MobilityDB 1.4 Manual de usuario

Esteban Zimányi

COLABORADORES			
	TÍTULO : MobilityDB 1.4 Manual de usuario		
ACCIÓN	*NOMBRE*	*FECHA*	*FIRMA*
ESCRITO POR	Esteban Zimányi	8 de noviembre de 2026	

HISTORIAL DE REVISIONES			
NÚMERO	FECHA	MODIFICACIONES	NOMBRE

Índice general

1. Introducción **1**

1.1. Comité directivo del proyecto . . . 1
1.2. Otros colaboradores del código . . . 2
1.3. Patrocinadores . . . 2
1.4. Licencias . . . 2
1.5. Instalación a partir de las fuentes . . . 2
1.5.1. Versión corta . . . 2
1.5.2. Obtener las fuentes . . . 3
1.5.3. Habilitación de la base de datos . . . 4
1.5.4. Dependencias . . . 4
1.5.5. Configuración . . . 5
1.5.6. Construir e instalar . . . 5
1.5.7. Pruebas . . . 6
1.5.8. Documentación . . . 6
1.6. Instalación a partir de binarios . . . 7
1.6.1. Distribuciones de Linux basadas en Debian . . . 7
1.6.2. Windows . . . 7
1.7. Soporte . . . 7
1.7.1. Reporte de problemas . . . 7
1.7.2. Listas de correo . . . 7
1.8. Migración de la versión 1.2 a la versión 1.3 . . . 8
1.9. Migración de la versión 1.3 a la versión 1.4 . . . 8

2. Tipos de conjunto y de rango **10**

2.1. Entrada y salida . . . 11
2.2. Constructores . . . 16
2.3. Conversión de tipos . . . 16
2.4. Accesores . . . 19
2.5. Transformaciones . . . 22
2.6. Sistema de referencia espacial . . . 25

2.7. Operaciones de conjuntos . . . 26
2.8. Operaciones de cuadro delimitador . . . 27
2.8.1. Operaciones topológicas . . . 27
2.8.2. Operaciones de posición . . . 28
2.8.3. Operaciones de división . . . 29
2.9. Operaciones de distancia . . . 30
2.10. Comparaciones . . . 31
2.11. Agregaciones . . . 32
2.12. Indexación . . . 33

3. Tipos de cuadro delimitador **35**
3.1. Entrada y salida . . . 35
3.2. Constructores . . . 38
3.3. Conversión de tipos . . . 39
3.4. Accesores . . . 40
3.5. Transformaciones . . . 42
3.6. Sistema de referencia espacial . . . 44
3.7. Operaciones de división . . . 44
3.8. Operaciones de conjuntos . . . 45
3.9. Operaciones de cuadro delimitador . . . 46
3.9.1. Operaciones topológicas . . . 46
3.9.2. Operaciones de posición . . . 47
3.10. Comparaciones . . . 49
3.11. Aggregaciones . . . 49
3.12. Indexación . . . 50

4. Tipos temporales (Parte 1) **51**
4.1. Introduction . . . 51
4.2. Ejemplos de tipos temporales . . . 54
4.3. Validez de los tipos temporales . . . 55
4.4. Temporalización de operaciones . . . 56
4.5. Notación . . . 57
4.6. Entrada y salida . . . 57
4.7. Constructores . . . 62
4.8. Conversión de tipos . . . 64
4.9. Accesores . . . 65
4.10. Transformaciones . . . 69

5. Tipos temporales (Parte 2) 72
5.1. Modificaciones . . . 72
5.2. Restricciones . . . 77
5.3. Operadores de cuadro delimitador . . . 79
5.4. Comparaciones . . . 80
5.4.1. Comparaciones tradicionales . . . 80
5.4.2. Comparaciones alguna vez y siempre . . . 81
5.4.3. Comparaciones temporales . . . 82
5.5. Funciones de utilidad . . . 83

6. Tipos alfanuméricos temporales 84
6.1. Notación . . . 84
6.2. Conversión de tipos . . . 84
6.3. Accessores . . . 85
6.4. Transformaciones . . . 86
6.5. Restrictions . . . 87
6.6. Operaciones booleanas . . . 88
6.7. Operaciones matemáticas . . . 88
6.8. Operaciones de texto . . . 92

7. Tipos temporales geométricos (Parte 1) 93
7.1. Tipos espaciotemporales . . . 93
7.2. Funciones sobre Geometrías . . . 95
7.3. Notación . . . 101
7.4. Entrada y salida . . . 102
7.5. Conversión de tipos . . . 104
7.6. Accessores . . . 105
7.7. Transformaciones . . . 109

8. Tipos temporales geométricos (Parte 2) 115
8.1. Restricciones . . . 115
8.2. Sistema de referencia espacial . . . 117
8.3. Operandos de dimensiones distintas . . . 118
8.4. Geometrías vacías . . . 119
8.5. Operaciones de cuadro delimitador . . . 119
8.6. Operaciones de distancia . . . 120
8.7. Cálculos planos y geodésicos . . . 123
8.8. Relaciones siempre y alguna vez . . . 124
8.9. Relaciones espaciotemporales . . . 126

9. Tipos temporales: Operaciones de análisis **128**
9.1. Simplificación . . . 128
9.2. Reducción . . . 129
9.3. Similaridad . . . 132
9.4. Filtro de Kalman extendido . . . 134
9.5. Operaciones de división . . . 135
9.6. Mosaicos multidimensionales . . . 138
9.6.1. Operaciones de intervalos . . . 139
9.6.2. Operaciones de mosaicos . . . 140
9.6.3. Operaciones de cuadro delimitador . . . 143
9.6.4. Operaciones de división . . . 145

10. Tipos temporales: Agregación e indexación **148**
10.1. Agregación . . . 148
10.2. Indexación . . . 150
10.3. Estadísticas y selectividad . . . 151
10.3.1. Colecta de estadísticas . . . 151
10.3.2. Estimación de la selectividad . . . 153

11. Puntos de red temporales **154**
11.1. Tipos de red estáticos . . . 154
11.1.1. Entrada y salida . . . 155
11.1.2. Constructores . . . 157
11.1.3. Conversión de tipos . . . 158
11.1.4. Accesores . . . 158
11.1.5. Transformaciones . . . 159
11.1.6. Operaciones espaciales . . . 159
11.1.7. Comparaciones . . . 160
11.2. Puntos de red temporales . . . 160
11.3. Validez de los puntos de red temporal . . . 161
11.4. Constructores . . . 161
11.5. Conversión de tipos . . . 162
11.6. Entrada y Salida MF-JSON . . . 162
11.7. Accessores . . . 163
11.8. Transformaciones . . . 164
11.9. Modificaciones . . . 164
11.10Restricciones . . . 165
11.11Operaciones espaciales . . . 165
11.12Operaciones de cuadro delimitador . . . 166

11.13Operaciones de distancia . . . 167
11.14Relaciones siempre y alguna vez . . . 168
11.15Relaciones espaciotemporales . . . 168
11.16Comparaciones . . . 169
11.17Agregacions . . . 170
11.18Indexación . . . 170
11.19Agregación . . . 171

12. Búferes circulares temporales **172**
12.1. Búferes circulares estáticos . . . 173
12.1.1. Entrada y salida . . . 173
12.1.2. Constructores . . . 174
12.1.3. Conversión de tipos . . . 174
12.1.4. Accesores . . . 175
12.1.5. Transformaciones . . . 175
12.1.6. Operaciones espaciales . . . 175
12.1.7. Operaciones de distancia . . . 176
12.1.8. Comparaciones . . . 177
12.2. Búferes circulares temporales . . . 177
12.3. Validez de los búferes circulares temporales . . . 178
12.4. Entrada y salida . . . 178
12.5. Constructores . . . 180
12.6. Conversión de tipos . . . 181
12.7. Accesores . . . 181
12.8. Transformaciones . . . 182
12.9. Modificaciones . . . 184
12.10Restricciones . . . 184
12.11Operaciones espaciales . . . 185
12.12Operaciones de cuadro delimitador . . . 186
12.13Operaciones de distancia . . . 187
12.14Relaciones siempre y alguna vez . . . 189
12.15Relaciones espaciotemporales . . . 189
12.16Comparaciones . . . 189
12.17Agregaciones . . . 190
12.18Simplificación . . . 191
12.19Indexación . . . 192

13. Poses temporales y cadenas de poses **193**
13.1. Notación . . . 194
13.2. Poses estáticas y cadenas de poses estáticas . . . 194
13.2.1. Entrada y salida . . . 195
13.2.2. Constructores . . . 196
13.2.3. Conversión de tipos . . . 197
13.2.4. Accesores . . . 198
13.2.5. Transformaciones . . . 198
13.2.6. Sistema de referencia espacial . . . 198
13.2.7. Operaciones de distancia . . . 199
13.2.8. Comparaciones . . . 200
13.3. Composición de cadenas de poses . . . 201
13.4. Poses temporales y cadenas de poses temporales . . . 202
13.5. Validez de las poses temporales y de las cadenas de poses . . . 203
13.6. Entrada y salida . . . 204
13.7. Constructores . . . 206
13.8. Conversión de tipos . . . 207
13.9. Accesores . . . 207
13.10Transformaciones . . . 209
13.11Modificaciones . . . 211
13.12Restricciones . . . 212
13.13Sistema de referencia espacial . . . 213
13.14Operaciones de cuadro delimitador . . . 214
13.15Operaciones de distancia . . . 215
13.16Relaciones siempre y alguna vez . . . 216
13.17Relaciones espaciotemporales . . . 217
13.18Comparaciones . . . 218
13.19Agregaciones . . . 219
13.20Operaciones espaciales . . . 220
13.21Simplificación . . . 220
13.22Indexación . . . 221
13.23Soporte de OGC GeoPose v1.0 . . . 221
13.23.1Entrada y salida JSON . . . 221
13.23.2Renormalización de cuaterniones . . . 222
13.23.3Accesores de ángulos de Euler . . . 222
13.23.4Registro de metadatos de marcos . . . 223
13.23.5Transformación rígida cuerpo↔mundo . . . 225
13.23.6E/S JSON temporal . . . 226
13.23.7Documentos compuestos de cadena y de grafo . . . 228

14. Geometrías rígidas temporales **230**

14.1. Entrada y salida . . . 230
14.2. Constructores . . . 232
14.3. Conversión de tipos . . . 232
14.4. Accesores . . . 233
14.5. Área recorrida . . . 234
14.6. Funciones espaciales . . . 234
14.7. Métricas de movimiento . . . 235
14.8. Transformaciones . . . 236
14.9. Modificaciones . . . 239
14.10Restricciones . . . 239
14.11Operaciones de distancia . . . 240
14.12Similitud . . . 242
14.13Operaciones de caja delimitadora . . . 242
14.14Relaciones siempre y alguna vez . . . 243
14.15Relaciones espaciotemporales . . . 243
14.16Comparaciones . . . 244
14.17Mosaicos . . . 245
14.18Cajas delimitadoras . . . 245
14.19Agregaciones . . . 246
14.20Indexación . . . 246

15. Tipos JSON en MobilityDB **247**

15.1. Operaciones sobre valores JSONB . . . 247
15.2. Operaciones sobre conjuntos JSONB . . . 251
15.3. Valores JSONB temporales . . . 257
15.4. Validez de los valores JSONB temporales . . . 258
15.5. Entrada y salida . . . 258
15.6. Constructores . . . 259
15.7. Conversiones . . . 260
15.8. Accesores . . . 261
15.9. Transformaciones . . . 261
15.10Operaciones JSON temporales . . . 262
15.11Restricciones . . . 269
15.12Operaciones de caja delimitadora . . . 269
15.13Comparaciones . . . 270
15.14Agregaciones . . . 271
15.15Indexación . . . 271

16. Teselaciones temporales **272**
16.1. Notación . . . 274
16.2. Entrada y salida . . . 274
16.3. Constructores . . . 278
16.4. Conversión de tipos . . . 281
16.5. Accesores . . . 283
16.6. Transformaciones . . . 286
16.7. Modificaciones . . . 288
16.8. Restricciones . . . 288
16.9. Cobertura de geometrías estáticas . . . 289
16.10Inspección . . . 290
16.11Jerarquía . . . 291
16.12Conversiones de latitud y longitud . . . 292
16.13Operaciones de cuadro delimitador . . . 296
16.14Funciones métricas . . . 298
16.15Funciones que devuelven conjuntos . . . 298
16.16Relaciones siempre y alguna vez . . . 300
16.17Relaciones espaciotemporales . . . 301
16.18Comparaciones . . . 301
16.19Agregaciones . . . 303
16.20Indexación . . . 303
16.21Operaciones específicas de H3 . . . 304
16.22Operaciones específicas de QUADBIN . . . 308
16.23Operaciones específicas de S2 . . . 309
16.24Operaciones específicas de ráster . . . 311

17. Nubes de puntos temporales **325**
17.1. Esquemas de nubes de puntos . . . 325
17.2. Nubes de puntos estáticas . . . 326
17.2.1. Entrada y salida . . . 327
17.2.2. Constructores . . . 327
17.2.3. Accesores . . . 328
17.3. Conjuntos de nubes de puntos . . . 329
17.3.1. Constructores . . . 329
17.4. Cajas delimitadoras de nubes de puntos . . . 330
17.4.1. Entrada y salida . . . 330
17.4.2. Constructores . . . 331
17.4.3. Conversiones . . . 331
17.4.4. Accesores . . . 332

17.4.5. Transformaciones . . . 333
17.4.6. Operaciones de conjuntos . . . 333
17.4.7. Operaciones topológicas . . . 333
17.4.8. Operaciones de posición . . . 334
17.4.9. Comparaciones . . . 334
17.5. Puntos de nube de puntos temporales . . . 335
17.5.1. Entrada y salida . . . 335
17.5.2. Constructores . . . 336
17.5.3. Conversiones . . . 337
17.5.4. Accesores . . . 338
17.5.5. Transformaciones . . . 338
17.5.6. Modificaciones . . . 339
17.5.7. Restricciones . . . 340
17.5.8. Operaciones de división . . . 341
17.5.9. Operaciones de caja delimitadora . . . 342
17.5.10.Operaciones de distancia . . . 342
17.5.11.Relaciones siempre y alguna vez . . . 343
17.5.12.Relaciones espaciotemporales . . . 344
17.5.13.Comparaciones . . . 345
17.5.14.Simplificación . . . 346
17.6. Parches de nube de puntos temporales . . . 346
17.6.1. Entrada y salida . . . 346
17.6.2. Constructores . . . 348
17.6.3. Conversiones . . . 348
17.6.4. Accesores . . . 349
17.6.5. Transformaciones . . . 349
17.6.6. Modificaciones . . . 350
17.6.7. Restricciones . . . 351
17.6.8. Operaciones de división . . . 352
17.6.9. Operaciones espaciales . . . 353
17.6.10.Operaciones de caja delimitadora . . . 354
17.6.11.Operaciones de distancia . . . 354
17.6.12.Relaciones siempre y alguna vez . . . 355
17.6.13.Relaciones espaciotemporales . . . 355
17.6.14.Comparaciones . . . 356
17.7. Indexación . . . 356
17.8. Agregaciones . . . 356

18. Escritura de consultas portables **359**
18.1. Operaciones de conjuntos . . . 359
18.2. Operaciones de cuadro delimitador . . . 360
18.2.1. Operaciones topológicas . . . 360
18.2.2. Operaciones de posición . . . 360
18.3. Operaciones de distancia . . . 361
18.4. Comparaciones . . . 361
18.4.1. Comparaciones tradicionales . . . 361
18.4.2. Comparaciones alguna vez y siempre . . . 361
18.4.3. Comparaciones temporales . . . 362
18.5. Operaciones booleanas . . . 362
18.6. Operaciones matemáticas . . . 362
18.7. Operaciones JSON temporales . . . 362
18.8. Valores base provistos por el anfitrión . . . 363
18.9. Agregaciones . . . 364
18.10Ejecución de una consulta en Apache Spark y Apache Flink . . . 365
18.10.1Los nombres que poseen los motores . . . 365
18.10.2Lectura y escritura de valores . . . 367
18.10.3Funciones que devuelven filas . . . 368
18.10.4Ejemplo . . . 368
18.11Los rásteres de los motores . . . 368
18.11.1DuckDB y su extensión de rásteres . . . 369
18.11.2Spark y Apache Sedona . . . 370

A. MobilityDB Reference **371**
A.1. Tipos de MobilityDB . . . 371
A.2. Set and Span Types . . . 371
A.2.1. Entrada y salida . . . 371
A.2.2. Constructores . . . 372
A.2.3. Conversión de tipos . . . 372
A.2.4. Accesores . . . 372
A.2.5. Transformaciones . . . 373
A.2.6. Sistema de referencia espacial . . . 373
A.2.7. Operaciones de conjuntos . . . 373
A.2.8. Operaciones de cuadro delimitador . . . 373
A.2.8.1. Operaciones topológicas . . . 373
A.2.8.2. Operaciones de posición . . . 374
A.2.8.3. Operaciones de división . . . 374
A.2.9. Operaciones de distancia . . . 374

A.2.10. Comparaciones . . . 374
A.2.11. Agregaciones . . . 374
A.3. Box Types . . . 374
A.3.1. Entrada y salida . . . 374
A.3.2. Constructores . . . 375
A.3.3. Conversión de tipos . . . 375
A.3.4. Accesores . . . 375
A.3.5. Transformaciones . . . 375
A.3.6. Sistema de referencia espacial . . . 375
A.3.7. Operaciones de división . . . 375
A.3.8. Operaciones de conjuntos . . . 376
A.3.9. Operaciones de cuadro delimitador . . . 376
A.3.9.1. Operaciones topológicas . . . 376
A.3.9.2. Operaciones de posición . . . 376
A.3.10. Comparaciones . . . 376
A.3.11. Aggregaciones . . . 376
A.4. Temporal Types . . . 376
A.4.1. Entrada y salida . . . 376
A.4.2. Constructores . . . 377
A.4.3. Conversión de tipos . . . 377
A.4.4. Accesores . . . 377
A.4.5. Transformaciones . . . 377
A.4.6. Modificaciones . . . 378
A.4.7. Restricciones . . . 378
A.4.8. Comparaciones . . . 378
A.4.8.1. Comparaciones tradicionales . . . 378
A.4.8.2. Comparaciones alguna vez y siempre . . . 378
A.4.8.3. Comparaciones temporales . . . 378
A.4.9. Funciones de utilidad . . . 378
A.5. Temporal Alphanumeric Types . . . 378
A.5.1. Conversión de tipos . . . 378
A.5.2. Accessores . . . 379
A.5.3. Transformaciones . . . 379
A.5.4. Restrictions . . . 379
A.5.5. Operaciones booleanas . . . 379
A.5.6. Operaciones matemáticas . . . 379
A.5.7. Operaciones de texto . . . 380
A.6. Temporal Geometry Types . . . 380
A.6.1. Funciones sobre Geometrías . . . 380

A.6.2. Entrada y salida . . . 381
A.6.3. Conversión de tipos . . . 381
A.6.4. Accessores . . . 381
A.6.5. Transformaciones . . . 382
A.6.6. Restricciones . . . 382
A.6.7. Sistema de referencia espacial . . . 382
A.6.8. Operaciones de cuadro delimitador . . . 382
A.6.9. Operaciones de distancia . . . 383
A.6.10. Relaciones siempre y alguna vez . . . 383
A.6.11. Relaciones espaciotemporales . . . 383
A.7. Operations for Temporal Types: Analytics Operations . . . 383
A.7.1. Simplificación . . . 383
A.7.2. Reducción . . . 384
A.7.3. Similaridad . . . 384
A.7.4. Filtro de Kalman extendido . . . 384
A.7.5. Operaciones de división . . . 384
A.7.6. Mosaicos multidimensionales . . . 384
A.7.6.1. Operaciones de intervalos . . . 384
A.7.6.2. Operaciones de mosaicos . . . 385
A.7.6.3. Operaciones de cuadro delimitador . . . 385
A.7.6.4. Operaciones de división . . . 385
A.7.7. Agregación . . . 385
A.8. Temporal Poses and Pose Chains . . . 385
A.8.1. Poses estáticas y cadenas de poses estáticas . . . 385
A.8.1.1. Entrada y salida . . . 385
A.8.1.2. Constructores . . . 386
A.8.1.3. Conversión de tipos . . . 386
A.8.1.4. Accesores . . . 386
A.8.1.5. Transformaciones . . . 386
A.8.1.6. Sistema de referencia espacial . . . 386
A.8.1.7. Operaciones de distancia . . . 386
A.8.1.8. Comparaciones . . . 386
A.8.2. Composición de cadenas de poses . . . 387
A.8.3. Entrada y salida . . . 387
A.8.4. Constructores . . . 387
A.8.5. Conversión de tipos . . . 387
A.8.6. Accesores . . . 388
A.8.7. Transformaciones . . . 388
A.8.8. Modificaciones . . . 388

A.8.9. Restricciones . . . 388
A.8.10. Sistema de referencia espacial . . . 389
A.8.11. Operaciones de cuadro delimitador . . . 389
A.8.12. Operaciones de distancia . . . 389
A.8.13. Relaciones siempre y alguna vez . . . 389
A.8.14. Relaciones espaciotemporales . . . 389
A.8.15. Comparaciones . . . 389
A.8.16. Agregaciones . . . 389
A.8.17. Operaciones espaciales . . . 390
A.8.18. Simplificación . . . 390
A.8.19. Soporte de OGC GeoPose v1.0 . . . 390
A.8.19.1. Entrada y salida JSON . . . 390
A.8.19.2. Renormalización de cuaterniones . . . 390
A.8.19.3. Accesores de ángulos de Euler . . . 390
A.8.19.4. Registro de metadatos de marcos . . . 390
A.8.19.5. Transformación rígida cuerpo↔mundo . . . 390
A.8.19.6. E/S JSON temporal . . . 391
A.8.19.7. Documentos compuestos de cadena y de grafo . . . 391
A.9. Temporal Network Points . . . 391
A.9.1. Tipos de red estáticos . . . 391
A.9.1.1. Entrada y salida . . . 391
A.9.1.2. Constructores . . . 391
A.9.1.3. Conversión de tipos . . . 391
A.9.1.4. Accesores . . . 392
A.9.1.5. Transformaciones . . . 392
A.9.1.6. Operaciones espaciales . . . 392
A.9.1.7. Comparaciones . . . 392
A.9.2. Constructores . . . 392
A.9.3. Conversión de tipos . . . 392
A.9.4. Entrada y Salida MF-JSON . . . 392
A.9.5. Accessores . . . 392
A.9.6. Transformaciones . . . 393
A.9.7. Modificaciones . . . 393
A.9.8. Restricciones . . . 393
A.9.9. Operaciones espaciales . . . 393
A.9.10. Operaciones de cuadro delimitador . . . 393
A.9.11. Operaciones de distancia . . . 393
A.9.12. Relaciones siempre y alguna vez . . . 393
A.9.13. Relaciones espaciotemporales . . . 393

A.9.14. Comparaciones . . . 394
A.9.15. Agregacions . . . 394
A.9.16. Agregación . . . 394
A.10. Temporal Circular Buffers . . . 394
A.10.1. Búferes circulares estáticos . . . 394
A.10.1.1. Entrada y salida . . . 394
A.10.1.2. Constructores . . . 394
A.10.1.3. Conversión de tipos . . . 394
A.10.1.4. Accesores . . . 394
A.10.1.5. Transformaciones . . . 395
A.10.1.6. Operaciones espaciales . . . 395
A.10.1.7. Operaciones de distancia . . . 395
A.10.1.8. Comparaciones . . . 395
A.10.2. Entrada y salida . . . 395
A.10.3. Constructores . . . 395
A.10.4. Conversión de tipos . . . 395
A.10.5. Accesores . . . 396
A.10.6. Transformaciones . . . 396
A.10.7. Modificaciones . . . 396
A.10.8. Restricciones . . . 396
A.10.9. Operaciones espaciales . . . 396
A.10.10 Operaciones de cuadro delimitador . . . 396
A.10.11 Operaciones de distancia . . . 397
A.10.12 Relaciones siempre y alguna vez . . . 397
A.10.13 Relaciones espaciotemporales . . . 397
A.10.14 Comparaciones . . . 397
A.10.15 Agregaciones . . . 397
A.10.16 Simplificación . . . 397
A.11. Temporal Rigid Geometries . . . 398
A.11.1. Entrada y salida . . . 398
A.11.2. Constructores . . . 398
A.11.3. Conversión de tipos . . . 398
A.11.4. Accesores . . . 398
A.11.5. Área recorrida . . . 398
A.11.6. Funciones espaciales . . . 398
A.11.7. Métricas de movimiento . . . 399
A.11.8. Transformaciones . . . 399
A.11.9. Modificaciones . . . 399
A.11.10 Restricciones . . . 399

A.11.11Operaciones de distancia . . . 399
A.11.12Operaciones de caja delimitadora . . . 400
A.11.13Relaciones siempre y alguna vez . . . 400
A.11.14Relaciones espaciotemporales . . . 400
A.11.15Comparaciones . . . 400
A.11.16Mosaicos . . . 400
A.11.17Cajas delimitadoras . . . 400
A.11.18Agregaciones . . . 400
A.12.Temporal JSON . . . 400
A.12.1. Operaciones sobre valores JSONB . . . 400
A.12.2. Operaciones sobre conjuntos JSONB . . . 401
A.12.3. Entrada y salida . . . 402
A.12.4. Constructores . . . 402
A.12.5. Conversiones . . . 402
A.12.6. Accesores . . . 402
A.12.7. Transformaciones . . . 402
A.12.8. Operaciones JSON temporales . . . 402
A.12.9. Restricciones . . . 403
A.12.10Operaciones de caja delimitadora . . . 403
A.12.11Comparaciones . . . 403
A.12.12Agregaciones . . . 403
A.13.Temporal Tessellations . . . 404
A.13.1. Entrada y salida . . . 404
A.13.2. Constructores . . . 404
A.13.3. Conversión de tipos . . . 404
A.13.4. Accesores . . . 405
A.13.5. Transformaciones . . . 405
A.13.6. Modificaciones . . . 405
A.13.7. Restricciones . . . 405
A.13.8. Cobertura de geometrías estáticas . . . 406
A.13.9. Inspección . . . 406
A.13.10Jerarquía . . . 406
A.13.11Conversiones de latitud y longitud . . . 406
A.13.12Operaciones de cuadro delimitador . . . 406
A.13.13Funciones métricas . . . 406
A.13.14Funciones que devuelven conjuntos . . . 406
A.13.15Relaciones siempre y alguna vez . . . 407
A.13.16Relaciones espaciotemporales . . . 407
A.13.17Comparaciones . . . 407

A.13.18Agregaciones . . . 407
A.13.19Operaciones específicas de H3 . . . 407
A.13.20Operaciones específicas de QUADBIN . . . 408
A.13.21Operaciones específicas de S2 . . . 408
A.13.22Operaciones específicas de ráster . . . 409
A.14. Temporal Point Clouds . . . 410
A.14.1. Esquemas de nubes de puntos . . . 410
A.14.2. Nubes de puntos estáticas . . . 410
A.14.2.1. Entrada y salida . . . 410
A.14.2.2. Constructores . . . 410
A.14.2.3. Accesores . . . 410
A.14.3. Conjuntos de nubes de puntos . . . 410
A.14.3.1. Constructores . . . 410
A.14.4. Cajas delimitadoras de nubes de puntos . . . 411
A.14.4.1. Entrada y salida . . . 411
A.14.4.2. Constructores . . . 411
A.14.4.3. Conversiones . . . 411
A.14.4.4. Accesores . . . 411
A.14.4.5. Transformaciones . . . 411
A.14.4.6. Operaciones de conjuntos . . . 411
A.14.4.7. Operaciones topológicas . . . 411
A.14.4.8. Operaciones de posición . . . 412
A.14.4.9. Comparaciones . . . 412
A.14.5. Puntos de nube de puntos temporales . . . 412
A.14.5.1. Entrada y salida . . . 412
A.14.5.2. Constructores . . . 412
A.14.5.3. Conversiones . . . 412
A.14.5.4. Accesores . . . 412
A.14.5.5. Transformaciones . . . 412
A.14.5.6. Modificaciones . . . 413
A.14.5.7. Restricciones . . . 413
A.14.5.8. Operaciones de división . . . 413
A.14.5.9. Operaciones de caja delimitadora . . . 413
A.14.5.10.Operaciones de distancia . . . 413
A.14.5.11.Relaciones siempre y alguna vez . . . 413
A.14.5.12.Relaciones espaciotemporales . . . 413
A.14.5.13.Comparaciones . . . 414
A.14.5.14.Simplificación . . . 414
A.14.6. Parches de nube de puntos temporales . . . 414

A.14.6.1. Entrada y salida . . . 414
A.14.6.2. Constructores . . . 414
A.14.6.3. Conversiones . . . 414
A.14.6.4. Accesores . . . 414
A.14.6.5. Transformaciones . . . 414
A.14.6.6. Modificaciones . . . 415
A.14.6.7. Restricciones . . . 415
A.14.6.8. Operaciones de división . . . 415
A.14.6.9. Operaciones espaciales . . . 415
A.14.6.10. Operaciones de caja delimitadora . . . 415
A.14.6.11. Operaciones de distancia . . . 415
A.14.6.12. Relaciones siempre y alguna vez . . . 415
A.14.6.13. Relaciones espaciotemporales . . . 416
A.14.6.14. Comparaciones . . . 416
A.14.7. Agregaciones . . . 416
A.14.8. Indexes . . . 416

B. Generador de datos sintéticos 417
B.1. Generador para tipos PostgreSQL . . . 417
B.2. Generador para tipos PostGIS . . . 418
B.3. Generador para tipos de rango, de tiempo y de cuadro delimitador MobilityDB . . . 419
B.4. Generador para tipos temporales MobilityDB . . . 419
B.5. Generación de tablas con valores aleatorios . . . 420
B.6. Generador para tipos de red temporales . . . 425
B.7. Generador para tipos JSONB temporales . . . 425

Índice de figuras

3.1. Grilla multiresolución sobre datos de Bruselas obtenidos mediante el generador BerlinMOD. Cada celda contiene como máximo 10.000 (izquierda) y 1.000 (derecha) instantes durante todo el período de simulación (cuatro días en este caso). A la izquierda, podemos ver la alta densidad de tráfico en el anillo alrededor de Bruselas, mientras que a la derecha podemos ver otros ejes principales de la ciudad. 45

4.1. Jerarquía de tipos temporales en MobilityDB. 51

5.1. Operación de inserción para valores temporales. 72
5.2. Operaciones de actualización y supresión para valores temporales. 73
5.3. Operaciones de modificación para tablas temporales en SQL. 73

7.1. Jerarquía de tipos espaciotemporales en MobilityDB. 93
7.2. Ilustración del uso de los tipos `tgeompoint` (arriba) y `tgeometry` (abajo) para modelizar la trayectoria y la franja de viento de las tormentas tropicales en 2024. 94
7.3. Visualización de la velocidad de un objeto móvil usando una rampa de color en QGIS. 110

9.1. Diferencia entre la distancia espacial y la distancia sincronizada. 129
9.2. Muestreo de flotantes temporales con interpolación discreta, escalonada y lineal. 130
9.3. Cambio de precisión de números flotantes temporales con interpolación discreta, escalonada y lineal. 132
9.4. Mosaicos multidimensionales para números flotantes temporales. 139

12.1. Ilustración del uso del tipo `tcbuffer` para modelar el búfer circular alrededor de la franja de viento de las tormentas tropicales en 2024. 172

16.1. La traza de un buque AIS que sale de Frederikshavn en las tres cuadrículas, con las celdas por las que pasa la traza sombreadas y un área marina protegida en verde. La fila superior presenta la traza completa en las resoluciones cuyas celdas son más próximas en área y delinea en negro el extracto que presenta la fila inferior, una resolución más fina en cada cuadrícula. Traza del buque y áreas protegidas de los datos AIS daneses; mapa base © colaboradores de OpenStreetMap. 273

Índice de cuadros

1.2. Variables para la documentación del usuario y del desarrollador 6

A.1. Instancias actuales de los tipos de plantilla en MobilityDB 371

Resumen

MobilityDB es una extensión del sistema de base de datos PostgreSQL y su extensión espacial PostGIS. Permite almacenar en la base de datos objetos temporales y espacio-temporales, es decir, objetos cuyos valores de atributo y/o ubicación evolucionan en el tiempo. MobilityDB incluye funciones para el análisis y procesamiento de objetos temporales y espacio-temporales y proporciona soporte para índices GiST y SP-GiST. MobilityDB es de código abierto y su código está disponible en Github. También está disponible un adaptador para el lenguaje de programación Python en Github.



MobilityDB es desarrollado por el Departamento de Ingeniería Informática y de Decisiones de la Université libre de Bruxelles (ULB) bajo la dirección del Prof. Esteban Zimányi. ULB es un miembro asociado de OGC y miembro del Grupo de trabajo de estandarización de características móviles de OGC (MF-SWG).



El manual de MobilityDB tiene una licencia Creative Commons Attribution-Share Alike 3.0 License 3. No dude en utilizar este material como desee, pero le pedimos que atribuya crédito al proyecto MobilityDB y, siempre que sea posible, un enlace a MobilityDB.



Capítulo 1

Introducción

MobilityDB es una extensión de PostgreSQL y PostGIS que proporciona *tipos temporales*. Dichos tipos de datos representan la evolución en el tiempo de los valores de algún tipo de elemento, llamado tipo base del tipo temporal. Por ejemplo, se pueden usar enteros temporales para representar la evolución en el tiempo de la marchar utilizada por un automóvil en movimiento. En este caso, el tipo de datos es *entero temporal* y el tipo base es *entero*. Del mismo modo, se puede utilizar un número flotante temporal para representar la evolución en el tiempo de la velocidad de un automóvil. Como otro ejemplo, se puede usar un punto temporal para representar la evolución en el tiempo de la ubicación de un automóvil, como lo reportan los dispositivos GPS. Los tipos temporales son útiles porque representar valores que evolucionan en el tiempo es esencial en muchas aplicaciones, por ejemplo, en aplicaciones de movilidad. Además, los operadores de los tipos base (como los operadores aritméticos y la agregación para números enteros y flotantes, las relaciones topológicas y la distancia para las geometrías) se pueden generalizar intuitivamente cuando los valores evolucionan en el tiempo.

MobilityDB proporciona los siguientes tipos temporales: `tbool`, `tint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint` y `tgeogpoint`. Estos tipos temporales se basan, respectivamente, en los tipos de base `boolean`, `integer`, `float` y `text` proporcionados por PostgreSQL, y en los tipos base de `geometry` y `geography` proporcionados por PostGIS, donde `tgeometry` y `tgeography` aceptan geometrías/geografías arbitrarias, mientras que `tgeompoint` y `tgeogpoint` solo aceptan puntos 2D o 3D.[1] Además, MobilityDB proporciona los tipos de plantilla *set*, *span* y *span set* para representar, respectivamente, un conjunto de valores, un rango de valores y un conjunto de rangos de valores de tipos de base o tipos de tiempo. Ejemplos de valores de tipos de conjunto son `intset`, `floatset` y `tstzset`, donde el último representa un conjunto de valores `timestamptz`. Ejemplos de valores de tipos de rango son `intspan`, `floatspan` y `tstzspan`. Ejemplos de valores de tipos de conjuntos de rangos son `intspanset`, `floatspanset` y `tstzspanset`.

1.1. Comité directivo del proyecto

El comité directivo del proyecto MobilityDB (Project Steering Committee o PSC) coordina la dirección general, los ciclos de publicación, la documentación y los esfuerzos de divulgación para el proyecto MobilityDB. Además, el PSC proporciona soporte general al usuario, acepta y aprueba parches de la comunidad general de MobilityDB y vota sobre diversos problemas relacionados con MobilityDB, como el acceso de commit de los desarrolladores, nuevos miembros del PSC o cambios significativos en la interfaz de programación de aplicaciones (Application Programming Interface o API).

A continuación se detallan los miembros actuales en orden alfabético y sus principales responsabilidades:

- Mohamed Bakli: MobilityDB-docker, versiones distribuidas y en la nube, integración con Citus
- Krishna Chaitanya Bommakanti: MEOS (Mobility Engine Open Source), pyMEOS
- Anita Graser: integración con Moving Pandas y el ecosistema de Python, integración con QGIS
- Darafei Praliaskouski: integración con PostGIS

[1] Aunque los puntos temporales 4D se pueden representar, la dimensión M actualmente no se tiene en cuenta.

- Mahmoud Sakr: cofundador del proyecto MobilityDB, MobilityDB workshop, copresidente del OGC Moving Feature Standard Working Group (MF-SWG)
- Vicky Vergara: integración con pgRouting, enlace con OSGeo
- Esteban Zimányi (chair): cofundador del proyecto MobilityDB, coordinación general del proyecto, principal contribuidor del código de backend, BerlinMOD generator

1.2. Otros colaboradores del código

- Arthur Lesuisse
- Xinyiang Li
- Maxime Schoemans

1.3. Patrocinadores

Estas son organizaciones de financiación de investigación (en orden alfabético) que han contribuido con financiación monetaria al proyecto MobilityDB.

- European Commission
- Fonds de la Recherche Scientifique (FNRS), Belgium
- Innoviris, Belgium

Estas son entidades corporativas (en orden alfabético) que han contribuido con tiempo de desarrollador o financiación monetaria al proyecto MobilityDB.

- Adonmo, India
- Georepublic, Germany
- Université libre de Bruxelles, Belgium

1.4. Licencias

Las siguientes licencias se pueden encontrar en MobilityDB:

Recurso	Licencia
Código MobilityDB	Licencia PostgreSQL
Documentación MobilityDB	Licencia Creative Commons Attribution-Share Alike 3.0

1.5. Instalación a partir de las fuentes

1.5.1. Versión corta

Para compilar asumiendo que tiene todas las dependencias en su ruta de búsqueda

```
git clone https://github.com/MobilityDB/MobilityDB
mkdir MobilityDB/build
cd MobilityDB/build
cmake  ..
make
sudo make install
```

Los comandos anteriores instalan la rama `master`. Si desea instalar otra rama, por ejemplo, `develop`, puede reemplazar el primer comando anterior de la siguiente manera:

```
git clone --branch develop https://github.com/MobilityDB/MobilityDB
```

También debe configurar lo siguiente en el archivo `postgresql.conf` según la versión de PostGIS que haya instalado (a continuación usamos PostGIS 3):

```
shared_preload_libraries = 'postgis-3'
max_locks_per_transaction = 128
```

Si no carga previamente la biblioteca PostGIS con la configuración anterior, no podrá cargar la biblioteca MobilityDB y obtendrá un mensaje de error como el siguiente:

```
ERROR:  could not load library "/usr/local/pgsql/lib/libMobilityDB-1.1.so":
  undefined symbol: ST_Distance
```

Puede encontrar la ubicación del archivo `postgresql.conf` de la manera siguiente.

```
$ which postgres
/usr/local/pgsql/bin/postgres
$ ls /usr/local/pgsql/data/postgresql.conf
/usr/local/pgsql/data/postgresql.conf
```

Como puede verse, los binarios de PostgreSQL están en el subdirectorio `bin` mientras que el archivo `postgresql.conf` está en el subdirectorio `data`.

Una vez que MobilityDB está instalado, debe habilitarse en cada base de datos en la que desee usarlo. En el siguiente ejemplo, usamos una base de datos llamada `mobility`.

```
createdb mobility
psql mobility -c "CREATE EXTENSION PostGIS"
psql mobility -c "CREATE EXTENSION MobilityDB"
```

Las dos extensiones PostGIS y MobilityDB también se pueden crear en un solo comando.

```
psql mobility -c "CREATE EXTENSION MobilityDB cascade"
```

1.5.2. Obtener las fuentes

La última versión de MobilityDB se puede encontrar en https://github.com/MobilityDB/MobilityDB/releases/latest

wget

Para descargar esta versión:

```
wget -O mobilitydb-1.3.tar.gz https://github.com/MobilityDB/MobilityDB/archive/v1.3.tar.gz
```

Ir a la Sección 1.5.1 para las instrucciones de extracción y compilación.

git

Para descargar el repositorio

```
git clone https://github.com/MobilityDB/MobilityDB.git
cd MobilityDB
git checkout v1.3
```

Ir a la Sección 1.5.1 para las instrucciones de compilación (no hay tar ball).

1.5.3. Habilitación de la base de datos

MobilityDB es una extensión que depende de PostGIS. Habilitar PostGIS antes de habilitar MobilityDB en la base de datos se puede hacer de la siguiente manera

```
CREATE EXTENSION postgis;
CREATE EXTENSION mobilitydb;
```

Alternativamente, esto se puede hacer con un solo comando usando `CASCADE`, que instala la extensión PostGIS requerida antes de instalar la extensión MobilityDB

```
CREATE EXTENSION mobilitydb CASCADE;
```

1.5.4. Dependencias

Dependencias de compilación

Para poder compilar MobilityDB, asegúrese de que se cumplan las siguientes dependencias:

- Sistema de compilación multiplataforma `CMake`.
- Compilador C `gcc` o `clang`. Se pueden usar otros compiladores ANSI C, pero pueden causar problemas al compilar algunas dependencias.
- GNU Make (`gmake` o `make`) versión 3.1 o superior. Para muchos sistemas, GNU make es la versión predeterminada de make. Verifique la versión invocando `make -v`.
- PostgreSQL versión 16 a 19. Fuera de ese rango, la configuración se detiene con un error explícito. PostgreSQL está disponible en http://www.postgresql.org.
- PostGIS versión 3 o superior. PostGIS está disponible en https://postgis.net/.

Nótese que PostGIS tiene sus propias dependencias, como Proj, GEOS, LibXML2 o JSON-C. MobilityDB también utiliza Proj y JSON-C, mientras que calcula por sí mismo las operaciones geométricas y no utiliza GEOS. Consulte http://trac.osgeo.org/postgis/wiki/UsersWikiPostgreSQLPostGIS para obtener una matriz de compatibilidad de PostGIS con PostgreSQL, GEOS y Proj.

Dependencias opcionales

Para la documentación del usuario

- Los archivos DocBook DTD y XSL son necesarios para crear la documentación. Para Ubuntu, son proporcionados por los paquetes `docbook` y `docbook-xsl`.
- El validador XML `xmllint` es necesario para validar los archivos XML de la documentación. Para Ubuntu, lo proporciona el paquete `libxml2`.
- El procesador XSLT `xsltproc` es necesario para crear la documentación en formato HTML. Para Ubuntu, lo proporciona el paquete `libxslt`.
- El programa `dblatex` es necesario para crear la documentación en formato PDF. Para Ubuntu, lo proporciona el paquete `dblatex`.

- El programa `dbtoepub` es necesario para construir la documentación en formato EPUB. Para Ubuntu, lo proporciona el paquete `dbtoepub`.

Para la documentación de los desarrolladores

- El programa `doxygen` es necesario para construir la documentación. Para Ubuntu, lo proporciona el paquete `doxygen`.

Ejemplo: instalar dependencias en distribuciones basadas en Debian

Dependencias de base de datos

```
sudo apt-get install postgresql-16 postgresql-server-dev-16 postgresql-16-postgis
```

Dependencias de construcción

```
sudo apt-get install cmake gcc
```

Ejemplo: instalar dependencias en distribuciones basadas en RPM

En Fedora, Red Hat Enterprise Linux y sus derivados, los paquetes de la distribución incluyen una versión compatible de PostgreSQL y de PostGIS, por lo que un solo comando cubre tanto las dependencias de base de datos como las de construcción.

```
sudo dnf install gcc gcc-c++ cmake make postgresql-server-devel postgis \
  proj-devel json-c-devel
```

Las familias opcionales que habilita `-DALL=ON` requieren dos paquetes adicionales, `gdal-devel` para los operadores de rásteres y `h3-devel` para el tipo de índice H3.

1.5.5. Configuración

MobilityDB usa el sistema `cmake` para realizar la configuración. El directorio de compilación deber ser diferente del directorio de origen.

Para crear el directorio de compilación

```
mkdir build
```

Para ver las variables que se pueden configurar

```
cd build
cmake -L ..
```

1.5.6. Construir e instalar

Nótese que la versión actual de MobilityDB solo se ha probado en sistemas Linux, MacOS y Windows. Puede funcionar en otros sistemas similares a Unix, pero no se ha probado. Buscamos voluntarios que nos ayuden a probar MobilityDB en múltiples plataformas.

Las siguientes instrucciones comienzan desde `path/to/MobilityDB` en un sistema Linux

```
mkdir build
cd build
cmake ..
make
sudo make install
```

Cuando cambia la configuración

```
rm -rf build
```

e inicie el proceso de construcción como se mencionó anteriormente.

1.5.7. Pruebas

MobilityDB utiliza `ctest`, el programa controlador de pruebas de CMake, para realizar pruebas. Este programa ejecutará las pruebas e informará los resultados.

Para ejecutar todas las pruebas

```
ctest
```

Para ejecutar un archivo de prueba dado

```
ctest -R '021_tbox'
```

Para ejecutar un conjunto de archivos de prueba determinados se pueden utilizar comodines

```
ctest -R '022_*'
```

Las pruebas se ejecutan en un clúster temporal de PostgreSQL en el directorio `tmptest` del directorio de compilación, cuyo servidor escucha en un socket de dominio Unix en un directorio corto bajo `/tmp`, ya que PostgreSQL no arranca cuando la ruta del socket supera los 107 bytes. Otro directorio para el clúster puede indicarse en la configuración con la variable `MOBILITYDB_TEST_TMPDIR`. El clúster se crea con la configuración regional C y en UTF-8, salvo que la variable `MOBILITYDB_TEST_ENCODING` indique otra codificación, como `SQL_ASCII`

```
cmake -DMOBILITYDB_TEST_TMPDIR=/tmp/mobilitydb-test ..
```

1.5.8. Documentación

La documentación del usuario de MobilityDB se puede generar en formato HTML, PDF y EPUB. Además, la documentación está disponible en inglés y en otros idiomas (actualmente, solo en español). La documentación del usuario se puede generar en todos los formatos y en todos los idiomas, o se pueden especificar formatos y/o idiomas específicos. La documentación del desarrollador de MobilityDB solo se puede generar en formato HTML y en inglés.

Las variables utilizadas para generar la documentación del usuario y del desarrollador son las siguientes:

Variable	Valor por defecto	Comentario
DOC_ALL	BOOL=OFF	La documentación del usuario se genera en formatos HTML, PDF y EPUB.
DOC_HTML	BOOL=OFF	La documentación del usuario se genera en formato HTML.
DOC_PDF	BOOL=OFF	La documentación del usuario se genera en formato PDF.
DOC_EPUB	BOOL=OFF	La documentación del usuario se genera en formato EPUB.
LANG_ALL	BOOL=OFF	La documentación del usuario se genera en inglés y en todas las traducciones disponibles.
ES	BOOL=OFF	La documentación del usuario se genera en inglés y en español.
DOC_DEV	BOOL=OFF	La documentación del desarrollador en inglés se genera en formato HTML

Cuadro 1.2: Variables para la documentación del usuario y del desarrollador

Generar la documentación del usuario y del desarrollador en todos los formatos y en todos los idiomas.

```
cmake -D DOC_ALL=ON -D LANG_ALL=ON -D DOC_DEV=ON ..
make doc
make doc_dev
```

Generar la documentación del usuario en formato HTML y en todos los idiomas.

```
cmake -D DOC_HTML=ON -D LANG_ALL=ON ..
make doc
```

Generar la documentación del usuario en inglés en todos los formatos.

```
cmake -D DOC_ALL=ON ..
make doc
```

Generar la documentación del usuario en formato PDF en inglés y en español.

```
cmake -D DOC_PDF=ON -D ES=ON ..
make doc
```

1.6. Instalación a partir de binarios

1.6.1. Distribuciones de Linux basadas en Debian

Se está desarrollando soporte para sistemas Linux basados en Debian, como Ubuntu y Arch Linux.

1.6.2. Windows

Desde la versión 3.3.3 de PostGIS, MobilityDB se distribuye en el PostGIS Bundle para Windows, que está disponible en application stackbuilder y en el sitio web de OSGeo. Para más information refiérase a la documentación PostGIS.

1.7. Soporte

El soporte de la comunidad de MobilityDB está disponible a través de la página github de MobilityDB, documentación, tutoriales, listas de correo y otros.

1.7.1. Reporte de problemas

Los errores son registrados y manejados en un issue tracker. Por favor siga los siguientes pasos:

1. Busque las entradas para ver si su problema ya ha sido informado. Si es así, agregue cualquier contexto adicional que haya encontrado, o al menos indique que usted también está teniendo el problema. Esto nos ayudará a priorizar los problemas comunes.
2. Si su problema no se ha informado, cree un nuevo asunto para ello.
3. En su informe, incluya instrucciones explícitas para replicar su problema. Las mejores etradas incluyen consultas SQL exactas que se necesitan para reproducir un problema. Reporte también el sistema operativo y las versiones de MobilityDB, PostGIS y PostgreSQL.
4. Se recomienda utilizar el siguiente envoltorio en su problema para determinar el paso que está causando el problema.

```
SET client_min_messages TO debug;
<your code>
SET client_min_messages TO notice;
```

1.7.2. Listas de correo

Hay dos listas de correo para MobilityDB alojadas en el servidor de listas de correo OSGeo:

- Lista de correo de usuarios: http://lists.osgeo.org/mailman/listinfo/mobilitydb-users
- Lista de distribución de desarrolladores: http://lists.osgeo.org/mailman/listinfo/mobilitydb-dev

Para preguntas generales y temas sobre cómo usar MobilityDB, escriba a la lista de correo de usuarios.

1.8. Migración de la versión 1.2 a la versión 1.3

MobilityDB 1.3 es una revisión importante con respecto a la versión 1.2. El cambio más significativo de la versión 1.3 fue habilitar nuevos tipos espaciotemporales. Mientras que la versión 1.2 definía los tipos espaciotemporales `tgeompoint` (punto de geometría temporal), `tgeogpoint` (punto de geografía temporal) y `tnpoint` (punto de red temporal), la versión 1.3 añadió los nuevos tipos `tgeometry` (geometría temporal), `tgeography` (geografía temporal), `tcbuffer` (búfer circular temporal), `tpose` (pose temporal) y `trgeometry` (geometría rígida temporal). A nivel conceptual, todos los tipos espaciotemporales se organizan en la jerarquía que muestra la Figura 7.1.

Para habilitar lo anterior fue necesaria una refactorización del código. Todos los tipos que proporcionaba la versión 1.2, además de los nuevos tipos `tgeometry` y `tgeography`, se definen como tipos centrales y, por lo tanto, se incluyen siempre en el proceso de construcción. Cada uno de los demás tipos espaciotemporales se puede incluir en el proceso de construcción indicando su bandera de compilación correspondiente. Por ejemplo, al añadir `-DCBUFFER=1` se incluyen en el proceso de construcción los tipos construidos sobre el tipo base `cbuffer`, es decir `cbuffer`, `cbufferset` y `tcbuffer`. La posibilidad de seleccionar los tipos que se incluyen en el programa ejecutable es importante en particular para el procesamiento de flujos, dada la capacidad limitada de los dispositivos de borde. Tenga en cuenta que los nuevos tipos `tcbuffer`, `tpose` y `trgeometry` están todavía en desarrollo y se finalizarán en una versión próxima.

Además, la API de MobilityDB y de MEOS se amplió con nuevas funcionalidades y se simplificó para mejorar su usabilidad. Por último, la versión 1.3 también habilita las últimas versiones PostgreSQL 18 y PostGIS 3.6.0. Por todas estas razones, el formato binario de los tipos temporales cambió de la versión 1.2 a la 1.3 y, por lo tanto, es necesario hacer una copia de seguridad y restaurarla para migrar a la nueva versión 1.3 de MobilityDB.

1.9. Migración de la versión 1.3 a la versión 1.4

Todo tipo que define la distancia de aproximación más cercana con respecto a una caja espaciotemporal la expresa en la versión 1.4 mediante la función `nearestApproachDistance` y el operador `|=|`. En la versión 1.3 los tipos `cbuffer` y `pose` la expresaban mediante la función `distance` y el operador `<->`, mientras que `geometry` y `geography` ya usaban la primera. Las signaturas afectadas son `(cbuffer, stbox)`, `(stbox, cbuffer)`, `(pose, stbox)` y `(stbox, pose)`.

Las dos funciones responden preguntas distintas, que es lo que expresa el cambio. Una caja espaciotemporal lleva un período, de modo que `nearestApproachDistance` responde la distancia mínima durante el tiempo que comparten sus dos argumentos, y responde el valor nulo cuando no comparten ninguno. La función `distance` mide sus dos argumentos como conjuntos de puntos y no lee ningún período. Las signaturas de `distance` cuyos argumentos no llevan período conservan el nombre.

La versión 1.4 da a MobilityDB un único dialecto SQL en sus enlaces SQL por lotes y de flujo, PostgreSQL, DuckDB, Apache Spark y Apache Flink, que nombran cada función de la misma manera. Un nombre de función debe por lo tanto estar libre en cada uno de estos motores. Las funciones de comparación alguna vez, siempre y temporales se nombran con un cuantificador de una letra, `e`, `a` o `t`, seguido de la comparación escrita por completo: un nombre más corto como `eLt` coincidiría con la función `elt` de Apache Spark y Apache Flink. Los operadores conservan sus símbolos. La tabla siguiente da las funciones detrás de cada operador en ambas versiones.

Operadores	Versión 1.3	Versión 1.4
`?=`, `%=`, `#=`	`ever_eq`, `always_eq`, `temporal_teq`, `tgeo_teq`	`eEqual`, `aEqual`, `tEqual`
`?<>`, `%<>`, `#<>`	`ever_ne`, `always_ne`, `temporal_tne`, `tgeo_tne`	`eNotEqual`, `aNotEqual`, `tNotEqual`
`?<`, `%<`, `#<`	`ever_lt`, `always_lt`, `temporal_tlt`	`eLessThan`, `aLessThan`, `tLessThan`
`?<=`, `%<=`, `#<=`	`ever_le`, `always_le`, `temporal_tle`	`eLessEqual`, `aLessEqual`, `tLessEqual`
`?>`, `%>`, `#>`	`ever_gt`, `always_gt`, `temporal_tgt`	`eGreaterThan`, `aGreaterThan`, `tGreaterTha`
`?>=`, `%>=`, `#>=`	`ever_ge`, `always_ge`, `temporal_tge`	`eGreaterEqual`, `aGreaterEqual`, `tGreaterE`

Las funciones de posición, entre las cuales `left` y `right` coinciden con funciones de Apache Spark y Apache Flink, llevan como prefijo la clase cuyos valores o cuadros delimitadores comparan: `set`, `span` o `spanset` para los tipos conjunto, rango y conjunto de rangos, y para un cuadro delimitador o un valor temporal el tipo de su cuadro delimitador, `tbox`, `stbox` o `tpcbox`. Un valor temporal booleano, de texto o JSONB está delimitado por su rango de tiempo y lleva `span`. Las posiciones en el tiempo y a lo largo de los tres ejes espaciales llevan el prefijo por igual. La siguiente tabla da las funciones en ambas versiones.

Versión 1.3	Versión 1.4
before	setBefore, spanBefore, spansetBefore, tboxBefore, stboxBefore
after	setAfter, spanAfter, spansetAfter, tboxAfter, stboxAfter, tpcb
overbefore	setOverbefore, spanOverbefore, spansetOverbefore, tboxOverbe
overafter	setOverafter, spanOverafter, spansetOverafter, tboxOverafter
left	setLeft, spanLeft, spansetLeft, tboxLeft, stboxLeft, tpcboxLeft
right	setRight, spanRight, spansetRight, tboxRight, stboxRight, tpcb
overleft	setOverleft, spanOverleft, spansetOverleft, tboxOverleft, stb
overright	setOverright, spanOverright, spansetOverright, tboxOverright
below, above, overbelow, overabove	stboxBelow, stboxAbove, stboxOverbelow, stboxOverabove, tpcbo
front, back, overfront, overback	stboxFront, stboxBack, stboxOverfront, stboxOverback, tpcboxF

Las funciones topológicas, entre las cuales contains y overlaps coinciden con una función o una palabra reservada de Apache Spark y Apache Flink, llevan como prefijo la clase cuyos valores o cuadros delimitadores comparan, como las funciones de posición. La función same de un búfer circular, un punto de red, una pose y una cadena de poses, que el manual documenta como función de estos tipos, conserva su nombre. Las funciones que comparan los identificadores de ruta de los puntos de red temporales se escriben sin guion bajo. La siguiente tabla da las funciones en ambas versiones.

Versión 1.3	Versión 1.4
overlaps	setOverlaps, spanOverlaps, spansetOverlaps, tboxOverlaps, stboxOverlaps, tpcboxO
contains	setContains, spanContains, spansetContains, tboxContains, stboxContains, tpcboxC
contained	setContained, spanContained, spansetContained, tboxContained, stboxContained, tp
adjacent	spanAdjacent, spansetAdjacent, tboxAdjacent, stboxAdjacent, tpcboxAdjacent
same	spanSame, tboxSame, stboxSame, tpcboxSame
same_rid	sameRid
contains_rid	containsRid
contained_rid	containedRid
overlaps_rid	overlapsRid

Capítulo 2

Tipos de conjunto y de rango

MobilityDB proporciona los tipos de *conjunto*, *rango* y *conjunto de rangos* para representar conjuntos de valores de otro tipo, que se denomina *tipo base*. Los tipos de conjunto son similares a los tipos de matrices de PostgreSQL restringidos a una dimensión, pero imponen la restricción de que los conjuntos no tienen duplicados. Los tipos de rango y conjunto de rangos en MobilityDB corresponden a los tipos de rango y multirango en PostgreSQL pero tienen restricciones adicionales. En particular, los tipos de rango en MobilityDB tienen una longitud fija y no permiten rangos vacíos ni límites infinitos. Si bien los tipos de rango en MobilityDB proporcionan una funcionalidad similar a los tipos de rango en PostgreSQL, los tipos de rango en MobilityDB permiten aumentar el rendimiento. En particular, se elimina la sobrecarga del procesamiento de tipos de longitud variable y, además, se pueden utilizar la aritmética de punteros y la búsqueda binaria.

Los tipos de base que se utilizan para construir tipos de conjunto, de rango y de conjunto de rangos son los tipos `integer`, `bigint`, `float`, `text`, `date`, and `timestamptz` (marca de tiempo con zona horaria) proporcionados por PostgreSQL, los tipos `geometry` y `geography` proporcionados por PostGIS, y los tipos `cbuffer` (búfer circular, ver Capítulo 12) y `npoint` (*network point* o punto de red, ver Capítulo 11) proporcionados por MobilityDB. MobilityDB proporciona los siguientes tipos de conjunto y de rango:

- `set`: `intset`, `bigintset`, `floatset`, `textset`, `dateset`, `tstzset`, `geomset`, `geogset`, `cbufferset`, `npointset`.
- `span`: `intspan`, `bigintspan`, `floatspan`, `datespan`, `tstzspan`.
- `spanset`: `intspanset`, `bigintspanset`, `floatspanset`, `datespanset`, `tstzspanset`.

A continuación presentamos las funciones y operadores para tipos de conjunto y de rango. Estas funciones y operadores son polimórficos, es decir, sus argumentos pueden ser de varios tipos y el tipo de resultado puede depender del tipo de los argumentos. Para expresar esto en la firma de las funciones y los operadores, utilizamos la siguiente notación:

- `set` representa cualquier tipo de conjunto, como `intset` o `tstzset`.
- `span` representa cualquier tipo de rango, como `intspan` o `tstzspan`.
- `spanset` representa cualquier tipo de conjunto de rangos, como `intspanset` o `tstzspanset`.
- `spans` representa cualquier tipo de rango o conjunto de rangos, como `intspan` o `tstzspanset`.
- `base` representa cualquier tipo de base de un tipo de conjunto o de rango, como `integer` o `timestamptz`.
- `number` representa cualquier tipo de base de un tipo de rango numérico, como `integer` o `float`,
- `numset` representa cualquier tipo de conjunto numérico, como `intset` o `floatset`.
- `numspans` representa cualquier tipo de rango o conjunto de rangos numérico, como `intspan` o `floatspanset`.
- `numspan` representa cualquier tipo de rango numérico, es decir, `intspan`, `bigintspan` o `floatspan`,

- `numspanset` representa cualquier tipo de conjunto de rangos numérico, es decir, `intspanset`, `bigintspanset` o `floatspanset`,
- `floatspans` representa un tipo de rango o conjunto de rangos de números flotantes, es decir, `floatspan` o `floatspanset`,
- `datespans` representa un tipo de rango o conjunto de rangos de fechas, es decir, `datespan` o `datespanset`,
- `tstzspans` representa un tipo de rango o conjunto de rangos de timestamptz, es decir, `tstzspan` o `tstzspanset`,
- `spatialset` representa cualquier tipo de conjunto espacial, como `geomset` o `geogset`.
- `numbers` representa cualquier tipo de conjunto o de rango numérico, como `integer`, `intset`, `intspan` o `intspanset`,
- `dates` representa cualquier tipo de tiempo con granularidad `date`, a saber `date`, `dateset`, `datespan` o `datespanset`,
- `times` representa cualquier tipo de tiempo con granularidad `timestamptz`, a saber `timestamptz`, `tstzset`, `tstzspan` o `tstzspanset`,
- Un conjunto de tipos como `{set,base}` representa cualquiera de los tipos enumerados,
- `type[]` representa una matriz de `type`.

Como ejemplo, la firma del operador contiene (`@>`) es como sigue:

```
{set,spans} @> {set,spans,base} → boolean
```

Nótese que la firma anterior es una versión abreviada de la firma más precisa a continuación

```
set @> {set,base} → boolean
spans @> {spans,base} → boolean
```

ya que los conjuntos y los rangos no se pueden mezclar en las operaciones y, por lo tanto, por ejemplo, no se puede preguntar si un rango contiene un conjunto. A continuación, por concisión, utilizamos el estilo abreviado de las firmas anteriores. Además, la parte de tiempo de las marcas de tiempo se omite en la mayoría de los ejemplos. Recuerde que en ese caso PostgreSQL asume el tiempo `00:00:00`.

A continuación, dado que los tipos de rango y conjunto de rangos tienen funciones y operadores similares, cuando hablamos de tipos rango nos referimos a los tipos de rango y conjuntos de rangos, a menos que nos refiramos explícitamente a los tipos de rango *unitarios* y a los tipos de *conjunto* de rangos para distinguirlos. Además, cuando nos referimos a tipos de tiempo, nos referimos a uno de los siguientes tipos: `timestamptz`, `tstzset`, `tstzspan` o `tstzspanset`.

2.1. Entrada y salida

MobilityDB generaliza los formatos de entrada y salida Well-Known Text (WKT) y Well-Known Binary (WKB) del Open Geospatial Consortium (OGC) a para todos sus tipos. De esta forma, las aplicaciones pueden intercambiar datos entre ellas utilizando un formato de intercambio estandarizado. El formato WKT es legible por humanos, mientras que el formato WKB es más compacto y más eficiente que el formato WKT. El formato WKB se puede generar como una cadena binaria o como una cadena de caracteres codificada en ASCII hexadecimal.

Los tipos de conjunto representan un conjunto *ordenado* de valores *diferentes*. Un conjunto debe contener al menos un elemento. Ejemplos de valores de tipos de conjunto son como sigue:

```
SELECT tstzset '{2001-01-01 08:00:00, 2001-01-03 09:30:00}';
-- Singleton set
SELECT textset '{"highway"}';
-- Erroneous set: unordered elements
SELECT floatset '{3.5, 1.2}';
-- Erroneous set: duplicate elements
SELECT geomset '{"Point(1 1)", "Point(1 1)"}';
-- Circular-buffer set: each element is a cbuffer WKT
SELECT cbufferset '{"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)"}';
```

Nótese que los elementos de los conjuntos `textset`, `geomset`, `geogset`, `cbufferset` y `npointset` deben estar delimitados entre commillas dobles. Nótese también que las geometrías y las geografías utilizan el orden definido por PostGIS.

Un valor de un tipo de rango unitario tiene dos límites, el *límite inferior* y el *límite superior*, que son valores del *tipo de base* subyacente. Por ejemplo, un valor del tipo `tstzspan` tiene dos límites, que son valores de `timestamptz`. Los límites pueden ser inclusivos o exclusivos. Un límite inclusivo significa que el instante límite está incluido en el rango, mientras que un límite exclusivo significa que el instante límite no está incluido en el rango. En el formato textual de un valor de un rango, los límites inferiores inclusivos y exclusivos están representados, respectivamente, por "`[`" y "`(`". Asimismo, los límites superiores inclusivos y exclusivos se representan, respectivamente, por "`]`" y "`)`". En un valor de un rango, el límite inferior debe ser menor o igual que el límite superior. Un valor de rango con límites iguales e inclusivos se llama *rango instantáneo* y corresponde a un valor del tipo de base. Ejemplos de valores de rango son como sigue:

```
SELECT intspan '[1, 3)';
SELECT floatspan '[1.5, 3.5]';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-03 09:30:00)';
-- Instant spans
SELECT intspan '[1, 1]';
SELECT floatspan '[1.5, 1.5]';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00]';
-- Erroneous span: invalid bounds
SELECT tstzspan '[2001-01-01 08:10:00, 2001-01-01 08:00:00]';
-- Erroneous span: empty span
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00)';
```

Los valores de `intspan`, `bigintspan` y `datespan` son convertidos en *forma normal* para que los valores equivalentes tengan representaciones idénticas. En la representación canónica de estos tipos, el límite inferior es inclusivo y el límite superior es exclusivo, como se muestra en los siguientes ejemplos:

```
SELECT intspan '[1, 1]';
-- [1, 2)
SELECT bigintspan '(1, 3]';
--[2, 4)
SELECT datespan '[2001-01-01, 2001-01-03]';
-- [2001-01-01, 2001-01-04)
```

La forma normal reemplaza un límite inferior exclusivo o un límite superior inclusivo por el valor siguiente del tipo base, por lo que un rango cuyo límite es el mayor valor del tipo base y requiere ese reemplazo es rechazado, como en los rangos correspondientes de PostgreSQL:

```
SELECT span(1, 2147483647, true, false);
-- [1, 2147483647)
SELECT span(1, 2147483647, true, true);
-- ERROR:  integer out of range
```

Un valor de un tipo de conjunto de rangos representa un conjunto *ordenado* de valores de rango *disjuntos*. Un valor de conjunto de rangos debe contener al menos un elemento, en cuyo caso corresponde a un único valor de rango. Ejemplos de valores conjunto de rangos son los siguientes:

```
SELECT floatspanset '{[8.1, 8.5],[9.2, 9.4]}';
-- Singleton spanset
SELECT tstzspanset '{[2001-01-01 08:00:00, 2001-01-01 08:10:00]}';
-- Erroneous spanset: unordered elements
SELECT intspanset '{[3,4],[1,2]}';
-- Erroneous spanset: overlapping elements
SELECT tstzspanset '{[2001-01-01 08:00:00, 2001-01-01 08:10:00],
  [2001-01-01 08:05:00, 2001-01-01 08:15:00]}';
```

Los valores de los tipos conjunto de rangos son convertidos en *forma normal* de modo que los valores equivalentes tengan representaciones idénticas. Para ello, los valores de rango consecutivos que son adyacentes se fusionan cuando es posible. Ejemplos de transformación a forma normal son los siguientes:

```
SELECT intspanset '{[1,2],[3,4]}';
-- {[1, 5)}
SELECT floatspanset '{[1.5,2.5],(2.5,4.5]}';
-- {[1.5, 4.5]}
SELECT tstzspanset '{[2001-01-01 08:00:00, 2001-01-01 08:10:00),
  [2001-01-01 08:10:00, 2001-01-01 08:10:00],
  (2001-01-01 08:10:00, 2001-01-01 08:20:00]}';
-- {[2001-01-01 08:00:00,2001-01-01 08:20:00]}
```

Damos a continuación las funciones de entrada y salida de tipos de conjunto y de rango en formato Well-Known Text (WKT) y Well-Known Binary (WKB). El formato de salida predeterminado de todos los tipos de conjuntos y de rango es el formato de texto conocido. La función `asText` que se da a continuación permite determinar la salida de valores de punto flotante.

- Devuelve la representación de texto conocido (WKT)

```
asText({floatset,floatspans},maxdecimaldigits integer=15) → text
```

El argumento `maxdecimaldigits` se puede utilizar para definir el número máximo de decimales para la salida de los valores de coma flotante (por defecto 15).

```
SELECT asText(floatset '{1.123456789,2.123456789}', 3);
-- {1.123, 2.123}
SELECT asText(floatspanset '{[1.55,2.55],[4,5]}',0);
-- {[2, 3], [4, 5]}
```

- Devuelve la representación extendida de texto conocido (EWKT) de un conjunto espacial

```
asEWKT(spatialset,maxdecimaldigits integer=15) → text
```

El SRID del conjunto precede a sus elementos. Un conjunto de índices de celda indica el SRID 4326 y un conjunto de nubes de puntos el SRID que declara su esquema, y ninguno de los dos toma el argumento `maxdecimaldigits`.

```
SELECT asEWKT(geomset 'SRID=5676;{"Point(1 1)", "Point(2 2)"}');
-- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(h3indexset '{8928308280fffff, 8928308280bffff}');
-- SRID=4326;{"8928308280bffff", "8928308280fffff"}
```

- Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)

```
asBinary({set,spans},endian text='') → bytea
asHexWKB({set,spans},endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(dateset '{2001-01-01, 2001-01-03}');
-- \x01050001020000006e01000070010000
SELECT asBinary(intspan '[1, 3)');
-- \x0113000101000000003000000
SELECT asBinary(floatspanset '{[1, 2], [4, 5]}', 'XDR');
-- \x00000e00000002033ff0000000000000040000000000000000340100000000000004014000000000000
SELECT asHexWKB(dateset '{2001-01-01, 2001-01-03}');
-- 01050001020000006E01000070010000
SELECT asHexWKB(intspan '[1, 3)');
-- 0113000101000000003000000
SELECT asHexWKB(floatspanset '{[1, 2], [4, 5]}', 'XDR');
-- 00000E00000002033FF0000000000000040000000000000000340100000000000004014000000000000
```

- Devuelve la representación extendida binaria conocida (EWKB) o la representación hexadecimal extendida binaria conocida (HexEWKB) de un conjunto espacial

```
asEWKB(spatialset,endian text='') → bytea
asHexEWKB(spatialset,endian text='') → text
```

El SRID del conjunto, cuando se conoce, se incluye una única vez en la cabecera del conjunto: `asEWKB` y `asHexEWKB` lo incluyen, mientras que `asBinary` y `asHexWKB` devuelven la representación binaria conocida (WKB) sin él, siguiendo la convención OGC y el `asBinary`/`asHexWKB` de los tipos base `cbuffer`, `npoint` y `pose`. Un conjunto de índices de celda indica el SRID 4326, y un conjunto de nubes de puntos el SRID que su esquema declara en el catálogo `pointcloud_formats`, por lo que las formas extendidas y simples de un conjunto de nubes de puntos coinciden cuando su esquema no declara ningún SRID.

```
SELECT asBinary(geomset 'SRID=4326;{Point(1 1)}');
-- \x012b0001010000000101000000000000000000f03f000000000000f03f
SELECT asHexWKB(geomset 'SRID=4326;{Point(1 1)}');
-- 012B0001010000000101000000000000000000F03F000000000000F03F
SELECT asEWKB(geomset 'SRID=4326;{Point(1 1)}');
-- \x012b0041e6100000010000000101000000000000000000f03f000000000000f03f
SELECT asHexEWKB(geomset 'SRID=4326;{Point(1 1)}');
-- 012B0041E6100000010000000101000000000000000000F03F000000000000F03F
SELECT asEWKB(cbufferset 'SRID=4326;{"Cbuffer(Point(1 1),0.5)"}');
-- \x013a0041e6100000010000000000000000000f03f000000000000f03f000000000000e03f
SELECT asHexEWKB(npointset '{"NPoint(1,0.5)"}');
-- 01310041E6100000010000000101000000000000000000000000000000E03F
SELECT asHexEWKB(h3indexset '{8a2a1072b59ffff}', 'XDR');
-- 00004561000010E60000000108A2A1072B59FFFF
```

- Entrada desde la representación de texto conocido (WKT)

```
settypeFromText(text) → set
spantypeFromText(text) → span
spansettypeFromText(text) → spanset
```

Hay una función por tipo de conjunto o de rango, el nombre de la función tiene como prefijo el nombre del tipo. La función es la inversa de `asText`: lee el texto en el que se escribe un literal del tipo, tanto de una constante como de una columna `text`. Un conjunto espacial lee el texto conocido de sus elementos; su texto conocido extendido, que indica el SRID, lo lee `FromEWKT` a continuación.

```
SELECT intsetFromText('{1, 2, 3}');
-- {1, 2, 3}
SELECT textsetFromText('{"highway", "car"}');
-- {"car", "highway"}
SELECT floatspanFromText('[1.5, 2.5)');
-- [1.5, 2.5)
SELECT tstzspanFromText('[2001-01-01, 2001-01-03)');
-- [2001-01-01, 2001-01-03)
SELECT intspansetFromText('{[1, 3), [4, 5]}');
-- {[1, 3), [4, 6)}
SELECT asText(geomsetFromText('{"Point(1 1)", "Point(2 2)"}'));
-- {"POINT(1 1)", "POINT(2 2)"}
```

- Entrada desde la representación binaria conocida (WKB)

```
settypeFromBinary(bytea) → set
spantypeFromBinary(bytea) → span
spansettypeFromBinary(bytea) → spanset
```

Hay una función por tipo de conjunto o de rango, el nombre de la función tiene como prefijo el nombre del tipo.

```
SELECT datesetFromBinary('\x01050001020000006e01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromBinary('\x011300010100000003000000');
```

```
-- [1, 3)
SELECT floatspansetFromBinary(
  '\x00000e00000002033ff0000000000004000000000000000034010000000000004014000000000000');
-- {[1, 2], [4, 5]}
```

- Entrada desde la representación hexadecimal binaria conocida (HexWKB)

```
settypeFromHexWKB(text) → set
spantypeFromHexWKB(text) → span
spansettypeFromHexWKB(text) → spanset
```

Hay una función por tipo de conjunto o de rango, el nombre de la función tiene como prefijo el nombre del tipo.

```
SELECT datesetFromHexWKB('01050001020000006E01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromHexWKB('011300010100000003000000');
-- [1, 3)
SELECT floatspanFromHexWKB('0106000100000000000F83F0000000000000440');
-- [1.5, 2.5)
SELECT floatspansetFromHexWKB(
  '00000E00000002033FF0000000000004000000000000000034010000000000004014000000000000');
-- {[1, 2], [4, 5]}
```

- Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT) de un conjunto espacial

```
spatialsettypeFromText(text) → spatialset
spatialsettypeFromEWKT(text) → spatialset
```

Hay una función por tipo de conjunto espacial, el nombre de la función tiene como prefijo el nombre del tipo. Un SRID indicado debe ser el que lleva un conjunto de índices de celda o de nubes de puntos, de lo contrario se genera un error.

```
SELECT asText(geomsetFromText('{"Point(1 1)", "Point(2 2)"}'));
-- {"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(geomsetFromEWKT('SRID=5676;{"Point(1 1)", "Point(2 2)"}'));
-- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(h3indexsetFromEWKT('SRID=4326;{8928308280fffff}'));
-- SRID=4326;{"8928308280fffff"}
SELECT h3indexsetFromEWKT('SRID=3857;{8928308280fffff}');
-- ERROR:  The SRID 3857 cannot be stated for the h3index
```

- Entrada desde la representación extendida binaria conocida (EWKB) o desde la representación hexadecimal extendida binaria conocida (HexEWKB) de un conjunto espacial

```
spatialsettypeFromEWKB(bytea) → spatialset
spatialsettypeFromHexEWKB(text) → spatialset
```

Hay una función por tipo de conjunto espacial, el nombre de la función tiene como prefijo el nombre del tipo. El SRID que indica la cabecera debe ser el que lleva un conjunto de índices de celda o de nubes de puntos, de lo contrario se genera un error.

```
SELECT asEWKT(geomsetFromEWKB(asEWKB(geomset 'SRID=5676;{"Point(1 1)"}')));
-- SRID=5676;{"POINT(1 1)"}
SELECT asEWKT(quadbinsetFromHexEWKB('01480041E610000001000000FFFFFFFFFFFF0F48'));
-- SRID=4326;{"480fffffffffffff"}
```

2.2. Constructores

La función constructora para los tipos de conjunto tiene un único argumento que es una matriz de valores del tipo base correspondiente. Los valores deben estar ordenados y no pueden tener nulos ni duplicados.

- Constructor para tipos de conjunto

```
set(base[]) → set
```

```
SELECT set(ARRAY['highway', 'primary', 'secondary']);
-- {"highway", "primary", "secondary"}
SELECT set(ARRAY[timestamptz '2001-01-01 08:00:00', '2001-01-03 09:30:00']);
-- {2001-01-01 08:00:00, 2001-01-03 09:30:00}
```

Los tipos de rango unitarios tienen una función constructora que acepta cuatro argumentos, donde los dos últimos son opcionales. Los primeros dos argumentos especifican, respectivamente, el límite inferior y el superior, y los dos últimos argumentos son valores booleanos que indican, respectivamente, si los límites inferior y el superior son inclusivos o no. Se supone que los dos últimos argumentos son, respectivamente, verdadero y falso si no se especifican. Nótese que los rangos de enteros se transforman en *forma normal*, es decir, con límite inferior inclusivo y límite superior exclusivo.

- Constructor para tipos de rango

```
span(lower base,upper base,lower_inc boolean=true,upper_inc boolean=false) → span
```

```
SELECT span(20.5, 25);
-- [20.5, 25)
SELECT span(20, 25, false, true);
-- [21, 26)
SELECT span(timestamptz '2001-01-01 08:00:00', '2001-01-03 09:30:00', false, true);
-- (2001-01-01 08:00:00, 2001-01-03 09:30:00]
```

La función constructora para los tipos de conjuntos de rangos tiene un solo argumento que es una matriz de rangos del mismo subtipo.

- Constructor for conjunto de rangos

```
spanset(span[]) → spanset
```

```
SELECT spanset(ARRAY[intspan '[10,12]', '[13,15]']);
-- {[10, 16)}
SELECT spanset(ARRAY[floatspan '[10.5,12.5]', '[13.5,15.5]']);
-- {[10.5, 12.5], [13.5, 15.5]}
SELECT spanset(ARRAY[tstzspan '[2001-01-01 08:00, 2001-01-01 08:10]',
  '[2001-01-01 08:20, 2001-01-01 08:40]']);
-- {[2001-01-01 08:00, 2001-01-01 08:10], [2001-01-01 08:20, 2001-01-01 08:40]};
```

2.3. Conversión de tipos

Los valores de los tipos de conjunto y de rango se pueden convertir entre sí o convertirse a tipos de rango de PostgreSQL y desde ellos mediante la función `CAST` o mediante la notación `::`.

- Convierte un valor de base en un valor de conjunto, rango o conjunto de rangos

```
base::{set,span,spanset}
set(base) → set
span(base) → span
spanset(base) → spanset
```

```
SELECT CAST(timestamptz '2001-01-01 08:00:00' AS tstzset);
-- {2001-01-01 08:00:00}
SELECT timestamptz '2001-01-01 08:00:00'::tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:00:00]
SELECT spanset(timestamptz '2001-01-01 08:00:00');
-- {[2001-01-01 08:00:00, 2001-01-01 08:00:00]}
```

- Convierte un valor de conjunto en un valor de conjunto de rangos

```
set::spanset
spanset(set) → spanset
```

```
SELECT spanset(tstzset '{2001-01-01 08:00:00, 2001-01-01 08:15:00, 2001-01-01 08:25:00}');
/* {[2001-01-01 08:00:00, 2001-01-01 08:00:00], [2001-01-01 08:15:00,
   2001-01-01 08:15:00],
   [2001-01-01 08:25:00, 2001-01-01 08:25:00]} */
```

- Convierte un conjunto geo en la colección de sus valores

```
geomset::geometry
geogset::geography
geometry(geomset) → geometry
geography(geogset) → geography
```

La colección toma su tipo únicamente de los tipos de los valores, como la construye a partir de ellos la función de PostGIS `ST_Collect`, de modo que un conjunto de puntos, incluido un conjunto de un punto, devuelve un multipunto.

```
SELECT ST_AsText(geomset '{"Point(1 1)", "Point(2 2)"}'::geometry);
-- MULTIPOINT((1 1),(2 2))
SELECT ST_AsText(geomset '{"Point(1 1)"}'::geometry);
-- MULTIPOINT((1 1))
SELECT ST_AsText(geometry(geomset '{"Point(1 1)", "Linestring(0 0,1 1)"}'));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(0 0,1 1))
```

- Convierte entre una geometría y una geografía

```
geomset::geogset
geogset::geomset
geogset(geomset) → geogset
geomset(geogset) → geomset
```

Un valor espacial de cualquier tipo se convierte a una geometría y a una geografía, sea planar o geodésico. La conversión cambia el tipo y nunca las coordenadas, de modo que una medida sigue luego al destino: sobre el plano para una geometría, sobre la superficie de la Tierra para una geografía. Como geografía, un valor sin SRID tiene el SRID 4326 y un valor en un sistema de referencia proyectado genera un error, como en PostGIS. La regla vale para todo tipo espacial y todo tipo temporal, cada uno de los cuales llega a la misma conversión de sus valores de geometría o geografía.

```
SELECT ST_AsEWKT(geometry 'Point(1 2)'::geography);
-- SRID=4326;POINT(1 2)
SELECT asEWKT(geomset '{"Point(1 2)"}'::geogset);
-- SRID=4326;{"POINT(1 2)"}
SELECT asEWKT(geogset '{"Point(1 2)"}'::geomset);
-- SRID=4326;{"POINT(1 2)"}
SELECT geomset 'SRID=3857;{"Point(1 2)"}'::geogset;
-- ERROR:  Only lon/lat coordinate systems are supported
```

- Convierte un valor de rango a un valor de conjunto de rangos

```
span::spanset
spanset(span) → spanset
```

```
SELECT floatspan '[1.5,2.5]'::floatspanset;
-- {[1.5, 2.5]}
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00)'::tstzspanset;
-- {[2001-01-01 08:00:00, 2001-01-01 08:30:00)}
```

- Convierte un conjunto de valores o un conjunto de rangos a un rango, ignorando las posibles brechas de tiempo

```
{set,spanset}::span
span({set,spanset}) → span
```

```
SELECT span(dateset '{2001-01-01, 2001-01-03, 2001-01-05}');
-- [2001-01-01, 2001-01-06)
SELECT span(tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- [2001-01-01, 2001-01-04)
```

- Convierte un valor de de rango en MobilityDB hacia y desde un valor de rango de PostgreSQL

```
{span,range}::{range,span}
range(span) → range
span(range) → span
```

Nótese que los valores de rango en PostgreSQL aceptan rangos vacíos y rangos con límites infinitos, que no están permitidos como valores de rango en MobilityDB

```
SELECT intspan '[10, 20)'::int4range;
-- [10,20)
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00)'::tstzrange;
-- ["2001-01-01 08:00:00","2001-01-01 08:30:00")
SELECT int4range '[10, 20)'::intspan;
-- [10,20)
SELECT int4range 'empty'::intspan;
-- ERROR:  Range cannot be empty
SELECT int4range '[10,)'::intspan;
-- ERROR:  Range bounds cannot be infinite
SELECT tstzrange '[2001-01-01 08:00:00, 2001-01-01 08:30:00)'::tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:30:00)
```

- Convierte un valor de conjunto de rangos de MobilityDB hacia y desde un valor de multirango de PostgreSQL

```
{spanset,multirange}::{multirange,spanset}
multirange(spanset) → multirange
spanset(multirange) → spanset
```

```
SELECT intspanset '{[1,2],[4,5]}'::int4multirange;
-- {[1,3),[4,6)}
SELECT tstzspanset '{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}'::tstzmultirange;
-- {[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}
SELECT int4multirange '{[1,2],[4,5]}'::intspanset;
-- {[1, 3), [4, 6)}
SELECT tstzmultirange '{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}'::tstzspanset;
-- {[2001-01-01, 2001-01-02], [2001-01-04, 2001-01-05]}
```

2.4. Accesores

- Devuelve el tamaño de la memoria en bytes

```
memSize({set,spanset}) → integer
```

```
SELECT memSize(tstzset '{2001-01-01, 2001-01-02, 2001-01-03}');
-- 48
SELECT memSize(tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04],
  [2001-01-05, 2001-01-06]}');
-- 112
```

- Devuelve el límite inferior o superior

```
lower(spans) → base
upper(spans) → base
```

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
--  1
```

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-01
SELECT upper(intspanset '{[1,2],[4,5]}');
--  6
SELECT lower(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
--  1
```

```
SELECT upper(floatspan '[20.5, 25.3)');
-- 25.3
SELECT upper(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-05
```

- ¿Es el límite inferior or superior inclusivo?

```
lowerInc(spans) → boolean
upperInc(spans) → boolean
```

```
SELECT lowerInc(datespan '[2001-01-01, 2001-01-05)');
-- true
SELECT lowerInc(intspanset '{[1,2],[4,5]}');
-- true
```

```
SELECT upper(floatspan '[20.5, 25.3]');
-- true
SELECT upperInc(tstzspan '[2001-01-01, 2001-01-05)');
-- false
```

- Devuelve el ancho del rango como un número de punto flotante

```
width(numspan) → float
width(numspanset,boundspan boolean=false) → float
```

Se puede establecer a verdadero un parámetro adicional para calcular el ancho del rango delimitador, ignorando así las posibles brechas de valors

```
SELECT width(floatspan '[1, 3)');
-- 2
SELECT width(intspanset '{[1,3),[5,7)}');
-- 4
SELECT width(intspanset '{[1,3),[5,7)}', true);
-- 6
```

- Devuelve el rango de tiempo

```
duration({datespan,tstzspan}) → interval
duration({datespanset,tstzspanset},boundspan boolean=false) → interval
```

Se puede establecer a verdadero un parámetro adicional para calcular la duración en el rango delimitador, ignorando así las posibles brechas de tiempo

```
SELECT duration(datespan '[2001-01-01, 2001-01-03)');
-- 2 days
SELECT duration(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}');
-- 3 days
SELECT duration(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}', true);
-- 4 days
```

- Devuelve el número de valores o los valores

```
numValues(set) → integer
getValues(set) → base[]
```

```
SELECT numValues(intset '{1,3,5,7}');
-- 4
SELECT getValues(tstzset '{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}');
-- {"2001-01-01","2001-01-03","2001-01-05","2001-01-07"}
```

- Devuelve el valor inicial, final o enésimo

```
startValue(set) → base
endValue(set) → base
valueN(set,integer) → base
```

```
SELECT startValue(intset '{1,3,5,7}');
-- 1
SELECT endValue(dateset '{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}');
-- 2001-01-07
SELECT valueN(floatset '{1,3,5,7}',2);
-- 3
```

- Transforma un conjunto en un conjunto de filas, una por cada valor del conjunto, en el orden del conjunto {}

```
unnest(set) → {base}
```

```
SELECT unnest(intset '{1,3,5}');
-- 1
-- 3
-- 5
SELECT unnest(textset '{"highway", "car"}');
-- car
-- highway
SELECT ST_AsText(unnest(geomset '{"Point(1 1)", "Point(2 2)"}'));
-- POINT(1 1)
-- POINT(2 2)
```

- Devuelve el número de rangos o los rangos

```
numSpans(spanset) → integer
spans(spanset) → span[]
```

```
SELECT numSpans(intspanset '{[1,3),[4,5),[6,7)}');
-- 3
SELECT numSpans(datespanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05),
  [2001-01-06, 2001-01-07)}');
-- 3
```

```
SELECT spans(floatspanset '{[1,3),[4,4],[6,7)}');
-- {"[1,3)","[4,4]","[6,7)"}
SELECT spans(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
  [2001-01-05, 2001-01-06)}');
-- {"[2001-01-01,2001-01-03)", "[2001-01-04,2001-01-04]", "[2001-01-05,2001-01-06)"}
```

- Devuelve el rango inicial, final o enésimo

```
startSpan(spanset) → span
endSpan(spanset) → span
spanN(spanset,integer) → span
```

```
SELECT startSpan(intspanset '{[1,3),[4,5),[6,7)}');
-- [1,3)
SELECT startSpan(datespanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05),
  [2001-01-06, 2001-01-07)}');
-- [2001-01-01,2001-01-03)
```

```
SELECT endSpan(floatspanset '{[1,3),[4,4],[6,7)}');
-- [6,7)
SELECT endSpan(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
  [2001-01-05, 2001-01-06)}');
-- [2001-01-05,2001-01-06)
```

```
SELECT spanN(floatspanset '{[1,3),[4,4],[6,7)}',2);
-- [4,4]
SELECT spanN(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
  [2001-01-05, 2001-01-06)}', 2);
-- [2001-01-04,2001-01-04]
```

- Devuelve el (number of) different dates

```
numDates(datespanset) → integer
dates(datespanset) → dateset
```

```
SELECT numDates(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 4
SELECT dates(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04}
```

- Devuelve el start, end, or n-th date

```
startDate(datespanset) → date
endDate(datespanset) → date
dateN(datespanset,integer) → date
```

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startDate(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 2001-01-01
SELECT endDate(datespanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 2001-01-05
SELECT dateN(datespanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}', 3);
-- 2001-01-05
```

- Devuelve el número o las marcas de tiempo diferentes

```
numTimestamps(tstzspanset) → integer
timestamps(tstzspanset) → tstzset
```

```
SELECT numTimestamps(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 3
SELECT timestamps(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- {"2001-01-01 00:00:00", "2001-01-03 00:00:00", "2001-01-05 00:00:00"}
```

- Devuelve la marca de tiempo inicial, final, o enésima

```
startTimestamp(tstzspanset) → timestamptz
endTimestamp(tstzspanset) → timestamptz
timestampN(tstzspanset,integer) → timestamptz
```

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startTimestamp(tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 2001-01-01
SELECT endTimestamp(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 2001-01-05
SELECT timestampN(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}', 3);
-- 2001-01-05
```

2.5. Transformaciones

- Expande o reducir los límites de un rango con un valor o un intervalo de tiempo

```
expand(numspan,base) → numspan
expand(tstzspan,interval) → tstzspan
```

La función devuelve NULL si el valor o intervalo dado como segundo argumento es negativo y el rango resultante de desplazar los límites con el argumento está vacío.

```
SELECT expand(floatspan '[1, 3]', 1);
-- [0, 4]
SELECT expand(floatspan '[1, 3]', -1);
-- [2, 2]
SELECT expand(floatspan '[1, 3)', -1);
-- NULL
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
-- [2000-12-31, 2001-01-04]
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-1 day');
-- [2001-01-02, 2001-01-02]
SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-2 day');
-- NULL
```

- Desplaza con un valor o rango de tiempo

```
shift(numbers,base) → numbers
shift(dates,integer) → dates
shift(times,interval) → times
```

```
SELECT shift(dateset '{2001-01-01, 2001-01-03, 2001-01-05}', 1);
-- {2001-01-02, 2001-01-04, 2001-01-06}
SELECT shift(intspan '[1, 4)', -1);
-- [0, 3)
SELECT shift(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
-- [2001-01-02, 2001-01-04]
SELECT shift(floatspanset '{[1, 2], [3, 4]}', -1);
-- {[0, 1], [2, 3]}
SELECT shift(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}',
  interval '1 day');
-- {[2001-01-02, 2001-01-04], [2001-01-05, 2001-01-06]}
```

- Escala con un valor o un rango de tiempo

```
scale(numbers,base) → numbers
scale(dates,integer) → dates
scale(times,interval) → times
```

Si el ancho o el lapso de tiempo del valor de entrada es cero (por ejemplo, para un conjunto de marcas de tiempo único), el resultado es el valor de entrada. El valor o rango de tiempo dado debe ser estrictamente mayor que cero.

```
SELECT scale(tstzset '{2001-01-01}', '1 day');
-- {2001-01-01}
SELECT scale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '2 days');
-- {2001-01-01, 2001-01-02, 2001-01-03}
SELECT scale(intspan '[1, 4)', 4);
-- [1, 6)
SELECT scale(datespan '[2001-01-01, 2001-01-04)', 4);
-- [2001-01-01, 2001-01-06)
SELECT scale(tstzspan '[2001-01-01, 2001-01-03]', '1 day');
-- [2001-01-01, 2001-01-02]
SELECT scale(floatspanset '{[1, 2], [3, 4]}', 6);
-- {[1, 3], [5, 7]}
SELECT scale(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}', '1 day');
/* {[2001-01-01 00:00:00, 2001-01-01 12:00:00],
   [2001-01-01 18:00:00, 2001-01-02 00:00:00]} */
SELECT scale(tstzset '{2001-01-01}', '-1 day');
-- ERROR:  The duration must be a positive interval: -1 days
```

- Desplaza y escalear con los valores o rangos de tiempo

```
shiftScale(numbers,base,base) → numbers
shiftScale(dates,integer,integer) → dates
shiftScale(times,interval,interval) → times
```

Estas funciones combinan las funciones shift y scale.

```
SELECT shiftScale(tstzset '{2001-01-01}', '1 day', '1 day');
-- {2001-01-02}
SELECT shiftScale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '1 day','2 days');
-- {2001-01-02, 2001-01-03, 2001-01-04}
SELECT shiftScale(intspan '[1, 4)', -1, 4);
-- [0, 5)
SELECT shiftScale(datespan '[2001-01-01, 2001-01-04)', -1, 4);
-- [2000-12-31, 2001-01-05)
SELECT shiftScale(tstzspan '[2001-01-01, 2001-01-03]', '1 day', '1 day');
-- [2001-01-02, 2001-01-03]
```

```
SELECT shiftScale(floatspanset '{[1, 2], [3, 4]}', -1, 6);
-- {[0, 2], [4, 6]}
SELECT shiftScale(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}',
  '1 day', '1 day');
/* {[2001-01-02 00:00:00, 2001-01-02 12:00:00],
   [2001-01-02 18:00:00, 2001-01-03 00:00:00]} */
```

- Redondea al entero inferior o superior

```
floor({floatset,floatspans}) → {floatset,floatspans}
ceil({floatset,floatspans}) → {floatset,floatspans}
```

```
SELECT floor(floatset '{1.5,2.5}');
-- {1, 2}
SELECT ceil(floatspan '[1.5,2.5)');
-- [2, 3)
SELECT floor(floatspan '(1.5, 1.6)');
-- [1, 1]
SELECT ceil(floatspanset '{[1.5, 2.5],[3.5,4.5]}');
-- {[2, 3], [4, 5]}
```

- Redondea a un número de decimales

```
round({floatset,floatspans},integer=0) → {floatset,floatspans}
```

```
SELECT round(floatset '{1.123456789,2.123456789}', 3);
-- {1.123, 2.123}
SELECT round(floatspan '[1.123456789,2.123456789)', 3);
-- [1.123,2.123)
SELECT round(floatspan '[1.123456789, 1000000)', 3);
-- [1.123, 1000000)
SELECT round(floatspanset '{[1.123456789, 2.123456789],[3.123456789,4.123456789]}', 3);
-- {[1.123, 2.123], [3.123, 4.123]}
```

- Convierte a grados o radianes

```
degrees({floatset,floatspans},normalize boolean=false) → {floatset,floatspans}
radians({floatset,floatspans}) → {floatset,floatspans}
```

El parámetro adicional en la función `degrees` puede ser utilizado para normalizar los valores entre 0 y 360 grados.

```
SELECT round(degrees(floatset '{0, 0.5, 0.7, 1.0}', true), 3);
-- {0, 28.648, 40.107, 57.296}
SELECT round(radians(floatspanset '{[0, 45], [90, 135]}'), 3);
-- {[0, 0.785], [1.571, 2.356]}
```

- Transforma en minúsculas, majúsculas o initcap

```
lower(textset) → textset
upper(textset) → textset
initcap(textset) → textset
```

```
SELECT lower(textset '{"AAA", "BBB", "CCC"}');
-- {"aaa", "bbb", "ccc"}
SELECT upper(textset '{"aaa", "bbb", "ccc"}');
-- {"AAA", "BBB", "CCC"}
SELECT initcap(textset '{"aaa", "bbb", "ccc"}');
-- {"Aaa", "Bbb", "Ccc"}
```

- Concatenación de texto

```
{text,textset} || {text,textset} → textset
```

```
SELECT textset '{aaa, bbb}' || text 'XX';
-- {"aaaXX", "bbbXX"}
SELECT text 'XX' || textset '{aaa, bbb}';
-- {"XXaaa", "XXbbb"}
```

- Establece la precisión temporal del valor de tiempo al rango con respecto al origen

```
tprecision(times,duration interval,torigin timestamptz='2000-01-03') → times
```

Si el origen no se especifica, su valor se establece por defecto en lunes 3 de enero de 2000.

```
SELECT tprecision(timestamptz '2001-12-03', '30 days');
-- 2001-11-23
SELECT tprecision(timestamptz '2001-12-03', '30 days', '2001-12-01');
-- 2001-12-01
SELECT tprecision(tstzset '{2001-01-01 08:00, 2001-01-01 08:10, 2001-01-01 09:00,
  2001-01-01 09:10}', '1 hour');
-- {"2001-01-01 08:00:00", "2001-01-01 09:00:00"}
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-01 09:00]', '1 day');
-- [2001-12-01, 2001-12-02)
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-15 09:00]', '1 day');
-- [2001-12-01, 2001-12-16)
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
  [2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02)}
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
  [2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02)}
```

2.6. Sistema de referencia espacial

- Devuelve o especifica el identificador de referencia espacial

```
SRID(spatialset) → integer
setSRID(spatialset,integer) → spatialset
```

```
SELECT SRID(geomset '{"Point(1 1)", "Point(2 2)"}');
-- 0
SELECT SRID(geogset '{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 4326
SELECT SRID(geomset 'SRID=5676;{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 5676
SELECT asEWKT(setSRID(geomset '{Point(1 1), Point(2 2)}', 5676));
-- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(setSRID(poseset '{"Pose(Point(1 1),1)", "Pose(Point(2 2),3)"}', 5676));
-- SRID=5676;{"Pose(POINT(2 2),3)", "Pose(POINT(1 1),1)"}
SELECT SRID(cbufferset 'SRID=4326;
  {"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)"}');
-- 4326
SELECT asEWKT(setSRID(cbufferset '{"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)"}',
  3812));
-- SRID=3812;{"Cbuffer(POINT(1 1),0.5)", "Cbuffer(POINT(2 2),1)"}
```

- Transforma a una referencia espacial diferente

```
transform(spatialset,srid integer) → spatialset
transformPipeline(spatialset,pipeline text,srid integer,
  is_forward boolean=true) → spatialset
```

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el conjunto tiene un SRID desconocido (representado por 0). La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas en el siguiente formato:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID del conjunto de entrada se ignora y el SRID de la conjunto de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(geomset 'SRID=4326;
  {Point(2.340088 49.400250), Point(6.575317 51.553167)}', 3812), 6);
-- SRID=3812;{"POINT(502773.429981 511805.120402)", "POINT(803028.908265 751590.742629)"}
WITH test(geoset, pipeline) AS (
  SELECT geogset 'SRID=4326;
    {"Point(4.3525 50.846667 100.0)", "Point(-0.1275 51.507222 100.0)"}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(geoset, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
-- SRID=4326;{"POINT Z (4.3525 50.846667 100)", "POINT Z (-0.1275 51.507222 100)"}
```

2.7. Operaciones de conjuntos

Los tipos de conjunto y de rango tienen operadores de conjuntos asociados, a saber, unión, diferencia e intersección, que se representan, respectivamente, por +, − y * . Los operadores de conjunto para los tipos de rango y tiempo se dan a continuación.

- Unión, diferencia o intersección de conjuntos o de rangos

```
set {+, -, *} set → set
spans {+, -, *} spans → spans
```

```
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' + dateset '{2001-01-03,
  2001-01-06}';
-- {2001-01-01, 2001-01-03, 2001-01-05, 2001-01-06}
SELECT intspan '[1, 3)' + intspan '[3, 5)';
-- [1, 5)
SELECT floatspan '[1, 3)' + floatspan '[4, 5)';
-- {[1, 3), [4, 5)}
SELECT tstzspanset '{[2001-01-01, 2001-01-03),
  [2001-01-04, 2001-01-05)}' + tstzspan '[2001-01-03, 2001-01-04)';
-- {[2001-01-01, 2001-01-05)}
```

```
SELECT intset '{1, 3, 5}' - intset '{3, 6}';
-- {1, 5}
SELECT datespan '[2001-01-01, 2001-01-05)' - datespan '[2001-01-03, 2001-01-07)';
-- {[2001-01-01, 2001-01-03)}
SELECT floatspan '[1, 5]' - floatspan '[3, 4]';
-- {[1, 3), (4, 5]}
SELECT tstzspanset '{[2001-01-01, 2001-01-06],
  [2001-01-07, 2001-01-10]}' - tstzspanset '{[2001-01-02, 2001-01-03],
  [2001-01-04, 2001-01-05], [2001-01-08, 2001-01-09]}';
/* {[2001-01-01,2001-01-02), (2001-01-03,2001-01-04), (2001-01-05,2001-01-06],
   [2001-01-07,2001-01-08), (2001-01-09,2001-01-10]} */
```

```
SELECT tstzset '{2001-01-01, 2001-01-03}' * tstzset '{2001-01-03, 2001-01-05}';
-- {2001-01-03}
SELECT intspan '[1, 5)' * intspan '[3, 6)';
-- [3, 5)
SELECT floatspanset '{[1, 5),[6, 8)}' * floatspan '[1, 6)';
-- {[1, 5)}
SELECT tstzspan '[2001-01-01, 2001-01-05)' * tstzspan '[2001-01-03, 2001-01-07)';
-- [2001-01-03, 2001-01-05)
```

2.8. Operaciones de cuadro delimitador

2.8.1. Operaciones topológicas

A continuación se presentan los operadores topológicos disponibles para los tipos de conjunto y de rango.

Un valor, el rango que contiene únicamente ese valor y el conjunto de rangos que contiene únicamente ese rango denotan el mismo conjunto, al igual que un rango y el conjunto de rangos que contiene únicamente a él. Todas las operaciones dan la misma respuesta para todas estas escrituras.

Un conjunto de rangos es una unión de rangos disjuntos. Las operaciones `&&`, `@>`, `<@` y `~=` responden sobre el valor mismo, por lo que un valor que cae en un hueco de un conjunto de rangos no está contenido en él. La operación `-|-` y las operaciones de posición de la sección siguiente responden sobre la *caja delimitadora* del valor, el rango más pequeño que lo contiene: los huecos de un conjunto de rangos no son fronteras de él.

- ¿Se superponen los valores (tienen valores en común)?

```
{set,spans} && {set,spans} → boolean
```

```
SELECT intset '{1, 3}' && intset '{2, 3, 4}';
-- true
SELECT floatspan '[1, 3)' && floatspan '[3, 4)';
-- false
SELECT tstzspan '[2001-01-01, 2001-01-05)' && tstzspan '[2001-01-02, 2001-01-07)';
-- true
SELECT floatspanset '{[1, 5),[6, 8)}' && floatspan '[1, 6)';
-- true
```

- ¿Contiene el primer valor el segundo?

```
{set,spans} @> {base,set,spans} → boolean
```

```
SELECT floatset '{1.5, 2.5}' @> 2.5;
-- true
SELECT tstzspan '[2001-01-01, 2001-05-01)' @> timestamptz '2001-02-01';
-- true
SELECT floatspanset '{[1, 2),(2, 3)}' @> 2.0;
-- false
```

- ¿Está el primer valor contenido en el segundo?

```
{base,set,spans} <@ {set,spans} → boolean
```

```
SELECT timestamptz '2001-01-10' <@ tstzspan '[2001-01-01, 2001-05-01)';
-- true
SELECT floatspan '[2, 5]' <@ floatspan '[1, 5)';
-- false
SELECT tstzspan '[2001-02-01, 2001-03-01)' <@ tstzspan '[2001-01-01, 2001-05-01)';
```

```
-- true
SELECT floatspanset '{[1,2],[3,4]}' <@ floatspan '[1, 6]';
-- true
```

- ¿Es el primer valor adyacente al segundo?

```
spans -|- spans → boolean
```

Dos valores son adyacentes cuando se tocan sin solaparse, es decir, cuando comparten una frontera y no tienen ningún valor en común en ella. Para los tipos base continuos `float` y `timestamptz`, un límite compartido es ese contacto. Para los tipos base discretos `int`, `bigint` y `date`, cuyos rangos se almacenan en forma normal, significa valores consecutivos.

```
SELECT intspan '[2, 6)' -|- intspan '[6, 7)';
-- true
SELECT intspan '[1, 5]' -|- intspan '[6, 10]';
-- true
SELECT intspan '[1, 5]' -|- intspan '[5, 10]';
-- false
SELECT floatspan '[2, 5)' -|- floatspan '(5, 6)';
-- true
SELECT floatspan '[1, 5]' -|- floatspan '[5, 9]';
-- true
SELECT floatspanset '{[2, 3],[4, 5)}' -|- floatspan '(5, 6)';
-- true
SELECT tstzspanset '{[2001-01-01, 2001-01-02]}' -|- tstzspan '[2001-01-02, 2001-01-03)';
-- true
```

Un conjunto de rangos responde como su caja delimitadora, y las tres escrituras de un valor coinciden:

```
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- 3;
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- intspan '[3,3]';
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- intspanset '{[3,3]}';
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- 12;
-- true
```

2.8.2. Operaciones de posición

Los operadores de posición disponibles para los tipos de conjunto y de rango se dan a continuación. Observe que los operadores para tipos de tiempo tienen un # adicional para distinguirlos de los operadores para tipos de números.

- ¿Está el primer valor estrictamente a la izquierda del segundo?

```
numbers << numbers → boolean
times <<# times → boolean
```

```
SELECT intspan '[15, 20)' << 20;
-- true
SELECT intspanset '{[15, 17],[18, 20)}' << 20;
-- true
SELECT floatspan '[15, 20)' << floatspan '(15, 20)';
-- false
SELECT dateset '{2001-01-01, 2001-01-02}' <<# dateset '{2001-01-03, 2001-01-05}';
-- true
```

- ¿Está el primer valor estrictamente a la derecha del segundo?

```
numbers >> numbers → boolean
times #>> times → boolean
```

```
SELECT intspan '[15, 20)' >> 10;
-- true
SELECT floatspan '[15, 20)' >> floatspan '[5, 10]';
-- true
SELECT floatspanset '{[15, 17], [18, 20)}' >> floatspan '[5, 10]';
-- true
SELECT tstzspan '[2001-01-04, 2001-01-05)' #>>
  tstzspanset '{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06)}';
-- true
```

- ¿No está el primer valor a la derecha del segundo?

```
numbers &< numbers → boolean
times &<# times → boolean
```

```
SELECT intspan '[15, 20)' &< 18;
-- false
SELECT intspanset '{[15, 16],[17, 18)}' &< 18;
-- true
SELECT floatspan '[15, 20)' &< floatspan '[10, 20]';
-- true
SELECT dateset '{2001-01-02, 2001-01-05}' &<# dateset '{2001-01-01, 2001-01-04}';
-- false
```

- ¿No está el primer valor a la izquierda del segundo?

```
numbers &> numbers → boolean
times #&> times → boolean
```

```
SELECT intspan '[15, 20)' &> 30;
-- true
SELECT floatspan '[1, 6]' &> floatspan '(1, 3)';
-- false
SELECT floatspanset '{[1, 2],[3, 4]}' &> floatspan '(1, 3)';
-- false
SELECT timestamp '2001-01-01' #&> tstzspan '[2001-01-01, 2001-01-05)';
-- true
```

2.8.3. Operaciones de división

Al crear índices para tipos de conjuntos o conjunto de rangos, lo que se almacena en el índice no es el valor real, sino un cuadro delimitador que *representa* el valor. En este caso, el índice proporcionará una lista de valores candidatos que *pueden* satisfacer el predicado de la consulta, y se necesita un segundo paso para filtrar los valores candidatos calculando el predicado de la consulta sobre los valores reales.

Sin embargo, cuando los cuadros delimitadores tienen un gran espacio vacío no cubierto por los valores reales, el índice generará muchos valores candidatos que no satisfacen el predicado de la consulta, lo que reduce la eficiencia del índice. En estas situaciones, puede ser mejor representar un valor no con un cuadro delimitador *único*, sino con *múltiples* cuadros delimitadores. Esto aumenta considerablemente la eficiencia del índice, siempre que el índice sea capaz de gestionar múltiples cuadros delimitadores por valor. Las siguientes funciones se utilizan para generar múltiples rangos a partir de un único valor de conjunto o conjunto de rangos.

- Devuelve una matriz de N rangos obtenida fusionando los elementos de un conjunto o los rangos de un conjunto de rangos

```
splitNSpans({set,spanset},integer) → span[]
```

El último argumento especifica el número de rangos de salida. Si el número de elementos o rangos de entrada es menor que el número especificado, la matriz resultante tendrá un rango por elemento o rango de entrada. De lo contrario, el número especificado de rangos de salida se obtendrá fusionando elementos o rangos de entrada consecutivos.

```
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
-- {"[1, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 3);
-- {"[1, 5)","[5, 8)","[8, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 3)","[3, 5)","[5, 7)","[7, 9)","[9, 10)","[10, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
/* {"[1, 2)","[2, 3)","[3, 4)","[4, 5)","[5, 6)","[6, 7)","[7, 8)","[8, 9)",
   "[9, 10)","[10, 11)"} */
```

```
SELECT splitNSpans(intspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}');
-- {"[1, 2)","[3, 4)","[5, 6)","[7, 8)","[9, 10)"}
SELECT splitNSpans(floatspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}', 3);
-- {"[1, 4)","[5, 8)","[9, 10)"}
SELECT splitNSpans(datespanset '{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-10)}', 3);
-- {"[2001-01-01, 2001-01-04)","[2001-01-05, 2001-01-10)"}
```

- Devuelve una matriz de rangos obtenida fusionando N elementos consecutivos de un conjunto o N rangos consecutivos de un conjunto de rangos

```
splitEachNSpans({set,spanset},integer) → span[]
```

El último argumento especifica el número de elementos de entrada que se fusionan para producir un rango de salida. Si la cantidad de elementos de entrada es menor que el número especificado, la matriz resultante tendrá un rango de salida por elemento. De lo contrario, la cantidad especificada de elementos de entrada consecutivos se fusionará en un único rango de salida en la respuesta. Observe que, a diferencia de la función splitNSpans, el número de rangos en el resultado depende del número de elementos o de rangos de entrada.

```
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
/* {"[1, 2)","[2, 3)","[3, 4)","[4, 5)","[5, 6)","[6, 7)","[7, 8)","[8, 9)",
   "[9, 10)","[10, 11)"} */
SELECT splitEachNSpans(intspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}', 3);
-- {"[1, 6)","[7, 10)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 7)","[7, 11)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
-- {"[1, 11)"}
```

2.9. Operaciones de distancia

El operador de distancia <-> para los tipos de rango o de tiempo considera el lapso o período delimitador y devuelve la distancia más pequeña entre los dos valores. La distancia tiene el tipo de la diferencia de dos valores, tal como la responde el operador de resta de PostgreSQL: el tipo base para los números, la cantidad de días como `integer` para las fechas, y un `interval` para las marcas de tiempo. El operador de distancia también se puede usar para búsquedas de vecinos más cercanos utilizando un índice GiST o SP-GiST (ver Sección 2.12).

- Devuelve la distancia mínima

```
numbers <-> numbers → base
dates <-> dates → integer
times <-> times → interval
{base,spatialset} <-> {base,spatialset} → float
setDistance({geography,geogset},{geography,geogset},spheroid boolean=true) → float
```

Para geografías, el operador calcula en el esferoide WGS 84, y la función `setDistance` calcula en la esfera, más rápido, si su último argumento es falso.

```
SELECT 3 <-> intspan '[6, 8)';
-- 3
SELECT floatspan '[1, 3]' <-> floatspan '(5.5, 7]';
-- 2.5
SELECT floatspan '[1, 3]' <-> floatspanset '{(5.5, 7],[8, 9]}';
-- 2.5
SELECT tstzspan '[2001-01-02, 2001-01-06)' <-> timestamptz '2001-01-07';
-- 1 day
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' <->
  dateset '{2001-01-02, 2001-01-04}';
--  0
SELECT round((geography 'Point(0 0)' <->
  geogset '{"Point(0 1)", "Point(0 2)"}')::numeric, 3);
-- 110574.389
SELECT round(setDistance(geography 'Point(0 0)', geogset '{"Point(0 1)", "Point(0 2)"}',
  false)::numeric, 3);
-- 111195.080
```

2.10. Comparaciones

Los operadores de comparación (=, <, etc.) requieren que los argumentos izquierdo y derecho sean del mismo tipo. Exceptuando la igualdad y la no igualdad, los otros operadores de comparación no son útiles en el mundo real, pero permiten construir índices de árbol B en tipos de rango o de tiempo. Para los valores de rango, los operadores comparan primero el límite inferior y luego el límite superior. Para los valores de conjunto de marcas de tiempo y conjunto de períodos, los operadores comparan primero los períodos delimitadores y, si son iguales, comparan los primeros N instantes o períodos, donde N es el mínimo del número de instantes o períodos que componen ambos valores.

Los operadores de comparación disponibles para los tipos de conjunto y de rango se dan a continuación. Recuerde que los rangos de enteros siempre se representan por su forma canónica.

- Comparaciones tradicionales

```
set {=, <>, <, >, <=, >=} set → boolean
spans {=, <>, <, >, <=, >=} spans → boolean
cmp({set,span,spanset},{set,span,spanset}) → integer
```

```
SELECT intspan '[1,3]' = intspan '[1,4)';
-- true
SELECT floatspanset '{[1, 2),[2,3)}' = floatspanset '{[1,3)}';
-- true
SELECT tstzset '{2001-01-01, 2001-01-04}' <> tstzset '{2001-01-01, 2001-01-05}';
-- false
SELECT tstzspan '[2001-01-01, 2001-01-04)' <> tstzspan '[2001-01-03, 2001-01-05)';
-- true
SELECT floatspan '[3, 4]' < floatspan '(3, 4]';
-- true
SELECT intspanset '{[1,2],[3,4]}' < intspanset '{[3, 4]}';
-- true
SELECT floatspan '[3, 4]' > floatspan '[3, 4)';
-- true
SELECT tstzspan '[2001-01-03, 2001-01-04)' > tstzspan '[2001-01-02, 2001-01-05)';
-- true
SELECT floatspanset '{[1, 4)}' <= floatspanset '{[1, 5), [6, 7)}';
-- true
SELECT tstzspanset '{[2001-01-01, 2001-01-04)}' <= tstzspanset '{[2001-01-01,
  2001-01-05), [2001-01-06, 2001-01-07)}';
```

```
-- true
SELECT tstzspan '[2001-01-03, 2001-01-05)' >= tstzspan '[2001-01-03, 2001-01-04)';
-- true
SELECT intspanset '{[1, 4)}' >= intspanset '{[1, 5), [6, 7)}';
-- false
SELECT cmp(intspanset '{[1, 4)}', intspanset '{[1, 5), [6, 7)}');
-- -1
```

- Valores hash

```
hash({set,span,spanset}) → integer
hashExtended({set,span,spanset},seed bigint) → bigint
```

La función `hash` devuelve un valor hash de 32 bits y la función `hashExtended` un valor hash de 64 bits calculado con la semilla dada. Los valores iguales tienen valores hash iguales, que es en lo que se basan los índices hash, las uniones hash y la agregación hash.

```
SELECT hash(intspan '[1,3]') = hash(intspan '[1,4)');
-- true
SELECT hash(intspanset '{[1,2],[3,4]}') = hash(intspanset '{[1,3),[3,5)}');
-- true
SELECT hash(tstzset '{2001-01-01, 2001-01-04}') =
  hash(tstzset '{2001-01-04, 2001-01-01}');
-- true
SELECT hashExtended(tstzspan '[2001-01-01, 2001-01-04)', 0) <>
  hashExtended(tstzspan '[2001-01-01, 2001-01-04)', 1);
-- true
```

2.11. Agregaciones

Hay varias funciones agregadas definidas para los tipos de conjunto y de rango. Estas se describen a continuación.

- La función `extent` devuelve el rango o período delimitador que engloba un conjunto de valores de rango o de tiempo.
- La unión es una operación muy útil para los tipos de conjunto y de rango. Como hemos visto en la Sección 2.7, podemos calcular la unión de dos valores de conjunto o de rango usando el operador +. Sin embargo, también es muy útil tener una versión agregada del operador de unión para combinar un número arbitrario de valores. Las funciones `setUnion` y `spanUnion` se pueden utilizar para este propósito.
- La función `tCount` generaliza la función de agregación tradicional `count`. El conteo temporal se puede utilizar para calcular en cada momento el número de objetos disponibles (por ejemplo, el número of períodos). La función `tCount` devuelve un entero temporal (ver el Capítulo 4). La función tiene dos parámetros opcionales que especifican la granularidad (un `interval`) y el origen del tiempo (un `timestamptz`). Cuando se dan estos parámetros, el conteo temporal se calcula en rangos de tiempo de la granularidad dada (ver Sección 9.6)

- Rango o período delimitador

```
extent({set,spans}) → span
```

```
WITH spans(r) AS (
  SELECT floatspan '[1, 4)' UNION SELECT floatspan '(5, 8)' UNION
  SELECT floatspan '(7, 9)' )
SELECT extent(r) FROM spans;
-- [1,9)
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
```

```
SELECT extent(ts) FROM times;
-- [2001-01-01, 2001-01-06]
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}' UNION
  SELECT tstzspanset '{[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06]}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-06]}' )
SELECT extent(ps) FROM periods;
-- [2001-01-01, 2001-01-06]
```

- Unión agregada

```
setUnion({base,set}) → set
spanUnion(spans) → spanset
spansetUnion(spanset) → spanset
```

```
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT setUnion(ts) FROM times;
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04, 2001-01-05, 2001-01-06}
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-03], [2001-01-05, 2001-01-06]}' UNION
  SELECT tstzspanset '{[2001-01-07, 2001-01-08]}' )
SELECT spansetUnion(ps) FROM periods;
-- {[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06], [2001-01-07, 2001-01-08]}
```

- Conteo temporal

```
tCount(times) → {tintSeq,tintSeqSet}
```

```
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT tCount(ts) FROM times;
-- {2@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-04, 1@2001-01-05, 1@2001-01-06}
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}' UNION
  SELECT tstzspanset '{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06)}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-06)}' )
SELECT tCount(ps) FROM periods;
-- {[2@2001-01-01, 3@2001-01-03, 1@2001-01-04, 2@2001-01-05, 2@2001-01-06)}
```

2.12. Indexación

Se pueden crear índices GiST y SP-GiST en columnas de tablas de los tipos de conjunto y de rango. El índice GiST implementa un árbol R, mientras que el índice SP-GiST implementa un árbol cuádruple. Un ejemplo de creación de un índice GiST en una columna `During` de tipo `tstzspan` en una tabla `Reservation` es como sigue:

```
CREATE TABLE Reservation (ReservationID integer PRIMARY KEY, RoomID integer,
  During tstzspan);
CREATE INDEX Reservation_During_Idx ON Reservation USING GIST(During);
```

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores: =, `&&`, `<@`, `@>`, `-|-`, `<<`, `>>`, `&<`, `&>`, `<<#`, `#>>`, `&<#`, `#&>` y `<->`.

Un conjunto espacial, es decir, un conjunto de geometrías, geografías, puntos de red, poses, cadenas de poses, buffers circulares, celdas H3, celdas quadbin o celdas S2, se indexa por el cuadro espaciotemporal que almacena y no por un rango. Su índice GiST implementa un árbol R y su índice SP-GiST un árbol cuádruple, o un árbol k-d cuando se nombra la clase de operadores, como en `USING SPGIST(Stops geomset_kdtree_ops)`. Un índice de este tipo puede acelerar las consultas que involucran a los operadores =, `&&`, `<@` y `@>` y, para los conjuntos cuyos elementos tienen una distancia, `<->`. Dado que el cuadro delimita los elementos de un conjunto sin enunciarlos, el índice devuelve cada conjunto cuyo cuadro satisface el operador, y luego el operador se verifica sobre el conjunto.

Además, se pueden crear índices de árbol B para columnas de tabla de un tipo de rango o de tiempo. Para estos tipos de índices, básicamente la única operación útil es la igualdad. Hay un orden de clasificación de árbol B definido para valores de tipos de rango o de tiempo con los correspondientes operadores <, <=, > y >=, pero el orden es bastante arbitrario y no suele ser útil en el mundo real. El soporte del árbol B está destinado principalmente a permitir la clasificación interna en las consultas, en lugar de la creación de índices reales.

Finalmente, se pueden crear índices hash para columnas de tabla de un tipo de rango o de tiempo. Para estos tipos de índices, la única operación definida es la igualdad.

Capítulo 3

Tipos de cuadro delimitador

A continuación presentamos las funciones y operadores para tipos cuadro delimitador. Estas funciones y operadores son polimórficos, es decir, sus argumentos pueden ser de varios tipos y el tipo del resultado puede depender del tipo de los argumentos. Para expresar esto, usamos la siguiente notación:

- `box` representa cualquier tipo de cuadro delimitador, es decir, `tbox` o `stbox`,
- `number` representa cualquier tipo de base numérico, es decir, `integer`, `bigint` o `float`,
- `numspan` representa cualquier tipo de rango numérico, es decir, `intspan`, `bigintspan` o `floatspan`,
- `numbers` representa cualquier tipo de conjunto o de rango numérico, como `integer`, `intset`, `intspan` o `intspanset`,
- `times` representa cualquier tipo de tiempo con granularidad `timestamptz`, a saber `timestamptz`, `tstzset`, `tstzspan` o `tstzspanset`,
- `tnumber` representa cualquier tipo de número temporal, es decir, `tint`, `tbigint` o `tfloat`,
- `geo` representa los tipos `geometry` o `geography`,
- `tgeo` representa un tipo temporal geometría/geografía, es decir, `tgeometry`, `tgeography`, `tgeompoint` o `tgeogpoint`.

3.1. Entrada y salida

MobilityDB generaliza los formatos de entrada y salida Well-Known Text (WKT) y Well-Known Binary (WKB) del Open Geospatial Consortium para todos los tipos temporales. Presentamos a continuación las funciones de entrada y salida para los tipos de cuadro delimitador.

Un `tbox` se compone de dimensiónes de valor numérico y/o de tiempo. Para cada dimensión, se proporciona un rango, es decir, ya sea un `intspan` o un `floatspan` para la dimensión de valor y un `tstzspan` para la dimensión de tiempo. Ejemplos de entrada de valores `tbox` son los siguientes:

```
-- Both value and time dimensions
SELECT tbox 'TBOXINT XT([1,3),[2001-01-01,2001-01-02])';
SELECT tbox 'TBOXBIGINT XT([1,3),[2001-01-01,2001-01-02])';
SELECT tbox 'TBOXFLOAT XT([1.5,2.5],[2001-01-01,2001-01-02])';
-- Only value dimension
SELECT tbox 'TBOXINT X([1,3))';
SELECT tbox 'TBOXBIGINT X([1,3))';
SELECT tbox 'TBOXFLOAT X((1.5,2.5))';
-- Only time dimension
SELECT tbox 'TBOX T((2001-01-01,2001-01-02))';
```

Un `stbox` se compone de dimensiónes espacial y/o temporal, donde las coordenadas de la dimensión espacial pueden ser 2D o 3D. Para la dimensión de tiempo se da un `tstzspan`, y para la dimensión espacial se dan los valores mínimos y máximos de las coordenadas, donde estas últimas pueden ser cartesianas (planas) o geodésicas (esféricas). Se puede especificar el SRID de las coordenadas; si no es el caso, se asume un valor de 0 (desconocido) y 4326 (correspondiente a WGS84), respectivamente, para coordenadas planas y geodésicas. Los cuadros geodésicos siempre tienen una dimensión Z para tener en cuenta la curvatura de la esfera o esferoide subyacente. Ejemplos de entrada de valores `stbox` son los siguientes:

```
-- Only value dimension with X and Y coordinates
SELECT stbox 'STBOX X((1.0,2.0),(1.0,2.0))';
-- Only value dimension with X, Y, and Z coordinates
SELECT stbox 'STBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
-- Both value (with X and Y coordinates) and time dimensions
SELECT stbox 'STBOX XT(((1.0,2.0),(1.0,2.0)),[2001-01-03,2001-01-03])';
-- Both value (with X, Y, and Z coordinates) and time dimensions
SELECT stbox 'STBOX ZT(((1.0,2.0,3.0),(1.0,2.0,3.0)),[2001-01-01,2001-01-03])';
-- Only time dimension
SELECT stbox 'STBOX T([2001-01-03,2001-01-03])';
-- Only value dimension with X, Y, and Z geodetic coordinates
SELECT stbox 'GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
-- Both value (with X, Y and Z geodetic coordinates) and time dimension
SELECT stbox 'GEODSTBOX ZT(((1.0,2.0,3.0),(1.0,2.0,3.0)),[2001-01-04,2001-01-04])';
-- Only time dimension for geodetic box
SELECT stbox 'GEODSTBOX T([2001-01-03,2001-01-03])';
-- SRID is given
SELECT stbox 'SRID=5676;STBOX XT(((1.0,2.0),(1.0,2.0)),[2001-01-04,2001-01-04])';
SELECT stbox 'SRID=4326;GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
```

Damos a continuación las funciones de entrada y salida de tipos de cuadro delimitador en formato Well-Known Text (WKT) y Well-Known Binary (WKB).

- Devuelve la representación de texto conocido (WKT)

```
asText(box,maxdecimaldigits integer=15) → text
```

El argumento `maxdecimaldigits` se puede utilizar para definir el número máximo de decimales para la salida de los valores de coma flotante (por defecto 15).

```
SELECT asText(tbox 'TBOXFLOAT XT([1.123456789,2.123456789),[2001-01-01,2001-01-02))', 3);
-- TBOXFLOAT XT([1.123, 2.123),[2001-01-01, 2001-01-02))
SELECT asText(stbox 'STBOX Z((1.55,1.55,1.55),(2.55,2.55,2.55))', 0);
-- STBOX Z((2,2,2),(3,3,3))
```

- Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)

```
asBinary(box,endian text='') → bytea
asHexWKB(box,endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina. Como para todo tipo espacial, la representación de una `stbox` no escribe su SRID, que escriben `asEWKB` y `asHexEWKB`.

```
SELECT asBinary(tbox 'TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))');
-- \x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040
SELECT asBinary(tbox 'TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))', 'XDR');
-- \x000300270100001cc1d3579c0000001cd5f12efc00000d013ff0000000000004000000000000000
SELECT asBinary(stbox 'STBOX X((1,1),(2,2))');
-- \x0101000000000000f03f0000000000000040000000000000f03f0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))');
-- 0103270001009C57D3C11C000000FC2EF1D51C00000D0001000000000000F03F0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2)[2001-01-01,2001-01-02))', 'XDR');
-- 000300270100001CC1D3579C0000001CD5F12EFC00000D013FF0000000000004000000000000000
```

```
SELECT asHexWKB(stbox 'STBOX X((1,1),(2,2))');
-- 0101000000000000F03F0000000000000040000000000000F03F0000000000000040
```

- Devuelve la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB) de una caja espaciotemporal

```
asEWKB(stbox,endian text='') → bytea
asHexEWKB(stbox,endian text='') → text
```

El resultado es la representación que escriben `asBinary` y `asHexWKB` con el SRID de la caja, como todo tipo espacial escribe su SRID en su representación extendida.

```
SELECT asEWKB(stbox 'SRID=3812;STBOX X((1,1),(2,2))');
-- \x0141e40e0000000000000000f03f0000000000000040000000000000f03f0000000000000040
SELECT asHexEWKB(stbox 'SRID=3812;STBOX X((1,1),(2,2))', 'XDR');
-- 004100000EE43FF0000000000000040000000000000003FF0000000000000400000000000000
```

- Entrada desde la representación de texto conocido (WKT)

```
boxFromText(text) → box
```

En la función anterior, `box` reemplaza cualquier tipo de cuadro delimitador, a saber, `tbox` o `stbox`. La función es la inversa de `asText`: lee el texto en el que se escribe un literal del tipo, tanto de una constante como de una columna `text`, incluido el SRID de un cuadro delimitador espaciotemporal.

```
SELECT tboxFromText('TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])');
-- TBOXFLOAT XT([1, 2],[2001-01-01, 2001-01-02])
SELECT stboxFromText('SRID=3812;STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])');
-- SRID=3812;STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])
```

- Entrada desde la representación binaria conocida (WKB) o desde la representación hexadecimal binaria conocida (HexWKB)

```
boxFromBinary(bytea) → box
boxFromHexWKB(text) → box
```

En las funciones anteriores, `box` reemplaza cualquier tipo de cuadro delimitador, a saber, `tbox` o `stbox`.

```
SELECT tboxFromBinary(
  '\x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040');
-- TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))
SELECT stboxFromBinary(
  '\x0101000000000000f03f0000000000000040000000000000f03f0000000000000040');
--  STBOX X((1,1),(2,2))
SELECT tboxFromHexWKB(
  '0103270001009C57D3C11C000000FC2EF1D51C00000D0001000000000000F03F0000000000000040');
-- TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02)))
SELECT stboxFromHexWKB(
  '0101000000000000F03F0000000000000040000000000000F03F0000000000000040');
-- STBOX X((1,1),(2,2))
```

- Entrada de una caja espaciotemporal desde la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB)

```
stboxFromEWKB(bytea) → stbox
stboxFromHexEWKB(text) → stbox
```

```
SELECT stboxFromEWKB(asEWKB(stbox 'SRID=3812;STBOX X((1,1),(2,2))'));
-- SRID=3812;STBOX X((1,1),(2,2))
SELECT stboxFromHexEWKB(
  '0141E40E0000000000000000F03F0000000000000040000000000000F03F0000000000000040');
-- SRID=3812;STBOX X((1,1),(2,2))
```

3.2. Constructores

El tipo `tbox` tienen varias funciones constructoras dependiendo de si se da la extensión de valor y/o de tiempo. La extensión de valor se puede especificar mediante un número o un intervalo, mientras que la extensión de tiempo se puede especificar mediante un tipo de tiempo.

- Constructor para `tbox`

```
tbox({number,numspan}) → tbox
tbox({timestamptz,tstzspan}) → tbox
tbox({number,numspan},{timestamptz,tstzspan}) → tbox
```

```
-- Both value and time dimensions
SELECT tbox(1.0, timestamptz '2001-01-01');
SELECT tbox(floatspan '[1.0, 2.0)', tstzspan '[2001-01-01,2001-01-02)');
-- Only value dimension
SELECT tbox(floatspan '[1.0,2.0)');
-- Only time dimension
SELECT tbox(tstzspan '[2001-01-01,2001-01-02)');
```

El tipo `stbox` tiene varias funciones constructoras dependiendo de si se da la extensión espacial y/o de tiempo. Las coordenadas de la extension espacial pueden ser 2D o 3D y pueden ser cartesianas o geodésicas. La extensión espacial se puede especificar mediante los valores de coordenadas mínimo y máximo. El SRID se puede especificar en un último argumento opcional. Si no se proporciona, se asume un valor 0 (respectivamente 4326) por defecto para los cuadros planos (respectivamente geodésicos). La extensión espacial también se puede especificar mediante una geometría o una geografía. La extensión temporal se puede especificar mediante un tipo de tiempo.

- Constructor para `stbox`

```
stboxX(float,float,float,float,srid integer=0) → stbox
stboxZ(float,float,float,float,float,float,srid integer=0) → stbox
stboxT({timestamptz,tstzspan}) → stbox
stboxXT(float,float,float,float,{timestamptz,tstzspan},srid integer=0) → stbox
stboxZT(float,float,float,float,float,float,{timestamptz,tstzspan},
  srid integer=0) → stbox
geodstboxZ(float,float,float,float,float,float,srid integer=4326) → stbox
geodstboxZT(float,float,float,float,float,float,{timestamptz,tstzspan},
  srid integer=4326) → stbox
stbox(geo) → stbox
stbox(geo,{timestamptz,tstzspan}) → stbox
```

```
-- Only value dimension with X and Y coordinates
SELECT stboxX(1.0,2.0,1.0,2.0);
-- Only value dimension with X, Y, and Z coordinates
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Only value dimension with X, Y, and Z coordinates and SRID
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0,5676);
-- Only time dimension
SELECT stboxT(tstzspan '[2001-01-03,2001-01-03]');
-- Both value (with X and Y coordinates) and time dimensions
SELECT stboxXT(1.0,2.0, 1.0,2.0, tstzspan '[2001-01-03,2001-01-03]');
-- Both value (with X, Y, and Z coordinates) and time dimensions
SELECT stboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-03]');
-- Only value dimension with X, Y, and Z geodetic coordinates
SELECT geodstboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Both value (with X, Y, and Z geodetic coordinates) and time dimensions
SELECT geodstboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-04]');
-- Geometry and time dimension
```

```
SELECT stbox(geometry 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03,2001-01-05]');
-- Geography and time dimension
SELECT stbox(geography 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03,2001-01-05]');
```

3.3. Conversión de tipos

- Convierte un `tbox` a otro tipo

```
tbox::{intspan,floatspan,tstzspan}
intspan(tbox) → tbox
floatspan(tbox) → tbox
tstzspan(tbox) → tbox
```

```
SELECT tbox 'TBOXINT XT([1,4),[2001-01-01,2001-01-02))'::intspan;
-- [1,4)
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02))'::floatspan;
-- (1, 2)
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02))'::tstzspan;
-- [2001-01-01, 2001-01-02)
```

- Convierte otro tipo a un `tbox`

```
{numbers,times,tnumber}::tbox
tbox({numbers,times,tnumber}) → tbox
```

```
SELECT intset '{1,2}'::tbox;
-- TBOXINT X([1, 3))
SELECT intspan '[1,3)'::tbox;
-- TBOXINT X([1, 3))
SELECT floatspan '(1.0,2.0)'::tbox;
-- TBOXFLOAT X((1, 2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)}'::tbox;
-- TBOX T((2001-01-01,2001-01-04))
```

- Convierte un `stbox` a otro tipo

```
stbox::{box2d,box3d,geo,tstzspan}
box2d(stbox) → box2d
box3d(stbox) → box3d
geometry(stbox) → geometry
geography(stbox) → geography
tstzspan(stbox) → tstzspan
```

Las conversiones siguen la regla de [?listitem].

```
SELECT stbox 'STBOX XT(((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-03])'::box2d;
-- BOX(1 2,3 4)
SELECT ST_AsEWKT(stbox 'SRID=4326;
  STBOX XT(((1,1),(5,5)),[2001-01-01,2001-01-05])':: geometry);
-- SRID=4326;POLYGON((1 1,1 5,5 5,5 1,1 1))
SELECT ST_AsEWKT(stbox 'STBOX XT(((1,1),(1,5)),[2001-01-01,2001-01-05])'::geometry);
-- LINESTRING(1 1,1 5)
SELECT ST_AsEWKT(stbox 'GEODSTBOX XT(((1,1),(1,1)),[2001-01-01,2001-01-05])'::geography);
-- SRID=4326;POINT(1 1)
SELECT ST_AsEWKT(stbox 'STBOX ZT(((1,1,1),(5,5,5)),[2001-01-01,2001-01-05])':: geometry);
/* POLYHEDRALSURFACE(((1 1 1,1 5 1,5 5 1,5 1 1,1 1 1)),
   ((1 1 5,5 1 5,5 5 5,1 5 5,1 1 5)),((1 1 1,1 1 5,1 5 5,1 5 1,1 1 1)),
   ((5 1 1,5 5 1,5 5 5,5 1 5,5 1 1)),((1 1 1,5 1 1,5 1 5,1 1 5,1 1 1)),
```

```
   ((1 5 1,1 5 5,5 5 5,5 5 1,1 5 1))) */
SELECT stbox 'STBOX XT(((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-03])'::tstzspan;
-- [2001-01-01, 2001-01-03]
```

- Convierte otro tipo a un `stbox`

```
{box2d,box3d,geo,times,tgeo}::stbox
stbox({box2d,box3d,geo,times,tgeo}) → stbox
```

```
SELECT geometry 'Linestring(1 1 1,2 2 2)'::box3d::stbox;
-- STBOX Z((1,1,1),(2,2,2))
SELECT geography 'Linestring(1 1,2 2)'::stbox;
-- SRID=4326;GEODSTBOX X((1,1),(2,2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)}'::stbox;
-- STBOX T((2001-01-01,2001-01-04))
```

La caja de un punto temporal geodésico contiene los círculos máximos que recorre entre sus posiciones, que se elevan por encima de ellas o descienden por debajo: un viaje de `Point(10 60)` a `Point(50 60)` alcanza la latitud 61.518762, y uno cuyas posiciones distan más de media vuelta al globo en longitud cruza el antimeridiano y abarca todas las longitudes.

```
SELECT stbox(tgeogpoint '[Point(10 60)@2001-01-01, Point(50 60)@2001-01-02]');
-- SRID=4326;GEODSTBOX XT(((10,60),(50,61.518762)),[2001-01-01, 2001-01-02])
SELECT stbox(tgeogpoint '[Point(-170 60)@2001-01-01, Point(170 60)@2001-01-02]');
-- SRID=4326;GEODSTBOX XT(((-180,60),(180,60.378349)),[2001-01-01, 2001-01-02])
```

3.4. Accesores

- ¿Tiene dimensión X/Z/T?

```
hasX(box) → boolean
hasZ(stbox) → boolean
hasT(box) → boolean
```

```
SELECT hasX(tbox 'TBOX T([2001-01-01,2001-01-03))');
-- false
SELECT hasX(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- true
SELECT hasZ(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false
SELECT hasT(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03])');
-- true
SELECT hasT(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false
```

- ¿Es geodética?

```
isGeodetic(stbox) → boolean
```

```
SELECT isGeodetic(stbox 'GEODSTBOX Z((1.0,1.0,0.0),(3.0,3.0,1.0))');
-- true
SELECT isGeodetic(stbox 'STBOX XT(((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-02])');
-- false
```

- Devuelve el valor mínimo de X/Y/Z/T

```
xMin(box) → float
yMin(stbox) → float
zMin(stbox) → float
tMin(box) → timestamptz
```

```
SELECT xMin(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- 1
SELECT yMin(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 2
SELECT zMin(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 3
SELECT tMin(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-01
```

Observe que para `tbox` el valor resultante de `xMin` y `xMax` se convierte a `float` para las cajas temporales con un span de enteros. Un límite de entero grande que ningún `float` representa exactamente se rechaza en lugar de redondearse; lea ese límite de `bigintspan(box)`.

- Devuelve el valor máximo de X/Y/Z/T

```
xMax(box) → float
yMax(stbox) → float
zMax(stbox) → float
tMax(box) → timestamptz
```

```
SELECT xMax(tbox 'TBOXINT X([1,4))');
-- 3
SELECT yMax(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 4
SELECT zMax(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 6
SELECT tMax(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-03
```

Observe que para `tbox` el valor resultante de `xMin` y `xMax` se convierte a `float` para las cajas temporales con un span de enteros. Un límite de entero grande que ningún `float` representa exactamente se rechaza en lugar de redondearse; lea ese límite de `bigintspan(box)`.

- Es el mínimo valor X/T inclusivo?

```
xMinInc(tbox) → boolean
tMinInc(box) → boolean
```

```
SELECT xMinInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMinInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true
```

- Es el máximo valor X/T inclusivo?

```
xMaxInc(tbox) → boolean
tMaxInc(box) → boolean
```

```
SELECT xMaxInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMaxInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true
```

- Área, volumen, perímetro

```
area(stbox,spheroid boolean=true) → float
volume(stbox) → float
perimeter(stbox,spheroid boolean=true) → float
```

Para cuadros geodésicos, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido. La función `volume` no acepta cuadros geodésicos.

```
SELECT area(stbox 'STBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))');
-- 4
SELECT volume(stbox 'STBOX ZT(((1,1,1),(3,3,3)),[2001-01-01,2001-01-03))');
-- 8
SELECT perimeter(stbox 'STBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))');
-- 8
SELECT area(stbox 'GEODSTBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))');
-- 49209676328.36632
SELECT perimeter(stbox 'GEODSTBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))', false);
-- 889221.9544681838
```

3.5. Transformaciones

- Desplaza y/o escalear el rango de valores de un cuadro delimitador con uno o dos valores

```
shiftValue(tbox,number) → tbox
scaleValue(tbox,number) → tbox
shiftScaleValue(tbox,number,number) → tbox
```

```
SELECT shiftValue(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])', 1.0);
-- TBOXFLOAT XT([2.5, 3.5],[2001-01-01, 2001-01-02])
SELECT scaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])', 2.0);
-- TBOXFLOAT XT([1.5, 3.5],[2001-01-01, 2001-01-02])
SELECT shiftScaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])', 2.0, 3.0);
-- TBOXFLOAT XT([3.5, 6.5],[2001-01-01, 2001-01-02])
```

- Desplaza y/o escalar el período de un cuadro delimitador con uno o dos intervalos

```
shiftTime(box,interval) → box
scaleTime(box,interval) → box
shiftScaleTime(box,interval,interval) → box
```

```
SELECT shiftTime(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])',
  interval '1 day');
-- TBOXFLOAT XT([1.5, 2.5],[2001-01-02, 2001-01-03])
SELECT shiftTime(stbox 'STBOX T([2001-01-01,2001-01-02])', interval '-1 day');
-- STBOX T([2000-12-31, 2001-01-01])
SELECT shiftTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '1 day');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-02, 2001-01-03])
SELECT scaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])',
  interval '2 days');
-- TBOXFLOAT XT([1.5, 2.5],[2001-01-01, 2001-01-03])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '1 hour');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01 00:00:00, 2001-01-01 01:00:00])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '-1 day');
-- ERROR:  The interval must be positive: -1 days
SELECT shiftScaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])',
  interval '1 day', interval '3 days');
-- TBOXFLOAT XT([1.5, 2.5],[2001-01-02, 2001-01-05])
SELECT shiftScaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '1 hour', interval '3 hours');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01 01:00:00, 2001-01-01 04:00:00])
```

Si el ancho o el lapso de tiempo del valor de tiempo es cero (es decir, los límites inferior y superior son iguales), el resultado es el cuadro delimitador. El valor o intervalo dado debe ser estrictamente mayor que cero.

- Devuelve la dimensión espacial del cuadro delimitador, eliminando la dimensión temporal, si existe

```
getSpace(stbox) → stbox
```

```
SELECT getSpace(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-03])');
-- STBOX Z((1,1,1),(2,2,2))
```

Las funciones dadas a continuación expanden los cuadros delimitadores en la dimensión de valor y de tiempo o establecen la precisión de la dimensión de valor. Estas funciones generan un error si la dimensión correspondiente no está presente.

- Extiende la dimensión numérica, espacial o temporal del cuadro delimitador con un valor o un intervalo

```
expandValue(tbox,number) → tbox
expandSpace(stbox,float) → stbox
expandTime(box,interval) → box
```

```
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-03])', 1.0);
-- TBOXFLOAT XT((0,3),[2001-01-01,2001-01-03])
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-03])', -1.0);
-- NULL
SELECT expandValue(tbox 'TBOX T([2001-01-01,2001-01-03))', 1);
-- The box must have value dimension
```

La función devuelve NULL si el valor o intervalo dado como segundo argumento es negativo y el rango resultante de desplazar los límites con el argumento está vacío.

```
SELECT expandSpace(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-03])', 1);
-- STBOX ZT(((0,0,0),(3,3,3)),[2001-01-01,2001-01-03])
SELECT expandSpace(stbox 'STBOX T([2001-01-01,2001-01-03))', 1);
-- The box must have space dimension
```

```
SELECT expandTime(tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-03])', interval '1 day');
-- TBOXFLOAT XT((1,2),[2000-12-31,2001-01-04])
SELECT expandTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-03])',
  interval '-1 day');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-02,2001-01-02])
SELECT expandTime(tbox 'TBOX XT((1,2),[2001-01-01,2001-01-03])', interval '-2 days');
-- NULL
```

- Redondea el valor o las coordenadas del cuadro delimitador a un número de decimales

```
round(box,integer=0) → box
```

```
SELECT round(tbox 'TBOXFLOAT XT((1.12345,2.12345),[2001-01-01,2001-01-02])', 2);
-- TBOXFLOAT XT((1.12,2.12),[2001-01-01, 2001-01-02])
SELECT round(stbox 'STBOX XT(((1.12345, 1.12345),(2.12345, 2.12345)),
  [2001-01-01,2001-01-02])', 2);
-- STBOX XT(((1.12,1.12),(2.12,2.12)),[2001-01-01, 2001-01-02])
SELECT round(tstzspan '[2001-01-01, 2001-01-02]'::tbox);
-- The tbox must have X dimension
```

3.6. Sistema de referencia espacial

- Devuelve o especifica el identificador de referencia espacial

```
SRID(stbox) → integer
setSRID(stbox,integer) → stbox
```

```
SELECT SRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)),[2001-01-01,2001-01-02])');
-- 0
SELECT SRID(stbox 'SRID=5676;STBOX XT(((1.0,2.0),(4.0,5.0)),[2001-01-01,2001-01-02])');
-- 5676
SELECT SRID(stbox 'GEODSTBOX T([2001-01-01,2001-01-02))');
-- ERROR:  The box must have space dimension
```

```
SELECT setSRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)), [2001-01-01,2001-01-02])',
  5676);
-- SRID=5676;STBOX ZT(((1,2,3),(4,5,6)),[2001-01-01,2001-01-02])
```

- Transforma a una referencia espacial diferente

```
transform(stbox,srid integer) → stbox
transformPipeline(stbox,pipeline text,srid integer,is_forward boolean=true) → stbox
```

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el cuadro de entrada tiene un SRID desconocido (representado por 0).

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID del cuadro de entrada se ignora y el SRID del cuadro de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT round(transform(stbox 'SRID=4326;STBOX XT(((2.340088, 49.400250),
  (6.575317, 51.553167)),[2001-01-01,2001-01-02])', 3812), 6);
/* SRID=3812;STBOX XT(((502773.429981,511805.120402),(803028.908265,751590.742629)),
   [2001-01-01, 2001-01-02]) */
WITH test(box, pipeline) AS (
  SELECT stbox 'SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(box, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
-- SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))
```

3.7. Operaciones de división

- Divide el cuadro delimitador en cuadrantes u octantes {}

```
quadSplit(stbox) → {stbox}
```

Como lo indica el símbolo {}, esta función es una *función de retorno de conjuntos* (también conocida como *función de tabla*) ya que normalmente devuelve más de un valor.

```
SELECT quadSplit(stbox 'STBOX XT(((0,0),(4,4)),[2001-01-01,2001-01-05])');
/* {"STBOX XT(((0,0),(2,2)),[2001-01-01, 2001-01-05])",
   "STBOX XT(((2,0),(4,2)),[2001-01-01, 2001-01-05])",
   "STBOX XT(((0,2),(2,4)),[2001-01-01, 2001-01-05])",
   "STBOX XT(((2,2),(4,4)),[2001-01-01, 2001-01-05])"} */
SELECT quadSplit(stbox 'STBOX Z((0,0,0),(4,4,4))');
/* {"STBOX Z((0,0,0),(2,2,2))","STBOX Z((2,0,0),(4,2,2))","STBOX Z((0,2,0),(2,4,2))",
   "STBOX Z((2,2,0),(4,4,2))","STBOX Z((0,0,2),(2,2,4))","STBOX Z((2,0,2),(4,2,4))",
   "STBOX Z((0,2,2),(2,4,4))","STBOX Z((2,2,2),(4,4,4))"} */
```

Esta función se usa normalmente para cuadrículas de resolución múltiple, donde el espacio se divide en celdas de modo que las celdas tengan un número máximo de elementos. Figura 3.1 muestra un ejemplo del resultado de usar esta función usando trayectorias sintéticas en Bruselas.

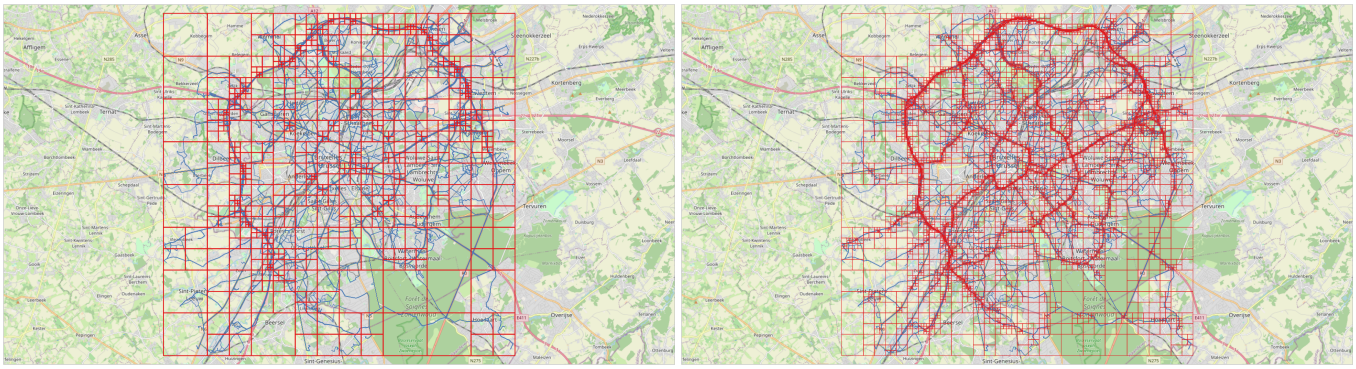


Figura 3.1: Grilla multiresolución sobre datos de Bruselas obtenidos mediante el generador BerlinMOD. Cada celda contiene como máximo 10.000 (izquierda) y 1.000 (derecha) instantes durante todo el período de simulación (cuatro días en este caso). A la izquierda, podemos ver la alta densidad de tráfico en el anillo alrededor de Bruselas, mientras que a la derecha podemos ver otros ejes principales de la ciudad.

3.8. Operaciones de conjuntos

Los operadores de conjuntos para los tipos de cuadro delimitador son la unión (+) y la intersección (*). En el caso de la unión, los operandos deben tener exactamente las mismas dimensiónes, de lo contrario se genera un error. Además, si los operandos no se superponen en todas las dimensiónes se genera un error, ya que esto daría como resultado una caja con valores disjuntos, que no se puede representar. El operador calcula la unión en todas las dimensiónes que están presentes en ambos argumentos. En el caso de intersección, los operandos deben tener al menos una dimensión común, de lo contrario se genera un error. El operador calcula la intersección en todas las dimensiónes que están presentes en ambos argumentos.

- Unión e intersección de dos cuadros delimitadores

```
box {+, *} box → box
```

```
SELECT tbox 'TBOXINT XT([1,3),[2001-01-01,2001-01-03])' + tbox 'TBOXINT XT([2,4),
  [2001-01-02,2001-01-04])';
-- TBOXINT XT([1,4),[2001-01-01,2001-01-04])
SELECT stbox 'STBOX ZT(((1,1,1),(2,2,2)),
  [2001-01-01,2001-01-02])' + stbox 'STBOX XT(((2,2),(3,3))),[2001-01-01,2001-01-03]';
-- ERROR:  The arguments must be of the same dimensionality
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-02])' + tbox 'TBOXFLOAT XT((3,4),
  [2001-01-03,2001-01-04])';
-- ERROR:  Result of box union would not be contiguous
```

```
SELECT tbox 'TBOXINT XT([1,3),
  [2001-01-01,2001-01-03])' * tbox 'TBOX T([2001-01-02,2001-01-04))';
-- TBOX T([2001-01-02,2001-01-03))
SELECT stbox 'STBOX ZT(((1,1,1),(3,3,3)),
  [2001-01-01,2001-01-02])' * stbox 'STBOX X((2,2),(4,4))';
-- STBOX X((2,2),(3,3))
```

3.9. Operaciones de cuadro delimitador

3.9.1. Operaciones topológicas

Hay cinco operadores topológicos: superposición (`&&`), contiene (`@>`), es contenido (`<@`), mismo (`~=`) y adyacente (`-|-`). Los operadores verifican la relación topológica entre los cuadros delimitadores teniendo en cuenta la dimensión de valor y/o de tiempo para todas las dimensiónes que estén presentes en ambos argumentos.

Un `numspan` o un `tstzspan` es operando de estos operadores frente a un `tbox`, en cualquier orden: el operador responde sobre el cuadro del span, que tiene solo la dimensión de valor o solo la de tiempo.

```
SELECT intspan '[1,5)' && tbox 'TBOXINT XT([3,8),[2001-01-01,2001-01-02])';
-- true
SELECT tstzspan '[2001-01-01,2001-01-03]' @> tbox 'TBOXINT XT([3,8) ↩
    ,[2001-01-01,2001-01-02])';
-- true
```

- ¿Se superponen los cuadros delimitadores?

```
box && box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' &&
  tbox 'TBOXFLOAT XT((2,4),[2001-01-02,2001-01-04])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' &&
  stbox 'STBOX T([2001-01-02,2001-01-02])';
-- true
```

- ¿Contiene el primer cuadro delimitador el segundo?

```
box @> box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' @>
  tbox 'TBOXFLOAT XT((2,3),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' @>
  stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])';
-- true
```

- ¿Está el primer cuadro delimitador contenido en el segundo?

```
box <@ box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <@ tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' <@
  stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])';
-- true
```

- ¿Son los cuadros delimitadores iguales en sus dimensiónes comunes?

```
box ~= box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])' ~= tbox 'TBOXFLOAT T([2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(3,3)),
  [2001-01-01,2001-01-03])' ~= stbox 'STBOX Z((1,1,1),(3,3,3))';
-- true
```

- ¿Son los cuadros delimitadores adyacentes?

```
box -|- box → boolean
```

Dos cuadros delimitadores son adyacentes si sus extensiones se encuentran en todas las dimensiones que los cuadros comparten y se tocan en un solo valor en al menos una de ellas. Una dimensión en la que los cuadros están separados los separa sea cual sea el comportamiento de las demás dimensiones.

```
SELECT tbox 'TBOXINT XT([1,2),[2001-01-01,2001-01-02])' -|- tbox 'TBOXINT XT([2,3),
  [2001-01-02,2001-01-03])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])' -|- tbox 'TBOX T([2001-01-02,2001-01-03])';
-- true
SELECT stbox 'STBOX XT(((1,1),(3,3)),
  [2001-01-01,2001-01-03])' -|- stbox 'STBOX XT(((2,2),(4,4)),[2001-01-03,2001-01-04])';
-- true
```

3.9.2. Operaciones de posición

Los operadores de posición consideran la posición relativa de los cuadros delimitadores. Los operadores `<<`, `>>`, `&<` y `&>` consideran el valor X para el tipo `tbox` y las coordenadas X para el tipo `stbox`, los operadores `<<|`, `|>>`, `&<|` y `|&>` consideran las coordenadas Y para el tipo `stbox`, los operadores `<</`, `/>>`, `&</` y `/&>` consideran las coordenadas Z para el tipo `stbox` y los operadores `<<#`, `#>>`, `#&<` y `#&>` consideran la dimensión de tiempo para los tipos `tbox` y `stbox`. Los operadores generan un error si ambos cuadros delimitadores no tienen la dimensión requerida.

Un `numspan` es operando de los operadores de la dimensión de valor frente a un `tbox`, y un `tstzspan` de los de la dimensión de tiempo, en cualquier orden, respondidos sobre el cuadro del span.

```
SELECT intspan '[1,5)' &< tbox 'TBOXINT XT([3,8),[2001-01-01,2001-01-02])';
-- true
SELECT tstzspan '[2001-01-05,2001-01-06]' #>> tbox 'TBOXINT XT([3,8) ↩
    ,[2001-01-01,2001-01-02])';
-- true
```

Los operadores para la dimensión numérica del tipo `tbox` se dan a continuación.

- ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente menores que los del segundo?

```
tbox {<<, <<#} tbox → boolean
stbox {<<, <<|, <</, <<#} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <<
  tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <<
  tbox 'TBOXFLOAT T([2001-01-03,2001-01-04])';
-- ERROR:  The box must have value dimension
```

```
SELECT stbox 'STBOX Z((1,1,1),(2,2,2))' <<| stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(2,2,2))' <</ stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])' <<# tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
```

- ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente mayores que los del segundo?

```
tbox {>>, #>>} tbox → boolean
stbox {>>, |>>, />>, #>>} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])' >>
  tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' |>> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' />> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX XT(((3,3),(4,4)),[2001-01-03,2001-01-04])' #>>
  stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])';
-- true
```

- ¿No son los valores X/Y/Z/T del primer cuadro delimitador mayores que los del segundo?

```
tbox {&<, &<#} box → boolean
stbox {&<, &<|, &</, &<#} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' &<
  tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &<| stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &</ stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,4),
  [2001-01-01,2001-01-04])' &<# tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
```

- ¿No son los valores X/Y/Z/T del primer cuadro delimitador menores que los del segundo?

```
tbox {&>, #&>} tbox → boolean
stbox {&>, |&>, /&>, #&>} stbox} → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' &>
  tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' |&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- false
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' /&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' #&>
  stbox 'STBOX XT(((1,1),(4,4)),[2001-01-01,2001-01-04])';
-- true
```

3.10. Comparaciones

Los operadores de comparación tradicionales (=, <, etc.) se pueden aplicar a tipos de cuadro delimitador. Exceptuando la igualdad y la no igualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten construir índices de árbol B en tipos de cuadro delimitador. Estos operadores definen un orden total. Comparan primero el SRID de los cuadros espaciotemporales, luego las dimensiones que tienen los cuadros, si los cuadros espaciotemporales son geodésicos y el tipo de valor de los cuadros temporales. Los cuadros que coinciden en todo esto se comparan en su dimensión temporal y, si son iguales, en su dimensión de valor o espacial.

- Comparaciones tradicionales

```
box {=, <>, <, >, <=, >=} box → boolean
cmp(box,box) → integer
```

La función `cmp` devuelve -1, 0 o 1 según que el primer argumento sea, respectivamente, menor que, igual a o mayor que el segundo.

```
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' = tbox 'TBOXINT XT([2,2],
  [2001-01-03,2001-01-05])';
-- false
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' <>
  tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <
  tbox 'TBOXINT XT([1,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-03,2001-01-04])' >
  tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <= tbox 'TBOXINT XT([2,2],
  [2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' >= tbox 'TBOXFLOAT XT([2,2],
  [2001-01-03,2001-01-05])';
-- false
SELECT cmp(tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])',
  tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])');
-- -1
```

- Valores hash

```
hash(box) → integer
hashExtended(box,seed bigint) → bigint
```

La función `hash` devuelve un valor hash de 32 bits y la función `hashExtended` un valor hash de 64 bits calculado con la semilla dada. Los valores iguales tienen valores hash iguales, que es en lo que se basan los índices hash, las uniones hash y la agregación hash.

```
SELECT hash(tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])') =
  hash(tbox 'TBOXINT XT([1,2),[2001-01-01,2001-01-04])');
-- true
SELECT hashExtended(stbox 'STBOX X((1,1),(2,2))', 0) <>
  hashExtended(stbox 'STBOX X((1,1),(2,2))', 1);
-- true
```

3.11. Aggregaciones

- Extensión del cuadro delimitador

```
extent(box) → box
```

```
WITH boxes(b) AS (
  SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' UNION
  SELECT tbox 'TBOXFLOAT XT((5,7),[2001-01-05,2001-01-07])' UNION
  SELECT tbox 'TBOXFLOAT XT((6,8),[2001-01-06,2001-01-08])' )
SELECT extent(b) FROM boxes;
-- TBOXFLOAT XT((1,8),[2001-01-01,2001-01-08])
WITH boxes(b) AS (
  SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' UNION
  SELECT stbox 'STBOX Z((5,5,5),(7,7,7))' UNION
  SELECT stbox 'STBOX Z((6,6,6),(8,8,8))' )
SELECT extent(b) FROM boxes;
-- STBOX Z((1,1,1),(8,8,8))
```

3.12. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tablas de los tipos `tbox` y `stbox`. El índice GiST implementa un árbol R, mientras que el índice SP-GiST implementa un árbol cuádruple n-dimensiónal. Un ejemplo de creación de un índice GiST en una columna `Box` de tipo `stbox` en una tabla `Trips` es el siguiente:

```
CREATE TABLE Trips(TripID integer PRIMARY KEY, Trip tgeompoint, Box stbox);
CREATE INDEX Trips_Box_Idx ON Trips USING GIST(bbox);
```

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores: `&&`, `<@`, `@>`, `~=`, `-|-`, `<<`, `>>`, `&<`, `&>`, `<<|`, `|>>`, `&<|`, `|&>`, `<</`, `/>>`, `&</`, `/&>`, `<<#`, `#>>`, `&<#` y `#&>`.

Además, se pueden crear índices de árbol B para columnas de tablas de un tipo cuadro delimitador. Para estos tipos de índices, básicamente la única operación útil es la igualdad. Hay un orden de clasificación de árbol B definido para valores de tipos cuadro delimitador con los correspondientes operadores < y >, pero el orden es bastante arbitrario y no suele ser útil en el mundo real. El soporte de árbol B está destinado principalmente a permitir la clasificación interna en las consultas, en lugar de la creación de índices reales.

Capítulo 4

Tipos temporales (Parte 1)

4.1. Introduction

MobilityDB proporciona varios tipos temporales basados en tipos PostgreSQL o PostGIS, a saber, `tbool`, `tint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint` y `tgeogpoint`, que se basan, respectivamente, en los tipos de base `boolean`, `integer`, `float`, `text`, `geometry` y `geography`, donde `tgeometry` y `tgeography` aceptan geometrías/geografías arbitrarias, mientras que `tgeompoint` y `tgeogpoint` solo aceptan puntos 2D o 3D con dimensión Z. Además, MobilityDB proporciona otros tipos espaciales específicos, a saber, `cbuffer` (buffer circular), `npoint` (punto de red) y `pose`, y sus tipos espaciotemporales correspondientes, a saber, `tcbuffer`, `tnpoint`, `tpose` y `trgeometry` (geometría rígida temporal). En este capítulo y en el siguiente, presentamos las funciones y operadores disponibles para todos los tipos temporales, mientras que las funciones específicas para los tipos espaciotemporales se detallarán en capítulos posteriores. A nivel conceptual, los tipos temporales se organizan en la jerarquía que se muestra en la Figura 4.1.

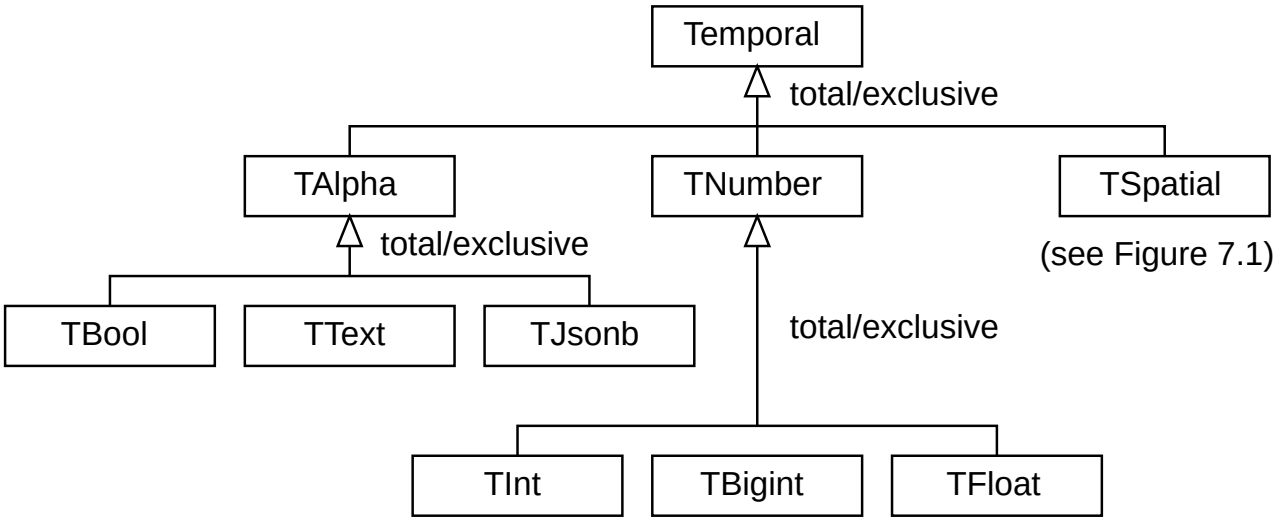


Figura 4.1: Jerarquía de tipos temporales en MobilityDB.

La *interpolación* de un valor temporal establece cómo evoluciona el valor entre instantes sucesivos. La interpolación es *discreta* cuando se desconoce el valor entre dos instantes sucesivos. Pueden representar, por ejemplo, entradas/salidas cuando se utiliza un lector de tarjetas RFID para entrar o salir de un edificio. La interpolación es *escalonada* cuando el valor permanece constante entre dos instantes sucesivos. Por ejemplo, la marcha que utiliza un automóvil en movimiento puede representarse con un número entero temporal, lo que indica que su valor es constante entre dos instantes de tiempo. Por otro lado, la interpolación es *lineal* cuando el valor evoluciona linealmente entre dos instantes sucesivos. Por ejemplo, la velocidad de un automóvil puede representarse con un flotante temporal, lo que indica que los valores se conocen en los instantes de tiempo pero evolucionan

continuamente entre ellos. De manera similar, la ubicación de un vehículo se puede representar mediante un punto temporal en el que la ubicación entre dos lecturas GPS consecutivas se obtiene mediante interpolación lineal. Los tipos temporales basados en tipos base discretos, es decir `tbool`, `tint` o `ttext` evolucionan necesariamente de manera escalonada. Por otro lado, los tipos temporales basados en tipos base continuos, es decir `tfloat`, `tgeompoint` o `tgeogpoint` pueden evolucionar de manera lineal o escalonada. Nótese que los tipos `tgeometry` y `tgeography` solo admiten interpolación discreta o por escalonada, ya que no es posible interpolar linealmente dos geometrías/geografías arbitrarias.

El *subtipo* de un valor temporal establece la extensión temporal en la que se registra la evolución de los valores. Los valores temporales vienen en tres subtipos, explicados a continuación.

Un valor temporal de subtipo *instante* (brevemente, un *valor de instante*) representa el valor en un instante de tiempo, por ejemplo

```
SELECT tfloat '17@2018-01-01 08:00:00';
```

Un valor temporal de subtipo *secuencia* (brevemente, un *valor de secuencia*) representa la evolución del valor durante una secuencia de instantes de tiempo, donde los valores entre estos instantes se interpolan usando una función discreta, escalonada, o lineal (ver arriba). Un ejemplo es el siguiente:

```
-- Discrete interpolation
SELECT tfloat '{17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00,
  18@2018-01-01 08:10:00}';
-- Step interpolation
SELECT tfloat 'Interp=Step;
  (10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00]';
-- Linear interpolation
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00]';
```

Como puede verse, un valor de secuencia tiene un límite superior e inferior que pueden ser inclusivos (representados por '`[`' y '`]`') o exclusivos (representados por '`(`' y '`)`'). Por definición, ambos límites deben ser inclusivos cuando la interpolación es discreta o cuando la secuencia tiene un solo instante (que se llama *secuencia instantánea*), como el ejemplo suiguiente.

```
SELECT tint '[10@2018-01-01 08:00:00]';
```

Los valores de secuencia deben ser *uniformes*, es decir, deben construirse a partir de valores de instante del mismo tipo de base. Los valores de secuencia con interpolación escalonada o lineal se denominan *secuencias continuas*.

El valor de una secuencia temporal se interpreta asumiendo que el período de tiempo definido por cada par de valores consecutivos `v1@t1` y `v2@t2` es inferior inclusivo y superior exclusivo, a menos que sean el primer o el último instantes de la secuencia y, en ese caso, se aplican los límites de toda la secuencia. Además, el valor que toma la secuencia temporal entre dos instantes consecutivos depende de si la interpolación es escalonada o lineal. Por ejemplo, la secuencia temporal anterior representa que el valor es `10` durante `(2018-01-01 08:00:00, 2018-01-01 08:05:00)`, `20` durante `[2018-01-01 08:05:00, 2018-01-01 08:10:00)` y `15` en el instante final `2018-01-01 08:10:00`. Por otro lado, la siguiente secuencia temporal

```
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00]';
```

representa que el valor evoluciona linealmente de `10` a `20` durante `(2018-01-01 08:00:00, 2018-01-01 08:05:00)` y evoluciona de `20` a `15` durante `[2018-01-01 08:05:00, 2018-01-01 08:10:00]`.

Finalmente, un valor temporal de subtipo *conjunto de secuencias* (brevemente, un *valor de conjunto de secuencias*) representa la evolución del valor en un conjunto de secuencias, donde se desconocen los valores entre estas secuencias. Un ejemplo es el siguiente:

```
SELECT tfloat '{[17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00],
  [18@2018-01-01 08:10:00, 18@2018-01-01 08:15:00]}';
```

Como se muestra en los ejemplos anteriores, los valores de conjunto de secuencias solo pueden ser de interpolación escalonada o lineal. Además, todas las secuencias que componen un valor de conjunto de secuencias deben ser del mismo tipo de base y de la misma interpolación.

Los valores de secuencia continuos se convierten en *forma normal* de modo que los valores equivalentes tengan representaciones idénticas. Para ello, los valores de instante consecutivos se fusionan cuando es posible. Para la interpolación escalonada, tres

valores instantáneos consecutivos se pueden fusionar en dos si tienen el mismo valor. Para la interpolación lineal, tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Esto se decide de forma exacta sobre los valores almacenados, de modo que un valor en forma normal es su propia forma normal y un valor leído de nuevo desde su representación binaria es el valor escrito. La interpolación de los puntos geodésicos sigue el círculo máximo, y un punto geodésico se fusiona cuando se encuentra sobre esa trayectoria dentro del redondeo del punto interpolado. Ejemplos de transformación en forma normal son los siguientes.

```
SELECT tint '[1@2001-01-01, 2@2001-01-03, 2@2001-01-04, 2@2001-01-05)';
-- [1@2001-01-01, 2@2001-01-03, 2@2001-01-05)
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
  Point(1 1)@2001-01-01 08:10:00)');
-- [POINT(1 1)@2001-01-01 08:00:00, POINT(1 1)@2001-01-01 08:10:00)
SELECT tfloat '[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]';
-- [1@2001-01-01, 3@2001-01-05]
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(2 2)@2001-01-01 08:05:00,
  Point(3 3)@2001-01-01 08:10:00]');
-- [POINT(1 1)@2001-01-01 08:00:00, POINT(3 3)@2001-01-01 08:10:00]
```

De manera similar, los valores del conjunto de secuencias temporales se convierten en forma normal. Para ello, los valores de secuencia consecutivos se fusionan cuando es posible. Ejemplos de transformación en forma normal son los siguientes.

```
SELECT tint '{[1@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)}';
-- '{[1@2001-01-01, 2@2001-01-03, 2@2001-01-05)}'
SELECT tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-03, 3@2001-01-05]}';
-- '{[1@2001-01-01, 3@2001-01-05]}'
SELECT tfloat '{[1@2001-01-01, 3@2001-01-05), [3@2001-01-05]}';
-- '{[1@2001-01-01, 3@2001-01-05]}'
SELECT asText(tgeompoint '{[Point(0 0)@2001-01-01 08:00:00,
  Point(1 1)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00),
  [Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)}');
/* {[[Point(0 0)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
   Point(1 1)@2001-01-01 08:15:00)} */
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,
  Point(2 2)@2001-01-01 08:05:00),
  [Point(2 2)@2001-01-01 08:05:00, Point(3 3)@2001-01-01 08:10:00]}');
-- {[POINT(1 1)@2001-01-01 08:00:00, POINT(3 3)@2001-01-01 08:10:00]}
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,
  Point(3 3)@2001-01-01 08:10:00), [Point(3 3)@2001-01-01 08:10:00]}');
-- {[POINT(1 1)@2001-01-01 08:00:00, POINT(3 3)@2001-01-01 08:10:00]}
```

Los tipos temporales soportan *modificadores de typo* (o `typmod` en terminología de PostgreSQL), que especifican información adicional para la definición de una columna. Por ejemplo, en la siguiente definición de tabla:

```
CREATE TABLE Department(DeptNo integer, DeptName varchar(25), NoEmps tint(Sequence));
```

el modificador de tipo para el tipo `varchar` es el valor 25, que indica la longitud máxima de los valores de la columna, mientras que el modificador de tipo para el tipo `tint` es la cadena de caracteres `Sequence`, que restringe el subtipo de los valores de la columna para que sean secuencias. En el caso de tipos alfanuméricos temporales (es decir, `tbool`, `tint`, `tfloat` y `ttext`), los valores posibles para el modificador de tipo son `Instant`, `Sequence` y `SequenceSet`. Si no se especifica ningún modificador de tipo para una columna, se permiten valores de cualquier subtipo.

Por otro lado, en el caso de tipos de puntos temporales (es decir, `tgeompoint` o `tgeogpoint`) el modificador de tipo se puede utilizar para especificar el subtipo, la dimensionalidad y/o el identificador de referencia espacial (SRID). Por ejemplo, en la siguiente definición de tabla:

```
CREATE TABLE Flight(FlightNo integer, Route tgeogpoint(Sequence, PointZ, 4326));
CREATE TABLE Storm(StormId integer, Route tgeography(Sequence, Linestring, 4326));
```

el modificador de typo para el tipo `tgeogpoint` se compone de tres valores, el primero indica el subtipo como arriba, el segundo el tipo espacial de las geografías que componen el punto temporal y el último el SRID de las geografías que componen. Para los puntos temporales, los valores posibles para el primer argumento del modificador de tipo son los anteriores, los del

segundo argumento son `Point` o `PointZ` y los del tercer argumento son SRID válidos. Los tres argumentos son opcionales y si alguno de ellos no se especifica para una columna, se permiten valores de cualquier subtipo, dimensionalidad y/o SRID.

Cada tipo temporal está asociado a otro tipo, conocido como su *cuadro delimitador*, que representan su extensión en la dimensión de valor y/o tiempo. El cuadro delimitador de los distintos tipos temporales es el siguiente:

- El tipo `tstzspan` para los tipos `tbool` y `ttext`, donde solo se considera la extensión temporal.
- El tipo `tbox` (temporal box) par los tipos `tint` y `tfloat`, donde la extensión del valor y de tiempo se define, respectivamente, por un rango numérico y un rango de tiempo.
- El tipo `stbox` (spatiotemporal box) para los tipos `tgeompoint` y `tgeogpoint`, donde la extensión espacial se define en las dimensiones X, Y y Z y la extensión de timpo se define en un rango de tiempo.

Un amplio conjunto de funciones y operadores son disponibles para realizar varias operaciones en los tipos temporales. Estos se explican a continuación y en los próximos capítulos. Algunas de estas operaciones, en particular las relacionadas con índices, manipulan cuadros delimitadores por razones de eficiencia.

4.2. Ejemplos de tipos temporales

A continuación se dan ejemplos de uso de tipos alfanuméricos temporales.

```
CREATE TABLE Department(DeptNo integer, DeptName varchar(25), NoEmps tint);
INSERT INTO Department VALUES
  (10, 'Research', tint '[10@2001-01-01, 12@2001-04-01, 12@2001-08-01)'),
  (20, 'Human Resources', tint '[4@2001-02-01, 6@2001-06-01, 6@2001-10-01)');
CREATE TABLE Temperature(RoomNo integer, Temp tfloat);
INSERT INTO Temperature VALUES
  (1001, tfloat '{18.5@2001-01-01 08:00:00, 20.0@2001-01-01 08:10:00}'),
  (2001, tfloat '{19.0@2001-01-01 08:00:00, 22.5@2001-01-01 08:10:00}');
-- Value at a timestamp
SELECT RoomNo, valueAtTimestamp(Temp, '2001-01-01 08:10:00') FROM temperature;
-- 1001 | 20
-- 2001 | 22.5
-- Restriction to a value
SELECT DeptNo, atValue(NoEmps, 10) FROM Department;
-- 10 | [10@2001-01-01, 10@2001-04-01)
-- 20 |
-- Restriction to a period
SELECT DeptNo, atTime(NoEmps, tstzspan '[2001-01-01, 2001-04-01]') FROM Department;
-- 10 | [10@2001-01-01, 12@2001-04-01]
-- 20 | [4@2001-02-01, 4@2001-04-01]
-- Temporal comparison
SELECT DeptNo, NoEmps #<= 10 FROM Department;
-- 10 | [t@2001-01-01, f@2001-04-01, f@2001-08-01)
-- 20 | [t@2001-02-01, t@2001-10-01)
-- Temporal aggregation
SELECT tsum(NoEmps) FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01,
   18@2001-06-01, 6@2001-08-01, 6@2001-10-01)} */
```

A continuación se dan ejemplos de uso de tipos de puntos temporales.

```
CREATE TABLE Trips(CarId integer, TripId integer, Trip tgeompoint);
INSERT INTO Trips VALUES
  (10, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:00:00, Point(2 0)@2001-01-01 08:10:00,
Point(2 1)@2001-01-01 08:15:00)}'),
  (20, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00,
  Point(3 3)@2001-01-01 08:20:00)}');
```

```
-- Value at a given timestamp
SELECT CarId,
  ST_AsText(valueAtTimestamp(Trip, timestamptz '2001-01-01 08:10:00')) FROM Trips;
-- 10 | POINT(2 0)
-- 20 | POINT(1 1)
-- Restriction to a value
SELECT CarId, asText(atValue(Trip, geometry 'Point(2 0)')) FROM Trips;
-- 10 | {"[POINT(2 0)@2001-01-01 08:10:00]"}
-- 20 |
-- Restriction to a period
SELECT CarId,
  asText(atTime(Trip, tstzspan '[2001-01-01 08:05:00,2001-01-01 08:10:00]')) FROM Trips;
-- 10 | {[POINT(1 0)@2001-01-01 08:05:00, POINT(2 0)@2001-01-01 08:10:00]}
-- 20 | {[POINT(0 0)@2001-01-01 08:05:00, POINT(1 1)@2001-01-01 08:10:00]}
-- Temporal distance
SELECT T1.CarId, T2.CarId, T1.Trip <-> T2.Trip FROM Trips T1,
  Trips T2 WHERE T1.CarId < T2.CarId;
/* 10 | 20 | {[1@2001-01-01 08:05:00, 1.4142135623731@2001-01-01 08:10:00,
   1@2001-01-01 08:15:00)} */
```

4.3. Validez de los tipos temporales

Los valores de los tipos temporales deben satisfacer varias restricciones para que estén bien definidos. Estas restricciones se dan a continuación.

- Las restricciones en el tipo base y el tipo `timestamptz` deben satisfacerse.
- Un valor de un `tint` o de un `tbigint` debe ser menor que el mayor valor de su tipo base, ya que el intervalo de valores del cuadro delimitador, cuyo límite superior es el valor siguiente, no tiene representación para él, como para un `intspan` o un `bigintspan`.
- Un valor de secuencia debe estar compuesto por al menos un valor de instante.
- Un valor de secuencia instantánea o con interpolación discreta debe tener límites inferior y superior inclusivos.
- En un valor de secuencia, las marcas de tiempo de los instantes que la componen deben ser diferentes y estar ordenadas.
- En un valor de secuencia con interpolación escalonada, los dos últimos valores deben ser iguales si el límite superior es exclusivo.
- Un valor de conjunto de secuencias debe estar compuesto por al menos un valor de secuencia.
- En un valor de conjunto de secuencias, los valores de las secuencias componentes no deben superponerse y deben estar ordenados.

Se genera un error cuando no se satisface una de estas restricciones. Ejemplos de valores temporales incorrectos son los siguientes.

```
-- Incorrect value for base type
SELECT tbool '1.5@2001-01-01 08:00:00';
-- Base type value is not a point
SELECT tgeompoint 'Linestring(0 0,1 1)@2001-01-01 08:05:00';
-- Value at the largest value of the base type
SELECT tint '2147483647@2001-01-01 08:00:00';
-- Incorrect timestamp
SELECT tint '2@2001-02-31 08:00:00';
-- Empty sequence
SELECT tint '';
-- Incorrect bounds for instantaneous sequence
```

```
SELECT tint '[1@2001-01-01 09:00:00)';
-- Duplicate timestamps
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00]';
-- Unordered timestamps
SELECT tint '[1@2001-01-01 08:10:00, 2@2001-01-01 08:00:00]';
-- Incorrect end value for step interpolation
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:10:00)';
-- Empty sequence set
SELECT tint '{[]}';
-- Duplicate timestamps
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00}';
-- Overlapping periods
SELECT tint '{[1@2001-01-01 08:00:00, 1@2001-01-01 10:00:00),
  [2@2001-01-01 09:00:00, 2@2001-01-01 11:00:00)}';
```

4.4. Temporalización de operaciones

Una forma común de generalizar las operaciones tradicionales a los tipos temporales es aplicar la operación en *cada instante*, lo que da un valor temporal como resultado. En ese caso, la operación sólo se define en la intersección de las extensiones temporales de los operandos; si las extensiones temporales son disjuntas, el resultado es nulo. Por ejemplo, los operadores de comparación temporal, como `#<`, determinan si los valores tomados por sus operandos en cada instante satisfacen la condición y devuelven un booleano temporal. A continuación se dan ejemplos de las diversas generalizaciones de los operadores.

```
-- Temporal comparison
SELECT tfloat '[2@2001-01-01, 2@2001-01-03)' #< tfloat '[1@2001-01-01, 3@2001-01-03)';
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
SELECT tfloat '[1@2001-01-01, 3@2001-01-03)' #< tfloat '[3@2001-01-03, 1@2001-01-05)';
-- NULL
-- Temporal addition
SELECT tint '[1@2001-01-01, 1@2001-01-03)' + tint '[2@2001-01-02, 2@2001-01-05)';
-- [3@2001-01-02, 3@2001-01-03)
-- Temporal intersects
SELECT tIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04)',
  geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04]}
-- Temporal distance
SELECT tgeompoint '[Point(0 0)@2001-01-01 08:00:00, Point(0 1)@2001-01-03 08:10:00)' <->
  tgeompoint '[Point(0 0)@2001-01-02 08:05:00, Point(1 1)@2001-01-05 08:15:00)';
-- [0.5@2001-01-02 08:05:00, 0.7451840337945570@2001-01-03 08:10:00)
```

Otro requisito común es determinar si los operandos satisfacen *alguna vez* o *siempre* una condición con respecto a una operación. Estos se pueden obtener aplicando los operadores de comparación alguna vez o siempre. Estos operadores se indican anteponiendo los operadores de comparación tradicionales con, respectivamente, `?` (alguna vez) y `%` (siempre). A continuación se dan ejemplos de operadores de comparación alguna vez y siempre.

```
-- Does the operands ever intersect?
SELECT eIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04)',
  geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))') ?= true;
-- true
-- Does the operands always intersect?
SELECT aIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04)',
  geometry 'Polygon((0 0,0 2,4 2,4 0,0 0))') %= true;
-- true
-- Is the left operand ever less than the right one ?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03)' #<
  tfloat '[3@2001-01-01, 1@2001-01-03)') ?= true;
-- true
-- Is the left operand always less than the right one ?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03)' #<
```

```
  tfloat '[2@2001-01-01, 4@2001-01-03)') %= true;
-- true
```

Por ejemplo, la función `eIntersects` determina si hay un instante en el que los dos argumentos se cruzan espacialmente.

A continuación describimos las funciones y operadores para tipos temporales. Para mayor concisión, en los ejemplos usamos principalmente secuencias compuestas por dos instantes.

4.5. Notación

A continuación presentamos las funciones y operadores para tipos temporales. Estas funciones y operadores son polimórficos, es decir, sus argumentos pueden ser de varios tipos y el tipo del resultado puede depender del tipo de los argumentos. Para expresar esto, usamos la siguiente notación:

- `times` representa cualquier tipo de tiempo con granularidad `timestamptz`, a saber `timestamptz`, `tstzset`, `tstzspan` o `tstzspanset`,
- `ttype` representa cualquier tipo temporal,
- `ttypeInst`, `ttypSeq` y `ttypeSeqSet` representan cualquier tipo temporal con subtipo instante, secuencia y conjunto de secuencias
- `tdisc` representa cualquier tipo temporal con tipo de base discreto, es decir, `tbool`, `tint`, or `ttext`,
- `tcont` cualquier tipo temporal con tipo de base contínuo, es decir, `tfloat`, `tgeompoint`, or `tgeogpoint`,
- `ttypeDiscSeq` y `ttypeContSeq` representan cualquier tipo temporal con subtipo secuencia y, respectivamente, interpolación discreta y contínua,
- `base` representa cualquier tipo de base de un tipo temporal, es decir, `boolean`, `integer`, `float`, `text`, `geometry` o `geography`,
- `values` representa cualquier conjunto de valores de un tipo base de un tipo temporal, por ejemplo, `integer`, `intset`, `intspan` y `intspanset` para el tipo base `integer`
- `type[]` representa una matriz de `type`.
- `<type>` en el nombre de una función representa las funciones obtenidas al remplazar `<type>` por un `type` específico. Por ejemplo, `tintDiscSeq` o `tfloatDiscSeq` son representadas por `ttypeDiscSeq`.

4.6. Entrada y salida

MobilityDB generaliza los formatos de entrada y salida Well-Known Text (WKT), Moving Features JSON (MF-JSON) y Well-Known Binary (WKB) del Open Geospatial Consortium para todos los tipos temporales. Presentamos a continuación las funciones de entrada y salida para los tipos temporales. Empezamos describiendo formato WKT.

Un valor de instante es un par de la forma `v@t`, donde `v` es un valor del tipo de base y `t` es un valor de `timestamptz`. Ejemplos de entrada de valores de instante son los siguientes:

```
SELECT tbool 'true@2001-01-01 08:00:00';
SELECT tint '1@2001-01-01 08:00:00';
SELECT tbigint '5@2001-01-01 08:00:00';
SELECT tfloat '1.5@2001-01-01 08:00:00';
SELECT ttext 'AAA@2001-01-01 08:00:00';
SELECT tgeompoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeogpoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeometry 'Linestring(0 0,1 1)@2017-01-01 08:00:05';
SELECT tgeography 'Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:00:05';
```

Un valor de secuencia es un conjunto de valores `v1@t1,...,vn@tn` delimitado por límites superior e inferior, que pueden ser inclusivo (representados por '`[`' y '`]`') o exclusivos (representados por '`(`' y '`)`'). Un valor de secuencia compuesto por una sola pareja `v@t` se denomina *secuencia instantánea*. Los valores de secuencia tienen una *función de interpolación* asociada que puede ser discreta, lineal o escalonada. Por definición, los límites inferior y superior de una secuencia instantánea o de un valor de secuencia con interpolación discreta son inclusivos. La extensión temporal de un valor de secuencia con interpolación discreta es un conjunto de marcas de tiempo. Ejemplos de valores de secuencia con interpolación discreta son los siguientes.

```
SELECT tbool '{true@2001-01-01 08:00:00, false@2001-01-03 08:00:00}';
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00}';
SELECT tbigint '{1@2001-01-01 08:00:00}'; -- Instantaneous sequence
SELECT tfloat '{1.0@2001-01-01 08:00:00, 2.0@2001-01-03 08:00:00}';
SELECT ttext '{AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00}';
SELECT tgeompoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeogpoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeometry '{Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,0 1)@2017-01-02 08:05:00}';
SELECT tgeography '{Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00}';
```

La extensión temporal de un valor de secuencia con interpolación lineal o escalonada es un período definido por el primer y el últimpo instante, así como por los límites inferior y superior. Ejemplos de valores de secuencia con interpolación lineal son los siguientes:

```
SELECT tbool '[true@2001-01-01 08:00:00, true@2001-01-03 08:00:00]';
SELECT tint '[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00]';
SELECT tfloat '[2.5@2001-01-01 08:00:00, 3@2001-01-03 08:00:00, 1@2001-01-04 08:00:00]';
SELECT tfloat '[1.5@2001-01-01 08:00:00]'; -- Instantaneous sequence
SELECT ttext '[BBB@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00]';
SELECT tgeompoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 0)@2017-01-01 08:05:00)';
SELECT tgeogpoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00,
  Point(0 0)@2017-01-01 08:10:00)';
SELECT tgeometry '[Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,0 1)@2017-01-02 08:05:00]';
SELECT tgeography '[Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00]';
```

Los valores de secuencia cuyo tipo base es continuo pueden especificar que la interpolación es escalonada con el prefijo `Interp=Step`. Si no se especifica, se supone que la interpolación es lineal por defecto. A continuación se dan ejemplos de valores de secuencia con interpolación escalonada:

```
SELECT tfloat 'Interp=Step;[2.5@2001-01-01 08:00:00, 3@2001-01-01 08:10:00]';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
  Point(1 1)@2017-01-01 08:05:00, Point(1 1)@2017-01-01 08:10:00)';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
 Point(1 1)@2017-01-01 08:05:00, Point(0 0)@2017-01-01 08:10:00)';
ERROR:  Invalid end value for temporal sequence with step interpolation
SELECT tgeogpoint 'Interp=Step;
  [Point(0 0)@2017-01-01 08:00:00, Point(1 1)@2017-01-01 08:10:00]';
```

Los dos últimos instantes de un valor de secuencia con interpolación discreta y límite superior exclusivo deben tener el mismo valor base, como se muestra en el segundo y tercer ejemplo anteriores.

El prefijo `Interp=Linear` indica explícitamente la interpolación lineal por defecto. Un tipo temporal cuyos valores no pueden interpolarse linealmente, como `tint` o `tgeometry`, rechaza este prefijo, como se muestra a continuación:

```
SELECT tfloat 'Interp=Linear;[2.5@2001-01-01 08:00:00, 3@2001-01-01 08:10:00]';
-- [2.5@2001-01-01 08:00:00+01, 3@2001-01-01 08:10:00+01]
SELECT tgeometry 'Interp=Linear;[Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,0 1)@2017-01-02 08:05:00]';
ERROR:  The temporal type cannot have linear interpolation
```

Un *valor de conjunto de secuencias* es un conjunto `{v1,...,vn}` donde cada `vi` es un valor de secuencia. La interpolación de los valores conjunto de secuencias solo puede ser lineal o escalonada, no discreta. Todas las secuencias que componen un valor de conjunto de secuencias deben tener la misma interpolación. La extensión temporal de un valor de conjunto de secuencias es un conjunto de períodos. Ejemplos de valores de conjunto de secuencias con interpolación lineal son los siguientes:

```
SELECT tbool '{[false@2001-01-01 08:00:00, false@2001-01-03 08:00:00),
  [true@2001-01-03 08:00:00], (false@2001-01-04 08:00:00, false@2001-01-06 08:00:00]}';
SELECT tint '{[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00),
  [2@2001-01-04 08:00:00, 3@2001-01-05 08:00:00, 3@2001-01-06 08:00:00]}';
SELECT tfloat '{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00, 2@2001-01-04 08:00:00,
  3@2001-01-06 08:00:00]}';
SELECT ttext '{[AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00,
  BBB@2001-01-04 08:00:00), [CCC@2001-01-05 08:00:00, CCC@2001-01-06 08:00:00]}';
SELECT tgeompoint '{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}';
SELECT tgeogpoint '{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}';
SELECT tgeometry '{[Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,1 1)@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}';
SELECT tgeography '{[Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Linestring(0 0,1 1)@2017-01-01 08:15:00)}';
```

Los valores de conjunto de secuencias cuyo tipo base es continuo pueden especificar que la interpolación es escalonada con el prefijo `Interp=Step`. Si no se especifica, se supone que la interpolación es lineal por defecto. A continuación se dan ejemplos de valores de conjunto de secuencias con interpolación escalonada:

```
SELECT tfloat 'Interp=Step;{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00,
  2@2001-01-04 08:00:00, 3@2001-01-06 08:00:00]}';
SELECT tgeompoint 'Interp=Step;
  {[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00],
  [Point(0 1)@2017-01-01 08:10:00, Point(0 1)@2017-01-01 08:15:00)}';
SELECT tgeogpoint 'Interp=Step;
  {[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00],
  [Point(0 1)@2017-01-01 08:10:00, Point(0 1)@2017-01-01 08:15:00)}';
```

Para geometrías temporales, es posible especificar el identificador de referencia espacial (SRID) utilizando la representación extendida de texto conocido (EWKT) de la siguiente manera:

```
SELECT tgeompoint 'SRID=5435;[Point(0 0)@2001-01-01,Point(0 1)@2001-01-02]'
SELECT tgeography 'SRID=7844;[Point(0 0)@2001-01-01,Linestring(1 0,0 1)@2001-01-02]';
```

Todas las geometrías componentes serán entonces del SRID dado. Además, cada geometría componente puede especificar su SRID con el formato EWKT como en el siguiente ejemplo

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=5435;Point(0 1)@2001-01-02]';
```

Se genera un error si las geometrías componentes no están todas en el mismo SRID o si el SRID de una geometría componente es diferente al del punto temporal.

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=4326;Point(0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (5435)
SELECT tgeography 'SRID=7844;
  [SRID=4326;Point(0 0)@2001-01-01, SRID=4326;Linestring(1 0,0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (7844)
```

Si no se especifica, el SRID predeterminado para los geometrías temporales es 0 (desconocido) y para los geografías temporales es 4326 (WGS 84). Los puntos temporales con interpolación escalonada también pueden especificar el SRID, como se muestra a continuación.

```
SELECT tgeompoint 'SRID=5435,Interp=Step;[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02]';
SELECT tgeogpoint 'Interp=Step;
  [SRID=4326;Point(0 0)@2001-01-01, SRID=4326;Point(0 1)@2001-01-02]';
```

Damos a continuación las funciones de entrada y salida en formato Well-Known Text (WKT), Well-Known Binary (WKB) y Moving Features JSON (MF-JSON) para los tipos alfanuméricos temporales. Las funciones correspondientes para los puntos temporales se detallan en la Sección 7.4. El formato de salida predeterminado de todos los tipos alfanuméricos temporales es el formato de texto conocido. La función `asText` que se da a continuación permite determinar la salida de valores temporales de punto flotante.

- Devuelve la representación de texto conocido (WKT)

```
asText(ttype,maxdecimaldigits integer=15) → text
```

El argumento `maxdecimaldigits` puede usarse para establecer el número máximo de decimales en la salida de los valores en punto flotante (por defecto 15).

```
SELECT asText(tfloat '[10.55@2001-01-01, 25.55@2001-01-02]', 0);
-- [11@2001-01-01, 26@2001-01-02]
SELECT asText(tgeometry '[Point(1.55 1.55)@2001-01-01,
  Linestring(1.55 1.55,3.55 3.55)@2001-01-02]', 0);
-- [POINT(1 1)@2001-01-01, LINESTRING(1 1,3 3)@2001-01-02]
```

- Devuelve la representación JSON de características móviles (MF-JSON)

```
asMFJSON(ttype,options integer=0,flags integer=0,maxdecimaldigits integer=15) → bytea
```

El argumento `options` puede usarse para agregar un cuadro delimitador en la salida MFJSON:

- 0: significa que no hay opción (valor por defecto)
- 1: cuadro delimitador MFJSON

El argumento `flags` puede usarse para personalizar la salida JSON, por ejemplo, para producir una salida JSON fácil de leer (para lectores humanos). Consulte la documentación de la biblioteca `json-c` para conocer los valores possible. Los valores típicos son los siguientes:

- 0: means no option (default value)
- 1: JSON_C_TO_STRING_SPACED
- 2: JSON_C_TO_STRING_PRETTY

El argumento `maxdecimaldigits` puede usarse para establecer el número máximo de decimales en la salida de los valores en punto flotante (por defecto 15).

```
SELECT asMFJSON(tbool 't@2001-01-01 18:00:00', 1);
/* {"type":"MovingBoolean","period":{"begin":"2001-01-01T18:00:00",
   "end":"2001-01-01T18:00:00","lowerInc":true,"upperInc":true},
   "values":[true],"datetimes":["2001-01-01T18:00:00"],"interpolation":"None"} */
SELECT asMFJSON(tint '{10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00}', 1);
/* {"type":"MovingInteger","bbox":[10,25],"period":{"begin":"2001-01-01T18:00:00",
   "end":"2001-01-01T18:10:00"},"values":[10,25],"datetimes":["2001-01-01T18:00:00",
   "2001-01-01T18:10:00"],"lowerInc":true,"upperInc":true, "interpolation":"Discrete"} */
SELECT asMFJSON(tfloat '[10.5@2001-01-01 18:00:00+02, 25.5@2001-01-01 18:10:00+02]');
/* {"type":"MovingFloat","values":[10.5,25.5],"datetimes":["2001-01-01T17:00:00",
   "2001-01-01T17:10:00"],"lowerInc":true,"upperInc":true,"interpolation":"Linear"} */
SELECT asMFJSON(ttext '{[walking@2001-01-01 18:00:00+02,
  driving@2001-01-01 18:10:00+02]}');
/* {"type":"MovingText","sequences":[{"values":["walking","driving"],
   "datetimes":["2001-01-01T17:00:00","2001-01-01T17:10:00"],
   "lowerInc":true,"upperInc":true}],"interpolation":"Step"} */
SELECT asMFJSON(tgeometry '{[Point(1 1)@2001-01-01 18:00:00+02,
```

```
  Linestring(1 1,2 2)@2001-01-01 18:10:00+02]}');
/* {"type":"MovingGeometry","sequences":[{"values":[{"type":"Point",
   "coordinates":[1,1]},{"type":"LineString","coordinates":[[1,1],[2,2]]}],
   "datetimes":["2001-01-01T17:00:00","2001-01-01T17:10:00"],
   "lower_inc":true,"upper_inc":true}],"interpolation":"Step"} */
```

- Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)

```
asBinary(ttype,endian text='') → bytea
asHexWKB(ttype,endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tbool 'true@2001-01-01');
-- \x011a000101009c57d3c11c0000
SELECT asBinary(tfloat '1.5@2001-01-01');
-- \x012100010000000000000f83f009c57d3c11c0000
SELECT asHexWKB(tint '1@2001-01-01', 'XDR');
-- 00002301000000100001CC1D3579C00
SELECT asHexWKB(ttext 'AAA@2001-01-01');
-- 012900010400000000000000041414100009C57D3C11C0000
```

- Entrada desde la representación de texto conocido (WKT)

```
ttypeFromText(text) → ttype
```

Hay una función por tipo temporal, el nombre de la función tiene como prefijo el nombre del tipo, que son `tbool`, `tint`, `tfloat` o `ttext` en los ejemplos a continuación. La función es la inversa de `asText`: lee el texto en el que se escribe un literal del tipo, tanto de una constante como de una columna `text`, de modo que `tintFromText(col)` y `col::tint` dan el mismo valor. La función de un tipo espaciotemporal se da en tspatialFromText y la de `tjsonb` en tjsonbFromText.

```
SELECT tboolFromText('[true@2001-01-01, false@2001-01-03]');
-- [t@2001-01-01, f@2001-01-03]
SELECT tintFromText('[1@2001-01-01, 2@2001-01-02]');
-- [1@2001-01-01, 2@2001-01-02]
SELECT tfloatFromText('Interp=Step;[1.5@2001-01-01, 2.5@2001-01-02]');
-- Interp=Step;[1.5@2001-01-01, 2.5@2001-01-02]
SELECT ttextFromText('{"walk"@2001-01-01, "bus"@2001-01-02}');
-- {"walk"@2001-01-01, "bus"@2001-01-02}
SELECT tintFromText(t) FROM (VALUES (text '[1@2001-01-01, 2@2001-01-02]')) v(t);
-- [1@2001-01-01, 2@2001-01-02]
```

- Entrada desde la representación JSON de características móviles (MF-JSON)

```
ttypeFromMFJSON(bytea) → ttype
```

Hay una función por tipo temporal, el nombre de esta función tiene como prefijo el nombre del tipo, que son `tbool` o `tint` en los ejemplos a continuación

```
SELECT tboolFromMFJSON(text '{"type":"MovingBoolean",
  "period":{"begin":"2001-01-01T18:00:00+01", "end":"2001-01-01T18:00:00+01",
  "lowerInc":true,"upperInc":true}, "values":[true],
  "datetimes":["2001-01-01T18:00:00+01"],"interpolation":"None"}');
-- t@2001-01-01 18:00:00
SELECT tintFromMFJSON(text '{"type":"MovingInteger","bbox":[10,25],
  "period":{"begin":"2001-01-01T18:00:00+01", "end":"2001-01-01T18:10:00+01"},
  "values":[10,25],"datetimes":["2001-01-01T18:00:00+01", "2001-01-01T18:10:00+01"],
  "lowerInc":true,"upperInc":true, "interpolation":"Discrete"}');
-- {10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00}
SELECT tfloatFromMFJSON(text '{"type":"MovingFloat","values":[10.5,25.5],
```

```
  "datetimes":["2001-01-01T17:00:00+01", "2001-01-01T17:10:00+01"],"lowerInc":true,
  "upperInc":true, "interpolation":"Linear"}');
-- [10.5@2001-01-01 18:00:00, 25.5@2001-01-01 18:10:00]'
SELECT ttextFromMFJSON(text '{"type":"MovingText",
  "sequences":[{"values":["walking","driving"],
  "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"], "lowerInc":true,
  "upperInc":true}],"interpolation":"Step"}');
-- {["walking"@2001-01-01 18:00:00, "driving"@2001-01-01 18:10:00]}');
SELECT asText(tgeometryFromMFJSON(text '{"type":"MovingGeometry",
  "sequences":[{"values":[{"type":"Point", "coordinates":[1,1]},
  {"type":"LineString","coordinates":[[1,1],[2,2]]}],
  "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"], "lower_inc":true,
  "upper_inc":true}],"interpolation":"Step"}'));
-- {[POINT(1 1)@2001-01-01 17:00:00, LINESTRING(1 1,2 2)@2001-01-01 17:10:00]}
```

- Entrada desde la representación binaria conocida (WKB) o desde la representación hexadecimal binaria conocida (HexWKB)

```
ttypeFromBinary(bytea) → ttype
ttypeFromHexWKB(text) → ttype
```

Hay una función por tipo temporal, el nombre de la función tiene como prefijo el nombre del tipo, que son `tbool` o `tint` en los ejemplos a continuación.

```
SELECT tboolFromBinary('\x011a000101009c57d3c11c0000');
-- t@2001-01-01
SELECT tintFromBinary('\x0000230100000000100001cc1d3579c00');
-- 1@2001-01-01
SELECT tfloatFromBinary('\x012100010000000000000f83f009c57d3c11c0000');
-- 1.5@2001-01-01
SELECT ttextFromBinary('\x012900010400000000000000041414100009c57d3c11c0000');
-- "AAA"@2001-01-01
SELECT tboolFromHexWKB('011A000101009C57D3C11C0000');
-- t@2001-01-01
SELECT tintFromHexWKB('0000230100000000100001CC1D3579C00');
-- 1@2001-01-01
SELECT tfloatFromHexWKB('012100010000000000000F83F009C57D3C11C0000');
-- 1.5@2001-01-01
SELECT ttextFromHexWKB('012900010400000000000000041414100009C57D3C11C0000');
-- "AAA"@2001-01-01
```

4.7. Constructores

A continuación, damos las funciones de constructor para los distintos subtipos. El uso de la función de constructor suele ser más conveniente que escribir una constante literal.

- Constructores para tipos temporales que tienen un valor constante

```
ttype(base,timestamptz) → ttypeInst
ttype(base,tstzset) → ttypeDiscSeq
ttype(base,tstzspan,interp text='linear') → ttypeContSeq
ttype(base,tstzspanset,interp text='linear') → ttypeSeqSet
```

Estos constructores tienen dos argumentos, un tipo base y un tipo de tiempo, donde el último es un valor de `timestamptz`, `tstzset`, `tstzspan` o `tstzspanset` para construir, respectivamente, un valor de subtipo instante, una secuencia con interpolación discreta, una secuencia con interpolación linear o escalonada o un conjunto de secuencias. Las funciones para valores de secuencia o de conjunto de secuencias con tipo de base continuo tienen además un tercer argumento opcional que indica si el valor temporal resultante tiene interpolación lineal o escalonada. Se asume por defecto una interpolación lineal si el argumento no se especifica.

```
SELECT tbool(true, timestamptz '2001-01-01');
SELECT tint(1, timestamptz '2001-01-01');
SELECT tfloat(1.5, tstzset '{2001-01-01, 2001-01-02}');
SELECT ttext('AAA', tstzset '{2001-01-01, 2001-01-02}');
SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]');
SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]', 'step');
SELECT tgeompoint('Point(0 0)', tstzspan '[2001-01-01, 2001-01-02]');
SELECT tgeography('SRID=7844;Point(0 0)',
  tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}', 'step');
```

- Constructores para tipos temporales de subtipo secuencia

```
ttypeSeq(ttypeInst[],interp text={'step','linear'},lowerInc boolean=true,
  upperInc boolean=true) → ttypeSeq
```

Estos constructores tienen un primer argumento obligatorio, que es una matriz de valores de los valores instantáneos correspondientes y argumentos opcionales adicionales. El segundo argumento establece la interpolación. Si no se proporciona el argumento, es por defecto escalonada para los tipos de base base discretos como los enteros y es lineal para tipos de base continuous como los flotantes. Se genera un error cuando se establece la interpolación lineal para valores temporales con tipos de base discretos. Para secuencias continuas, el tercero y el cuarto argumentos son valores booleanos que indican, respectivamente, si los límites izquierdo y derecho son inclusivos o exclusivos. Se supone que estos argumentos son verdaderos de forma predeterminada si no se especifican.

```
SELECT tboolSeq(ARRAY[tbool 'true@2001-01-01 08:00:00','false@2001-01-01 08:05:00'],
  'discrete');
SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00']);
SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00'], 'linear');
-- ERROR:  The temporal type cannot have linear interpolation
SELECT tfloatSeq(ARRAY[tfloat '1.0@2001-01-01 08:00:00', '2.0@2001-01-01 08:05:00'],
  'step', false, true);
SELECT ttextSeq(ARRAY[ttext 'AAA@2001-01-01 08:00:00', 'BBB@2001-01-01 08:05:00']);
SELECT tgeompointSeq(ARRAY[tgeompoint 'Point(0 0)@2001-01-01 08:00:00',
  'Point(0 1)@2001-01-01 08:05:00', 'Point(1 1)@2001-01-01 08:10:00'], 'discrete');
SELECT tgeographySeq(ARRAY[tgeography 'Point(1 1)@2001-01-01 08:00:00',
  'Point(2 2)@2001-01-01 08:05:00']);
```

- Constructores para tipos temporales de subtipo conjunto de secuencias

```
ttypeSeqSet(ttypeContSeq[]) → ttypeSeqSet
ttypeSeqSetGaps(ttypeInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → ttypeSeqSet
```

Un primer conjunto de constructores tiene un solo argumento, que es una matriz de valores de los valores de *secuencia* correspondientes. La interpolación del valor temporal resultante depende de la interpolación de las secuencias que lo componen. Se genera un error si las secuencias que componen la matriz tienen una interpolación diferente.

```
SELECT tboolSeqSet(ARRAY[tbool '[false@2001-01-01 08:00:00, false@2001-01-01 08:05:00)',
  '[true@2001-01-01 08:05:00]','(false@2001-01-01 08:05:00, false@2001-01-01 08:10:00)']);
SELECT tintSeqSet(ARRAY[tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:05:00,
  2@2001-01-01 08:10:00, 2@2001-01-01 08:15:00)']);
SELECT tfloatSeqSet(ARRAY[tfloat '[1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00,
  2.0@2001-01-01 08:10:00]', '[2.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']);
SELECT tfloatSeqSet(ARRAY[tfloat 'Interp=Step;
  [1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
  'Interp=Step;[3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']);
SELECT ttextSeqSet(ARRAY[ttext '[AAA@2001-01-01 08:00:00, AAA@2001-01-01 08:05:00)',
  '[BBB@2001-01-01 08:10:00, BBB@2001-01-01 08:15:00)']);
SELECT tgeompointSeqSet(ARRAY[tgeompoint '[Point(0 0)@2001-01-01 08:00:00,
  Point(0 1)@2001-01-01 08:05:00, Point(0 1)@2001-01-01 08:10:00)',
  '[Point(0 1)@2001-01-01 08:15:00, Point(0 0)@2001-01-01 08:20:00)']);
```

```
SELECT tgeographySeqSet(ARRAY[tgeography '[Point(0 0)@2001-01-01 08:00:00,
  Point(0 0)@2001-01-01 08:05:00)',
  '[Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)']);
SELECT tfloatSeqSet(ARRAY[tfloat 'Interp=Step;
  [1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
  '[3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']);
-- ERROR:  The temporal values must have the same interpolation
```

Otro conjunto de constructores para valores de conjunto de secuencias tiene como primer argumento una matriz de valores de *instante* correspondientes, y dos argumentos que establecen un intervalo de tiempo máximo y una distancia máxima tal que se introduce un espacio entre secuencias del resultado siempre que dos instantes de entrada consecutivos tengan un intervalo de tiempo o una distancia mayor que estos valores. Para puntos temporales, la distancia se especifica en unidades del sistema de coordenadas. Estos dos argumentos de brechas son opcionales, pero al menos uno de ellos debe especificarse. Además, cuando el tipo base es continuo, un último argumento adicional establece si la interpolación es escalonada o lineal. Si no se especifica este argumento, se asume que es lineal por defecto.

Los parámetros de la función dependen del tipo temporal. Por ejemplo, el parámetro de interpolación no está permitido para tipos temporales con subtipos discretos como `tint`. De manera similar, el parámetro `maxdist` no está permitido para tipos escalares como `ttext` que no tienen una función de distancia estándar.

```
SELECT tintSeqSetGaps(ARRAY[tint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
  '5@2001-01-05']);
-- {[1@2001-01-01, 3@2001-01-02, 4@2001-01-03, 5@2001-01-05]}
  SELECT tbigintSeqSetGaps(ARRAY[tbigint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
    '5@2001-01-05'], '1 day', 1);
-- {[1@2001-01-01], [3@2001-01-02, 4@2001-01-03], [5@2001-01-05]}
SELECT ttextSeqSetGaps(ARRAY[ttext 'AA@2001-01-01', 'BB@2001-01-02', 'AA@2001-01-03',
  'CC@2001-01-05'], '1 day');
-- {["AA"@2001-01-01, "BB"@2001-01-02, "AA"@2001-01-03], ["CC"@2001-01-05]}
SELECT asText(tgeometrySeqSetGaps(ARRAY[tgeometry 'Point(1 1)@2001-01-01',
  'Linestring(2 2,3 3)@2001-01-02', 'Point(4 2)@2001-01-03',
  'Polygon((5 0,6 1,7 0,5 0))@2001-01-05'], '1 day', 1));
/* {[POINT(1 1)@2001-01-01], [LINESTRING(2 2,3 3)@2001-01-02],
   [POINT(4 2)@2001-01-03], [POLYGON((5 0,6 1,7 0,5 0))@2001-01-05]} */
SELECT asText(tgeompointSeqSetGaps(ARRAY[tgeompoint 'Point(1 1)@2001-01-01',
  'Point(2 2)@2001-01-02', 'Point(3 2)@2001-01-03', 'Point(3 2)@2001-01-05'], '1 day', 1,
  'step'));
/* Interp=Step;{[POINT(1 1)@2001-01-01], [POINT(2 2)@2001-01-02, POINT(3 2)@2001-01-03],
   [POINT(3 2)@2001-01-05]} */
```

4.8. Conversión de tipos

Un valor temporal se puede convertir en un tipo compatible usando la notación `CAST(ttype1 AS ttype2)` o `ttype1::ttype2`.

- Convierte un valor temporal a un intervalo de marcas de tiempo

```
ttype::tstzspan
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::tstzspan;
-- [2001-01-01, 2001-01-03]
SELECT ttext '(A@2001-01-01, B@2001-01-03, C@2001-01-05]'::tstzspan;
-- (2001-01-01, 2001-01-05]
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]'::tstzspan;
-- [2001-01-01, 2001-01-03]
```

4.9. Accesores

- Devuelve el tamaño de la memoria en bytes

```
memSize(ttype) → integer
```

```
SELECT memSize(tint '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- 176
```

- Devuelve el subtipo temporal

```
tempSubtype(ttype) → {'Instant','Sequence','SequenceSet'}
```

```
SELECT tempSubtype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Sequence
```

- Devuelve el tipo base

```
tempBasetype(ttype) → text
```

```
SELECT tempBasetype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- integer
SELECT tempBasetype(tgeompoint 'Point(1 1)@2001-01-01');
-- geometry
```

- Devuelve la interpolación

```
interp(ttype) → {'None','Discrete','Step','Linear'}
```

```
SELECT interp(tbool 'true@2001-01-01');
-- None
SELECT interp(tfloat '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- Discrete
SELECT interp(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Linear
SELECT interp(tfloat 'Interp=Step;[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tgeompoint 'Interp=Step;
  [Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(3 3)@2001-01-03]');
-- Step
SELECT interp(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02,
  Polygon((3 3,4 4,5 3,3 3))@2001-01-03]');
-- Step
```

- Devuelve el valor o la marca de tiempo de un instantes

```
getValue(ttypeInst) → base
getTimestamp(ttypeInst) → timestamptz
```

```
SELECT getValue(tint '1@2001-01-01');
-- 1
SELECT getTimestamp(tfloat '1@2001-01-01');
-- 2001-01-01
```

- Devuelve los valores o el tiempo en el que se define el valor temporal

```
getValues(ttype) → {boolean[],numspanset,set}
getTime(ttype) → tstzspanset
```

```
SELECT getValues(tbool '[false@2001-01-01, true@2001-01-02, false@2001-01-03]');
-- {f,t}
SELECT getValues(tint '[1@2001-01-01, 3@2001-01-02, 1@2001-01-03]');
-- {[1, 2), [3, 4)}
SELECT getValues(tint '{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [4@2001-01-04, 4@2001-01-05]}');
-- {[1, 3), [4, 5)}
SELECT getValues(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[1, 1], [2, 2]}
SELECT getValues(tfloat 'Interp=Step;
  {[1@2001-01-01, 2@2001-01-02, 1@2001-01-03], [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 1], [2, 2], [3, 3]}
SELECT getValues(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- {[1, 2]}
SELECT getValues(tfloat '{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 2], [3, 3]}
```

```
SELECT getTime(ttext 'walking@2001-01-01');
-- {[2001-01-01, 2001-01-01]}
SELECT getTime(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[2001-01-01, 2001-01-01], [2001-01-02, 2001-01-02], [2001-01-03, 2001-01-03]}
SELECT getTime(tint '[1@2001-01-01, 1@2001-01-15)');
-- {[2001-01-01, 2001-01-15)}
SELECT getTime(tfloat '{[1@2001-01-01, 1@2001-01-10), [12@2001-01-12, 12@2001-01-15]}');
-- {[2001-01-01, 2001-01-10), [2001-01-12, 2001-01-15]}
```

- Devuelve el período delimitador en el que se define el valor temporal

```
timeSpan(ttype) → tstzspan
```

```
SELECT timeSpan(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- [2001-01-01, 2001-01-03]
SELECT timeSpan(tint '[1@2001-01-01, 1@2001-01-15)');
-- [2001-01-01, 2001-01-15)
```

- Devuelve el valor inicial, final, o enésimo

```
startValue(ttype) → base
endValue(ttype) → base
valueN(ttype,integer) → base
```

La función no tiene en cuenta si los límites son inclusivos o no.

```
SELECT startValue(tfloat '(1@2001-01-01, 2@2001-01-03)');
-- 1
SELECT endValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 5
SELECT valueN(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}', 3);
-- 3
```

- Devuelve el valor en una marca de tiempo

```
valueAtTimestamp(ttype,timestamptz) → base
```

```
SELECT valueAtTimestamp(tfloat '[1@2001-01-01, 4@2001-01-04)', '2001-01-02');
-- 2
```

- Devuelve el intervalo de tiempo

```
duration(ttype,boundspan boolean=false) → interval
```

Se puede poner en verdadero un parámetro adicional para calcular la duración del período limitador, ignorando así los posibles intervalos de tiempo

```
SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}');
-- 00:00:00
SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}', true);
-- 4 days
SELECT duration(tfloat '[1@2001-01-01, 2@2001-01-03, 2@2001-01-05)');
-- 4 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05)}');
-- 3 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05)}',
  true);
-- 4 days
```

- ¿Es el instante inicial/final inclusivo?

```
lowerInc(ttype) → boolean
upperInc(ttype) → boolean
```

```
SELECT lowerInc(tint '[1@2001-01-01, 2@2001-01-02)');
-- true
SELECT upperInc(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- false
```

- Devuelve el número o los instantes diferentes

```
numInstants(ttype) → integer
instants(ttype) → ttypeInst[]
```

```
SELECT numInstants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- 3
SELECT instants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- {"1@2001-01-01","2@2001-01-02","3@2001-01-03"}
```

- Devuelve el instante inicial, final o enésimo

```
startInstant(ttype) → ttypeInst
endInstant(ttype) → ttypeInst
instantN(ttype,integer) → ttypeInst
```

Las funciones no tienen en cuenta si los límites son inclusivos o no.

```
SELECT startInstant(tfloat '{[1@2001-01-01, 2@2001-01-02),
  (2@2001-01-02, 3@2001-01-03)}');
-- 1@2001-01-01
SELECT endInstant(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- 3@2001-01-03
SELECT instantN(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}', 3);
-- 3@2001-01-03
```

- Devuelve el número o las marcas de tiempo diferentes

```
numTimestamps(ttype) → integer
timestamps(ttype) → timestamptz[]
```

```
SELECT numTimestamps(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 3
SELECT timestamps(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- {"2001-01-01", "2001-01-03", "2001-01-05"}
```

- Devuelve la marca de tiempo inicial, final o enésima

```
startTimestamp(ttype) → timestamptz
endTimestamp(ttype) → timestamptz
timestampN(ttype,integer) → timestamptz
```

Las funciones no tienen en cuenta si los límites son inclusivos o no.

```
SELECT startTimestamp(tfloat '[1@2001-01-01, 2@2001-01-03)');
-- 2001-01-01
SELECT endTimestamp(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 2001-01-05
SELECT timestampN(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}',
  3);
-- 2001-01-05
```

- Devuelve el número o las secuencias

```
numSequences({ttypeContSeq,ttypeSeqSet}) → integer
sequences({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq[]
```

```
SELECT numSequences(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 2
SELECT sequences(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- {"[1@2001-01-01, 2@2001-01-03)", "[3@2001-01-03, 5@2001-01-05)"}
```

- Devuelve la secuencia inicial, final, o enésima

```
startSequence({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq
endSequence({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq
sequenceN({ttypeContSeq,ttypeSeqSet},integer) → ttypeContSeq
```

```
SELECT startSequence(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- [1@2001-01-01, 2@2001-01-03)
SELECT endSequence(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- [3@2001-01-03, 5@2001-01-05)
SELECT sequenceN(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}',
  2);
-- [3@2001-01-03, 5@2001-01-05)
```

- Devuelve los segmentos

```
segments({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq[]
```

```
SELECT segments(tint '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
  (3@2001-01-03, 5@2001-01-05]}');
/* {"[1@2001-01-01, 1@2001-01-02)","[3@2001-01-02, 3@2001-01-03)","[2@2001-01-03]",
   "(3@2001-01-03, 3@2001-01-05)","[5@2001-01-05]"} */
SELECT segments(tfloat '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
  (3@2001-01-03, 5@2001-01-05]}');
/* {"[1@2001-01-01, 3@2001-01-02)","[3@2001-01-02, 2@2001-01-03]",
   "(3@2001-01-03, 5@2001-01-05]"} */
```

4.10. Transformaciones

Un valor temporal se puede transformar en otro subtipo. Se genera un error si los subtipos son incompatibles.

- Transforma un tipo temporal a otro subtipo

```
ttypeInst(ttype) → ttypeInst
ttypeSeq(ttype) → ttypeSeq
ttypeSeqSet(ttype) → ttypeSeqSet
```

```
SELECT tboolInst(tbool '{[true@2001-01-01]}');
-- t@2001-01-01
SELECT tboolInst(tbool '{[true@2001-01-01, true@2001-01-02]}');
-- ERROR: Cannot transform input value to a temporal instant
SELECT tintSeq(tint '1@2001-01-01');
-- [1@2001-01-01]
SELECT tfloatSeqSet(tfloat '2.5@2001-01-01');
-- {[2.5@2001-01-01]}
SELECT tfloatSeqSet(tfloat '{2.5@2001-01-01, 1.5@2001-01-02, 3.5@2001-01-03}');
-- {[2.5@2001-01-01],[1.5@2001-01-02],[3.5@2001-01-03]}
```

- Transforma un valor temporal a otra interpolación

```
setInterp(ttype,interp text) → ttype
```

```
SELECT setInterp(tbool 'true@2001-01-01','discrete');
-- {t@2001-01-01}
SELECT setInterp(tfloat '{[1@2001-01-01], [2@2001-01-02], [1@2001-01-03]}', 'discrete');
-- {1@2001-01-01, 2@2001-01-02, 1@2001-01-03}
SELECT setInterp(tfloat 'Interp=Step;
  [1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04]', 'linear');
/* {[1@2001-01-01, 1@2001-01-02), [2@2001-01-02, 2@2001-01-03),
   [1@2001-01-03, 1@2001-01-04), [2@2001-01-04]} */
SELECT asText(setInterp(tgeompoint 'Interp=Step;{[Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-02], [Point(3 3)@2001-01-05, Point(4 4)@2001-01-06]}', 'linear'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02), [POINT(2 2)@2001-01-02],
   [POINT(3 3)@2001-01-05, POINT(3 3)@2001-01-06), [POINT(4 4)@2001-01-06]} */
SELECT setInterp(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
  'linear');
-- ERROR:  The temporal type cannot have linear interpolation
```

- Devuelve un valor booleano temporal que indica si cada segmento de una secuencia (o conjunto de sequencias) temporal(es) continua tiene, respectivamente, al menos o como máximo una duración determinada

```
segmentMinDuration(ttype,interval,strict boolean=true) → tbool
segmentMaxDuration(ttype,interval,strict boolean=true) → tbool
```

Si el argumento `strict` no se especifica, se asume el valor `true` por defecto y, por lo tanto, la función verifica si la duración del segmento es estrictamente menor o mayor que el intervalo. Si el argumento es `false`, la función verifica si la duración del segmento es menor/mayor o *igual* que el intervalo.

```
SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day');
-- {[t@2001-01-01, f@2001-01-03, t@2001-01-04, t@2001-01-06]}
SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day', false);
-- {[t@2001-01-01, t@2001-01-06]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day');
-- {[f@2001-01-01, f@2001-01-06]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day', false);
-- {[f@2001-01-01, t@2001-01-03, f@2001-01-04, f@2001-01-06]}
```

- Desplaza y/o escalear el intervalo de tiempo de un valor temporal a uno o dos intervalos

```
shiftTime(ttype,interval) → ttype
scaleTime(ttype,interval) → ttype
shiftScaleTime(ttype,interval,interval) → ttype
```

Cuando se escalea, si el rango de valores o el intervalo de tiempo del valor temporal es cero (por ejemplo, para un instante temporal), el resultado es el valor temporal. Igualmente, el ancho o intervalo dado debe ser estrictamente mayor que cero.

```
SELECT shiftTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-02, 2@2001-01-04, 1@2001-01-06}
SELECT shiftTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-02, 2@2001-01-04]
SELECT asText(shiftTime(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-03],
  [Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]}', '1 day'));
/* {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-04],
   [POINT(2 2)@2001-01-05, POINT(1 1)@2001-01-06]} */
SELECT scaleTime(tint '1@2001-01-01', '1 day');
-- 1@2001-01-01
SELECT scaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-01, 2@2001-01-01 12:00:00, 1@2001-01-02}
SELECT scaleTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-01, 2@2001-01-02]
SELECT asText(scaleTime(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03], [Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]}', '1 day'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-01 06:00:00,
   POINT(1 1)@2001-01-01 12:00:00], [POINT(2 2) @2001-01-01 18:00:00,
   POINT(1 1)@2001-01-02]} */
SELECT scaleTime(tint '1@2001-01-01', '-1 day');
-- ERROR:  The interval must be positive: -1 days
SELECT shiftScaleTime(tint '1@2001-01-01', '1 day', '1 day');
-- 1@2001-01-02
SELECT shiftScaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day','1 day');
-- {1@2001-01-02, 2@2001-01-02 12:00:00, 1@2001-01-03}
SELECT shiftScaleTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day', '1 day');
-- [1@2001-01-02, 2@2001-01-03]
SELECT asText(shiftScaleTime(tgeometry '{[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03],
  [Point(2 2)@2001-01-04, Polygon((1 1,2 2,3 1,1 1))@2001-01-05]}', '1 day', '1 day'));
/* {[POINT(1 1)@2001-01-02 00:00:00, LINESTRING(1 1,2 2)@2001-01-02 06:00:00,
   POINT(1 1)@2001-01-02 12:00:00], [POINT(2 2)@2001-01-02 18:00:00,
   POLYGON((1 1,2 2,3 1,1 1))@2001-01-03 00:00:00]} */
```

- Transforma un valor temporal no lineal en un conjunto de filas, cada una es una pareja compuesta de un valor de base y un conjunto de períodos durante el cual el valor temporal tiene el valor de base {}

```
unnest(ttype) → {(value,time)}
```

```
SELECT (un).value, (un).time
FROM (SELECT unnest(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}') AS un) t;
-- 1 | {[2001-01-01, 2001-01-01], [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-02]}
SELECT (un).value, (un).time
FROM (SELECT unnest(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- 1 | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-03)}
SELECT (un).value, (un).time
FROM (SELECT unnest(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- ERROR:  Operation not supported for temporal values with linear interpolation
SELECT ST_AsText((un).value),
  (un).time FROM (SELECT unnest(tgeompoint 'Interp=Step;
  [Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
--  POINT(1 1) | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
--  POINT(2 2) | {[2001-01-02, 2001-01-03)}
SELECT ST_AsText((un).value),
  (un).time FROM (SELECT unnest(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
-- POINT(1 1)          | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
-- LINESTRING(1 1,2 2) | {[2001-01-02, 2001-01-03)}
```

Capítulo 5

Tipos temporales (Parte 2)

5.1. Modificaciones

A continuación, explicamos la semántica de las operaciones de modificación (es decir, `insert`, `update` y `delete`) para tipos temporales. Estas operaciones tienen una semántica similar a las operaciones correspondientes para tablas temporales de tiempo de aplicación (*application-time temporal tables*) introducidas en el estándar SQL:2011. La principal diferencia es que SQL usa marcas de tiempo de tuplas (donde las marcas de tiempo se adjuntan a las tuplas), mientras que los valores temporales en MobilityDB usan marcas de tiempo de atributos (donde las marcas de tiempo se adjuntan a los valores de los atributos). Please refer to this article for a detailed discussion about tuple versus attribute timestamping in temporal databases.

La operación `insert` agrega a un valor temporal los instantes de otro sin modificar los instantes existentes, como se ilustra en la Figura 5.1.

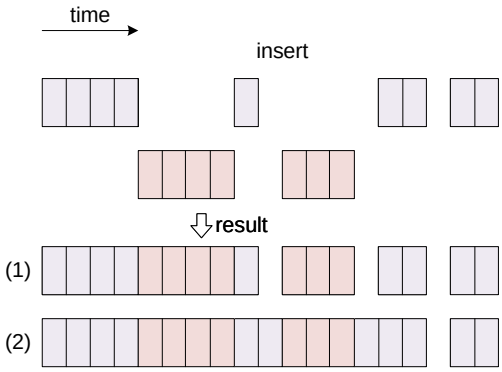


Figura 5.1: Operación de inserción para valores temporales.

Como se muestra en la figura, los valores temporales solo pueden intersectarse en su límite y, en ese caso, deben tener el mismo valor de base en sus marcas de tiempo comunes; de lo contrario, se genera un error. El resultado de la operación es la unión de los instantes para ambos valores temporales, como se muestra en el primer resultado de la figura. Esto es equivalente a una operación `merge` que se explica a continuación. Alternativamente, como se muestra en el segundo resultado de la figura, los fragmentos insertados que son disjuntos con el valor original se conectan al último instante anterior y al primer instante posterior al fragmento. Se utiliza un parámetro booleano `connect` para elegir entre los dos resultados, y el parámetro es verdadero por defecto. Nótese que esto solo se aplica a valores temporales continuos.

La operación `update` reemplaza los instantes en un primer valor temporal con los de un segundo valor como se ilustra en la Figura 5.2.

Como se muestra en la figura, el valor resultante contiene los instantes del segundo valor, independientemente de los instantes anteriores que tenía en el valor temporal original. Como en el caso de una operación `insert`, un parámetro booleano adicional determina si los nuevos fragmentos desconectados están conectados en el valor resultante, como se muestra en los dos posibles

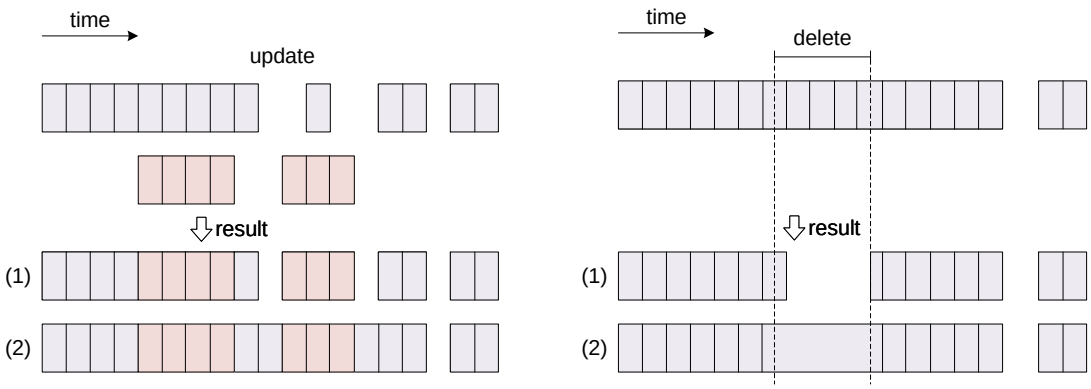


Figura 5.2: Operaciones de actualización y supresión para valores temporales.

resultados de la figura. Cuando los dos valores temporales son disjuntos o solo se intersectan en su límite, esto corresponde a una operación `insert` como se explicó anteriormente. En este caso, la operación `update` se comporta como una operación `upsert` en SQL.

La operación `deleteTime` elimina los instantes de un valor temporal que intersectan un valor de tiempo. Esta operación se puede utilizar en dos situaciones diferentes, ilustradas en la Figura 5.2.

1. En el primer caso, que se muestra como el resultado superior de la figura, el significado de la operación es introducir brechas de tiempo después de eliminar los instantes del valor temporal que intersectan el valor de tiempo. Esto es equivalente a las operaciones de restricción (Sección 5.2), que restringen un valor temporal al complemento del valor de tiempo.
2. El segundo caso, que se muestra como el resultado inferior de la figura, se usa para eliminar valores erróneos (por ejemplo, detectados como valores atípicos) sin introducir una brecha de tiempo, o para eliminar una brecha de tiempo. En este caso, los valores en el fragmento del valor temporal se eliminan y el último instante anterior y el primer instante posterior a une fragmento suprimido se conectan. Este comportamiento se especifica estableciendo un parámetro booleano adicional de la operación. Nótese que esto solo se aplica a valores temporales continuos.

La Figura 5.3 muestra las operaciones de modificación equivalentes para tablas temporales en el estándar SQL. Intuitivamente, estas figuras se obtienen girando 90 grados en el sentido de las agujas del reloj las figuras correspondientes para los valores temporales (Figura 5.1 y Figura 5.2). Esto se debe al hecho de que en SQL, las tuplas consecutivas ordenadas por tiempo generalmente se conectan a través de las funciones de ventana `LEAD` y `LAG`.

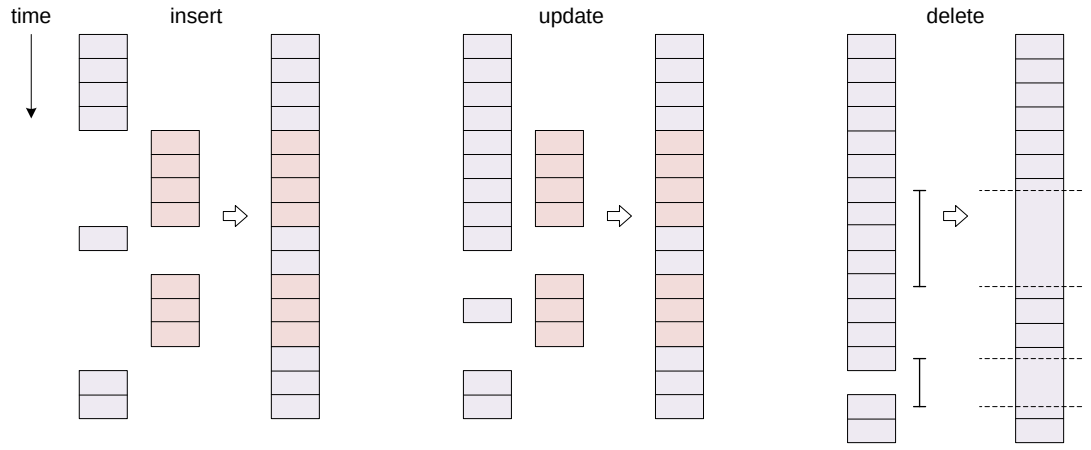


Figura 5.3: Operaciones de modificación para tablas temporales en SQL.

- Inserta un valor temporal en otro

```
insert(ttype,ttype,connect boolean=true) → ttype
```

```
SELECT insert(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{3@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 3@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT insert(tbigint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tbigint '{5@2001-01-03, 7@2001-01-07}');
-- ERROR: The temporal values have different value at their overlapping instant 2001-01-03
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-05]
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]', false);
-- {[1@2001-01-01, 2@2001-01-02], [1@2001-01-03, 1@2001-01-05]}
SELECT asText(insert(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04],[Point(1 1 1)@2001-01-05]}',
  tgeompoint 'Point(1 1 1)@2001-01-03'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (1 1 1)@2001-01-03,
   POINT Z (3 3 3)@2001-01-04], [POINT Z (1 1 1)@2001-01-05]} */
```

- Actualiza un valor temporal con otro

```
update(ttype,ttype,connect boolean=true) → ttype
```

```
SELECT update(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{5@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 5@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT update(tfloat '[1@2001-01-01, 1@2001-01-05]',
  tfloat '[1@2001-01-02, 3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 1@2001-01-02, 3@2001-01-03, 1@2001-01-04, 1@2001-01-05]}
SELECT asText(update(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(1 1 1)@2001-01-05], [Point(1 1 1)@2001-01-07]}',
  tgeompoint '[Point(2 2 2)@2001-01-02, Point(2 2 2)@2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (2 2 2)@2001-01-04,
   POINT Z (1 1 1)@2001-01-05], [POINT Z (1 1 1)@2001-01-07]} */
SELECT asText(update( tgeometry '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeometry '[Linestring(2 2,3 3)@2001-01-02, Linestring(3 3,2 2)@2001-01-04]'));
/* [POINT(1 1)@2001-01-01, LINESTRING(2 2,3 3)@2001-01-02,
   LINESTRING(3 3,2 2)@2001-01-04], (POINT(3 3)@2001-01-04, POINT(1 1)@2001-01-05] */
```

- Elimina de un valor temporal un valor de tiempo

```
deleteTime(ttype,times,connect boolean=true) → ttype
```

```
SELECT deleteTime(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-02', false);
-- {[1@2001-01-01, 1@2001-01-02), (1@2001-01-02, 1@2001-01-03]}
SELECT deleteTime(tbigint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-04');
-- [1@2001-01-01, 1@2001-01-03]
SELECT deleteTime(tfloat '[1@2001-01-01, 4@2001-01-02, 2@2001-01-04, 5@2001-01-05]',
  tstzspan '[2001-01-02, 2001-01-04]');
-- [1@2001-01-01, 5@2001-01-05]
SELECT asText(deleteTime(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tstzspan '[2001-01-02, 2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-04,
   POINT Z (3 3 3)@2001-01-05]} */
```

- Anexa un instante temporal a un valor temporal

```
appendInstant(ttype,ttypeInst,interp text=NULL) → ttype
appendInstant(ttypeInst,interp text=NULL,maxdist float=NULL,
  maxt interval=NULL) → ttypeSeq
```

La primera versión de la función devuelve el resultado de agregar el segundo argumento al primero. Si cualquiera de las entradas es NULL, se devuelve NULL. La segunda versión, la *agregación* `appendInstant`, devuelve el resultado de agregar sucesivamente un conjunto de filas de valores temporales. Esto significa que funciona de la misma manera que las funciones `SUM()` y `AVG()` y, como la mayoría de los agregados, también ignora los valores NULL. Dos argumentos opcionales establecen una distancia máxima y un intervalo de tiempo máximo de modo que se introduce una brecha de tiempo cada vez que dos instantes consecutivos tienen una distancia o un intervalo de tiempo mayor que estos valores. Para puntos temporales, la distancia se especifica en unidades del sistema de coordenadas. Si uno de estos argumentos no se dan, no se tiene en cuenta para determinar las brechas. La distancia máxima es un argumento de los tipos cuyos valores tienen una distancia: los números, puntos, geometrías y geografías temporales, los búferes circulares, los puntos de red, las poses y las geometrías rígidas; los demás tipos temporales toman solo la interpolación y el intervalo de tiempo máximo, como en `appendInstant(ttext, interp text, maxt interval)`. Sobre una sola fila la agregación devuelve una secuencia de un instante. DuckDB, Apache Spark y Apache Flink llaman a esta agregación `appendInstantAgg` (Sección 18.10.1).

```
SELECT appendInstant(tint '1@2001-01-01', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT appendInstant(tbigint '1@2001-01-01', tbigint '1@2001-01-02', 'discrete');
-- {1@2001-01-01, 1@2001-01-02}
SELECT appendInstant(tint '[1@2001-01-01]', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT asText(appendInstant(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint 'Point(1 1 1)@2001-01-06'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
   [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
   POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendInstant(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Linestring(2 2,3 3)@2001-01-04, Point(3 3)@2001-01-05]}',
  tgeometry 'Linestring(1 1,2 2)@2001-01-06'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(2 2,3 3)@2001-01-04,
   POINT(3 3)@2001-01-05, LINESTRING(1 1,2 2)@2001-01-06]} */
```

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst ORDER BY inst) FROM temp;
-- [1@2001-01-01, 5@2001-01-05]
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst, 'discrete' ORDER BY inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
WITH temp(inst) AS (
  SELECT tgeogpoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeogpoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeogpoint 'Point(3 3)@2001-01-03' UNION
  SELECT tgeogpoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeogpoint 'Point(5 5)@2001-01-05' )
SELECT asText(appendInstant(inst ORDER BY inst)) FROM temp;
/* [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03,
   POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05] */
```

Observe que en la primera consulta con `tfloat`, las observaciones intermedias fueron eliminadas por el proceso de normalización ya que eran redundantes debido a la interpolación lineal. Este no es el caso de la segunda consulta con interpolación discreta o la tercera consulta con `tgeogpoint`, donde se utilizan coordenadas geodésicas.

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '4@2001-01-04' UNION SELECT tfloat '5@2001-01-05' UNION
  SELECT tfloat '7@2001-01-07' )
SELECT appendInstant(inst, NULL, 0.0, '1 day' ORDER BY inst) FROM temp;
-- {[1@2001-01-01, 2@2001-01-02], [4@2001-01-04, 5@2001-01-05], [7@2001-01-07]}
WITH temp(inst) AS (
  SELECT tgeompoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeompoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeompoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeompoint 'Point(5 5)@2001-01-05' UNION
  SELECT tgeompoint 'Point(7 7)@2001-01-07' )
SELECT asText(appendInstant(inst, NULL, sqrt(2), '1 day' ORDER BY inst)) FROM temp;
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02],
   [POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05], [POINT(7 7)@2001-01-07]} */
```

- Anexa una secuencia temporal a un valor temporal

```
appendSequence(ttype,ttypeSeq) → {ttypeSeq,ttypeSeqSet}
appendSequence(ttypeSeq) → {ttypeSeq,ttypeSeqSet}
```

La primera versión de la función devuelve el resultado de agregar el segundo argumento al primero. Si cualquiera de las entradas es NULL, se devuelve NULL. La segunda versión, la *agregación* `appendSequence`, devuelve el resultado de agregar sucesivamente un conjunto de filas de valores temporales. Esto significa que funciona de la misma manera que las funciones `SUM()` y `AVG()` y, como la mayoría de los agregados, también ignora los valores NULL. El nombre `appendSequence` designa la misma función de agregación y se mantiene por compatibilidad con versiones anteriores.

```
SELECT appendSequence(tint '1@2001-01-01', tint '{2@2001-01-02, 3@2001-01-03}');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT appendSequence(tbigint '[1@2001-01-01, 2@2001-01-02]',
  tbigint '[2@2001-01-02, 3@2001-01-03]');
-- [1@2001-01-01, 2@2001-01-02, 3@2001-01-03]
SELECT asText(appendSequence(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint '[Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
   [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
   POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendSequence(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Linestring(3 3,2 2)@2001-01-04, Point(3 3)@2001-01-05]}',
  tgeometry '[Point(3 3)@2001-01-05, Linestring(3 3,1 1)@2001-01-06]'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(3 3,2 2)@2001-01-04,
   POINT(3 3)@2001-01-05, LINESTRING(3 3,1 1)@2001-01-06]} */
```

- Fusiona los valores temporales

```
merge(ttype,ttype) → ttype
merge(ttype[]) → ttype
merge(ttype) → ttype
```

Los valores temporales solo pueden intersectar en su límite. En ese caso, para todos los tipos temporales excepto `tgeometry` y `tgeography`, sus valores de base en las marcas de tiempo comunes deben ser los mismos, de lo contrario se genera un error. Para los tipos `tgeometry` y `tgeography`, si el valor en sus marcas de tiempo comunes es diferente, el valor resultante después de la `merge` será la unión espacial de los valores. La razón de un comportamiento diferente para estos tipos es que son los únicos tipos temporales que no son tipos *escalares*, es decir, su valor en una marca de tiempo dada no es un único elemento del dominio del tipo base, sino más bien un subconjunto de su dominio. Un operando NULL de la función sobre dos valores se ignora, de modo que `merge(v, NULL)` devuelve `v`. La función `merge` es una función de *agregación* que devuelve el resultado de agregar sucesivamente un conjunto de filas de valores temporales. Esto significa que funciona de la misma manera que las funciones `SUM()` y `AVG()` y, como la mayoría de los agregados, también ignora los valores NULL. El nombre `merge` designa la misma función de agregación y se mantiene por compatibilidad con versiones anteriores.

```
SELECT merge(tint '1@2001-01-01', tint '1@2001-01-02');
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(tbigint '[1@2001-01-01, 2@2001-01-02]',
  tbigint '[2@2001-01-02, 1@2001-01-03]');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 1@2001-01-04]}
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[1@2001-01-02, 2@2001-01-03]');
-- ERROR:  The temporal values have different value at their common timestamp 2001-01-02
SELECT asText(merge(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint '{[Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06]}'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
   [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
   POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(merge(tgeometry 'Linestring(1 1,5 1)@2001-01-01',
  tgeometry '{Linestring(5 1,10 1)@2001-01-01, Point(3 3)@2001-01-02}'));
-- {LINESTRING(1 1,5 1,10 1)@2001-01-01, POINT(3 3)@2001-01-02}
```

```
SELECT merge(ARRAY[tint '1@2001-01-01', '1@2001-01-02']);
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{2@2001-01-02, 3@2001-01-03}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{3@2001-01-03, 4@2001-01-04}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04}
SELECT merge(ARRAY[tbigint '[1@2001-01-01, 2@2001-01-02]',
  '[2@2001-01-02, 1@2001-01-03]']);
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(ARRAY[tbigint '[1@2001-01-01, 2@2001-01-02]',
  '[3@2001-01-03, 4@2001-01-04]']);
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 4@2001-01-04]}
SELECT asText(merge(ARRAY[tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(3 3)@2001-01-03, Point(4 4)@2001-01-04]}', '{[Point(4 4)@2001-01-04,
  Point(3 3)@2001-01-05], [Point(6 6)@2001-01-06, Point(7 7)@2001-01-07]}']));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [POINT(3 3)@2001-01-03,
   POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-05],
   [POINT(6 6)@2001-01-06, POINT(7 7)@2001-01-07]} */
SELECT asText(merge(ARRAY[tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]}',
  '{[Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(1 1)@2001-01-03]
SELECT asText(merge(ARRAY[tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]}',
  '{[Linestring(2 2,1 1)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- {[POINT(1 1)@2001-01-01, LINESTRING(2 2,1 1)@2001-01-02, POINT(1 1)@2001-01-03]}
```

```
WITH temp(inst) AS (
SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
SELECT tfloat '5@2001-01-05' ) SELECT merge(inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
```

5.2. Restricciones

Hay dos conjuntos complementarios de funciones de restricción. El primer conjunto de funciones restringe el valor temporal con respecto a un valor o una extensión de tiempo. Dos ejemplos son `atValues` o `atTime`. El segundo conjunto de funciones restringe el valor temporal con respecto al *complement* de un valor o una extensión de tiempo. Dos ejemplos son `minusValues` o `minusTime`

- Restringe a (al complemento de) un valor o un conjunto de valores o, para un número temporal, un span o un conjunto de spans de valores

```
atValue(ttype,base) → ttype
minusValue(ttype,base) → ttype
atValues(ttype,values) → ttype
minusValues(ttype,values) → ttype
atSpan(tnumber,span) → tnumber
minusSpan(tnumber,span) → tnumber
atSpanset(tnumber,spanset) → tnumber
minusSpanset(tnumber,spanset) → tnumber
```

```
SELECT atValue(tint '[1@2001-01-01, 1@2001-01-15)', 1);
-- [1@2001-01-01, 1@2001-01-15)
SELECT atValues(tfloat '[1@2001-01-01, 4@2001-01-4)', floatset '{1, 3, 5}');
-- {[1@2001-01-01], [3@2001-01-03]}
SELECT atSpan(tfloat '[1@2001-01-01, 4@2001-01-4)', floatspan '[1,3]');
-- [1@2001-01-01, 3@2001-01-03]
SELECT atSpanset(tfloat '[1@2001-01-01, 5@2001-01-05)', floatspanset '{[1,2], [3,4]}');
-- {[1@2001-01-01, 2@2001-01-02],[3@2001-01-03, 4@2001-01-04]}
SELECT asText(atValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03)',
  geometry 'Point(1 1 1)'));
-- {[POINT Z (1 1 1)@2001-01-02]}
SELECT asText(atValues(tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03)',
  geomset '{"Point(0 0)", "Point(1 1)"}'));
-- {[POINT(0 0)@2001-01-01], [POINT(1 1)@2001-01-02]}
SELECT asText(atValue(tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,2 2)@2001-01-02,
  Linestring(0 0,2 2)@2001-01-03]', geometry 'Linestring(0 0,2 2)'));
-- {[LINESTRING(0 0,2 2)@2001-01-02, LINESTRING(0 0,2 2)@2001-01-03]}
```

```
SELECT minusValue(tint '[1@2001-01-01, 2@2001-01-02, 2@2001-01-03)', 1);
-- {[2@2001-01-02, 2@2001-01-03)}
SELECT minusValues(tfloat '[1@2001-01-01, 4@2001-01-4)', floatset '{2, 3}');
/* {[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03),
   (3@2001-01-03, 4@2001-01-04)} */
SELECT minusSpan(tfloat '[1@2001-01-01, 4@2001-01-4)', floatspan '[2,3]');
-- {[1@2001-01-01, 2@2001-01-02), (3@2001-01-03, 4@2001-01-04)}
SELECT minusSpanset(tfloat '[1@2001-01-01, 5@2001-01-05)', floatspanset '{[1,2], [3,4]}');
-- {(2@2001-01-02, 3@2001-01-03), (4@2001-01-04, 5@2001-01-05)}
SELECT asText(minusValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03)',
  geometry 'Point(1 1 1)'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
   (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03)} */
SELECT asText(minusValues(tgeompoint '[Point(0 0 0)@2001-01-01, Point(3 3 3)@2001-01-04)',
  geomset '{"Point(1 1 1)', 'Point(2 2 2)"}'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
   (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03),
   (POINT Z (2 2 2)@2001-01-03, POINT Z (3 3 3)@2001-01-04)} */
SELECT asText(minusValue(tgeometry '[Point(0 0)@2001-01-01,
  Linestring(0 0,2 2)@2001-01-02, Linestring(0 0,2 2)@2001-01-03]',
  geometry 'Linestring(0 0,2 2)'));
--  {[POINT(0 0)@2001-01-01, POINT(0 0)@2001-01-02)}
```

- Restringe a (al complemento de) un valor de tiempo

```
atTime(ttype,times) → ttype
minusTime(ttype,times) → ttype
```

```
SELECT atTime(tfloat '[1@2001-01-01, 5@2001-01-05)', timestamptz '2001-01-02');
-- 2@2001-01-02
SELECT atTime(tbigint '[1@2001-01-01, 1@2001-01-15)', tstzset '{2001-01-01, 2001-01-03}');
```

```
-- {1@2001-01-01, 1@2001-01-03}
SELECT atTime(tfloat '{[1@2001-01-01, 3@2001-01-03), [3@2001-01-04, 1@2001-01-06)}',
  tstzspan '[2001-01-02,2001-01-05)');
-- {[2@2001-01-02, 3@2001-01-03), [3@2001-01-04, 2@2001-01-05)}
SELECT atTime(tint '[1@2001-01-01, 1@2001-01-15)',
  tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}');
-- {[1@2001-01-01, 1@2001-01-03),[1@2001-01-04, 1@2001-01-05)}
```

```
SELECT minusTime(tfloat '[1@2001-01-01, 5@2001-01-05)', timestamptz '2001-01-02');
-- {[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 5@2001-01-05)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15)', tstzset '{2001-01-02, 2001-01-03}');
/* {[1@2001-01-01, 1@2001-01-02), (1@2001-01-02, 1@2001-01-03),
    (1@2001-01-03, 1@2001-01-15)} */
SELECT minusTime(tfloat '{[1@2001-01-01, 3@2001-01-03), [3@2001-01-04, 1@2001-01-06)}',
  tstzspan '[2001-01-02,2001-01-05)');
-- {[1@2001-01-01, 2@2001-01-02), [2@2001-01-05, 1@2001-01-06)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15)',
  tstzspanset '{[2001-01-02, 2001-01-03), [2001-01-04, 2001-01-05)}');
/* {[1@2001-01-01, 1@2001-01-02), [1@2001-01-03, 1@2001-01-04),
    [1@2001-01-05, 1@2001-01-15)} */
```

- Mantiene los instantes de un valor temporal que son anteriores o posteriores a una marca de tiempo, o iguales a ella

```
beforeTimestamp(ttype,timestamptz,strict boolean=true) → ttype
afterTimestamp(ttype,timestamptz,strict boolean=true) → ttype
```

```
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-02');
-- [1@2001-01-01, 1@2001-01-02)
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-01');
-- NULL
SELECT beforeTimestamp(tbigint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-01',
  false);
-- [1@2001-01-01]
SELECT afterTimestamp(tfloat '[1@2001-01-01, 3@2001-01-03]', timestamptz '2001-01-02',
  false);
-- [2@2001-01-02, 3@2001-01-03]
SELECT asText(afterTimestamp(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  timestamptz '2001-01-03'));
-- {[POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05]}
```

5.3. Operadores de cuadro delimitador

Estos operadores determinan si los cuadros delimitadores de sus argumentos satisfacen el predicado y dan como resultado un valor booleano. Como se indica en el Capítulo 4, el cuadro delimitador asociado a un tipo temporal depende del tipo base: es el tipo `tstzspan` para los tipos `tbool` y `ttext`, el tipo `tbox` para los tipos `tint` y `tfloat` y el tipo `stbox` para los tipos `tgeompoint` y `tgeogpoint`. Además, como se dijo en la Sección 3.3, muchos tipos PostgreSQL, PostGIS o MobilityDB se pueden convertir a los tipos `tbox` y `stbox`. Por ejemplo, los tipos numéricos y los rangos se pueden convertir al tipo `tbox`, los tipos `geometry` y `geography` se pueden convertir al tipo `stbox` y los tipos de tiempo y los tipos temporales se pueden convertir a los tipos `tbox` y `stbox`.

Un primer conjunto de operadores considera las relaciones topológicas entre los cuadros delimitadores. Hay cinco operadores topológicos: superposición (`&&`), contiene (`@>`), está contenido (`<@`), mismo (`~=`) y adyacente (`-|-`). Los argumentos de estos operadores pueden ser un tipo base, una cuadro delimitador o un tipo temporal y los operadores verifican la relación topológica teniendo en cuenta el valor y/o la dimensión temporal según el tipo de los argumentos.

Otro conjunto de operadores considera la posición relativa de los cuadros delimitadores. Los operadores `<<`, `>>`, `&<` y `&>` consideran la dimensión de valor para los tipos `tint` y `tfloat` y las coordenadas X para los tipos `tgeompoint` y `tgeogpoint`,

los operadores `<<|`, `|>>`, `&<|` y `|&>` consideran las coordenadas Y para los tipos `tgeompoint` y `tgeogpoint`, los operadores `<</`, `/>>`, `&</` y `/&>` consideran las coordenadas Z para los tipos `tgeompoint` y `tgeogpoint` y los operadores `<<#`, `#>>`, `#&<` y `#&>` consideran la dimensión tiempo para todos los tipos temporales.

Finalmente, cabe destacar que los operadores de cuadro delimitador leen las dimensiones que ambos operandos tienen, véase Sección 8.3.

Refiérase a la Sección 3.9 para los operadores de cuadro delimitador.

5.4. Comparaciones

5.4.1. Comparaciones tradicionales

Los operadores de comparación tradicionales (=, <, etc.) requieren que los operandos izquierdo y derecho sean del mismo tipo base. Excepto la igualdad y la no igualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan sobre tipos temporales. Estos operadores comparan los períodos delimitadores (ver la Sección 2.10), después los cuadros delimitadores (ver la Sección 3.10) y si son iguales, entonces la comparación depende del subtipo. Para los valores de instante, primero comparan las marcas de tiempo y, si son iguales, comparan los valores. Para los valores de secuencia, comparan los primeros N instantes, donde N es el mínimo del número de instantes que componen ambos valores. Finalmente, para los valores de conjuntos de secuencias, comparan los primeros N valores de secuencia, donde N es el mínimo del número de secuencias que componen ambos valores.

Los operadores de igualdad y no igualdad consideran la representación equivalente para diferentes subtipos como se muestra a continuación.

```
SELECT tint '1@2001-01-01' = tint '{1@2001-01-01}';
-- true
SELECT tfloat '1.5@2001-01-01' = tfloat '[1.5@2001-01-01]';
-- true
SELECT ttext 'AAA@2001-01-01' = ttext '{[AAA@2001-01-01]}';
-- true
SELECT tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}' =
  tgeompoint '{[Point(1 1)@2001-01-01], [Point(2 2)@2001-01-02]}';
-- true
SELECT tgeography '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]' =
  tgeography '{[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]}';
-- true
```

- Comparaciones tradicionales

```
ttype {=, <>, <, >, <=, >=} ttype → boolean
cmp(ttype,ttype) → integer
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-04)' = tint '[2@2001-01-03, 2@2001-01-05)';
-- false
SELECT tint '[1@2001-01-01, 1@2001-01-04)' <> tint '[2@2001-01-03, 2@2001-01-05)';
-- true
SELECT tint '[1@2001-01-01, 1@2001-01-04)' < tint '[2@2001-01-03, 2@2001-01-05)';
-- true
SELECT ttfloat '[1@2001-01-01, 1@2001-01-04)' > ttfloat '[2@2001-01-03, 2@2001-01-05)';
-- false
SELECT tbigint '[1@2001-01-01, 1@2001-01-04)' <= tbigint '[2@2001-01-03, 2@2001-01-05)';
-- true
SELECT tbigint '[1@2001-01-01, 1@2001-01-04)' >= tbigint '[2@2001-01-03, 2@2001-01-05)';
-- false
SELECT cmp(tfloat '[1@2001-01-01, 1@2001-01-04)', '[2@2001-01-03, 2@2001-01-05)');
-- -1
```

- Valores hash

```
hash(ttype) → integer
hashExtended(ttype,seed bigint) → bigint
```

La función `hash` devuelve un valor hash de 32 bits y la función `hashExtended` un valor hash de 64 bits calculado con la semilla dada. Los valores iguales tienen valores hash iguales, que es en lo que se basan los índices hash, las uniones hash y la agregación hash; esto se cumple entre subtipos, de modo que un instante tiene el valor hash de una secuencia o de un conjunto de secuencias que contiene solo ese instante.

```
SELECT hash(tint '1@2001-01-01') = hash(tint '{1@2001-01-01}');
-- true
SELECT hash(tfloat '1.5@2001-01-01') = hash(tfloat '{[1.5@2001-01-01]}');
-- true
SELECT hash(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}') =
  hash(tgeompoint '{[Point(1 1)@2001-01-01], [Point(2 2)@2001-01-02]}');
-- true
SELECT hashExtended(tint '[1@2001-01-01, 2@2001-01-02]', 0) <>
  hashExtended(tint '[1@2001-01-01, 2@2001-01-02]', 1);
-- true
```

5.4.2. Comparaciones alguna vez y siempre

Una posible generalización de los operadores de comparación tradicionales (=, <>, <, <=, etc.) a tipos temporales consiste en determinar si la comparación es alguna vez o siempre verdadera. En este caso, el resultado es un valor booleano. MobilityDB proporciona operadores para probar si la comparación de un valor temporal y un valor del tipo base or dos valores temporales es alguna vez o siempre verdadera. Estos operadores se indican anteponiendo los operadores de comparación tradicionales con, respectivamente, `?` (alguna vez) y `%` (siempre). Algunos ejemplos son `?=`, `%<>` o `?<=`. La igualdad y la no igualdad alguna vez o siempre están disponibles para todos los tipos temporales, mientras que las desigualdades alguna vez o siempre sólo están disponibles para los tipos temporales cuyo tipo base tiene un orden total definido, es decir, `tint`, `tfloat` o `ttext`. Las comparaciones alguna vez y siempre son operadores inversos: por ejemplo, `?=` es el inverso de `%<>` y `?>` es el inverso de `%<=`.

- Comparaciones alguna vez y siempre

```
{base,ttype} {?=, ?<>, ?<, ?>, ?<=, ?>=} {base,ttype} → boolean
{base,ttype} {%=, %<>, %<, %>, %<=, %>=} {base,ttype} → boolean
```

Los operadores no tienen en cuenta si los límites son inclusivos o no.

```
SELECT tint '[1@2001-01-01, 3@2001-01-04]' ?= 2;
-- false
SELECT tgeometry '[Point(0 0)@2001-01-01,
  Linestring(0 0,1 1)@2001-01-04]' ?= geometry 'Linestring(1 1,0 0)';
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-04)' %= 1;
-- true
SELECT tgeometry '[Linestring(0 0,1 1)@2001-01-01,
  Linestring(1 1,0 0)@2001-01-04)' %= geometry 'Linestring(0 0,1 1)';
-- true
```

```
SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' ?<> 2;
-- true
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' ?<>
  geometry 'Point(1 1)';
-- false
SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' %<> 2;
-- false
SELECT tgeogpoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' %<>
  geography 'Point(2 2)';
-- true
```

```
SELECT tint '[1@2001-01-01, 4@2001-01-04]' ?< 2;
-- true
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' %< 2;
-- false
```

```
SELECT tint '[1@2001-01-03, 1@2001-01-05]' ?> 0;
-- true
SELECT tfloat '[1@2001-01-03, 1@2001-01-05)' %> 1;
-- false
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-05]' ?<= 2;
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-05)' %<= 4;
-- true
```

```
SELECT ttext '{[AAA@2001-01-01, AAA@2001-01-03),
  [BBB@2001-01-04, BBB@2001-01-05)}' ?> 'AAA'::text;
-- true
SELECT ttext '{[AAA@2001-01-01, AAA@2001-01-03),
  [BBB@2001-01-04, BBB@2001-01-05)}' %> 'AAA'::text;
-- false
```

5.4.3. Comparaciones temporales

Otra posible generalización de los operadores de comparación tradicionales (=, <>, <, <=, etc.) a tipos temporales consiste en determinar si la comparación es verdadera o falsa en cada instante. En este caso, el resultado es un booleano temporal. Los operadores de comparación temporal se indican anteponiendo los operadores de comparación tradicionales con #. Algunos ejemplos son #= o #<=. La igualdad y no igualdad temporal están disponibles para todos los tipos temporales, mientras que las desigualdades temporales sólo están disponibles para los tipos temporales cuyo tipo base tiene un orden total definido, es decir, `tint`, `tfloat` o `ttext`.

- Comparaciones temporales

```
{base,ttype} {#=, #<>, #<, #>, #<=, #>=} {base,ttype} → boolean
```

```
SELECT tfloat '[1@2001-01-01, 2@2001-01-04)' #= 3;
-- {[f@2001-01-01, f@2001-01-04)}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #= tfloat '[4@2001-01-02, 1@2001-01-05)';
-- {[f@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04)}
SELECT tgeompoint '[Point(0 0)@2001-01-01,
  Point(2 2)@2001-01-03)' #= geometry 'Point(1 1)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tgeometry '[Point(0 0)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-03]' #= geometry 'Linestring(2 2,1 1)';
-- {[f@2001-01-01, t@2001-01-03]}
```

```
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #<> 2;
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, 2001-01-04)}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #<> tfloat '[2@2001-01-02, 2@2001-01-05)';
-- {[f@2001-01-02], (t@2001-01-02, t@2001-01-04)}
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(1 1,2 2)@2001-01-03]' #<>
  geometry 'Linestring(2 2,1 1)';
-- {[t@2001-01-01, f@2001-01-03]}
```

```
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #< 2;
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-04)}
SELECT 1 #> tint '[1@2001-01-03, 1@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
SELECT tfloat '[1@2001-01-01, 1@2001-01-05)' #<= tfloat '{2@2001-01-03, 3@2001-01-04}';
-- {t@2001-01-03, t@2001-01-04}
SELECT 'AAA'::text #> ttext '{[AAA@2001-01-01, AAA@2001-01-03),
  [BBB@2001-01-04, BBB@2001-01-05)}';
-- {[f@2001-01-01, f@2001-01-03), [t@2001-01-04, t@2001-01-05)}
```

5.5. Funciones de utilidad

- Versión de la extensión MobilityDB

```
mobilitydbVersion() → text
```

```
SELECT mobilitydbVersion();
-- MobilityDB 1.3.0
```

- Versión de la extensión MobilityDB y de sus dependencias

```
mobilitydbFullVersion() → text
```

```
SELECT mobilitydbFullVersion();
/* MobilityDB 1.3.0, PostgreSQL 17.2, PostGIS 3.5.2, PROJ 9.2.0, JSON-C 0.13.1 */
```

Capítulo 6

Tipos alfanuméricos temporales

6.1. Notación

En Sección 4.5 presentamos la notación utilizada para definir la firma de las funciones y operadores para tipos temporales. A continuación, ampliamos estas notaciones para tipos alfanuméricos temporales.

- `torder` representa cualquier tipo temporal cuyo tipo de base tiene definido un orden total, es decir, `tint`, `tbigint`, `tfloat` o `ttext`,
- `talpha` representa cualquier tipo temporal alfanumérico, por ejemplo, `tint` o `ttext`,
- `tnumber` representa cualquier tipo de número temporal, es decir, `tint`, `tbigint` o `tfloat`,
- `number` representa cualquier tipo de base numérico, es decir, `integer`, `bigint` o `float`,
- `numspan` representa cualquier tipo de rango numérico, es decir, `intspan`, `bigintspan` o `floatspan`,

6.2. Conversión de tipos

Un valor temporal se puede convertir en un tipo compatible utilizando la notación `CAST(ttype1 AS ttype2)` o la notación `ttype1::ttype2`.

- Convierte un número temporal en un rango o un cuadro delimitador temporal

```
tnumber::{span,tbox}
valueSpan(tnumber) → numspan
timeSpan(tnumber) → tstzspan
tbox(tnumber) → tbox
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::intspan;
-- [1, 3)
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]'::floatspan;
-- (1, 3]
SELECT valueSpan(tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]');
-- (1, 3]
SELECT timeSpan(tfloat 'Interp=Step;(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]');
-- (2001-01-01, 2001-01-05]
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::tbox;
-- TBOXINT XT((1,2),[2001-01-01,2001-01-03])
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]'::tbox;
-- TBOXFLOAT XT((1,3),[2001-01-01,2001-01-05])
```

- Convierte entre un booleano temporal y un entero temporal

```
tbool::tint
tint(tbool) → tint
```

```
SELECT tbool '[true@2001-01-01, false@2001-01-03]'::tint;
-- [1@2001-01-01, 0@2001-01-03]
```

- Convierte entre enteros temporales, enteros grandes temporales y flotantes temporales

```
tnumber::tnumber
tnumber(tnumber) → tnumber
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::tfloat;
-- Interp=Step;[1@2001-01-01, 2@2001-01-03]
SELECT tbigint '[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]'::tfloat;
-- Interp=Step;[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]
SELECT tfloat 'Interp=Step;[1.5@2001-01-01, 2.5@2001-01-03]'::tint;
-- [1@2001-01-01, 2@2001-01-03]
SELECT tint(tfloat '[1.5@2001-01-01, 2.5@2001-01-03]');
-- ERROR:  Cannot cast temporal float with linear interpolation to temporal integer
```

6.3. Accessores

- Devuelve el intervalo de valores ignorando las brechas potenciales

```
valueSpan(tnumber) → numspan
```

```
SELECT valueSpan(tint '{1@2001-01-01, 5@2001-01-02, 3@2001-01-03}');
-- [1, 6)
SELECT valueSpan(tfloat '[1.5@2001-01-01, 4.5@2001-01-03]');
-- [1.5, 4.5]
```

- Devuelve el conjunto de valores de un número temporal

```
valueSet(tnumber) → numset
```

```
SELECT valueSet(tint '[1@2001-01-01, 2@2001-01-03]');
-- {1, 2}
SELECT valueSet(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 4@2001-01-05)}');
-- {1, 2, 3, 4}
```

- Devuelve el valor mínimo, máximo, o promedio

```
minValue(torder) → base
maxValue(torder) → base
avgValue(tnumber) → base
```

Las funciones no tienen en cuenta si los límites son inclusivos o no.

```
SELECT minValue(tfloat '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- 1
SELECT maxValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 5
SELECT avgValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 2.75
```

```
SELECT minValue(ttext '{[A@2001-01-01, B@2001-01-02], [C@2001-01-03, E@2001-01-05]}');
-- A
SELECT maxValue(ttext '{[A@2001-01-01, B@2001-01-02], [C@2001-01-03, E@2001-01-05]}');
-- E
```

- Devuelve el instante con el valor mínimo o máximo

```
minInstant(torder) → base
maxInstant(torder) → base
```

Las funciones no tienen en cuenta si los límites son inclusivos o no. Si varios instantes tienen el valor mínimo, se devuelve el primero.

```
SELECT minInstant(tfloat '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- 1@2001-01-01
SELECT maxInstant(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 5@2001-01-05
```

6.4. Transformaciones

- Desplaza y/o escalear el rango de valores de un número temporal con uno o dos números

```
shiftValue(tnumber,base) → tnumber
scaleValue(tnumber,base) → tnumber
shiftScaleValue(tnumber,base,base) → tnumber
```

Cuando se escalea, si el rango de valores del valor temporal es un valor único (por ejemplo, para un instante temporal), el resultado es el valor temporal. Además, el ancho data debe ser estrictamente superio que cero.

```
SELECT shiftValue(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', 1);
-- {2@2001-01-02, 3@2001-01-04, 2@2001-01-06}
SELECT shiftValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}', -1);
--  {[0@2001-01-01, 1@2001-01-02], [2@2001-01-03, 3@2001-01-04]}
SELECT scaleValue(tint '1@2001-01-01', 1);
-- 1@2001-01-01
SELECT scaleValue(tfloat '{[1@2001-01-01,2@2001-01-02], [3@2001-01-03,4@2001-01-04]}', 6);
-- {[1@2001-01-01, 3@2001-01-03], [5@2001-01-05, 7@2001-01-07]}
SELECT scaleValue(tint '1@2001-01-01', -1);
-- ERROR:  The value must be strictly positive: -1
SELECT shiftScaleValue(tint '1@2001-01-01', 1, 1);
-- 2@2001-01-01
SELECT shiftScaleValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}',
  -1, 6);
--  {[0@2001-01-01, 2@2001-01-02], [4@2001-01-03, 6@2001-01-04]}
```

- Extrae de un flotante temporal con interpolación lineal las subsecuencias donde los valores permanecen en un rango de un ancho dado durante al menos una duración dada

```
stops(tfloat,maxdist float=0.0,minduration interval='0 minutes') → tfloat
```

Si `maxDist` no se especifica, el valor 0.0 se asume por defecto y por lo tanto, la función extrae los segmentos constantes del flotante temporal. Una parada que comienza en el primer instante de una secuencia incluye ese instante solo si la secuencia lo incluye, y lo mismo vale para una parada que termina en su último instante. Las paradas se buscan en orden temporal: cada una comienza en el primer instante a partir del cual el valor permanece dentro de la distancia máxima durante la duración mínima, y termina en el último instante antes de salir de ella.

```
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]');
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]', 0.0, '2 days');
-- NULL
```

6.5. Restrictions

- Restringe al (complemento del) valor mínimo o máximo

```
atMin(torder) → torder
atMax(torder) → torder
minusMin(torder) → torder
minusMax(torder) → torder
```

La función devuelve un valor nulo si el valor mínimo o máximo sólo ocurre en límites exclusivos.

```
SELECT atMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {1@2001-01-01, 1@2001-01-05}
SELECT atMin(tint '(1@2001-01-01, 3@2001-01-03]');
-- {(1@2001-01-01, 1@2001-01-03)}
SELECT atMin(tfloat '(1@2001-01-01, 3@2001-01-03]');
-- NULL
SELECT atMin(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05]}');
-- {(AA@2001-01-01, AA@2001-01-03), [AA@2001-01-05]}
SELECT atMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {3@2001-01-05}
SELECT atMax(tfloat '(1@2001-01-01, 3@2001-01-03)');
-- NULL
SELECT atMax(tfloat '{(2@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)}');
-- {[2@2001-01-03, 2@2001-01-05]}
SELECT atMax(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05]}');
-- {("BB"@2001-01-03, "BB"@2001-01-05)}
```

```
SELECT minusMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {2@2001-01-03}
SELECT minusMin(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {(1@2001-01-01, 3@2001-01-03]}
SELECT minusMin(tfloat '(1@2001-01-01, 3@2001-01-03)');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMin(tint '{[1@2001-01-01, 1@2001-01-03), (1@2001-01-03, 1@2001-01-05)}');
-- NULL
SELECT minusMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {1@2001-01-01, 2@2001-01-03}
SELECT minusMax(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {[1@2001-01-01, 3@2001-01-03)}
SELECT minusMax(tfloat '(1@2001-01-01, 3@2001-01-03)');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMax(tfloat '{[2@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)}');
-- {(2@2001-01-01, 1@2001-01-03)}
SELECT minusMax(tfloat '{[1@2001-01-01, 3@2001-01-03), (3@2001-01-03, 1@2001-01-05)}');
-- {[1@2001-01-01, 3@2001-01-03), (3@2001-01-03, 1@2001-01-05)}
```

- Restringe a (al complemento de) un `tbox`

```
atTbox(tnumber,tbox) → tnumber
minusTbox(tnumber,tbox) → tnumber
```

When the bounding box has both value and time dimensions, the functions restrict the temporal number with respect to the value *and* the time extents of the box.

```
SELECT atTbox(tfloat '[0@2001-01-01, 3@2001-01-04)',
  tbox 'TBOXFLOAT XT((0,2),[2001-01-02, 2001-01-04])');
-- {[1@2001-01-02, 2@2001-01-03]}
```

```
SELECT minusTbox(tfloat '[1@2001-01-01, 4@2001-01-04)',
  'TBOXFLOAT XT((1,4),[2001-01-03, 2001-01-04])');
-- {[1@2001-01-01, 3@2001-01-03)}
WITH temp(temp, box) AS (
  SELECT tfloat '[1@2001-01-01, 4@2001-01-04)',
    tbox 'TBOXFLOAT XT((1,2),[2001-01-03, 2001-01-04])' )
SELECT minusSpan(minusTime(temp, box::tstzspan), box::floatspan) FROM temp;
-- {[1@2001-01-01], [2@2001-01-02, 3@2001-01-03)}
```

6.6. Operaciones booleanas

- Y temporal, o temporal

```
{boolean,tbool} {&, |} {boolean,tbool} → tbool
```

```
SELECT tbool '[true@2001-01-03, true@2001-01-05)' &
  tbool '[false@2001-01-03, false@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
SELECT tbool '[true@2001-01-03, true@2001-01-05)' &
  tbool '{[false@2001-01-03, false@2001-01-04), [true@2001-01-04, true@2001-01-05)}';
-- {[f@2001-01-03, t@2001-01-04, t@2001-01-05)}
SELECT tbool '[true@2001-01-03, true@2001-01-05)' | tbool '[false@2001-01-03,
  false@2001-01-05)';
-- [t@2001-01-03, t@2001-01-05)
```

- No temporal

```
~tbool → tbool
```

```
SELECT ~tbool '[true@2001-01-03, true@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
```

- Devuelve el tiempo cuando un booleano temporal toma el valor verdadero

```
whenTrue(tbool) → tstzspanset
```

```
SELECT whenTrue(tfloat '[1@2001-01-01, 4@2001-01-04, 1@2001-01-07]' #> 2);
-- {(2001-01-02, 2001-01-06)}
SELECT whenTrue(tdwithin(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04,
  Point(1 1)@2001-01-07]', geometry 'Point(1 1)', sqrt(2)));
-- {[2001-01-01, 2001-01-02], [2001-01-06, 2001-01-07]}
```

6.7. Operaciones matemáticas

- Adición, resta, multiplicación y división temporal

```
{number,tnumber} {+, -, *, /} {number,tnumber} → tnumber
```

La división temporal genera un error si el denominador es alguna vez igual a cero durante el intervalo de tiempo común de los argumentos.

El producto de dos flotantes temporales con interpolación lineal es, entre dos instantes consecutivos, un polinomio de grado dos en el tiempo, y el cociente por un flotante temporal con interpolación lineal es una razón de dos funciones lineales, ninguno de los cuales puede expresar la interpolación lineal. El resultado contiene entonces el valor en los instantes de los argumentos

y, dentro de un segmento, en el extremo del producto y en un instante adicional de un cociente de dos flotantes temporales, y es lineal entre los instantes que contiene. Cada valor que contiene el resultado es exacto, y también lo son su mínimo y su máximo, ya que un cociente varía de forma monótona entre dos instantes, pero entre los instantes que contiene el resultado es la cuerda del producto o del cociente. En los ejemplos siguientes, el producto vale 3.75 al mediodía del 1 de enero, donde el resultado interpola 3.5, y 2 dividido por el flotante temporal vale 1 al mediodía del 2 de enero, donde el resultado interpola 1.333. El producto de un flotante temporal por un número o por un flotante temporal con interpolación escalonada, y su cociente por un número o por un flotante temporal con interpolación escalonada, son lineales entre dos instantes y son exactos.

```
SELECT tint '[2@2001-01-01, 2@2001-01-04)' + 1;
-- [3@2001-01-01, 3@2001-01-04)
SELECT tfloat '[2@2001-01-01, 2@2001-01-04)' + tfloat '[1@2001-01-01, 4@2001-01-04)';
-- [3@2001-01-01, 6@2001-01-04)
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' + tfloat '{[1@2001-01-01, 2@2001-01-02),
  [1@2001-01-02, 2@2001-01-04)}';
-- {[2@2001-01-01, 4@2001-01-04), [3@2001-01-02, 6@2001-01-04)}
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-04)' - tint '[2@2001-01-03, 2@2001-01-05)';
-- [-1@2001-01-03, -1@2001-01-04)
SELECT tfloat '[3@2001-01-01, 6@2001-01-04)' - tfloat '[2@2001-01-01, 2@2001-01-04)';
-- [1@2001-01-01, 4@2001-01-04)
```

```
SELECT tint '[1@2001-01-01, 4@2001-01-04]' * 2;
-- [2@2001-01-01, 8@2001-01-04]
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' * tfloat '[2@2001-01-01, 2@2001-01-04)';
-- [2@2001-01-01, 8@2001-01-04)
SELECT tfloat '[1@2001-01-01, 3@2001-01-03)' * '[3@2001-01-01, 1@2001-01-03)';
-- [3@2001-01-01, 4@2001-01-02, 3@2001-01-03)
SELECT valueAtTimestamp(tfloat '[1@2001-01-01, 3@2001-01-03)' *
  '[3@2001-01-01, 1@2001-01-03)', '2001-01-01 12:00');
-- 3.5
```

```
SELECT 2 / tfloat '[1@2001-01-01, 3@2001-01-04)';
-- [2@2001-01-01, 0.666666666666667@2001-01-04)
SELECT valueAtTimestamp(2 / tfloat '[1@2001-01-01, 3@2001-01-04)', '2001-01-02 12:00');
-- 1.3333333333333333
SELECT tfloat '[1@2001-01-01, 5@2001-01-05)' / tfloat '[5@2001-01-01, 1@2001-01-05)';
-- [0.2@2001-01-01, 1@2001-01-03, 5@2001-01-05)
SELECT 2 / tfloat '[-1@2001-01-01, 1@2001-01-02]';
-- ERROR:  Division by zero
SELECT tfloat '[-1@2001-01-04, 1@2001-01-05]' / tfloat '[-1@2001-01-01, 1@2001-01-05]';
-- [-2@2001-01-04, 1@2001-01-05]
```

- Devuelve el valor absoluto del número temporal

```
abs(tnumber) → tnumber
```

```
SELECT abs(tfloat '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03, 0@2001-01-04, 1@2001-01-05]
SELECT abs(tint '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 1@2001-01-05]
```

- Redondea al entero inferior o superior

```
floor(tfloat) → tfloat
ceil(tfloat) → tfloat
```

```
SELECT floor(tfloat '[0.5@2001-01-01, 1.5@2001-01-02]');
-- [0@2001-01-01, 1@2001-01-02]
SELECT ceil(tfloat '[0.5@2001-01-01, 0.6@2001-01-02, 0.7@2001-01-03]');
-- [1@2001-01-01, 1@2001-01-03]
```

- Redondea a un número de posiciones decimales

```
round(tfloat,integer=0) → tfloat
```

```
SELECT round(tfloat '[0.7853981633974448@2001-01-01, 2.3561944901923450@2001-01-02]', 2);
-- [0.79@2001-01-01, 2.36@2001-01-02]
```

- Convierte a grados o radianes

```
degrees({float,tfloat},normalize boolean=false) → tfloat
radians(tfloat) → tfloat
```

El parámetro adicional en la función `degrees` puede ser utilizado para normalizar los valores entre 0 y 360 grados.

```
SELECT degrees(pi() * 5);
-- 900
SELECT degrees(pi() * 5, true);
-- 180
SELECT round(degrees(tfloat '[0.7853981633974448@2001-01-01,
  2.3561944901923450@2001-01-02]'));
-- [45@2001-01-01, 135@2001-01-02]
SELECT radians(tfloat '[45@2001-01-01, 135@2001-01-02]');
-- [0.7853981633974448@2001-01-01, 2.3561944901923450@2001-01-02]
```

- Devuelve la diferencia de valor entre instantes consectivos del número temporal

```
deltaValue(tnumber) → tnumber
```

```
SELECT deltaValue(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- [1@2001-01-01, -1@2001-01-02, -1@2001-01-03)
SELECT deltaValue(tfloat '{[1.5@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [2@2001-01-04, 2@2001-01-05]}');
/* Interp=Step;{[0.5@2001-01-01, -1@2001-01-02, -1@2001-01-03),
   [0@2001-01-04, 0@2001-01-05)} */
```

- Devuelve la tendencia de un flotante temporal con interpolación lineal, que indica si su valor es creciente, constante o decreciente, representado, respectivamente, por 1, 0 y -1

```
trend(tfloat) → tint
```

Nótese que la tendencia es NULL para secuencias instantáneas.

```
SELECT trend(tfloat '[1@2001-01-01, 2@2001-01-02, 4@2001-01-03, 4@2001-01-04,
  3@2001-01-05, 2@2001-01-06]');
-- [1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]
SELECT trend(tfloat '[1@2001-01-01]');
-- NULL
```

- Devuelve la derivada sobre el tiempo del número flotante temporal en unidades por segundo

```
derivative(tfloat) → tfloat
```

El número flotante temporal debe tener interpolación linear. Nótese que esta función corresponde a la función speed para los puntos temporales.

```
SELECT derivative(tfloat '{[0@2001-01-01, 10@2001-01-02, 5@2001-01-03],
  [1@2001-01-04, 0@2001-01-05]}') * 3600 * 24;
/* Interp=Step;{[-10@2001-01-01, 5@2001-01-02, 5@2001-01-03],
   [1@2001-01-04, 1@2001-01-05]} */
SELECT derivative(tfloat 'Interp=Step;[0@2001-01-01, 10@2001-01-02, 5@2001-01-03]');
-- ERROR:  The temporal value must have linear interpolation
```

- Devuelve el área bajo la curva

```
integral(tnumber) → float
```

```
SELECT integral(tint '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1
SELECT integral(tfloat '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1.5
```

- Devuelve el promedio ponderado en el tiempo

```
twAvg(tnumber) → float
```

```
SELECT twAvg(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-06)}');
-- 1.75
```

- Devuelve el logaritmo natural y el logaritmo base 10 de un número flotante temporal

```
ln(tfloat) → tfloat
log10(tfloat) → tfloat
```

El número flotante temporal no puede ser cero o negativo. Con interpolación lineal, el resultado tiene, en cada segmento, los instantes del segmento y el instante en el que la interpolación lineal más se aleja del logaritmo, y se interpola linealmente entre ellos

```
SELECT ln(tfloat '{[1@2001-01-01, 10@2001-01-02, 5@2001-01-03],
  [1@2001-01-04, 1@2001-01-05]}');
/* {[0@2001-01-01, 1.3631921320765 12@2001-01-01 07:45:23.043236,
   2.3025850929940 46@2001-01-02, 1.9759508330173 18@2001-01-02 13:22:31.148467,
   1.6094379124341@2001-01-03], [0@2001-01-04, 0@2001-01-05]} */
SELECT log10(tfloat 'Interp=Step;[-10@2001-01-01, 10@2001-01-02]');
-- ERROR:  Cannot take logarithm of zero or a negative number
```

- Devuelve el exponencial (e elevado a la potencia dada) de un número flotante temporal

```
exp(tfloat) → tfloat
```

Con interpolación lineal, el resultado tiene, en cada segmento, los instantes del segmento y el instante en el que la interpolación lineal más se aleja de la exponencial, y se interpola linealmente entre ellos

```
SELECT exp(tfloat '{[1@2001-01-01, 10@2001-01-02], [1@2001-01-04, 1@2001-01-05]}');
/* {[2.718281828459045@2001-01-01, 2447.0830569101154@2001-01-01 18:08:25.45925,
   22026.465794806718@2001-01-02],
   [2.718281828459045@2001-01-04, 2.718281828459045@2001-01-05]} */
SELECT exp(tfloat '{-10@2001-01-01, 0@2001-01-02, 10@2001-01-03}');
-- {0.00004539992976 2@2001-01-01, 1@2001-01-02, 22026.465794806718@2001-01-03}
```

- Devuelve el seno, coseno o tangente de un número flotante temporal (argumento en radianes)

```
sin(tfloat) → tfloat
cos(tfloat) → tfloat
tan(tfloat) → tfloat
```

Con interpolación lineal, el resultado de `sin` y `cos` tiene, en cada segmento, los instantes del segmento y los de los extremos de la función, el resultado de `tan` se refina por bisección donde la función es empinada, y el resultado se interpola linealmente entre sus instantes. El número flotante temporal de `tan` no puede alcanzar $\pi/2$ más un múltiplo de π, en un instante o a lo largo de un segmento con interpolación lineal

```
SELECT round(sin(tfloat '[0@2001-01-01, 6.283185307@2001-01-02]'), 6);
-- [0@2001-01-01, 1@2001-01-01 06:00:00.000001, -1@2001-01-01 18:00:00.000002, 0@2001 ↩
    -01-02]
SELECT round(cos(tfloat '[0@2001-01-01, 3.141592654@2001-01-02]'), 6);
-- [1@2001-01-01, -1@2001-01-02]
SELECT round(tan(tfloat '{0@2001-01-01, 0.785398163@2001-01-02}'), 6);
-- {0@2001-01-01, 1@2001-01-02}
SELECT tan(tfloat '[0@2001-01-01, 3@2001-01-02]');
-- ERROR:  Cannot take the tangent of pi/2 plus a multiple of pi
```

6.8. Operaciones de texto

- Concatenación de texto

```
{text,ttext} || {text,ttext} → ttext
```

```
SELECT ttext '[AA@2001-01-01, AA@2001-01-04)' || text 'B';
-- ["AAB"@2001-01-01, "AAB"@2001-01-04)
SELECT ttext '[AA@2001-01-01, AA@2001-01-04)' || ttext '[BB@2001-01-02, BB@2001-01-05)';
-- ["AABB"@2001-01-02, "AABB"@2001-01-04)
SELECT ttext '[A@2001-01-01, B@2001-01-03,
  C@2001-01-04]' || ttext '{[D@2001-01-01, D@2001-01-02), [E@2001-01-02, E@2001-01-04)}';
-- {["AD"@2001-01-01, "AE"@2001-01-02, "BE"@2001-01-03, "BE"@2001-01-04)}
```

- Transforma en minúsculas, mayúsculas o initcap

```
upper(ttext) → ttext
lower(ttext) → ttext
initcap(ttext) → ttext
```

```
SELECT lower(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["aa"@2001-01-01, "bb"@2001-01-02]
SELECT upper(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["AA"@2001-01-01, "BB"@2001-01-02]
SELECT initcap(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["Aa"@2001-01-01, "Bb"@2001-01-02]
```

Capítulo 7

Tipos temporales geométricos (Parte 1)

7.1. Tipos espaciotemporales

MobilityDB proporciona un conjunto de tipos espaciotemporales, a saber, `tgeometry` (geometría temporal), `tgeography` (geografía temporal), `tgeompoint` (punto geométrico temporal), `tgeogpoint` (punto geográfico temporal), `tcbuffer` (buffer circular temporal), `tnpoint` (punto de red temporal), `tpose` (pose temporal), `tposechain` (cadena de poses temporal), `trgeometry` (geometría rígida temporal), `th3index`, `tquadbin` y `ts2cell` (los índices de celdas DGGS temporales), y `tpcpoint` (punto de nube de puntos temporal) y `tpcpatch` (parche de nube de puntos temporal). A nivel conceptual, estos tipos espaciotemporales se organizan en la jerarquía que se muestra en la Figura 7.1.

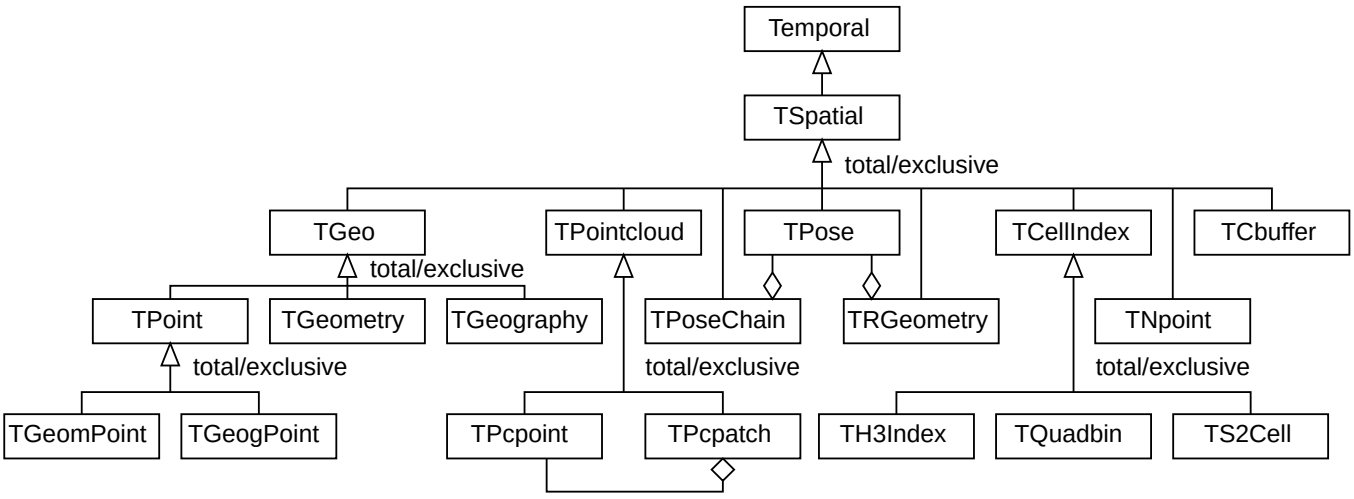


Figura 7.1: Jerarquía de tipos espaciotemporales en MobilityDB.

En este capítulo y el siguiente cubrimos el tipo `TGeo` y sus subtipos, es decir, los tipos espaciotemporales derivados de los tipos `geometry` y `geography` de PostGIS. En los capítulos siguientes, continuamos describiendo los tipos espaciotemporales restantes.

La Figura 7.2 ilustra el uso de los tipos `tgeompoint` (arriba) y `tgeometry` (abajo) para modelizar la trayectoria y la franja de viento de las tormentas tropicales en 2024. Los datos provienen de National Oceanic and Atmospherique Administration (NOAA). Como se ilustra en la figura, el tipo `tgeompoint` permite la interpolación *lineal*, mientras que el tipo `tgeometry` solo permite la interpolación *escalonada*.

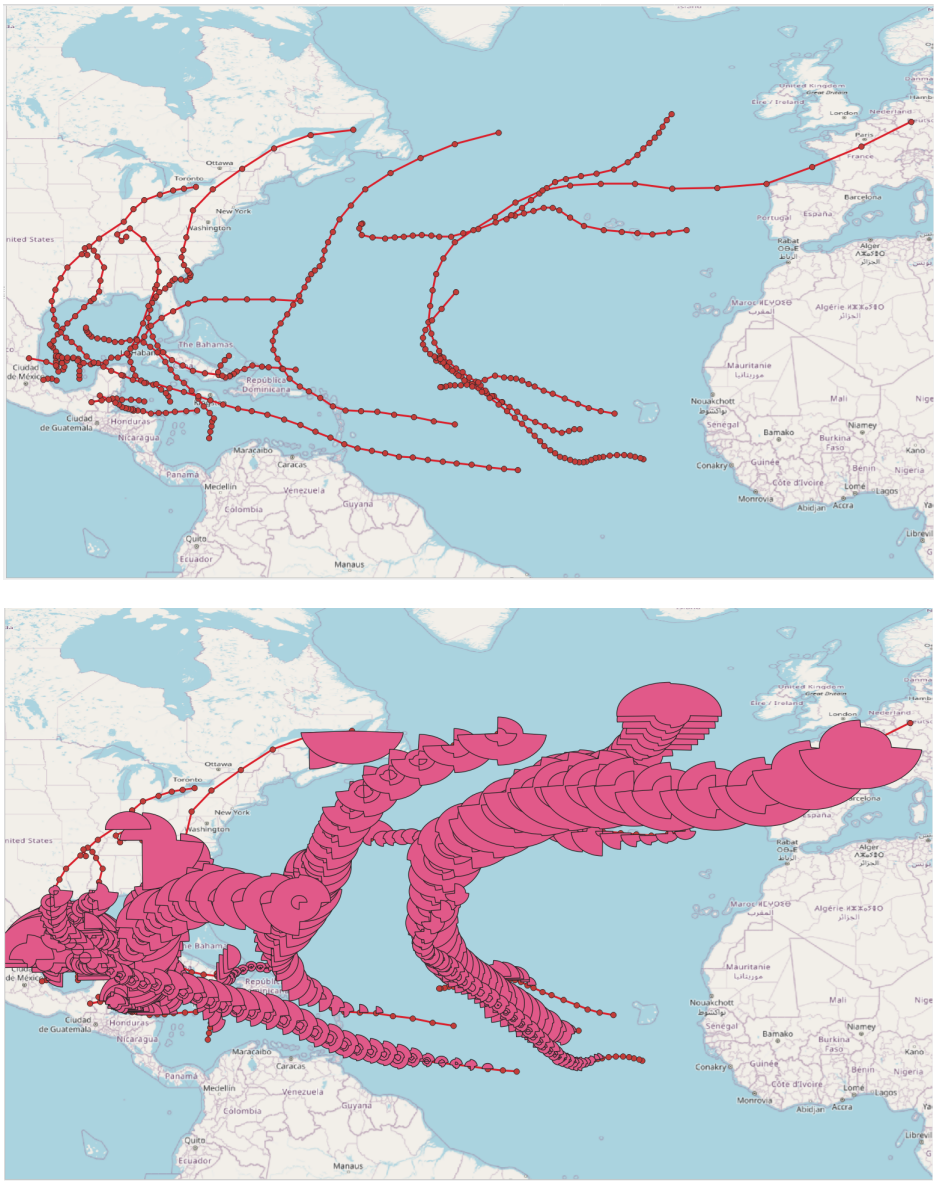


Figura 7.2: Ilustración del uso de los tipos `tgeompoint` (arriba) y `tgeometry` (abajo) para modelizar la trayectoria y la franja de viento de las tormentas tropicales en 2024.

7.2. Funciones sobre Geometrías

Las funciones de esta sección responden una pregunta sobre una geometría y no sobre un tipo temporal. Cada una indica cómo lee una geometría delimitada por arcos circulares, ya que una respuesta construida a partir de los arcos mismos y una construida a partir de una aproximación poligonal de ellos no son la misma respuesta.

- Devuelve el rectángulo de área mínima que encierra una geometría

```
orientedEnvelope(geometry) → geometry
```

El rectángulo se orienta en el ángulo que lo hace más pequeño, que en general no es el ángulo de los ejes. Se devuelve como una línea cuando la geometría está contenida en una y como un punto cuando la geometría es uno. Una geometría que lleva un arco circular se responde sobre el arco mismo: el rectángulo se coloca contra el círculo en el que se encuentra el arco y no contra la cadena de segmentos que lo aproxima.

```
SELECT ST_AsText(round(orientedEnvelope(geometry 'Polygon((0 0,3 4,-1 7,-4 3,0 0))'), 6));
-- POLYGON((0 0,3 4,-1 7,-4 3,0 0))
SELECT ST_AsText(round(orientedEnvelope(geometry 'Linestring(0 0,10 0)'), 6));
-- LINESTRING(0 0,10 0)
SELECT ST_AsText(round(orientedEnvelope(geometry 'Circularstring(0 0,0.5 0.1,1 0)'), 6));
-- POLYGON((1 0.1,0 0.1,0 0,1 0,1 0.1))
```

- Devuelve la geometría convexa más pequeña que encierra una geometría

```
convexHull(geometry) → geometry
```

El resultado es un polígono en el que cada esquina es un punto de la geometría, una línea cuando la geometría está contenida en una y un punto cuando la geometría es uno. Una geometría que lleva un arco circular se responde sobre el arco mismo, por lo que la envolvente está delimitada por el arco y se devuelve como una geometría curva y no como un polígono que la aproxima.

```
SELECT ST_AsText(round(convexHull(geometry 'Multipoint(0 0,10 0,10 10,0 10,5 5)'), 6));
-- POLYGON((0 0,0 10,10 10,10 0,0 0))
SELECT ST_AsText(round(convexHull(geometry 'Polygon((0 0,10 0,10 10,5 5,0 10,0 0))'), 6));
-- POLYGON((0 0,0 10,10 10,10 0,0 0))
SELECT ST_AsText(round(convexHull(geometry 'Circularstring(5 0,0 5,-5 0)'), 6));
-- CURVEPOLYGON(COMPOUNDCURVE((5 0,-5 0),CIRCULARSTRING(-5 0,0 5,5 0)))
```

- Devuelve verdadero si una geometría no tiene ningún punto en el que se cruce o se toque a sí misma

```
isSimple(geometry) → boolean
```

Un punto siempre es simple, un multipunto es simple cuando no repite ningún punto, una línea es simple cuando se encuentra consigo misma solo donde dos de sus segmentos se siguen y, cuando se cierra, en el punto donde se cierra, y una geometría de área es simple cuando cada uno de sus anillos lo es. Además, dos líneas de una multilínea pueden encontrarse en un punto que termina ambas. Una geometría que lleva un arco circular se encuentra consigo misma a lo largo de un arco y no en un punto, lo que la intersección de segmentos en la que esto se basa no lee, por lo que tal geometría se responde sobre la cadena de segmentos que aproxima sus arcos.

```
SELECT isSimple(geometry 'Linestring(0 0,10 10,10 0,0 10)');
-- false
SELECT isSimple(geometry 'Linestring(0 0,10 0,10 10,0 0)');
-- true
SELECT isSimple(geometry 'Multilinestring((0 0,10 0),(10 0,10 10))');
-- true
```

- Devuelve la geometría que contiene todos los puntos situados a una distancia dada de una geometría

```
buffer(geometry,float,options text='') → geometry
```

La frontera del resultado está formada por los dos desplazamientos de la geometría a la distancia dada, por la unión que rellena el lado exterior de cada giro y por el extremo que cierra cada final de una línea abierta. Una distancia negativa contrae una geometría de área y devuelve una geometría vacía para una de cualquier otra dimensión. La cadena `options` lleva los estilos como pares `clave=valor` separados por espacios: `endcap` es uno de `round`, `flat` o `square`, `join` uno de `round`, `mitre` o `bevel`, y `mitre_limit` la razón a partir de la cual una unión en inglete pasa a ser biselada.

```
SELECT ST_AsText(round(buffer(geometry 'Point(0 0)', 1), 6));
-- CURVEPOLYGON(CIRCULARSTRING(-1 0,1 0,-1 0))
SELECT ST_AsText(round(buffer(geometry 'Polygon((0 0,10 0,10 10,0 10,0 0))', -1,
  'join=mitre'), 6));
-- CURVEPOLYGON(COMPOUNDCURVE((1 1,9 1),(9 1,9 9),(9 9,1 9),(1 9,1 1)))
SELECT ST_AsText(round(buffer(geometry 'Curvepolygon(Circularstring(0 0,2 2,4 0,2 -2,
  0 0))', 1), 6));
-- CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(-1 0,2 3,5 0),CIRCULARSTRING(5 0,2 -3,-1 0)))
```

- Devuelve la matriz de intersección DE-9IM de dos geometrías o si coincide con un patrón

```
relate(geometry,geometry) → text
relate(geometry,geometry,text) → boolean
```

La matriz indica, para cada par formado por el interior, la frontera y el exterior de la primera geometría y de la segunda, la dimensión de lo que comparten: `F` cuando no comparten nada, `0`, `1` o `2` cuando comparten puntos, una curva o un área. Un arco circular se lee sobre su propio círculo y no sobre una cadena de segmentos que lo aproxima, y el interior y la frontera de una colección de geometrías son los de la unión de sus miembros. Con un patrón, la función responde si la matriz coincide con él, como lo hace `ST_Relate` con tres argumentos: un carácter `T` del patrón coincide con cualquiera de `0`, `1` y `2`, y un carácter `*` coincide con cualquier cosa.

```
SELECT relate(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Linestring(2 2,6 2)');
-- 1020F1102
SELECT relate(geometry 'Polygon((0 0,2 0,2 2,0 2,0 0))',
  geometry 'Polygon((2 0,4 0,4 2,2 2,2 0))');
-- FF2F11212
SELECT relate(geometry 'Curvepolygon(Circularstring(0 0,2 2,4 0,2 -2,0 0))',
  geometry 'Point(2 1.9)');
-- 0F2FF1FF2
SELECT relate(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Point(2 2)',
  'T*****FF*');
-- true
```

- Devuelve verdadero si dos geometrías satisfacen una relación espacial

```
contains(geometry,geometry) → boolean
covers(geometry,geometry) → boolean
touches(geometry,geometry) → boolean
geoEquals(geometry,geometry) → boolean
disjoint({geometry,geography},{geometry,geography}) → boolean
intersects({geometry,geography},{geometry,geography}) → boolean
```

Cada una responde como la función de PostGIS del mismo nombre con el prefijo `ST_` sobre los mismos argumentos, sin llamar a GEOS. Sobre dos geometrías con Z, `disjoint` e `intersects` se evalúan en 3D, como `ST_3DIntersects`, y en 2D cuando una de ellas no tiene Z. Las relaciones de contención `contains`, `covers` y `touches` no tienen forma 3D y rechazan una geometría con Z. Dos geografías se intersecan sobre el esferoide WGS 84, como lo decide `ST_Intersects`. La igualdad de dos conjuntos de puntos se llama `geoEquals` en todos los motores, ya que PostGIS declara `equals(geometry, geometry)` como un alias obsoleto de `ST_Equals`; lee los puntos, mientras que el operador = compara los vértices.

```
SELECT contains(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Point(2 2)');
-- true
SELECT touches(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Point(4 2)');
-- true
SELECT intersects(geometry 'Point(1 1 1)', geometry 'Point(1 1 2)');
-- false
```

```
SELECT geoEquals(geometry 'Linestring(0 0,2 0)', geometry 'Linestring(0 0,1 0,2 0)');
-- true
```

- Devuelve verdadero si dos geometrías están a una distancia dada

```
dwithin(geometry,geometry,float) → boolean
dwithin(geography,geography,float,spheroid boolean=true) → boolean
```

Dos geometrías con Z se miden en 3D, como `ST_3DDWithin`, y en 2D cuando una de ellas no tiene Z, véase Sección 8.3. Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT dwithin(geometry 'Point(0 0 0)', geometry 'Point(0 0 2)', 1.5);
-- false
SELECT dwithin(geometry 'Point(0 0 0)', geometry 'Point(0 0)', 1.5);
-- true
SELECT dwithin(geography 'Point(0 0)', geography 'Point(0 1)', 110800);
-- true
SELECT dwithin(geography 'Point(0 0)', geography 'Point(0 1)', 110800, false);
-- false
```

- Devuelve el área, el perímetro o el centroide de una geometría o una geografía

```
area(geometry) → float
area(geography,spheroid boolean=true) → float
perimeter(geometry) → float
perimeter(geography,spheroid boolean=true) → float
centroid(geometry) → geometry
centroid(geography,spheroid boolean=true) → geography
```

Cada una responde como la función de PostGIS del mismo nombre con el prefijo `ST_`, sin llamar a GEOS, salvo que el área de una superficie delimitada por arcos de circunferencia se mide sobre sus arcos, por la forma cerrada del segmento circular que cada arco añade a su cuerda: para el disco de radio 1 a continuación el área es π, que `ST_Area`, al reemplazar cada cuarto de circunferencia por 32 segmentos, da como 3,1403. Una geografía se mide en metros cuadrados o en metros sobre el esferoide WGS 84 por defecto, y sobre la esfera, un cálculo más rápido, si el último argumento es falso. Sobre la esfera, el área es la que delimitan los círculos máximos que unen los vértices, que `ST_Area` sobre una geografía en la esfera aproxima: para el cuadrado de un grado en el ecuador a continuación, la aproximación es un 0,67 por ciento menor.

```
SELECT area(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))');
-- 16
SELECT area(geometry 'CurvePolygon(CircularString(0 0,1 1,2 0,1 -1,0 0))');
-- 3.141592653589793
SELECT ST_AsText(centroid(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))'));
-- POINT(2 2)
SELECT round(area(geography 'Polygon((0 0,1 0,1 1,0 1,0 0))')::numeric);
-- 12308778361
SELECT round(area(geography 'Polygon((0 0,1 0,1 1,0 1,0 0))', false)::numeric);
-- 12364031799
```

- Devuelve la distancia, la línea que une los puntos más cercanos o la distancia máxima de dos geometrías o geografías

```
distance(geometry,geometry) → float
distance(geography,geography,spheroid boolean=true) → float
shortestLine(geometry,geometry) → geometry
shortestLine(geography,geography,spheroid boolean=true) → geography
maxDistance(geometry,geometry) → float
```

Dos geometrías con Z se miden en 3D, como las funciones de PostGIS con el prefijo `ST_3D`, y una geometría con Z y otra sin ella se rechazan. Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido. La línea más corta de dos geografías une sus puntos más cercanos sobre la esfera, como los une `ST_ShortestLine` sobre geografías.

```
SELECT distance(geometry 'Point(0 0 0)', geometry 'Point(3 4 12)');
-- 13
SELECT ST_AsText(shortestLine(geometry 'Point(0 0 0)',
  geometry 'Linestring(1 -1 1,1 1 1)'));
-- LINESTRING Z (0 0 0,1 0 1)
SELECT round(maxDistance(geometry 'Linestring(0 0 0,1 0 0)',
  geometry 'Point(0 0 2)')::numeric, 6);
-- 2.236068
SELECT round(distance(geography 'Point(0 0)', geography 'Point(0 1)', false)::numeric, 3);
-- 111195.080
```

- Devuelve la frontera, la inversión o las partes de una geometría

```
boundary(geometry) → geometry
reverse(geometry) → geometry
numGeometries(geometry) → integer
geometryN(geometry,integer) → geometry
numPoints(geometry) → integer
```

Cada una responde como la función de PostGIS del mismo nombre con el prefijo `ST_`, `numPoints` contando todos los vértices como lo hace `ST_NPoints` y `geometryN` contando desde 1. La frontera de una geometría vacía es la geometría vacía de la dimensión de una frontera. La frontera de una curva compuesta son sus dos puntos extremos, o vacía cuando se cierra, y la frontera de una multisuperficie son las fronteras de sus miembros; `ST_Boundary` rechaza ambas.

```
SELECT ST_AsText(boundary(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))'));
-- LINESTRING(0 0,4 0,4 4,0 4,0 0)
SELECT ST_AsText(reverse(geometry 'Linestring(0 0,4 0,4 3)'));
-- LINESTRING(4 3,4 0,0 0)
SELECT numGeometries(geometry 'MultiPoint(1 1,5 5)');
-- 2
SELECT ST_AsText(geometryN(geometry 'MultiPoint(1 1,5 5)', 2));
-- POINT(5 5)
SELECT numPoints(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))');
-- 5
```

- Devuelve el punto en una fracción de una línea, la parte entre dos fracciones o la fracción más cercana a un punto

```
lineInterpolatePoint(geometry,float) → geometry
lineSubstring(geometry,float,float) → geometry
lineLocatePoint(geometry,geometry) → float
```

Cada una responde como la función de PostGIS del mismo nombre con el prefijo `ST_`, siendo una fracción una parte de la longitud de la línea entre 0 y 1.

```
SELECT ST_AsText(lineInterpolatePoint(geometry 'Linestring(0 0,4 0,4 3)', 0.25));
-- POINT(1.75 0)
SELECT ST_AsText(lineSubstring(geometry 'Linestring(0 0,4 0,4 3)', 0.25, 0.8));
-- LINESTRING(1.75 0,4 0,4 1.6)
SELECT lineLocatePoint(geometry 'Linestring(0 0,4 0,4 3)', geometry 'Point(4 4)');
-- 1
```

- Devuelve la colección de un arreglo de geometrías o la línea que las une

```
collect(geometry[]) → geometry
makeLine(geometry[]) → geometry
```

Una colección toma su tipo únicamente de los tipos de las geometrías, como lo da la función de PostGIS `ST_Collect`, de modo que un arreglo de un punto devuelve un multipunto, y una línea une los puntos y las líneas del arreglo en su orden, como los une `ST_MakeLine`. Un arreglo vacío devuelve `NULL`.

```
SELECT ST_AsText(collect(ARRAY[geometry 'Point(1 1)']));
-- MULTIPOINT((1 1))
SELECT ST_AsText(collect(ARRAY[geometry 'Point(1 1)', geometry 'Linestring(0 0,1 1)']));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(0 0,1 1))
SELECT ST_AsText(makeLine(ARRAY[geometry 'Point(1 1)', geometry 'Point(2 2)']));
-- LINESTRING(1 1,2 2)
```

- Devuelve la representación Well-Known Text, Extended Well-Known Text, Extended Well-Known Binary o GeoJSON de una geometría o una geografía

```
asText({geometry,geography},maxdecimaldigits integer=15) → text
asEWKT({geometry,geography},maxdecimaldigits integer=15) → text
asEWKB({geometry,geography},endian text='') → bytea
asGeoJSON(geometry,maxdecimaldigits integer=9,options integer=8) → text
asGeoJSON(geography,maxdecimaldigits integer=9,options integer=0) → text
```

Cada una responde como la función de PostGIS del mismo nombre con el prefijo `ST_`. El GeoJSON indica el sistema de referencia de coordenadas que piden sus opciones, la suma de 1 para el rectángulo delimitador, 2 para el nombre corto del sistema, como en `EPSG:3857`, 4 para su nombre largo y 8 para el nombre corto de cualquier sistema salvo WGS 84, que es el valor por defecto para una geometría; una geografía no toma ninguna opción por defecto.

```
SELECT asText(geometry 'SRID=3857;Point(1.123456789 2)', 3);
-- POINT(1.123 2)
SELECT asEWKT(geometry 'SRID=3857;Point(1 2)');
-- SRID=3857;POINT(1 2)
SELECT asEWKB(geometry 'Point(1 1)', 'XDR');
-- \x00000000013ff00000000000003ff0000000000000
SELECT asGeoJSON(geometry 'SRID=3857;Point(1 2)');
-- {"type":"Point","crs":{"type":"name","properties":{"name":"EPSG:3857"}},"coordinates ↩
    ":[1,2]}
SELECT asGeoJSON(geometry 'SRID=4326;Point(1 2)');
-- {"type":"Point","coordinates":[1,2]}
```

- Transforma una geometría o una geografía a un identificador de referencia espacial

```
transform(geometry,integer) → geometry
transform(geography,integer) → geography
```

Una geometría se transforma como la transforma la función de PostGIS `ST_Transform`. Una geografía se transforma únicamente a un sistema de coordenadas de longitud y latitud, como lo exige la conversión de una geometría en una geografía.

```
SELECT asEWKT(transform(geometry 'SRID=4326;Point(4.35 50.85)', 3857), 2);
-- SRID=3857;POINT(484239.78 6594803.23)
SELECT asEWKT(transform(geography 'Point(4.35 50.85)', 3857));
-- ERROR:  Only lon/lat coordinate systems are supported
```

- Densifica una geometría o una geografía de modo que ningún segmento sea más largo que una distancia

```
segmentize(geometry,float) → geometry
segmentize(geography,float) → geography
```

Una geometría se densifica en el plano, la distancia en las unidades de su sistema de referencia espacial, y una geografía sobre la esfera, la distancia en metros, como las densifica la función de PostGIS `ST_Segmentize`. Un punto, un multipunto y un valor vacío se devuelven sin cambios, al igual que un triángulo, un TIN y una superficie poliédrica. Una distancia que no es positiva se rechaza.

```
SELECT ST_AsText(segmentize(geometry 'Linestring(0 0,3 0)', 1));
-- LINESTRING(0 0,1 0,2 0,3 0)
SELECT asText(segmentize(geography 'Linestring(4.35 50.85,2.35 48.85)', 100000), 4);
-- LINESTRING(4.35 50.85,3.8343 50.3533,3.3293 49.8543,2.8347 49.3532,2.35 48.85)
```

- Devuelve los grupos de un arreglo de geometrías

```
clusterKMeans(geometry[],k integer) → integer[]
clusterDBSCAN(geometry[],eps float,minpoints integer) → integer[]
clusterIntersecting(geometry[]) → geometry[]
clusterWithin(geometry[],distance float) → geometry[]
```

`clusterKMeans` y `clusterDBSCAN` devuelven el número del grupo de cada geometría, en el orden del arreglo, como las numeran las funciones de ventana de PostGIS `ST_ClusterKMeans` y `ST_ClusterDBSCAN` sobre las filas en ese orden: una geometría vacía está en el grupo -1 de k-means, y una geometría que DBSCAN no ubica en ningún grupo tiene el número `NULL`. `clusterIntersecting` y `clusterWithin` devuelven una colección de geometrías por grupo, como las devuelven los agregados de PostGIS `ST_ClusterIntersecting` y `ST_ClusterWithin`. Un arreglo vacío devuelve `NULL`.

```
SELECT clusterKMeans(ARRAY[geometry 'Point(0 0)', geometry 'Point(1 0)', geometry 'Point ↩
    (10 10)',
  geometry 'Point(11 10)', geometry 'Point(30 30)'], 2);
-- {1,1,1,1,0}
SELECT clusterDBSCAN(ARRAY[geometry 'Point(0 0)', geometry 'Point(1 0)', geometry 'Point ↩
    (10 10)',
  geometry 'Point(11 10)', geometry 'Point(30 30)'], 1.5, 2);
-- {0,0,1,1,NULL}
SELECT array_length(clusterWithin(ARRAY[geometry 'Point(0 0)', geometry 'Point(1 0)',  ↩
    geometry 'Point(10 10)',
  geometry 'Point(11 10)', geometry 'Point(30 30)'], 2), 1);
-- 3
```

- Devuelve la región que dos geometrías comparten

```
intersection(geometry,geometry) → geometry
```

Las cuatro operaciones booleanas de esta sección responden sobre los (multi)polígonos que llevan los tipos temporales cuyos valores son regiones, y cada una toma el nombre de la función de PostGIS para la que responde sin el prefijo `ST_`, de modo que una consulta alcanza este motor quitando el prefijo y PostGIS conservándolo. Son planas y bidimensionales: se rechazan una geografía, una dimensión Z y una geometría que no sea un (multi)polígono. Un resultado vacío se responde como `NULL`.

```
SELECT ST_AsText(intersection(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
-- POLYGON((5 3,3 3,3 5,5 5,5 3))
SELECT ST_AsText(intersection(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'MultiPolygon(((2 2,2 4,3 4,3 2,2 2)),((4 2,4 4,6 4,6 2,4 2)))'));
-- MULTIPOLYGON(((3 2,2 2,2 4,3 4,3 2)),((5 2,4 2,4 4,5 4,5 2)))
SELECT intersection(geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))',
  geometry 'Polygon((10 10,10 11,11 11,11 10,10 10))') IS NULL;
-- true
```

- Devuelve la región que dos geometrías cubren en conjunto

```
geoUnion(geometry,geometry) → geometry
```

`UNION` es una palabra reservada de SQL, por lo que esta lleva el nombre del tipo sobre el que responde, como hacen `setUnion` y `spanUnion`. Dos geometrías que no se tocan se responden como un multipolígono.

```
SELECT ST_AsText(geoUnion(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
-- POLYGON((5 1,1 1,1 5,3 5,3 7,7 7,7 3,5 3,5 1))
```

- Devuelve la región de la primera geometría que la segunda no cubre

```
difference(geometry,geometry) → geometry
```

Una segunda geometría situada dentro de la primera deja un agujero en lugar de dividirla.

```
SELECT ST_AsText(difference(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
-- POLYGON((5 1,1 1,1 5,3 5,3 3,5 3,5 1))
SELECT ST_AsText(difference(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((2 2,2 4,4 4,4 2,2 2))'));
-- POLYGON((5 1,1 1,1 5,5 5,5 1),(2 4,2 2,4 2,4 4,2 4))
```

- Devuelve la región que cubre exactamente una de dos geometrías

```
symDifference(geometry,geometry) → geometry
```

Es la unión de las dos geometrías menos la región que comparten, por lo que una geometría contra sí misma se responde como `NULL`.

```
SELECT ST_AsText(symDifference(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
-- MULTIPOLYGON(((7 3,5 3,5 5,3 5,3 7,7 7,7 3)),((5 1,1 1,1 5,3 5,3 3,5 3,5 1)))
SELECT symDifference(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))') IS NULL;
-- true
```

7.3. Notación

Presentamos en la Sección 4.5 y en la Sección 6.1 la notación utilizada para definir la firma de las funciones y operadores para tipos temporales. A continuación, ampliamos estas notaciones para tipos espaciotemporales.

- `tspatial` representa un tipo espaciotemporal, por ejemplo, `tgeometry`, `tgeompoint`, `tpose`, o `tnpoint`,
- `tgeo` representa un tipo temporal geometría/geografía, es decir, `tgeometry`, `tgeography`, `tgeompoint`, o `tgeogpoint`,
- `tgeom` representa un tipo temporal geometría, es decir, `tgeometry` o `tgeompoint`,
- `tgeog` representa un tipo temporal geografía, es decir, `tgeography` o `tgeogpoint`,
- `tpoint` representa un tipo temporal punto, es decir, `tgeompoint` o `tgeogpoint`,
- `spatial` representa un tipo de base espacial, por ejemplo, `geometry`, `geography`, `pose`, o `npoint`,
- `geo` representa los tipos `geometry` o `geography`,
- `geompoint` representa el tipo `geometry` restringido a un punto.
- `geogpoint` representa el tipo `geography` restringido a un punto.
- `point` representa los tipos `geometry` o `geography` restringidos a un punto.
- `lines` representa los tipos `geometry` o `geography` restringidos a una (multi)línea.

A continuación, se especifica con el símbolo ⬡ que la función admite geometrías en 3D y con el símbolo 🌐 que la función admite geografías.

7.4. Entrada y salida

- Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)

```
asText({tspatial,tspatial[],spatial[]}) → {text,text[]}
asEWKT({tspatial,tspatial[],spatial[]}) → {text,text[]}
```

```
SELECT asText(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02)');
-- [POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02)
SELECT asText(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
/* {"[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
   "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"} */
SELECT asText(ARRAY[geometry 'Point(0 0)', 'Point(1 1)']);
-- {"POINT(0 0)","POINT(1 1)"}
```

```
SELECT asEWKT(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02)');
-- SRID=4326;[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02)
SELECT asEWKT(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
-- {"SRID=4326;[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
    "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"}
SELECT asEWKT(ARRAY[geometry 'SRID=5676;Point(0 0)', 'SRID=5676;Point(1 1)']);
-- {"SRID=5676;POINT(0 0)","SRID=5676;POINT(1 1)"}
```

- Devuelve la representación JSON de características móviles (MF-JSON)

```
asMFJSON(tspatial,options integer=0,flags integer=0,maxdecimaldigits integer=15) → text
```

El argumento `options` puede usarse para agregar BBOX y/o CRS en la salida MFJSON:

- 0: significa que no hay opción (valor por defecto)
- 1: MFJSON BBOX
- 2: MFJSON Short CRS (e.g., EPSG:4326)
- 4: MFJSON Long CRS (e.g., urn:ogc:def:crs:EPSG::4326)

El argumento `flags` puede usarse para personalizar la salida JSON, por ejemplo, para producir una salida JSON fácil de leer (para lectores humanos). Consulte la documentación de la biblioteca `json-c` para conocer los valores posibles. Los valores típicos son los siguientes:

- 0: means no option (default value)
- 1: JSON_C_TO_STRING_SPACED
- 2: JSON_C_TO_STRING_PRETTY

El argumento `maxdecimaldigits` puede usarse para establecer el número máximo de decimales en la salida de los valores en punto flotante (por defecto 15).

```
SELECT asMFJSON(tgeompoint 'Point(1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingPoint","coordinates":[[1,2]],"datetimes":["2019-01-01T17:00:00.15"],
   "interpolation":"None"} */
SELECT asMFJSON(tgeometry 'SRID=3812;Linestring(1 1,1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingGeometry", "crs":{"type":"Name","properties":{"name":"EPSG:3812"}},
   "values":[{"type":"LineString","coordinates":[[1,1],[1,2]]}],
   "datetimes":["2019-01-01T17:00:00.15"],"interpolation":"None"} */
SELECT asMFJSON(tgeompoint 'SRID=4326;
  Point(50.813810 4.384260)@2019-01-01 18:00:00.15+02', 3, 0, 2);
/* {"type":"MovingPoint","crs":{"type":"name","properties":{"name":"EPSG:4326"}},
   "stBoundedBy":{"bbox":[50.81,4.38,50.81,4.38],
```

```
   "period":{"begin":"2019-01-01 17:00:00.15","end":"2019-01-01 17:00:00.15"}},
   "coordinates":[[50.81,4.38]],"datetimes":["2019-01-01T17:00:00.15"],
   "interpolations":"None"} */
```

- Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
asBinary(tspatial,endian text='') → bytea
asEWKB(tspatial,endian text='') → bytea
asHexEWKB({tspatial,geo},endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tgeompoint 'Point(1 2 3)@2001-01-01');
-- \x012e001101e90300000000000000000f03f0000000000000040000000000000000840009c57d3c11c0000
SELECT asEWKB(tgeogpoint 'SRID=7844;Point(1 2 3)@2001-01-01');
-- \x012f0071a41e0000000000000000f03f0000000000000040000000000000000840009c57d3c11c0000
SELECT asHexEWKB(tgeompoint 'SRID=3812;Point(1 2 3)@2001-01-01');
-- 012E0051E40E0000000000000000F03F0000000000000040000000000000000840009C57D3C11C0000
SELECT asHexEWKB(geometry 'SRID=3812;Point(1 2)');
-- 0101000020E40E0000000000000000F03F0000000000000040
```

- Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)

```
tspatialFromText(text) → tspatial
tspatialFromEWKT(text) → tspatial
```

En las funciones anteriores, `tspatial` reemplaza cualquier tipo espacial, como `tgeompoint`, `tgeometry` o `tpose`

```
SELECT asEWKT(tgeompointFromText(text '[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]'));
-- [POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
SELECT asEWKT(tgeographyFromText(text '[Point(1 2)@2001-01-01,
  Linestring(1 2,3 4)@2001-01-02]'));
-- SRID=4326;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
```

```
SELECT asEWKT(tgeompointFromEWKT(text 'SRID=3812;
  [Point(1 2)@2001-01-01, Point(3 4)@2001-01-02]'));
-- SRID=3812;[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
SELECT asEWKT(tgeographyFromEWKT(text 'SRID=7844;
  [Point(1 2)@2001-01-01, Linestring(1 2,3 4)@2001-01-02]'));
-- SRID=7844;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
```

- Entrada desde la representación JSON de características móviles (MF-JSON)

```
tspatialFromMFJSON(text) → tspatial
```

En la funcion anterior, `tspatial` reemplaza cualquier tipo espacial, como `tgeompoint`, `tgeometry` o `tpose`

```
SELECT asEWKT(tgeompointFromMFJSON(text '{"type":"MovingPoint",
  "crs":{"type":"name", "properties":{"name":"EPSG:4326"}},"coordinates":[[50.81,4.38]],
  "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15
SELECT asEWKT(tgeogpointFromMFJSON(text '{"type":"MovingPoint",
  "crs":{"type":"name", "properties":{"name":"EPSG:4326"}},"coordinates":[[50.81,4.38]],
  "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15
SELECT asEWKT(tgeographyFromMFJSON(text '{"type":"MovingGeometry",
  "crs":{"type":"Name","properties":{"name":"EPSG:7844"}},
```

```
  "values":[{"type":"LineString","coordinates":[[1,1],[1,2]]}],
  "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
-- SRID=7844;LINESTRING(1 1,1 2)@2019-01-01 17:00:00.15
```

- Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

```
tspatialFromBinary(bytea) → tspatial
tspatialFromEWKB(bytea) → tspatial
tspatialFromHexWKB(text) → tspatial
tspatialFromHexEWKB(text) → tspatial
geometryFromHexEWKB(text) → geometry
geographyFromHexEWKB(text) → geography
```

En las funciones anteriores, `tspatial` reemplaza cualquier tipo espacial, como `tgeompoint`, `tgeometry` o `tpose`

```
SELECT asEWKT(tgeompointFromBinary(
  '\x012e0011000000000000f03f0000000000000040000000000000000840009c57d3c11c0000'));
-- POINT Z (1 2 3)@2001-01-01
SELECT asEWKT(tgeogpointFromEWKB(
  '\x012f0071a41e0000000000000000f03f0000000000000040000000000000000840009c57d3c11c0000'));
-- SRID=7844;POINT Z (1 1 1)@2001-01-01
SELECT asEWKT(tgeompointFromHexEWKB(
  '012E0051E40E0000000000000000F03F0000000000000040000000000000000840009C57D3C11C0000'));
-- SRID=3812;POINT(1 2 3)@2001-01-01
SELECT ST_AsEWKT(geographyFromHexEWKB(
  '0101000020E61000006666666666661140CDCCCCCCCC6C4940'));
-- SRID=4326;POINT(4.35 50.85)
```

7.5. Conversión de tipos

- Convierte un valor espaciotemporal a un rango temporal o a un cuadro delimitador espaciotemporal

```
tspatial::stbox
stbox(tspatial)
```

```
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]'::stbox;
-- STBOX XT(((1,1),(3,3)),[2001-01-01, 2001-01-03])
SELECT tgeography '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03]'::stbox;
-- SRID=4326;GEODSTBOX ZT(((1,1,1),(3,3,3)),[2001-01-01, 2001-01-03])
```

- Convierte entre una geometría temporal y una geografía temporal

```
{tgeometry,tgeography}::{tgeography,tgeometry}
{tgeompoint,tgeogpoint}::{tgeogpoint,tgeompoint}
```

Las conversiones siguen la regla de [?listitem].

```
SELECT asText((tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02)')::tgeogpoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02)
SELECT asText((tgeography 'Linestring(0 0,1 1)@2001-01-01')::tgeometry);
-- LINESTRING(0 0,1 1)@2001-01-01
```

Una forma común de almacenar puntos temporales en PostGIS es representarlos como geometrías de tipo `LINESTRING M` y utilizar la dimensión M para codificar marcas de tiempo como segundos desde 1970-01-01 00:00:00. Estas geometrías aumentadas con tiempo, llamadas trayectorias, se pueden validar con la función `ST_IsValidTrajectory` para verificar quel el

valor M está creciendo de cada vértice al siguiente. Las trayectorias se pueden manipular con las funciones `ST_ClosestPointOfApproach`, `ST_DistanceCPA` y `ST_CPAWithin`. Los valores de puntos temporales se pueden convertir a/desde trayectorias de PostGIS.

- Convierte entre un punto temporal y una trayectoria PostGIS

```
{tpoint,geo}::{geo,tpoint}
```

```
SELECT ST_AsText((tgeompoint 'Point(0 0)@2001-01-01')::geometry);
-- POINT M (0 0 978307200)
SELECT ST_AsText((tgeompoint '{Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 1)@2001-01-03}')::geometry);
-- MULTIPOINT M (0 0 978307200,1 1 978393600,1 1 978480000)"
SELECT ST_AsText((tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02)')::geometry);
-- LINESTRING M (0 0 978307200,1 1 978393600)
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02),
  [Point(1 1)@2001-01-03, Point(1 1)@2001-01-04),
  [Point(1 1)@2001-01-05, Point(0 0)@2001-01-06)}')::geometry);
/* MULTILINESTRING M ((0 0 978307200,1 1 978393600),(1 1 978480000,1 1 978566400),
   (1 1 978652800,0 0 978739200)) */
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02),
  [Point(1 1)@2001-01-03], [Point(1 1)@2001-01-05, Point(0 0)@2001-01-06)}')::geometry);
/* GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
   POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200)) */
```

```
SELECT asText(geometry 'LINESTRING M (0 0 978307200,0 1 978393600,
  1 1 978480000)'::tgeompoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02, POINT(1 1)@2001-01-03];
SELECT asText(geometry 'GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
  POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200))'::tgeompoint);
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(1 1)@2001-01-03],
   [POINT(1 1)@2001-01-05, POINT(0 0)@2001-01-06]} */
```

7.6. Accessores

- Devuelve la trayectoria o el área atravesada

```
trajectory(tpoint,unary_union boolean=false) → geo
traversedArea(tgeo,unary_union boolean=false) → geo
```

Esta función es equivalente a getValues para los valores temporales alphanuméricos. El último argumento indica si se aplica la función PostGIS ST_UnaryUnion para eliminar geometrías redundantes en el resultado. Tenga en cuenta que establecer este argumento como verdadero implica un alto consumo computacional.

```
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
  Point(0 0)@2001-01-03]'));
-- LINESTRING(0 0,0 1,0 0)
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
  Point(0 0)@2001-01-03]', true));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajectory(tgeompoint '{[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02),
  [Point(0 1)@2001-01-03, Point(0 0)@2001-01-04)}'));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajectory(tgeompoint 'Interp=Step;{[Point(0 0)@2001-01-01,
  Point(0 1)@2001-01-02], [Point(0 1)@2001-01-03, Point(1 1)@2001-01-04]}'));
-- MULTIPOINT((0 0),(1 1),(0 1))
```

```
SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]'));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(1 1,2 2))
SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]', true));
-- LINESTRING(1 1,2 2)
```

- Devuelve el centroide como un punto temporal

```
centroid(tgeo) → tpoint
centroid(tgeography,spheroid boolean=true) → tgeogpoint
```

Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT asText(centroid(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,3 3)@2001-01-02,
  Polygon((1 1,4 4,7 1,1 1))@2001-01-03]'));
-- Interp=Step;[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(4 2)@2001-01-03]
SELECT asText(centroid(tgeography '[MultiPoint(1 1,4 4,7 1)@2001-01-01,
  Polygon((1 1,4 4,7 1,1 1))@2001-01-02]'),6);
-- Interp=Step;[POINT(4 2.001727)@2001-01-01, POINT(4 2.001727)@2001-01-02]
SELECT asText(centroid(tgeography '{Linestring(0 0,0 1,1 1)@2001-01-01}', false), 6);
-- {POINT(0.249967 0.750007)@2001-01-01}
```

- Devuelve los valores de las coordenadas X/Y/Z como un número flotante temporal

```
getX(tpoint) → tfloat
getY(tpoint) → tfloat
getZ(tpoint) → tfloat
```

```
SELECT getX(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
  Point(5 6)@2001-01-03}');
-- {1@2001-01-01, 3@2001-01-02, 5@2001-01-03}
SELECT getX(tgeogpoint 'Interp=Step;
  [Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02, Point(7 8 9)@2001-01-03]');
-- Interp=Step;[1@2001-01-01, 4@2001-01-02, 7@2001-01-03]
SELECT getY(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
  Point(5 6)@2001-01-03}');
-- {2@2001-01-01, 4@2001-01-02, 6@2001-01-03}
SELECT getY(tgeogpoint 'Interp=Step;
  [Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02, Point(7 8 9)@2001-01-03]');
-- Interp=Step;[2@2001-01-01, 5@2001-01-02, 8@2001-01-03]
SELECT getZ(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
  Point(5 6)@2001-01-03}');
-- The temporal point must have Z dimension
SELECT getZ(tgeogpoint 'Interp=Step;
  [Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02, Point(7 8 9)@2001-01-03]');
-- Interp=Step;[3@2001-01-01, 6@2001-01-02, 9@2001-01-03]
```

- Devuelve verdadero si el punto temporal no se auto-intersecta espacialmente

```
isSimple(tpoint) → boolean
```

Nótese que un punto temporal de conjunto de secuencias es simple si cada una de las secuencias que lo componen es simple.

```
SELECT isSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 0)@2001-01-03]');
-- false
SELECT isSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]');
```

```
-- false
SELECT isSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}');
-- true
```

- Devuelve la longitud atravesada por el punto temporal

```
length(tpoint) → float
length(geo) → float
length({geography,tgeogpoint},spheroid boolean=true) → float
```

La función `length` devuelve la longitud de una geometría o de una geografía, la de una geografía en metros sobre el esferoide, como la función `ST_Length` de PostGIS. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- 1.73205080756888
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(0 0 0)@2001-01-03]');
-- 3.46410161513775
SELECT length(tgeompoint 'Interp=Step;
  [Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- 0
SELECT length(geometry 'Linestring(0 0,3 4)');
-- 5
SELECT round(length(geography 'Linestring(4.35 50.85,4.40 50.85)'), 6);
-- 3521.188745
SELECT round(length(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  false), 3);
-- 111195.08
```

- Devuelve la longitud acumulada atravesada por el punto temporal

```
cumulativeLength(tpoint) → tfloatSeq
cumulativeLength(tgeogpoint,spheroid boolean=true) → tfloatSeq
```

Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT round(cumulativeLength(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}'), 6);
-- {[0@2001-01-01, 1.414214@2001-01-02, 2.414214@2001-01-03],
  [2.414214@2001-01-04, 3.414214@2001-01-05]}
SELECT cumulativeLength(tgeompoint 'Interp=Step;
  [Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- Interp=Step;[0@2001-01-01, 0@2001-01-03]
SELECT round(cumulativeLength(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  false), 3);
-- [0@2001-01-01, 111195.08@2001-01-02]
```

- Devuelve la velocidad del punto temporal en unidades por segundo

```
speed(tpoint) → tfloatSeqSet
speed(tgeogpoint,spheroid boolean=true) → tfloatSeqSet
```

El punto temporal debe tener interpolación linear. Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT speed(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}') * 3600 * 24;
/* Interp=Step;{[1.41421356237731@2001-01-01, 1@2001-01-02, 1@2001-01-03],
   [1@2001-01-04, 1@2001-01-05]} */
```

```
SELECT speed(tgeompoint 'Interp=Step;
  [Point(0 0)@2001-01-01, Point(1 1)@2001-01-02, Point(1 0)@2001-01-03]');
-- ERROR:  The temporal value must have linear interpolation
SELECT round(speed(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  false), 6);
-- Interp=Step;[1.28698@2001-01-01, 1.28698@2001-01-02]
```

- Devuelve el centroide ponderado en el tiempo

```
twCentroid(tgeompoint) → point
```

```
SELECT ST_AsText(twCentroid(tgeompoint '{[Point(0 0 0)@2001-01-01,
  Point(0 1 1)@2001-01-02, Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04)}'));
-- POINT Z (0 0.666666666666667 0.666666666666667)
```

- Devuelve la dirección, es decir, el acimut entre las ubicaciones inicial y final

```
direction(tpoint) → float
```

El resultado se expresa en radianes. Es NULL si solo hay una ubicación o si las ubicaciones inicial y final son iguales.

```
SELECT round(degrees(direction(tgeompoint '[Point(0 0)@2001-01-01,
  Point(-1 -1)@2001-01-02, Point(1 1)@2001-01-03]'))::numeric, 6);
-- 45.000000
SELECT direction(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(0 1 1)@2001-01-02,
  Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04)}');
-- NULL
```

- Devuelve el acimut temporal

```
azimuth(tpoint) → tfloat
```

El resultado se expresa en radianes. El azimut es indefinido cuando dos localizaciones sucesivas son iguales y en este caso se añade una brecha de tempo.

```
SELECT round(degrees(azimuth(tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-02, Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04)')));
-- Interp=Step;{[45@2001-01-01, 45@2001-01-02], [225@2001-01-03, 225@2001-01-04)}
```

- Devuelve la diferencia angular temporal

```
angularDifference(tpoint) → tfloat
```

El resultado se expresa en grados.

```
SELECT round(angularDifference(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]'), 3);
-- {0@2001-01-01, 180@2001-01-02, 0@2001-01-03}
SELECT round(degrees(angularDifference(tgeompoint '{[Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-02], [Point(2 2)@2001-01-03, Point(1 1)@2001-01-04]}')), 3);
-- {0@2001-01-01, 0@2001-01-02, 0@2001-01-03, 0@2001-01-04}
```

- Devuelve el rumbo temporal

```
bearing({tpoint,point},{tpoint,point}) → tfloat
```

Nótese que esta función no acepta dos puntos geográficos temporales.

```
SELECT degrees(bearing(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]',
  geometry 'Point(2 2)'));
-- [45@2001-01-01, 0@2001-01-02, 225@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]',
  tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]')), 3);
--  [63.435@2001-01-01, 0@2001-01-02, 296.565@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]',
  tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]')), 3);
-- [243.435@2001-01-01, 116.565@2001-01-03]
```

7.7. Transformaciones

- Redondea los valores de las coordenadas a un número de decimales

```
round(tspatial,integer=0) → tspatial
```

```
SELECT asText(round(tgeompoint '{Point(1.12345 1.12345 1.12345)@2001-01-01,
  Point(2 2 2)@2001-01-02, Point(1.12345 1.12345 1.12345)@2001-01-03}', 2));
/* {POINT Z (1.12 1.12 1.12)@2001-01-01, POINT Z (2 2 2)@2001-01-02,
   POINT Z (1.12 1.12 1.12)@2001-01-03} */
SELECT asText(round(tgeography 'Linestring(1.12345 1.12345,2.12345 2.12345)@2001-01-01',
  2));
-- LINESTRING(1.12 1.12,2.12 2.12)@2001-01-01
```

- Devuelve una matriz de fragmentos del punto temporal que son simples

```
makeSimple(tpoint) → tgeompoint[]
```

```
SELECT asText(makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 0)@2001-01-03]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02)",
   "[POINT(1 1)@2001-01-02, POINT(0 0)@2001-01-03]"} */
SELECT asText(makeSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]'));
/* {"[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 0 2)@2001-01-03,
   POINT Z (0 0 0)@2001-01-04]"} */
SELECT asText(makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 1)@2001-01-03, Point(1 0)@2001-01-04]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02, POINT(0 1)@2001-01-03)",
   "[POINT(0 1)@2001-01-03, POINT(1 0)@2001-01-04]"} */
SELECT asText(makeSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}'));
/* {"{[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02],
   [POINT Z (1 1 1)@2001-01-03, POINT Z (0 0 0)@2001-01-04]}"} */
```

- Construye una geometría/geografía con medida M a partir de un punto temporal y un número flotante temporal

```
geoMeasure(tpoint,tfloat,segmentize boolean=false) → geo
```

El último argumento `segmentize` establece si el valor resultado ya sea es un `Linestring M` o un `MultiLinestring M` donde cada componente es un segmento de dos puntos.

```
SELECT ST_AsText(geoMeasure(tgeompoint '{Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02}', '{5@2001-01-01, 5@2001-01-02}'));
-- MULTIPOINT ZM (1 1 1 5,2 2 2 5)
SELECT ST_AsText(geoMeasure(tgeogpoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
```

```
  [Point(1 1)@2001-01-03, Point(1 1)@2001-01-04]}',
  '{[5@2001-01-01, 5@2001-01-02],[7@2001-01-03, 7@2001-01-04]}'));
-- GEOMETRYCOLLECTION M (POINT M (1 1 7),LINESTRING M (1 1 5,2 2 5))
SELECT ST_AsText(geoMeasure(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]', '[5@2001-01-01, 7@2001-01-02, 5@2001-01-03]', true));
-- MULTILINESTRING M ((1 1 5,2 2 5),(2 2 7,1 1 7))
```

Una visualización típica de los datos de movilidad es mostrar en un mapa la trayectoria del objeto móvil utilizando diferentes colores según la velocidad. La Figura 7.3 muestra el resultado de la consulta a continuación usando una rampa de color en QGIS.

```
WITH Temp(t) AS (
  SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-05, Point(2 0)@2001-01-08,
    Point(3 1)@2001-01-10, Point(4 0)@2001-01-11]' )
SELECT ST_AsText(geoMeasure(t, round(speed(t) * 3600 * 24, 2), true)) FROM Temp;
/* MULTILINESTRING M ((0 0 0.35,1 1 0.35),(1 1 0.47,2 0 0.47),(2 0 0.71,3 1 0.71),
   (3 1 1.41,4 0 1.41)) */
```

La siguiente expresión se usa en QGIS para lograr esto. La función `scale_linear` transforma el valor M de cada segmento componente al rango [0, 1]. Este valor luego se pasa a la función `ramp_color`.

```
ramp_color('RdYlBu', scale_linear(
    m(start_point(geometry_n($geometry,@geometry_part_num))), 0, 2, 0, 1) )
```



Figura 7.3: Visualización de la velocidad de un objeto móvil usando una rampa de color en QGIS.

- Devuelve una geometría temporal sin Z o con una Z, como lo hacen las funciones de PostGIS `ST_Force2D` y `ST_Force3D` para una geometría

```
force2D(tgeo) → tgeo
force3D(tgeo,zvalue float=0.0) → tgeo
```

El tipo y la interpolación del valor se mantienen. `force3D` da la Z de un valor que no la tiene, y una Z que el valor tiene se mantiene. `force2D` puede dejar instantes que un movimiento lineal en dos dimensiones no necesita, y el resultado se normaliza, por lo que puede tener menos instantes. Estas funciones eligen las dimensiones que lee una relación entre dos valores, véase Sección 8.3. Como una geografía se mide sobre la superficie de la Tierra, su Z no cambia sus distancias.

```
SELECT asEWKT(force2D(tgeompoint '[Point(0 0 7)@2001-01-01, Point(2 0 7)@2001-01-03]'));
-- [POINT(0 0)@2001-01-01, POINT(2 0)@2001-01-03]
SELECT asEWKT(force3D(tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]', 50));
-- [POINT Z (0 0 50)@2001-01-01, POINT Z (2 0 50)@2001-01-03]
SELECT force3D(tgeompoint '[Point(0 0)@2001-01-01, Point(10 0)@2001-01-02]') |=|
  tgeompoint '[Point(0 0 10000)@2001-01-01, Point(10 0 10000)@2001-01-02]';
-- 10000
```

- Devuelve la transformación afín 3D de una geometría temporal para traducirla, rotarla y/o escalarla en un solo paso

```
affine(tgeo,a float,b float,c float,d float,e float,f float,g float,
  h float,i float,xoff float,yoff float,zoff float) → tgeo
affine(tgeo,a float,b float,d float,e float,xoff float,yoff float) → tgeo
affine(geometry,a float,b float,c float,d float,e float,f float,g float,
  h float,i float,xoff float,yoff float,zoff float) → geometry
affine(geometry,a float,b float,d float,e float,xoff float,yoff float) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano. Un valor bidimensional que la transformación saca de su plano se vuelve tridimensional, como una pose.

```
-- Rotate a 3D temporal point 180 degrees about the z axis
SELECT asEWKT(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), 0, 0, 0, 1,
  0, 0, 0))
FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01, POINT(1 4 3)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
SELECT asEWKT(rotate(temp, pi())) FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01,
  POINT(1 4 3)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
-- Rotate a 3D temporal point 180 degrees in both the x and z axis
SELECT asEWKT(round(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), -sin(pi() ↩
    ),
   0, sin(pi()), cos(pi()), 0, 0, 0), 6)) FROM (SELECT tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -1 0)@2001-01-01, LINESTRING Z (-1 -1 0,-2 -2 0)@2001-01-02]
SELECT ST_AsEWKT(affine(geometry 'Point(1 2 3)', cos(pi()), -sin(pi()), 0,
  sin(pi()), cos(pi()), 0, 0, 0, 1, 0, 0, 0));
-- POINT(-1 -2 3)
```

- Devuelve el punto temporal rotado en sentido antihorario sobre el punto de origen

```
rotate(tgeo,radians float) → tgeo
rotate(tgeo,radians float,x0 float,y0 float) → tgeo
rotate(tgeo,radians float,origin geometry) → tgeo
rotate(geometry,radians float) → geometry
rotate(geometry,radians float,x0 float,y0 float) → geometry
rotate(geometry,radians float,origin geometry) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano.

Un origen dado como geometría es un punto no vacío que declara el SRID del valor temporal o ninguno; de lo contrario se produce un error.

```
-- Rotate a temporal point 180 degrees
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()), 6);
-- [POINT(-5 -10)@2001-01-01, POINT(-5 -5)@2001-01-02, POINT(-10 -5)@2001-01-03]
-- Rotate 30 degrees counter-clockwise at x=5, y=10
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()/6, 5, 10), 6);
-- [POINT(5 10)@2001-01-01, POINT(7.5 5.67)@2001-01-02, POINT(11.83 8.17)@2001-01-03]
-- Rotate 60 degrees clockwise from centroid
SELECT asEWKT(rotate(temp, -pi()/3, ST_Centroid(traversedArea(temp))),
  2) FROM (SELECT tgeometry '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Linestring(5 5,10 5)@2001-01-03]' AS temp) AS t;
/* [POINT(10.58 9.67)@2001-01-01, POINT(6.25 7.17)@2001-01-02,
   LINESTRING(6.25 7.17,8.75 2.83)@2001-01-03] */
SELECT ST_AsEWKT(rotate(geometry 'Linestring(5 10,5 5)', pi()/6, 5, 10), 2);
-- LINESTRING(5 10,7.5 5.67)
```

- Devuelve una geometría temporal rotada en sentido antihorario alrededor del eje x 

```
rotateX(tgeo,radians float) → tgeo
rotateX(geometry,radians float) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano. Un valor bidimensional que la transformación saca de su plano se vuelve tridimensional, como una pose.

```
SELECT asEWKT(rotateX(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 4 3)@2001-01-02]',
  pi()), 6);
-- [POINT Z (1 -2 -3)@2001-01-01, POINT Z (1 -4 -3)@2001-01-02]
SELECT ST_AsEWKT(round(rotateX(geometry 'Point(1 1)', pi()/2), 6));
-- POINT(1 0 1)
SELECT asEWKT(round(rotateX(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]',
  pi()/2), 6));
-- [POINT Z (1 0 1)@2001-01-01, POINT Z (2 0 2)@2001-01-02]
```

- Devuelve una geometría temporal rotada en sentido antihorario alrededor del eje y

```
rotateY(tgeo,radians float) → tgeo
rotateY(geometry,radians float) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano. Un valor bidimensional que la transformación saca de su plano se vuelve tridimensional, como una pose.

```
SELECT asEWKT(rotateY(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 4 3)@2001-01-02]',
  pi()), 6);
-- [POINT Z (-1 2 -3)@2001-01-01, POINT Z (-1 4 -3)@2001-01-02]
SELECT ST_AsEWKT(round(rotateY(geometry 'Linestring(1 1,2 2)', pi()/2), 6));
-- LINESTRING(0 1 -1,0 2 -2)
```

- Devuelve una geometría temporal rotada en sentido antihorario alrededor del eje z

```
rotateZ(tgeo,radians float) → tgeo
rotateZ(geometry,radians float) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano.

```
SELECT asEWKT(rotateZ(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
  pi()/2), 6);
-- [POINT(-1 1)@2001-01-01, LINESTRING(-1 1,-2 2)@2001-01-02]
SELECT ST_AsEWKT(rotateZ(geometry 'Linestring(1 1,2 2)', pi()/2), 6);
-- LINESTRING(-1 1,-2 2)
```

- Devuelve un punto temporal escalado por factores dados

```
scale(tgeo,xfactor float,yfactor float,zfactor float) → tgeo
scale(tgeo,xfactor float,yfactor float) → tgeo
scale(tgeo,factor geometry) → tgeo
scale(tgeo,factor geometry,origin geometry) → tgeo
scale(geometry,xfactor float,yfactor float,zfactor float) → geometry
scale(geometry,xfactor float,yfactor float) → geometry
scale(geometry,factor geometry) → geometry
scale(geometry,factor geometry,origin geometry) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano. Un valor bidimensional que la transformación saca de su plano se vuelve tridimensional, como una pose.

```
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]', 0.5,
  0.75, 0.8));
--  [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]', 0.5,
  0.75));
-- [POINT Z (0.5 1.5 3)@2001-01-01, POINT Z (0.5 0.75 1)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
  geometry 'Point(0.5 0.75 0.8)'));
-- [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
```

```
SELECT asEWKT(scale(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
  geometry 'Point(2 2)', geometry 'Point(1 1)'));
-- [POINT(1 1)@2001-01-01, LINESTRING(1 1,3 3)@2001-01-02]
SELECT ST_AsEWKT(scale(geometry 'Linestring(1 1,2 2)', geometry 'Point(2 2)',
  geometry 'Point(1 1)'));
-- LINESTRING(1 1,3 3)
```

- Devuelve una geometría temporal trasladada por desplazamientos dados y luego escalada por factores dados

```
transscale(tgeo,deltax float,deltay float,xfactor float,yfactor float) → tgeo
transscale(geometry,deltax float,deltay float,xfactor float,yfactor float) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano.

```
SELECT asEWKT(transscale(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]',
  1, 1, 2, 3));
-- [POINT(4 6)@2001-01-01, POINT(6 9)@2001-01-02]
SELECT ST_AsEWKT(transscale(geometry 'Point(1 1)', 1, 1, 2, 3));
-- POINT(4 6)
```

- Devuelve una geometría temporal trasladada por desplazamientos dados

```
translate(tgeo,deltax float,deltay float) → tgeo
translate(tgeo,deltax float,deltay float,deltaz float) → tgeo
translate(geometry,deltax float,deltay float) → geometry
translate(geometry,deltax float,deltay float,deltaz float) → geometry
```

Sobre una geometría, la función responde como su equivalente de PostGIS para una geometría que la transformación mantiene en su plano. Un valor bidimensional que la transformación saca de su plano se vuelve tridimensional, como una pose.

```
SELECT asEWKT(translate(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]',
  10, 20));
-- [POINT(11 21)@2001-01-01, POINT(12 22)@2001-01-02]
SELECT asEWKT(translate(tgeompoint '[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02]',
  10, 20, 30));
-- [POINT Z (11 21 31)@2001-01-01, POINT Z (12 22 32)@2001-01-02]
SELECT ST_AsEWKT(translate(geometry 'Point(1 1)', 10, 20));
-- POINT(11 21)
SELECT ST_AsEWKT(translate(geometry 'Point(1 1)', 0, 0, 5));
-- POINT(1 1 5)
```

- Transforma un punto geométrico temporal en el espacio de coordenadas de un Mapbox Vector Tile

```
asMVTGeom(tpoint,bounds stbox,extent integer=4096,buffer integer=256,
  clip boolean=true) → (geom,times)
```

El resultado es un par compuesto de un valor `geometry` y una matriz de valores de marca de tiempo asociados codificados como época de Unix. Los parámetros son los siguientes:

- `tpoint` es el punto temporal para transformar
- `bounds` es un `stbox` que define los límites geométricos del contenido del mosaico sin búfer
- `extent` es la extensión del mosaico en el espacio de coordenadas del mosaico
- `buffer` es la distancia del búfer en el espacio de coordenadas de mosaico
- `clip` es un booleano que determina si las geometrías resultantes y las marcas de tiempo deben recortarse o no

```
SELECT ST_AsText((mvt).geom),
  (mvt).times FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01,
  Point(100 100)@2001-01-02]', stbox 'STBOX X((40,40),(60,60))') AS mvt ) AS t;
-- LINESTRING(-256 4352,4352 -256) | {946714680,946734120}
SELECT ST_AsText((mvt).geom), (mvt).times
FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01, Point(100 100)@2001-01-02]',
  stbox 'STBOX X((40,40),(60,60))', clip:=false) AS mvt ) AS t;
-- LINESTRING(-8192 12288,12288 -8192) | {946681200,946767600}
```

- Extrae de un punto temporal con interpolación lineal las subsecuencias donde el punto permanece dentro de un área con un tamaño máximo especificado durante al menos la duración dada

```
stops(tpoint,maxdist float=0.0,minduration interval='0 minutes') → tpoint
```

El tamaño del área es la mayor distancia entre dos puntos de la subsecuencia, que es la cantidad que la forma de flotante temporal de la función lee como la diferencia entre el mayor y el menor valor que toma. Si no se especifica `maxDist`, se asume 0.0 y, por lo tanto, la función extrae los segmentos constantes del punto temporal dado. La distancia se calcula en las unidades del sistema de coordenadas para un `tgeompoint` y en metros sobre el esferoide WGS84 para un `tgeogpoint`. Tenga en cuenta que, aunque la función acepta geometrías 3D, el cálculo siempre se realiza en 2D. Una parada que comienza en el primer instante de una secuencia incluye ese instante solo si la secuencia lo incluye, y lo mismo vale para una parada que termina en su último instante. Las paradas se buscan en orden temporal: cada una comienza en el primer instante a partir del cual el valor permanece dentro de un área del tamaño máximo durante la duración mínima, y termina en el último instante antes de salir del área.

```
SELECT asText(stops(tgeompoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-02,
  Point(2 2)@2001-01-03, Point(2 2)@2001-01-04]'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(2 2)@2001-01-03,
   POINT(2 2)@2001-01-04]} */
SELECT asText(stops(tgeompoint '[Point(1 1 1)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 2 2)@2001-01-03, Point(2 2 2)@2001-01-04]', 1.75));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03,
   POINT Z (2 2 2)@2001-01-04]} */
```

Capítulo 8

Tipos temporales geométricos (Parte 2)

8.1. Restricciones

- Restringe a (al complemento de) una geometría

```
atGeometry(tgeom,geometry) → tgeom
minusGeometry(tgeom,geometry) → tgeom
```

La geometría debe ser 2D y el cálculo con respecto a ella se realiza en 2D. El resultado conserva la dimensión Z del punto temporal, si existe.

Un punto geodésico se restringe a una `geography`, que debe ser un polígono o un multipolígono, y el cálculo se realiza sobre la esfera: el punto recorre el círculo máximo entre dos posiciones y la región está delimitada por los círculos máximos que pasan por sus propios vértices, de modo que el viaje entra y sale donde esos caminos se cruzan. La diferencia crece con la latitud, ya que un viaje a lo largo de un paralelo se sitúa bastante al norte de él.

```
SELECT asText(atGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {"[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]"}
SELECT astext(atGeometry(tgeompoint '[Point(0 0 0)@2001-01-01, Point(4 4 4)@2001-01-05]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {[POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03]}
SELECT asText(atGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (2 1 1)@2001-01-02, POINT Z (3 1 1)@2001-01-03, POINT Z (3 1 3)@2001-01-05]}
SELECT asText(atGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(1 1,5 1)@2001-01-01
```

```
SELECT asText(minusGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02), (POINT(2 2)@2001-01-03,
   POINT(3 3)@2001-01-04)} */
SELECT astext(minusGeometry(tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(4 4 4)@2001-01-05]', geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
   (POINT Z (2 2 2)@2001-01-03, POINT Z (4 4 4)@2001-01-05]} */
SELECT asText(minusGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 1 1)@2001-01-02)}
SELECT asText(minusGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(5 1,10 1)@2001-01-01
```

```
SELECT asText(atGeometry(tgeogpoint 'SRID=4326;[Point(-170 65)@2001-01-01,
  Point(-100 65)@2001-01-03]',
  geography 'SRID=4326;Polygon((-150 60,-150 70,-120 70,-120 60,-150 60))'), 6);
/* {[POINT(-150 68.423603)@2001-01-01 14:39:22.550164,
   POINT(-120 68.423603)@2001-01-02 09:20:37.449835]} */
```

- Restringe a (al complemento de) un `stbox`

```
atStbox(tgeom,stbox,borderInc boolean=true) → tgeompoint
minusStbox(tgeom,stbox,borderInc boolean=true) → tgeompoint
```

El tercer argumento opcional se utiliza para mosaicos multidimensionales (ver Sección 9.6) para excluir el borde superior de los mosaicos cuando un valor temporal se divide en varios mosaicos, de modo que todos los fragmentos de la geometría temporal sean exclusivos.

Un punto temporal se restringe en 3D. La dimensión Z se tiene en cuenta únicamente cuando tanto el punto temporal como la caja la tienen, véase Sección 8.3, y el resultado conserva la dimensión Z del punto temporal. El cuerpo de una geometría temporal, en cambio, se recorta en 2D: una caja que contiene coordenadas rechaza una geometría temporal que tiene dimensión Z, mientras que una caja que contiene únicamente un período la restringe en el tiempo como para cualquier otro tipo temporal. La restricción de un `tgeography` a una caja espaciotemporal no está implementada.

Un punto temporal geodésico recorre el círculo máximo entre dos posiciones, y la caja lo contiene donde ese círculo entra en ella y sale de ella: un viaje de `Point(10 60)` a `Point(50 60)` alcanza la latitud 61.518762, y un viaje que cruza el antimeridiano recorre las longitudes por el camino corto. Dos cajas que se encuentran en un cruce indican el mismo instante para él, de modo que las piezas de un viaje contienen el período del viaje y sus posiciones.

```
SELECT asText(atStbox(tgeogpoint '[Point(10 60)@2001-01-01, Point(50 60)@2001-01-02]',
  stbox 'SRID=4326;GEODSTBOX X((0 55),(60 61))', false), 6);
/* {[POINT(10 60)@2001-01-01, POINT(18.168064 61)@2001-01-01 05:02:50.433397),
   [POINT(41.831936 61)@2001-01-01 18:57:09.566602, POINT(50 60)@2001-01-02]} */
```

```
SELECT asText(atStbox(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)',
  stbox 'STBOX XT(((0,0),(2,2)),[2001-01-02, 2001-01-04])'));
-- {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]}
SELECT asText(atStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2),(3,3,3))'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03, POINT Z (3 3 2)@2001-01-04,
   POINT Z (3 3 3)@2001-01-05]} */
SELECT asText(atStbox(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,3 3)@2001-01-03,
  Point(2 2)@2001-01-04, Linestring(3 3,4 4)@2001-01-09]', stbox 'STBOX X((2,2),(3,3))'));
-- {[LINESTRING(2 2,3 3)@2001-01-03, POINT(2 2)@2001-01-04, POINT(3 3)@2001-01-09]}
```

```
SELECT asText(minusStbox(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04)',
  stbox 'STBOX XT(((1,1),(2,2)),[2001-01-03,2001-01-04])'));
-- {(POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03)}
SELECT asText(minusStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2),(3,3,3))'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02),
   (POINT Z (3 3 3)@2001-01-05, POINT Z (3 3 7)@2001-01-09]} */
SELECT asText(minusStbox(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,3 3)@2001-01-03, Point(2 2)@2001-01-04, Linestring(1 1,4 4)@2001-01-09]',
  stbox 'STBOX X((2,2),(3,3))'));
/* {[POINT(1 1)@2001-01-01, LINESTRING(1 1,2 2)@2001-01-03,
   LINESTRING(1 1,2 2)@2001-01-04),[MULTILINESTRING((1 1,2 2),(3 3,4 4))@2001-01-09]} */
```

- Restringe a (al complemento de) un rango de altitud

```
atElevation(tpoint,floatspan) → tpoint
minusElevation(tpoint,floatspan) → tpoint
```

La altitud de un punto temporal es su coordenada Z, que varía linealmente en el tiempo tanto si el punto es planar como si es geodésico, de modo que los períodos que el rango admite son los mismos para ambos. Las posiciones en los límites de esos períodos no lo son: un punto geodésico recorre el círculo máximo entre dos posiciones y alcanza allí la altitud de un límite, y no sobre la línea recta en longitud y latitud.

```
SELECT astext(atElevation(tgeompoint '[Point(1 1 1)@2001-01-01, Point(4 4 4)@2001-01-04,
  Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03],
   [POINT Z (3 3 3)@2001-01-05, POINT Z (2 2 2)@2001-01-06]} */
SELECT astext(minusElevation(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02),
   (POINT Z (3 3 3)@2001-01-03, POINT Z (4 4 4)@2001-01-04,
   POINT Z (3 3 3)@2001-01-05), (POINT Z (2 2 2)@2001-01-06,
   POINT Z (1 1 1)@2001-01-07]} */
```

```
SELECT asText(atElevation(tgeogpoint 'SRID=4326;[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'), 6);
/* {[POINT Z (1.998577 2.00071 2)@2001-01-02, POINT Z (2.998373 3.000812 3)@2001-01-03],
   [POINT Z (2.998373 3.000812 3)@2001-01-05, POINT Z (1.998577 2.00071 2)@2001-01-06]} */
```

8.2. Sistema de referencia espacial

- Devuelve o establece el identificador de referencia espacial

```
SRID(tspatial) → integer
setSRID(tspatial,integer) → tspatial
```

```
SELECT SRID(tgeompoint 'Point(0 0)@2001-01-01');
-- 0
SELECT asEWKT(setSRID(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02)', 4326));
-- SRID=4326;[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02)
```

- Transforma a una referencia espacial diferente

```
transform(tspatial,integer) → tspatial
transformPipeline(tspatial,pipeline text,srid integer,
  is_forward boolean=true) → tspatial
transformPipeline(geo,pipeline text,srid integer,
  is_forward boolean=true) → geo
```

La función `transformPipeline` transforma de la misma manera una geometría o una geografía con una canalización.

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el punto temporal tiene un SRID desconocido (representado por 0).

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID del punto temporal de entrada se ignora y el SRID del punto temporal de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(tgeompoint 'SRID=4326;Point(4.35 50.85)@2001-01-01', 3812));
-- SRID=3812;POINT(648679.018035303 671067.055638114)@2001-01-01
SELECT ST_AsText(transformPipeline(geometry 'SRID=4326;Point(2 49)',
  'urn:ogc:def:coordinateOperation:EPSG::16031'), 3);
-- POINT(426857.988 5427937.523)
```

```
WITH test(tgeo, pipeline) AS (
  SELECT tgeogpoint 'SRID=4326;{Point(4.3525 50.846667 100.0)@2001-01-01,
    Point(-0.1275 51.507222 100.0)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(tgeo, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
/* SRID=4326;{POINT Z (4.3525 50.846667 100)@2001-01-01,
   POINT Z (-0.1275 51.507222 100)@2001-01-02} */
```

8.3. Operandos de dimensiones distintas

Una operación que relaciona dos valores espaciales lee únicamente las dimensiones que ambos tienen. Dos valores con Z se leen en 3D. Si uno de ellos no tiene Z, ambos se leen en 2D, se ignora la Z del otro, y no se produce ni un error ni un aviso. El orden de los operandos no cambia la respuesta.

```
-- A car with no Z and a plane above it are at the distance of the map
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(10 0)@2001-01-02]' |=|
  tgeompoint '[Point(0 0 10000)@2001-01-01, Point(10 0 10000)@2001-01-02]';
-- 0
-- Two values with Z are read in 3D
SELECT tgeompoint '[Point(0 0 0)@2001-01-01, Point(10 0 0)@2001-01-02]' |=|
  tgeompoint '[Point(0 0 10000)@2001-01-01, Point(10 0 10000)@2001-01-02]';
-- 10000
-- An operation building a value requires the same dimensions
SELECT merge(tgeompoint 'Point(0 0)@2001-01-01', tgeompoint 'Point(0 0 0)@2001-01-02');
-- ERROR:  The arguments must be of the same dimensionality
-- Two reference systems are not comparable
SELECT tgeompoint 'SRID=3857;Point(0 0)@2001-01-01' |=|
  tgeompoint 'SRID=4326;Point(0 0)@2001-01-01';
-- ERROR:  Operation on mixed SRID
```

- La regla abarca las distancias (`|=|`, `<->`, `nearestApproachInstant`, `shortestLine`, `distance`, `maxDistance`, y las distancias de similitud), la distancia dentro de un umbral (`eDwithin`, `aDwithin`, `tDwithin`), las relaciones `intersects` y `disjoint` en sus formas alguna vez, siempre y temporal, y los operadores de cuadro delimitador. El instante de máxima aproximación conserva la Z del valor del que se toma, y la línea más corta de operandos de dimensiones distintas no tiene Z.
- Una operación que construye o compara valores requiere las mismas dimensiones y produce un error en caso contrario. Es el caso de la igualdad y la comparación, de `merge` y los constructores, de la restricción a un valor, de los agregados, de la unión y la intersección de cuadros, y de las teselas.
- Una operación sin definición en 3D, como `contains`, `covers` o `touches`, produce un error con cualquier operando con Z.
- Un tipo de dos dimensiones, como un búfer circular temporal o una geometría rígida temporal, lee la X y la Y de un operando 3D de sus distancias. Sus demás relaciones producen un error con un operando con Z.
- Una geografía se mide sobre la superficie de la Tierra: su Z no se lee, tenga o no una el otro operando.
- Los valores en dos sistemas de referencia espacial siempre producen un error, ya que sus unidades no son comparables.

Como la regla no produce ningún error, un valor al que le falta su Z por equivocación responde en 2D sin ningún aviso. Las funciones [?listitem] indican las dimensiones que se leen. Una columna declarada con dimensión Z, como `tgeompoint(Sequence, PointZ)`, no acepta un valor sin Z, de modo que una tabla no puede contenerlo.

8.4. Geometrías vacías

Una geometría vacía es el conjunto vacío de puntos: no está en ninguna parte. Como en PostGIS, una operación le da la respuesta que da el conjunto vacío, y no se produce ni un error ni un aviso, salvo donde la geometría debe ser la posición de un valor.

```
-- It meets nothing
SELECT eIntersects(geometry 'Point empty',
  tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]');
-- f
SELECT tDisjoint(geometry 'Point empty',
  tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]');
-- [t@2001-01-01, t@2001-01-03]
-- Nothing can be measured to it
SELECT geometry 'Point empty' <->
  tgeompoint '[Point(2 2)@2001-01-01, Point(1 1)@2001-01-02, Point(2 2)@2001-01-03]';
-- NULL
-- It removes nothing and keeps nothing
SELECT asText(minusGeometry(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]', geometry 'Linestring empty'));
-- [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(1 1)@2001-01-03]
SELECT asText(atGeometry(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]', geometry 'Linestring empty'));
-- NULL
-- It cannot be the position of a value
SELECT tgeompoint(geometry 'Point empty', timestamptz '2001-01-01');
-- ERROR:  Only non-empty geometries accepted
```

- **No toca nada.** Una relación que pide un punto común, como `intersects`, `contains`, `covers`, `touches`, la distancia dentro de un umbral y la igualdad, es falsa, mientras que `disjoint` y la desigualdad son verdaderas. Una relación o comparación temporal toma esta respuesta durante todo el tiempo del valor temporal, y las formas alguna vez y siempre responden lo mismo.
- **Nada puede medirse hasta ella.** Las distancias (`|=|`, `<->`, `nearestApproachInstant`, `shortestLine`) y `bearing` son `NULL`. No tiene cuadro delimitador, de modo que `stbox`, `expandSpace`, `stboxes`, `splitNStboxes` y `splitEachNStboxes` son `NULL`.
- **No quita nada y no conserva nada.** Las restricciones `atGeometry` y `atValue` son `NULL`, ya que no queda ningún instante, y `minusGeometry` y `minusValue` devuelven el valor temporal sin cambios.
- **No puede ser la posición de un valor.** Construir un valor temporal o un conjunto a partir de ella, o darla como origen de `rotate` y `scale`, produce un error.

8.5. Operaciones de cuadro delimitador

- Devuelve el cuadro delimitador espaciotemporal expandido en la dimensión espacial por un valor flotante

```
expandSpace({spatial,tspatial},float) → stbox
```

```
SELECT expandSpace(geography 'Linestring(0 0,1 1)', 2);
-- SRID=4326;GEODSTBOX X((-2,-2),(3,3))
SELECT expandSpace(tgeompoint 'Point(0 0)@2001-01-01', 2);
-- STBOX XT(((-2,-2),(2,2)),[2001-01-01,2001-01-01])
```

8.6. Operaciones de distancia

Los operandos de dimensiones distintas se leen en las dimensiones que ambos tienen, véase Sección 8.3.

- Devuelve la distancia más pequeña que haya existido

```
{geo,tgeo,stbox} |=| {geo,tgeo,stbox} → float
nearestApproachDistance({geography,tgeog,stbox},{geography,tgeog,stbox},
  spheroid boolean=true) → float
```

Para geografías, el operador calcula en el esferoide WGS 84, y la función `nearestApproachDistance` calcula en la esfera, más rápido, si su último argumento es falso.

```
SELECT tgeompoint '[Point(0 0)@2001-01-02, Point(1 1)@2001-01-04,
  Point(0 0)@2001-01-06)' |=| geometry 'Linestring(2 2,2 1,3 1)';
-- 1
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03, Point(0 0)@2001-01-05)'
  |=| tgeompoint '[Point(2 0)@2001-01-02, Point(1 1)@2001-01-04, Point(2 2)@2001-01-06)';
-- 0.5
SELECT tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05)' |=| tgeompoint '[Point(2 0 0)@2001-01-02,
  Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06)';
-- 0.5
SELECT tgeometry '(Point(1 1)@2001-01-01,
  Linestring(3 1,1 1)@2001-01-03]' |=| geometry 'Linestring(1 3,2 2,3 3)';
-- 1
SELECT round((tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]' |=|
  geography 'Point(0 0)')::numeric, 3);
-- 110574.389
SELECT round(nearestApproachDistance(
  tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  geography 'Point(0 0)', false)::numeric, 3);
-- 111195.080
```

El operador `|=|` se puede utilizar para realizar una búsqueda de vecino más cercano utilizando un índice GiST o SP-GIST (ver la Sección 10.2). Esta función corresponde a la función `ST_DistanceCPA` proporcionada por PostGIS, aunque este última requiere que ambos argumentos sean una trayectoria.

Cuando un argumento es una caja espaciotemporal que lleva un período, la distancia se mide desde la parte del valor temporal que está dentro de ese período, y el resultado es `NULL` cuando ninguna parte de él lo está. Una caja que no lleva período mide el valor temporal entero.

```
SELECT ST_DistanceCPA(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05)'::geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
  Point(2 2 2)@2001-01-06)'::geometry);
-- 0.5
```

- Devuelve el instante del primer punto temporal en el que los dos argumentos están a la distancia más cercana

```
nearestApproachInstant({geo,tgeo},{geo,tgeo}) → tgeo
nearestApproachInstant({geography,tgeog},{geography,tgeog},spheroid boolean=true) → tgeog
```

La función sólo devuelve el primer instante que encuentre si hay más de uno. El instante resultante puede tener un límite exclusivo. Para geografías, los puntos más cercanos de cada segmento se localizan en la esfera y sus distancias se comparan en el esferoide WGS 84 por defecto, como la función `ST_Distance` de PostGIS mide una distancia entre geografías. Si el último argumento es falso, se comparan en la esfera.

```
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(2 1)@2001-01-02
```

```
SELECT asText(nearestApproachInstant(tgeompoint 'Interp=Step;
  (Point(1 1)@2001-01-01, Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(1 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-03]', tgeompoint '(Point(1 1)@2001-01-01, Point(4 1)@2001-01-03]'));
-- POINT(1 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeometry
  '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
  '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,1 1 1)@2001-01-02
SELECT asText(nearestApproachInstant(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)'));
-- POINT(0 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)', false));
-- POINT(0.998 0)@2001-01-02
```

La función `nearestApproachInstant` generaliza the la función PostGIS `ST_ClosestPointOfApproach`. Primero, la última función requiere que ambos argumentos sean trayectorias. Segundo, la función `nearestApproachInstant` devuelve tanto el punto como la marca de tiempo del punto de aproximación más cercano, mientras que la función PostGIS sólo proporciona la marca de tiempo como se muestra a continuación.

```
SELECT to_timestamp(ST_ClosestPointOfApproach( tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-05)'::geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
  Point(2 2 2)@2001-01-06)'::geometry));
-- 2001-01-03 12:00:00
```

- Devuelve la distancia espacial mínima entre dos conjuntos de geometrías temporales, ignorando el tiempo

```
minDistance(tgeo[],tgeo[]) → float
minDistance(tgeog[],tgeog[],spheroid boolean=true) → float
```

La distancia se mide entre las trayectorias descartando el tiempo, igual a `ST_Distance` de la trayectoria de cada argumento, y el resultado es el mínimo sobre todos los pares tomados de los dos conjuntos. Por lo tanto, dos valores que siguen el mismo camino en momentos distintos están a distancia cero. La cantidad sincronizada en el tiempo, que compara dos argumentos solo en los instantes que comparten, es nearestApproachDistance. Un solo par es el caso de un elemento. Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT minDistance(ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)',
  tgeompoint '[Point(5 5)@2001-01-01, Point(6 6)@2001-01-03)'],
  ARRAY[tgeompoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03)']);
-- 0
SELECT round(minDistance(
  ARRAY[tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]'],
  ARRAY[tgeogpoint 'Point(0 0)@2001-01-01'], false)::numeric, 3);
-- 111195.080
```

- Devuelve los pares de índices que cumplen la relación entre dos arreglos de geometrías temporales, generalizando las relaciones espaciales escalares a una unión espacial conjunto-conjunto

```
eDwithinPairs(tgeo[],tgeo[],float) → setof(i integer,j integer)
aDwithinPairs(tgeo[],tgeo[],float) → setof(i integer,j integer)
eIntersectsPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
aIntersectsPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
eDisjointPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
aDisjointPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
eTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer,j integer)
aTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer,j integer)
tDwithinPairs(tgeo[],tgeo[],float) → setof(i integer,j integer,periods tstzspanset)
tIntersectsPairs(tgeo[],tgeo[]) → setof(i integer,j integer,periods tstzspanset)
```

```
tDisjointPairs(tgeo[],tgeo[]) → setof(i integer,j integer,periods tstzspanset)
tTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer,j integer,periods tstzspanset)
eDwithinPairs(tgeog[],tgeog[],float,spheroid boolean=true) → setof(i integer,j integer)
aDwithinPairs(tgeog[],tgeog[],float,spheroid boolean=true) → setof(i integer,j integer)
```

Cada función resuelve el producto cruzado podado de los dos arreglos de entrada en una sola llamada y devuelve los índices basados en uno `i` y `j` de los pares que cumplen la relación. Las funciones con prefijo `e` devuelven los pares para los que la relación se cumple alguna vez, las funciones con prefijo `a` los pares para los que se cumple siempre, y las funciones con prefijo `t` devuelven además los períodos durante los cuales se cumple. La caja espaciotemporal de cada entrada se usa como prefiltro de caja envolvente, de modo que la relación exacta se calcula solo sobre los pares candidatos que sobreviven, y los pares cuyas extensiones temporales no se solapan se excluyen, igual que en las relaciones escalares. Las funciones están definidas para puntos temporales y geometrías temporales tanto en el caso planar como en el geodésico, excepto la relación de toca, que está definida solo para geometrías planares. Los índices se relacionan con las identidades originales mediante un `array_agg(identidad ORDER BY ...)` paralelo. Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT i, j FROM eDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]',
        tgeompoint '[Point(50 50)@2001-01-01, Point(50 60)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- 1 | 1
SELECT i, j, periods FROM tDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- one pair, with the period during which it is within the distance
SELECT i, j FROM aDisjointPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(5 0)@2001-01-01, Point(5 10)@2001-01-02]']);
-- 1 | 1
```

- Devuelve la línea que conecta el punto de aproximación más cercano

```
shortestLine({geo,tgeo},{geo,tgeo}) → geo
shortestLine({geography,tgeog},{geography,tgeog},spheroid boolean=true) → geography
```

La función sólo devolverá la primera línea que encuentre si hay más de una. Para geografías, la línea une los puntos del instante de máxima aproximación, que decide el esferoide WGS 84 por defecto y la esfera si el último argumento es falso.

```
SELECT ST_AsText(shortestLine(tgeompoint '(Point(1 1)@2001-01-01, Point(3 1)@2001-01-03]',
  geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(2 1,2 2)
SELECT ST_AsText(shortestLine(tgeompoint 'Interp=Step;
  (Point(1 1)@2001-01-01, Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(1 1,2 2)
SELECT ST_AsText(shortestLine(tgeometry
  '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
  '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,2 0 0)
SELECT ST_AsText(shortestLine(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)'));
-- LINESTRING(0 1,0 0)
SELECT ST_AsText(shortestLine(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)', false));
-- LINESTRING(0.998 0,0 0)
```

La función `shortestLine` se puede utilizar para obtener el resultado proporcionado por la función PostGIS `ST_CPAWithin` cuando ambos argumentos son trayectorias como se muestra a continuación.

```
SELECT ST_Length(shortestLine( tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-05)',
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
```

```
  Point(2 2 2)@2001-01-06')) <= 0.5;
-- true
SELECT ST_CPAWithin(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05)'::geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
  Point(2 2 2)@2001-01-06)'::geometry, 0.5);
-- true
```

El operador de distancia temporal, denotado <->, calcula la distancia en cada instante de la intersección de las extensiones temporales de sus argumentos y da como resultado un número flotante temporal. Calcular la distancia temporal es útil en muchas aplicaciones de movilidad. Por ejemplo, un grupo en movimiento (también conocido como convoy o bandada) se define como un conjunto de objetos que se mueven cerca unos de otros durante un intervalo de tiempo prolongado. Esto requiere calcular la distancia temporal entre dos objetos en movimiento.

Un punto temporal con interpolación lineal toma como otro argumento un punto temporal, un punto o, cuando es plano y 2D, una geometría de cualquier otro tipo. Un punto temporal de cualquier otra interpolación, un instante incluido, tiene un valor sólo en sus instantes y toma cualquier geometría o geografía, siendo su distancia en cada instante la distancia del punto a ella. Observe que los tipos temporales sólo consideran la interpolación lineal entre valores, mientras que la distancia es una raíz de una función cuadrática. Por lo tanto, el operador de distancia temporal proporciona una aproximación lineal del valor de distancia real para los puntos de secuencia temporal. En este caso, los argumentos se sincronizan en la dimensión de tiempo y para cada uno de los segmentos de línea que componen los argumentos, se calcula la distancia espacial entre el punto inicial, el punto final y el punto de aproximación más cercano, como se muestra en los ejemplos a continuación.

- Devuelve la distancia temporal

```
{geo,tgeo} <-> {geo,tgeo} → tfloat
tDistance({geography,tgeog},{geography,tgeog},spheroid boolean=true) → tfloat
```

  Para geografías, el operador calcula en el esferoide WGS 84, y la función `tDistance` calcula en la esfera, más rápido, si su último argumento es falso.

```
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)' <->
  geometry 'Point(0 1)';
-- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03)
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)' <->
  tgeompoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03)';
-- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03)
SELECT tgeompoint '[Point(0 1)@2001-01-01, Point(0 0)@2001-01-03)' <->
  tgeompoint '[Point(0 0)@2001-01-01, Point(1 0)@2001-01-03)';
-- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03)
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,1 1)@2001-01-02]' <->
  tgeometry '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-02)';
-- Interp=Step;[1@2001-01-01, 1@2001-01-02]
SELECT round(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]' <->
  geography 'Point(0 0)', 3);
-- [110574.389@2001-01-01, 221149.453@2001-01-02]
SELECT round(tDistance(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  geography 'Point(0 0)', false), 3);
-- [111195.08@2001-01-01, 222390.159@2001-01-02]
```

8.7. Cálculos planos y geodésicos

Las coordenadas de una geometría temporal (`tgeometry` o `tgeompoint`) se interpretan en el plano de su sistema de referencia espacial, y las de una geografía temporal (`tgeography` o `tgeogpoint`) como longitudes y latitudes sobre el elipsoide WGS 84. Qué operaciones admite una geografía temporal, y con qué exactitud, se deduce de tres reglas.

1. Una operación está disponible para las geografías cuando está definida sobre el elipsoide. Es el caso de las operaciones métricas, como la distancia, la distancia dentro de un umbral, la longitud, el área, el acimut o el centroide, que se calculan sobre el esferoide WGS 84, o sobre la esfera, más rápido, cuando su argumento `spheroid` es falso. Es también el caso de la intersección y de la disyunción, ya que dos geografías se intersecan cuando su distancia es cero. No es el caso de `contains`, `covers` y `touches`, que se definen mediante el modelo de nueve intersecciones extendido dimensionalmente de la topología plana, ni de las construcciones planas, como la unión o la intersección de dos geometrías, que PostGIS tampoco define para las geografías. Estas funciones producen un error para las geografías.
2. Las versiones alguna vez, siempre y temporal de una operación disponible para las geografías se calculan de forma exacta. El valor de una geografía temporal, y el de un punto geográfico temporal con interpolación discreta o escalonada, es constante entre dos instantes, por lo que la operación se evalúa en cada instante. Un punto geográfico temporal con interpolación lineal se desplaza a lo largo de un círculo máximo entre dos instantes, y el resultado de una relación temporal cambia en los instantes en que su distancia geodésica al otro argumento cruza la distancia dada. Estos instantes se calculan cuando el otro argumento es un punto geográfico o un punto geográfico temporal. Frente a otras geografías, como líneas y polígonos, se calculan a partir de los cruces del círculo máximo de cada segmento con los vértices y las aristas de la geografía en la esfera. En el esferoide, cada instante en que la distancia en la esfera deja de crecer o de decrecer cerca de la distancia dada se desplaza al extremo de la distancia en el esferoide, y cada cruce se calcula sobre la distancia en el esferoide.
3. Una relación temporal disponible para las geografías acepta una geografía o una geografía temporal en cualquiera de sus argumentos, como su versión plana acepta una geometría o una geometría temporal.

Una geometría temporal en el SRID 4326 se convierte en una geografía temporal con los mismos valores mediante la conversión `::tgeography`, y a la inversa mediante la conversión `::tgeometry`. La función `transform` la reproyecta en cambio a un sistema de referencia espacial plano cuya unidad es el metro, en el que las distancias y las áreas son planas.

```
SELECT round(nearestApproachDistance(tgeometry 'SRID=4326;[Polygon((4.35 50.84,
  4.40 50.84,4.40 50.88,4.35 50.84))@2001-01-01, Point(4.37 50.85)@2001-01-02]',
  geometry 'SRID=4326;Point(4.45 50.84)')::numeric, 4);
-- 0.0500
SELECT round(nearestApproachDistance(tgeometry 'SRID=4326;[Polygon((4.35 50.84,
  4.40 50.84,4.40 50.88,4.35 50.84))@2001-01-01, Point(4.37 50.85)@2001-01-02]'
  ::tgeography, geography 'Point(4.45 50.84)')::numeric, 1);
-- 3521.9
```

8.8. Relaciones siempre y alguna vez

Los operandos de dimensiones distintas se leen en las dimensiones que ambos tienen, véase Sección 8.3.

- Contiene alguna vez o siempre

```
eContains({geometry,tgeom},{geometry,tgeom}) → boolean
aContains({geometry,tgeom},{geometry,tgeom}) → boolean
```

Esta función devuelve verdadero si el punto temporal está alguna vez en el interior de la geometría. Recuerde que una geometría no contiene cosas en su borde y, por lo tanto, los polígonos y las líneas no contienen líneas y puntos que se encuentran en su borde. Consulte la documentación de la función ST_Contains en PostGIS.

```
SELECT eContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

- Cubre alguna vez o siempre

```
eCovers({geometry,tgeom},{geometry,tgeom}) → boolean
aCovers({geometry,tgeom},{geometry,tgeom}) → boolean
```

Please refer to the documentation of the ST_Contains and the ST_Covers function in PostGIS for detailed explanations about the difference between the two functions.

```
SELECT eCovers(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

- Está disjunto alguna vez o siempre

```
eDisjoint({geo,tgeo},{geo,tgeo}) → boolean
aDisjoint({geo,tgeo},{geo,tgeo}) → boolean
```

```
SELECT eDisjoint(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)');
-- false
SELECT eDisjoint(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeometry '[Linestring(1 1 1,2 2 2)@2001-01-01, Point(2 2 2)@2001-01-03]');
-- true
```

- Está alguna vez o siempre a distancia de

```
eDwithin({geo,tgeo},{geo,tgeo},float) → boolean
aDwithin({geometry,tgeom},{geometry,tgeom},float) → boolean
eDwithin({geography,tgeog},{geography,tgeog},float,spheroid boolean=true) → boolean
aDwithin({geography,tgeog},{geography,tgeog},float,spheroid boolean=true) → boolean
```

Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido. Las geografías se admiten según lo indicado en Sección 8.7.

```
SELECT eDwithin(geometry 'Point(1 1 1)',
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 0)@2001-01-02]', 1);
-- true
SELECT eDwithin(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 2 2)@2001-01-01,Point(2 2 2)@2001-01-02]', 1);
-- false
SELECT eDwithin(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  geography 'Point(0 0)', 110800);
-- true
SELECT eDwithin(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  geography 'Point(0 0)', 110800, false);
-- false
SELECT aDwithin(tgeogpoint 'Interp=Step;[Point(0.5 2.5)@2001-01-01, Point(2.5 2.5)@2001 ↩
    -01-02]',
  geography 'Polygon((0 0,3 0,3 3,2 3,2 1,1 1,1 3,0 3,0 0))', 50000);
-- true
```

- Intersecta alguna vez o siempre

```
eIntersects({geo,tgeo},{geo,tgeo}) → boolean
aIntersects({geometry,tgeom},{geometry,tgeom}) → boolean
```

```
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03)');
-- false
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03)');
-- true
```

- Toca alguna vez o siempre

```
eTouches({geometry,tgeom},{geometry,tgeom}) → boolean
aTouches({geometry,tgeom},{geometry,tgeom}) → boolean
```

```
SELECT eTouches(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-03)');
-- true
```

8.9. Relaciones espaciotemporales

- Contiene temporal

```
tContains(geometry,tgeom) → tbool
```

```
SELECT tContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
SELECT tContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03]}
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(1 4)@2001-01-01, Point(4 1)@2001-01-04]');
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, f@2001-01-03, f@2001-01-04]}
```

- Disjunto temporal

```
tDisjoint({geo,tgeo},{geo,tgeo}) → tbool
```

La función es la negación de `tIntersects`. Las geografías se admiten según lo indicado en Sección 8.7.

```
SELECT tDisjoint(geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)');
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-03], (t@2001-01-03, t@2001-01-04]}
SELECT tDisjoint(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05)');
-- {[t@2001-01-01, f@2001-01-03], (t@2001-01-03, t@2001-01-05)}
```

- Está a distancia de temporal

```
tDwithin({geo,tgeo},{geo,tgeo},float) → tbool
tDwithin({geography,tgeog},{geography,tgeog},float,spheroid boolean=true) → tbool
```

Las geografías se admiten según lo indicado en Sección 8.7. La distancia se mide en el esferoide WGS 84 por defecto. Si el último argumento es falso, se mide en la esfera. Un segmento de un punto geográfico temporal se desplaza a lo largo de un círculo máximo, y el resultado cambia de valor en cada instante en que la distancia geodésica cruza la distancia dada, que un segmento puede cruzar varias veces.

```
SELECT tDwithin(geometry 'Point(1 1)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03)', sqrt(2));
--  {[t@2001-01-01, t@2001-01-03)}
SELECT tDwithin(tgeompoint '[Point(1 0)@2001-01-01, Point(1 4)@2001-01-05]',
  tgeompoint 'Interp=Step;[Point(1 2)@2001-01-01, Point(1 3)@2001-01-05]', 1);
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-04], (f@2001-01-04, t@2001-01-05]}
SELECT tDwithin(geography 'Point(1 0.1)',
  tgeogpoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]', 15000);
/* {[f@2001-01-01, t@2001-01-01 21:48:53.195554, t@2001-01-02 02:11:06.804445],
   (f@2001-01-02 02:11:06.804445, f@2001-01-03]} */
SELECT tDwithin(tgeogpoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]',
  tgeogpoint '[Point(1 1)@2001-01-01, Point(1 -1)@2001-01-03]', 20000, false);
/* {[f@2001-01-01, t@2001-01-01 20:56:51.37512, t@2001-01-02 03:03:08.624879],
   (f@2001-01-02 03:03:08.624879, f@2001-01-03]} */
SELECT tDwithin(tgeography '[Polygon((0 0,2 0,2 2,0 2,0 0))@2001-01-01,
  Point(3 3)@2001-01-02, Polygon((2 2,4 2,4 4,2 4,2 2))@2001-01-03]',
  geography 'Point(1 1)', 200000);
-- [t@2001-01-01, f@2001-01-02, t@2001-01-03]
```

- Intersección temporal

```
tIntersects({geo,tgeo},{geo,tgeo}) → tbool
```

Dos puntos temporales se intersecan en los instantes en que son iguales. Dos geometrías o geografías temporales, cuyos valores son constantes entre sus instantes, se intersecan en los instantes en que sus valores se intersecan, y dos geografías se intersecan donde su distancia es cero. Las geografías se admiten según lo indicado en Sección 8.7.

```
SELECT tIntersects(geometry 'MultiPoint(1 1,2 2)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)');
/* {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, t@2001-01-03],
   (f@2001-01-03, f@2001-01-04]} */
SELECT tIntersects(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05)');
-- {[f@2001-01-01, t@2001-01-03], (f@2001-01-03, f@2001-01-05)}
SELECT tIntersects(tgeometry '[Polygon((0 0,2 0,2 2,0 2,0 0))@2001-01-01,
  Point(3 3)@2001-01-02]', tgeometry '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-02]');
-- [t@2001-01-01, f@2001-01-02]
SELECT tIntersects(tgeography '[Polygon((0 0,2 0,2 2,0 2,0 0))@2001-01-01,
  Point(3 3)@2001-01-02, Polygon((2 2,4 2,4 4,2 4,2 2))@2001-01-03]',
  geography 'Point(1 1)');
-- [t@2001-01-01, f@2001-01-02, f@2001-01-03]
```

- Toca temporal

```
tTouches({geometry,tgeom},{geometry,tgeom}) → tbool
```

```
SELECT tTouches(geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 0)@2001-01-04)');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04]}
```

Capítulo 9

Tipos temporales: Operaciones de análisis

9.1. Simplificación

- Simplifica un flotante o una geometría/geografía temporal asegurándose de que los valores consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo

```
minDistSimplify({tfloat,tgeo},mindist float) → {tfloat,tgeo}
minTimeDeltaSimplify({tfloat,tgeo},mint interval) → {tfloat,tgeo}
```

En el caso de puntos temporales la distancia se especifica en las unidades del sistema de coordenadas. Observe que la simplificación se aplica sólo a secuencias temporales o conjuntos de secuencias con interpolación lineal. En todos los demás casos, se devuelve una copia del punto temporal dado.

```
SELECT minDistSimplify(tfloat '[1@2001-01-01,2@2001-01-02,3@2001-01-04,4@2001-01-05]', 1);
-- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05)', sqrt(3)));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05)
SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(4 4 4)@2001-01-05)', sqrt(3)));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (4 4 4)@2001-01-05)
```

```
SELECT minTimeDeltaSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-04,
  4@2001-01-05]', '1 day');
-- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
SELECT asText(minTimeDeltaSimplify(tgeogpoint '[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05)', '1 day'));
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05)
```

- Simplifica un flotante o un punto temporal usando el algoritmo de Douglas-Peucker

```
maxDistSimplify({tfloat,tgeompoint,tgeometry},maxdist float,
  syncdist boolean=true) → {tfloat,tgeompoint,tgeometry}
douglasPeuckerSimplify({tfloat,tgeompoint,tgeometry},maxdist float,
  syncdist boolean=true) → {tfloat,tgeompoint,tgeometry}
```

La diferencia entre las dos funciones es que `maxDistSimplify` usa una versión del algoritmo de un solo recorrido, mientras que `douglasPeuckerSimplify` usa el algoritmo recursivo estándar. La función elimina los valores or los puntos cuya distancia es menor que la distancia pasada como segundo argumento. En el caso de puntos temporales la distancia se especifica en las unidades del sistema de coordenadas. El tercer argumento se aplica solo a puntos temporales y especifica si se utiliza la distancia espacial o la distancia sincronizada. Observe que la simplificación se aplica sólo a secuencias temporales o conjuntos de secuencias con interpolación lineal. En todos los demás casos, se devuelve una copia del punto temporal dado.

```
-- Only synchronous distance for temporal floats
SELECT maxDistSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 3@2001-01-04,
  1@2001-01-05]', 1, false);
-- [1@2001-01-01, 1@2001-01-03, 3@2001-01-04, 1@2001-01-05]
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-05, POINT(5 1)@2001-01-06]
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2, false));
-- [POINT(1 1)@2001-01-01, POINT(5 1)@2001-01-06]
```

```
-- Spatial vs synchronized Euclidean distance
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
  Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, false));
-- [POINT(1 1)@2001-01-01, POINT(7 4)@2001-01-07]
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
  Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, true));
-- [POINT(1 1)@2001-01-01, POINT(6 1)@2001-01-06, POINT(7 4)@2001-01-07]
```

La diferencia entre la distancia espacial y la distancia sincronizada se ilustra en los dos últimos ejemplos anteriores y en la Figura 9.1. En el primer ejemplo, que usa la distancia espacial, se elimina el segundo instante ya que la distancia perpendicular entre `POINT(2 2)` y la línea definida por `POINT(1 1)` y `POINT(7 4)` es igual a 2.23. Por el contrario, en el segundo ejemplo se mantiene el segundo instante dado que la proyección de `Point(6 2)` en la marca de tiempo `2001-01-06` sobre el segmento de línea temporal da como resultado `Point(6 3.5)` y la distancia entre el punto original y su proyección es 2.5.

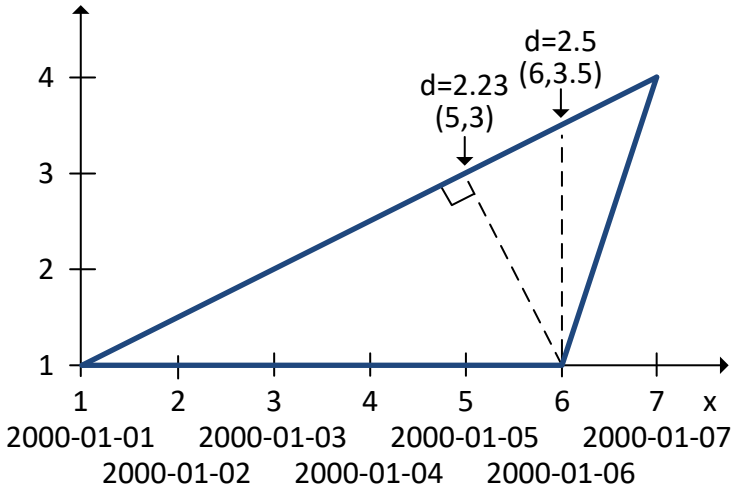


Figura 9.1: Diferencia entre la distancia espacial y la distancia sincronizada.

Un uso típico de la función `douglasPeuckerSimplify` es reducir el tamaño de un conjunto de datos, en particular con fines de visualización. Si la visualización es estática, se debe preferir la distancia espacial; si la visualización es dinámica o animada, se debe preferir la distancia sincronizada.

9.2. Reducción

- Muestrea un valor temporal con respecto a un intervalo

```
tsample(ttype,duration interval,torigin timestamptz='2000-01-03',
  interp text='discrete') → ttype
```

Si el origen no se especifica, su valor se establece por defecto en lunes 3 de enero de 2000. El intervalo dado debe ser estrictamente mayor que cero.

```
SELECT tsample(tbool '[true@2001-01-01,false@2001-01-02]', '12 hours', '2001-01-01',
  'step');
-- [t@2001-01-01, f@2001-01-02]
SELECT tsample(tint '{1@2001-01-01,5@2001-01-05}', '3 days', '2001-01-01');
-- {1@2001-01-01}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '1 day', '2001-01-01');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01');
-- {1@2001-01-01, 4@2001-01-04}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01', 'linear');
-- [1@2001-01-01, 4@2001-01-04]
```

```
SELECT asText(tsample(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05]',
  '2 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03, POINT(5 5)@2001-01-05}
SELECT asText(tsample(tgeompoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05],
  [Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-07}
SELECT asText(tsample(tgeompoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05],
  [Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01', 'step'));
-- Interp=Step;{[POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04], [POINT(3 3)@2001-01-07]}
```

La Figura 9.2 ilustra el muestreo para números flotantes temporales con varias interpolaciones. Como ilustra la figura, la operación de muestreo es más adecuada para valores temporales con interpolación continua.

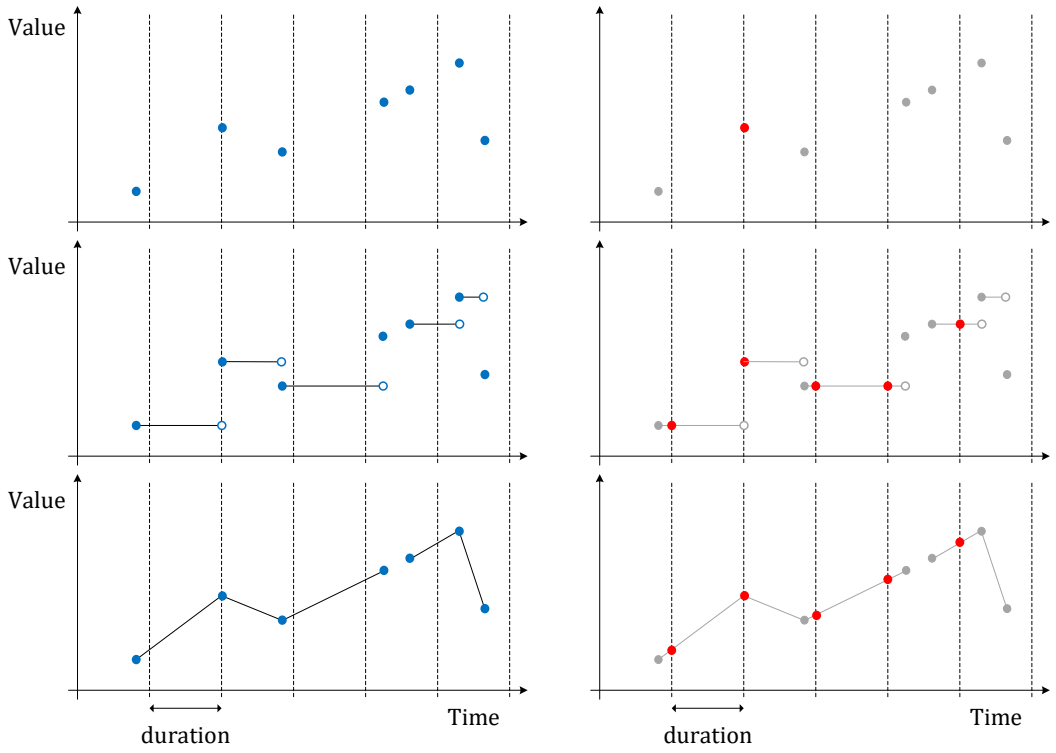


Figura 9.2: Muestreo de flotantes temporales con interpolación discreta, escalonada y lineal.

- Reduce la precisión temporal de un valor temporal con respecto a un intervalo calculando el promedio/centroide ponderado por el tiempo en cada intervalo de tiempo

```
tprecision({tnumber,tgeompoint,tcbuffer,tpose,tgeo,trgeometry},duration interval,
  torigin timestamptz='2000-01-03') → {tfloat,tpoint,tcbuffer,tpose,tgeo,trgeometry}
```

Si el origen no se especifica, su valor se establece por defecto en lunes 3 de enero de 2000. El intervalo dado debe ser estrictamente mayor que cero. Un número temporal da un flotante temporal, ya que el promedio ponderado por el tiempo de los valores de un intervalo no es necesariamente un entero, como `twAvg` da un flotante.

```
SELECT tprecision(tint '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','1 day', '2001-01-01');
-- Interp=Step;[1@2001-01-01, 5@2001-01-05, 1@2001-01-09]
SELECT tprecision(tint '[1@2001-01-01 00:00, 3@2001-01-01 00:05, 3@2001-01-01 00:10]',
  '10 minutes');
-- Interp=Step;[2@2001-01-01 00:00:00, 3@2001-01-01 00:10:00]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09)','1 day',
  '2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','1 day',
  '2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08, 1@2001-01-09]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09)','2 days',
  '2001-01-01');
-- [2@2001-01-01, 4@2001-01-03, 4@2001-01-05, 2@2001-01-07]
```

```
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
  Point(1 1)@2001-01-09)', '1 day', '2001-01-01'));
/* [POINT(1.5 1.5)@2001-01-01, POINT(4.5 4.5)@2001-01-04, POINT(4.5 4.5)@2001-01-05,
   POINT(1.5 1.5)@2001-01-08] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
  Point(1 1)@2001-01-09)', '2 days', '2001-01-01'));
/* [POINT(2 2)@2001-01-01, POINT(4 4)@2001-01-03, POINT(4 4)@2001-01-05, POINT(2
   2)@2001-01-07] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
  Point(1 1)@2001-01-09)', '4 days', '2001-01-01'));
-- [POINT(3 3)@2001-01-01, POINT(3 3)@2001-01-05]
```

Cambiar la precisión de un valor temporal es similar a cambiar su *granularidad temporal*, por ejemplo, de marcas de tiempo a horas o días, aunque la precisión se puede establecer en un intervalo arbitrario, como 2 horas y 15 minutos. La Figura 9.3 ilustra un cambio de precisión temporal para números flotantes temporales con varias interpolaciones.

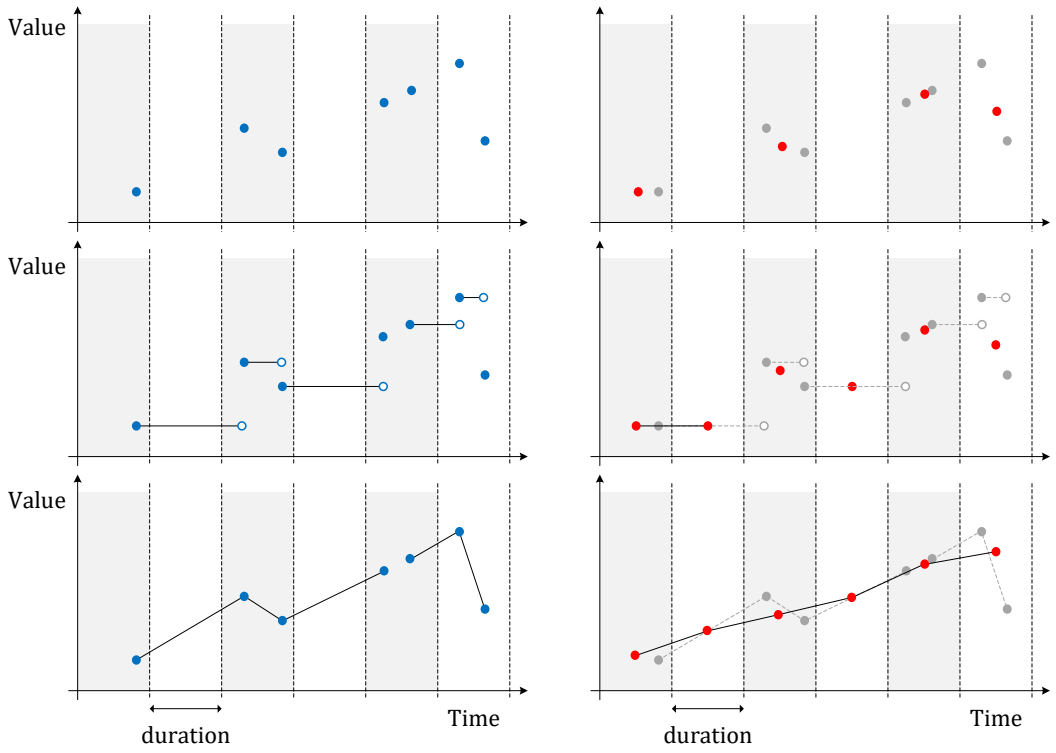


Figura 9.3: Cambio de precisión de números flotantes temporales con interpolación discreta, escalonada y lineal.

9.3. Similaridad

- Devuelve la distancia de Hausdorff discreta, la distancia de Fréchet discreta, la distancia de distorsión de tiempo dinámica (Dynamic Time Warp o DTW), la distancia de Hausdorff promedio, o la distancia de subsecuencia común más larga (Longest Common SubSequence o LCSS) entre dos valores temporales

```
hausdorffDistance({tnumber,tgeo},{tnumber,tgeo}) → float
frechetDistance({tnumber,tgeo},{tnumber,tgeo}) → float
dynTimeWarpDistance({tnumber,tgeo},{tnumber,tgeo}) → float
averageHausdorffDistance({tnumber,tgeo},{tnumber,tgeo}) → float
lcssDistance({tnumber,tgeo},{tnumber,tgeo},float) → float
hausdorffDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
frechetDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
dynTimeWarpDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
averageHausdorffDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
lcssDistance(tgeogpoint,tgeogpoint,float,spheroid boolean=true) → float
```

La función `lcssDistance` requiere un parámetro adicional que especifica el umbral de distancia por debajo del cual dos valores se consideran coincidentes. Estas funciones tienen una complejidad de espacio lineal ya que solo dos filas de la matriz de distancia son asignadas en la memoria. Sin embargo, su complejidad de tiempo es cuadrática en el número de instantes de los valores temporales. Por lo tanto, las funciones requerien un tiempo considerable para valores temporales con gran número de instantes. Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT hausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 0.5
SELECT round(hausdorffDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
```

```
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 1.414214
SELECT round(frechetDistance(tgeogpoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02]',
  tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]', false), 3);
-- 111195.08
```

```
SELECT frechetDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 0.5
SELECT round(frechetDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 1.414214
```

```
SELECT dynTimeWarpDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 2
SELECT round(dynTimeWarpDistance(tgeompoint '[Point(1 1)@2001-01-01,
  Point(3 3)@2001-01-03, Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 3.380776
```

```
SELECT round(averageHausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04,
  1.5@2001-01-05]')::numeric, 6);
-- 0.283333
SELECT round(averageHausdorffDistance(tgeompoint '[Point(1 1)@2001-01-01,
  Point(3 3)@2001-01-03, Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 0.385218
```

```
SELECT lcssDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]',
  1.0);
-- 3
SELECT round(lcssDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]', 1.0)::numeric, 6);
-- 2.000000
```

- Devuelve las parejas de correspondencia entre dos valores temporales con respecto a la distancia de Fréchet discreta o la distance de distorsión de tiempo dinámica (Dynamic Time Warp o DTW) 📦🌐{}

```
frechetDistancePath({tnumber,tgeo},{tnumber,tgeo}) → {(i,j)}
dynTimeWarpPath({tnumber,tgeo},{tnumber,tgeo}) → {(i,j)}
frechetDistancePath(tgeogpoint,tgeogpoint,spheroid boolean=true) → {(i,j)}
dynTimeWarpPath(tgeogpoint,tgeogpoint,spheroid boolean=true) → {(i,j)}
```

El resultado es un conjunto de pares `(i,j)`. Esta función requiere ubicar en memoria una matriz de distancias cuyo tamaño es cuadrático en el número de instantes de los valores temporales. Por tanto, la función fallará para valores temporales con gran número de instantes dependiendo de la memoria disponible. Para geografías, el cálculo se realiza en el esferoide WGS 84 por defecto. Si el último argumento es falso, se utiliza un cálculo esférico más rápido.

```
SELECT frechetDistancePath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
```

```
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT frechetDistancePath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]');
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)
```

```
SELECT dynTimeWarpPath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT dynTimeWarpPath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]');
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)
```

9.4. Filtro de Kalman extendido

- Devuelve un flotante temporal o un punto temporal suavizado, obtenido al filtrar una trayectoria ruidosa con un filtro de Kalman extendido

```
extendedKalmanFilter(tfloat,gate float,q float,variance float,
  to_drop boolean=true) → tfloat
extendedKalmanFilter(tgeompoint,gate float,q float,variance float,
  to_drop boolean=true) → tgeompoint
```

El filtro modela el movimiento subyacente de un objeto móvil y lo separa del ruido de medición y de los valores atípicos ocasionales. El parámetro `gate` controla el rigor de la detección de atípicos, `q` es la escala del ruido de proceso, `variance` es la varianza del ruido de medición y `to_drop` elige si los atípicos se eliminan (`true`) o se reemplazan por la trayectoria predicha (`false`). Estas funciones están pensadas para trayectorias con al menos tres instantes y se aplican normalmente a valores temporales con interpolación lineal.

```
SELECT asEWKT(extendedKalmanFilter(tgeompoint '[Point(1 1)@2001-01-01,
  Point(50 50)@2001-01-02, Point(2 2)@2001-01-03]', 1e-5, 5e-10, 5e-10, true));
-- [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-03]
SELECT asEWKT(extendedKalmanFilter(tgeompoint '[Point(1 1)@2001-01-01,
  Point(50 50)@2001-01-02, Point(2 2)@2001-01-03]', 1e-5, 5e-10, 5e-10, false));
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]
```

9.5. Operaciones de división

Al crear índices para tipos temporales, lo que se almacena en el índice no es el valor real sino un cuadro delimitador que *representa* el valor. En este caso, el índice proporcionará una lista de valores candidatos que *pueden* satisfacer el predicado de la consulta, y se necesita un segundo paso para filtrar los valores candidatos calculando el predicado de la consulta sobre los valores reales.

Sin embargo, cuando los cuadros delimitadores tienen un gran espacio vacío no cubierto por los valores reales, el índice generará muchos valores candidatos que no satisfacen el predicado de la consulta, lo que reduce la eficiencia del índice. En estas situaciones, puede ser mejor representar un valor no con un cuadro delimitador *único*, sino con *múltiples* cuadros delimitadores. Esto aumenta considerablemente la eficiencia del índice, siempre que el índice sea capaz de gestionar múltiples cuadros delimitadores por valor. Las siguientes funciones se utilizan para generar múltiples cuadros delimitadores para un único valor temporal.

- Devuelve una matriz de N rangos de tiempo a partir de los instantes o segmentos de un valor temporal

```
splitNSpans(ttype,integer) → tstzspan[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de rangos de salida. Si el número de instantes o segmentos es inferior o igual al número especificado, la matriz resultante tendrá un rango por instante o segmento del valor temporal. En caso contrario, el número de rangos especificado se obtendrá fusionando instantes o segmentos consecutivos.

```
SELECT splitNSpans(ttext '{A@2001-01-01, B@2001-01-02, A@2001-01-03, B@2001-01-04,
  A@2001-01-05}', 1);
-- {"[2001-01-01, 2001-01-05]"}
SELECT splitNSpans(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04,
  1@2001-01-05}', 2);
-- {"[2001-01-01, 2001-01-03]","[2001-01-04, 2001-01-05]"}
SELECT splitNSpans(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03, Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]', 2);
-- {"[2001-01-01, 2001-01-03]","[2001-01-03, 2001-01-05]"}
SELECT splitNSpans(tgeogpoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04], [Point(1 1 1)@2001-01-05]}', 2);
-- {"[2001-01-01, 2001-01-04]","[2001-01-05, 2001-01-05]"}
SELECT splitNSpans(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04,
  2@2001-01-05}', 6);
/* {"[2001-01-01, 2001-01-01]","[2001-01-02, 2001-01-02]","[2001-01-03,
   2001-01-03]","[2001-01-04, 2001-01-04]","[2001-01-05, 2001-01-05]"} */
SELECT splitNSpans(ttext '[A@2001-01-01, B@2001-01-02, A@2001-01-03, B@2001-01-04,
  A@2001-01-05]', 6);
/* {"[2001-01-01, 2001-01-02]","[2001-01-02, 2001-01-03]","[2001-01-03,
   2001-01-04]","[2001-01-04, 2001-01-05]"} */
```

- Devuelve una matriz de rangos de tiempo obtenida fusionando N instantes o segmentos consecutivos de un valor temporal

```
splitEachNSpans(ttype,integer) → tstzspan[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. El último argumento especifica el número de instantes o segmentos de entrada que se fusionan para producir un rango de salida. Si el número de instantes o segmentos es inferior o igual al número especificado, la matriz resultante tendrá un único rango por secuencia. En caso contrario, el número especificado de instantes o segmentos consecutivos será fusionado en cada rango de salida. Observe que, a diferencia de la función splitNSpans, el número de cuadros en el resultado depende del número de instantes o de segmentos de entrada.

```
SELECT splitEachNSpans(ttext '{A@2001-01-01, B@2001-01-02, A@2001-01-03, B@2001-01-04,
  A@2001-01-05}', 1);
/* {"[2001-01-01, 2001-01-01]","[2001-01-02, 2001-01-02]","[2001-01-03,
   2001-01-03]","[2001-01-04, 2001-01-04]","[2001-01-05, 2001-01-05]"} */
SELECT splitEachNSpans(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04,
```

```
  1@2001-01-05]', 2);
-- {"[2001-01-01, 2001-01-03]","[2001-01-03, 2001-01-05]"}
SELECT splitEachNSpans(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04], [Point(1 1 1)@2001-01-05]}', 2);
-- {"[2001-01-01, 2001-01-02]","[2001-01-03, 2001-01-04]","[2001-01-05, 2001-01-05]"}
SELECT splitEachNSpans(tgeogpoint '[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02,
  Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04, Point(1 1 1)@2001-01-05]', 6);
-- {"[2001-01-01, 2001-01-05]"}
SELECT splitEachNSpans(tgeogpoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04], [Point(1 1 1)@2001-01-05]}', 6);
-- {"[2001-01-01, 2001-01-02]","[2001-01-03, 2001-01-04]","[2001-01-05, 2001-01-05]"}
```

- Devuelve una matriz de N cuadros temporales obtenida fusionando los instantes o segmentos de un número temporal

```
splitNTboxes(tnumber,integer) → tbox[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta or continua. El último argumento especifica el número de cuadros de salida. Si el número de instantes o segmentos es inferior o igual al numéro especificado, la matriz resultante tendrá un cuadro por instante o segmento del número temporal. En caso contrario, En caso contrario, el número de cuadros especificado se obtendrá fusionando instantes o segmentos consecutivos.

```
SELECT splitNTboxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 1);
-- {"TBOXINT XT([1, 5),[2001-01-01, 2001-01-05])"}
SELECT splitNTboxes(tfloat '{[1@2001-01-01, 2@2001-01-02], [1@2001-01-03, 4@2001-01-04],
  [1@2001-01-05]}', 2);
/* {"TBOXFLOAT XT([1, 4],[2001-01-01, 2001-01-04])","TBOXFLOAT XT([1, 1],[2001-01-05,
   2001-01-05])"} */
SELECT splitNTboxes(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3);
/* {"TBOXFLOAT XT([1, 2],[2001-01-01, 2001-01-03])","TBOXFLOAT XT([1, 4],[2001-01-03,
   2001-01-04])","TBOXFLOAT XT([1, 4],[2001-01-04, 2001-01-05])"} */
SELECT splitNTboxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 6);
/* {"TBOXINT XT([1, 2),[2001-01-01, 2001-01-01])","TBOXINT XT([2, 3),[2001-01-02,
   2001-01-02])","TBOXINT XT([1, 2),[2001-01-03, 2001-01-03])","TBOXINT XT([4,
   5),[2001-01-04, 2001-01-04])","TBOXINT XT([1, 2),[2001-01-05, 2001-01-05])"} */
SELECT splitNTboxes(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 6);
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-02])","TBOXINT XT([1, 3),[2001-01-02,
   2001-01-03])","TBOXINT XT([1, 5),[2001-01-03, 2001-01-04])","TBOXINT XT([1,
   5),[2001-01-04, 2001-01-05])"} */
```

- Devuelve una matriz de cuadros temporales obtenida fusionando N instantes o segmentos consecutivos de un número temporal

```
splitEachNTboxes(tnumber,integer) → tbox[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta or continua. El último argumento especifica el número de instantes o segmentos de entrada que se fusionan para producir un cuadro de salida. Si el número de instantes o segmentos es inferior o igual al número especificado, la matriz resultante tendrá un único cuadro por secuencia. En caso contrario, el número especificado de instantes o segmentos consecutivos será fusionado en cada cuadro de salida. Observe que, a diferencia de la función splitNTboxes, el número de cuadros en el resultado depende del número de instantes o de segmentos de entrada.

```
SELECT splitEachNTboxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 1);
/* {"TBOXINT XT([1, 2),[2001-01-01, 2001-01-01])","TBOXINT XT([2, 3),[2001-01-02,
   2001-01-02])","TBOXINT XT([1, 2),[2001-01-03, 2001-01-03])","TBOXINT XT([4,
   5),[2001-01-04, 2001-01-04])","TBOXINT XT([1, 2),[2001-01-05, 2001-01-05])"} */
SELECT splitEachNTboxes(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 1);
```

```
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-02])","TBOXINT XT([1, 3),[2001-01-02,
   2001-01-03])","TBOXINT XT([1, 5),[2001-01-03, 2001-01-04])","TBOXINT XT([1,
   5),[2001-01-04, 2001-01-05])"} */
SELECT splitEachNTboxes(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3);
/* {"TBOXFLOAT XT([1, 4],[2001-01-01, 2001-01-04])","TBOXFLOAT XT([1, 4],[2001-01-04,
   2001-01-05])"} */
SELECT splitEachNTboxes(tfloat '{[1@2001-01-01, 2@2001-01-02],
  [1@2001-01-03, 4@2001-01-04], [1@2001-01-05]}', 6);
/* {"TBOXFLOAT XT([1, 2],[2001-01-01, 2001-01-02])","TBOXFLOAT XT([1, 4],[2001-01-03,
   2001-01-04])","TBOXFLOAT XT([1, 1],[2001-01-05, 2001-01-05])"} */
```

- Devuelve ya sea una matriz de N cuadros espaciales obtenida fusionando los segmentos de una (multi)línea o una matriz de N cuadros espaciotemporales obtenida fusionando los instantes o segmentos de un punto temporal

```
splitNStboxes({lines,tpoint},integer) → stbox[]
```

Para puntos temporales, la elección entre instantes o segmentos depende de si la interpolación es discreta or continua. El último argumento especifica el número de cuadros de salida. Si el número de instantes o segmentos es menor que el número dado, la matriz resultante tendrá un cuadro por segmento de la (multi)línea o un cuadro por instante o segmento del punto temporal. En caso contrario, el número de cuadros especificado se obtendrá fusionando instantes o segmentos consecutivos.

```
SELECT splitNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
-- {"STBOX X((1,1),(5,2))"}
SELECT splitNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1),(3,2))","STBOX X((3,1),(5,2))"}
SELECT splitNStboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
/* {"SRID=4326;GEODSTBOX Z((1,1,1),(2,2,1))","SRID=4326;GEODSTBOX
   Z((2,1,1),(3,2,1))","SRID=4326;GEODSTBOX Z((3,1,1),(4,2,1))","SRID=4326;GEODSTBOX
   Z((4,1,1),(5,2,1))"} */
SELECT splitNStboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1),(4,2))","STBOX X((5,1),(6,2))"}
```

```
SELECT splitNStboxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05}', 1);
-- {"STBOX XT(((1,1),(5,2)),[2001-01-01, 2001-01-05])"}
SELECT splitNStboxes(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05]', 4);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])",
   "STBOX XT(((2,1),(3,2)),[2001-01-02, 2001-01-03])",
   "STBOX XT(((3,1),(4,2)),[2001-01-03, 2001-01-04])",
   "STBOX XT(((4,1),(5,2)),[2001-01-04, 2001-01-05])"} */
SELECT splitNStboxes(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(3 1)@2001-01-03, Point(4 2)@2001-01-04], [Point(5 1)@2001-01-05]}', 2);
/* {"STBOX XT(((1,1),(4,2)),[2001-01-01, 2001-01-04])","STBOX
   XT(((5,1),(5,1)),[2001-01-05, 2001-01-05])"} */
SELECT splitNStboxes(tgeogpoint '{Point(1 1 1)@2001-01-01, Point(2 2 1)@2001-01-02,
  Point(3 1 1)@2001-01-03, Point(4 2 1)@2001-01-04, Point(5 1 1)@2001-01-05}', 6);
/* {"SRID=4326;GEODSTBOX ZT(((1,1,1),(1,1,1)),[2001-01-01,
   2001-01-01])","SRID=4326;GEODSTBOX ZT(((2,2,1),(2,2,1)),[2001-01-02,
   2001-01-02])","SRID=4326;GEODSTBOX ZT(((3,1,1),(3,1,1)),[2001-01-03,
   2001-01-03])","SRID=4326;GEODSTBOX ZT(((4,2,1),(4,2,1)),[2001-01-04,
   2001-01-04])","SRID=4326;GEODSTBOX ZT(((5,1,1),(5,1,1)),[2001-01-05, 2001-01-05])"} */
SELECT splitNStboxes(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05]', 6);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])","STBOX
   XT(((2,1),(3,2)),[2001-01-02, 2001-01-03])","STBOX XT(((3,1),(4,2)),[2001-01-03,
   2001-01-04])","STBOX XT(((4,1),(5,2)),[2001-01-04, 2001-01-05])"} */
```

- Devuelve ya sea una matriz de cuadros espaciales obtenida fusionando N segmentos consecutivos de una (multi)línea o una matriz de cuadros espaciotemporales obtenida fusionando N instantes o segmentos consecutivos de un punto temporal

```
splitEachNStboxes({lines,tpoint},integer) → stbox[]
```

Para puntos temporales, la elección entre instantes o segmentos depende de si la interpolación es discreta or continua. El último argumento especifica el número de instantes o segmentos de entrada que se fusionan para producir un cuadro de salida. Si el número de instantes o segmentos es menor que el número dado, la matriz resultante tendrá un único cuadro por secuencia. En caso contrario, el número especificado de instantes o segmentos consecutivos será fusionado en cada cuadro de salida. Observe que, a diferencia de la función splitNStboxes, el número de cuadros en el resultado depende del número de instantes o de segmentos de entrada.

```
SELECT splitEachNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
/* {"STBOX X((1,1),(2,2))","STBOX X((2,1),(3,2))","STBOX X((3,1),(4,2))","STBOX
   X((4,1),(5,2))"} */
SELECT splitEachNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1),(3,2))","STBOX X((3,1),(5,2))"}
SELECT splitEachNStboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
-- {"SRID=4326;GEODSTBOX Z((1,1,1),(5,2,1))"}
SELECT splitEachNStboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1),(2,2))","STBOX X((3,1),(4,2))","STBOX X((5,1),(6,2))"}
```

```
SELECT splitEachNStboxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05}', 1);
/* {"STBOX XT(((1,1),(1,1)),[2001-01-01, 2001-01-01])","STBOX
   XT(((2,2),(2,2)),[2001-01-02, 2001-01-02])","STBOX XT(((3,1),(3,1)),[2001-01-03,
   2001-01-03])","STBOX XT(((4,2),(4,2)),[2001-01-04, 2001-01-04])","STBOX
   XT(((5,1),(5,1)),[2001-01-05, 2001-01-05])"} */
SELECT splitEachNStboxes(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05]', 1);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])","STBOX
   XT(((2,1),(3,2)),[2001-01-02, 2001-01-03])","STBOX XT(((3,1),(4,2)),[2001-01-03,
   2001-01-04])","STBOX XT(((4,1),(5,2)),[2001-01-04, 2001-01-05])"} */
SELECT splitEachNStboxes(tgeogpoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(3 1)@2001-01-03, Point(4 2)@2001-01-04], [Point(5 1)@2001-01-05]}', 2);
/* {"SRID=4326;GEODSTBOX XT(((1,1),(2,2)),[2001-01-01,
   2001-01-02])","SRID=4326;GEODSTBOX XT(((3,1),(4,2)),[2001-01-03,
   2001-01-04])","SRID=4326;GEODSTBOX XT(((5,1),(5,1)),[2001-01-05, 2001-01-05])"} */
```

9.6. Mosaicos multidimensionales

Los mosaicos multidimensionales son un mecanismo que se utiliza para dividir el dominio de valores temporales en intervalos o mosaicos de un número variable de dimensiones. En el caso de una sola dimensión, el dominio se puede dividir por valor o por tiempo utilizando intervalos del mismo ancho o la misma duración, respectivamente. Para los números temporales, el dominio se puede dividir en mosaicos bidimensionales del mismo ancho para la dimensión de valor y la misma duración para la dimensión de tiempo. Para los puntos temporales, el dominio se puede dividir en el espacio en mosaicos bidimensionales o tridimensionales, dependiendo del número de dimensiones de las coordenadas espaciales. Finalmente, para los puntos temporales, el dominio se puede dividir por espacio y por tiempo usando mosaicos tridimensionales o tetradimensionales. Además, los valores temporales también se pueden fragmentar de acuerdo con una malla multidimensional definida sobre el dominio subyacente.

Los mosaicos multidimensionales se pueden utilizar para diversos fines. Se pueden utilizar para calcular histogramas multidimensionales, donde los valores temporales se agregan de acuerdo con la partición subyacente del dominio. Por otro lado, los mosaicos multidimensionales también se pueden utilizar para fines de indexación, donde el cuadro delimitador de un valor temporal se puede fragmentar en múltiples cuadros para mejorar la eficiencia del índice. Finalmente, los mosaicos multidimensionales se pueden utilizar para fragmentar valores temporales de acuerdo con una malla multidimensional definida sobre el dominio subyacente. Esto permite la distribución de un conjunto de datos en un clúster de servidores, donde cada servidor contiene una partición del conjunto de datos. La ventaja de este mecanismo de partición es que preserva la proximidad en valor/espacio y tiempo, a diferencia de los mecanismos de partición tradicionales basados en hash.

La Figura 9.4 ilustra un mosaico multidimensional para números flotantes temporales. El dominio bidimensional se divide en mosaicos que tienen el mismo tamaño para la dimensión de valor y la misma duración para la dimensión de tiempo. Suponga que

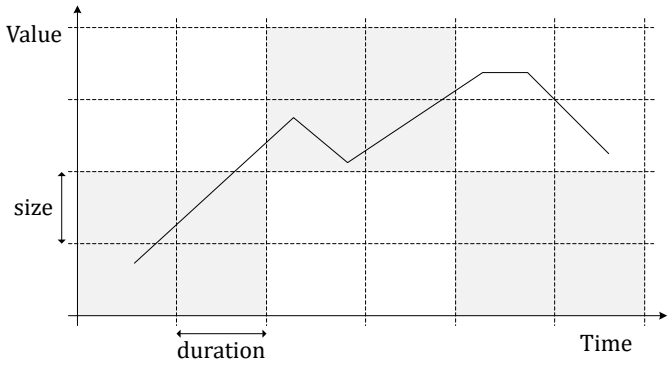


Figura 9.4: Mosaicos multidimensionales para números flotantes temporales.

este esquema de mosaicos se usa para distribuir un conjunto de datos en un clúster de seis servidores, como sugiere el patrón gris en la figura. En este caso, los valores se fragmentan para que cada servidor reciba los datos de mosaicos contiguos. Esto implica en particular que cuatro nodos recibirán un fragmento del número flotante temporal que se muestra en la figura. Una ventaja de esta distribución de datos basada en mosaicos multidimensionales es que reduce los datos que deben intercambiarse entre nodos cuando se procesan consultas, un proceso que generalmente se denomina *reshuffling*.

Muchas de las funciones de esta sección son *funciones de retorno de conjuntos* (también conocidas como *funciones de tabla*) ya que normalmente devuelven más de un valor. En este caso, las funciones están marcadas con el símbolo {}.

Cada intervalo, mosaico y caja contiene su límite inferior y excluye el superior, de modo que una posición en el borde entre dos de ellos pertenece al que comienza allí. La extensión de un valor o de una caja contiene su límite superior, de modo que cuando ese límite cae en un borde de la malla, el intervalo o mosaico que comienza allí no contiene de la extensión más que su borde superior. El argumento opcional `borderInc`, verdadero por defecto, indica si ese intervalo o mosaico forma parte de la malla. Cuando es verdadero, el borde tiene su propio intervalo, mosaico, caja o fragmento, de modo que los fragmentos de una división cubren todo el valor; cuando es falso, el borde se omite, como para una extensión leída sin su límite superior. Un valor cuya extensión excluye su límite superior no tiene borde que omitir y responde igual. Las mallas de celdas H3 y S2, que no tienen un eje alineado con la extensión, asignan una posición en el borde de una celda a una de las celdas que se encuentran allí, y la celda en la que una trayectoria entra en su último instante es la única celda que contiene solo el borde superior de su extensión: `borderInc` indica si esa celda forma parte del resultado.

9.6.1. Operaciones de intervalos

- Devuelve una matriz de intervalos que cubre el rango de valores o de tiempo con intervalos de la misma amplitud o duración

```
bins(numspans,vsize number,vorigin number=0,borderInc boolean=true) → span[]
bins(datespans,duration interval,torigin date='2000-01-01',
  borderInc boolean=true) → span[]
bins(tstzspans,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → span[]
```

Si el origen no se especifica, su valor se establece por defecto en 0 para rangos de valores, en sábado 1 de enero de 2000 para rangos de fechas y en lunes 3 de enero de 2000 para rangos de marcas de tiempo. El argumento opcional `borderInc` indica si se incluye el borde superior del rango y, por lo tanto, se genera el intervalo que comienza en ese borde y solo lo contiene.

```
SELECT bins(floatspan '[-10, -1]', 2.5, -7);
-- {"[-12, -9.5)","[-9.5, -7)","[-7, -4.5)","[-4.5, -2)","[-2, 0.5)"}
SELECT bins(intspan '[15, 25]', 2);
-- {"[14, 16)","[16, 18)","[18, 20)","[20, 22)","[22, 24)","[24, 26)"}
SELECT index, span FROM unnest(bins(datespan '[2001-01-15, 2001-01-25]', '2 days'))
  WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-17)
-- 2 | [2001-01-17, 2001-01-19)
-- 3 | [2001-01-19, 2001-01-21)
```

```
-- ...
SELECT index, span
FROM unnest(bins(tstzspanset '{[2001-01-15, 2001-01-17],[2001-01-22, 2001-01-25]}',
  '2 days', '2001-01-02')) WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-16)
-- 2 | [2001-01-16, 2001-01-17]
-- 3 | [2001-01-22, 2001-01-24)
-- 4 | [2001-01-24, 2001-01-25]
```

- Devuelve el rango que contiene un número o una marca de tiempo

```
getBin(number,vsize number,vorigin number=0) → span
getBin(date,duration interval,torigin date='2000-01-03') → datespan
getBin(timestamptz,duration interval,torigin timestamptz='2000-01-03') → tstzspan
```

Si el origen no se especifica, su valor se establece por defecto en 0 para rangos de valores y en lunes 3 de enero de 2000 para rangos de tiempo.

```
SELECT getBin(2, 2);
-- [2, 4)
SELECT getBin(2, 2.5, 1.5);
-- [1.5, 4)
SELECT getBin('2001-01-04', interval '1 week');
-- [2001-01-01 08:00:00, 2001-01-08 08:00:00)
SELECT getBin('2001-01-04', interval '1 week', '2001-01-07');
-- [2000-12-31, 2001-01-07)
```

9.6.2. Operaciones de mosaicos

- Devuelve el conjunto de mosaicos que cubre un cuadro delimitador temporal con mosaicos del mismo tamaño y/o duración {}

```
valueTiles(tbox,vsize number,vorigin number=0,borderInc boolean=true) → {(index,tile)}
timeTiles(tbox,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(index,tile)}
valueTimeTiles(tbox,vsize number,duration interval,vorigin number=0,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(index,tile)}
```

El resultado es un conjunto de pares `(index,tile)`. Si el origen de la dimensión de valores y/o de tiempo no se especifica, su valor se establece por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente. El tamaño de los valores y la duración de una malla de valores y tiempo son estrictamente positivos; de lo contrario se produce un error. El tamaño y el origen de los valores tienen el tipo base de los valores de la caja, y una caja de flotantes también acepta valores enteros. Los mosaicos de tiempo de una caja con dimensión de valores conservan el intervalo de valores de la caja, con sus límites. El argumento opcional `borderInc` indica si se incluye el borde superior de la caja y, por lo tanto, se generan mosaicos adicionales que contienen el borde.

```
SELECT (gr).index, (gr).tile
FROM (SELECT valueTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox, 2.0) AS gr) t;
-- 1 | TBOXFLOAT X([14, 16))
-- 2 | TBOXFLOAT X([16, 18))
-- 3 | TBOXFLOAT X([18, 20))
-- ...
SELECT (gr).index, (gr).tile
FROM (SELECT timeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox, '2 days') AS gr) t;
-- 1 | TBOXFLOAT XT([15, 25],[2001-01-15, 2001-01-17))
-- 2 | TBOXFLOAT XT([15, 25],[2001-01-17, 2001-01-19))
-- 3 | TBOXFLOAT XT([15, 25],[2001-01-19, 2001-01-21))
-- ...
SELECT (gr).index,
  (gr).tile FROM (SELECT valueTimeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox,
```

```
  2.0, '2 days') AS gr) t;
-- 1 | TBOX XT([14,16),[2001-01-15,2001-01-17))
-- 2 | TBOX XT([16,18),[2001-01-15,2001-01-17))
-- 3 | TBOX XT([18,20),[2001-01-15,2001-01-17))
-- ...
SELECT valueTiles(tbox 'TBOXINT XT([15, 25],[2001-01-15, 2001-01-25])', 2, 15);
-- (1,"TBOXINT X([15, 17))")
-- (2,"TBOXINT X([17, 19))")
-- (3,"TBOXINT X([19, 21))")
-- ...
SELECT valueTimeTiles(tfloat '[15@2001-01-15,25@2001-01-25]'::tbox, 2.0, '2 days', 11.5);
-- (1,"TBOX XT([13.5,15.5),[2001-01-15,2001-01-17))")
-- (2,"TBOX XT([15.5,17.5),[2001-01-15,2001-01-17))")
-- (3,"TBOX XT([17.5,19.5),[2001-01-15,2001-01-17))")
-- ...
```

- Devuelve el conjunto de mosaicos que cubre un cuadro delimitador espaciotemporal con mosaicos del mismo tamaño y/o duración ⍁{}

```
spaceTiles({stbox,tgeompoint},xsize float,[ysize float,zsize float,]
  sorigin geompoint='Point(0 0 0)',borderInc boolean=true) → {(index,tile)}
timeTiles({stbox,tgeompoint},duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(index,tile)}
spaceTimeTiles({stbox,tgeompoint},xsize float,[ysize float,zsize float,]duration interval,
  sorigin geompoint='Point(0 0 0)',torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(index,tile)}
```

Si el origen de las dimensiones espacial y/o de tiempo no se especifican, su valor se establece por defecto en `'Point (0 0 0)'` y en el lunes 3 de enero de 2000, respectivamente. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se generan mosaicos adicionales que contienen el borde. En el caso de una malla espacio-temporal, `ysize` y `zsize` son opcionales. Un tamaño omitido, o dado como 0, toma el valor de `xsize`, y `zsize` solo se lee para un valor con dimensión Z. Todo tamaño es un número finito que no es negativo; de lo contrario se produce un error. El tamaño `xsize` y la duración de una malla espacio-temporal son estrictamente positivos; de lo contrario se produce un error. El SRID de las coordenadas de los mosaicos está determinado por el del cuadro de entrada y el tamaño se da en las unidades del SRID. Si se especifica el origen de las coordenadas espaciales, que debe ser un punto, su dimensionalidad y SRID deben ser iguales al del cuadro delimitador, de lo contrario se genera un error.

```
SELECT spaceTiles(tgeompoint '[Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]', 2.0);
-- (1,"STBOX X((2,2),(4,4))")
-- (2,"STBOX X((4,2),(6,4))")
-- (3,"STBOX X((6,2),(8,4))")
-- ...
SELECT timeTiles(tgeompoint '[Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]', '2 days');
-- (1,"STBOX T([2001-01-15, 2001-01-17))")
-- (2,"STBOX T([2001-01-17, 2001-01-19))")
-- (3,"STBOX T([2001-01-19, 2001-01-21))")
-- ...
SELECT spaceTiles(tgeompoint 'SRID=3812;
  [Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]'::stbox, 2.0, geometry 'Point(3 3)');
-- (1,"SRID=3812;STBOX X((3,3),(5,5))")
-- (2,"SRID=3812;STBOX X((5,3),(7,5))")
-- (3,"SRID=3812;STBOX X((7,3),(9,5))")
-- ...
SELECT spaceTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
  Point(15 15 15)@2001-01-25]'::stbox, 2.0, geometry 'Point(3 3 3)');
-- (1,"STBOX Z((3,3,3),(5,5,5))")
-- (2,"STBOX Z((5,3,3),(7,5,5))")
-- (3,"STBOX Z((7,3,3),(9,5,5))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]', 2.0,
  interval '2 days');
```

```
-- (1,"STBOX XT(((2,2),(4,4)),[2001-01-15,2001-01-17))")
-- (2,"STBOX XT(((4,2),(6,4)),[2001-01-15,2001-01-17))")
-- (3,"STBOX XT(((6,2),(8,4)),[2001-01-15,2001-01-17))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
  Point(15 15 15)@2001-01-25]'::stbox, 2.0, interval '2 days', 'Point(3 3 3)',
  '2001-01-15');
-- (1,"STBOX ZT(((3,3,3),(5,5,5)),[2001-01-15,2001-01-17))")
-- (2,"STBOX ZT(((5,3,3),(7,5,5))),[2001-01-15,2001-01-17)")
-- (3,"STBOX ZT(((7,3,3),(9,5,5))),[2001-01-15,2001-01-17)")
-- ...
```

- Devuelve el mosaico temporal que cubre un valor y/o una marca de tiempo

```
getValueTile(v float,vsize float,vorigin float=0.0) → tbox
getTboxTimeTile(time timestamptz,duration interval,
  torigin timestamptz='2000-01-03') → tbox
getValueTimeTile(v float,t timestamptz,vsize float,duration interval,
  vorigin float=0.0,torigin timestamptz='2000-01-03') → tbox
```

Si el origen de las dimensiones de valores y/o de tiempo no se especifica, su valor se establece por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente.

```
SELECT getValueTile(15, 2);
-- TBOXFLOAT X([14, 16))
SELECT getTboxTimeTile('2001-01-15', interval '2 days');
-- TBOX T([2001-01-13 08:00:00, 2001-01-15 08:00:00))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days');
-- TBOXFLOAT XT([14, 16),[2001-01-13 08:00:00, 2001-01-15 08:00:00))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days', 1, '2001-01-02');
-- TBOXFLOAT XT([15, 17),[2001-01-14, 2001-01-16))
```

- Devuelve el mosaico espaciotemporal que cubre un punto y/o una marca de tiempo 

```
getSpaceTile(point geometry,xsize float,[ysize float,zsize float],
  sorigin geompoint='Point(0 0 0)') → stbox
getStboxTimeTile(time timestamptz,duration interval,
  torigin timestamptz='2000-01-03') → stbox
getSpaceTimeTile(point geometry,time timestamptz,xsize float,
  [ysize float,zsize float,]duration interval,sorigin geompoint='Point(0 0 0)',
  torigin timestamptz='2000-01-03') → stbox
```

Si el origen de la dimensión espacial y/o de tiempo no se especifica, su valor se establece por defecto en `'Point(0 0 0)'` y en el lunes 3 de enero de 2000, respectivamente. En el caso de una malla espacio-temporal, `ysize` y `zsize` son opcionales. Un tamaño omitido, o dado como 0, toma el valor de `xsize`, y `zsize` solo se lee para un valor con dimensión Z. Todo tamaño es un número finito que no es negativo; de lo contrario se produce un error. El tamaño `xsize` y la duración de una malla espacio-temporal son estrictamente positivos; de lo contrario se produce un error. El SRID de las coordenadas de los mosaicos está determinado por el del cuadro de entrada y el tamaño se da en las unidades del SRID. Si se especifica el origen de las coordenadas espaciales, que debe ser un punto, su dimensionalidad y SRID deben ser iguales al del cuadro delimitador, de lo contrario se genera un error.

```
SELECT getSpaceTile(geometry 'Point(1 1 1)', 2.0);
-- STBOX Z((0,0,0),(2,2,2))
SELECT getStboxTimeTile(timestamptz '2001-01-01', interval '2 days');
-- STBOX T([2000-12-30 08:00:00, 2001-01-01 08:00:00))
SELECT getSpaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days');
-- STBOX XT(((0,0),(2,2)),[2000-12-30 08:00:00, 2001-01-01 08:00:00))
SELECT getSpaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days',
  'Point(1 1)', '2001-01-02');
-- STBOX XT(((1,1),(3,3)),[2000-12-31, 2001-01-02))
```

9.6.3. Operaciones de cuadro delimitador

Estas operaciones fragmentan el cuadro delimitador de un valor temporal con respecto a un mosaico multidimensional. Ofrecen una alternativa a las operaciones de la Sección 9.5 para dividir los cuadros delimitadores especificando el tamaño máximo de los cuadros en las distintas dimensiones.

- Devuelve una matriz de rangos de valores o de tiempo obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores o de tiempo

```
valueBins(tnumber,vsize number,vorigin number=0,borderInc boolean=true) → numspan[]
timeBins(ttype,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → tstzspan[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. Si no se especifica el origen de los valores, se establece por defecto en 0 para los rangos de valores y en el lunes 3 de enero de 2000 para los rangos de tiempo. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se genera el rango que comienza en ese borde y solo lo contiene.

```
SELECT valueBins(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 3);
-- {"[0, 3)","[3, 6)"}
SELECT valueBins(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3, 1);
-- {"[1, 4)","[4, 7)"}
```

```
SELECT timeBins(ttext '{AAA@2001-01-01, BBB@2001-01-02, AAA@2001-01-03, CCC@2001-01-04,
  AAA@2001-01-05}', '3 days');
/* {"[1999-12-31 08:00:00, 2001-01-03 08:00:00)","[2001-01-03 08:00:00, 2001-01-06
   08:00:00)"} */
SELECT timeBins(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', '3 days', '2001-01-01');
-- {"[2001-01-01, 2001-01-04)","[2001-01-04, 2001-01-07)"}
```

- Devuelve una matriz de cuadros temporales obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores y/o tiempo

```
valueBoxes(tnumber,vsize number,vorigin number=0,borderInc boolean=true) → tbox[]
timeBoxes(tnumber,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → tbox[]
valueTimeBoxes(tnumber,vsize number,duration interval,vorigin number=0,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → tbox[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. Si el origen de la dimensión de valores y/o de tiempo no se especifica, se establece por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente. El tamaño de los valores y la duración de una malla de valores y tiempo son estrictamente positivos; de lo contrario se produce un error. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se generan las cajas que comienzan en ese borde y solo lo contienen.

```
SELECT valueBoxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 3);
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-05])","TBOXINT XT([4, 5),[2001-01-04,
   2001-01-04])"} */
SELECT timeBoxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', '3 days');
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-03])","TBOXINT XT([1, 5),[2001-01-04,
   2001-01-05])"} */
SELECT valueTimeBoxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 3, '3 days');
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-03])","TBOXINT XT([1, 2),[2001-01-05,
   2001-01-05])","TBOXINT XT([4, 5),[2001-01-04, 2001-01-04])"} */
```

```
SELECT valueTimeBoxes(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3, '3 days', 1, '2001-01-01');
/* {"TBOXFLOAT XT([1, 4),[2001-01-01, 2001-01-04))","TBOXFLOAT XT([1, 4),(2001-01-04,
   2001-01-05])","TBOXFLOAT XT([4, 4],[2001-01-04, 2001-01-04])"} */
```

- Devuelve una matriz de cuadros espaciotemporales obtenidos a partir de los instantes o segmentos de un punto temporal con respecto a un mosaico espacial y/o temporal

```
spaceBoxes({tgeompoint,tgeometry,tpose,tpcpoint,trgeometry},xsize float,
  [ysize float,zsize float,]
  sorigin geompoint='Point(0 0 0)',borderInc boolean=true) → stbox[]
timeBoxes({tgeompoint,tgeometry,tpose,tpcpoint,trgeometry},duration interval,
  torigin timestamptz='2000-01-03',
  borderInc boolean=true) → stbox[]
spaceTimeBoxes({tgeompoint,tgeometry,tpose,tpcpoint,trgeometry},xsize float,
  [ysize float,zsize float,]duration interval,
  sorigin geompoint='Point(0 0 0)',torigin timestamptz='2000-01-03',
  borderInc boolean=true) → stbox[]
```

La elección entre instantes o segmentos depende de si la interpolación es discreta o continua. Los argumentos `ysize` y `zsize` son opcionales. Un tamaño omitido, o dado como 0, toma el valor de `xsize`, y `zsize` solo se lee para un valor con dimensión Z. Todo tamaño es un número finito que no es negativo; de lo contrario se produce un error. El tamaño `xsize` y la duración de una malla espacio-temporal son estrictamente positivos; de lo contrario se produce un error. El SRID de las coordenadas del mosaico se determina por el punto temporal y los tamaños se dan en las unidades del SRID. Si se proporciona el origen de las coordenadas espaciales, que debe ser un punto, su dimensionalidad y SRID deben ser iguales a los del punto temporal, de lo contrario se genera un error. Si el origen de la dimensión espacial y/o el tiempo no se especifica, su valor se establece por defecto en `'Point(0 0 0)'` y en el lunes 3 de enero de 2000, respectivamente. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se generan mosaicos adicionales que contienen el borde. Una pose temporal y un punto de nube de puntos temporal responden a estas funciones en el punto geométrico temporal al que se resuelve su posición, de modo que un valor con interpolación lineal se divide en mosaicos a lo largo de su trayectoria y no solo en sus instantes.

```
SELECT spaceBoxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03, Point(4 4)@2001-01-04, Point(1 1)@2001-01-05}', 3);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-05])","STBOX
   XT(((4,4),(4,4)),[2001-01-04, 2001-01-04])"} */
SELECT timeBoxes(tgeompoint '{Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02,
  Point(1 1 1)@2001-01-03, Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-05}',
  interval '2 days', '2001-01-01');
/* {"STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01, 2001-01-02])","STBOX
   ZT(((1,1,1),(4,4,4)),[2001-01-03, 2001-01-04])","STBOX
   ZT(((1,1,1),(1,1,1)),[2001-01-05, 2001-01-05])"} */
SELECT spaceTimeBoxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03, Point(4 4)@2001-01-04, Point(1 1)@2001-01-05}', 3, '3 days');
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-03])","STBOX
   XT(((1,1),(1,1)),[2001-01-05, 2001-01-05])","STBOX XT(((4,4),(4,4)),[2001-01-04,
   2001-01-04])"} */
SELECT spaceTimeBoxes(tgeompoint '[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02,
  Point(1 1 1)@2001-01-03, Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-05]', 3,
  interval '3 days', 'Point(1 1 1)', '2001-01-01');
/* {"STBOX ZT(((1,1,1),(4,4,4)),[2001-01-01, 2001-01-04))","STBOX
   ZT(((1,1,1),(4,4,4)),(2001-01-04, 2001-01-05])","STBOX
   ZT(((4,4,4),(4,4,4)),[2001-01-04, 2001-01-04])"} */
SELECT spaceBoxes(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]', 2.0);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02))","STBOX
   XT(((2,2),(3,3)),[2001-01-02, 2001-01-03])"} */
```

9.6.4. Operaciones de división

Estas funciones fragmentan un valor temporal con respecto a una secuencia de intervalos (ver la Sección 9.6.1) o un mosaico multidimensional (ver la Sección 9.6.2).

- Fragmenta un número temporal con respecto a intervalos de valores y/o de tiempo {}

```
valueSplit(tnumber,vsize number,vorigin number=0,
  borderInc boolean=true) → {(number,tnumber)}
timeSplit(ttype,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(time,temp)}
valueTimeSplit(tnumber,vsize number,duration interval,vorigin number=0,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(number,time,tnumber)}
```

El resultado es un conjunto de pares o un conjunto de triples. Si no se especifica el origen de la dimensión de valores o temporal, se establecen por defecto en 0 y en el lunes 3 de enero de 2000, respectivamente. El tamaño de los valores y la duración de una malla de valores y tiempo son estrictamente positivos; de lo contrario se produce un error. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se generan los fragmentos que comienzan en ese borde y solo lo contienen.

```
SELECT (sp).number,
  (sp).tnumber FROM (SELECT valueSplit(tint '[1@2001-01-01, 2@2001-01-02, 5@2001-01-05,
  10@2001-01-10]', 2) AS sp) t;
--  0 | {[1@2001-01-01, 1@2001-01-02)}
--  2 | {[2@2001-01-02, 2@2001-01-05)}
--  4 | {[5@2001-01-05, 5@2001-01-10)}
-- 10 | {[10@2001-01-10]}
SELECT valueSplit(tfloat '[1@2001-01-01, 10@2001-01-10)', 2.0, 1.0);
-- (1,"{[1@2001-01-01, 3@2001-01-03)}")
-- (3,"{[3@2001-01-03, 5@2001-01-05)}")
-- (5,"{[5@2001-01-05, 7@2001-01-07)}")
-- (7,"{[7@2001-01-07, 9@2001-01-09)}")
-- (9,"{[9@2001-01-09, 10@2001-01-10)}")
```

```
SELECT (ts).time, (ts).temp
FROM (SELECT timeSplit(tfloat '[1@2001-02-01, 10@2001-02-10)', '2 days') AS ts) t;
-- 2001-01-31 | [1@2001-02-01, 2@2001-02-02)
-- 2001-02-02 | [2@2001-02-02, 4@2001-02-04)
-- 2001-02-04 | [4@2001-02-04, 6@2001-02-06)
-- ...
SELECT (ts).time,
  astext((ts).temp) AS temp FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01,
  Point(10 10)@2001-02-10]', '2 days', '2001-02-01') AS ts) AS t;
-- 2001-02-01 | [POINT(1 1)@2001-02-01, POINT(3 3)@2001-02-03)
-- 2001-02-03 | [POINT(3 3)@2001-02-03, POINT(5 5)@2001-02-05)
-- 2001-02-05 | [POINT(5 5)@2001-02-05, POINT(7 7)@2001-02-07)
-- ...
```

Observe que se puede fragmentar un valor temporal en intervalos de tiempo cíclicos (en lugar de lineales). Los siguientes dos ejemplos muestran cómo fragmentar un valor temporal por hora y por día de la semana.

```
SELECT (ts).time::time AS hour,
  merge((ts).temp) AS temp FROM (SELECT timeSplit(tfloat '[1@2001-01-01, 10@2001-01-03]',
  '1 hour') AS ts) t GROUP BY hour ORDER BY hour;
/* 00:00:00 | {[1@2001-01-01, 1.1875@2001-01-01 01:00:00),
   [5.5@2001-01-02, 5.6875@2001-01-02 01:00:00)} */
/* 01:00:00 | {[1.1875@2001-01-01 01:00:00, 1.375@2001-01-01 02:00:00),
   [5.6875@2001-01-02 01:00:00, 5.875@2001-01-02 02:00:00)} */
/* 02:00:00 | {[1.375@2001-01-01 02:00:00, 1.5625@2001-01-01 03:00:00),
   [5.875@2001-01-02 02:00:00, 6.0625@2001-01-02 03:00:00)} */
/* 03:00:00 | {[1.5625@2001-01-01 03:00:00, 1.75@2001-01-01 04:00:00),
```

```
   [6.0625@2001-01-02 03:00:00, 6.25@2001-01-02 04:00:00)} */
-- ...
SELECT EXTRACT(DOW FROM (ts).time) AS dow_no, TO_CHAR((ts).time, 'Dy') AS dow,
  asText(round(merge((ts).temp),
  2)) AS temp FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-01-01,
  Point(10 10)@2001-01-14)', '1 hour') AS ts) t GROUP BY dow, dow_no ORDER BY dow_no;
/* 0 | Sun | {[POINT(1 1)@2001-01-01, POINT(1.69 1.69)@2001-01-02),
   [POINT(5.85 5.85)@2001-01-08, POINT(6.54 6.54)@2001-01-09)} */
/* 1 | Mon | {[POINT(1.69 1.69)@2001-01-02, POINT(2.38 2.38)@2001-01-03),
   [POINT(6.54 6.54)@2001-01-09, POINT(7.23 7.23)@2001-01-10)} */
/* 2 | Tue | {[POINT(2.38 2.38)@2001-01-03, POINT(3.08 3.08)@2001-01-04),
   [POINT(7.23 7.23)@2001-01-10, POINT(7.92 7.92)@2001-01-11)} */
-- ...
```

```
SELECT (sp).number, (sp).time,
  (sp).tnumber FROM (SELECT valueTimeSplit(tint '[1@2001-02-01, 2@2001-02-02,
  5@2001-02-05, 10@2001-02-10]', 5, '5 days') AS sp) t;
--  0 | 2001-02-01 | {[1@2001-02-01, 2@2001-02-02, 2@2001-02-05)}
--  5 | 2001-02-01 | {[5@2001-02-05, 5@2001-02-06)}
--  5 | 2001-02-06 | {[5@2001-02-06, 5@2001-02-10)}
-- 10 | 2001-02-06 | {[10@2001-02-10]}
SELECT (sp).number, (sp).time,
  (sp).tnumber FROM (SELECT valueTimeSplit(tfloat '[1@2001-02-01, 10@2001-02-10)', 5.0,
  '5 days', 1.0, '2001-02-01') AS sp) t;
-- 1 | 2001-01-01 | [1@2001-01-01, 6@2001-01-06)
-- 6 | 2001-01-06 | [6@2001-01-06, 10@2001-01-10)
```

- Fragmenta un punto temporal con respecto a una malla espacial o espacio-temporal

```
spaceSplit({tgeompoint,tgeogpoint,tgeometry,tpose,tpcpoint},xsize float,
  [ysize float,zsize float,]
  origin point='Point(0 0 0)',bitmatrix boolean=true,
  borderInc boolean=true) → {(point,tpoint)}
timeSplit(tpoint,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(time,tpoint)}
spaceTimeSplit({tgeompoint,tgeogpoint,tgeometry,tpose,tpcpoint},xsize float,
  [ysize float,zsize float,]
  duration interval,sorigin point='Point(0 0 0)',
  torigin timestamptz='2000-01-03',bitmatrix boolean=true,
  borderInc boolean=true) → {(point,time,tpoint)}
```

Una pose temporal y un punto de nube de puntos temporal responden a estas funciones en el punto geométrico temporal al que se resuelve su posición, de modo que un valor con interpolación lineal se divide en mosaicos a lo largo de su trayectoria y no solo en sus instantes. La segunda columna de una división lleva el tipo del primer argumento, de modo que dividir una pose temporal devuelve poses temporales. El resultado es un conjunto de pares o un conjunto de triples. Si el origen de la dimensión espacial y/o el tiempo no se especifica, su valor se establece por defecto en `'Point(0 0 0)'` y en el lunes 3 de enero de 2000, respectivamente. Los argumentos `ysize` y `zsize` son opcionales. Un tamaño omitido, o dado como 0, toma el valor de `xsize`, y `zsize` solo se lee para un valor con dimensión Z. Todo tamaño es un número finito que no es negativo; de lo contrario se produce un error. El tamaño `xsize` y la duración de una malla espacio-temporal son estrictamente positivos; de lo contrario se produce un error. Si no se especifica el argumento `bitmatrix`, el cálculo utilizará una matriz de bits para acelerar el proceso. El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se generan mosaicos adicionales que contienen el borde.

Un punto temporal geodésico recorre el círculo máximo entre dos posiciones, de modo que se divide en los mosaicos que ese círculo cruza y no en los de la línea recta en longitud y latitud: un viaje cuyas dos posiciones están en la latitud 61 alcanza la latitud 62.485889 y entra en la fila de mosaicos al norte de aquella en la que esa línea permanece. La cuadrícula que se aplica a un valor geodésico toma un origen de tipo geografía y sus mosaicos son geodésicos, de modo que un mosaico y el valor restringido a él expresan coordenadas de una misma clase. La matriz de bits se rellena a partir de la línea recta entre dos posiciones, que un círculo máximo abandona, de modo que un valor geodésico se divide sin ella y en su lugar se lee cada mosaico de su caja delimitadora.

```
SELECT ST_AsText((sp).point) AS point, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1)@2001-03-01, Point(10 10)@2001-03-10]',
  2.0) AS sp) t;
-- POINT(0 0) | {[POINT(1 1)@2001-03-01, POINT(2 2)@2001-03-02)}
-- POINT(2 2) | {[POINT(2 2)@2001-03-02, POINT(4 4)@2001-03-04)}
-- POINT(4 4) | {[POINT(4 4)@2001-03-04, POINT(6 6)@2001-03-06)}
-- ...
SELECT ST_AsText((sp).point) AS point, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1 1)@2001-03-01,
  Point(10 10 10)@2001-03-10]', 2.0, geometry 'Point(1 1 1)') AS sp) t;
-- POINT Z(1 1 1) | {[POINT Z (1 1 1)@2001-03-01, POINT Z (3 3 3)@2001-03-03)}
-- POINT Z(3 3 3) | {[POINT Z (3 3 3)@2001-03-03, POINT Z (5 5 5)@2001-03-05)}
-- POINT Z(5 5 5) | {[POINT Z (5 5 5)@2001-03-05, POINT Z (7 7 7)@2001-03-07)}
-- ...
SELECT ST_AsText((sp).point) AS point, asText((sp).tpose) AS tpose
FROM (SELECT spaceSplit(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]', 2.0) AS sp) t;
-- POINT(0 0) | {[Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.1)@2001-01-02)}
-- POINT(2 2) | {[Pose(POINT(2 2),0.1)@2001-01-02, Pose(POINT(3 3),0.1)@2001-01-03]}
SELECT ST_AsText((sp).point) AS point, asText((sp).tpoint, 6) AS tpoint
FROM (SELECT spaceSplit(tgeogpoint '[Point(10 61)@2001-03-01,
  Point(50 61)@2001-03-03]', 20.0) AS sp) t;
-- POINT(0 60)  | {[POINT(10 61)@2001-03-01, POINT(20 62.124941)@2001-03-01  ↩
    12:17:24.687202)}
-- POINT(20 60) | {[POINT(20 62.124941)@2001-03-01 12:17:24.687202,
--                 POINT(40 62.124941)@2001-03-02 11:42:35.312797)}
-- POINT(40 60) | {[POINT(40 62.124941)@2001-03-02 11:42:35.312797, POINT(50 61)@2001 ↩
    -03-03]}
```

```
SELECT (sp).time,
  astext((sp).temp) AS tpoint FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01,
  Point(10 10)@2001-02-10]', interval '2 days') AS sp) t;
-- 2001-01-31 | [POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02)
-- 2001-02-02 | [POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04)
-- 2001-02-04 | [POINT(4 4)@2001-02-04, POINT(6 6)@2001-02-06)
-- ...
```

```
SELECT ST_AsText((sp).point) AS point, (sp).time, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
  2.0, interval '2 days') AS sp) t;
-- POINT(0 0) | 2001-01-31 | {[POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02)}
-- POINT(2 2) | 2001-01-31 | {[POINT(2 2)@2001-02-02]}
-- POINT(2 2) | 2001-02-02 | {[POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04)}
-- ...
SELECT ST_AsText((sp).point) AS point, (sp).time, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1 1)@2001-02-01,
  Point(10 10 10)@2001-02-10]', 2.0, interval '2 days', 'Point(1 1 1)',
  '2001-03-01') AS sp) t;
-- POINT Z(1 1 1) | 2001-02-01 | {[POINT Z(1 1 1)@2001-02-01, POINT Z(3 3 3)@2001-02-03)}
-- POINT Z(3 3 3) | 2001-02-01 | {[POINT Z(3 3 3)@2001-02-03]}
-- POINT Z(3 3 3) | 2001-02-03 | {[POINT Z(3 3 3)@2001-02-03, POINT Z (5 5 5)@2001-02-05)}
-- ...
```

Capítulo 10

Tipos temporales: Agregación e indexación

10.1. Agregación

Las funciones agregadas temporales generalizan las funciones agregadas tradicionales. Su semántica es que calculan el valor de la función en cada instante de la *unión* de las extensiones temporales de los valores a agregar. En contraste, recuerde que todas las otras funciones que manipulan tipos temporales calculan el valor de la función en cada instante de la *intersección* de las extensiones temporales de los argumentos.

Las funciones agregadas temporales son las siguientes:

- Para todos los tipos temporales, la función `tCount` generaliza la función traditional `count`. El conteo temporal se puede utilizar para calcular en cada momento el número de objetos disponibles (por ejemplo, el número de coches en un área).
- Para todos los tipos temporales, la función `extent` devuelve un cuadro delimitador que engloba un conjunto de valores temporales. Dependiendo del tipo de base, el resultado de esta función puede ser un `tstzspan`, un `tbox` o un `stbox`.
- Para el tipo booleano temporal, las funciones `tAnd` y `tOr` generalizan las funciones traditionales `and` y `or`.
- Para los tipos numéricos temporales hay dos tipos de funciones agregadas temporales. Las funciones `tmin`, `tMax`, `tSum` y `tAvg` generalizan las funciones traditionales `min`, `max`, `sum` y `avg`. Además, las funciones `wMin`, `wMax`, `wCount`, `wSum` y `wAvg` son versiones de ventana (o acumulativas) de las funciones tradicionales que, dado un intervalo de tiempo w, calculan el valor de la función en un instante t considerando los valores durante el intervalo [t-w, t]. Todas las funciones agregadas de ventana están disponibles para enteros temporales, mientras que para flotantes temporales sólo son significativos el mínimo y el máximo de ventana.
- Para el tipo texto temporal, las funciones `tMin` y `tMax` generalizan las funciones traditionales `min` y `max`.
- Finalmente, para puntos temporales, la función `tCentroid` generaliza la función `ST_Centroid` proporcionada por PostGIS. Por ejemplo, dado un conjunto de objetos que se mueven juntos (es decir, un convoy o una bandada), el centroide temporal producirá un punto temporal que representa en cada instante el centro geométrico (o el centro de masa) de todos los objetos en movimiento.

---

nota
Los nombres `tMin`, `tMax`, `merge`, `appendInstant` y `appendSequence` designan tanto una función escalar como una agregación, que PostgreSQL distingue por sus argumentos: la función escalar `tMin(tbox)` es el mínimo de una caja, la agregación `tMin(tint)` el mínimo temporal de un conjunto de filas. DuckDB, Apache Spark y Apache Flink admiten una sola función por nombre y llaman a estas agregaciones `tMinAgg`, `tMaxAgg`, `mergeAgg`, `appendInstantAgg` y `appendSequenceAgg` (Sección 18.10.1). DuckDB llama `wavg` a su media ponderada y compara los nombres sin distinguir mayúsculas, por lo que los tres motores llaman `wAvgAgg` a la media de ventana.

En los ejemplos que siguen, suponemos que las tablas `Department` y `Trip` contienen las dos tuplas introducidas en la Sección 4.2.

- Conteo temporal

```
tCount(ttype) → {tintSeq,tintSeqSet}
```

```
SELECT tCount(NoEmps) FROM Department;
-- {[1@2001-01-01, 2@2001-02-01, 1@2001-08-01, 1@2001-10-01)}
```

- Extensión del cuadro delimitador

```
extent(ttype) → {tstzspan,tbox,stbox}
```

```
SELECT extent(noEmps) FROM Department;
-- TBOX XT((4,12),[2001-01-01,2001-10-01])
SELECT extent(Trip) FROM Trips;
-- STBOX XT(((0,0),(3,3)),[2001-01-01 08:00:00, 2001-01-01 08:20:00))
```

- Y, o temporal

```
tOr(tbool) → tbool
tAnd(tbool) → tbool
```

```
SELECT tAnd(NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-04-01, f@2001-10-01)}
SELECT tOr(NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-08-01, f@2001-10-01)}
```

- Mínimo, máximo, suma y promedio temporal

```
tMin(ttype) → ttype
tMax(ttype) → ttype
tSum(tnumber) → {tnumberSeq,tnumberSeqSet}
tAvg(tnumber) → {tfloatSeq,tfloatSeqSet}
```

```
SELECT tMin(NoEmps) FROM Department;
-- {[10@2001-01-01, 4@2001-02-01, 6@2001-06-01, 6@2001-10-01)}
SELECT tMax(NoEmps) FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-01, 6@2001-10-01)}
SELECT tSum(NoEmps) FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01, 18@2001-06-01, 6@2001-08-01,
   6@2001-10-01)} */
SELECT tAvg(NoEmps) FROM Department;
/* {[10@2001-01-01, 10@2001-02-01), [7@2001-02-01, 7@2001-04-01),
   [8@2001-04-01, 8@2001-06-01), [9@2001-06-01, 9@2001-08-01),
   [6@2001-08-01, 6@2001-10-01) */
```

- Conteo de ventana

```
wCount(tnumber,interval) → {tintSeq,tintSeqSet}
```

```
SELECT wCount(NoEmps, interval '2 days') FROM Department;
/* {[1@2001-01-01, 2@2001-02-01, 3@2001-04-01, 2@2001-04-03, 3@2001-06-01, 2@2001-06-03,
   1@2001-08-03, 1@2001-10-03)} */
```

- Mínimo, máximo, suma y promedio de ventana

```
wMin(tnumber,interval) → {tnumberSeq,tnumberSeqSet}
wMax(tnumber,interval) → {tnumberDiscSeq,tnumberSeqSet}
wSum(tint,interval) → {tintSeq,tintSeqSet}
wAvg(tint,interval) → {tfloatSeq,tfloatSeqSet}
```

```
SELECT wMin(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 4@2001-04-01, 6@2001-06-03, 6@2001-10-03)}
SELECT wMax(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-03, 6@2001-10-03)}
SELECT wSum(NoEmps, interval '2 days') FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 26@2001-04-01, 16@2001-04-03, 22@2001-06-01,
   18@2001-06-03, 6@2001-08-03, 6@2001-10-03)} */
SELECT round(wAvg(NoEmps, interval '2 days'), 3) FROM Department;
/*  Interp=Step;{[10@2001-01-01, 7@2001-02-01, 8.667@2001-04-01, 8@2001-04-03,
   7.333@2001-06-01, 9@2001-06-03, 6@2001-08-03, 6@2001-10-03)} */
```

- Centroide temporal

```
tCentroid(tgeompoint) → tgeompoint
```

```
SELECT tCentroid(Trip) FROM Trips;
/* {[POINT(0 0)@2001-01-01 08:00:00, POINT(1 0)@2001-01-01 08:05:00),
   [POINT(0.5 0)@2001-01-01 08:05:00, POINT(1.5 0.5)@2001-01-01 08:10:00,
   POINT(2 1.5)@2001-01-01 08:15:00),
   [POINT(2 2)@2001-01-01 08:15:00, POINT(3 3)@2001-01-01 08:20:00)} */
```

10.2. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tabla de tipos temporales. El índice GiST implementa un árbol R, mientras que el índice SP-GiST implementa un árbol cuádruple n-dimensional. Ejemplos de creación de índices son los siguientes:

```
CREATE INDEX Department_NoEmps_Gist_Idx ON Department USING Gist(NoEmps);
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador para los tipos temporales. Como se explica en el Capítulo 4, estos son

- el tipo `tstzspan` para los tipos `tbool` y `ttext`,
- el tipo `tbox` par los tipos `tint` y `tfloat`,
- el tipo `stbox` para los tipos `tgeompoint` y `tgeogpoint`.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores (consulte la Sección 5.3 para obtener más información):

- `<<`, `&<`, `&>`, `>>`, que sólo consideran la dimensión de valores en tipos alfanuméricos temporales,
- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, `&</`, `<</`, `/>>` y `/&>`, que sólo consideran la dimensión espacial en tipos de puntos temporales,
- `&<#`, `<<#`, `#>>`, `#&>`, que sólo consideran la dimensión temporal para todos los tipos temporales,
- `&&`, `@>`, `<@`, `~=` y `|=|`, que consideran tantas dimensiones como compartan la columna indexada y el argumento de consulta. Estos operadores trabajan en cuadros delimitadores (es decir, `tstzspan`, `tbox` o `stbox`), no los valores completos.

Por ejemplo, dado el índice definido anteriormente en la tabla `Department` y una consulta que implica una condición con el operador `&&` (superposición), si el argumento derecho es un flotante temporal, entonces se consideran tanto el valor como las dimensiones de tiempo para filtrar las tuplas de la relación, mientras que si el argumento derecho es un valor flotante, un rango flotante o un tipo de tiempo, entonces el valor o la dimensión de tiempo se utilizará para filtrar las tuplas de la relación. Además, se puede construir un cuadro delimitador a partir de un valor/rango y/o una marca de tiempo/período, que se puede usar para filtrar las tuplas de la relación. Ejemplos de consultas que utilizan el índice en la tabla `Department` definida anteriormente se dan a continuación.

```
SELECT * FROM Department WHERE NoEmps && intspan '[1, 5)';
SELECT * FROM Department WHERE NoEmps && tstzspan '[2001-04-01, 2001-05-01)';
SELECT * FROM Department WHERE NoEmps &&
  tbox(intspan '[1, 5)', tstzspan '[2001-04-01, 2001-05-01)');
SELECT * FROM Department WHERE NoEmps &&
  tfloat '{[1@2001-01-01, 1@2001-02-01), [5@2001-04-01, 5@2001-05-01)}';
```

Del mismo modo, los ejemplos de consultas que utilizan el índice en la tabla `Trips` definida anteriormente se dan a continuación.

```
SELECT * FROM Trips WHERE Trip && geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))';
SELECT * FROM Trips WHERE Trip && timestamptz '2001-01-01';
SELECT * FROM Trips WHERE Trip && tstzspan '[2001-01-01, 2001-01-05)';
SELECT * FROM Trips WHERE Trip &&
  stbox(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))', tstzspan '[2001-01-01, 2001-01-05]');
SELECT * FROM Trips WHERE Trip &&
  tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02, Point(1 1)@2001-01-05)}';
```

Finalmente, se pueden crear índices de árbol B para columnas de tabla de todos los tipos temporales. Para este tipo de índice, la única operación útil es la igualdad. Hay un orden de clasificación de árbol B definido para valores de tipos temporales, con los correspondientes operadores <, <=, > y >=, pero el orden es bastante arbitrario y no suele ser útil en el mundo real. El soporte de árbol B para tipos temporales está destinado principalmente a permitir la clasificación interna en las consultas, en lugar de la creación de índices reales.

Para acelerar algunas de las funciones de los tipos temporales, se puede agregar en la cláusula `WHERE` de las consultas una comparación de cuadro delimitador que hace uso de los índices disponibles. Por ejemplo, este sería típicamente el caso de las funciones que proyectan los tipos temporales a las dimensiones de valor/espacio y/o tiempo. Esto filtrará las tuplas con un índice como se muestra en la siguiente consulta.

```
SELECT atTime(T.Trip, tstzspan '[2001-01-01, 2001-01-02)') FROM Trips T
-- Bounding box index filtering
WHERE T.Trip && tstzspan '[2001-01-01, 2001-01-02)';
```

En el caso de los geometrías temporales, todas las relaciones espaciales con la semántica alguna vez o siempre (ver la Sección 8.9) incluyen automáticamente una comparación de cuadro delimitador que hará uso de cualquier índice que esté disponible en los puntos temporales. Por esta razón, la primera versión de las relaciones se usa típicamente para filtrar las tuplas con la ayuda de un índice al calcular las relaciones temporales como se muestra en la siguiente consulta.

```
SELECT tIntersects(T.Trip, R.Geom) FROM Trips T, Regions R
-- Bounding box index filtering
WHERE eIntersects(T.Trip, R.Geom);
```

10.3. Estadísticas y selectividad

10.3.1. Colecta de estadísticas

El planificador de PostgreSQL se basa en información estadística sobre el contenido de las tablas para generar el plan de ejecución más eficiente para las consultas. Estas estadísticas incluyen una lista de algunos de los valores más comunes en cada columna y un histograma que muestra la distribución aproximada de datos en cada columna. Para tablas grandes, se toma una muestra aleatoria del contenido de la tabla, en lugar de examinar cada fila. Esto permite analizar tablas grandes en poco tiempo. La información estadística es recopilada por el comando `ANALYZE` y es almacenada en la tabla de catálogo `pg_statistic`. Dado que diferentes tipos de estadísticas pueden ser apropiados para diferentes tipos de datos, la tabla sólo almacena estadísticas muy generales (como el número de valores nulos) en columnas dedicadas. Todo lo demás se almacena en cinco “slots”, que son pares de columnas de matriz que almacenan las estadísticas de una columna de un tipo arbitrario.

Las estadísticas recopiladas para tipos de tiempo y tipos temporales se basan en las recopiladas por PostgreSQL para tipos escalares y tipos de rango. Para tipos escalares, como `float`, se recopilan las siguientes estadísticas:

1. fracción de valores nulos,
2. ancho promedio, en bytes, de valores no nulos,
3. número de diferentes valores no nulos,
4. matriz de los valores más comunes y matriz de sus frecuencias,
5. histograma de valores, donde se excluyen los valores más comunes,
6. correlación entre el orden de filas físico y lógico.

Para los tipos de rango, como `tstzspan`, se recopilan tres histogramas adicionales:

7. histograma de longitud de rangos no vacíos,
8. histogramas de límites superior e inferior.

Para geometrías, además de (1)–(3), se recopilan las siguientes estadísticas:

9. número de dimensiones de los valores, cuadro delimitador N-dimensional, número de filas en la tabla, número de filas en la muestra, número de valores no nulos,
10. Histograma N-dimensional que divide el cuadro delimitador en varias celdas y mantiene la proporción de valores que se cruzan con cada celda.

Las estadísticas recopiladas para columnas de los tipos de tiempo y de rango `tstzset`, `tstzspan`, `tstzspanset`, `intspan` y `floatspan` replican las recopiladas por PostgreSQL para `tstzrange`. Esto es claro para los tipos de rango en MobilityDB, que son versiones más eficientes de los tipos de rango en PostgreSQL. Para los tipos `tstzset` y `tstzspanset`, un valor se convierte en su período delimitador y luego se recopilan las estadísticas del tipo `tstzspan`.

Las estadísticas recopiladas para columnas de los tipos temporales dependen del subtipo temporal y del tipo base. Además de las estadísticas (1)–(3) que se recopilan para todos los tipos temporales, las estadísticas se recopilan para la dimensiónes de tiempo y de valor de forma independiente. Más precisamente, se recopilan las siguientes estadísticas para la dimensión de tiempo:

- Para columnas de subtipo instante, las estadísticas (4)–(6) se recopilan para las marcas de tiempo.
- Para columnas de otro subtipo, las estadísticas (7)–(8) se recopilan para los períodos delimitadores.

Las siguientes estadísticas se recopilan para la dimensión de valores:

- Para columnas de tipos temporales con interpolación escalonada (es decir, `tbool`, `ttext` o `tint`):
  - Para el subtipo instante, las estadísticas (4)–(6) se recopilan para los valores.
  - Para todas los demás subtipos, las estadísticas (7)–(8) se recopilan para los valores.
- Para columnas de tipos temporales flotantes (es decir, `tfloat`):
  - Para el subtipo instante, las estadísticas (4)–(6) se recopilan para los valores.
  - Para todas los demás subtipos, las estadísticas (7)–(8) se recopilan por los rangos delimitadores de valores.
- Para columnas de tipos de puntos temporales (es decir, `tgeompoint` y `tgeogpoint`) las estadísticas (9)–(10) se compilan para los puntos.

10.3.2. Estimación de la selectividad

Los operadores booleanos en PostgreSQL se pueden asociar con dos funciones de selectividad, que calculan la probabilidad de que un valor de un tipo dado coincida con un criterio dado. Estas funciones de selectividad se basan en las estadísticas recopiladas. Hay dos tipos de funciones de selectividad. Las funciones de selectividad de *restricción* intentan estimar el porcentaje de filas en una tabla que satisfacen una condición en la cláusula `WHERE` de la forma `column OP constant`. Por otro lado, las funciones de selectividad de *unión* intentan estimar el porcentaje de filas en una tabla que satisfacen una condición en la cláusula `WHERE` de la forma `table1.column1 OP table2.column2`.

MobilityDB define 23 clases de operadores booleanos (como=, `<`, `&&`, `<<`, etc.), cada uno de los cuales puede tener como argumentos izquierdo o derecho un tipo PostgreSQL (como `integer`, `timestamptz`, etc.) o un tipo MobilityDB (como `tstzspan`, `tint`, etc.). Como consecuencia, existe un número muy elevado de operadores con diferentes argumentos a considerar para las funciones de selectividad. El enfoque adoptado fue agrupar estas combinaciones en clases correspondientes a las dimensiones de valor o de tiempo. Las clases corresponden al tipo de estadísticas recopiladas como se explica en la sección anterior.

MobilityDB estima la selectividad de restricción y de combinación para tipos de tiempo, de rango y temporales.

Capítulo 11

Puntos de red temporales

Los puntos temporales que hemos considerado hasta ahora representan el movimiento de objetos que pueden moverse libremente en el espacio ya que se supone que pueden cambiar su posición de un lugar a otro sin ninguna restricción de movimiento. Este es el caso de los animales y de los objetos voladores como aviones o drones. Sin embargo, en muchos casos, los objetos no se mueven libremente en el espacio sino dentro de redes integradas espacialmente, como rutas o ferrocarriles. En este caso, es necesario tener en cuenta de las redes integradas al describir los movimientos de estos objetos en movimiento. Los puntos de red temporales tienen en cuenta estos requisitos.

En comparación con los puntos temporales de espacio libre, los puntos basados en red tienen las siguientes ventajas:

- Los puntos de red temporales proporcionan restricciones que reflejan los movimientos reales de los objetos en movimiento.
- La información geométrica no se almacena con el punto móvil, sino de una vez por todas en las redes fijas. De esta forma, las representaciones e interpolaciones de la ubicación son más precisas.
- Los puntos de red temporales son más eficientes en términos de almacenamiento de datos, actualización de ubicación, formulación de consultas e indexación. Estos aspectos se tratan más adelante en este documento.

Los puntos de red temporales se basan en pgRouting, una extensión de PostgreSQL para desarrollar aplicaciones de enrutamiento de red y realizar análisis de gráficos. Por lo tanto, los puntos de red temporal asumen que la red subyacente está definida en una tabla llamada `ways`, que tiene al menos tres columnas: `gid` que contiene el identificador de ruta único, `length` que contiene la longitud de la ruta y `the_geom` que contiene la geometría de la ruta.

Hay dos tipos de red estáticos, `npoint` (abreviatura de *network point*) y `nsegment` (abreviatura de *network segment*), que representan, respectivamente, un punto y un segmento de una ruta. Un valor `npoint` se compone de un identificador de ruta y un número flotante en el rango [0,1] que determina una posición relativa de la ruta, donde 0 corresponde al comienzo de la ruta y 1 al final de la ruta. Un valor de `nsegment` se compone de un identificador de ruta y dos números flotantes en el rango [0,1] que determinan las posiciones relativas de inicio y finalización. Un valor de `nsegment` cuyas posiciones inicial y final son iguales corresponde a un valor de `npoint`.

El tipo `npoint` sirve como tipo base para definir el tipo punto de red temporal `tnpoint`. El tipo `tnpoint` tiene una funcionalidad similar al tipo de punto temporal `tgeompoint` con la excepción de que solo considera dos dimensiones. Por lo tanto, todas las funciones y operadores descritos anteriormente para el tipo `tgeompoint` también son aplicables para el tipo `tnpoint`. Además, hay funciones específicas definidas para el tipo `tnpoint`.

11.1. Tipos de red estáticos

Un valor `npoint` es un par de la forma `(rid, position)` donde `rid` es un valor `bigint` que representa un identificador de ruta y `position` es un valor `float` en el rango [0,1] que indica su posición relativa. Los valores 0 y 1 de `position` denotan, respectivamente, la posición inicial y final de la ruta. La distancia de la ruta entre un valor de `npoint` y la posición inicial de la ruta con el identificador `rid` se calcula multiplicando `position` por `length`, donde este último es la longitud de la ruta. Ejemplos de entrada de valores de puntos de red son los siguientes:

```
SELECT npoint 'Npoint(76, 0.3)';
SELECT npoint 'Npoint(64, 1.0)';
```

La función de constructor para puntos de red tiene un argumento para el identificador de ruta y un argumento para la posición relativa. Un ejemplo de un valor de punto de red definido con la función constructora es el siguiente:

```
SELECT npoint(76, 0.3);
```

Un valor `nsegment` es un triple de la forma `(rid, startPosition, endPosition)` donde `rid` es un valor `bigint` que representa un identificador de ruta y `startPosition` y `endPosition` son valores de `float` en el rango [0,1] tal que `startPosition` $\leq$ `endPosition`. Semánticamente, un segmento de red representa un conjunto de puntos de red `(rid, position)` con `startPosition` $\leq$ `position` $\leq$ `endPosition`. Si `startPosition = 0` y `endPosition = 1`, el segmento de red es equivalente a la ruta completa. Si `startPosition = endPosition`, el segmento de red representa un único punto de red. Ejemplos de entrada de valores de puntos de red son los siguientes:

```
SELECT nsegment 'Nsegment(76, 0.3, 0.5)';
SELECT nsegment 'Nsegment(64, 0.5, 0.5)';
SELECT nsegment 'Nsegment(64, 0.0, 1.0)';
SELECT nsegment 'Nsegment(64, 1.0, 0.0)';
-- converted to nsegment 'Nsegment(64, 0.0, 1.0)';
```

Como se puede ver en el último ejemplo, los valores `startPosition` y `endPosition` se invertirán para asegurar que la condición `startPosition` $\leq$ `endPosition` siempre se satisface. La función de constructor para segmentos de red tiene un argumento para el identificador de ruta y dos argumentos opcionales para las posiciones inicial y final. Los ejemplos de valores de segmento de red definidos con la función constructora son los siguientes:

```
SELECT nsegment(76, 0.3, 0.3);
SELECT nsegment(76); -- start and end position assumed to be 0 and 1 respectively
SELECT nsegment(76, 0.5); -- end position assumed to be 1
```

Los valores del tipo `npoint` se pueden convertir al tipo `nsegment` usando un `CAST` explícito o usando la notación `::` como se muestra a continuación.

```
SELECT npoint(76, 0.33)::nsegment;
```

Los valores de los tipos de red estáticos deben satisfacer varias restricciones para que estén bien definidos. Estas restricciones se dan a continuación.

- El identificador de ruta `rid` debe encontrarse en la columna `gid` de la tabla `ways`.
- Los valores de `position`, `startPosition` y `endPosition` deben estar en el rango [0,1]. Se genera un error cuando no se cumple una de estas restricciones.

Ejemplos de valores de tipo de red estática incorrectos son los siguientes.

```
-- Incorrect rid value
SELECT npoint 'Npoint(87.5, 1.0)';
-- Incorrect position value
SELECT npoint 'Npoint(87, 2.0)';
-- rid value not found in the ways table
SELECT npoint 'Npoint(99999999, 1.0)';
```

Damos a continuación las funciones y operadores para los tipos de redes estáticas.

11.1.1. Entrada y salida

- Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)

```
asText({npoint,npoint[]}) → {text,text[]}
asText(nsegment) → text
asEWKT({npoint,npoint[]}) → {text,text[]}
asEWKT(nsegment) → text
```

```
SELECT asText(npoint 'SRID=5676;Npoint(1,0.5)');
-- NPoint(1,0.5)
SELECT asText(ARRAY[npoint 'Npoint(1,0.2)', 'Npoint(1,0.5)']);
-- {"NPoint(1,0.2)","NPoint(1,0.5)"}
SELECT asText(nsegment 'Nsegment(1,0.3,0.5)');
-- NSegment(1,0.3,0.5)
SELECT asEWKT(npoint 'SRID=5676;Npoint(1,0.5)');
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(ARRAY[npoint 'SRID=5676;Npoint(1,0.2)', 'SRID=5676;Npoint(1,0.5)']);
-- {"SRID=5676;NPoint(1,0.2)","SRID=5676;NPoint(1,0.5)"}
SELECT asEWKT(nsegment 'Nsegment(1,0.3,0.5)');
-- SRID=5676;NSegment(1,0.3,0.5)
```

- Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
asBinary(npoint,endian text='') → bytea
asEWKB(npoint,endian text='') → bytea
asHexWKB(npoint,endian text='') → text
asHexEWKB(npoint,endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina. Un punto de red toma su SRID de la red de rutas subyacente: `asEWKB` y `asHexEWKB` lo incluyen, mientras que las formas simples `asBinary` y `asHexWKB` lo omiten.

```
SELECT asBinary(npoint 'SRID=5676;Npoint(1,0.5)');
-- \x0101010000000000000000000000000000e03f
SELECT asEWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- \x01412c160000010000000000000000000000000e03f
SELECT asHexWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- 0101010000000000000000000000000E03F
SELECT asHexEWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- 01412C160000010000000000000000000000000E03F
```

- Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)

```
npointFromText(text) → npoint
npointFromEWKT(text) → npoint
nsegmentFromText(text) → nsegment
nsegmentFromEWKT(text) → nsegment
```

Un SRID indicado por la entrada extendida debe ser el SRID de la red de rutas. Cuando no hay ninguna red de rutas cargada, un punto de red no tiene SRID y el indicado no se conserva.

```
SELECT asEWKT(npointFromText(text 'Npoint(1,0.5)'));
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(npointFromEWKT(text 'SRID=5676;Npoint(1,0.5)'));
-- SRID=5676;NPoint(1,0.5)
SELECT npointFromEWKT(text 'SRID=4326;Npoint(1,0.5)');
-- ERROR:  The SRID of the npoint (4326) does not match the SRID of the network (5676)
SELECT asText(nsegmentFromText(text 'Nsegment(1,0.3,0.5)'));
-- NSegment(1,0.3,0.5)
SELECT asEWKT(nsegmentFromEWKT(text 'SRID=5676;Nsegment(1,0.3,0.5)'));
-- SRID=5676;NSegment(1,0.3,0.5)
```

- Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

```
npointFromBinary(bytea) → npoint
npointFromEWKB(bytea) → npoint
npointFromHexWKB(text) → npoint
npointFromHexEWKB(text) → npoint
```

```
SELECT asEWKT(npointFromEWKB('\x01412c160000010000000000000000000000000e03f'));
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(npointFromHexEWKB('01412C160000010000000000000000000000000E03F'));
-- SRID=5676;NPoint(1,0.5)
```

- Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB) de un segmento de red

```
asBinary(nsegment,endian text='') → bytea
asEWKB(nsegment,endian text='') → bytea
asHexWKB(nsegment,endian text='') → text
asHexEWKB(nsegment,endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina. Un segmento de red toma su SRID de la red de rutas subyacente, como un punto de red: `asEWKB` y `asHexEWKB` lo incluyen, mientras que las formas simples `asBinary` y `asHexWKB` lo omiten.

```
SELECT asBinary(nsegment 'NSegment(1,0.5,0.7)');
-- \x0101010000000000000000000000000e03f666666666666e63f
SELECT asEWKB(nsegment 'NSegment(1,0.5,0.7)');
-- \x01412c160000010000000000000000000000000e03f666666666666e63f
SELECT asHexWKB(nsegment 'NSegment(1,0.5,0.7)');
-- 0101010000000000000000000000000E03F666666666666E63F
SELECT asHexEWKB(nsegment 'NSegment(1,0.5,0.7)', 'XDR');
-- 00410000162C0000000000000013FE00000000000003FE6666666666666
```

- Entrada de un segmento de red desde la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
nsegmentFromBinary(bytea) → nsegment
nsegmentFromEWKB(bytea) → nsegment
nsegmentFromHexWKB(text) → nsegment
nsegmentFromHexEWKB(text) → nsegment
```

Un SRID indicado por una entrada extendida debe ser el SRID de la red de rutas, y la ruta debe ser una de la red.

```
SELECT nsegmentFromBinary(asBinary(nsegment 'NSegment(1,0.5,0.7)'));
-- NSegment(1,0.5,0.7)
SELECT nsegmentFromHexEWKB(asHexEWKB(nsegment 'NSegment(1,0.5,0.7)'));
-- NSegment(1,0.5,0.7)
```

11.1.2. Constructores

- Constructores de puntos o segmentos de red

```
npoint(bigint,float) → npoint
nsegment(bigint,float,float) → nsegment
```

```
SELECT npoint(76, 0.3);
SELECT nsegment(76, 0.3, 0.5);
```

11.1.3. Conversión de tipos

Los valores de los tipos `npoint` y `nsegment` se pueden convertir al tipo `geometry` utilizando un `CAST` explícito o utilizando la notación `::` como se muestra a continuación. De manera similar, los valores `geometry` del subtipo `point` o `linestring` (restringidos a dos puntos) se pueden convertir, respectivamente, a valores `npoint` y `nsegment`. Para ello, se debe encontrar la ruta sobre la que se encuentran los puntos dados, con la precisión del redondeo de un punto calculado sobre una ruta, cerca de 1e-13 para coordenadas de 100, de modo que un punto obtenido de un punto de red vuelve a él mientras que un punto fuera de la red no. Si no se encuentra dicha ruta, se devuelve un valor nulo. Un punto medido cerca de la red, como una posición GPS, se convierte con la función `npoint(geometry,float)`, cuyo segundo argumento es la distancia máxima entre el punto y la ruta: el resultado se sitúa sobre la ruta más cercana dentro de esa distancia, la de menor identificador entre rutas igualmente cercanas, y es nulo cuando ninguna ruta está dentro de esa distancia. Un índice espacial sobre la columna `the_geom` de la tabla `ways` sirve a esta búsqueda.

- Convierte entre un punto o un segmento de red y una geometría

```
{npoint,nsegment}::{geometry,geography}
geometry::{npoint,nsegment}
npoint(geometry,float) → npoint
```

Las conversiones siguen la regla de [?listitem].

```
SELECT ST_AsText(npoint(76, 0.33)::geometry, 6);
-- POINT(27.842471 43.855103)
SELECT ST_AsText(nsegment(76, 0.33, 0.66)::geometry, 6);
-- LINESTRING(27.842471 43.855103,45.316951 45.399688)
SELECT ST_AsText(nsegment(76, 0.33, 0.33)::geometry, 6);
-- POINT(27.842471 43.855103)
```

```
SELECT round(geometry 'SRID=5676;Point(27.842471 43.855103)'::npoint, 6);
-- NPoint(76,0.33)
SELECT round(geometry 'SRID=5676;LINESTRING(27.842471 43.855103,
  45.316951 45.399688)'::nsegment, 6);
-- NSegment(76,0.33,0.66)
SELECT geometry 'SRID=5676;Point(28 44)'::npoint;
-- NULL
SELECT npoint(geometry 'SRID=5676;Point(28 44)', 0.1);
-- NULL
SELECT round(npoint(geometry 'SRID=5676;Point(28 44)', 0.5), 6);
-- NPoint(76,0.333192)
```

- Construye una caja espaciotemporal a partir de un punto de red y, opcionalmente, una marca de tiempo o un período

```
stbox(npoint) → stbox
stbox(npoint,{timestamptz,tstzspan}) → stbox
```

```
SELECT stbox(npoint 'NPoint(1,0.3)');
-- SRID=5676;STBOX X((62.786633,80.143556),(62.786633,80.143556))
SELECT stbox(npoint 'NPoint(1,0.3)', timestamptz '2001-01-01');
/* SRID=5676;STBOX XT(((62.786633,80.143556),(62.786633,80.143556)),[2001-01-01,
   2001-01-01]) */
SELECT stbox(npoint 'NPoint(1,0.3)', tstzspan '[2001-01-01,2001-01-02]');
/* SRID=5676;STBOX XT(((62.786633,80.143556),(62.786633,80.143556)),[2001-01-01,
   2001-01-02]) */
```

11.1.4. Accesores

- Devuelve el identificador de ruta

```
route({npoint,nsegment}) → bigint
```

```
SELECT route(npoint 'Npoint(63, 0.3)');
-- 63
SELECT route(nsegment 'Nsegment(76, 0.3, 0.3)');
-- 76
```

- Devuelve la posición

```
getPosition(npoint) → float
```

```
SELECT getPosition(npoint 'Npoint(63, 0.3)');
-- 0.3
```

- Devuelve la posición inicial/final

```
startPosition(nsegment) → float
endPosition(nsegment) → float
```

```
SELECT startPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.3
SELECT endPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.5
```

11.1.5. Transformaciones

- Redondea la(s) posición(es) del punto de red or el segmento de red en el número de posiciones decimales

```
round({npoint,nsegment},integer=0) → {npoint,nsegment}
```

```
SELECT round(npoint(76, 0.123456789), 6);
--  NPoint(76,0.123457)
SELECT round(nsegment(76, 0.123456789, 0.223456789), 6);
--  NSegment(76,0.123457,0.223457)
```

11.1.6. Operaciones espaciales

- Devuelve el identificador de referencia espacial

```
SRID({npoint,nsegment}) → integer
```

```
SELECT SRID(npoint 'Npoint(76, 0.3)');
-- 5676
SELECT SRID(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 5676
```

Dos valores `npoint` con identificadores de ruta diferentes pueden denotar la misma ubicación, en la intersección de sus rutas. Los operadores de comparación comparan el identificador de ruta y la posición, por lo que distinguen estos dos valores. Si denotan la misma ubicación se responde sobre sus geometrías. Como el punto de un punto de red se interpola a lo largo de su ruta, dos de estos puntos calculados a lo largo de rutas diferentes pueden diferir en el redondeo de esa interpolación, por lo que sus geometrías se comparan dentro de una distancia, por ejemplo con `ST_DWithin`, en lugar de con =:

```
WITH inter(geom) AS (
  SELECT st_intersection(t1.the_geom, t2.the_geom)
  FROM ways t1, ways t2 WHERE t1.gid = 1 AND t2.gid = 2), fractions(f1, f2) AS (
  SELECT ST_LineLocatePoint(t1.the_geom, i.geom), ST_LineLocatePoint(t2.the_geom, i.geom)
  FROM ways t1, ways t2, inter i WHERE t1.gid = 1 AND t2.gid = 2)
SELECT npoint(1, f1) = npoint(2, f2) FROM fractions;
-- false
SELECT ST_DWithin(npoint(1, f1)::geometry, npoint(2, f2)::geometry, 1e-9) FROM fractions;
-- true
```

11.1.7. Comparaciones

Los operadores de comparación (=, < y así sucesivamente) para tipos de red estáticos requieren que los argumentos izquierdo y derecho sean del mismo tipo. Excepto la igualdad y la desigualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan en tipos de red estáticos.

- Comparaciones tradicionales

```
npoint {=, <>, <, >, <=, >=} npoint
nsegment {=, <>, <, >, <=, >=} nsegment
```

```
SELECT npoint 'Npoint(3, 0.5)' = npoint 'Npoint(3, 0.5)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' <> nsegment 'Nsegment(3, 0.5, 0.5)';
-- false
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' < nsegment 'Nsegment(3, 0.5, 0.6)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' > nsegment 'Nsegment(2, 0.5, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.5)' <= npoint 'Npoint(2, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.6)' >= npoint 'Npoint(1, 0.5)';
-- true
```

11.2. Puntos de red temporales

El tipo de punto de red temporal `tnpoint` permite representar el movimiento de objetos en una red. Corresponde al tipo de punto temporal `tgeompoint` restringido a coordenadas bidimensionales. Como todos los demás tipos temporales, se presenta en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores de `tnpoint` en estos subtipos.

```
SELECT tnpoint 'Npoint(1, 0.5)@2001-01-01';
SELECT tnpoint '{Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03}';
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint '{[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03], [Npoint(2, 0.6)@2001-01-04, Npoint(2, 0.6)@2001-01-05]}';
```

El tipo de punto de red temporal acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL). Los valores posibles para el modificador de tipo son `Instant`, `Sequence` y `SequenceSet`. Si no se especifica ningún modificador de tipo para una columna, se permiten valores de cualquier subtipo.

```
SELECT tnpoint(Sequence) '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint(Sequence) 'Npoint(1, 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Los valores de puntos de red temporales del subtipo de secuencia et interpolación linear o escalonada deben definirse en una única ruta. Por lo tanto, se necesita un valor de subtipo de conjunto de secuencias para representar el movimiento de un objeto que atraviesa varias rutas, incluso si no hay un espacio temporal. Por ejemplo, en el siguiente valor

```
SELECT tnpoint '{[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.5)@2001-01-03),
  [NPoint(2, 0.4)@2001-01-03, NPoint(2, 0.6)@2001-01-04)}';
```

el punto de red cambia su ruta en 2001-01-03.

Los valores de puntos de red temporal del subtipo de secuencia o conjunto de secuencias se convierten en una forma normal para que los valores equivalentes tengan representaciones idénticas. Para ello, los valores instantáneos consecutivos se fusionan cuando es posible. Tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Los ejemplos de transformación a una forma normal son los siguientes.

```
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
  NPoint(1, 0.6)@2001-01-03)';
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.6)@2001-01-03)
SELECT tnpoint '{[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.3)@2001-01-02,
  NPoint(1, 0.5)@2001-01-03), [NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04)}';
-- {[NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02, NPoint(1,0.7)@2001-01-04)}
```

11.3. Validez de los puntos de red temporal

Los valores de los puntos de red temporal deben satisfacer las restricciones especificadas en la Sección 4.3 para que estén bien definidos. Se genera un error cuando no se cumple una de estas restricciones. Ejemplos de valores incorrectos son los siguientes.

```
-- Null values are not allowed
SELECT tnpoint 'NULL@2001-01-01 08:05:00';
SELECT tnpoint 'Point(0 0)@NULL';
-- Base type is not a network point
SELECT tnpoint 'Point(0 0)@2001-01-01 08:05:00';
-- Multiple routes in a continuous sequence
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01 09:00:00, Npoint(2, 0.2)@2001-01-01 09:05:00)';
```

Damos a continuación las funciones y operadores para los tipos de puntos de red. La mayoría de las funciones para tipos temporales descritas en los capítulos precendentes se pueden aplicar para tipos de puntos de red temporales. Por lo tanto, en las firmas de las funciones, la notación `base` también representa un `npoint` y las notaciones `ttype`, `tpoint` y `tgeompoint` también representan un `tnpoint`. Además, las funciones que tienen un argumento de tipo `geometry` aceptan además un argumento de tipo `npoint`. Para evitar la redundancia, a continuación solo presentamos algunos ejemplos de estas funciones y operadores para puntos de red temporales.

11.4. Constructores

- Constructor para puntos de red temporales con valor constante

```
tnpoint(npoint,timestamptz) → tnpointInst
tnpoint(npoint,tstzset) → tnpointDiscSeq
tnpoint(npoint,tstzspan,interp text='linear') → tnpointContSeq
tnpoint(npoint,tstzspanset,interp text='linear') → tnpointSeqSet
```

```
SELECT tnpoint('Npoint(1, 0.5)', timestamptz '2001-01-01');
-- NPoint(1,0.5)@2001-01-01
SELECT tnpointSeq('Npoint(1, 0.3)', tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
-- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.3)@2001-01-03, NPoint(1,0.3)@2001-01-05}
SELECT tnpointSeq('Npoint(1, 0.5)', tstzspan '[2001-01-01, 2001-01-02]');
-- [NPoint(1,0.5)@2001-01-01, NPoint(1,0.5)@2001-01-02]
SELECT tnpointSeqSet('Npoint(1, 0.2)', tstzspanset '{[2001-01-01, 2001-01-03]}', 'step');
-- Interp=Step;{[NPoint(1,0.2)@2001-01-01, NPoint(1,0.2)@2001-01-03]}
```

- Constructor para puntos de red temporal de subtipo secuencia

```
tnpointSeq(tnpointInst[],interp text={'step','linear'},lower_inc boolean=true,
  upper_inc boolean=true) → tnpointSeq
```

```
SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.3)@2001-01-01', 'Npoint(1, 0.5)@2001-01-02',
  'Npoint(1, 0.5)@2001-01-03']);
-- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.5)@2001-01-02, NPoint(1,0.5)@2001-01-03}
SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.2)@2001-01-01', 'Npoint(1, 0.4)@2001-01-02',
  'Npoint(1, 0.5)@2001-01-03']);
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.4)@2001-01-02, NPoint(1,0.5)@2001-01-03]
```

- Constructor para puntos de red temporal de subtipo conjunto de secuencias

```
tnpointSeqSet(tnpoint[]) → tnpointSeqSet
tnpointSeqSetGaps(tnpointInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → tnpointSeqSet
```

```
SELECT tnpointSeqSet(ARRAY[tnpoint '[Npoint(1,0.2)@2001-01-01, Npoint(1,0.4)@2001-01-02,
  Npoint(1,0.5)@2001-01-03]', '[Npoint(2,0.6)@2001-01-04, Npoint(2,0.6)@2001-01-05]']);
/* {[NPoint(1,0.2)@2001-01-01, NPoint(1,0.4)@2001-01-02, NPoint(1,0.5)@2001-01-03],
   [NPoint(2,0.6)@2001-01-04, NPoint(2,0.6)@2001-01-05]} */
SELECT tnpointSeqSetGaps(ARRAY[tnpoint 'NPoint(1,0.1)@2001-01-01',
  'NPoint(1,0.3)@2001-01-03', 'NPoint(1,0.5)@2001-01-05'], '1 day');
-- {[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.3)@2001-01-03], [NPoint(1,0.5)@2001-01-05]}
```

11.5. Conversión de tipos

Un valor de punto de red temporal se puede convertir en y desde un punto de geometría temporal. Esto se puede hacer usando un `CAST` explícito o usando la notación `::`. Se devuelve un valor nulo si alguno de los valores de puntos de geometría que componen no se puede convertir en un valor `npoint`.

- Convierte entre un punto de red temporal y un punto de geometría temporal

```
{tnpoint,tgeompoint}::{tgeompoint,tnpoint}
tnpoint::tgeogpoint
```

Las conversiones siguen la regla de [?listitem].

```
SELECT asText((tnpoint '[NPoint(1, 0.2)@2001-01-01,
  NPoint(1, 0.3)@2001-01-02)')::tgeompoint);
/* [POINT(23.057077727326 28.7666335767956)@2001-01-01,
   POINT(48.7117553116406 20.9256801894708)@2001-01-02) */
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,
  POINT(48.7117553116406 20.9256801894708)@2001-01-02)'::tnpoint;
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02)
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,
  POINT(48.7117553116406 20.9)@2001-01-02)'::tnpoint;
-- NULL
```

11.6. Entrada y Salida MF-JSON

Un punto de red temporal se escribe y se lee en Moving-Features JSON. Cada instante representa su punto de red como `{"route":<route>,"position":<position>}`, el id de ruta como un número JSON y la posición como un float en `[0, 1]`, coincidiendo con la superficie de accesores SQL (`route`, `getPosition`).

- Genera un punto de red temporal como MF-JSON y lo lee de vuelta; el ciclo de ida y vuelta es sin pérdida

```
asMFJSON(tnpoint,options integer,flags integer,maxdecimaldigits integer) → text
tnpointFromMFJSON(text) → tnpoint
```

La salida toma los argumentos de opciones, banderas y dígitos decimales compartidos con los demás tipos temporales.

```
SELECT tnpointFromMFJSON(asMFJSON(tnpoint 'NPoint(1, 0.5)@2001-01-01'))
  = tnpoint 'NPoint(1, 0.5)@2001-01-01';
-- true
```

11.7. Accessores

- Devuelve los valores

```
getValues(tnpoint) → npointset
```

```
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02)}');
-- {"NPoint(1,0.3)","NPoint(1,0.5)"}
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02)}');
-- {"NPoint(1,0.3)"}
```

- Devuelve los identificadores de ruta

```
routes(tnpoint) → bigintset
```

```
SELECT routes(tnpoint '{NPoint(3, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02}');
-- {1, 3}
```

- Devuelve el valor en una marca de tiempo

```
valueAtTimestamp(tnpoint,timestamptz) → npoint
```

```
SELECT valueAtTimestamp(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03)',
  '2001-01-02');
-- NPoint(1,0.4)
```

- Devuelve la longitud atravesada por el punto de red temporal

```
length(tnpoint) → float
```

```
SELECT length(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]');
-- 54.3757408468784
```

- Devuelve la longitud acumulada atravesada por el punto de red temporal

```
cumulativeLength(tnpoint) → tfloat
```

```
SELECT cumulativeLength(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02,
  NPoint(1, 0.5)@2001-01-03], [NPoint(1, 0.6)@2001-01-04, NPoint(1, 0.7)@2001-01-05]}');
/* {[0@2001-01-01, 54.3757408468784@2001-01-02, 54.3757408468784@2001-01-03],
   [54.3757408468784@2001-01-04, 81.5636112703177@2001-01-05]} */
```

- Devuelve la velocidad del punto de red temporal en unidades por segundo

```
speed({tnpointSeq,tnpointSeqSet}) → tfloatSeqSet
```

```
SELECT speed(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
  NPoint(1, 0.6)@2001-01-03]') * 3600 * 24;
/* Interp=Step;[21.4016800272077@2001-01-01, 14.2677866848051@2001-01-02,
   14.2677866848051@2001-01-03] */
```

11.8. Transformaciones

- Transforma un punto de red temporal en otro subtipo

```
tnpointInst(tnpoint) → tnpointInst
tnpointSeq(tnpoint) → tnpointSeq
tnpointSeqSet(tnpoint) → tnpointSeqSet
```

```
SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01', 'discrete');
-- {NPoint(1,0.5)@2001-01-01}
SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- [NPoint(1,0.5)@2001-01-01]
SELECT tnpointSeqSet(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- {[NPoint(1,0.5)@2001-01-01]}
```

- Transforma un punto de red temporal en otra interpolación

```
setInterp(tnpoint,interp text) → tnpoint
```

```
SELECT setInterp(tnpoint 'NPoint(1,0.2)@2001-01-01','linear');
-- [NPoint(1,0.2)@2001-01-01]
SELECT setInterp(tnpoint '{[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.2)@2001-01-02]}',
  'discrete');
-- {NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02}
```

- Redondea la fracción del punto de red temporal en el número de lugares decimales

```
round(tnpoint,integer=0) → tnpoint
```

```
SELECT round(tnpoint '{[NPoint(1,0.123456789)@2001-01-01, NPoint(1,0.5)@2001-01-02)}', 6);
-- {[NPoint(1,0.123457)@2001-01-01, NPoint(1,0.5)@2001-01-02)}
```

11.9. Modificaciones

- Agrega un instante temporal o una secuencia temporal

```
appendInstant(tnpoint,tnpointInst) → tnpoint
appendSequence(tnpoint,tnpointSeq) → tnpoint
```

```
SELECT asText(appendInstant(tnpoint 'Npoint(1, 0.1)@2001-01-01',
  'Npoint(1, 0.2)@2001-01-02'));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02]
SELECT asText(appendSequence(tnpoint '[Npoint(1, 0.1)@2001-01-01]',
  '[Npoint(1, 0.2)@2001-01-02]'));
-- {[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.2)@2001-01-02]}
```

- Fusiona los puntos de red temporales

```
merge(tnpoint,tnpoint) → tnpoint
merge(tnpoint[]) → tnpoint
merge(tnpoint) → tnpoint
```

```
SELECT asText(merge(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  '[Npoint(1, 0.3)@2001-01-03, Npoint(1, 0.5)@2001-01-05]'));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.5)@2001-01-05]
SELECT asText(merge(ARRAY[tnpoint '[Npoint(1, 0.1)@2001-01-01,
  Npoint(1, 0.2)@2001-01-02]',
  '[Npoint(1, 0.2)@2001-01-02, Npoint(1, 0.3)@2001-01-03]']));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.3)@2001-01-03]
WITH temp(inst) AS (
  SELECT tnpoint 'Npoint(1, 0.1)@2001-01-01' UNION
  SELECT tnpoint 'Npoint(1, 0.2)@2001-01-02' UNION
  SELECT tnpoint 'Npoint(1, 0.3)@2001-01-03' )
SELECT asText(merge(inst)) FROM temp;
-- {NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02, NPoint(1,0.3)@2001-01-03}
```

11.10. Restricciones

- Restringe al (complemento de) un valor o un conjunto de valores

```
atValue(tnpoint,base) → tnpoint
minusValue(tnpoint,base) → tnpoint
atValues(tnpoint,values) → tnpoint
minusValues(tnpoint,values) → tnpoint
```

```
SELECT atValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
-- {[NPoint(2,0.5)@2001-01-02]}
SELECT minusValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
/* {[NPoint(2,0.3)@2001-01-01, NPoint(2,0.5)@2001-01-02),
    (NPoint(2,0.5)@2001-01-02, NPoint(2,0.7)@2001-01-03]} */
```

- Restringe al (complemento de) una geometría

```
atGeometry(tnpoint,geometry) → tnpoint
minusGeometry(tnpoint,geometry) → tnpoint
```

```
SELECT atGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
-- {(NPoint(2,0.342593)@2001-01-01 05:06:40.364673, NPoint(2,0.7)@2001-01-03]}
```

11.11. Operaciones espaciales

- Devuelve el centroide ponderado en el tiempo

```
twCentroid(tnpoint) → geometry(Point)
```

```
SELECT ST_AsText(twCentroid(tnpoint '{[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-02, NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04)}'));
-- POINT(79.9787466444847 46.2385558051041)
```

11.12. Operaciones de cuadro delimitador

- Operadores de rutas, que comparan los identificadores de ruta que visita un punto de red temporal: contenido en, contiene, se superpone e igual

```
{bigint,bigintset,npoint,tnpoint} {?@, @?, @@, @=} {bigint,bigintset,npoint,
  tnpoint} → boolean
```

```
SELECT bigint '1' ?@ tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' @? bigint '1';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01, NPoint(3, 0.5)@2001-01-02}' @@ bigintset '{3,
  7}';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01, NPoint(3, 0.5)@2001-01-02}' @= bigintset '{1,
  3}';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01,
  NPoint(3, 0.5)@2001-01-02}' @= bigintset '{1}';
-- false
```

- Operadores topológicos

```
{tstzspan,stbox,tnpoint} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tnpoint} → boolean
```

```
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' &&
  tstzspan '[2001-01-02,2001-01-03]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' @>
  stbox(npoint 'NPoint(1, 0.5)');
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-03]' ~= tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.35)@2001-01-02, NPoint(1, 0.5)@2001-01-03]';
-- true
```

- Operadores de posición

```
{stbox,tnpoint} {<<, &<, >>, &>} {stbox,tnpoint} → boolean
{stbox,tnpoint} {<<|, &<|, |>>, |&>} {stbox,tnpoint} → boolean
{tstzspan,stbox,tnpoint} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tnpoint} → boolean
```

```
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' <<
  stbox(npoint 'NPoint(1, 0.2)');
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-02]' <<| stbox(npoint 'NPoint(1, 0.5)');
-- false
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' #&>
  tstzspan '[2001-01-01,2001-01-03]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.3)@2001-01-05]' #>>
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02]';
-- true
```

11.13. Operaciones de distancia

Los operandos de dimensiones distintas se leen en las dimensiones que ambos tienen, véase Sección 8.3.

- Devuelve la distancia más cercana

```
{geo,npoint,tnpoint,stbox} |=| {geo,npoint,tnpoint,stbox} → float
```

El operador `|=|` se puede utilizar para realizar búsquedas más cercanas utilizando un índice GiST o SP-GIST (ver Sección 10.2).

```
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]' |=| geometry 'SRID=5676;Linestring(50 50,55 55)';
-- 31.69220882252415
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]' |=| npoint 'NPoint(1, 0.5)';
-- 19.49691305292373
SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.7)@2001-01-02]';
-- 5.231180723735304
```

- Devuelve el instante del primer punto de red temporal en el que los dos argumentos están a la distancia más cercana

```
nearestApproachInstant({geo,npoint,tnpoint},{geo,npoint,tnpoint}) → tnpoint
```

```
SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)');
-- NPoint(2,0.349928)@2001-01-01 02:59:44.402905
SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)');
-- NPoint(2,0.592181)@2001-01-01 17:31:51.080405
```

- Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos

```
shortestLine({geo,npoint,tpoint},{geo,npoint,tpoint}) → geometry
```

La función devuelve la primera línea que encuentre si hay más de una.

```
SELECT ST_AsText(shortestLine(tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(50.7960725266492 48.8266286733015,50 50)
SELECT ST_AsText(shortestLine(tnpoint '[NPoint(2, 0.3)@2001-01-01,
  NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)'));
-- LINESTRING(77.0902838115125 66.6659083092593,90.8134936900394 46.4385792121146)
```

- Devuelve la distancia temporal

```
tnpoint <-> tnpoint → tfloat
```

```
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
  npoint 'NPoint(1, 0.2)';
-- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
  geometry 'Point(50 50)';
-- [25.04966669450440@2001-01-01, 26.40856884262320@2001-01-03]
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
  tnpoint '[NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-04]';
-- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]
```

11.14. Relaciones siempre y alguna vez

- Relaciones alguna vez y siempre

```
eContains(geometry,tnpoint) → boolean
eCovers(geometry,tnpoint) → boolean
eDisjoint({geometry,tnpoint},{geometry,tnpoint}) → boolean
eDwithin({geometry,tnpoint},{geometry,tnpoint},float) → boolean
eIntersects({geometry,tnpoint},{geometry,tnpoint}) → boolean
eTouches(geometry,tnpoint) → boolean
eTouches(tnpoint,geometry) → boolean
aContains(geometry,tnpoint) → boolean
aCovers(geometry,tnpoint) → boolean
aDisjoint({geometry,tnpoint},{geometry,tnpoint}) → boolean
aDwithin({geometry,tnpoint},{geometry,tnpoint},float) → boolean
aIntersects({geometry,tnpoint},{geometry,tnpoint}) → boolean
aTouches(geometry,tnpoint) → boolean
aTouches(tnpoint,geometry) → boolean
```

Las relaciones *alguna vez* y *siempre* determinan si la relación topológica o de distancia se satisface alguna vez o siempre y dan como resultado un `boolean`. Un punto de red temporal las responde en la posición a la que se resuelven su ruta y su fracción, de modo que una secuencia con interpolación lineal se responde a lo largo de su ruta y no solo en sus instantes. El punto de red temporal declara las relaciones que responde un punto en movimiento: `eContains`, `aContains`, `eCovers` y `aCovers` toman la geometría en primer lugar únicamente, y `eTouches` y `aTouches` no tienen dirección entre dos puntos de red temporales, ya que un punto en movimiento no contiene ni cubre una geometría y dos puntos en movimiento no se tocan.

```
SELECT eContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- false
SELECT eCovers(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint 'Npoint(1, 0.1)@2001-01-01');
-- false
SELECT eDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- true
SELECT eIntersects(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  tnpoint '[Npoint(2, 0.0)@2001-01-01, Npoint(2, 1)@2001-01-03]');
-- false
SELECT eTouches(tnpoint 'Npoint(1, 0.1)@2001-01-01', geometry 'SRID=5676;
  Polygon((0 0,0 50,50 50,50 0,0 0))');
-- false
SELECT aDwithin(tnpoint '[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-03]',
  tnpoint '[Npoint(1, 0.5)@2001-01-01, Npoint(1, 0.3)@2001-01-03]', 1);
-- false
```

11.15. Relaciones espaciotemporales

- Relaciones espaciotemporales

```
tContains(geometry,tnpoint) → tbool
tCovers(geometry,tnpoint) → tbool
tDisjoint({geometry,tnpoint},{geometry,tnpoint}) → tbool
tDwithin({geometry,tnpoint},{geometry,tnpoint},float) → tbool
tIntersects({geometry,tnpoint},{geometry,tnpoint}) → tbool
tTouches(geometry,tnpoint) → tbool
tTouches(tnpoint,geometry) → tbool
```

Las relaciones *temporales* calculan la relación topológica o de distancia en cada instante y dan como resultado un `tbool`. Un punto de red temporal las responde en la posición a la que se resuelven su ruta y su fracción. El punto de red temporal declara las relaciones que responde un punto en movimiento: `tContains` y `tCovers` toman la geometría en primer lugar únicamente, y `tTouches` no tiene dirección entre dos puntos de red temporales, ya que un punto en movimiento no contiene ni cubre una geometría y dos puntos en movimiento no se tocan.

```
SELECT tContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- {[f@2001-01-01, f@2001-01-03]}
SELECT tCovers(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint 'Npoint(1, 0.1)@2001-01-01');
-- f@2001-01-01
SELECT tDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- {[t@2001-01-01, t@2001-01-03]}
SELECT tTouches(tnpoint 'Npoint(1, 0.1)@2001-01-01', geometry 'SRID=5676;
  Polygon((0 0,0 50,50 50,50 0,0 0))');
-- f@2001-01-01
SELECT tIntersects(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  tnpoint '[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.1)@2001-01-03]');
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03]}
SELECT tDwithin(tnpoint '[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-03]',
  tnpoint '[Npoint(1, 0.5)@2001-01-01, Npoint(1, 0.3)@2001-01-03]', 1);
/* {[f@2001-01-01, t@2001-01-01 22:19:04.400634,
   t@2001-01-02 01:40:55.599365], (f@2001-01-02 01:40:55.599365, f@2001-01-03]} */
```

11.16. Comparaciones

- Comparaciones tradicionales

```
tnpoint {=, <>, <, >, <=, >=} tnpoint → boolean
```

```
SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-02),
  [NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-03]}' =
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- true
SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]}' <>
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- false
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.6)@2001-01-03]';
-- true
```

- Comparaciones alguna vez y siempre

```
{npoint,tnpoint} {?=, ?<>, %=, %<>} {npoint,tnpoint} → boolean
```

```
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-04)' ?= Npoint(1, 0.3);
-- true
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.2)@2001-01-04)' &= Npoint(1, 0.2);
-- true
```

- Comparaciones temporales

```
{npoint,tnpoint} {#=, #<>} {npoint,tnpoint} → tbool
```

```
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01,
  NPoint(1, 0.4)@2001-01-03)' #= npoint 'NPoint(1, 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.8)@2001-01-03)' #<>
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.7)@2001-01-03)';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```

11.17. Agregacions

Las tres funciones agregadas para puntos de red temporales se ilustran a continuación.

- Conteo temporal

```
tCount(tnpoint) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03)' UNION
  SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04)' UNION
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05)' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05)}
```

- Conteo de ventana

```
wCount(tnpoint) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03)' UNION
  SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04)' UNION
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05)' )
SELECT wCount(Temp, '1 day') FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
   1@2001-01-06)} */
```

- Centroide temporal

```
tCentroid(tnpoint) → tgeompoint
```

```
WITH Temp(temp) AS (
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03)' UNION
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-03)' UNION
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-03)' ) SELECT astext(tCentroid(Temp)) FROM Temp;
/* {[POINT(72.451531682218 76.5231414472853)@2001-01-01,
   POINT(55.7001249027598 72.9552602410653)@2001-01-03)} */
```

11.18. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tabla de puntos de redes temporales. A continuación, se muestra un ejemplo de creación de índice:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador para los puntos de la red temporal, que es un `stbox` y, por lo tanto, almacena las coordenadas absolutas del espacio subyacente.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en puntos de la red temporal,
- `<<#`, `&<#`, `#&>`, `#>>`, que solo consideran la dimensión temporal en puntos de la red temporal,
- `&&`, `@>`, `<@`, `~=`, `-|-` y `|=|`, que consideran tantas dimensiones como compartan la columna indexada y el argumento de consulta.

Estos operadores trabajan con cuadros delimitadores, no con los valores completos.

11.19. Agregación

- Devuelve el cuadro delimitador espaciotemporal que cubre cada posición materializada en un conjunto de puntos de red

```
extent(tnpoint) → stbox
```

El agregado es polimórfico con los agregados `extent` de los demás tipos temporales espaciales, por lo que una sola consulta puede calcular un único cuadro a través de un conjunto heterogéneo de columnas `tgeompoint`, `tgeometry` y `tnpoint`. La extensión espacial de cada instante se resuelve a través del registro de ways, lo que domina el coste sobre una entrada grande.

```
SELECT extent(temp) FROM tbl_tnpoint;
```

Capítulo 12

Búferes circulares temporales

Un tipo de búfer circular estático `cbuffer` representa un punto 2D y un radio mayor o igual que 0 alrededor del punto. El radio representa el «impacto» del punto sobre su entorno. Este impacto puede interpretarse como ruido, contaminación o aspectos similares según los requisitos de la aplicación.

El tipo `cbuffer` sirve como tipo base para definir el tipo de búfer circular temporal `tcbuffer`. El tipo `tcbuffer` tiene una funcionalidad similar al tipo de punto temporal `tgeompoint` con la excepción de que solo considera dos dimensiones. Por lo tanto, la mayoría de las funciones y operadores descritos anteriormente para el tipo `tgeompoint` también son aplicables para el tipo `tcbuffer`. Además, hay funciones específicas definidas para el tipo `tcbuffer`.

La Figura 12.1 ilustra el uso del tipo `tcbuffer` para modelar el búfer circular alrededor de la franja de viento de las tormentas tropicales en 2024. Los datos se obtienen de la National Oceanic and Atmospherique Administration (NOAA). Como se ilustra en la figura, los tipos `tgeompoint` y `tcbuffer` permiten la interpolación *lineal*, mientras que, como hemos visto en la Figura 7.2, el tipo `tgeometry` solo permite la interpolación *escalonada*.

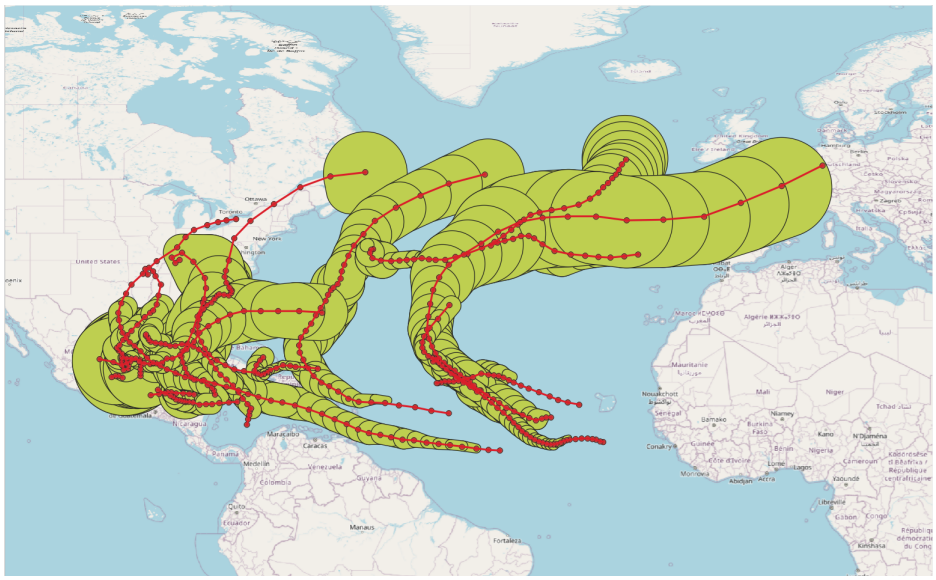


Figura 12.1: Ilustración del uso del tipo `tcbuffer` para modelar el búfer circular alrededor de la franja de viento de las tormentas tropicales en 2024.

12.1. Búferes circulares estáticos

Un valor `cbuffer` es un par de la forma `(point,radius)` donde `point` es un punto geométrico y `radius` es un valor `float` mayor o igual que cero que representa un radio alrededor del punto expresado utilizando las unidades del SRID del punto. Ejemplos de entrada de valores de búfer circular son los siguientes:

```
SELECT cbuffer 'Cbuffer(Point(1 1), 0.3)';
SELECT cbuffer 'Cbuffer(Point(1 1), 10.0)';
```

El tipo `cbuffer` corresponde al resultado de la función de PostGIS ST_MinimumBoundingRadius.

Los valores del tipo de búfer circular deben satisfacer varias restricciones para que estén bien definidos. Ejemplos de valores incorrectos del tipo de búfer circular son los siguientes.

```
-- Incorrect point value
SELECT cbuffer 'Cbuffer(Linestring(1 1,2 2), 1.0)';
-- Incorrect 3D point
SELECT cbuffer 'Cbuffer(Point Z(1 1 1), 1.0)';
-- Incorrect radius value
SELECT cbuffer 'Cbuffer(Point(1 1), -2.0)';
```

A continuación damos las funciones y operadores para el tipo de búfer circular.

12.1.1. Entrada y salida

- Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)

```
asText({cbuffer,cbuffer[]}) → {text,text[]}
asEWKT({cbuffer,cbuffer[]}) → {text,text[]}
```

```
SELECT asText(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- Cbuffer(POINT(1 1),0.5)
SELECT asText(ARRAY[cbuffer 'Cbuffer(Point(1 1),0.2)', 'Cbuffer(Point(1 1),0.5)']);
-- {"Cbuffer(POINT(1 1),0.2)","Cbuffer(POINT(1 1),0.5)"}
SELECT asEWKT(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(ARRAY[cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.2)',
  'Cbuffer(SRID=5676;Point(1 1),0.5)']);
-- {"SRID=5676;Cbuffer(POINT(1 1),0.2)","SRID=5676;Cbuffer(POINT(1 1),0.5)"}
```

- Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
asBinary(cbuffer,endian text='') → bytea
asEWKB(cbuffer,endian text='') → bytea
asHexWKB(cbuffer,endian text='') → text
asHexEWKB(cbuffer,endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina. Un búfer circular toma su SRID de su punto subyacente: `asEWKB` y `asHexEWKB` lo incluyen, mientras que las formas simples `asBinary` y `asHexWKB` lo omiten.

```
SELECT asBinary(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- \x0100000000000000f03f000000000000f03f000000000000e03f
SELECT asEWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- \x01402c160000000000000000f03f000000000000f03f000000000000e03f
SELECT asHexWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- 0100000000000000F03F000000000000F03F000000000000E03F
SELECT asHexEWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
-- 01402C160000000000000000F03F000000000000F03F000000000000E03F
```

- Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)

```
cbufferFromText(text) → cbuffer
cbufferFromEWKT(text) → cbuffer
```

```
SELECT asText(cbufferFromText(text 'Cbuffer(Point(1 1),0.5)'));
-- Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromEWKT(text 'SRID=5676;Cbuffer(Point(1 1),0.5)'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
```

- Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

```
cbufferFromBinary(bytea) → cbuffer
cbufferFromEWKB(bytea) → cbuffer
cbufferFromHexWKB(text) → cbuffer
cbufferFromHexEWKB(text) → cbuffer
```

```
SELECT asEWKT(cbufferFromBinary(
  '\x0100000000000000f03f000000000000f03f000000000000e03f'));
-- Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromEWKB(
  '\x01402c16000000000000000f03f000000000000f03f000000000000e03f'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromHexEWKB(
  '01402C16000000000000000F03F000000000000F03F000000000000E03F'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
```

12.1.2. Constructores

- Constructor para búferes circulares

```
cbuffer(geompoint,float) → cbuffer
```

```
SELECT asText(cbuffer(ST_Point(1,1), 0.3));
-- Cbuffer(POINT(1 1),0.3)
```

12.1.3. Conversión de tipos

Los valores del tipo `cbuffer` se pueden convertir al tipo `geometry` utilizando un `CAST` explícito o utilizando la notación `::` como se muestra a continuación. De manera similar, una `geometry` se puede convertir a un valor `cbuffer` utilizando la función `ST_MinimumBoundingRadius` de PostGIS.

- Convierte entre un búfer circular y una geometría

```
{geometry,cbuffer}::{cbuffer,geometry}
cbuffer::geography
```

Como geografía, que no tiene curvas, el círculo se lee como un polígono de 32 segmentos por cuadrante, y no es un círculo sobre la superficie de la Tierra. Las conversiones siguen la regla de [?listitem].

```
SELECT ST_AsText(cbuffer(ST_Point(1,1), 1)::geometry);
-- CURVEPOLYGON(CIRCULARSTRING(0 1,2 1,0 1))
SELECT asText(geometry 'SRID=5676;CIRCULARSTRING(0 1,2 1,0 1)'::cbuffer);
-- Cbuffer(POINT(1 1),1)
SELECT asText(geometry 'SRID=5676;LINESTRING(0 0,2 2)'::cbuffer);
-- Cbuffer(POINT(1 1),1.414213562373095)
```

- Convierte un búfer circular y, opcionalmente, una marca de tiempo o un período, a un cuadro espaciotemporal

```
stbox(cbuffer) → stbox
stbox(cbuffer,{timestamptz,tstzspan}) → stbox
```

```
SELECT stbox(cbuffer 'SRID=5676;Cbuffer(Point(1 1),0.3)');
-- SRID=5676;STBOX X((0.7,0.7),(1.3,1.3))
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', timestamptz '2001-01-01');
-- STBOX XT(((0.7,0.7),(1.3,1.3)),[2001-01-01, 2001-01-01])
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT(((0.7,0.7),(1.3,1.3)),[2001-01-01, 2001-01-02])
```

12.1.4. Accesores

- Devuelve el punto

```
point(cbuffer) → geompoint
```

```
SELECT ST_AsText(point(cbuffer 'Cbuffer(Point(1 1), 0.3)'));
-- POINT(1 1)
```

- Devuelve el radio

```
radius(cbuffer) → float
```

```
SELECT radius(cbuffer 'Cbuffer(Point(1 1), 0.3)');
-- 0.3
```

12.1.5. Transformaciones

- Redondea el punto y el radio del búfer circular en el número de lugares decimales

```
round({cbuffer,cbuffer[]},integer=0) → {cbuffer,cbuffer[]}
```

```
SELECT asText(round(cbuffer(ST_Point(1.123456789,1.123456789), 0.123456789), 6));
-- Cbuffer(POINT(1.123457 1.123457),0.123457)
SELECT asText(round(ARRAY[cbuffer(ST_Point(1.123456789,1.123456789), 0.123456789),
  cbuffer(ST_Point(2.987654321,2.987654321), 0.987654321)], 6));
-- {"Cbuffer(POINT(1.123457 1.123457),0.123457)","Cbuffer(POINT(2.987654 2.987654) ↩
   ,0.987654)"}
```

12.1.6. Operaciones espaciales

- Devuelve o establece el identificador de referencia espacial

```
SRID(cbuffer) → integer
setSRID(cbuffer,integer) → cbuffer
```

```
SELECT SRID(cbuffer 'Cbuffer(SRID=5676;Point(1 1), 0.3)');
-- 5676
SELECT asEWKT(setSRID(cbuffer 'Cbuffer(Point(0 0),1)', 4326));
-- SRID=4326;Cbuffer(POINT(0 0),1)
```

- Transforma a una referencia espacial diferente

```
transform(cbuffer,integer) → cbuffer
transformPipeline(cbuffer,pipeline text,srid integer,
  is_forward boolean=true) → cbuffer
```

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el búfer circular de entrada tiene un SRID desconocido (representado por 0).

La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato de cadena:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

El SRID del búfer circular de entrada se ignora y el SRID del búfer circular de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(cbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)', 3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)
```

```
WITH test(cbuffer, pipeline) AS (
  SELECT cbuffer 'Cbuffer(SRID=4326;Point(4.3525 50.846667),1)',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(cbuffer, pipeline, 4326), pipeline,
  4326, false), 6) FROM test;
-- SRID=4326;Cbuffer(POINT(4.3525 50.846667),1)
```

12.1.7. Operaciones de distancia

- Devuelve la distancia

```
distance({geo,cbuffer},cbuffer) → float
distance(cbuffer,{geo,cbuffer}) → float
{geo,cbuffer} <-> cbuffer → float
```

La distancia entre dos búferes circulares es la distancia entre sus puntos reducida por ambos radios, y es cero cuando los búferes se superponen. El resultado es `NULL` cuando la geometría está vacía.

```
SELECT round(distance(geometry 'Point(1 0)', cbuffer 'Cbuffer(Point(4 0),0.5)'), 6);
-- 2.5
SELECT round(cbuffer 'Cbuffer(Point(0 0),0.5)' <-> cbuffer 'Cbuffer(Point(3 4),0.5)', 6);
-- 4
SELECT round(cbuffer 'Cbuffer(Point(0 0),3)' <-> cbuffer 'Cbuffer(Point(3 4),3)', 6);
-- 0
```

- Devuelve la distancia de aproximación más cercana

```
nearestApproachDistance({stbox,cbuffer},{cbuffer,stbox}) → float
{stbox,cbuffer} |=| {cbuffer,stbox} → float
```

El valor es la menor distancia sobre el tiempo que comparten los dos argumentos, y es `NULL` cuando no comparten ninguno. Un búfer circular no lleva período, por lo que está presente en todo tiempo que abarca la caja.

```
SELECT round(nearestApproachDistance(stbox 'STBOX X((1,-1),(3,1))', cbuffer 'Cbuffer(Point ↩
    (4 0),0.5)'), 6);
-- 0.5
SELECT round(cbuffer 'Cbuffer(Point(4 0),0.5)' |=| stbox 'STBOX X((1,-1),(3,1))', 6);
-- 0.5
```

12.1.8. Comparaciones

Los operadores de comparación (=, < y así sucesivamente) están disponibles para búferes circulares. Comparan primero los puntos y luego el radio de los argumentos. Excepto la igualdad y la desigualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan en búferes circulares.

- Comparaciones tradicionales

```
cbuffer {=, <>, <, >, <=, >=} cbuffer
```

```
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' = cbuffer 'Cbuffer(Point(3 3), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' <> cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' < cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.6)' > cbuffer 'Cbuffer(Point(2 2), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.5)' <= cbuffer 'Cbuffer(Point(2 2), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.6)' >= cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- true
```

12.2. Búferes circulares temporales

El tipo de búfer circular temporal `tcbuffer` permite representar el movimiento de objetos junto con un radio circular alrededor de ellos. Como todos los tipos temporales, se presenta en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores de `tcbuffer` en estos subtipos.

```
SELECT tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01';
SELECT tcbuffer '{Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03}';
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
SELECT tcbuffer '{[Cbuffer(Point(1 1), 1)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-02],
  [Cbuffer(Point(2 2), 2)@2001-01-04, Cbuffer(Point(2 2), 3)@2001-01-05]}';
```

El tipo de búfer circular temporal acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL) para especificar el subtipo y/o el identificador de referencia espacial (SRID). Los valores posibles para el subtipo son `Instant`, `Sequence` y `SequenceSet`. Los dos argumentos son opcionales y si alguno de ellos no se especifica para una columna, se permiten valores de cualquier subtipo y/o SRID.

```
SELECT asEWKT(tcbuffer(Sequence,5676) 'SRID=5676;
  [Cbuffer(Point(1 1), 0.2)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-03]
SELECT tcbuffer(Sequence) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal subtype (Instant) does not match column type (Sequence)
SELECT tcbuffer(5676) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR:  Temporal circular buffer SRID (0) does not match column SRID (5676)
```

Los valores de búfer circular temporal del subtipo de secuencia o conjunto de secuencias se convierten en una forma normal para que los valores equivalentes tengan representaciones idénticas. Para ello, los valores instantáneos consecutivos se fusionan cuando es posible. Tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Los ejemplos de transformación a una forma normal son los siguientes.

```
SELECT asText(tcbuffer '[Cbuffer(Point(1 1), 0.25)@2001-01-01,
  Cbuffer(Point(2 2), 0.5)@2001-01-02, Cbuffer(Point(3 3), 0.75)@2001-01-03)');
-- [Cbuffer(POINT(1 1),0.25)@2001-01-01, Cbuffer(POINT(3 3),0.75)@2001-01-03)
```

```
SELECT asText(tcbuffer '{[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(2 2), 0.3)@2001-01-02, Cbuffer(Point(2 2), 0.5)@2001-01-03),
  [Cbuffer(Point(2 2), 0.5)@2001-01-03, Cbuffer(Point(2 2), 0.7)@2001-01-04)}');
/* {[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(2 2),0.3)@2001-01-02,
   Cbuffer(POINT(2 2),0.7)@2001-01-04)} */
```

12.3. Validez de los búferes circulares temporales

Los valores de búfer circular temporal deben satisfacer las restricciones especificadas en la Sección 4.3 para que estén bien definidos. Se genera un error cuando no se cumple una de estas restricciones. Ejemplos de valores incorrectos son los siguientes.

```
-- Null values are not allowed
SELECT tcbuffer 'NULL@2001-01-01 08:05:00';
SELECT tcbuffer 'Point(0 0)@NULL';
-- Base type is not a circular buffer
SELECT tcbuffer 'Point(0 0)@2001-01-01 08:05:00';
```

A continuación damos las funciones y operadores para el búfer circular temporal. La mayoría de las funciones y operadores para tipos temporales descritos en los capítulos precedentes se pueden aplicar para los tipos de búfer circular temporal. Por lo tanto, en las firmas de las funciones, la notación `base` representa un `cbuffer` y las notaciones `ttype`, `tpoint` y `tgeompoint` también representan un `tcbuffer`. Además, las funciones que tienen un argumento de tipo `geometry` aceptan además un argumento de tipo `cbuffer`. Para evitar la redundancia, a continuación solo presentamos algunos ejemplos de estas funciones y operadores para búferes circulares temporales.

12.4. Entrada y salida

- Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)

```
asText({tcbuffer,tcbuffer[],cbuffer[]}) → {text,text[]}
asEWKT({tcbuffer,tcbuffer[],cbuffer[]}) → {text,text[]}
```

```
SELECT asText(tcbuffer 'SRID=4326;
  [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)');
-- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)
SELECT asText(ARRAY[cbuffer 'Cbuffer(Point(0 0),1)', 'Cbuffer(Point(1 1),2)']);
-- {"Cbuffer(POINT(0 0),1)","Cbuffer(POINT(1 1),2)"}
SELECT asEWKT(tcbuffer 'SRID=4326;
  [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)');
-- SRID=4326;[Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)
SELECT asEWKT(ARRAY[cbuffer 'Cbuffer(SRID=5676;Point(0 0),1)',
  'Cbuffer(SRID=5676;Point(1 1),2)']);
-- {"SRID=5676;Cbuffer(POINT(0 0),1)","SRID=5676;Cbuffer(POINT(1 1),2)"}
```

- Devuelve la representación JSON de características móviles (MF-JSON)

```
asMFJSON(tcbuffer) → text
```

Un búfer circular siempre es planar 2D. Cada valor es un objeto con un miembro `point`, un arreglo de coordenadas `[x, y]` y un miembro numérico `radius`, que reflejan los accesores `point` y `radius`.

```
SELECT asMFJSON(tcbuffer 'Cbuffer(Point(1 2),0.5)@2001-01-01', 3);
/* {"type":"MovingCircularBuffer","bbox":[[0.5,1.5],[1.5,2.5]],
   "period":{"begin":"2001-01-01T00:00:00","end":"2001-01-01T00:00:00",
   "lower_inc":true,"upper_inc":true}, "values":[{"point":[1,2],"radius":0.5}],
   "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
SELECT asMFJSON(tcbuffer 'SRID=4326;
```

```
  [Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.4)@2001-01-02)', 3);
/* {"type":"MovingCircularBuffer","crs":{"type":"Name",
   "properties":{"name":"EPSG:4326"}},"bbox":[[0.8,0.8],[2.4,2.4]],
   "period":{"begin":"2001-01-01T00:00:00","end":"2001-01-02T00:00:00",
   "lower_inc":true,"upper_inc":false},
   "values":[{"point":[1,1],"radius":0.2},{"point":[2,2],"radius":0.4}],
   "datetimes":["2001-01-01T00:00:00","2001-01-02T00:00:00"],
   "lower_inc":true,"upper_inc":false,"interpolation":"Linear"} */
```

- Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
asBinary(tcbuffer,endian text='') → bytea
asEWKB(tcbuffer,endian text='') → bytea
asHexWKB(tcbuffer,endian text='') → text
asHexEWKB(tcbuffer,endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tcbuffer 'Cbuffer(Point(1 2),1)@2001-01-01');
-- \x013b0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
SELECT asEWKB(tcbuffer 'SRID=7844;Cbuffer(Point(1 2),1)@2001-01-01');
-- \x013b0041a41e0000000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
SELECT asHexWKB(tcbuffer 'Cbuffer(Point(1 2),1)@2001-01-01');
-- 013B0001000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
SELECT asHexEWKB(tcbuffer 'SRID=3812;Cbuffer(Point(1 2),1)@2001-01-01');
-- 013B0041E40E0000000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
```

- Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)

```
tcbufferFromText(text) → tcbuffer
tcbufferFromEWKT(text) → tcbuffer
```

```
SELECT asEWKT(tcbufferFromText(text '[Cbuffer(Point(1 2),1)@2001-01-01,
  Cbuffer(Point(3 4),2)@2001-01-02]'));
-- [Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
SELECT asEWKT(tcbufferFromEWKT(text 'SRID=3812;
  [Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02]'));
-- SRID=3812;[Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
```

- Entrada desde la representación JSON de características móviles (MF-JSON)

```
tcbufferFromMFJSON(text) → tcbuffer
```

```
SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer 'SRID=3812;
  Cbuffer(Point(1 2),0.5)@2001-01-01', 3)));
-- SRID=3812;Cbuffer(POINT(1 2),0.5)@2001-01-01
SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer 'SRID=5676;
  [Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02]', 3)));
-- SRID=5676;[Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
```

- Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

```
tcbufferFromBinary(bytea) → tcbuffer
tcbufferFromEWKB(bytea) → tcbuffer
tcbufferFromHexWKB(text) → tcbuffer
tcbufferFromHexEWKB(text) → tcbuffer
```

```
SELECT asEWKT(tcbufferFromBinary(
  '\x013b0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- Cbuffer(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tcbufferFromEWKB(
  '\x013b0041a41e0000000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- SRID=7844;Cbuffer(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tcbufferFromHexEWKB(
  '013B0041E40E000000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000'));
-- SRID=3812;Cbuffer(POINT(1 2),1)@2001-01-01
```

12.5. Constructores

- Constructor para búferes circulares temporales con valor constante

```
tcbuffer(cbuffer,timestamptz) → tcbufferInst
tcbuffer(cbuffer,tstzset) → tcbufferDiscSeq
tcbuffer(cbuffer,tstzspan,interp text='linear') → tcbufferContSeq
tcbuffer(cbuffer,tstzspanset,interp text='linear') → tcbufferSeqSet
```

```
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)', timestamptz '2001-01-01'));
-- Cbuffer(POINT(1 1),0.5)@2001-01-01
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.3)',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}'));
/* {Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(1 1),0.3)@2001-01-03,
   Cbuffer(POINT(1 1),0.3)@2001-01-05} */
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)', tstzspan '[2001-01-01, 2001-01-02]'));
-- [Cbuffer(POINT(1 1),0.5)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-02]
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.2)',
  tstzspanset '{[2001-01-01, 2001-01-03]}', 'step'));
-- Interp=Step;{[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.2)@2001-01-03]}
```

- Constructor para búferes circulares temporales de subtipo secuencia

```
tcbufferSeq(tcbufferInst[],interp text='linear',lower_inc boolean=true,
  upper_inc boolean=true) → tcbufferSeq
```

```
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
  'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03']));
/* [Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02,
   Cbuffer(POINT(1 1),0.5)@2001-01-03] */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
  'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03'],
  'discrete'));
/* {Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02,
   Cbuffer(POINT(1 1),0.5)@2001-01-03} */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.2)@2001-01-01',
  'Cbuffer(Point(1 1), 0.4)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03'], 'step'));
/* Interp=Step;[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.4)@2001-01-02,
   Cbuffer(POINT(1 1),0.5)@2001-01-03] */
```

- Constructor para búferes circulares temporales de subtipo conjunto de secuencias

```
tcbufferSeqSet(tcbuffer[]) → tcbufferSeqSet
tcbufferSeqSetGaps(tcbufferInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → tcbufferSeqSet
```

```
SELECT asText(tcbufferSeqSet(ARRAY[tcbuffer '[Cbuffer(Point(1 1),0.2)@2001-01-01,
  Cbuffer(Point(2 2),0.4)@2001-01-02]',
  '[Cbuffer(Point(2 2),0.6)@2001-01-03, Cbuffer(Point(2 2),0.8)@2001-01-04]']));
/* {[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(2 2),0.4)@2001-01-02],
   [Cbuffer(POINT(2 2),0.6)@2001-01-03, Cbuffer(POINT(2 2),0.8)@2001-01-04]} */
SELECT asText(tcbufferSeqSetGaps(ARRAY[tcbuffer 'Cbuffer(Point(1 1),0.1)@2001-01-01',
  'Cbuffer(Point(1 1),0.3)@2001-01-03', 'Cbuffer(Point(1 1),0.5)@2001-01-05'], '1 day'));
/* {[Cbuffer(POINT(1 1),0.1)@2001-01-01], [Cbuffer(POINT(1 1),0.3)@2001-01-03],
   [Cbuffer(POINT(1 1),0.5)@2001-01-05]} */
```

- Construye un búfer circular temporal a partir de un punto de geometría temporal y un flotante temporal

```
tcbuffer(tgeompoint,tfloat) → tcbuffer
```

El marco de tiempo del resultado es la intersección de los marcos de tiempo del punto temporal y el flotante temporal. Si no se intersecan, se devuelve un valor nulo

```
SELECT asText(tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
  tfloat '[1@2001-01-01, 2@2001-01-02)'));
-- [Cbuffer(POINT(1 1),1)@2001-01-01, Cbuffer(POINT(2 2),2)@2001-01-02)
SELECT asText(tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
  tfloat '[2@2001-01-03, 3@2001-01-04)'));
-- NULL
```

12.6. Conversión de tipos

Un valor de búfer circular temporal se puede convertir en y desde un punto de geometría temporal. Esto se puede hacer usando un `CAST` explícito o usando la notación `::`. Se devuelve un valor nulo si alguno de los valores de puntos de geometría que componen no se puede convertir en un valor `cbuffer`.

- Convierte un búfer circular temporal a un punto de geometría temporal

```
tcbuffer::{tgeompoint,tgeogpoint}
```

Las conversiones siguen la regla de [?listitem].

```
SELECT asText((tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02)'::tgeompoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
```

- Convierte un búfer circular temporal a un flotante temporal

```
tcbuffer::tfloat
```

```
SELECT (tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02)'::tfloat;
-- [0.2@2001-01-01, 0.3@2001-01-02)
```

12.7. Accesores

- Devuelve los valores

```
getValues(tcbuffer) → cbufferset
```

```
SELECT asText(getValues(tcbuffer '{[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02)}'));
-- {"Cbuffer(POINT(1 1),0.3)", "Cbuffer(POINT(1 1),0.5)"}
SELECT asEWKT(getValues(tcbuffer 'SRID=5676;
  {[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-02)}'));
-- SRID=5676;{"Cbuffer(POINT(1 1),0.3)"}
```

- Devuelve los puntos o los radios

```
points(tcbuffer) → geomset
radius(tcbuffer) → floatset
```

```
SELECT asEWKT(points(tcbuffer 'SRID=5676;
  {Cbuffer(Point(3 3), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02}'));
-- SRID=5676;{"POINT(1 1)", "POINT(3 3)"}
SELECT radius(tcbuffer 'SRID=5676;
  {Cbuffer(Point(3 3), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02}');
-- {0.3, 0.5}
```

- Devuelve el valor en una marca de tiempo

```
valueAtTimestamp(tcbuffer,timestamptz) → cbuffer
```

```
SELECT asText(valueAtTimestamp(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03)', '2001-01-02'));
-- Cbuffer(POINT(2 2),0.4)
```

12.8. Transformaciones

- Transforma un búfer circular temporal en otro subtipo

```
tcbufferInst(tcbuffer) → tcbufferInst
tcbufferSeq(tcbuffer) → tcbufferSeq
tcbufferSeqSet(tcbuffer) → tcbufferSeqSet
```

```
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Cbuffer(POINT(1 1),0.5)@2001-01-01}
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- [Cbuffer(POINT(1 1),0.5)@2001-01-01]
SELECT asText(tcbufferSeqSet(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- {[Cbuffer(POINT(1 1),0.5)@2001-01-01]}
```

- Transforma un búfer circular temporal en otra interpolación

```
setInterp(tcbuffer,interp text) → tcbuffer
```

```
SELECT asText(setInterp(tcbuffer 'Cbuffer(Point(1 1),0.2)@2001-01-01','linear'));
-- [Cbuffer(POINT(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tcbuffer '{[Cbuffer(Point(1 1),0.1)@2001-01-01],
  [Cbuffer(Point(1 1),0.2)@2001-01-02]}', 'discrete'));
-- {Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.2)@2001-01-02}
```

- Redondea los puntos y los radios del búfer circular temporal en el número de lugares decimales

```
round(tcbuffer,integer) → tcbuffer
```

```
SELECT asText(round(tcbuffer '{[Cbuffer(Point(1 1.123456789),0.123456789)@2001-01-01,
  Cbuffer(Point(1 1),0.5)@2001-01-02)}', 3));
-- {[Cbuffer(POINT(1 1.123),0.123)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-02)}
```

- Extrae las subsecuencias en las que el búfer circular temporal permanece dentro de un área de un tamaño máximo dado durante al menos la duración dada

```
stops(tcbuffer,maxdist float=0.0,minduration interval='0 minutes') → tcbuffer
```

El tamaño del área es la mayor distancia entre dos centros de la subsecuencia, y una parada que comienza o termina en el primer o el último instante de una secuencia incluye ese instante solo si la secuencia lo incluye, como se indica para un punto temporal en stops.

```
SELECT asText(stops(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02, Cbuffer(Point(2 2), 0.5)@2001-01-03,
  Cbuffer(Point(2 2), 0.5)@2001-01-04]'));
/* {[Cbuffer(POINT(1 1),0.5)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-02],
   [Cbuffer(POINT(2 2),0.5)@2001-01-03, Cbuffer(POINT(2 2),0.5)@2001-01-04]} */
```

- Devuelve un valor temporal trasladado por los desplazamientos dados

```
translate(tcbuffer,deltax float,deltay float) → tcbuffer
```

El movimiento desplaza el centro de cada buffer circular y conserva su radio. Un buffer circular es plano, por lo que no admite desplazamiento vertical ni rotación alrededor de los ejes x o y.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(translate(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
  Cbuffer(Point(3 2),1)@2001-01-02]', 1, 2), 6);
-- [Cbuffer(POINT(2 3),0.5)@2001-01-01, Cbuffer(POINT(4 4),1)@2001-01-02]
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del origen o de un punto

```
rotate(tcbuffer,angle float) → tcbuffer
rotate(tcbuffer,angle float,x0 float,y0 float) → tcbuffer
```

El movimiento desplaza el centro de cada buffer circular y conserva su radio. Un buffer circular es plano, por lo que no admite desplazamiento vertical ni rotación alrededor de los ejes x o y.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotate(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
  Cbuffer(Point(3 2),1)@2001-01-02]', 0.7, 1, 1), 6);
/* [Cbuffer(POINT(1 1),0.5)@2001-01-01,
   Cbuffer(POINT(1.885467 3.053278),1)@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del eje z

```
rotateZ(tcbuffer,angle float) → tcbuffer
```

El movimiento desplaza el centro de cada buffer circular y conserva su radio. Un buffer circular es plano, por lo que no admite desplazamiento vertical ni rotación alrededor de los ejes x o y.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotateZ(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
  Cbuffer(Point(3 2),1)@2001-01-02]', 0.7), 6);
/* [Cbuffer(POINT(0.120625 1.40906),0.5)@2001-01-01,
   Cbuffer(POINT(1.006091 3.462337),1)@2001-01-02] */
```

12.9. Modificaciones

- Agrega un instante temporal o una secuencia temporal

```
appendInstant(tcbuffer,tcbufferInst) → tcbuffer
appendSequence(tcbuffer,tcbufferSeq) → tcbuffer
```

```
SELECT asText(appendInstant(tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01',
  tcbuffer 'Cbuffer(Point(2 2), 0.2)@2001-01-02'));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(2 2),0.2)@2001-01-02]
SELECT asText(appendSequence(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01]',
  '[Cbuffer(Point(2 2), 0.2)@2001-01-02]'));
-- {[Cbuffer(POINT(1 1),0.1)@2001-01-01], [Cbuffer(POINT(2 2),0.2)@2001-01-02]}
```

- Fusiona los búferes circulares temporales

```
merge(tcbuffer,tcbuffer) → tcbuffer
merge(tcbuffer[]) → tcbuffer
merge(tcbuffer) → tcbuffer
```

```
SELECT asText(merge(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03]',
  '[Cbuffer(Point(1 1), 0.3)@2001-01-03, Cbuffer(Point(1 1), 0.5)@2001-01-05]'));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-05]
SELECT asText(merge(ARRAY[tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-02]',
  '[Cbuffer(Point(1 1), 0.2)@2001-01-02, Cbuffer(Point(1 1), 0.3)@2001-01-03]']));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.3)@2001-01-03]
WITH temp(inst) AS (
  SELECT tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01' UNION
  SELECT tcbuffer 'Cbuffer(Point(2 2), 0.2)@2001-01-02' )
SELECT asText(merge(inst)) FROM temp;
-- {Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(2 2),0.2)@2001-01-02}
```

12.10. Restricciones

- Restringe al (complemento de) un valor o un conjunto de valores

```
atValue(tcbuffer,base) → tcbuffer
minusValue(tcbuffer,base) → tcbuffer
atValues(tcbuffer,values) → tcbuffer
minusValues(tcbuffer,values) → tcbuffer
```

```
SELECT asText(atValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
-- {[Cbuffer(POINT(2 2),0.5)@2001-01-02]}
SELECT asText(minusValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
/* {[Cbuffer(POINT(2 2),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02),
   (Cbuffer(POINT(2 2),0.5)@2001-01-02, Cbuffer(POINT(2 2),0.7)@2001-01-03]} */
```

- Restringe al (complemento de) una geometría

```
atGeometry(tcbuffer,geometry) → tcbuffer
minusGeometry(tcbuffer,geometry) → tcbuffer
```

```
SELECT asText(atGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0,0 2,2 2,2 0,0 0))'));
-- {[Cbuffer(POINT(2 2),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.7)@2001-01-03]}
SELECT asText(minusGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0,0 2,2 2,2 0,0 0))'));
-- NULL
```

- Restringe un búfer circular temporal a (el complemento de) una caja espaciotemporal

```
atStbox(tcbuffer,stbox,borderInc boolean=true) → tcbuffer
minusStbox(tcbuffer,stbox,borderInc boolean=true) → tcbuffer
```

```
SELECT asText(atStbox(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]', stbox 'STBOX X((0,0),(2,2))'));
/* {[Cbuffer(POINT(1 1),0.5)@2001-01-01,
   Cbuffer(POINT(2.353553390590278 2.353553390590278),0.5)@2001-01-02 08:29:07.012947]} */
```

12.11. Operaciones espaciales

- Devuelve o establece el identificador de referencia espacial

```
SRID(tcbuffer) → integer
setSRID(tcbuffer,integer) → tcbuffer
```

```
SELECT SRID(tcbuffer 'SRID=5676;
  [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)');
-- 5676
SELECT asEWKT(setSRID(tcbuffer '[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(1 1),2)@2001-01-02)', 5676));
-- SRID=5676;[Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)
```

- Transforma a una referencia espacial diferente

```
transform(tcbuffer,integer) → tcbuffer
transformPipeline(tcbuffer,pipeline text,srid integer,
  is_forward boolean=true) → tcbuffer
```

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando el búfer circular temporal de entrada tiene un SRID desconocido (representado por 0). La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato de cadena: `urn:ogc:def:coordinateOperation:AUTHORITY::CODE`. El SRID del búfer circular temporal de entrada se ignora y el SRID del búfer circular temporal de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `srid`. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
SELECT asEWKT(transform(tcbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)@2001-01-01',
  3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tcbuffer, pipeline) AS (
  SELECT tcbuffer 'SRID=4326;{Cbuffer(Point(4.3525 50.846667),1)@2001-01-01,
    Cbuffer(Point(-0.1275 51.507222),2)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(tcbuffer, pipeline, 4326), pipeline,
  4326, false), 6) FROM test;
/* SRID=4326;{Cbuffer(POINT(4.3525 50.846667),1)@2001-01-01,
   Cbuffer(POINT(-0.1275 51.507222),2)@2001-01-02} */
```

- Devuelve el área atravesada por el búfer circular temporal

```
traversedArea(tcbuffer,unary_union boolean=false) → geometry
```

```
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]'));
-- CURVEPOLYGON(CIRCULARSTRING(0.5 1,1.5 1,0.5 1))
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 1), 0.5)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]', true));
/* CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(3 0.5,3.5 1,3 1.5),(3 1.5,1 1.5),
   CIRCULARSTRING(1 1.5,0.646446609406726 1.353553390593274,0.5 1),
   CIRCULARSTRING(0.5 1,0.646446609406726 0.646446609406726,1 0.5),(1 0.5,3 0.5))) */
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0)@2001-01-01,
  Cbuffer(Point(3 1), 0)@2001-01-02, Cbuffer(Point(4 1), 0.5)@2001-01-03]', true));
/* GEOMETRYCOLLECTION(CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(3.75 0.566987298107781,
   4.5 1,3.75 1.433012701892219),(3.75 1.433012701892219,3 1,3.75 0.566987298107781))),
   LINESTRING(1 1,3 1)) */
```

El argumento es falso salvo que se indique, y la función responde entonces las piezas que barre el búfer, cada una un disco o la región que un disco barre entre dos instantes consecutivos, reunidas en una geometría. Con `unary_union = true` se disuelven en la región que cubren, calculada de forma exacta: cada encuentro de los círculos y de los segmentos que los unen se decide sobre las coordenadas de entrada, ningún arco se reemplaza por segmentos y solo se redondean los vértices del resultado, cada uno una vez. El resultado conserva sus arcos, como `CURVEPOLYGON` o `MULTISURFACE`. Donde el búfer mantiene un radio nulo, la línea o el punto que traza y que ningún disco cubre se devuelve junto a las superficies en una `GEOMETRYCOLLECTION`. Donde el cálculo no puede decidir un encuentro o un orden de los bordes, la función genera un error en lugar de responder una aproximación.

- Devuelve el punto geométrico temporal dado por los centros de un búfer circular temporal

```
centroid(tcbuffer) → tgeompoint
```

```
SELECT asText(centroid(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]'));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03]
```

- Devuelve la envolvente convexa del área atravesada por el búfer circular temporal

```
convexHull(tcbuffer) → geometry
```

La envolvente convexa se calcula sobre el área atravesada exacta, arcos incluidos: un arco alcanza más allá de los puntos que lo definen, por lo que aporta a la envolvente el trozo de su círculo que ninguna otra característica del área alcanza. Por lo tanto, el resultado es un `CURVEPOLYGON` curvo delimitado por un `COMPOUNDCURVE` de arcos circulares y segmentos rectos, tan exacto como la región que devuelve `traversedArea`. La envolvente es lineal solo cuando el búfer mantiene un radio cero, ya que el área atravesada no lleva entonces ningún arco.

```
SELECT ST_AsText(convexHull(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(1 3), 0.5)@2001-01-02]'));
/* CURVEPOLYGON(COMPOUNDCURVE((1.5 3,1.5 1),CIRCULARSTRING(1.5 1,1 0.5,0.5 1),
   (0.5 1,0.5 3),CIRCULARSTRING(0.5 3,1 3.5,1.5 3))) */
SELECT ST_AsText(convexHull(tcbuffer '[Cbuffer(Point(1 1), 0)@2001-01-01,
  Cbuffer(Point(1 3), 0)@2001-01-02, Cbuffer(Point(3 3), 0)@2001-01-03]'));
-- POLYGON((1 1,1 3,3 3,1 1))
```

12.12. Operaciones de cuadro delimitador

- Operadores topológicos

```
{tstzspan,stbox,tcbuffer} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tcbuffer} → boolean
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' && stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' @> stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- true
SELECT stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)') <@
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' ~= tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.35)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true
```

- Operadores de posición

```
{stbox,tcbuffer} {<<, &<, >>, &>} {stbox,tcbuffer} → boolean
{stbox,tcbuffer} {<<|, &<|, |>>, |&>} {stbox,tcbuffer} → boolean
{tstzspan,stbox,tcbuffer} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tcbuffer} → boolean
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' <<
  stbox(cbuffer 'Cbuffer(Point(1 1), 0.2)');
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' <<| stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
  Cbuffer(Point(1 1), 0.5)@2001-01-05]' #&> tstzspan '[2001-01-01,2001-01-03]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
  Cbuffer(Point(1 1), 0.5)@2001-01-05]' #>>
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02]';
-- true
```

- Devuelve la caja delimitadora espaciotemporal expandida en la dimensión espacial por un valor flotante

```
expandSpace(tcbuffer,float) → stbox
```

```
SELECT expandSpace(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]', 2);
-- STBOX XT(((-1.5,-1.5),(5.5,5.5)),[2001-01-01, 2001-01-03])
```

12.13. Operaciones de distancia

Los operandos de dimensiones distintas se leen en las dimensiones que ambos tienen, véase Sección 8.3.

- Devuelve la distancia más cercana

```
{geo,cbuffer,tcbuffer,stbox} |=| {geo,cbuffer,tcbuffer,stbox} → float
```

El operador `|=|` se puede utilizar para realizar búsquedas del vecino más cercano utilizando un índice GiST o SP-GiST (ver Sección 10.2).

```
SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' |=| geometry 'Linestring(50 50,55 55)';
-- 67.18225099390857
```

```
SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' |=| cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- 0.2142135623730952
```

- Devuelve el instante del primer búfer circular temporal en el que los dos argumentos están a la distancia más cercana

```
nearestApproachInstant({geo,cbuffer,tcbuffer},{geo,cbuffer,tcbuffer}) → tcbuffer
```

```
SELECT asText(nearestApproachInstant(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- Cbuffer(POINT(2 2),0.7)@2001-01-02
SELECT asText(nearestApproachInstant(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)'));
-- Cbuffer(POINT(2 2),0.7)@2001-01-02
```

- Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos

```
shortestLine({geo,cbuffer,tcbuffer},{geo,cbuffer,tcbuffer}) → geometry
```

La función devuelve la primera línea que encuentre si hay más de una

```
SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(2.494974746830583 2.494974746830583,50 50)
SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)'));
-- LINESTRING(1.505025253169417 1.505025253169417,1.353553390593274 1.353553390593274)
```

- Devuelve la distancia temporal

```
{geo,cbuffer} <-> tcbuffer → tfloat
tcbuffer <-> {geo,cbuffer,tcbuffer} → tfloat
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- [0@2001-01-01, 0@2001-01-03]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)';
-- [68.99646455628165@2001-01-01, 68.79646455628166@2001-01-03]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <->
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-04]';
-- [0@2001-01-02, 0@2001-01-03]
```

- Devuelve la distancia espacial mínima entre dos conjuntos de búferes circulares temporales, ignorando el tiempo

```
minDistance(tcbuffer[],tcbuffer[]) → float
```

La distancia se mide entre las áreas recorridas sin tener en cuenta el tiempo, igual a `ST_Distance` del área recorrida de cada argumento, y el resultado es el mínimo sobre todos los pares tomados de los dos conjuntos. Dos valores cuyos discos barren la misma región en momentos distintos están por lo tanto a distancia cero. La magnitud sincronizada en el tiempo, que compara dos argumentos solo en los instantes que comparten, es |=|. Un único par es el caso de un elemento.

```
SELECT minDistance(ARRAY[tcbuffer '[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(2 0),1)@2001-01-02]'], ARRAY[tcbuffer
  '[Cbuffer(Point(0 5),0.5)@2001-01-03, Cbuffer(Point(2 5),0.5)@2001-01-04]']);
-- 3.5
```

12.14. Relaciones siempre y alguna vez

- Relaciones alguna vez y siempre

```
eContains({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aContains({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean
aDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean
eIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
```

Las relaciones *alguna vez* y *siempre* determinan si la relación topológica o de distancia se satisface alguna vez o siempre (ver Sección 5.4.2) y devuelven un `boolean`

```
SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)');
-- true
SELECT eDisjoint(cbuffer 'Cbuffer(Point(2 2), 0.0)',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)');
-- true
SELECT eIntersects(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03)',
  tcbuffer '[Cbuffer(Point(2 2), 0.0)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-03)');
-- false
```

12.15. Relaciones espaciotemporales

- Relaciones espaciotemporales

```
tContains({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → tbool
tIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
```

Las relaciones *temporales* calculan la relación topológica o de distancia en cada instante y dan como resultado un `tbool`

```
SELECT tDisjoint(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)');
-- {[f@2001-01-01, f@2001-01-03)}
SELECT tDwithin(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03)', tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03)', 1);
-- {[t@2001-01-01, t@2001-01-03)}
```

12.16. Comparaciones

- Comparaciones tradicionales

```
tcbuffer {=, <>, <, >, <=, >=} tcbuffer → boolean
```

```
SELECT tcbuffer '{[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02),
  [Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]}' =
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tcbuffer '{[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]}' <>
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.6)@2001-01-03]';
-- false
```

- Comparaciones alguna vez y siempre

```
{tcbuffer,cbuffer} {?=, ?<>, %=, %<>} {tcbuffer,cbuffer} → boolean
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03)' ?= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03)' ?= tcbuffer '[Cbuffer(Point(1 1), 0.4)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-03)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-04)' %= cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- true
```

- Comparaciones temporales

```
{tcbuffer,cbuffer} {#=, #<>} {tcbuffer,cbuffer} → tbool
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03)' #= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.8)@2001-01-03)' #<>
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.7)@2001-01-03)';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```

12.17. Agregaciones

Las cuatro funciones agregadas para búferes circulares temporales se ilustran a continuación.

- Extensión del cuadro delimitador

```
extent(tcbuffer) → stbox
```

```
SELECT extent(temp) FROM (VALUES
  (tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'),
  (tcbuffer '[Cbuffer(Point(2 2), 0.2)@2001-01-02,
    Cbuffer(Point(3 3), 0.4)@2001-01-03]')) t(temp);
-- STBOX XT(((0.5,0.5),(3.4,3.4)),[2001-01-01, 2001-01-03])
```

- Conteo temporal

```
tCount(tcbuffer) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
    Cbuffer(Point(1 1), 0.5)@2001-01-05)' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05)}
```

- Conteo de ventana

```
wCount(tcbuffer) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
    Cbuffer(Point(1 1), 0.5)@2001-01-05)' )
SELECT wCount(Temp, '1 day') FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
   1@2001-01-06)} */
```

- Agrega los centros de un conjunto de búferes circulares temporales en su centroide temporal, convirtiendo a `tgeompoint`

```
tCentroid(tcbuffer::tgeompoint) → tgeompoint
```

La agregación pertenece al tipo `tgeompoint`: la trayectoria del centro de un búfer circular es un punto temporal, de modo que la conversión alcanza la agregación en lugar de que el tipo búfer lleve su propia copia.

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tcbuffer '[Cbuffer(Point(3 3), 0.2)@2001-01-01,
    Cbuffer(Point(3 3), 0.4)@2001-01-03)' )
SELECT asText(tCentroid(temp::tgeompoint)) FROM Temp;
-- {[POINT(2 2)@2001-01-01, POINT(2 2)@2001-01-03)}
```

12.18. Simplificación

- Devuelve un búfer circular temporal simplificado asegurándose de que los centros consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo

```
minDistSimplify(tcbuffer,mindist float) → tcbuffer
minTimeDeltaSimplify(tcbuffer,mint interval) → tcbuffer
```

La simplificación opera sobre la trayectoria de los centros del búfer circular temporal y conserva el radio en cada instante que sobrevive. La simplificación se aplica sólo a secuencias temporales o conjuntos de secuencias con interpolación lineal. En todos los demás casos, se devuelve una copia del valor dado.

```
SELECT asText(minDistSimplify(tcbuffer '[Cbuffer(Point(0 0), 1)@2001-01-01,
  Cbuffer(Point(1 0), 1)@2001-01-02, Cbuffer(Point(4 0), 1)@2001-01-03]', 2));
-- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(4 0),1)@2001-01-03]
```

```
-- The result keeps the interpolation of the value it simplifies.
SELECT interp(minDistSimplify(tcbuffer '[Cbuffer(Point(0 0), 1)@2001-01-01,
  Cbuffer(Point(1 0), 1)@2001-01-02, Cbuffer(Point(4 0), 1)@2001-01-03]', 2));
-- Linear
SELECT asText(minTimeDeltaSimplify(tcbuffer '[Cbuffer(Point(0 0), 1)@2001-01-01,
  Cbuffer(Point(1 0), 1)@2001-01-02, Cbuffer(Point(4 0), 1)@2001-01-04]',
  interval '2 days'));
-- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(4 0),1)@2001-01-04]
```

- Devuelve un búfer circular temporal simplificado usando el algoritmo de Douglas-Peucker

```
maxDistSimplify(tcbuffer,maxdist float,syncdist boolean=true) → tcbuffer
douglasPeuckerSimplify(tcbuffer,maxdist float,syncdist boolean=true) → tcbuffer
```

La diferencia entre las dos funciones es que `maxDistSimplify` utiliza una versión de un solo paso del algoritmo mientras que `douglasPeuckerSimplify` utiliza el algoritmo recursivo estándar. Al igual que las funciones anteriores, la simplificación opera sobre la trayectoria de los centros y conserva el radio en cada instante que sobrevive.

```
SELECT asText(douglasPeuckerSimplify(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
  Cbuffer(Point(2 2),0.5)@2001-01-02, Cbuffer(Point(3 1),0.5)@2001-01-03,
  Cbuffer(Point(4 4),0.5)@2001-01-04]', 2.0));
-- [Cbuffer(POINT(1 1),0.5)@2001-01-01, Cbuffer(POINT(4 4),0.5)@2001-01-04]
```

12.19. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tabla de búferes circulares temporales. A continuación, se muestra un ejemplo de creación de índice:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador para los búferes circulares temporales, que es un `stbox`, como todos los tipos espaciotemporales.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en búferes circulares temporales,
- `<<#`, `&<#`, `#&>`, `#>>`, que solo consideran la dimensión temporal en búferes circulares temporales,
- `&&`, `@>`, `<@`, `~=`, `-|-` y `|=|`, que consideran tantas dimensiones como compartan la columna indexada y el argumento de consulta.

Estos operadores trabajan con cuadros delimitadores, no con los valores completos.

Capítulo 13

Poses temporales y cadenas de poses

El tipo `pose` se utiliza para representar la *ubicación* y la *orientación* de objetos geométricos dentro de sistemas de coordenadas anclados a la superficie de la Tierra o dentro de otros sistemas de coordenadas astronómicas. La ubicación se representa mediante un punto 2D o 3D. Para las poses 2D, la orientación se define mediante un ángulo de rotación en (-π, π] expresado en radianes. Para las poses 3D, la orientación se define mediante cuatro valores flotantes `W`, `X`, `Y`, `Z`, que representan un cuaternión unitario $Q = \langle W, X, Y, Z \rangle$ donde $\| Q^2 \| = \sqrt{W^2 + X^2 + Y^2 + Z^2} = 1$.

El constructor de poses acepta cuaterniones cuya norma dista como mucho `1e-3` de la unidad (una tolerancia lo bastante amplia como para absorber la deriva del integrador típica de los clientes de fusión de sensores, como las IMU y los entornos de ejecución de RA/RV), y a continuación los renormaliza a norma exactamente unitaria antes de almacenarlos. Las normas muy alejadas (por ejemplo, el caso de error evidente `(1, 1, 1, 1)`, donde `|q| = 2`), los cuaterniones nulos y los cuaterniones con componentes `NaN`/`Inf` se rechazan de entrada. La representación en disco es por tanto independiente de la higiene de coma flotante de quien llama, y el código posterior de comparación, hash, SLERP y descomposición de Euler puede confiar en el invariante de norma unitaria.

El grupo de trabajo de estándares GeoPose (SWG), que trabaja bajo los auspicios del Open Geospatial Consortium, ha definido un estándar para intercambiar información de poses entre distintos usuarios, dispositivos y plataformas. Se puede encontrar más información sobre el estándar en el repositorio de GitHub del SWG GeoPose.

El tipo `pose` sirve como tipo base para definir el tipo de pose temporal `tpose`. El tipo `tpose` tiene una funcionalidad similar al tipo de punto temporal `tgeompoint`. Por lo tanto, la mayoría de las funciones y operadores descritos anteriormente para el tipo `tgeompoint` también son aplicables para el tipo `tpose`. Además, hay funciones específicas definidas para el tipo `tpose`. En este capítulo cubrimos estas funciones.

El tipo `tpose` se utiliza para definir el tipo `trgeometry` (es decir, geometría rígida temporal) definido en el siguiente capítulo. La implementación de estos tipos en MobilityDB se ha estudiado en la siguiente tesis doctoral.

- Maxime Schoemans, *Managing Rigid Temporal Geometries in Moving Object Databases*, tesis doctoral, Université libre de Bruxelles, 2025.

El tipo `posechain` representa un cuerpo cuyas partes se sitúan unas respecto de otras: una cámara sobre un cardán sobre un vehículo, o un brazo cuyo antebrazo se sitúa respecto del brazo. Una cadena de poses es una lista ordenada de al menos una `pose`. El primer eslabón se expresa en el marco exterior de la cadena, y cada eslabón posterior se expresa en el marco que define el eslabón anterior, de modo que un eslabón indica dónde se sitúa una parte respecto de la parte que la sostiene y no respecto del mundo.

El estándar OGC GeoPose denomina a esta estructura una cadena de transformaciones de marcos, donde cada transformación es un par de marcos de referencia en el que el marco exterior es el dominio y el marco interior es el rango. El estándar establece que un marco interior no puede ser un marco topocéntrico, por lo que una cadena lleva exactamente un marco que nombra un lugar sobre la tierra, en su exterior. De ello se derivan dos propiedades, ambas verificadas: toda la cadena tiene un solo SRID y todos los eslabones tienen la misma dimensión. Solo el eslabón exterior puede ser geodésico y solo el eslabón exterior puede llevar un SRID.

Cuando el marco exterior es geográfico, la posición del eslabón exterior está en grados mientras que la traslación de un eslabón posterior está en metros a lo largo de los ejes locales Este-Norte-Arriba en su marco padre. La composición realiza esa suma en

coordenadas geocéntricas y lee de vuelta la posición como geográfica, de modo que una cadena sobre un marco geográfico se compone sobre el elipsoide y no en el plano, y la orientación se vuelve a expresar respecto de la base Este-Norte-Arriba en la posición que alcanza la composición.

La composición de todos los eslabones da la pose del marco más interior en el marco exterior de la cadena, que es lo que devuelve la conversión al tipo `pose`. Esa única conversión otorga a una cadena toda la superficie definida para las poses, de modo que el tipo en sí permanece reducido. Los dos tipos responden por tanto a las mismas preguntas, y este capítulo declara cada operación una sola vez, nombrando el tipo solo donde la respuesta depende de él.

Lo que declara	Tipo base	Tipo conjunto	Tipo temporal
Una colocación	`pose`	`poseset`	`tpose`
Una composición de colocaciones	`posechain`	`posechainset`	`tposechain`

13.1. Notación

La mayoría de las funciones y los operadores para tipos temporales descritos en los capítulos anteriores pueden aplicarse a las poses y a las cadenas de poses. Por tanto, en las signaturas de las funciones, la notación `base` representa una `pose` o una `posechain` y la notación `ttype` un `tpose` o un `tposechain`; las notaciones `geompoint2D` y `geompoint3D` representan un `geompoint` con, respectivamente, dos y tres dimensiones, y las notaciones `geogpoint2D` y `geogpoint3D` representan un `geogpoint` con, respectivamente, dos y tres dimensiones. Para evitar redundancia, presentamos a continuación solo lo que es específico de los dos tipos.

Una signatura que responden ambos tipos los nombra como una alternativa, `{pose,posechain}`, y una signatura que responde solo uno de ellos nombra ese. Una función cuyo *nombre* difiere entre los dos conserva una línea propia, de modo que `tpose(pose,timestamptz)` y `tposechain(posechain,timestamptz)` nunca se colapsan en un nombre invocable que no existe.

13.2. Poses estáticas y cadenas de poses estáticas

Una `pose` 2D es un par de la forma `(point2D,radius)` donde `point2D` es un punto geométrico 2D y `radius` es un valor `float` que representa un ángulo de rotación en $(-\pi, \pi]$ expresado en radianes. Una `pose` 3D es una tupla de la forma `(point3D,W,Y,X,Z)` donde `point3D` es un punto geométrico 3D, y `W`, `X`, `Y` y `Z` son cuatro `floats` que representan un cuaternión *unitario*. Ejemplos de entrada de valores de pose son los siguientes:

```
SELECT pose 'Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

Se puede especificar un SRID para una pose ya sea al comienzo del literal de pose o antes del literal de punto, como se muestra a continuación.

```
SELECT pose 'SRID=3812;Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(SRID=5676;Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

Al igual que para otros tipos espaciales, un `pose` puede ser planar (geométrico) o geodésico (geográfico). Un pose geodésico se denota con la palabra clave `GeodPose`, de la misma manera que un `stbox` geodésico se denota con `GEODSTBOX`. Un pose geodésico que no indica un SRID tiene el SRID 4326 de una geografía, mientras que un pose planar tiene el SRID 0.

```
SELECT pose 'GeodPose(Point(1 1), 0.5)';
SELECT pose 'SRID=4326;GeodPose(Point Z(1 1 1), 1, 0, 0, 0)';
```

Los valores del tipo pose deben satisfacer varias restricciones para que estén bien definidos. Ejemplos de valores incorrectos del tipo pose son los siguientes.

```
-- Empty point
select pose 'Pose(Point empty, 0.5)';
-- Incorrect point value
SELECT pose 'Pose(Linestring(1 1,2 2), 1.0)';
-- Incorrect radius value
SELECT pose 'Pose(Point(1 1), -10.0)';
-- Incorrect 3D point
SELECT pose 'Pose(Point Z(1 1), 1.0)';
-- Incomplete 3D orientation
SELECT pose 'Pose(Point Z(1 1 1), 1.0)';
```

A continuación damos las funciones y operadores para el tipo pose.

13.2.1. Entrada y salida

- Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)

```
asText({pose,posechain,pose[],posechain[]}) → {text,text[]}
asEWKT({pose,posechain,pose[],posechain[]}) → {text,text[]}
```

```
SELECT asText(pose 'SRID=4326;Pose(Point(0 0),1)');
-- Pose(POINT(0 0),1)
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(POINT(0 0),1)","Pose(POINT(1 1),2)"}
SELECT asEWKT(pose 'SRID=4326;Pose(Point(0 0),1)');
-- SRID=4326;Pose(POINT(0 0),1)
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)', 'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2))"}
```

- Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
asBinary({pose,posechain},endian text='') → bytea
asEWKB({pose,posechain},endian text='') → bytea
asHexWKB({pose,posechain},endian text='') → text
asHexEWKB({pose,posechain},endian text='') → text
```

La cadena escribe sus indicadores y su único SRID una sola vez, después el número de eslabones y después los valores de cada eslabón en orden. El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(pose 'Pose(Point(1 2),1)');
-- \x0101000000000000f03f0000000000000040000000000000f03f
SELECT asEWKB(pose 'SRID=7844;Pose(Point(1 2),1)');
-- \x0141a41e0000000000000000f03f0000000000000040000000000000f03f
SELECT asHexWKB(pose 'Pose(Point(1 2),1)');
-- 0101000000000000F03F0000000000000040000000000000F03F
SELECT asHexEWKB(pose 'SRID=3812;Pose(Point(1 2),1)');
-- 0141E40E0000000000000000F03F0000000000000040000000000000F03F
```

- Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)

```
poseFromText(text) → pose
poseFromEWKT(text) → pose
posechainFromText(text) → posechain
posechainFromEWKT(text) → posechain
```

Estas funciones leen solo la forma de texto. La entrada del tipo, `pose '...'`, lee además un documento OGC GeoPose, como la entrada de `geometry` lee uno GeoJSON, y poseFromGeoPose lee solo un documento GeoPose.

```
SELECT asEWKT(poseFromText(text 'Pose(Point(1 2),1)'));
-- Pose(POINT(1 2),1)
SELECT asEWKT(poseFromEWKT(text 'SRID=3812;Pose(Point(1 2),1)'));
-- SRID=3812;Pose(POINT(1 2),1)
SELECT asEWKT(posechainFromText('PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),1))'));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),1))
SELECT asEWKT(posechainFromEWKT('SRID=3812;PoseChain(Pose(Point(0 0),0))'));
-- SRID=3812;PoseChain(Pose(POINT(0 0),0))
```

- Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

```
poseFromBinary(bytea) → pose
poseFromEWKB(bytea) → pose
poseFromHexWKB(text) → pose
poseFromHexEWKB(text) → pose
posechainFromBinary(bytea) → posechain
posechainFromEWKB(bytea) → posechain
posechainFromHexWKB(text) → posechain
posechainFromHexEWKB(text) → posechain
```

```
SELECT asEWKT(poseFromBinary('\x0101000000000000f03f000000000000000040000000000000f03f'));
-- Pose(POINT(1 2),1)
SELECT asEWKT(poseFromEWKB(
  '\x0141a41e0000000000000000f03f000000000000000040000000000000f03f'));
-- SRID=7844;Pose(POINT(1 2),1)
SELECT asEWKT(poseFromHexEWKB(
  '0141E40E0000000000000000F03F000000000000000040000000000000F03F'));
-- SRID=3812;Pose(POINT(1 2),1)
SELECT posechainFromBinary(asBinary(posechain 'PoseChain(Pose(Point(1 2),0.5))')) =
  posechain 'PoseChain(Pose(Point(1 2),0.5))';
-- t
SELECT posechainFromHexEWKB(asHexEWKB( posechain 'SRID=3812;
  PoseChain(Pose(Point(1 2),0.5))')) = posechain 'SRID=3812;PoseChain(Pose(Point(1 2),
  0.5))';
-- t
```

13.2.2. Constructores

- Constructor de poses geodésicos a partir de un punto geográfico

```
geodpose(geogpoint2D,float) → pose
geodpose(geogpoint3D,float,float,float,float) → pose
geodpose(geogpoint3D,yaw float,pitch float,roll float) → pose
```

Es una función propia, como `geodstbox`, ya que un punto escrito como un literal sin tipo se resuelve a una geometría en `pose`.

```
SELECT asText(geodpose(geography 'Point(1 2)', 0.5));
-- GeodPose(POINT(1 2),0.5)
SELECT asText(geodpose(geography 'Point(1 2 3)', 1, 0, 0, 0));
-- GeodPose(POINT Z (1 2 3),1,0,0,0)
```

- Constructor para poses

```
pose(geompoint2D,float) → pose
pose(geompoint3D,float,float,float,float) → pose
pose(geompoint3D,yaw float,pitch float,roll float) → pose
```

La forma de tres ángulos toma la orientación en la otra codificación que prescribe el estándar OGC GeoPose v1.0, una terna yaw / pitch / roll en radianes bajo la convención Tait-Bryan intrínseca ZYX, y almacena el cuaternión que denota. Es la inversa de ypr.

```
SELECT asText(pose(ST_Point(1,1), radians(45)), 6);
-- Pose(POINT(1 1),0.785398)
SELECT asEWKT(pose(ST_Point(1,1,3812), radians(45)), 6);
-- SRID=3812;Pose(POINT(1 1),0.785398)
SELECT asText(pose(ST_PointZ(1,1,1), 1, 0, 0, 0));
-- Pose(POINT Z (1 1 1),1,0,0,0)
SELECT asText(pose(ST_PointZ(1,1,1), radians(90), 0, 0), 6);
-- Pose(POINT Z (1 1 1),0.707107,0,0,0.707107)
```

13.2.3. Conversión de tipos

Los valores del tipo `pose` se pueden convertir al tipo de punto `geometry` utilizando un `CAST` explícito o utilizando la notación `::` como se muestra a continuación.

- Convierte una pose y, opcionalmente, una marca de tiempo o un período, en una caja espaciotemporal

```
{pose,posechain}::stbox
stbox({pose,posechain}) → stbox
stbox({pose,posechain},{timestamptz,tstzspan}) → stbox
```

La caja cubre la posición compuesta de cada prefijo de la cadena y no solo la del conjunto: cada articulación es un lugar, y una ventana de consulta que alcanza el codo pero no la mano alcanza igualmente la cadena.

```
SELECT stbox(pose 'SRID=5676;Pose(Point(1 1),0.3)');
-- SRID=5676;STBOX X((1,1),(1,1))
SELECT stbox(pose 'Pose(Point(1 1),0.3)', timestamptz '2001-01-01');
-- STBOX XT(((1,1),(1.3,1.3)),[2001-01-01, 2001-01-01])
SELECT stbox(pose 'Pose(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT(((1,1),(1.3,1.3)),[2001-01-01, 2001-01-02])
```

- Convierte una pose, planar o geodésica, en un punto geográfico

  Las conversiones siguen la regla de [?listitem].

```
{pose,posechain}::geogpoint
```

```
SELECT ST_AsEWKT(pose 'GeodPose(Point(1 2), 0.5)'::geography);
-- SRID=4326;POINT(1 2)
SELECT ST_AsEWKT(pose 'Pose(Point(1 2), 0.5)'::geography);
-- SRID=4326;POINT(1 2)
SELECT pose 'SRID=3857;Pose(Point(1 2), 0.5)'::geography;
-- ERROR:  Only lon/lat coordinate systems are supported in geography.
```

- Convierte una pose, planar o geodésica, en un punto geométrico

```
{pose,posechain}::geompoint
```

```
SELECT ST_AsText(pose(ST_Point(1, 1), 1)::geometry);
-- POINT(1 1)
SELECT ST_AsEWKT(pose(ST_PointZ(1, 1, 1, 5676), 1, 0, 0, 0)::geometry);
-- SRID=5676;POINT(1 1 1)
```

13.2.4. Accesores

- Devuelve el punto, que para una cadena de poses es el punto de su marco más interior

```
point({pose,posechain}) → geometry
```

```
SELECT ST_AsText(point(pose 'Pose(Point(1 1), 0.3)'));
-- POINT(1 1)
```

- Devuelve el cuaternión de orientación de una pose

```
quaternion(pose) → quaternion
```

Las componentes se devuelven en el orden `W`, `X`, `Y`, `Z`, la convención de Hamilton en la que una pose 3D las almacena. Esta es una de las dos codificaciones de orientación que prescribe el estándar OGC GeoPose v1.0; la otra, yaw / pitch / roll, la proporciona ypr. Ambas están definidas para las dos dimensiones: la orientación de una pose 2D es un giro alrededor de la vertical local por su ángulo almacenado, de modo que coloca la mitad de ese ángulo en `W` y `Z` y deja `X` e `Y` en cero. Este es el cuaternión que un documento GeoPose Basic-Quaternion lleva para una pose plana.

```
SELECT quaternion(pose 'Pose(Point Z(1 1 1), 0, 0, 0, 1)');
-- (0,0,0,1)
SELECT quaternion(pose 'Pose(Point(1 1), 0)');
-- (1,0,0,0)
```

13.2.5. Transformaciones

- Redondea el punto y la orientación de la pose al número de posiciones decimales

```
round({pose,pose[],posechain},integer=0) → {pose,pose[],posechain}
```

```
SELECT asText(round(pose(ST_Point(1.123456789,1.123456789), 0.123456789), 6));
-- Pose(POINT(1.123457 1.123457),0.123457)
SELECT asText(round(ARRAY[pose(ST_Point(1.123456789,1.123456789), 0.123456789),
  pose(ST_Point(2.987654321,2.987654321), 0.987654321)], 6));
-- {"Pose(POINT(1.123457 1.123457),0.123457)","Pose(POINT(2.987654 2.987654),0.987654)"}
```

13.2.6. Sistema de referencia espacial

- Devuelve o establece el identificador de referencia espacial, que para una cadena de poses es el identificador de su marco exterior

```
SRID({pose,posechain}) → integer
setSRID({pose,posechain},integer) → {pose,posechain}
```

```
SELECT SRID(pose 'Pose(SRID=5676;Point(1 1), 0.3)');
-- 5676
SELECT asEWKT(setSRID(pose 'Pose(Point(0 0),1)', 4326));
-- SRID=4326;Pose(POINT(0 0),1)
```

- Transforma a un identificador de referencia espacial

```
transform({pose,posechain},integer) → {pose,posechain}
transformPipeline({pose,posechain},pipeline text,srid integer,
  is_forward boolean=true) → {pose,posechain}
```

La función `transform` especifica la transformación con un SRID de destino. Se genera un error cuando la entrada tiene un SRID desconocido (representado por 0). La posición se transforma como se transforma un punto, y la orientación se escribe en los ejes del marco de destino en la posición transformada, para cualquier par de sistemas de referencia. Un sistema geográfico escribe una orientación en el marco Este-Norte-Arriba en la posición, y cualquier otro sistema en sus ejes de coordenadas. La orientación gira el ángulo entre los dos marcos en la posición, que para una proyección es su convergencia de meridianos, de modo que el cuerpo mira hacia la misma dirección sobre el terreno antes y después de la transformación. Una proyección que no conserva los ángulos lleva esa dirección exactamente, y los demás ejes de una pose 3D son los más próximos que le son ortogonales. En una cadena solo el eslabón exterior nombra un marco y solo él se transforma, siendo cada uno de los demás eslabones una transformación rígida leída en los ejes de su marco padre, que un cambio del marco exterior deja como estaban. La función `transformPipeline` especifica la transformación con una canalización de transformación de coordenadas definida representada con el siguiente formato de cadena:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

La canalización transforma la posición sea cual sea el SRID de la pose de entrada, y el SRID de la pose de salida se establecerá en cero a menos que se proporcione un valor a través del parámetro opcional `srid`. La orientación se escribe en el marco de destino como en `transform`, siendo los marcos los que nombran el SRID de la pose de entrada y el parámetro `srid`; se mantiene tal cual cuando alguno de ellos es desconocido. Como se indica en el último parámetro, la canalización se ejecuta de forma predeterminada en dirección hacia adelante; al establecer el parámetro en falso, la canalización se ejecuta en la dirección inversa.

```
-- The Belgian Lambert projection turns the North by -0.007111 degrees at this position
SELECT asEWKT(transform(pose 'SRID=4326;Pose(Point(4.35 50.85),1)', 3812), 6);
-- SRID=3812;Pose(POINT(648679.018035 671067.055638),0.999876)
-- A body facing East at 12E 45N faces 2.1223 degrees North of the East of the grid of
-- UTM zone 32
SELECT round(degrees(yaw(transform(pose 'SRID=4326;Pose(Point(12 45),0)', 32632)))::
    numeric, 6);
-- 2.122300
```

```
-- Round-tripping a 3D pose through ECEF and back lands at the input pose
SELECT asEWKT(round( transform(transform(pose 'SRID=4326;Pose(Point(8 47 0), 1, 0, 0, 0)',
  4978), 4326), 6));
-- SRID=4326;Pose(POINT Z (8 47 0),1,0,0,0)
-- At the equator-meridian (lat=lon=0), the body identity quaternion in the ECEF basis is
-- the canonical East-North-Up to ECEF rotation
SELECT asEWKT(round(transform(pose 'SRID=4326;Pose(Point(0 0 0), 1, 0, 0, 0)', 4978), 6));
-- SRID=4978;Pose(POINT Z (6378137 0 0),0.5,0.5,0.5,0.5)
```

```
WITH test(pose, pipeline) AS (
  SELECT pose 'Pose(SRID=4326;Point(4.3525 50.846667),1)',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(pose, pipeline, 32631), pipeline, 4326,
  false), 6) FROM test;
-- SRID=4326;Pose(POINT(4.3525 50.846667),1)
```

13.2.7. Operaciones de distancia

- Devuelve la distancia

```
distance({geo,pose},pose) → float
distance(pose,{geo,pose}) → float
distance({geo,posechain},posechain) → float
distance(posechain,{geo,posechain}) → float
{geo,pose} <-> pose → float
{geo,posechain} <-> posechain → float
```

Una cadena de poses se mide por la pose que da su composición. Solo se tiene en cuenta el componente de punto de la pose, la orientación se ignora. La distancia se calcula en dos dimensiones, incluso cuando los argumentos son tridimensionales. El resultado es `NULL` cuando la geometría está vacía.

Un pose geodésico y una geografía se miden sobre la superficie de la Tierra, en metros, cualquiera que sea su Z.

```
SELECT round(pose 'GeodPose(Point(0 0), 0)' <-> geography 'Point(1 0)', 3);
-- 111319.491
SELECT round(distance(geometry 'Point(1 0)', pose 'Pose(Point(4 0),0)'), 6);
-- 3
SELECT round(pose 'Pose(Point(0 0),0)' <-> pose 'Pose(Point(3 4),0)', 6);
-- 5
SELECT round(pose 'Pose(Point(0 0 0), 1, 0, 0, 0)' <->
  pose 'Pose(Point(3 4 12), 1, 0, 0, 0)', 6);
-- 5
SELECT round(geometry 'Point empty' <-> pose 'Pose(Point(4 0),0)', 6);
-- NULL
SELECT round(distance(posechain 'PoseChain(Pose(Point(1 0),0), Pose(Point(1 0),0))',
  geometry 'Point(5 0)'), 6);
-- 3
```

- Devuelve la distancia de aproximación más cercana

```
nearestApproachDistance({stbox,pose},{pose,stbox}) → float
nearestApproachDistance({stbox,posechain},{posechain,stbox}) → float
{stbox,pose} |=| {pose,stbox} → float
{stbox,posechain} |=| {posechain,stbox} → float
```

El valor es la menor distancia sobre el tiempo que comparten los dos argumentos, y es `NULL` cuando no comparten ninguno. Una pose no lleva período, por lo que está presente en todo tiempo que abarca la caja. Solo se tiene en cuenta el componente de punto de la pose, la orientación se ignora.

```
SELECT round(nearestApproachDistance(stbox 'STBOX X((1,-1),(3,1))', pose 'Pose(Point(4 0) ↩
    ,0)'), 6);
-- 1
SELECT round(pose 'Pose(Point(4 0),0)' |=| stbox 'STBOX X((1,-1),(3,1))', 6);
-- 1
```

13.2.8. Comparaciones

Los operadores de comparación (=, < y así sucesivamente) están disponibles para poses. Excepto la igualdad y la desigualdad, los otros operadores de comparación no son útiles en el mundo real pero permiten que los índices de árbol B se construyan en poses.

- Comparaciones tradicionales

```
{pose,posechain} {=, <>, <, >, <=, >=} {pose,posechain} → boolean
```

El orden compara la dimensión, después el SRID, después los eslabones en orden y por último el número de eslabones, de modo que una cadena precede a toda cadena que la extiende.

```
SELECT pose 'Pose(Point(3 3), 0.5)' = pose 'Pose(Point(3 3), 0.5)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' <> pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' < pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.6)' > pose 'Pose(Point(2 2), 0.6)';
-- true
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)' <=
  pose 'Pose(Point Z(2 2 2), 0.5, 0.5, 0.5, 0.5)';
-- true
SELECT pose 'Pose(Point(1 1), 0.6)' >= pose 'Pose(Point(1 1), 0.5)';
-- true
```

13.3. Composición de cadenas de poses

Una cadena de poses declara sus eslabones, y esta sección contiene lo que solo responde una cadena: cómo se construye a partir de poses, cuántos eslabones tiene, qué pose es un eslabón y cómo la cadena entera se compone de nuevo en la única pose que denota.

- Construye una cadena de poses a partir de un arreglo de poses ordenadas desde el marco más exterior hacia dentro

```
posechain(pose[]) → posechain
```

```
SELECT asEWKT(posechain(ARRAY[pose 'Pose(Point(0 0),0)', pose 'Pose(Point(1 0),0)']));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),0))
```

- Convierte una pose en una cadena de poses de un solo eslabón

```
posechain(pose) → posechain
```

```
SELECT asEWKT(posechain(pose 'SRID=3812;Pose(Point(1 2),0.5)'));
-- SRID=3812;PoseChain(Pose(POINT(1 2),0.5))
```

- Devuelve la pose de un marco que define la cadena, leída en el marco exterior de la cadena

```
pose(posechain) → pose
pose(posechain,integer) → pose
```

Cada instante contiene la pose a la que compone toda la cadena, que es donde se sitúa su marco más interno en el marco de la cadena. El resultado es una función escalonada: componer una cadena multiplica las rotaciones de sus eslabones, por lo que la pose de la cadena interpolada entre dos instantes no es la pose interpolada entre las dos poses compuestas, y un resultado lineal respondería una pose que la cadena nunca tiene. Una cadena de un solo eslabón se compone a sí misma, y allí la interpolación del argumento se conserva sin cambios.

Con un argumento se compone toda la cadena, lo que da la pose de su marco más interior. Con dos se componen los primeros `n` eslabones, lo que da la pose del marco que define el eslabón n-ésimo, es decir, dónde se sitúa esa articulación de la cadena.

```
SELECT asEWKT(pose(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),
  Pose(Point(1 0),0))'));
-- Pose(POINT(2 0),0)
SELECT asEWKT(pose(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),
  Pose(Point(1 0),0))', 2));
-- Pose(POINT(1 0),0)
```

- Devuelve una cadena de poses con una pose añadida como su eslabón más interior

```
appendPose(posechain,pose) → posechain
```

La pose se lee en el marco que define el último eslabón de la cadena, por lo que no lleva ni un SRID ni el marcador geodésico.

```
SELECT asEWKT(appendPose(posechain 'PoseChain(Pose(Point(0 0),0))',
  pose 'Pose(Point(1 0),0)'));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),0))
```

- Devuelve el número de eslabones de una cadena de poses

```
numPoses(posechain) → integer
```

```
SELECT numPoses(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),
  Pose(Point(2 0),0))');
-- 3
```

- Devuelve los eslabones de una cadena de poses

```
startPose(posechain) → pose
endPose(posechain) → pose
poseN(posechain,integer) → pose
poses(posechain) → pose[]
```

Un eslabón se devuelve tal como está almacenado, es decir, en el marco que define el eslabón anterior. Dónde se sitúa ese marco en el marco exterior de la cadena es lo que responde `pose(posechain,integer)`. La función `poseN` devuelve `NULL` si la cadena tiene menos eslabones que el número dado.

```
SELECT asEWKT(startPose(posechain 'SRID=3812;
  PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- SRID=3812;Pose(POINT(0 0),0)
SELECT asEWKT(endPose(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- Pose(POINT(1 0),0)
SELECT asEWKT(poseN(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))', 2));
-- Pose(POINT(1 0),0)
SELECT asEWKT(poses(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- {"Pose(POINT(0 0),0)","Pose(POINT(1 0),0)"}
```

- Devuelve el número de eslabones que tiene todo valor de una cadena de poses temporal

```
numPoses(tposechain) → integer
```

El número es el mismo en todo instante, por lo que un instante responde por todo el valor.

```
SELECT numPoses(tposechain 'PoseChain(Pose(Point(0 0), 0),
  Pose(Point(10 0), 0))@2001-01-01');
-- 2
```

13.4. Poses temporales y cadenas de poses temporales

El tipo de cadena de poses temporal `tposechain` representa la evolución en el tiempo de una cadena de marcos anidados, es decir, de un objeto articulado cuyas articulaciones se mueven unas respecto de otras. Como todos los tipos temporales, tiene tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores `tposechain` en estos subtipos.

Todos los valores de una cadena de poses temporal tienen el mismo número de eslabones. Una cadena que gana una articulación es una estructura distinta y no un valor posterior de la misma, y ese número constante es también lo que hace que la interpolación siguiente esté bien definida. En caso contrario se genera un error.

Una cadena de poses temporal se interpola eslabón por eslabón, cada eslabón como lo hace una pose, es decir, linealmente en la posición y a lo largo del arco más corto en la rotación. En el ejemplo siguiente un brazo de dos eslabones gira un cuarto de vuelta en el hombro mientras el codo permanece donde está: a la mitad, el hombro ha girado un octavo de vuelta y el codo no ha cambiado.

Una cadena de poses se ubica donde se ubica la pose que da su composición, por lo que su distancia es la de esa pose, y la distancia de una cadena de poses temporal es la de la pose temporal que da su composición, `tpose(tposechain)`. Esa composición multiplica las rotaciones de los eslabones, por lo que para una cadena de más de un eslabón es una función escalonada, y la distancia conserva entonces el valor de cada instante hasta el siguiente.

El tipo de pose temporal `tpose` permite representar la evolución en el tiempo de la posición de los objetos y su orientación. Como todos los tipos temporales, se presenta en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores de `tpose` en estos subtipos.

```
SELECT tpose 'Pose(Point(1 1), 0.5)@2001-01-01';
SELECT tpose '{Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03}';
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.4)@2001-01-02,
```

```
  Pose(Point(1 1), 0.5)@2001-01-03]';
SELECT tpose '{[Pose(Point(1 1), 1)@2001-01-01, Pose(Point(2 2), 1)@2001-01-02],
  [Pose(Point(2 2), 2)@2001-01-04, Pose(Point(2 2), 3)@2001-01-05]}';
```

Al igual que para los tipos de punto temporal, el tipo de pose temporal acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL) para especificar el subtipo, la dimensionalidad del punto y/o el identificador de referencia espacial (SRID). Los valores posibles para el subtipo son `Instant`, `Sequence` y `SequenceSet` y los valores posibles para el tipo de geometría son `Point` o `PointZ`. Los tres argumentos son opcionales y si alguno de ellos no se especifica para una columna, se permiten valores de cualquier subtipo, dimensionalidad y/o SRID.

```
SELECT asEWKT(tpose(Sequence,5676) 'SRID=5676;
  [Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-03]
SELECT tpose(Sequence) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
SELECT tpose(PointZ) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- Column has Z dimension but the tpose value does not
SELECT tpose(5676) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR:  Temporal pose SRID (0) does not match column SRID (5676)
SELECT asEWKT(tposechain(Sequence,5676) 'SRID=5676;
  [PoseChain(Pose(Point(1 1), 0.2))@2001-01-01,
  PoseChain(Pose(Point(1 1), 0.5))@2001-01-03]');
/* SRID=5676;[PoseChain(Pose(POINT(1 1),0.2))@2001-01-01,
   PoseChain(Pose(POINT(1 1),0.5))@2001-01-03] */
```

Los valores de pose temporal del subtipo de secuencia o conjunto de secuencias se convierten en una forma normal para que los valores equivalentes tengan representaciones idénticas. Para ello, los valores instantáneos consecutivos se fusionan cuando es posible. Tres valores instantáneos consecutivos se pueden fusionar en dos si las funciones lineales que definen la evolución de los valores son las mismas. Los ejemplos de transformación a una forma normal son los siguientes.

```
SELECT asText(tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(2 2), 0.4)@2001-01-02,
  Pose(Point(3 3), 0.6)@2001-01-03)');
-- [Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(3 3),0.6)@2001-01-03)
SELECT asText(tpose '{[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(2 2), 0.3)@2001-01-02,
  Pose(Point(2 2), 0.5)@2001-01-03),
  [Pose(Point(2 2), 0.5)@2001-01-03, Pose(Point(2 2), 0.7)@2001-01-04)}');
/* {[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(2 2),0.3)@2001-01-02,
   Pose(POINT(2 2),0.7)@2001-01-04)} */
```

13.5. Validez de las poses temporales y de las cadenas de poses

Los valores de pose temporal deben satisfacer las restricciones especificadas en la Sección 4.3 para que estén bien definidos. Se genera un error cuando no se cumple una de estas restricciones. Ejemplos de valores incorrectos son los siguientes.

```
-- Null values are not allowed
SELECT tpose 'NULL@2001-01-01 08:05:00';
SELECT tpose 'Point(0 0)@NULL';
-- Base type is not a pose
SELECT tpose 'Point(0 0)@2001-01-01 08:05:00';
```

A continuación damos las funciones y operadores para las poses temporales. La mayoría de las funciones y operadores para tipos temporales y tipos espaciotemporales descritos en los capítulos precedentes se pueden aplicar para los tipos de pose temporal. Por lo tanto, en las firmas de las funciones, el tipo `pose` se puede utilizar siempre que se especifiquen los tipos `base` y `spatial`, y el tipo `tpose` se puede utilizar siempre que se especifiquen los tipos `ttype` y `tspatial`. Para evitar la redundancia, a continuación solo presentamos algunos ejemplos de estas funciones y operadores para poses temporales y nos remitimos a los capítulos precedentes para explicaciones detalladas sobre ellos.

13.6. Entrada y salida

- Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)

```
asText({tpose,tposechain,tpose[],tposechain[]}) → {text,text[]}
asEWKT({tpose,tposechain,tpose[],tposechain[]}) → {text,text[]}
```

```
SELECT asText(tpose 'SRID=4326;
  [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)');
-- [Pose(POINT(0 0),1)@2001-01-01, Pose(POINT(1 1),2)@2001-01-02)
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(POINT(0 0),1)","Pose(POINT(1 1),2)"}
SELECT asEWKT(tpose 'SRID=4326;
  [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)');
-- SRID=4326;[Pose(POINT(0 0),1)@2001-01-01, Pose(POINT(1 1),2)@2001-01-02)
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)', 'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2))"}
```

- Devuelve la representación JSON de características móviles (MF-JSON)

```
asMFJSON({tpose,tposechain}) → text
```

Una cadena se escribe como el arreglo de sus eslabones, en el orden que indica en el marco de quién se lee cada uno de ellos.

```
SELECT asMFJSON(tpose 'Pose(Point(1 2),0.5)@2001-01-01', 3);
/* {"type":"MovingPose","bbox":[[1,2],[1,2]],"period":{"begin":"2001-01-01T00:00:00",
   "end":"2001-01-01T00:00:00","lower_inc":true,"upper_inc":true},
   "values":[{"position":{"lat":2,"lon":1},"yaw":0.5}],
   "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
SELECT asMFJSON(tpose 'Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3);
/* {"type":"MovingPose","bbox":[[1,2,3],[1,2,3]],
   "period":{"begin":"2001-01-01T00:00:00","end":"2001-01-01T00:00:00",
   "lower_inc":true,"upper_inc":true},
   "values":[{"position":{"lat":2,"lon":1,"h":3},"quaternion":{"x":0,"y":0,"z":0,"w":1}}],
   "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
```

- Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
asBinary({tpose,tposechain},endian text='') → bytea
asEWKB({tpose,tposechain},endian text='') → bytea
asHexWKB({tpose,tposechain},endian text='') → text
asHexEWKB({tpose,tposechain},endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tpose 'Pose(Point(1 2),1)@2001-01-01');
-- \x013800010100000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
SELECT asEWKB(tpose 'SRID=7844;Pose(Point(1 2),1)@2001-01-01');
-- \x01380041a41e00000100000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
SELECT asHexWKB(tpose 'Pose(Point(1 2),1)@2001-01-01');
-- 013800010100000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
SELECT asHexEWKB(tpose 'SRID=3812;Pose(Point(1 2),1)@2001-01-01');
-- 01380041E40E00000100000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
```

- Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)

```
tposeFromText(text) → tpose
tposeFromEWKT(text) → tpose
tposechainFromText(text) → tposechain
```

Estas funciones leen solo la forma de texto temporal, como la entrada del tipo `tpose '...'`. Como en el texto de todo tipo temporal, una llave inicial abre una secuencia discreta o un conjunto de secuencias, y un valor dentro de ella puede ir entre comillas. Un documento GeoPose se lee con tposeFromGeoPose.

```
SELECT asEWKT(tposeFromText(text '[Pose(Point(1 2),1)@2001-01-01,
  Pose(Point(3 4),2)@2001-01-02]'));
-- [Pose(POINT(1 2),1)@2001-01-01, Pose(POINT(3 4),2)@2001-01-02]
SELECT asEWKT(tposeFromEWKT(text 'SRID=3812;
  [Pose(Point(1 2),1)@2001-01-01, Pose(Point(3 4),2)@2001-01-02]'));
-- SRID=3812;[Pose(POINT(1 2),1)@2001-01-01, Pose(POINT(3 4),2)@2001-01-02]
SELECT asEWKT(tposechainFromText('SRID=5676;PoseChain(Pose(Point(1 1),0.5))@2001-01-01'));
-- SRID=5676;PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
SELECT asEWKT(tposechainFromBinary(asBinary(tposechain
  'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01')));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
SELECT asEWKT(tposechainFromHexEWKB(asHexEWKB(tposechain 'SRID=5676;
  PoseChain(Pose(Point(1 1), 0.5))@2001-01-01')));
-- SRID=5676;PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
```

- Entrada desde la representación JSON de características móviles (MF-JSON)

```
tposeFromMFJSON(text) → tpose
tposechainFromMFJSON(text) → tposechain
```

Una cadena de poses se escribe como el arreglo de sus eslabones, de modo que un documento `MovingPoseChain` contiene un arreglo de objetos pose para cada instante

```
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose 'SRID=3812;Pose(Point(1 2),0.5)@2001-01-01',
  3)));
-- SRID=3812;Pose(POINT(1 2),0.5)@2001-01-01
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose 'SRID=5676;
  Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3)));
-- SRID=5676;Pose(POINT Z(1 2 3),1,0,0,0)@2001-01-01
SELECT asEWKT(tposechainFromMFJSON(asMFJSON(tposechain 'SRID=3812;
  PoseChain(Pose(Point(1 2),0.5),Pose(Point(3 4),0.25))@2001-01-01', 3)));
-- SRID=3812;PoseChain(Pose(POINT(1 2),0.5),Pose(POINT(3 4),0.25))@2001-01-01
```

- Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

```
tposeFromBinary(bytea) → tpose
tposeFromEWKB(bytea) → tpose
tposeFromHexWKB(text) → tpose
tposeFromHexEWKB(text) → tpose
tposechainFromBinary(bytea) → tposechain
tposechainFromHexWKB(text) → tposechain
tposechainFromHexEWKB(text) → tposechain
```

```
SELECT asEWKT(tposeFromBinary(
  '\x013a0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- Pose(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tposeFromEWKB(
  '\x013a0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- SRID=7844;Pose(POINT(1 1),2)@2001-01-01
SELECT asEWKT(tposeFromHexEWKB(
  '013A0041E40E0000E40E00000000000000000F03F...40000000000000F03F009C57D3C11C0000'));
-- SRID=3812;Pose(POINT(1 2),1)@2001-01-01
```

13.7. Constructores

- Constructor para poses temporales con valor constante

```
tpose(pose,timestamptz) → tposeInst
tpose(pose,tstzset) → tposeDiscSeq
tpose(pose,tstzspan,interp text='linear') → tposeContSeq
tpose(pose,tstzspanset,interp text='linear') → tposeSeqSet
tposechain(posechain,{timestamptz,tstzset}) → tposechain
tposechain(posechain,{tstzspan,tstzspanset},interp text='linear') → tposechain
```

```
SELECT asText(tpose('Pose(Point(1 1), 0.5)', timestamptz '2001-01-01'));
-- Pose(POINT(1 1),0.5)@2001-01-01
SELECT asText(tpose('Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)',
  tstzset '{2001-01-01, 2001-01-05}'));
/* {Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)@2001-01-01,
   Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)@2001-01-05} */
SELECT asText(tpose('Pose(Point(1 1), 0.5)', tstzspan '[2001-01-01, 2001-01-02]'));
-- [Pose(POINT(1 1),0.5)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-02]
SELECT asText(tpose('Pose(Point(1 1), 0.2)', tstzspanset '{[2001-01-01, 2001-01-03]}',
  'step'));
-- Interp=Step;{[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.2)@2001-01-03]}
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  timestamptz '2001-01-01'));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  tstzset '{2001-01-01, 2001-01-02}'));
/* {PoseChain(Pose(POINT(1 1),0.5))@2001-01-01,
   PoseChain(Pose(POINT(1 1),0.5))@2001-01-02} */
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  tstzspan '[2001-01-01, 2001-01-02]'));
/* [PoseChain(Pose(POINT(1 1),0.5))@2001-01-01,
   PoseChain(Pose(POINT(1 1),0.5))@2001-01-02] */
```

- Constructor para poses temporales de subtipo secuencia

```
tposeSeq(tposeInst[],interp text={'step','linear'},lower_inc boolean=true,
  upper_inc boolean=true) → tposeSeq
```

```
SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.3)@2001-01-01',
  'Pose(Point(2 2), 0.5)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* {Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02,
   Pose(Point(1 1),0.5)@2001-01-03} */
SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.2)@2001-01-01',
  'Pose(Point(1 1), 0.4)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* [Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.4)@2001-01-02,
   Pose(POINT(1 1),0.5)@2001-01-03] */
```

- Constructor para poses temporales de subtipo conjunto de secuencias

```
tposeSeqSet(tpose[]) → tposeSeqSet
tposeSeqSetGaps(tposeInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → tposeSeqSet
```

```
SELECT asText(tposeSeqSet(ARRAY[tpose '[Pose(Point(1 1),0.2)@2001-01-01,
  Pose(Point(2 2),0.4)@2001-01-02]',
  '[Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.8)@2001-01-04]']));
/* {[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.4)@2001-01-02],
   [Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.6)@2001-01-04]} */
SELECT asText(tposeSeqSetGaps(ARRAY[tpose 'Pose(Point(1 1),0.1)@2001-01-01',
```

```
  'Pose(Point(1 1),0.3)@2001-01-03', 'Pose(Point(1 1),0.5)@2001-01-05'], '1 day'));
/* {[Pose(Point(1 1),0.1)@2001-01-01], [Pose(Point(1 1),0.3)@2001-01-03],
   [Pose(Point(1 1),0.5)@2001-01-05]} */
```

- Construye una pose temporal 2D a partir de un punto geométrico temporal y un flotante temporal

```
tpose(tgeompoint,tfloat) → tpose
```

El marco temporal del resultado es la intersección de los marcos temporales del punto temporal y el flotante temporal. Si no se intersecan, se devuelve un valor nulo

```
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
  tfloat '[1@2001-01-01, 2@2001-01-02)'));
-- [Pose(POINT(1 1),1)@2001-01-01, Pose(POINT(2 2),2)@2001-01-02)
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
  tfloat '[2@2001-01-03, 3@2001-01-04)'));
-- NULL
```

13.8. Conversión de tipos

Un valor de pose temporal se puede convertir hacia y desde un punto temporal, que es un punto geométrico temporal o un punto geográfico temporal cualquiera que sea el tipo de la pose, véase [?listitem]. Esto se puede hacer utilizando un `CAST` explícito o utilizando la notación `::`. Convertir una pose temporal al tipo de punto temporal del otro marco de referencia genera un error. Se devuelve un valor nulo si alguno de los valores de punto geométrico que lo componen no se puede convertir en un valor `pose`.

- Convierte una pose temporal, planar o geodésica, en un punto geométrico temporal

```
{tpose,tposechain}::tgeompoint
```

```
SELECT asText((tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.3)@2001-01-02)')::tgeompoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
SELECT asEWKT((tpose '[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
  Pose(Point Z(2 2 2), 1, 0, 0, 0)@2001-01-02)')::tgeompoint);
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02)
```

- Convierte una pose temporal, planar o geodésica, en un punto geográfico temporal

```
{tpose,tposechain}::tgeogpoint
```

```
SELECT asText((tpose '[GeodPose(Point(1 1), 0.2)@2001-01-01,
  GeodPose(Point(1 1), 0.3)@2001-01-02)')::tgeogpoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
```

13.9. Accesores

- Devuelve los valores

```
getValues({tpose,tposechain}) → {poseset,posechainset}
```

```
SELECT asText(getValues(tpose '{[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02)}'));
-- {"Pose(POINT(1 1),0.3)","Pose(POINT(1 1),0.5)"}
SELECT asEWKT(getValues(tpose 'SRID=5676;
  {[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.3)@2001-01-02)}'));
-- SRID=5676;{"Pose(POINT(1 1),0.3)"}
```

- Devuelve los puntos

```
points(tpose) → geomset
```

```
SELECT asEWKT(points(tpose 'SRID=5676;
  {Pose(Point(3 3), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02}'));
-- SRID=5676;{POINT(1 1), POINT(3 3)}
```

- Devuelve la trayectoria

```
trajectory(tpose) → geometry
```

Una pose tiene una posición y una orientación, pero no una extensión, por lo que la geometría que cubre a lo largo del tiempo es la trayectoria de su posición, como para un punto temporal. Una pose que interpola se desplaza por la línea entre dos posiciones consecutivas; una que no interpola permanece en cada una de ellas y no cubre nada entre ellas.

```
SELECT ST_AsText(trajectory(tpose '[Pose(Point(1 1),0.5)@2001-01-01,
  Pose(Point(3 3),0.5)@2001-01-02]'));
-- LINESTRING(1 1,3 3)
SELECT ST_AsText(trajectory(tpose 'Interp=Step;
  [Pose(Point(1 1),0.5)@2001-01-01, Pose(Point(3 3),0.5)@2001-01-02]'));
-- MULTIPOINT((1 1),(3 3))
```

- Devuelve el valor en una marca de tiempo

```
valueAtTimestamp(tpose,timestamptz) → pose
```

```
SELECT asText(valueAtTimestamp(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(3 3), 0.5)@2001-01-03)', '2001-01-02'));
-- Pose(POINT(2 2),0.4)
SELECT asText(valueAtTimestamp(tpose
  '[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
  Pose(Point Z(3 3 3), 1, 0, 0, 0)@2001-01-03)', timestamptz '2001-01-02'), 6);
-- Pose(POINT Z (2 2 2),0.866025,0.288675,0.288675,0.288675)
```

- Devuelve la velocidad de una pose temporal como un flotante temporal: la magnitud de la velocidad de la componente de posición (distancia recorrida por unidad de tiempo)

```
speed(tpose) → tfloat
```

La orientación no es considerada; para la velocidad angular véase `angularSpeed`.

La respuesta se da en las unidades del CRS del SRID, que multiplicadas por 3600 en este caso corresponden a metros por hora.

```
SELECT asText(speed(tpose '[Pose(Point(0 0), 0)@2001-01-01 08:00,
  Pose(Point(1 1), 0.5)@2001-01-02 08:00]') * 3600);
-- Interp=Step;[1.4142135623730950@2001-01-01, 1.4142135623730950@2001-01-02]
```

- Devuelve la velocidad angular de una pose temporal como un flotante temporal con interpolación por pasos (radianes por unidad de tiempo)

```
angularSpeed(tpose) → tfloat
```

Para las poses 2D es el delta theta del arco más corto por segmento dividido por la duración del segmento; para las poses 3D es el ángulo de arco de SLERP `2 · acos(|q1 · q2|)` dividido por la duración del segmento. SLERP tiene por construcción velocidad angular constante a lo largo de un segmento, por lo que el resultado es constante a trozos.

```
-- 90-degree yaw rotation over one day -> pi/2 rad / 86400 s
SELECT asText(angularSpeed(tpose '[Pose(Point(0 0 0), 1, 0, 0, 0)@2001-01-01,
  Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)@2001-01-02]'));
-- Interp=Step;[1.8181e-5@2001-01-01, 1.8181e-5@2001-01-02]
```

- Devuelve el valor de un instante temporal, el conjunto de valores o el lapso de tiempo

```
getValue({tpose,tposechain}) → {pose,posechain}
timeSpan({tpose,tposechain}) → tstzspan
```

```
SELECT asEWKT(getValue(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01'));
-- PoseChain(Pose(POINT(1 1),0.5))
SELECT asEWKT(getValues(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}'));
-- {"PoseChain(Pose(POINT(1 1),0.5))", "PoseChain(Pose(POINT(2 2),0.5))"}
SELECT timeSpan(tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02]');
-- [2001-01-01, 2001-01-02]
```

13.10. Transformaciones

- Transforma a otro subtipo

```
tposeInst(tpose) → tposeInst
tposeSeq(tpose) → tposeSeq
tposeSeqSet(tpose) → tposeSeqSet
```

```
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Pose(POINT(1 1),0.5)@2001-01-01}
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01'));
-- [Pose(POINT(1 1),0.5)@2001-01-01]
SELECT asText(tposeSeqSet(tpose 'Pose(PointZ(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01'));
-- {[Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)@2001-01-01]}
```

- Transforma a otra interpolación

```
setInterp(tpose,interp text) → tpose
```

```
SELECT asText(setInterp(tpose 'Pose(Point(1 1),0.2)@2001-01-01','linear'));
-- [Pose(POINT(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tpose '{[Pose(Point Z(1 1 1),1,0,0,0)@2001-01-01],
  [Pose(Point Z(2 2 2),0,0,0,1)@2001-01-02]}', 'discrete'));
-- {Pose(POINT Z (1 1 1),1,0,0,0)@2001-01-01, Pose(POINT Z (2 2 2),0,0,0,1)@2001-01-02}
```

- Redondea los puntos y las orientaciones de la pose temporal al número de posiciones decimales

```
round({tpose,tposechain},integer=0) → {tpose,tposechain}
shiftTime({tpose,tposechain},interval) → {tpose,tposechain}
```

```
SELECT asText(round(tpose '{[Pose(Point(1 1.123456789),0.123456789)@2001-01-01,
  Pose(Point(1 1),0.5)@2001-01-02)}', 3));
-- {[Pose(POINT(1 1.123),0.123)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-02)}
SELECT asEWKT(round(tposechain
  'PoseChain(Pose(Point(1.123456789 2.123456789), 0.5))@2001-01-01', 3));
-- PoseChain(Pose(POINT(1.123 2.123),0.5))@2001-01-01
SELECT asEWKT(shiftTime(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01',
  interval '1 day'));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-02
```

- Devuelve un valor temporal trasladado por los desplazamientos dados

```
translate({tpose,tposechain},deltax float,deltay float,deltaz float=0)
  → {tpose,tposechain}
```

Un desplazamiento vertical saca un valor bidimensional de su plano, por lo que el resultado es tridimensional.

La cadena de cada instante mueve su primer eslabón, que se expresa en el marco exterior de la cadena, y conserva los demás.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(translate(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 1, 2), 6);
-- [Pose(POINT(2 3),0.3)@2001-01-01, Pose(POINT(4 4),1.1)@2001-01-02]
SELECT asText(translate(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 1, 2, 3), 6);
/* [Pose(POINT Z (2 3 3),0.988771,0,0,0.149438)@2001-01-01,
   Pose(POINT Z (4 4 3),0.852525,0,0,0.522687)@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del origen o de un punto

```
rotate({tpose,tposechain},angle float) → {tpose,tposechain}
rotate({tpose,tposechain},angle float,x0 float,y0 float) → {tpose,tposechain}
```

La cadena de cada instante mueve su primer eslabón, que se expresa en el marco exterior de la cadena, y conserva los demás.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotate(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7, 1, 1), 6);
-- [Pose(POINT(1 1),1)@2001-01-01, Pose(POINT(1.885467 3.053278),1.8)@2001-01-02]
SELECT asText(rotate(tposechain '[PoseChain(Pose(Point(1 1),0.3),Pose(Point(2 0),0.5)) ↩
    @2001-01-01,
  PoseChain(Pose(Point(3 2),1.1),Pose(Point(2 0),0.5))@2001-01-02]', 0.7, 1, 1), 6);
/* [PoseChain(Pose(POINT(1 1),1),Pose(POINT(2 0),0.5))@2001-01-01,
   PoseChain(Pose(POINT(1.885467 3.053278),1.8),Pose(POINT(2 0),0.5))@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del eje x

```
rotateX({tpose,tposechain},angle float) → {tpose,tposechain}
```

La rotación saca un valor bidimensional de su plano, por lo que el resultado es tridimensional: cada posición gana la altura que le da la rotación y cada orientación es la rotación compuesta con el ángulo alrededor de la vertical que indica el valor.

La cadena de cada instante mueve su primer eslabón, que se expresa en el marco exterior de la cadena, y conserva los demás.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotateX(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* [Pose(POINT Z (1 0.764842 0.644218),0.928825,0.339047,-0.051242,0.140378)@2001-01-01,
   Pose(POINT Z (3 1.529684 1.288435),0.800838,0.292329,-0.179228,0.490998)@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del eje y

```
rotateY({tpose,tposechain},angle float) → {tpose,tposechain}
```

La rotación saca un valor bidimensional de su plano, por lo que el resultado es tridimensional: cada posición gana la altura que le da la rotación y cada orientación es la rotación compuesta con el ángulo alrededor de la vertical que indica el valor.

La cadena de cada instante mueve su primer eslabón, que se expresa en el marco exterior de la cadena, y conserva los demás.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotateY(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* [Pose(POINT Z (0.764842 1 -0.644218),0.928825,0.051242,0.339047,0.140378)@2001-01-01,
   Pose(POINT Z (2.294527 2 -1.932653),0.800838,0.179228,0.292329,0.490998)@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del eje z

```
rotateZ({tpose,tposechain},angle float) → {tpose,tposechain}
```

La cadena de cada instante mueve su primer eslabón, que se expresa en el marco exterior de la cadena, y conserva los demás. El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotateZ(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* [Pose(POINT(0.120625 1.40906),1)@2001-01-01,
   Pose(POINT(1.006091 3.462337),1.8)@2001-01-02] */
```

- Devuelve una pose temporal proyectada sobre el suelo

```
force2D(tpose) → tpose
```

Cada instante conserva la x y la y de su posición y la guiñada de su orientación. La rotación de un segmento en tres dimensiones toma el arco más corto entre dos cuaterniones, de modo que el resultado coincide con el valor en los instantes, y entre ellos donde la rotación es alrededor de la vertical. Un valor en dos dimensiones queda como está.

```
SELECT asText(force2D(tpose '[Pose(Point(1 1 5),0.7071067811865476,0,0,0.7071067811865476) ↩
    @2001-01-01,
  Pose(Point(3 2 7),1,0,0,0)@2001-01-02]'), 6);
-- [Pose(POINT(1 1),1.570796)@2001-01-01, Pose(POINT(3 2),0)@2001-01-02]
```

13.11. Modificaciones

- Agrega un instante temporal o una secuencia temporal

```
appendInstant({tpose,tposechain},{tposeInst,tposechainInst}) → {tpose,tposechain}
appendSequence({tpose,tposechain},{tposeSeq,tposechainSeq}) → {tpose,tposechain}
```

```
SELECT asText(appendInstant(tpose 'Pose(Point(1 1), 0.5)@2001-01-01',
  'Pose(Point(2 2), 1)@2001-01-02'));
-- [Pose(POINT(1 1),0.5)@2001-01-01, Pose(POINT(2 2),1)@2001-01-02]
SELECT asText(appendSequence(tpose '[Pose(Point(1 1), 0.5)@2001-01-01]',
  '[Pose(Point(2 2), 1)@2001-01-02]'));
-- {[Pose(POINT(1 1),0.5)@2001-01-01], [Pose(POINT(2 2),1)@2001-01-02]}
```

- Fusiona las poses temporales

```
merge({tpose,tposechain},{tpose,tposechain}) → {tpose,tposechain}
merge({tpose,tposechain}[]) → {tpose,tposechain}
merge({tpose,tposechain}) → {tpose,tposechain}
```

El nombre `merge` designa también una agregación, que fusiona los valores de un conjunto de filas.

```
SELECT asText(merge(tpose '[Pose(Point(1 1 1),1,0,0,0)@2001-01-01,
  Pose(Point(2 2 2),0,1,0,0)@2001-01-02]',
  '[Pose(Point(2 2 2),0,1,0,0)@2001-01-02, Pose(Point(3 3 3),0,0,0,1)@2001-01-03]'));
/* [Pose(POINT Z (1 1 1),1,0,0,0)@2001-01-01, Pose(POINT Z (2 2 2),0,1,0,0)@2001-01-02,
   Pose(POINT Z (3 3 3),0,0,0,1)@2001-01-03] */
SELECT asText(merge(ARRAY[tpose '{[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(2 2),0.2)@2001-01-02],
  [Pose(Point(3 3),0.3)@2001-01-03, Pose(Point(4 4),0.4)@2001-01-04]}',
  '[Pose(Point(4 4),0.4)@2001-01-04, Pose(Point(5 5),0.5)@2001-01-05]']));
/* {[Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.2)@2001-01-02],
   [Pose(POINT(3 3),0.3)@2001-01-03, Pose(POINT(5 5),0.5)@2001-01-05]} */
```

```
WITH temp(inst) AS (
  SELECT tpose 'Pose(Point(1 1),0.1)@2001-01-01' UNION
  SELECT tpose 'Pose(Point(2 2),0.2)@2001-01-02' UNION
  SELECT tpose 'Pose(Point(3 3),0.3)@2001-01-03' )
SELECT asText(merge(inst)) FROM temp;
/* {Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.2)@2001-01-02,
   Pose(POINT(3 3),0.3)@2001-01-03} */
SELECT asText(merge(temp)) FROM (VALUES
  (tposechain 'PoseChain(Pose(Point(1 1), 0.1))@2001-01-01'),
  (tposechain 'PoseChain(Pose(Point(2 2), 0.2))@2001-01-02')) t(temp);
/* {PoseChain(Pose(POINT(1 1),0.1))@2001-01-01,
   PoseChain(Pose(POINT(2 2),0.2))@2001-01-02} */
```

- Inserta o actualizar la segunda cadena de poses temporal en la primera, o eliminar de ella un valor de tiempo

```
insert({tpose,tposechain},{tpose,tposechain},connect boolean=true) → {tpose,tposechain}
update({tpose,tposechain},{tpose,tposechain},connect boolean=true) → {tpose,tposechain}
deleteTime({tpose,tposechain},times,connect boolean=true) → {tpose,tposechain}
```

```
SELECT asText(deleteTime(tposechain
  '{PoseChain(Pose(Point(1 1),0.5), Pose(Point(2 0),0))@2001-01-01,
  PoseChain(Pose(Point(3 3),0.5), Pose(Point(4 0),0))@2001-01-02}',
  timestamptz '2001-01-02'));
-- {PoseChain(Pose(POINT(1 1),0.5),Pose(POINT(2 0),0))@2001-01-01}
```

13.12. Restricciones

- Restringe al (complemento de) un valor o un conjunto de valores

```
atValue({tpose,tposechain},{pose,posechain}) → {tpose,tposechain}
minusValue({tpose,tposechain},{pose,posechain}) → {tpose,tposechain}
atValues({tpose,tposechain},{poseset,posechainset}) → {tpose,tposechain}
minusValues({tpose,tposechain},{poseset,posechainset}) → {tpose,tposechain}
```

```
SELECT atValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
-- {[Pose(POINT(2 2),0.5)@2001-01-02]}
SELECT minusValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
/* {[Pose(Point(2 2),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02),
   (Pose(Point(2 2),0.5)@2001-01-02, Pose(Point(2 2),0.7)@2001-01-03]} */
```

- Restringe al (complemento de) una geometría

```
atGeometry(tpose,geometry) → tpose
minusGeometry(tpose,geometry) → tpose
```

```
SELECT atGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Polygon((40 40,40 50,50 50,50 40,40 40))');
/* {(Pose(Point(2 2),0.342593)@2001-01-01 05:06:40.364673,
   Pose(Point(2 2),0.7)@2001-01-03]} */
```

- Restringe al (complemento de) un span de elevación

```
atElevation(tpose,floatspan) → tpose
minusElevation(tpose,floatspan) → tpose
```

La elevación de una pose es la elevación de su posición, por lo que la restricción conserva los tiempos en los que esa posición se encuentra dentro del span. La pose temporal debe tener dimensión Z.

```
SELECT asText(atElevation(tpose '[Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
  Pose(Point(0 0 4),1,0,0,0)@2001-01-05]', floatspan '[1, 2]'));
/* {[Pose(POINT Z (0 0 1),1,0,0,0)@2001-01-02,
   Pose(POINT Z (0 0 2),1,0,0,0)@2001-01-03]} */
SELECT asText(minusElevation(tpose '[Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
  Pose(Point(0 0 4),1,0,0,0)@2001-01-05]', floatspan '[1, 2]'));
/* {[Pose(POINT Z (0 0 0),1,0,0,0)@2001-01-01, Pose(POINT Z (0 0 1),1,0,0,0)@2001-01-02),
   (Pose(POINT Z (0 0 2),1,0,0,0)@2001-01-03,
   Pose(POINT Z (0 0 4),1,0,0,0)@2001-01-05]} */
```

- Restringe una cadena de poses temporal a un tiempo o a un valor

```
atTime({tpose,tposechain},times) → {tpose,tposechain}
```

```
SELECT asEWKT(atTime(tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-03]', timestamptz '2001-01-02'));
-- PoseChain(Pose(POINT(1.5 1.5),0.5))@2001-01-02
SELECT asEWKT(atValue(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}',
  posechain 'PoseChain(Pose(Point(2 2), 0.5))'));
-- {PoseChain(Pose(POINT(2 2),0.5))@2001-01-02}
```

13.13. Sistema de referencia espacial

- Devuelve o establece el identificador de referencia espacial, que para una cadena de poses temporal es el identificador de su marco exterior

```
SRID({tpose,tposechain}) → integer
setSRID({tpose,tposechain},integer) → {tpose,tposechain}
```

```
SELECT SRID(tpose 'SRID=5676;
  [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)'));
-- 5676
SELECT asEWKT(setSRID(tpose '[Pose(Point(0 0),1)@2001-01-01,
  Pose(Point(1 1),2)@2001-01-02)', 5676));
-- SRID=5676;[Pose(POINT(0 0),1)@2001-01-01, Pose(POINT(1 1),2)@2001-01-02)
```

- Transforma a un identificador de referencia espacial

```
transform({tpose,tposechain},integer) → {tpose,tposechain}
transformPipeline({tpose,tposechain},pipeline text,srid integer,
  is_forward boolean=true) → {tpose,tposechain}
```

La cadena de cada instante se transforma como se transforma la cadena estática correspondiente, es decir, solo se transforma el eslabón exterior, tal como se describe para transform.

La pose de cada instante se transforma igual que la pose estática correspondiente, con la corrección de orientación incluida, como se describe para transform. Lo mismo ocurre con un `poseset`. Una `trgeometry` asocia sus poses con una geometría de referencia que comparte su marco, y la transformación a otro SRID no está admitida para ese tipo.

```
SELECT asEWKT(transform(tpose 'SRID=4326;Pose(Point(4.35 50.85),1)@2001-01-01', 3812));
-- SRID=3812;Pose(POINT(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tpose, pipeline) AS (
  SELECT tpose 'SRID=4326;{Pose(Point(4.3525 50.846667),1)@2001-01-01,
    Pose(Point(-0.1275 51.507222),2)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(tpose, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
/* SRID=4326;{Pose(POINT(4.3525 50.846667),1)@2001-01-01,
   Pose(POINT(-0.1275 51.507222),2)@2001-01-02} */
```

13.14. Operaciones de cuadro delimitador

- Operadores topológicos

```
{tstzspan,stbox,tpose,tposechain} {&&, <@, @>, ~=, -|-}
  {tstzspan,stbox,tpose,tposechain} → boolean
```

```
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' && tstzspan '[2001-01-02,2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' @> stbox(pose 'Pose(Point(1 1), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' ~= tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.35)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true
```

- Operadores de posición

```
{stbox,tpose} {<<, &<, >>, &>} {stbox,tpose}  → boolean
{stbox,tpose} {<<|, &<|, |>>, |&>} {stbox,tpose}  → boolean
{stbox,tpose} {<</, &</, />>, /&>} {stbox,tpose}  → boolean
{tstzspan,stbox,tpose} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tpose} → boolean
```

```
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' << stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' <<| stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1 1), 1,0,0,0)@2001-01-01,
  Pose(Point(1 1 1), 1,0,0,0)@2001-01-02]' <</ stbox(pose 'Pose(Point(2 2 2), 1,0,0,0)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03, Pose(Point(1 1), 0.5)@2001-01-05]' #>>
  tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02]';
-- true
```

- Devuelve la caja delimitadora espaciotemporal expandida en la dimensión espacial, o la matriz de cajas que cubren la cadena de poses temporal

```
expandSpace({tpose,tposechain},float) → stbox
stboxes({tpose,tposechain}) → stbox[]
```

```
SELECT expandSpace(tposechain
  '[PoseChain(Pose(Point(1 1),0.5), Pose(Point(2 0),0))@2001-01-01,
  PoseChain(Pose(Point(3 3),0.5), Pose(Point(4 0),0))@2001-01-03]', 2);
/* STBOX XT(((-1,-1),(8.510330247561491,6.917702154416812)),
   [2001-01-01, 2001-01-03]) */
```

13.15. Operaciones de distancia

Los operandos de dimensiones distintas se leen en las dimensiones que ambos tienen, véase Sección 8.3.

A continuación presentamos las funciones que calculan operaciones de distancia para poses temporales y cadenas de poses temporales. Tenga en cuenta que para estas operaciones solo se considera el componente de punto, no la orientación. Una cadena de poses temporal se mide por la pose temporal que da su composición, `tpose(tposechain)`, que es una función escalonada para una cadena de más de un eslabón.

- Devuelve la distancia más pequeña alguna vez

```
{geo,pose,tpose,stbox} |=| {geo,pose,tpose,stbox} → float
{geo,posechain,tposechain,stbox} |=| {geo,posechain,tposechain,stbox} → float
```

```
SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]' |=| geometry 'Linestring(50 50,55 55)';
-- 67.88225099390856
SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]' |=| pose 'Pose(Point(1 1), 0.5)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' |=| pose 'Pose(Point(2 2), 0.2)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' |=| geometry 'Linestring(2 2,2 1,3 1)';
-- 1
```

El operador `|=|` se puede utilizar para realizar búsquedas del vecino más cercano utilizando un índice GiST o SP-GiST (ver Sección 10.2).

- Devuelve el instante de la primera pose temporal en el que los dos argumentos están a la distancia más cercana

```
nearestApproachInstant({geo,pose,tpose},{geo,pose,tpose}) → tpose
nearestApproachInstant({geo,posechain,tposechain},{geo,posechain,tposechain}) → ↩
    tposechain
```

```
SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- Pose(POINT(2 2),0.3)@2001-01-01
SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- Pose(POINT(2 2),0.3)@2001-01-01
```

- Devuelve la línea que conecta el punto de aproximación más cercano

```
shortestLine({geo,pose,tpose},{geo,pose,tpose}) → geometry
shortestLine({geo,posechain,tposechain},{geo,posechain,tposechain}) → geometry
```

La función devuelve la primera línea que encuentre si hay más de una.

```
SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(2 2,50 50)
SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- LINESTRING(2 2,1 1)
```

- Devuelve la distancia temporal

```
tDistance({geompoint,geogpoint,pose,tpose},{geompoint,geogpoint,pose,tpose}) → tfloat
tDistance({geompoint,geogpoint,posechain,tposechain},{geompoint,geogpoint,posechain, ↩
    tposechain}) → tfloat
{geompoint,geogpoint,pose,tpose} <-> {geompoint,geogpoint,pose,tpose} → tfloat
{geompoint,geogpoint,posechain,tposechain} <-> {geompoint,geogpoint,posechain,tposechain}  ↩
    → tfloat
```

```
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <-> pose 'Pose(Point(2 2), 0.2)', 6);
-- [1.414214@2001-01-01, 1.414214@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)', 6);
-- [69.296465@2001-01-01, 69.296465@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.5)@2001-01-03]' <->
  tpose '[Pose(Point(1 2), 0.3)@2001-01-02, Pose(Point(2 1), 0.5)@2001-01-04]', 6);
-- [0.707107@2001-01-02, 0.5@2001-01-02 12:00:00, 0.707107@2001-01-03]
SELECT asText(round(tposechain '[PoseChain(Pose(Point(0 0),0), Pose(Point(1 0),0))@2001 ↩
    -01-01,
  PoseChain(Pose(Point(4 0),0), Pose(Point(1 0),0))@2001-01-05]' <->
  geometry 'Point(1 0)', 6));
-- Interp=Step;[0@2001-01-01, 4@2001-01-05]
```

13.16. Relaciones siempre y alguna vez

- Relaciones alguna vez y siempre

```
eContains(geometry,{tpose,tposechain}) → boolean
eCovers(geometry,{tpose,tposechain}) → boolean
eDisjoint({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
eDwithin({geo,tpose,tposechain},{geo,tpose,tposechain},float) → boolean
eIntersects({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
eTouches(geometry,{tpose,tposechain}) → boolean
eTouches({tpose,tposechain},geometry) → boolean
aContains(geometry,{tpose,tposechain}) → boolean
aCovers(geometry,{tpose,tposechain}) → boolean
aDisjoint({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
aDwithin({geometry,tpose,tposechain},{geometry,tpose,tposechain},float) → boolean
aIntersects({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
aTouches(geometry,{tpose,tposechain}) → boolean
aTouches({tpose,tposechain},geometry) → boolean
```

Las relaciones *alguna vez* y *siempre* determinan si la relación topológica o de distancia se satisface alguna vez o siempre y dan como resultado un `boolean`. Una pose temporal las responde en la posición que ocupa, de modo que un valor con interpolación lineal se responde a lo largo de su trayectoria y no solo en sus instantes. La pose temporal declara las relaciones que responde un punto en movimiento: `eContains`, `aContains`, `eCovers` y `aCovers` toman la geometría en primer lugar únicamente, y `eTouches` y `aTouches` no tienen dirección entre dos poses temporales, ya que un punto en movimiento no contiene ni cubre una geometría y dos puntos en movimiento no se tocan. Una cadena de poses temporal las responde en la posición de la pose temporal que da su composición, `tpose(tposechain)`.

```
SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(3 3),0.3)@2001-01-03]');
-- true
SELECT aCovers(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(3 3),0.3)@2001-01-03]');
-- true
SELECT eDisjoint(geometry(pose 'Pose(Point(2 2), 0.0)'),
  tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03)');
-- true
SELECT eIntersects(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]',
  tpose '[Pose(Point(3 3),0.1)@2001-01-01, Pose(Point(1 1),0.1)@2001-01-03]');
-- true
SELECT eTouches(tpose 'Pose(Point(0 0), 0.1)@2001-01-01',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- true
SELECT aDwithin(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]', geometry 'Point(9 9)', 1.0);
-- false
SELECT eIntersects(tposechain 'PoseChain(Pose(Point(1 1),0), Pose(Point(9 -1),0))@2001 ↩
    -01-01',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- false
```

13.17. Relaciones espaciotemporales

- Relaciones espaciotemporales

```
tContains(geometry,{tpose,tposechain}) → tbool
tCovers(geometry,{tpose,tposechain}) → tbool
tDisjoint({geometry,tpose,tposechain},{geometry,tpose,tposechain}) → tbool
tDwithin({geo,tpose,tposechain},{geo,tpose,tposechain},float) → tbool
tIntersects({geometry,tpose,tposechain},{geometry,tpose,tposechain}) → tbool
tTouches(geometry,{tpose,tposechain}) → tbool
tTouches({tpose,tposechain},geometry) → tbool
```

Las relaciones *temporales* calculan la relación topológica o de distancia en cada instante y dan como resultado un `tbool`. Una pose temporal las responde en la posición que ocupa, de modo que un valor con interpolación lineal se responde a lo largo de su trayectoria y no solo en sus instantes. La pose temporal declara las relaciones que responde un punto en movimiento: `tContains` y `tCovers` toman la geometría en primer lugar únicamente, y `tTouches` no tiene dirección entre dos poses temporales, ya que un punto en movimiento no contiene ni cubre una geometría y dos puntos en movimiento no se tocan.

```
SELECT tIntersects(tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(3 3), 0.1)@2001-01-03]', geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- {[t@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03]}
SELECT tDisjoint(tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(3 3), 0.1)@2001-01-03]', geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03]}
SELECT tContains(geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))',
  tpose 'Pose(Point(0 0), 0.1)@2001-01-01');
-- f@2001-01-01
SELECT tCovers(geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))',
  tpose 'Pose(Point(0 0), 0.1)@2001-01-01');
-- t@2001-01-01
SELECT tTouches(tpose 'Pose(Point(0 0), 0.1)@2001-01-01',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-01
SELECT tDwithin(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
```

```
  Pose(Point(1 1),0.5)@2001-01-03)',
  tpose '[Pose(Point(1 1),0.5)@2001-01-01, Pose(Point(3 3),0.3)@2001-01-03)', 1);
/* {[t@2001-01-01, t@2001-01-01 16:58:14.025894],
   (f@2001-01-01 16:58:14.025894, f@2001-01-03)} */
```

13.18. Comparaciones

- Comparaciones tradicionales

```
tpose {=, <>, <, >, <=, >=} tpose → boolean
```

```
SELECT tpose '{[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.3)@2001-01-02),
  [Pose(Point(1 1), 0.3)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]}' =
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tpose '{[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]}' <>
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]' <
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.6)@2001-01-03]';
-- true
```

- Comparaciones alguna vez y siempre

```
tpose {?=, ?<>, %=, %<>} tpose → boolean
```

```
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.4)@2001-01-04)' ?= Pose(Point(1 1), 0.3);
-- true
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.2)@2001-01-04)' &= Pose(Point(1 1), 0.2);
-- true
```

- Comparaciones temporales

```
tpose {#=, #<>} tpose → tbool
```

```
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.4)@2001-01-03)' #= pose 'Pose(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.8)@2001-01-03)' #<>
  tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.7)@2001-01-03)';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```

- Compara dos cadenas de poses temporales, o una cadena de poses temporal y una cadena de poses

```
eEqual({pose,posechain,tpose,tposechain},{pose,posechain,tpose,tposechain}) → boolean
```

```
SELECT tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01' =
  tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01';
-- t
SELECT eEqual(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}',
  posechain 'PoseChain(Pose(Point(2 2), 0.5))');
-- t
```

13.19. Agregaciones

Las tres funciones de agregación para poses temporales se ilustran a continuación.

- Conteo temporal

```
tCount({tpose,tposechain}) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
    Pose(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
    Pose(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03, Pose(Point(1 1), 0.5)@2001-01-05)' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05)}
```

- Conteo de ventana

```
wCount({tpose,tposechain},interval) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
    Pose(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
    Pose(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03, Pose(Point(1 1), 0.5)@2001-01-05)' )
SELECT wCount(Temp, '1 day') FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
   1@2001-01-06)} */
```

- Extensión del cuadro delimitador

```
extent({tpose,tposechain}) → stbox
```

```
SELECT extent(temp) FROM (VALUES (NULL::tposechain),
  (tposechain 'PoseChain(Pose(Point(1 1), 0.1))@2001-01-01'),
  (tposechain '{PoseChain(Pose(Point(2 2), 0.2))@2001-01-01,
  PoseChain(Pose(Point(3 3), 0.3))@2001-01-02}')) t(temp);
-- STBOX XT(((1,1),(3,3)),[2001-01-01, 2001-01-02])
```

- Agrega los instantes agregados en una secuencia

```
appendInstant({tpose,tposechain}) → {tpose,tposechain}
appendInstant({tpose,tposechain},interp text) → {tpose,tposechain}
appendInstant({tpose,tposechain},interp text,maxdist float,maxt interval) → {tpose, ↩
    tposechain}
appendInstant(tposechain,interp text,maxt interval) → tposechain
```

Las brechas se establecen por una distancia máxima entre instantes consecutivos, siendo la distancia de una cadena de poses la de la pose que da su composición, y por un tiempo máximo.

```
SELECT asText(appendInstant(inst)) FROM (VALUES
  (tposechain 'PoseChain(Pose(Point(1 1), 0.1))@2001-01-01'),
  (tposechain 'PoseChain(Pose(Point(2 2), 0.2))@2001-01-02')) t(inst);
/* [PoseChain(Pose(POINT(1 1),0.1))@2001-01-01,
   PoseChain(Pose(POINT(2 2),0.2))@2001-01-02] */
```

- Agrega las secuencias agregadas en un conjunto de secuencias

```
appendSequence({tpose,tposechain}) → {tpose,tposechain}
```

```
SELECT asText(appendSequence(seq)) FROM (VALUES
  (tposechain '[PoseChain(Pose(Point(1 1), 0.1))@2001-01-01]'),
  (tposechain '[PoseChain(Pose(Point(2 2), 0.2))@2001-01-02]')) t(seq);
/* {[PoseChain(Pose(POINT(1 1),0.1))@2001-01-01],
   [PoseChain(Pose(POINT(2 2),0.2))@2001-01-02]} */
```

13.20. Operaciones espaciales

- Devuelve el punto de geometría temporal dado por los puntos de una pose temporal

```
centroid(tpose) → tgeompoint
```

```
SELECT asText(centroid(tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(3 3), 0.5)@2001-01-03]'));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03]
```

- Devuelve la envolvente convexa de los puntos de una pose temporal

```
convexHull(tpose) → geometry
```

La envolvente se toma sobre las posiciones que ocupa la pose; la orientación no tiene extensión y no participa en ella.

```
SELECT ST_AsText(convexHull(tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(3 1), 0.3)@2001-01-02, Pose(Point(2 3), 0.5)@2001-01-03]'));
-- POLYGON((1 1,2 3,3 1,1 1))
```

13.21. Simplificación

- Devuelve una pose temporal simplificada asegurando que los puntos consecutivos estén separados al menos una cierta distancia o intervalo de tiempo

```
minDistSimplify(tpose,mindist float) → tpose
minTimeDeltaSimplify(tpose,mint interval) → tpose
```

La simplificación opera sobre la trayectoria de los puntos de la pose temporal y preserva la orientación en cada instante que sobrevive. La trayectoria se simplifica en el plano, por lo que una pose geodésica con interpolación lineal genera un error. La simplificación se aplica únicamente a secuencias temporales o conjuntos de secuencias con interpolación lineal. En todos los demás casos, se devuelve una copia del valor dado.

```
SELECT asText(minDistSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 2), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-03]', 2));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-03]
SELECT asText(minTimeDeltaSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 2), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-04]', interval '2 days'));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-04]
```

- Devuelve una pose temporal simplificada mediante el algoritmo de Douglas-Peucker

```
maxDistSimplify(tpose,maxdist float,syncdist boolean=true) → tpose
douglasPeuckerSimplify(tpose,maxdist float,syncdist boolean=true) → tpose
```

El argumento `syncdist` indica si la distancia se mide entre instantes sincronizados; cuando es falso la distancia es la espacial, ignorando el tiempo.

```
SELECT asText(maxDistSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 1), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-03]', 1));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-03]
SELECT asText(douglasPeuckerSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 1), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-03]', 1));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-03]
```

13.22. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tablas de poses temporales y de cadenas de poses temporales, y de conjuntos de poses y de cadenas de poses. Un ejemplo de creación de índice es el siguiente:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador de los valores, que es un `stbox`, como para todos los tipos espaciotemporales; el cuadro de una cadena de poses cubre la posición compuesta de cada una de sus articulaciones. Un conjunto se indexa por el cuadro que almacena, como se describe en Sección 2.12.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en las poses temporales,
- `<<#`, `&<#`, `#&>`, `#>>`, que solo consideran la dimensión temporal en las poses temporales,
- `&&`, `@>`, `<@`, `~=`, `-|-`, y `|=|` , que consideran tantas dimensiones como las que comparten la columna indexada y el argumento de la consulta.

Estos operadores trabajan con cuadros delimitadores, no con los valores completos.

13.23. Soporte de OGC GeoPose v1.0

El tipo `pose` implementa el modelo de datos que subyace al estándar OGC GeoPose v1.0: una `position` más una `orientation` en un marco de referencia conocido, donde la orientación es un cuaternión unitario en la convención de Hamilton o, de forma equivalente, una terna yaw / pitch / roll bajo la convención Tait-Bryan intrínseca ZYX. Esta sección agrupa la superficie SQL que expone esa interoperabilidad: E/S JSON para las clases de conformidad Basic y Advanced del estándar, una función auxiliar de renormalización explícita para quienes ejecutan composiciones largas, y los accesores de ángulos de Euler que esperan los consumidores Basic-YPR.

13.23.1. Entrada y salida JSON

- Convierte hacia o desde la codificación JSON de OGC GeoPose v1.0 (clase de conformidad Basic-Quaternion, Basic-YPR o Advanced)

```
asGeoPose(pose,conformance integer=0,maxdecimaldigits integer=-1) → text
poseFromGeoPose(text) → pose
```

  El argumento `conformance` selecciona la clase de salida: `0` = Basic-Quaternion (predeterminado, sin pérdida), `1` = Basic-YPR (yaw, pitch, roll en grados, Tait-Bryan intrínseco ZYX), `2` = Advanced. El argumento `maxdecimaldigits` es el número de dígitos significativos que se conservan en los números JSON; `-1` utiliza el valor predeterminado sin pérdida de json-c. La función de entrada detecta automáticamente la clase de conformidad a partir de las claves JSON, donde el miembro `frameSpecification` implica Advanced, el miembro `quaternion` implica Basic-Quaternion y el miembro `angles` implica Basic-YPR. Las clases Basic llevan la posición en un miembro `position` y dejan el marco implícito. La clase Advanced no tiene miembro de posición: nombra su marco exterior de forma explícita y la pose se sitúa en el origen de ese marco, de modo que las dos codificaciones de una misma pose se releen iguales. Todas las clases exigen un marco exterior geográfico, por lo que la pose debe ser geodésica; una pose planar se rechaza en el límite de la conversión, al igual que un SRID proyectado.

```
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 0, 6);
/* {"position":{"lat":47,"lon":8,"h":1500},"quaternion":{"x":0,"y":0,"z":0.707107,
   "w":0.707107}} */
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 1, 6);
-- {"position":{"lat":47,"lon":8,"h":1500},"angles":{"yaw":90,"pitch":0,"roll":0}}
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 2, 6);
/* {"frameSpecification":{"authority":"/geopose/1.0","id":"LTP-ENU","parameters":
   "longitude=8&latitude=47&height=1500&crs=EPSG:4979"},
   "quaternion":{"x":0,"y":0,"z":0.707107,"w":0.707107}} */
SELECT asEWKT(poseFromGeoPose( '{"position":{"lat":47,"lon":8,"h":1500},
  "angles":{"yaw":90,"pitch":0,"roll":0}}'), 6);
-- SRID=4326;GeodPose(POINT Z (8 47 1500),0.707107,0,0,0.707107)
```

13.23.2. Renormalización de cuaterniones

- Devuelve una pose cuyo cuaternión de orientación se ha llevado a una norma unitaria

```
poseNormalize(pose) → pose
```

Una pose 2D se devuelve sin cambios, ya que su orientación es un único ángulo. A una pose 3D se le divide el cuaternión por su norma euclidiana, de modo que composiciones largas de SLERP (o cualquier otra vía que acumule deriva de punto flotante en `|q|`) pueden restaurarse al invariante `|q| = 1` que requiere el código de SLERP y de descomposición de Euler.

```
SELECT asText(poseNormalize(pose 'Pose(Point(1 1 1), 0.5, 0.5, 0.5, 0.5)'));
-- Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)
```

- Devuelve la inversa de una pose

```
poseInverse(pose) → pose
```

La inversa invierte la relación entre marcos: donde una pose lleva un valor del marco que nombra al marco en el que ella misma está expresada, la inversa lo lleva de vuelta. Es `R_BA = R_AB^T` y `t_BA = -R_AB^T t_AB`, de modo que componer una pose con su inversa da la identidad. Esto es lo que expresa una posición del mundo en el marco de un vehículo. Una pose sobre un marco geográfico no tiene inversa: el marco que nombraría no es un marco del elipsoide.

```
SELECT asEWKT(round(poseInverse(pose(ST_Point(10,5), pi()/2)), 6));
-- Pose(POINT(-5 10),-1.570796)
SELECT asEWKT(round(applyPose(poseInverse(pose(ST_Point(10,5), pi()/2)),
  pose(ST_Point(10,5), pi()/2)), 6));
-- Pose(POINT(0 0),0)
SELECT asEWKT(round(poseInverse(pose 'Pose(Point(1 0 0), 0, 0, 0, 1)'), 6));
-- Pose(POINT Z (1 0 0),0,0,0,-1)
```

13.23.3. Accesores de ángulos de Euler

- Devuelve el ángulo yaw, pitch o roll de una pose, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose

```
yaw({pose,tpose}) → {float,tfloat}
pitch({pose,tpose}) → {float,tfloat}
roll({pose,tpose}) → {float,tfloat}
```

Para una pose 2D `yaw` devuelve la rotación almacenada `theta`, por convención el yaw del marco del cuerpo, y `pitch` y `roll` devuelven `0`. Para una pose 3D los tres valores provienen de la descomposición Tait-Bryan intrínseca ZYX del cuaternión de orientación. El término `asin` del pitch se acota a `[-1, 1]` para absorber la pequeña deriva numérica que pueden introducir las composiciones largas de cuaterniones. Sobre una pose temporal la interpolación del resultado sigue a la dimensión. Una pose 2D interpola su ángulo almacenado linealmente y ese ángulo es su yaw, de modo que un `tpose` 2D lineal se proyecta

a un `tfloat` lineal. Una pose 3D interpola mediante SLERP, cuya descomposición Tait-Bryan no es lineal en el tiempo, de modo que un `tpose` 3D se proyecta a un `tfloat` de pasos: leerlo entre dos instantes responde el ángulo del instante de la izquierda en lugar de un ángulo que ninguna pose tiene.

```
SELECT yaw(pose 'Pose(Point(1 1), 0.5)');
-- 0.5
SELECT pitch(pose 'Pose(Point(1 1), 0.5)');
-- 0
SELECT roll(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 0
SELECT yaw(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 1.5707963267948966
```

```
SELECT asText(yaw(tpose '[Pose(Point(0 0), 0.0)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]'));
-- [0@2001-01-01, 0.5@2001-01-02]
```

- Devuelve la orientación de una pose como una terna yaw / pitch / roll, en radianes

```
ypr(pose) → ypr
```

Esta es la codificación Basic-YPR de la orientación en OGC GeoPose, la contraparte de quaternion, y devuelve los tres ángulos que yaw, `pitch` y `roll` responden de uno en uno. Al igual que `quaternion`, está definida para ambas dimensiones: una pose 2D gira por su ángulo almacenado y no cabecea ni alabea. Los ángulos están en radianes, mientras que la codificación JSON de GeoPose los escribe en grados.

```
SELECT ypr(pose 'Pose(Point(1 1), 0.5)');
-- (0.5,0,0)
SELECT ypr(pose 'Pose(Point Z(1 1 1), 1, 0, 0, 0)');
-- (0,0,0)
SELECT ypr(pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)');
-- (1.5707963267948966,0,1.5707963267948966)
```

- Eleva los accesores de ángulos de Euler sobre una pose temporal, devolviendo un flotante temporal en radianes

```
yaw(tpose) → tfloat
pitch(tpose) → tfloat
roll(tpose) → tfloat
```

El resultado lleva interpolación de pasos, ya que un ángulo leído de una rotación no varía linealmente entre dos poses.

```
SELECT round(yaw(tpose '[Pose(Point(0 0),0.5)@2001-01-01,
  Pose(Point(1 1),1.5)@2001-01-02]'), 6);
-- Interp=Step;[0.5@2001-01-01, 1.5@2001-01-02]
SELECT round(pitch(tpose 'Pose(Point(0 0 0),1,0,0,0)@2001-01-01'), 6);
-- 0@2001-01-01
```

13.23.4. Registro de metadatos de marcos

El estándar OGC GeoPose v1.0 distingue el *marco exterior* (la referencia global, por ejemplo, WGS-84 geográfico o ECEF) del *marco interior* (el marco del cuerpo cuya orientación es el cuaternión de la pose). Las clases de conformidad Basic exigen WGS-84 geográfico como marco exterior y un marco interior implícito de ejes de cuerpo dextrógiros; la clase Advanced nombra su marco exterior de forma explícita y sitúa la pose en el origen de ese marco.

En MobilityDB, el tipo `pose` codifica el marco exterior de forma implícita mediante su SRID y utiliza el marco interior convencional de ejes de cuerpo dextrógiros. La tabla `geopose_frames` registra esta correspondencia y declara todos los identificadores de marco que un documento puede nombrar: una serie de secuencia compuesta nombra `LTP-ENU` y `RotateTranslate`, y una cadena nombra esos mismos dos marcos como `/Extrinsic/LTP-ENU` e `/Intrinsic/Translate-Rotate`, todos bajo la autoridad `/geopose/1.0`. Las pilas de marcos de la clase Advanced no están admitidas.

```
SELECT frame_id, authority, code, name FROM geopose_frames ORDER BY frame_id;
-- 1 | EPSG | 4326 | WGS-84 geographic (lat/lon/h)
-- 2 | EPSG | 4978 | WGS-84 ECEF (Earth-Centred Earth-Fixed)
-- 3 | OGC | LTP | Local Tangent Plane (East-North-Up)
-- 4 | OGC | BODY | Right-handed body axes (default inner frame)
-- 5 | /geopose/1.0 | LTP-ENU | GeoPose outer frame of a Composite Sequence Series
-- 6 | /geopose/1.0 | RotateTranslate | GeoPose inner frame of a Composite Sequence Series
-- 7 | /geopose/1.0 | /Extrinsic/LTP-ENU | GeoPose outer frame of a Chain
-- 8 | /geopose/1.0 | /Intrinsic/Translate-Rotate | GeoPose inner frame of a Chain
```

Las filas anteriores provienen de MEOS, que las construye a partir de los identificadores que escriben los codificadores, de modo que un marco que esta compilación puede emitir nunca falta en el catálogo. Un usuario registra marcos adicionales insertándolos en la tabla. La columna `is_geographic`, omitida arriba, es verdadera únicamente para el marco geográfico WGS-84.

- Devuelve todos los marcos que nombran el estándar y los codificadores

```
geoPoseFrames() → setof record
```

```
SELECT count(*) FROM geoPoseFrames();
-- 8
```

Tres funciones responden para un marco que MEOS registra, una de las filas que devuelve `geoPoseFrames`, y generan un error para cualquier otro identificador; un marco que registra un usuario se lee directamente de la tabla `geopose_frames`:

- Devuelve el SRID de un marco que MEOS registra, o `NULL` si el marco es paramétrico (LTP, BODY)

```
geoPoseFrameSRID(integer) → integer
```

```
SELECT geoPoseFrameSRID(1), geoPoseFrameSRID(2);
-- 4326 | 4978
SELECT geoPoseFrameSRID(3);
-- NULL
```

- Devuelve el nombre legible de un marco que MEOS registra

```
geoPoseFrameName(integer) → text
```

```
SELECT geoPoseFrameName(1);
-- WGS-84 geographic (lat/lon/h)
SELECT geoPoseFrameName(2), geoPoseFrameName(3);
-- WGS-84 ECEF (Earth-Centred Earth-Fixed) | Local Tangent Plane (East-North-Up)
```

- Devuelve `true` para un marco lat/lon/h que MEOS registra, `false` para uno cartesiano o proyectado

```
geoPoseFrameIsGeographic(integer) → boolean
```

```
SELECT geoPoseFrameIsGeographic(1), geoPoseFrameIsGeographic(2);
-- true | false
```

Los usuarios pueden registrar marcos personalizados insertándolos en `geopose_frames`; el catálogo está marcado como una tabla de configuración, por lo que `pg_dump` preserva las filas del usuario.

13.23.5. Transformación rígida cuerpo↔mundo

- Aplica la transformación de cuerpo rígido codificada por una pose (la correspondencia *cuerpo al mundo* de OGC GeoPose) a una geometría en el marco del cuerpo, produciendo la geometría correspondiente en el marco del mundo

```
applyPose(geometry,pose) → geometry
applyPose(geometry,tpose) → tgeompoint
applyPose(pose,pose) → pose
```

La transformación es

```
world = R(q) · body + p
```

donde `(p, q)` son la posición y la orientación de la pose. La forma estática toma una única pose y una geometría estática; la forma temporal eleva la transformación rígida por instante de una `tpose` a lo largo de sus instantes, produciendo una trayectoria `tgeompoint` en el marco del mundo de la geometría del cuerpo. La forma estática admite una geometría de cuerpo de cualquier tipo y lleva su forma al marco de la pose; la forma temporal devuelve una `tgeompoint`, por lo que su geometría de cuerpo debe ser un punto. La pose y la geometría del cuerpo deben tener la misma dimensionalidad y deben coincidir en su SRID, adoptando el desconocido el del otro. La interpolación lineal de la trayectoria resultante es la cuerda de cada segmento, que aproxima la verdadera trayectoria de cuerpo rígido (un arco circular bajo SLERP), la misma concesión que MobilityDB ya adopta para las trayectorias espaciales.

```
-- A body sensor offset 1 unit along the body X axis, traced through a tpose that ends 90
-- degrees yawed and translated to (10, 20).
SELECT asText(applyPose(ST_Point(1,0), tpose '[Pose(Point(0 0), 0)@2026-01-01,
  Pose(Point(10 20), 1.5707963267948966)@2026-01-02]'));
-- [POINT(1 0)@2026-01-01, POINT(10 21)@2026-01-02]
```

```
-- The shape of the body geometry is carried into the pose's frame: a hull 4 by 2 about
-- the body origin, placed at (10, 20)
SELECT ST_AsText(applyPose(ST_MakeEnvelope(-2,-1,2,1), pose 'Pose(Point(10 20), 0)'));
-- POLYGON((8 19,8 21,12 21,12 19,8 19))
```

Cualquiera de los dos marcos puede moverse. Aplicar una pose a una `tpose` lleva un cuerpo fijo a través de todo el movimiento de su padre, que es lo que pide un sensor montado sobre un vehículo en movimiento; aplicar una `tpose` a una pose lleva un cuerpo en movimiento a un marco que permanece quieto; y aplicar una `tpose` a una `tpose` compone dos marcos que ambos se mueven, sobre el tiempo que comparten. `poseInverse` se eleva de la misma manera, cambiando el punto de vista sobre un objeto en movimiento durante todo su movimiento.

```
applyPose(pose,tpose) → tpose
applyPose(tpose,pose) → tpose
applyPose(tpose,tpose) → tpose
poseInverse(tpose) → tpose
```

```
-- A sensor one unit ahead of a body that ends yawed a quarter turn at (10, 20) ends one
-- unit north of it
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)',
  tpose '[Pose(Point(0 0), 0)@2026-01-01,
  Pose(Point(10 20), 1.5707963267948966)@2026-01-02]'), 6));
-- [Pose(POINT(1 0),0)@2026-01-01, Pose(POINT(10 21),1.570796)@2026-01-02]
```

Una pose nombra un marco, por lo que ella misma es un valor en el marco del cuerpo: aplicar una pose a otra compone las dos relaciones entre marcos. Escribiendo `P_WV` para la pose de un vehículo en el mundo y `P_VS` para la pose de un sensor en el vehículo, la pose del sensor en el mundo es `P_WS = P_WV ∘ P_VS`. Esta es la operación que una cadena de poses pliega sobre sus eslabones, de modo que una cadena de dos eslabones se compone en lo que esto devuelve.

```
-- A sensor one unit ahead of a vehicle turned a quarter turn is one unit North of it, and
-- carries the vehicle's orientation
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)', pose(ST_Point(0,0), pi()/2)),
  6));
```

```
-- Pose(POINT(0 1),1.570796)
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)', pose(ST_Point(10,5), pi()/2)),
  6));
-- Pose(POINT(10 6),1.570796)
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0 0), 1, 0, 0, 0)',
  pose 'Pose(Point(0 0 5), 1, 0, 0, 0)'), 6));
-- Pose(POINT Z (1 0 5),1,0,0,0)
```

13.23.6. E/S JSON temporal

- Convierte una pose temporal hacia o desde su codificación OGC GeoPose v1.0

```
asGeoPose(tpose,conformance integer=0,maxdecimaldigits integer=-1) → text
tposeFromGeoPose(text) → tpose
```

La clase de conformidad se deduce del valor. Un instante único es un documento Basic que lleva su `validTime`; una secuencia o un conjunto de secuencias es una serie de secuencia compuesta, de la clase Regular cuando los instantes están igualmente espaciados por un número entero de milisegundos y de la clase Irregular en caso contrario. El argumento `conformance` elige la codificación de la orientación de un documento Basic, como lo hace para una `pose`, y no tiene efecto sobre una serie, cuyos marcos interiores llevan un cuaternión y no ofrecen tal elección. Una serie declara su marco exterior una sola vez, como el marco Este-Norte-Arriba del plano tangente local en la posición de la primera pose, y da cada pose como un marco interior que contiene su traslación desde ese punto de tangencia, en metros, y su rotación relativa a la base de ese marco. Los tiempos son valores `GeoPose_Instant`, es decir, tiempo Unix en milisegundos enteros, que es lo que el estándar exige de toda posición temporal que define. Por lo tanto, el muestreo por debajo del milisegundo no se lleva; la codificación MF-JSON, que el estándar que la sustenta tipa como una cadena de fecha y hora, sí lo lleva. El modelo de transición informa de la interpolación. La interpolación lineal es el modelo `interpolate` del estándar y las interpolaciones de pasos y discreta son su modelo `none`; puesto que esos dos identificadores no distinguen la de pasos de la discreta, los `parameters` del modelo nombran la interpolación con las mismas palabras que usa la codificación MF-JSON.

```
SELECT asGeoPose(tpose '[Pose(Point(0 0 0), 0.5, 0.5, 0.5, 0.5)@2026-01-01,
  Pose(Point(0 0 100), 0.5, 0.5, 0.5, 0.5)@2026-01-02,
  Pose(Point(0 0 300), 0.5, 0.5, 0.5, 0.5)@2026-01-05]', 0, 6);
-- {
--  "header": {
--   "poseCount": 3,
--   "startInstant": 1767254400000,
--   "stopInstant": 1767600000000,
--   "transitionModel": {
--    "authority": "/geopose/1.0",
--    "id": "interpolate",
--    "parameters": "interpolation=Linear"
--   }
--  },
--  "outerFrame": {
--   "authority": "/geopose/1.0",
--   "id": "LTP-ENU",
--   "parameters": "longitude=0&latitude=0&height=0&crs=EPSG:4979"
--  },
--  "innerFrameAndTimeSeries": [
--   {
--    "frame": {
--     "authority": "/geopose/1.0",
--     "id": "RotateTranslate",
--     "parameters": "translation=[0, 0, 0]&rotation=[0.5, 0.5, 0.5, 0.5]"
--    },
--    "validTime": 1767254400000
--   },
--   {
--    "frame": {
```

```
--      "authority": "/geopose/1.0",
--      "id": "RotateTranslate",
--      "parameters": "translation=[0, 0, 100]&rotation=[0.5, 0.5, 0.5, 0.5]"
--     },
--     "validTime": 1767340800000
--    },
--    {
--     "frame": {
--      "authority": "/geopose/1.0",
--      "id": "RotateTranslate",
--      "parameters": "translation=[0, 0, 300]&rotation=[0.5, 0.5, 0.5, 0.5]"
--     },
--     "validTime": 1767600000000
--    }
--   ],
--   "trailer": {
--    "poseCount": 3
--   }
-- }
```

Una serie no tiene ni huecos ni límites abiertos, de modo que un conjunto de secuencias se aplana en una única secuencia cerrada y la inclusión de los límites de una secuencia no se conserva. La lectura acepta además la envoltura `TemporalGeoPose` que escribían versiones anteriores, un arreglo de objetos de la clase Basic cada uno con un `validTime` añadido, de modo que los datos ya almacenados en esa forma siguen cargándose.

La forma de esa envoltura es

```
{
  "type":          "TemporalGeoPose",
  "version":       "1.0",
  "conformance":   "Basic-Quaternion" | "Basic-YPR",
  "interpolation": "None" | "Discrete" | "Step" | "Linear",
  "instants":      [...]                  // for TInstant + TSequence
  "lower_inc":     true|false,            // for TSequence only
  "upper_inc":     true|false,            // for TSequence only
  "sequences":     [{...}, ...]           // for TSequenceSet only
}
```

```
SELECT asGeoPose(tpose '[Pose(Point(8 47), 0)@2026-01-01,
  Pose(Point(9 48), 0.5)@2026-01-02]', 1, 4);
/* {"type":"TemporalGeoPose","version":"1.0","conformance":"Basic-YPR",
   "interpolation":"Linear","lower_inc":true,"upper_inc":true,
   "instants":[
   {"position":{"lat":47,"lon":8,"h":0},
   "angles":{"yaw":0,"pitch":0,"roll":0},
   "validTime":"2026-01-01"},
   {"position":{"lat":48,"lon":9,"h":0},
   "angles":{"yaw":28.65,"pitch":0,"roll":0},
   "validTime":"2026-01-02"}]} */
```

- Escribe una pose temporal como un Stream de OGC GeoPose

```
asGeoPoseStream(tpose,maxdecimaldigits integer=-1) → text
```

Un stream lleva los marcos de una Serie Irregular sin declarar cuántas poses hay ni cuándo terminan, ya que pueden llegar más. El estándar lo escribe como un `header`, que contiene el modelo de transición y el marco exterior, y un arreglo `streamElements` con un elemento por pose. El marco exterior es el plano tangente local Este-Norte-Arriba en la posición de la primera pose, de modo que la cabecera y cada elemento hablan de un mismo punto de tangencia. Esta función escribe un valor que ya se posee entero; asGeoPoseStreamHeader y `asGeoPoseStreamElement` escriben los mismos dos documentos por partes.

```
SELECT asGeoPoseStream(tpose 'Geodpose(Point(0 0 0), 1, 0, 0, 0)@2026-01-01', 6);
/* {"header":{"transitionModel":{"authority":"/geopose/1.0","id":"none",
   "parameters":"interpolation=None"},"outerFrame":{"authority":"/geopose/1.0",
   "id":"LTP-ENU","parameters":"longitude=0&latitude=0&height=0&crs=EPSG:4979"}},
   "streamElements":[{"streamElement":{"frame":{"authority":"/geopose/1.0",
   "id":"RotateTranslate","parameters":"translation=[0, 0, 0]&rotation=[1, 0, 0, 0]"},
   "validTime":1767254400000}}]} */
```

- Escribe la cabecera de Stream de OGC GeoPose de una pose temporal, o el elemento de Stream de uno de sus instantes

```
asGeoPoseStreamHeader(tpose,maxdecimaldigits integer=-1) → text
asGeoPoseStreamElement(tpose,tpose,maxdecimaldigits integer=-1) → text
```

Son los dos documentos que un motor de flujos escribe uno a la vez: la cabecera una vez y luego un elemento por cada instante a medida que llega. El segundo argumento de `asGeoPoseStreamElement` es un instante del primero, y el elemento expresa su marco respecto del marco exterior que la cabecera ancla en la primera pose, de modo que los documentos son los miembros del stream que `asGeoPoseStream` escribe entero.

```
SELECT asGeoPoseStreamHeader(tpose '[Geodpose(Point(0 0 0), 1, 0, 0, 0)@2026-01-01,
  Geodpose(Point(0 0 0), 0.707107, 0, 0, 0.707107)@2026-01-02]', 6);
/* {"transitionModel":{"authority":"/geopose/1.0","id":"interpolate",
   "parameters":"interpolation=Linear"},"outerFrame":{"authority":"/geopose/1.0",
   "id":"LTP-ENU","parameters":"longitude=0&latitude=0&height=0&crs=EPSG:4979"}} */
SELECT asGeoPoseStreamElement(tpose '[Geodpose(Point(0 0 0), 1, 0, 0, 0)@2026-01-01,
  Geodpose(Point(0 0 0), 0.707107, 0, 0, 0.707107)@2026-01-02]',
  tpose 'Geodpose(Point(0 0 0), 0.707107, 0, 0, 0.707107)@2026-01-02', 6);
/* {"streamElement":{"frame":{"authority":"/geopose/1.0","id":"RotateTranslate",
   "parameters":"translation=[0, 0, 0]&rotation=[0.707107, 0, 0, 0.707107]"},
   "validTime":1767340800000}} */
```

13.23.7. Documentos compuestos de cadena y de grafo

El estándar declara una composición como un documento de cadena y un conjunto de marcos como un documento de grafo, que es lo que escribe una cadena de poses.

- Escribe o lee un documento de cadena compuesta OGC GeoPose

```
asGeoPose(tposechain,maxdecimaldigits integer=-1) → text
tposechainFromGeoPose(text) → tposechain
```

Un documento de cadena es un marco exterior y una secuencia de transformaciones que alcanza un marco interior final, que es lo que contiene una cadena de poses. El documento nombra el marco LTP-ENU tangente en la posición del eslabón exterior y una transformación por eslabón: la primera lleva ese marco tangente al marco propio del eslabón exterior, y cada una posterior es el eslabón tal como se almacena, leído en los ejes de su padre. El documento lleva un único tiempo de validez, por lo que se escribe a partir de un solo instante; atTime obtiene uno de un valor más largo. Su cadena de marcos contiene al menos dos marcos, por lo que una cadena de un solo eslabón no tiene documento de cadena. La clase de cadena nombra sus dos marcos `/Extrinsic/LTP-ENU` e `/Intrinsic/Translate-Rotate`, mientras que las clases Advanced, Series y Stream nombran esos mismos dos marcos `LTP-ENU` y `RotateTranslate`. Un documento se escribe con el par que usa su propia clase, y cualquiera de los dos pares se acepta al leer.

```
SELECT asGeoPose(tposechain 'SRID=4326;
  PoseChain( GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
  Pose(Point Z(2 0 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00', 6);
/* {"validTime":1619588170083,"outerFrame":{"authority":"/geopose/1.0",
   "id":"/Extrinsic/LTP-ENU","parameters":"longitude=-122.3&latitude=47.7&
   height=11&crs=EPSG:4979"},"frameChain":[{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
   rotation=[1, 0, -1.38778e-17, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[2, 0, 0]&
   rotation=[1, 0, 0, 0]"}]} */
```

- Escribe un documento de grafo compuesto OGC GeoPose

```
asGeoPose(tposechain[],maxdecimaldigits integer=-1) → text
```

Un documento de grafo es un conjunto de marcos y las transformaciones entre ellos, que es lo que contienen varias cadenas de poses que comparten su marco más exterior. La lista de marcos nombra el marco LTP-ENU tangente en ese eslabón más exterior seguido de los eslabones de cada cadena, y la lista de transformaciones nombra el padre y el hijo de cada arista por su posición en esa lista. Las aristas no llevan transformación propia; esta reside en los marcos que nombran. El documento lleva un único tiempo de validez, por lo que se escribe a partir de cadenas leídas en uno y el mismo instante; atTime las obtiene de valores más largos. Su lista de marcos contiene al menos dos marcos, que un solo eslabón de una sola cadena ya proporciona junto al marco tangente.

```
SELECT asGeoPose(ARRAY[tposechain 'SRID=4326;
  PoseChain( GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
  Pose(Point Z(2 0 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00',
  tposechain 'SRID=4326;
  PoseChain(GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
  Pose(Point Z(0 3 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00'], 6);
/* {"validTime":1619588170083,"frameList":[{"authority":"/geopose/1.0",
   "id":"/Extrinsic/LTP-ENU","parameters":"longitude=-122.3&latitude=47.7&
   height=11&crs=EPSG:4979"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
   rotation=[1, 0, -1.38778e-17, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[2, 0, 0]&
   rotation=[1, 0, 0, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
   rotation=[1, 0, -1.38778e-17, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 3, 0]&
   rotation=[1, 0, 0, 0]"}],"transformList":[{"link":[0,1]},{"link":[1,2]},
   {"link":[0,3]},{"link":[3,4]}]} */
```

Capítulo 14

Geometrías rígidas temporales

Una geometría rígida temporal (`trgeometry`) es una geometría de referencia estática, en dos o tres dimensiones, normalmente un polígono que describe la forma de un cuerpo en movimiento— emparejada con una pose temporal (`tpose`) que describe cómo esa geometría se traslada y rota a lo largo del tiempo. La geometría materializada en el instante `t` es la geometría de referencia rotada y trasladada de acuerdo con la pose interpolada en `t`.

Esta es la representación natural para objetos en movimiento cuya *forma* importa, no solo su posición. El ejemplo motivador es el tráfico marítimo AIS, donde el casco de cada barco (un pentágono derivado de los desplazamientos de antena `A`, `B`, `C`, `D`) sigue una trayectoria de pose definida por informes de latitud/longitud y rumbo. La geometría materializada en cualquier instante es el contorno real del barco en ese momento.

El tipo `trgeometry` se construye sobre Capítulo 13. La mayoría de los operadores de acceso, comparación, topológicos, de posición y de indexación tienen la misma forma que las funciones `tpose` correspondientes; este capítulo cubre la superficie específica de trgeometry: la materialización de pose a forma, el área recorrida, la distancia materializada y las restricciones espaciales sobre el cuerpo en movimiento.

14.1. Entrada y salida

Un literal `trgeometry` empareja una geometría de referencia con una pose temporal. La geometría de referencia va primero, separada de la pose por un punto y coma; la pose sigue la sintaxis estándar de `tpose` (instante, secuencia o conjunto de secuencias).

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.5)@2001-01-01';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0)@2001-01-01,
  Pose(Point(5 0), 0.5)@2001-01-02, Pose(Point(0 0), 0.0)@2001-01-03}';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 1.5)@2001-01-02]';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.0)@2001-01-02],
  [Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(2 0), 0.0)@2001-01-05]}';
```

La geometría de referencia puede ser cualquier polígono o superficie poliédrica, con la dimensión de la pose: una pose con un cuaternión necesita una geometría de referencia en tres dimensiones, y `ST_Force3D` da una plana, en el plano z = 0 del cuerpo. Otros valores dan un error. La interpolación de la pose es lineal en la traslación y de camino angular más corto en la rotación. Los SRID se heredan de la geometría de referencia y de la pose; deben coincidir.

Se puede especificar un SRID para una trgeometry al comienzo del literal, como se muestra a continuación.

```
SELECT trgeometry 'SRID=25832;Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(0 0), 0.5)@2001-01-01';
```

La interpolación escalonada también se puede especificar de la misma manera que para otros tipos temporales:

```
SELECT trgeometry 'Interp=Step;Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
```

Se admiten los formatos de transmisión estándar.

- Devuelve la representación de texto conocido (WKT)

```
asText({trgeometry,trgeometry[]} [, maxdecimaldigits integer=15]) → text
```

```
SELECT asText(trgeometry 'Polygon((1 1,2 2,3 1,1 1));
  Pose(Point(1.123456789 1.123456789), 0.5)@2001-01-01', 6);
-- POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1.123457 1.123457),0.5)@2001-01-01
```

- Devuelve la representación extendida de texto conocido (EWKT), prefijada con el SRID

```
asEWKT({trgeometry,trgeometry[]} [, maxdecimaldigits integer=15]) → text
```

```
SELECT asEWKT(trgeometry 'SRID=25832;Polygon((1 1,2 2,3 1,1 1));
  Pose(Point(1 1),0.5)@2001-01-01');
-- SRID=25832;POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1 1),0.5)@2001-01-01
```

- Devuelve la representación JSON de características móviles (MF-JSON)

```
asMFJSON(trgeometry [, options integer=0 [, flags integer=0
  [, maxdecimaldigits integer=15]]]) → text
```

```
SELECT asMFJSON(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(0 0),0)@2001-01-01');
```

`asMFJSON` aún no admite una superficie poliédrica.

- Devuelve la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

```
asEWKB(trgeometry [, endian text]) → bytea
asHexWKB(trgeometry [, endian text]) → text
asHexEWKB(trgeometry [, endian text]) → text
```

Las funciones complementarias `asHexWKB` y `asHexEWKB` devuelven el texto codificado en hexadecimal de, respectivamente, la representación simple y la que incluye el SRID; ambas coinciden solo cuando el valor no tiene SRID, como en el ejemplo siguiente.

```
SELECT length(asEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01'));
-- 130
-- The bytes round-trip through the matching input function.
SELECT asText(trgeometryFromHexEWKB(asHexEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
SELECT asText(trgeometryFromHexEWKB(asHexWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
```

- Entrada desde la representación de texto conocido (WKT), desde la representación extendida de texto conocido (EWKT), desde la representación JSON de características móviles (MF-JSON), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

```
trgeometryFromText(text) → trgeometry
trgeometryFromEWKT(text) → trgeometry
trgeometryFromMFJSON(text) → trgeometry
trgeometryFromEWKB(bytea) → trgeometry
trgeometryFromHexWKB(text) → trgeometry
trgeometryFromHexEWKB(text) → trgeometry
```

```
SELECT asText(trgeometryFromEWKT('SRID=4326;Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
SELECT asText(trgeometryFromHexEWKB(asHexEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
```

14.2. Constructores

Una `trgeometry` se puede construir directamente a partir de sus tipos componentes: una geometría de referencia más una pose temporal:

- Construye una geometría rígida temporal a partir de una geometría de referencia y una pose temporal

```
trgeometry(geometry,tpose) → trgeometry
```

```
SELECT asText(trgeometry(geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
  tpose 'Pose(Point(0 0), 0.0)@2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(0 0),0)@2001-01-01
```

Ambos argumentos deben compartir el mismo SRID. La geometría de referencia puede ser un polígono o una superficie poliédrica; la pose puede ser de cualquier subtipo temporal.

14.3. Conversión de tipos

La `trgeometry` se puede proyectar a sus tipos componentes o a una trayectoria de puntos puros.

- Devuelve la pose temporal subyacente

```
tpose(trgeometry) → tpose
```

```
SELECT asText(tpose(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.5)@2001-01-02]'));
-- [Pose(POINT(0 0),0)@..., Pose(POINT(10 0),0.5)@...]
```

- Devuelve la trayectoria del centroide (antena) como un punto temporal

```
tgeompoint(trgeometry) → tgeompoint
tgeogpoint(trgeometry) → tgeogpoint
tgeography(trgeometry) → tgeography
```

Las conversiones siguen la regla de [?listitem].

```
SELECT asText(tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- [POINT(0 0)@..., POINT(10 0)@...]
```

14.4. Accesores

Se aplican los accesores estándar de los tipos temporales: cada uno devuelve la geometría materializada en la marca de tiempo solicitada (la forma de referencia rotada y trasladada de acuerdo con la pose interpolada), o un valor de metadatos derivado de la trayectoria de pose.

- Devuelve la geometría materializada al inicio, al final o en una marca de tiempo elegida

```
startValue(trgeometry) → geometry
endValue(trgeometry) → geometry
valueAtTimestamp(trgeometry,timestamptz) → geometry
```

```
SELECT asText(startValue(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0))
SELECT asText(valueAtTimestamp( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  timestamptz '2001-01-01 12:00:00'));
-- POLYGON((5 0,6 0,6 1,5 1,5 0))
```

Tenga en cuenta que la rotación interpola linealmente a lo largo del camino angular más corto; una rotación de 90° entre dos instantes separados por un día devuelve un polígono rotado 45° en el punto medio.

- Devuelve el número de instantes, secuencias o marcas de tiempo distintos

```
numInstants(trgeometry) → integer
numSequences(trgeometry) → integer
numTimestamps(trgeometry) → integer
```

```
SELECT numInstants(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 2
SELECT numSequences(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 1
SELECT numTimestamps(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 2
```

- Devuelve el arreglo de instantes, secuencias o segmentos entre instantes constituyentes

```
instants(trgeometry) → trgeometry[]
sequences(trgeometry) → trgeometry[]
segments(trgeometry) → trgeometry[]
```

```
SELECT array_length( instants(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.5)@2001-01-02,
  Pose(Point(0 0), 0.0)@2001-01-03}'), 1);
-- 3
```

- Devuelve el conjunto de puntos del centroide (antena) distintos vistos a lo largo de la trgeometry

```
points(trgeometry) → geomset
```

```
SELECT asText(points(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 5), 0.0)@2001-01-02,
  Pose(Point(0 0), 0.0)@2001-01-03}'));
-- {"POINT(0 0)", "POINT(5 5)"}
```

- Devuelve el ángulo yaw, pitch o roll como un flotante temporal, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose

```
yaw(trgeometry) → tfloat
pitch(trgeometry) → tfloat
roll(trgeometry) → tfloat
```

Una geometría rígida lleva su orientación en sus poses, de modo que cada una de estas funciones se proyecta sobre la pose temporal y se lo pregunta, exactamente como lo hacen los accesores homónimos de tpose. Una geometría rígida 2D gira por el ángulo almacenado de cada pose y no cabecea ni alabea.

```
SELECT asText(yaw(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(0 0), 1.5)@2001-01-02]'));
-- [0@2001-01-01, 1.5@2001-01-02]
SELECT asText(pitch(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(0 0), 1.5)@2001-01-02]'));
-- [0@2001-01-01, 0@2001-01-02]
```

- Devuelve la geometría de referencia estática

```
geometry(trgeometry) → geometry
```

```
SELECT ST_AsText(geometry(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(5 0), 0.5)@2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0))
```

14.5. Área recorrida

La función `traversedArea` devuelve la región que el cuerpo en movimiento cubre sobre el dominio temporal de la trgeometry: las colocaciones que toma, o la unión de ellas cuando se le pide.

- Devuelve el área atravesada por la geometría rígida temporal

```
traversedArea(trgeometry,unary_union boolean=false) → geometry
```

El argumento es falso salvo que se indique, y la función responde entonces las colocaciones mismas reunidas en una geometría, un multipolígono para un cuerpo que es un polígono, que es lo que quiere quien llama y hace su propio posprocesamiento. Con `unary_union = true` se disuelven en la región que cubren. *Advertencia de muestreo:* la implementación muestrea en los instantes de entrada, de modo que los segmentos de traslación pura son exactos, pero la cinta barrida entre dos instantes donde ocurre una rotación se aproxima mediante la envolvente convexa de los dos polígonos extremos. Sobremuestrear la `tpose` antes de construir la `trgeometry` ajusta la aproximación.

La forma disuelta de un valor en tres dimensiones es la región sobre el suelo, sin Z. Donde el cuerpo gira, su área aún no es la que tiene el mismo valor en dos dimensiones.

```
SELECT ST_AsText(traversedArea( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- MULTIPOLYGON(((0 0,1 0,1 1,0 1,0 0)),((10 0,11 0,11 1,10 1,10 0)))
SELECT ST_AsText(traversedArea( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', true));
-- POLYGON((0 0,0 1,11 1,11 0,0 0))
```

14.6. Funciones espaciales

Estas funciones derivan un valor espacial del cuerpo en movimiento. `centroid` y `bodyPointTrajectory` devuelven puntos temporales que siguen una posición a lo largo del tiempo; `convexHull` devuelve una geometría estática, sin Z para un valor en tres dimensiones.

- Devuelve el centroide del cuerpo en movimiento como un punto temporal

```
centroid(trgeometry) → tgeompoint
```

```
SELECT asText(centroid( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-02]'));
-- [POINT(0.5 0.5)@2001-01-01, POINT(2.5 0.5)@2001-01-02]
```

- Devuelve la envolvente convexa del área recorrida por el cuerpo en movimiento

```
convexHull(trgeometry) → geometry
```

```
SELECT ST_GeometryType(convexHull( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-02]'));
-- ST_Polygon
```

- Devuelve la trayectoria en el marco del mundo de un punto del marco del cuerpo transportado por la geometría rígida en movimiento

```
bodyPointTrajectory(trgeometry,geometry) → tgeompoint
```

El punto se expresa en el marco local de la forma de referencia y la pose variable en el tiempo se le aplica en cada instante. El resultado tiene la interpolación de la geometría rígida temporal. Con interpolación lineal, el resultado es exacto a lo largo de un segmento en el que el cuerpo no rota. A lo largo de un segmento en el que el cuerpo rota, el punto sigue una curva, que el resultado reemplaza por el segmento que une sus posiciones en los dos instantes, como en el segundo ejemplo a continuación, donde el punto se desplaza de (1 0) a (0 1) a lo largo de un cuarto de circunferencia.

```
SELECT asText(bodyPointTrajectory( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-02]',
  geometry 'Point(0 0)'));
-- [POINT(0 0)@2001-01-01, POINT(2 0)@2001-01-02]
SELECT asText(round(bodyPointTrajectory( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(1 0)'), 6));
-- [POINT(1 0)@2001-01-01, POINT(0 1)@2001-01-02]
```

14.7. Métricas de movimiento

Estas funciones informan sobre el movimiento de una geometría rígida temporal a lo largo de su trayectoria del centroide.

- Devuelve la longitud total recorrida por el centroide, o su longitud acumulada a lo largo del tiempo

```
length(trgeometry) → float
cumulativeLength(trgeometry) → tfloat
```

```
SELECT length(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 4),0)@2001-01-02]');
-- 5
```

- Devuelve la velocidad del centroide como un flotante temporal

```
speed(trgeometry) → tfloat
```

```
SELECT speed(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- Interp=Step;[0.000115740740741@2001-01-01, 0.000115740740741@2001-01-02]
```

- Devuelve la velocidad angular como un flotante temporal

```
angularSpeed(trgeometry) → tfloat
```

La forma de una geometría rígida nunca cambia, por lo que su velocidad angular es la de la pose que la lleva, en radianes por unidad de tiempo.

```
SELECT asText(angularSpeed(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1)@2001-01-02]'));
-- Interp=Step;[0.000011574074074@2001-01-01, 0.000011574074074@2001-01-02]
```

- Devuelve el centroide ponderado en el tiempo de la trayectoria del centroide como una geometría

```
twCentroid(trgeometry) → geometry
```

```
SELECT ST_AsText(twCentroid(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 4),0)@2001-01-02]'));
-- POINT(1.5 2)
```

14.8. Transformaciones

Se aplican las funciones estándar de modificación de secuencias: `round`, `setInterp`, `shiftTime`, `scaleTime`, `shiftScaleTime`, `deleteTimestamp`, etc. `round` aún no admite una superficie poliédrica, ni pocos dígitos para una rotación en tres dimensiones, y la transformación a otro SRID, `transform` y `transformPipeline`, aún no se admite para una geometría rígida temporal.

- Redondea todos los componentes numéricos a un número dado de posiciones decimales

```
round(trgeometry,integer) → trgeometry
```

```
SELECT asText(round( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1.123456789 1.123456789), 0.123456)@2001-01-01', 3));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1.123 1.123),0.123)@...
```

- Convierte entre interpolaciones lineal y escalonada

```
setInterp(trgeometry,text) → trgeometry
```

```
SELECT asText(setInterp(trgeometry 'Interp=Step;Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 'linear'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- {[Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(0 0),0)@2001-01-02),
--  [Pose(POINT(10 0),0)@2001-01-02]}
```

- Desplaza, escalar, o desplazar y escalar el dominio temporal de una trgeometry

```
shiftTime(trgeometry,interval) → trgeometry
scaleTime(trgeometry,interval) → trgeometry
shiftScaleTime(trgeometry,interval,interval) → trgeometry
```

```
SELECT asText(shiftTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
  0.0)@2001-01-02]', interval '1 day'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-02, Pose(POINT(10 0),0)@2001-01-03]
SELECT asText(scaleTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
```

```
  0.0)@2001-01-02]', interval '2 days'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(10 0),0)@2001-01-03]
SELECT asText(shiftScaleTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
  0.0)@2001-01-02]', interval '1 day', interval '2 days'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-02, Pose(POINT(10 0),0)@2001-01-04]
```

- Transforma una trgeometry no lineal en un conjunto de filas, una por pose distinta, cada una es una pareja compuesta de la geometría de referencia con la pose aplicada y el conjunto de períodos durante el cual la trgeometry toma esa pose {}

```
unnest(trgeometry) → {(value,time)}
```

```
SELECT ST_AsText((un).value), (un).time
FROM (SELECT unnest(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-02,
  Pose(Point(0 0), 0.0)@2001-01-03}') AS un) t;
-- POLYGON((0 0,1 0,1 1,0 1,0 0)) | {[2001-01-01, 2001-01-01], [2001-01-03, 2001-01-03]}
-- POLYGON((4 0,5 0,5 1,4 1,4 0)) | {[2001-01-02, 2001-01-02]}
SELECT ST_AsText(ST_SnapToGrid((un).value, 0.000001)), (un).time
FROM (SELECT unnest(merge(
  trgeometry(geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
    pose(geometry 'Point(0 0)', 0), timestamptz '2001-01-01'),
  trgeometry(geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
    pose(geometry 'Point(3 2)', pi() / 2), timestamptz '2001-01-02'))) AS un) t;
-- POLYGON((0 0,1 0,1 1,0 1,0 0)) | {[2001-01-01, 2001-01-01]}
-- POLYGON((3 2,3 3,2 3,2 2,3 2)) | {[2001-01-02, 2001-01-02]}
```

La pose se aplica a la geometría de referencia en cada fila del resultado: el valor de una fila es la geometría de referencia rotada según la orientación de la pose y luego trasladada a su posición, la geometría que `startValue` y `valueAtTimestamp` devuelven en esa pose. En el segundo ejemplo la pose rota el cuadrado 90° y lo desplaza a (3, 2). Una geometría de referencia que una rotación lleva sobre sí misma produce filas con geometrías iguales, una por pose.

- Devuelve un valor temporal trasladado por los desplazamientos dados

```
translate(trgeometry,deltax float,deltay float,deltaz float=0) → trgeometry
```

Un desplazamiento vertical saca un valor bidimensional de su plano, por lo que el resultado es tridimensional.

El movimiento desplaza la pose de cada instante y conserva la geometría de referencia. Un movimiento que saca un valor bidimensional de su plano lleva consigo la geometría de referencia, en el plano z = 0.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(translate(trgeometry 'Polygon((0 0,2 0,2 1,0 1,0 0));
  [Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(3 2),1.1)@2001-01-02]', 1, 2), 6);
/* POLYGON((0 0,2 0,2 1,0 1,0 0));[Pose(POINT(2 3),0.3)@2001-01-01,
   Pose(POINT(4 4),1.1)@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del origen o de un punto

```
rotate(trgeometry,angle float) → trgeometry
rotate(trgeometry,angle float,x0 float,y0 float) → trgeometry
```

El movimiento desplaza la pose de cada instante y conserva la geometría de referencia. Un movimiento que saca un valor bidimensional de su plano lleva consigo la geometría de referencia, en el plano z = 0.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotate(trgeometry 'Polygon((0 0,2 0,2 1,0 1,0 0));
  [Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(3 2),1.1)@2001-01-02]', 0.7, 1, 1), 6);
/* POLYGON((0 0,2 0,2 1,0 1,0 0));[Pose(POINT(1 1),1)@2001-01-01,
   Pose(POINT(1.885467 3.053278),1.8)@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del eje x

```
rotateX(trgeometry,angle float) → trgeometry
```

La rotación saca un valor bidimensional de su plano, por lo que el resultado es tridimensional: cada posición gana la altura que le da la rotación y cada orientación es la rotación compuesta con el ángulo alrededor de la vertical que indica el valor.

El movimiento desplaza la pose de cada instante y conserva la geometría de referencia. Un movimiento que saca un valor bidimensional de su plano lleva consigo la geometría de referencia, en el plano z = 0.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

```
SELECT asText(rotateX(trgeometry 'Polygon((0 0,2 0,2 1,0 1,0 0));
  [Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* POLYGON Z ((0 0 0,2 0 0,2 1 0,0 1 0,0 0 0));
   [Pose(POINT Z (1 0.764842 0.644218),0.928825,0.339047,-0.051242,0.140378)@2001-01-01,
   Pose(POINT Z (3 1.529684 1.288435),0.800838,0.292329,-0.179228,0.490998)@2001-01-02] */
```

- Devuelve un valor temporal rotado en sentido antihorario alrededor del eje y

```
rotateY(trgeometry,angle float) → trgeometry
```

La rotación saca un valor bidimensional de su plano, por lo que el resultado es tridimensional: cada posición gana la altura que le da la rotación y cada orientación es la rotación compuesta con el ángulo alrededor de la vertical que indica el valor.

El movimiento desplaza la pose de cada instante y conserva la geometría de referencia. Un movimiento que saca un valor bidimensional de su plano lleva consigo la geometría de referencia, en el plano z = 0.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

- Devuelve un valor temporal rotado en sentido antihorario alrededor del eje z

```
rotateZ(trgeometry,angle float) → trgeometry
```

El movimiento desplaza la pose de cada instante y conserva la geometría de referencia. Un movimiento que saca un valor bidimensional de su plano lleva consigo la geometría de referencia, en el plano z = 0.

El movimiento es una aplicación plana, por lo que un valor geodésico genera un error.

- Devuelve una geometría rígida temporal proyectada sobre el suelo

```
force2D(trgeometry) → trgeometry
```

La geometría de referencia pierde su Z y cada pose se proyecta como force2D proyecta una pose temporal, de modo que las funciones en dos dimensiones responden sobre el suelo: en el segundo ejemplo, dos cuerpos planos a distintas alturas se tocan. Una geometría de referencia que no sea un polígono en un plano da un error, y un valor en dos dimensiones queda como está.

```
SELECT asText(force2D(trgeometry 'Polygon((-2 -1 0,2 -1 0,2 1 0,-2 1 0,-2 -1 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
  Pose(Point(10 0 0),0.7071067811865476,0,0,0.7071067811865476)@2001-01-02]'), 6);
/* POLYGON((-2 -1,2 -1,2 1,-2 1,-2 -1));[Pose(POINT(0 0),0)@2001-01-01,
   Pose(POINT(10 0),1.570796)@2001-01-02] */
SELECT round(nearestApproachDistance(
  force2D(trgeometry 'Polygon((-2 -1 0,2 -1 0,2 1 0,-2 1 0,-2 -1 0));
    [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(10 0 0),1,0,0,0)@2001-01-02]'),
  force2D(trgeometry 'Polygon((-2 -1 0,2 -1 0,2 1 0,-2 1 0,-2 -1 0));
    [Pose(Point(5 0 0.8),1,0,0,0)@2001-01-01, Pose(Point(5 0 0.8),1,0,0,0)@2001-01-02]')) ↩
      ::numeric, 6);
-- 0.000000
```

14.9. Modificaciones

- Agrega un único instante a una trgeometry existente, ampliando la trayectoria de pose subyacente

```
appendInstant(trgeometry,trgeometry [, maxdist float [, maxt interval]]) → trgeometry
```

```
SELECT appendInstant( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(5 0), 0.0)@2001-01-02');
```

- Agrega una secuencia completa a una trgeometry existente

```
appendSequence(trgeometry,trgeometry) → trgeometry
```

```
SELECT asText(appendSequence(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(20 0), 0.0)@2001-01-03, Pose(Point(30 0), 0.0)@2001-01-04]'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- {[Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(10 0),0)@2001-01-02],
--    [Pose(POINT(20 0),0)@2001-01-03, Pose(POINT(30 0),0)@2001-01-04]}
```

- Combina dos trgeometries con la misma geometría de referencia en un único conjunto de secuencias

```
merge(trgeometry,trgeometry) → trgeometry
merge(trgeometry) → trgeometry
```

```
SELECT asText(merge( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(5 0), 0.0)@2001-01-05]'));
```

14.10. Restricciones

Las funciones `atGeometry`, `minusGeometry`, `atStbox` y `minusStbox` restringen una trgeometry a (el complemento de) una geometría o una caja espaciotemporal. La semántica refleja la de `tpose`: una geometría rígida temporal está «en» una región en el instante `t` si y solo si su cuerpo, la geometría de referencia situada por la pose en `t`, toca esa región. `atGeometry` y `minusGeometry` aún no se admiten para un valor en tres dimensiones, ni para una geometría con Z o con agujeros, y `atStbox` y `minusStbox` aún no leen la dimensión Z de una caja.

- Restringe al (complemento de) una pose o un conjunto de poses

```
atValue(trgeometry,pose) → trgeometry
minusValue(trgeometry,pose) → trgeometry
atValues(trgeometry,poseset) → trgeometry
minusValues(trgeometry,poseset) → trgeometry
```

```
SELECT asText(atValue(trgeometry 'Polygon((1 1,2 2,3 1,1 1));
  [Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.4)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03]', pose 'Pose(Point(1 1), 0.5)'));
-- POLYGON((1 1,2 2,3 1,1 1));{[Pose(POINT(1 1),0.5)@2001-01-03]}
SELECT asText(minusValue(trgeometry 'Polygon((1 1,2 2,3 1,1 1));
  {Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03}', pose 'Pose(Point(1 1), 0.5)'));
-- POLYGON((1 1,2 2,3 1,1 1));{Pose(POINT(1 1),0.3)@2001-01-01}
```

El valor de restricción es una pose, el valor base que interpola una geometría rígida temporal. La geometría de referencia se conserva sin cambios, de modo que el resultado describe el mismo cuerpo.

- Restringe una trgeometry a (el complemento de) una geometría

```
atGeometry(trgeometry,geometry) → trgeometry
minusGeometry(trgeometry,geometry) → trgeometry
```

```
SELECT getTime(atGeometry( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))'));
-- {[2001-01-01, 2001-01-04]}
SELECT getTime(minusGeometry( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))'));
-- {(2001-01-04, 2001-01-05]}
```

La restricción contra una geometría vacía devuelve NULL para `atGeometry` y la entrada original para `minusGeometry`.

- Restringe una trgeometry a (el complemento de) una caja espaciotemporal

```
atStbox(trgeometry,stbox[,border_inc boolean=true]) → trgeometry
minusStbox(trgeometry,stbox[,border_inc boolean=true]) → trgeometry
```

```
SELECT getTime(atStbox( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  stbox 'STBOX X((1, -1), (3, 1))'));
-- {[2001-01-01, 2001-01-04]}
```

Una caja con solo un componente temporal aún no se admite, y `atTime` responde esa pregunta; si la caja solo tiene un componente espacial, se conserva el dominio temporal.

14.11. Operaciones de distancia

Los operandos de dimensiones distintas se leen en las dimensiones que ambos tienen, véase Sección 8.3.

Las funciones de distancia calculan la distancia entre el cuerpo de una geometría rígida temporal y una geometría, un punto temporal o el cuerpo de otra geometría rígida temporal. El cuerpo en un instante es la geometría de referencia desplazada y girada por la pose en ese instante.

Los valores de una distancia temporal son exactos. Cuando un cuerpo gira, la distancia entre dos instantes no varía linealmente, por lo que la distancia temporal tiene también un instante en cada mínimo y cada máximo de la distancia. Así, la distancia de aproximación más cercana y el instante de aproximación más cercana son exactos.

Las funciones de distancia requieren una geometría de referencia que sea un polígono convexo sin agujeros. Un polígono del otro argumento también debe ser convexo y sin agujeros. Otros valores dan un error. La distancia de una geometría rígida temporal en tres dimensiones aún no se admite, ni la distancia a una colección de geometrías o a una curva. Por tanto, la distancia temporal entre dos geometrías cóncavas tampoco se admite aún.

```
SELECT tDistance(trgeometry 'Polygon((0 0,4 0,4 4,2 1,0 4,0 0));
  Pose(Point(0 0),0)@2001-01-01', geometry 'Point(7 3)');
-- ERROR:  The distance of a temporal rigid geometry needs a reference geometry that is
-- a convex polygon without holes
```

- Devuelve la distancia temporal

```
{geometry,tgeompoint,trgeometry} <-> {geometry,tgeompoint,trgeometry} → tfloat
tDistance({geometry,tgeompoint,trgeometry},{geometry,tgeompoint,trgeometry}) → tfloat
```

En el primer ejemplo, un cuadrado pasa junto a un punto. Su lado permanece a distancia 2 del punto desde las 10:00 hasta las 14:00. En el segundo ejemplo, el cuadrado gira un cuarto de vuelta junto al punto. Una esquina del cuadrado se acerca más al punto a las 12:00, y el resultado tiene un instante allí.

```
SELECT trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(-6 0),0)@2001-01-01, Pose(Point(6 0),0)@2001-01-02]' <->
  geometry 'Point(0 3)';
-- [5.385164807134504@2001-01-01, 2@2001-01-01 10:00:00, 2@2001-01-01 14:00:00,
--  5.385164807134504@2001-01-02]
SELECT round(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]' <->
  geometry 'Point(3 0)', 6);
-- [2@2001-01-01, 1.828427@2001-01-01 05:11:32.371809, 1.585786@2001-01-01 12:00:00,
--  1.828427@2001-01-01 18:48:27.628191, 2@2001-01-02]
SELECT trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),0)@2001-01-02]' <->
  tgeompoint '[Point(3 -4)@2001-01-01, Point(3 4)@2001-01-02]';
-- [3.605551275463989@2001-01-01, 2@2001-01-01 09:00:00, 2@2001-01-01 15:00:00,
--  3.605551275463989@2001-01-02]
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),0)@2001-01-02]' <->
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(2 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]';
-- [1@2001-01-01, 9@2001-01-02]
```

- Devuelve la distancia de aproximación más cercana

```
{geometry,stbox,tgeompoint,trgeometry} |=| {geometry,stbox,tgeompoint,trgeometry} → float
nearestApproachDistance({geometry,stbox,tgeompoint,trgeometry},
  {geometry,stbox,tgeompoint,trgeometry}) → float
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]' |=|
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(50 0),0)@2001-01-01, Pose(Point(50 0),0)@2001-01-02]';
-- 39
SELECT round(nearestApproachDistance(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)')::numeric, 6);
-- 1.585786
```

El operador `|=|` se puede utilizar para búsquedas de vecino más cercano con índices GiST o SP-GiST (véase Sección 14.20).

- Devuelve el instante del primer argumento en el que los dos argumentos están más cerca

```
nearestApproachInstant({geometry,tgeompoint,trgeometry},
  {geometry,tgeompoint,trgeometry}) → trgeometry
```

Si la distancia permanece en su mínimo durante un intervalo, el resultado es el primer instante del intervalo. En el segundo ejemplo, el cuadrado está a distancia 4 del punto desde las 09:36 hasta las 12:00.

```
SELECT asText(nearestApproachInstant(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)'), 6);
-- POLYGON((-1 -1,1 -1,1 1,-1 1,-1 -1));Pose(POINT(0 0),0.785398)@2001-01-01 12:00:00
SELECT asText(nearestApproachInstant(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]',
  geometry 'Point(5 5)'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(4 0),0)@2001-01-01 09:36:00
```

- Devuelve la línea entre los puntos más cercanos de los dos argumentos en su instante de aproximación más cercana

```
shortestLine({geometry,tgeompoint,trgeometry},{geometry,tgeompoint,trgeometry})
  → geometry
```

```
SELECT ST_AsText(shortestLine(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(0 0),0)@2001-01-01', geometry 'Point(10 0)'));
-- LINESTRING(1 0,10 0)
```

14.12. Similitud

Una geometría rígida temporal no tiene una distancia de similitud propia. Comparar dos poses combina una diferencia de posiciones con una diferencia de orientaciones, lo que requiere una longitud que una pose no tiene, y las distancias que admite un cuerpo rígido, como la distancia de Hausdorff de las formas ubicadas, el mayor desplazamiento de un punto del cuerpo o el área de su diferencia simétrica, ordenan los pares de valores de manera distinta. Las trayectorias de los puntos de referencia de dos geometrías rígidas temporales, los orígenes de sus marcos locales, se comparan con las funciones de similitud de los puntos temporales sobre su conversión a `tgeompoint`.

- Compara las trayectorias de los puntos de referencia de dos geometrías rígidas temporales

```
SELECT round(frechetDistance(
  tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-03]'),
  tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 2),0)@2001-01-03]')), 6);
-- 2.828427
```

14.13. Operaciones de caja delimitadora

Los operadores topológicos y de posición comparan las cajas delimitadoras espaciotemporales de los operandos; ambos pueden ser una `trgeometry`, un `stbox`, una `geometry` o un `tstzspan`.

- Operadores topológicos estándar sobre cajas delimitadoras

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]' &&
       stbox 'STBOX X((5, -1), (15, 1))';
-- t
```

- Operadores de posición espacial y temporal

  El conjunto completo `<<`, `>>`, `&<`, `&>`, `<<|`, `&<|`, `|>>`, `|&>`, `<</`, `&</`, `/>>`, `/&>`, `<<#`, `&<#`, `#>>`, `#&>` acepta `trgeometry` en cualquiera de los lados. Los operadores `<</`, `&</`, `/>>` y `/&>` comparan el eje Z y requieren que ambos argumentos tengan dimensión Z.

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01,
  Pose(Point(3 0),0)@2001-01-02]' << stbox 'STBOX X((5,5),(6,6))';
-- true
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01,
  Pose(Point(3 0),0)@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';
-- true
SELECT trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(3 0 0),1,0,0,0)@2001-01-02]'
  <</ stbox 'STBOX Z((5,5,5),(6,6,6))';
-- true
SELECT trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(3 0 0),1,0,0,0)@2001-01-02]'
  /&> stbox 'STBOX Z((5,5,5),(6,6,6))';
-- false
```

14.14. Relaciones siempre y alguna vez

- Relaciones siempre y alguna vez

```
eContains({geometry,trgeometry},{geometry,trgeometry}) → boolean
aContains({geometry,trgeometry},{geometry,trgeometry}) → boolean
eCovers({geometry,trgeometry},{geometry,trgeometry}) → boolean
aCovers({geometry,trgeometry},{geometry,trgeometry}) → boolean
eDisjoint({geometry,trgeometry},{geometry,trgeometry}) → boolean
aDisjoint({geometry,trgeometry},{geometry,trgeometry}) → boolean
eDwithin({geometry,trgeometry},{geometry,trgeometry},float) → boolean
aDwithin({geometry,trgeometry},{geometry,trgeometry},float) → boolean
eIntersects({geometry,trgeometry},{geometry,trgeometry}) → boolean
aIntersects({geometry,trgeometry},{geometry,trgeometry}) → boolean
eTouches({geometry,trgeometry},{geometry,trgeometry}) → boolean
aTouches({geometry,trgeometry},{geometry,trgeometry}) → boolean
```

Las relaciones *alguna vez* determinan si la relación se cumple en algún instante, y las relaciones *siempre* si se cumple en todos los instantes. Una geometría rígida temporal las responde sobre el cuerpo que su geometría de referencia sitúa en cada instante, de modo que la relación leída es la del cuerpo situado y no la de la geometría de referencia en el origen. Con interpolación lineal el cuerpo se mueve entre sus instantes, y `eIntersects`, `aIntersects`, `eDisjoint` y `aDisjoint` leen ese movimiento mediante la distancia temporal, por lo que necesitan lo que necesita la distancia: un cuerpo convexo en dos dimensiones, frente a una geometría cuyos polígonos son convexos y sin agujeros. Las demás relaciones aún no leen el movimiento entre los instantes. Estas relaciones aún no siguen Sección 8.3: un operando de otra dimensión da un error. Para un valor en tres dimensiones, `eIntersects`, `eDisjoint`, `aDisjoint`, `eContains`, `eCovers` y `eTouches` aún no leen Z, de modo que una geometría sobre el cuerpo puede responder como si lo tocara, y `aContains` y `aCovers` aún no admiten una geometría con Z en todas sus formas.

```
SELECT eIntersects(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 3),0)@2001-01-03]',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- true
SELECT aIntersects(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 3),0)@2001-01-03]',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- false
```

`eDwithin` y `aDwithin` utilizan la distancia temporal de Sección 14.11, con los mismos requisitos sobre las geometrías. En el ejemplo, un cuadrado gira junto a un punto. Está a distancia 2 del punto en los dos instantes, pero se acerca a menos de 1.7 del punto entre ellos.

```
SELECT eDwithin(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)', 1.7);
-- true
SELECT aDwithin(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)', 1.7);
-- false
```

14.15. Relaciones espaciotemporales

- Relaciones espaciotemporales

```
tContains({geometry,trgeometry},{geometry,trgeometry}) → tbool
tCovers({geometry,trgeometry},{geometry,trgeometry}) → tbool
tDisjoint({geometry,trgeometry},{geometry,trgeometry}) → tbool
tDwithin({geometry,trgeometry},{geometry,trgeometry},float) → tbool
```

```
tIntersects({geometry,trgeometry},{geometry,trgeometry}) → tbool
tTouches({geometry,trgeometry},{geometry,trgeometry}) → tbool
```

Las relaciones *temporales* calculan la relación topológica o de distancia en cada instante y dan como resultado un `tbool`. Una geometría rígida cubre un área, por lo que todas las relaciones se declaran en ambas direcciones y entre dos geometrías rígidas temporales. Por tanto, un cuerpo que toca a otro solo entre dos instantes de un valor lineal responde falso en ambos. Para un valor en tres dimensiones, `tIntersects` y `tDisjoint` aún no responden correctamente, pues un punto dentro del cuerpo en un instante se indica como que no lo toca, y `tContains`, `tCovers` y `tTouches` aún no se admiten.

```
SELECT tIntersects(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 3),0)@2001-01-03]',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- [t@2001-01-01, f@2001-01-03]
```

`tDwithin` utiliza la distancia temporal de Sección 14.11, con los mismos requisitos sobre las geometrías. Su resultado cambia en los instantes exactos en los que la distancia cruza la distancia dada. En el ejemplo, un cuadrado gira un cuarto de vuelta junto a un punto. Está a menos de 1.7 del punto desde las 07:26:20 hasta las 16:33:39.

```
SELECT tDwithin(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)', 1.7);
-- {[f@2001-01-01, t@2001-01-01 07:26:20.193911, t@2001-01-01 16:33:39.806089],
--  (f@2001-01-01 16:33:39.806089, f@2001-01-02]}
```

14.16. Comparaciones

Los operadores de igualdad y de siempre / alguna vez comparan dos geometrías rígidas temporales sobre la geometría materializada: tanto la forma de referencia como la trayectoria de pose deben coincidir en cada instante compartido.

- Comparaciones tradicionales

```
trgeometry {=, <>, <, >, <=, >=} trgeometry → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]' =
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- t
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),0.0)@2001-01-02]' <>
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- f
```

- Comparaciones alguna vez y siempre

```
{geometry,trgeometry} {?=, ?<>, %=, %<>} {geometry,trgeometry} → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01,
  Pose(Point(5 0), 0.0)@2001-01-02]' ?= geometry 'Polygon((5 0,6 0,6 1,5 1,5 0))';
-- t
```

- Comparaciones temporales

```
{geometry,trgeometry} {#=, #<>} {geometry,trgeometry} → tbool
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
  0.0)@2001-01-02]' %= trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- t
```

14.17. Mosaicos

Estas funciones devuelven las cajas espaciotemporales de una cuadrícula multidimensional que la geometría rígida temporal interseca, calculadas sobre su trayectoria del centroide.

- Devuelve las cajas espaciotemporales de una geometría rígida temporal dividida con respecto a una cuadrícula espacial, temporal o espaciotemporal

```
spaceBoxes(trgeometry,xsize float [, ysize float [, zsize float]],
  sorigin geometry='Point(0 0 0)',bitmatrix boolean=true,
  borderInc boolean=true) → stbox[]
timeBoxes(trgeometry,duration interval,torigin timestamptz='2000-01-03',
  bitmatrix boolean=true,borderInc boolean=true) → stbox[]
spaceTimeBoxes(trgeometry,xsize float [, ysize float [, zsize float]],duration interval,
  sorigin geometry='Point(0 0 0)',torigin timestamptz='2000-01-03',
  bitmatrix boolean=true,borderInc boolean=true) → stbox[]
```

```
SELECT array_length(spaceBoxes( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]', 2.0), 1);
-- 6
```

14.18. Cajas delimitadoras

Estas funciones descomponen una geometría rígida temporal en las cajas delimitadoras o los lapsos de tiempo de sus segmentos o de un número solicitado de contenedores.

- Devuelve el arreglo de cajas espaciotemporales o lapsos de tiempo de los segmentos de una geometría rígida temporal

```
stboxes(trgeometry) → stbox[]
spans(trgeometry) → tstzspan[]
```

```
SELECT stboxes(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {"STBOX XT(((0,0),(11,1)),[2001-01-01, 2001-01-02])"}
SELECT spans(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {"[2001-01-01, 2001-01-02]"}
```

- Devuelve las cajas espaciotemporales o los lapsos de tiempo de una geometría rígida temporal dividida en un número dado de contenedores, o con un número dado de segmentos por contenedor

```
splitNStboxes(trgeometry,integer) → stbox[]
splitEachNStboxes(trgeometry,integer) → stbox[]
splitNSpans(trgeometry,integer) → tstzspan[]
splitEachNSpans(trgeometry,integer) → tstzspan[]
```

```
SELECT splitNStboxes(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 2);
-- {"STBOX XT(((0,0),(11,1)),[2001-01-01, 2001-01-02])"}
SELECT splitNSpans(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 2);
-- {"[2001-01-01, 2001-01-02]"}
```

14.19. Agregaciones

- Número de geometrías rígidas temporales solapadas en cada instante, equivalente a `tCount` sobre la tpose subyacente

```
tCount(trgeometry) → tint
```

```
SELECT tCount(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {[1@2001-01-01, 1@2001-01-02]}
```

- Caja delimitadora agregada de una columna de geometrías rígidas temporales

```
extent(trgeometry) → stbox
```

```
SELECT extent(temp) FROM tbl_trgeometry;
```

14.20. Indexación

Hay tres tipos de índices disponibles sobre columnas `trgeometry`. Cada uno utiliza la caja delimitadora espaciotemporal de la geometría materializada y admite consultas topológicas, de posición y de distancia KNN.

- `GIST(temp)`, GiST de árbol R.
- `SPGIST(temp trgeometry_quadtree_ops)`, SP-GiST de árbol cuádruple.
- `SPGIST(temp trgeometry_kdtree_ops)`, SP-GiST de árbol kd.

```
CREATE INDEX tbl_trgeometry_gist_idx ON tbl_trgeometry USING GIST(temp);
-- Topological lookup
SELECT count(*) FROM tbl_trgeometry WHERE temp && stbox 'STBOX XT(((0, 0),
  (10, 10)),[2001-01-01, 2001-01-02])';
-- KNN (ordered scan)
SELECT mmsi FROM tbl_trgeometry ORDER BY temp <-> trgeometry 'Polygon((0 0,1 0,1 1,
  0 1,0 0));Pose(Point(100 100), 0.0)@2001-01-01' LIMIT 10;
```

La variante de árbol kd suele ganar en cargas de trabajo dominadas por consultas KNN con una cota de ordenación ajustada; la variante de árbol cuádruple es más uniforme. El GiST de árbol R es el valor predeterminado seguro para cargas de trabajo mixtas de lectura/escritura.

Capítulo 15

Tipos JSON en MobilityDB

PostgreSQL proporciona dos tipos para almacenar datos JSON: `json` y `jsonb`. El tipo `json` almacena una copia exacta del texto de entrada, que debe volver a analizarse cada vez que se procesa. Por otro lado, el tipo `jsonb` almacena los datos JSON en un formato binario descompuesto que hace que su entrada sea ligeramente más lenta debido a la sobrecarga de conversión añadida, pero significativamente más rápida de procesar, ya que no es necesario volver a analizarla. El tipo `jsonb` también admite indexación.

En MobilityDB, los tipos `textset` y `ttext` se utilizan para representar, respectivamente, conjuntos de valores JSON y valores JSON temporales. Además, el tipo `jsonb` sirve como tipo base para definir los tipos `jsonbset` y `tjsonb`. La mayoría de las funciones y operadores descritos en los capítulos anteriores para los tipos conjunto y temporal también son aplicables a los tipos JSON correspondientes. Además, hay funciones específicas definidas para estos tipos, que se derivan de las funciones correspondientes de los tipos `json` y `jsonb`.

En este capítulo, describimos los tipos JSON de MobilityDB y sus operaciones asociadas. Remitimos a la documentación de PostgreSQL para una explicación detallada de los tipos `json` y `jsonb` y su funcionalidad. Hemos buscado habilitar la misma sintaxis de las funciones originales de PostgreSQL para los tipos correspondientes de MobilityDB, como se ilustra en este documento.

Considere por ejemplo el operador `->`, que extrae un campo de objeto JSON con una clave dada.

```
SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10
```

Aplicar el mismo operador a un conjunto JSONB y a un valor JSONB temporal produce los siguientes resultados

```
SELECT jsonbset '{"{\"unit\": \"km\", \"speed\": 10}",
  "{\"unit\": \"km\", \"speed\": 20}"}' -> text 'speed';
-- {10, 20}
SELECT tjsonb '[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
```

que se obtienen aplicando el operador de PostgreSQL `->` a cada elemento del conjunto JSONB y a cada instante del valor JSONB temporal.

15.1. Operaciones sobre valores JSONB

PostgreSQL proporciona las siguientes operaciones sobre valores JSONB con sus propios nombres y operadores. MobilityDB las nombra como sus equivalentes sobre conjuntos JSONB y JSONB temporales, de modo que una consulta que utiliza estos nombres se ejecuta sin cambios en todos los motores del ecosistema (Sección 18.8). Los operadores de PostgreSQL conservan sus símbolos.

- Extrae un campo de objeto JSON especificado por una clave

```
jsonbObjectField(jsonb,text) → jsonb
jsonbObjectFieldText(jsonb,text) → text
```

```
SELECT jsonbObjectField(jsonb '{"a": {"b": 1}}', 'a');
-- {"b": 1}
SELECT jsonbObjectFieldText(jsonb '{"a": "x"}', 'a');
-- x
```

- Extrae un valor JSON especificado por una ruta

```
jsonbExtractPath(jsonb,path text[]) → jsonb
jsonbExtractPathText(jsonb,path text[]) → text
```

```
SELECT jsonbExtractPath(jsonb '{"a": {"b": [10, 20]}}', ARRAY['a', 'b', '1']);
-- 20
SELECT jsonbExtractPathText(jsonb '{"a": {"b": "z"}}', ARRAY['a', 'b']);
-- z
```

- Extrae un elemento de un arreglo JSON

```
jsonbArrayElement(jsonb,integer) → jsonb
jsonbArrayElementText(jsonb,integer) → text
```

Los elementos se numeran desde cero, y un entero negativo cuenta desde el final del arreglo.

```
SELECT jsonbArrayElement(jsonb '[1, [2, 3]]', 1);
-- [2, 3]
SELECT jsonbArrayElement(jsonb '[1, 2, 3]', -1);
-- 3
SELECT jsonbArrayElementText(jsonb '["x", "y"]', 0);
-- x
```

- Devuelve el número de elementos de un arreglo JSON

```
jsonbArrayLength(jsonb) → integer
```

```
SELECT jsonbArrayLength(jsonb '[1, 2, 3]');
-- 3
```

- Devuelve los elementos de un arreglo JSON como filas

```
jsonbArrayElements(jsonb) → setof jsonb
jsonbArrayElementsText(jsonb) → setof text
```

```
SELECT * FROM jsonbArrayElements(jsonb '[1, "two", {"three": 3}]');
-- 1
-- "two"
-- {"three": 3}
SELECT * FROM jsonbArrayElementsText(jsonb '[1, "two"]');
-- 1
-- two
```

- Devuelve los campos de un objeto JSON como filas de una clave y un valor

```
jsonbEach(jsonb) → setof (key text,value jsonb)
jsonbEachText(jsonb) → setof (key text,value text)
```

```
SELECT * FROM jsonbEach(jsonb '{"a": 1, "b": [2]}') ORDER BY key;
-- a | 1
-- b | [2]
SELECT * FROM jsonbEachText(jsonb '{"a": 1, "b": "x"}') ORDER BY key;
-- a | 1
-- b | x
```

- Devuelve las claves de un objeto JSON como filas

```
jsonbObjectKeys(jsonb) → setof text
```

```
SELECT * FROM jsonbObjectKeys(jsonb '{"b": 1, "a": 2}') ORDER BY 1;
-- a
-- b
```

- Devuelve el texto indentado de un valor JSON

```
jsonbPretty(jsonb) → text
```

```
SELECT jsonbPretty(jsonb '{"a": 1}');
-- {
--     "a": 1
-- }
```

- Concatena dos valores JSON

```
jsonbConcat(jsonb,jsonb) → jsonb
```

```
SELECT jsonbConcat(jsonb '{"a": 1}', jsonb '{"b": 2}');
-- {"a": 1, "b": 2}
```

- Elimina de un valor JSON una clave, las claves de un arreglo, un elemento de arreglo o el elemento en una ruta

```
jsonbDelete(jsonb,text) → jsonb
jsonbDeleteArray(jsonb,text[]) → jsonb
jsonbDeleteIndex(jsonb,integer) → jsonb
jsonbDeletePath(jsonb,path text[]) → jsonb
```

```
SELECT jsonbDelete(jsonb '{"a": 1, "b": 2}', 'a');
-- {"b": 2}
SELECT jsonbDeleteArray(jsonb '{"a": 1, "b": 2, "c": 3}', ARRAY['a', 'c']);
-- {"b": 2}
SELECT jsonbDeleteIndex(jsonb '[1, 2, 3]', -1);
-- [1, 2]
SELECT jsonbDeletePath(jsonb '{"a": {"b": 1, "c": 2}}', ARRAY['a', 'b']);
-- {"a": {"c": 2}}
```

- ¿Existe un texto, o alguno o todos los textos de un arreglo, como clave de nivel superior o elemento de arreglo de un valor JSON?

```
jsonbExists(jsonb,text) → boolean
jsonbExistsAny(jsonb,text[]) → boolean
jsonbExistsAll(jsonb,text[]) → boolean
```

```
SELECT jsonbExists(jsonb '{"a": 1}', 'a');
-- true
SELECT jsonbExistsAny(jsonb '{"a": 1}', ARRAY['b', 'a']);
-- true
SELECT jsonbExistsAll(jsonb '{"a": 1}', ARRAY['b', 'a']);
-- false
```

- ¿Contiene el primer valor JSON al segundo, o está contenido en él?

```
jsonbContains(jsonb,jsonb) → boolean
jsonbContained(jsonb,jsonb) → boolean
```

```
SELECT jsonbContains(jsonb '{"a": 1, "b": 2}', jsonb '{"a": 1}');
-- true
SELECT jsonbContained(jsonb '{"a": 1}', jsonb '{"a": 1, "b": 2}');
-- true
```

- Devuelve un valor JSON con el elemento en una ruta establecido en un valor

```
jsonbSet(jsonb,path text[],val jsonb,create_missing boolean=true) → jsonb
jsonbSetLax(jsonb,path text[],val jsonb,create_missing boolean=true,
  handle_null text='use_json_null') → jsonb
```

El elemento se añade cuando no existe y `create_missing` es verdadero. La función `jsonbSetLax` trata un valor NULL según `handle_null`, que es uno de `'raise_exception'`, `'use_json_null'`, `'delete_key'` o `'return_target'`.

```
SELECT jsonbSet(jsonb '{"a": 1}', ARRAY['b'], jsonb '2');
-- {"a": 1, "b": 2}
SELECT jsonbSet(jsonb '{"a": 1}', ARRAY['b'], jsonb '2', false);
-- {"a": 1}
SELECT jsonbSetLax(jsonb '{"a": 1}', ARRAY['a'], NULL);
-- {"a": null}
SELECT jsonbSetLax(jsonb '{"a": 1}', ARRAY['a'], NULL, true, 'delete_key');
-- {}
```

- Devuelve un valor JSON con un valor insertado en una ruta

```
jsonbInsert(jsonb,path text[],val jsonb,after boolean=false) → jsonb
```

```
SELECT jsonbInsert(jsonb '[1, 3]', ARRAY['1'], jsonb '2');
-- [1, 2, 3]
SELECT jsonbInsert(jsonb '[1, 3]', ARRAY['1'], jsonb '2', true);
-- [1, 3, 2]
```

- Devuelve un valor JSON sin sus campos de objeto cuyo valor es nulo

```
jsonbStripNulls(jsonb,strip_in_arrays boolean=false) → jsonb
```

El último argumento indica si los elementos de arreglo nulos también se eliminan.

```
SELECT jsonbStripNulls(jsonb '{"a": null, "b": [1, null]}');
-- {"b": [1, null]}
SELECT jsonbStripNulls(jsonb '{"a": null, "b": [1, null]}', true);
-- {"b": [1]}
```

- ¿Devuelve la ruta JSON algún elemento para un valor JSON?

```
jsonbPathExists(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
jsonbPathExistsTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
```

El último argumento suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos; el resultado de un error suprimido es NULL. Las funciones que terminan en `Tz` comparan valores de fecha y hora entre zonas horarias.

```
SELECT jsonbPathExists(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ > 2)');
-- true
SELECT jsonbPathExists(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ > $x)', '{"x": 5}');
-- false
```

- Devuelve el resultado de una comprobación de predicado de ruta JSON para un valor JSON

```
jsonbPathMatch(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
jsonbPathMatchTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
```

```
SELECT jsonbPathMatch(jsonb '{"a": [1, 2, 3]}', 'exists($.a[*] ? (@ > 2))');
-- true
```

- Devuelve los elementos que una ruta JSON devuelve para un valor JSON, como filas, como un arreglo JSON, o el primero de ellos

```
jsonbPathQuery(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → setof jsonb
jsonbPathQueryTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → setof jsonb
jsonbPathQueryArray(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
jsonbPathQueryArrayTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
jsonbPathQueryFirst(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
jsonbPathQueryFirstTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
```

```
SELECT * FROM jsonbPathQuery(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ >= 2)');
-- 2
-- 3
SELECT jsonbPathQueryArray(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ >= 2)');
-- [2, 3]
SELECT jsonbPathQueryFirst(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ >= 2)');
-- 2
```

- Compara dos valores JSON, o devuelve el valor hash de un valor JSON

```
jsonb {=, <>, <, <=, >=, >} jsonb → boolean
cmp(jsonb,jsonb) → integer
hash(jsonb) → integer
hashExtended(jsonb,seed bigint) → bigint
```

Los operadores son los de PostgreSQL, y las funciones `eq`, `ne`, `lt`, `le`, `ge` y `gt` responden como ellos, junto con `cmp`, que devuelve -1, 0 o 1. Las funciones `hash` y `hashExtended` responden como las funciones `jsonb_hash` y `jsonb_hash_extended` de PostgreSQL.

```
SELECT eq(jsonb '{"a": 1}', jsonb '{"a": 1}');
-- true
SELECT cmp(jsonb '1', jsonb '2');
-- -1
```

15.2. Operaciones sobre conjuntos JSONB

Al manipular colecciones JSON de estructura variable, puede ocurrir que un elemento esté definido en algunos de los documentos pero no en todos. PostgreSQL proporciona el *modo lax* para este propósito, donde el argumento `null_value_treatment` determina el comportamiento en el caso en que una función devuelve un valor NULL. El argumento puede tomar uno de los siguientes valores: `'raise_exception'`, `'use_json_null'`, `'delete_key'`, `'return_target'`, donde `'use_json_null'` es el valor por defecto. En PostgreSQL el modo lax se admite solo para operaciones de *actualización* (no de *consulta*) con la función `jsonb_set_lax`. En MobilityDB, hemos mantenido la misma semántica para la función correspondiente `jsonbsetSetLax`, pero habilitamos un comportamiento similar para *todas* las operaciones JSON que pueden devolver un valor nulo, salvo que reemplazamos el valor `'return_target'` por `'return_null'`, ya que el primero no tiene sentido para operaciones sobre conjuntos. Para operadores como `->`, se utiliza el valor por defecto `'use_json_null'` y no se puede cambiar, mientras que para las funciones correspondientes `jsonbsetObjectField`, el último argumento especifica el comportamiento en el caso en que la función devuelve NULL. Ilustramos este comportamiento para la función `jsonbsetObjectField` a continuación.

- Extrae un campo de objeto JSON especificado por una clave

```
{textset,jsonbset} -> text → {textset,jsonbset}
jsonbset ->> text → textset
jsonbsetObjectField(jsonbset,text,null_handle text='use_json_null') → jsonbset
jsonbsetObjectFieldText(jsonbset,text,null_handle text='use_json_null') → textset
```

```
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km", "speed": 20}']) -> text 'speed';
-- {10, 20}
SELECT set(ARRAY[jsonb '{"position":"Point(1 1)", "speed":10}',
  '{"position":"Point(2 2)", "speed":20}']) ->> text 'position';
-- {"Point(1 1)", "Point(2 2)"}
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10}', '{"unit": "km"}',
  '{"unit": "km", "speed": 10}']) -> text 'speed';
-- {"null", 10}
```

Mostramos a continuación el comportamiento de la función `jsonbsetObjectField` para los posibles valores del argumento `null_handle`. Todas las funciones JSON que pueden devolver un valor NULL se comportan de forma similar en MobilityDB.

```
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'raise_exception');
-- ERROR:  The lifted operation returned NULL
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'use_json_null');
-- {"null", 10}
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'delete_key');
-- {10}
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'return_null');
-- NULL
```

- Extrae un campo de objeto JSON especificado por una ruta

```
{textset,jsonbset} #> text[] → {textset,jsonbset}
jsonbsetExtractPath(jsonbset,path text[],null_handle text='use_json_null') → jsonbset
jsonbsetExtractPathText(jsonbset,path text[],null_handle text='use_json_null') → textset
```

```
SELECT set(ARRAY[jsonb '{"speed": {"unit": "km", "value": 10}}',
  '{"speed": {"unit": "km", "value": 20}}']) #> ARRAY[text 'speed', 'value'];
-- {10, 20}
SELECT set(ARRAY[jsonb '{"position":"Point(1 1)", "PoIs":["Grand Place", "La Bourse"]}',
  '{"position":"Point(2 2)", "PoIs":["Palais Royal", "Manneken Pis"]}']) 
  #>> ARRAY[text 'PoIs', '1'];
-- {"La Bourse", "Manneken Pis"}
```

- Extrae un elemento de un arreglo JSON

```
jsonbsetArrayElement(jsonbset,integer,null_handle text='use_json_null') → jsonbset
jsonbsetArrayElementText(jsonbset,integer,null_handle text='use_json_null') → textset
```

```
SELECT jsonbsetArrayElement(set(ARRAY[jsonb '["Grand Place", "La Bourse"]',
  '["Palais Royal", "Manneken Pis"]']), 0);
-- {"\"Grand Place\"", "\"Palais Royal\""}
SELECT jsonbsetArrayElement(set(ARRAY[jsonb '["Grand Place", "La Bourse"]',
  '["Palais Royal", "Manneken Pis"]']), 1);
-- {"\"La Bourse\"", "\"Manneken Pis\""}
```

- Extrae un valor alfanumérico de un conjunto JSONB dado por una clave

```
intset(jsonbset,text,null_handle text='raise_exception') → tint
floatset(jsonbset,text,null_handle text='raise_exception') → tfloat
textset(jsonbset,text,null_handle text='raise_exception') → textset
```

Tenga en cuenta que el valor `use_json_null` no puede utilizarse para las funciones anteriores y por lo tanto se utiliza el valor por defecto `'raise_exception'`.

```
SELECT intset(jsonbset '{"{\"speed\": 10, \"units\": \"km/h\"}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}', 'speed');
-- {10, 20}
SELECT floatset(jsonbset '{"{\"speed\": 10, \"units\": \"km/h\"}",
  "{\"speed\": 20, \"units\": \"km/h\"}", "{\"speed\": 25}"}', 'speed');
-- {10, 20, 25}
SELECT textset(jsonbset '{"{\"road\": \"Bvd Gén. Jacques\", \"category\": \"primary\"}",
  "{\"road\": \"Bvd de la Cambre\", \"category\": \"residential\"}"}', 'category');
-- {"primary", "residential"}
```

- Concatenación de un conjunto JSONB

```
{jsonb,jsonbset} || {jsonbset,jsonb} → jsonbset
```

El valor se concatena con cada elemento del conjunto, y el orden de los operandos es libre.

```
SELECT set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']) || '{"unit":"km"}'::jsonb;
-- {"{\"unit\": \"km\", \"speed\": 10}", "{\"unit\": \"km\", \"speed\": 20}"}
SELECT jsonb '{"unit":"km"}' || set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']);
-- {"{\"unit\": \"km\", \"speed\": 10}", "{\"unit\": \"km\", \"speed\": 20}"}
```

- Eliminación en un conjunto JSONB

```
jsonbset - {integer,text,text[]} → jsonbset
jsonbset #- text[] → jsonbset
```

```
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km", "speed": 20}']) - text 'unit';
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
SELECT set(ARRAY[jsonb '["Grand Place", "La Bourse"]',
  '["Palais Royal", "Manneken Pis"]']) - 1;
-- {"[\"Grand Place\"]", "[\"Palais Royal\"]"}
SELECT set(ARRAY[jsonb '{"location":"Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]}',
  '{"location":"Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]}']
  #- ARRAY[text 'PoIs', '1'];
/* {"{\"PoIs\": [\"Grand Place\"], \"location\": \"Point(1 1)\"}", "{\"PoIs\":
   [\"Palais Royal\"], \"location\": \"Point(2 2)\"}"} */
```

Como se muestra a continuación, el resultado de una operación de eliminación puede ser un registro vacío o un arreglo vacío. Para eliminar estos valores, puede utilizarse la función `setMinus`.

```
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10, "light": true}',
  '{"unit": "km", "speed": 20}']) - ARRAY[text 'unit', 'speed'];
-- {"{}", "{\"light\": true}"}
SELECT set(ARRAY[jsonb '["Grand Place", "La Bourse"]', '["Palais Royal"]']) - 0;
-- {[], "[\"La Bourse\"]"}
```

```
SELECT setMinus(set(ARRAY[jsonb '{"unit": "km", "speed": 10, "light": true}',
  '{"unit": "km", "speed": 20}']) - ARRAY[text 'unit', 'speed'],
  set(ARRAY[jsonb '{}', '[]']));
-- {"{\"light\": true}"}
SELECT setMinus(set(ARRAY[jsonb '["Grand Place", "La Bourse"]', '["Palais Royal"]']) - 0,
  set(ARRAY[jsonb '{}', '[]']));
-- {"[\"La Bourse\"]"}
```

- Existencia en un conjunto JSONB

```
jsonbset ? text → boolean[]
jsonbset {?|, ?&} text[] → boolean[]
```

Como en PostgreSQL, los operadores `?|` y `?&` comprueban, respectivamente, si *alguna* o *todas* las cadenas del arreglo de texto existen como claves de nivel superior o elementos de arreglo. El resultado es un booleano por cada valor del conjunto, en el orden de los valores del conjunto.

```
SELECT jsonbset '{"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}';
-- {"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' ? text 'units';
-- {f,t}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' ?| ARRAY[text 'units', text 'lights'];
-- {f,t}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' ?& ARRAY[text 'speed', text 'units'];
-- {f,t}
```

- Asignación en un conjunto JSONB

```
jsonbsetSet(jsonbset,path text[],val jsonb,create_missing boolean=true) → jsonbset
jsonbsetSetLax(jsonbset,path text[],val jsonb,create_missing boolean=true,
  handle_null text='use_json_null') → jsonbset
```

Devuelve el valor `jsonbset` con el elemento especificado por la ruta reemplazado por `jsonb`, o añadido si `create` es verdadero y el elemento no existe. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, y create_if_missing es verdadero, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo. La función `jsonbsetSetLax` se comporta de forma idéntica a `jsonbsetSet` si el valor dado no es NULL; de lo contrario, se comporta según el valor de `handle_null`, que debe ser uno de `'raise_exception'`, `'use_json_null'`, `'delete_key'` o `'return_target'`. Como se indicó anteriormente, mantuvimos el comportamiento de PostgreSQL para esta función habilitando el valor `'return_target'`, mientras que para todas las demás operaciones de *consulta*, este valor se ha reemplazado por `'return_null'`.

```
SELECT jsonbsetSet(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb);
-- {"{\"speed\": 10, \"units\": \"km/h\"}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbsetSet(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb, false);
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
SELECT jsonbsetSet(set(ARRAY[jsonb '{"speed": 10, "units": "km/h"}',
  '{"speed": 20, "units": "km/h"}']), ARRAY['units'], '"mi/h"'::jsonb);
-- {"{\"speed\": 10, \"units\": \"mi/h\"}", "{\"speed\": 20, \"units\": \"mi/h\"}"}
SELECT jsonbsetSetLax(set(ARRAY[jsonb '{"speed": 10, "units": "km/h"}', '{"speed": 20}']),
  ARRAY['units'], NULL, true, 'delete_key');
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
```

- Devuelve el valor `jsonbset` con `jsonb` insertado

```
jsonbsetInsert(jsonbset,path text[],val jsonb,after boolean=false) → jsonbset
```

Si el elemento especificado por la ruta es un elemento de arreglo, el nuevo valor se insertará antes de ese elemento si `after` es falso, o después en caso contrario. Si el elemento especificado por la ruta es un campo de objeto, el nuevo valor se insertará solo si el objeto no contiene ya esa clave. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo.

```
SELECT jsonbsetInsert(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb);
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
SELECT jsonbsetInsert(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb, false);
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
SELECT jsonbsetInsert(set(ARRAY[jsonb '{"speed": 10, "units": "km/h"}',
  '{"speed": 20, "units": "km/h"}']), ARRAY['units'], '"mi/h"'::jsonb);
-- {"{\"speed\": 10, \"units\": \"mi/h\"}", "{\"speed\": 20, \"units\": \"mi/h\"}"}
```

- Devuelve un conjunto JSONB sin nulos

```
jsonbsetStripNulls(jsonbset,strip_in_arrays boolean=false) → jsonbset
```

El último argumento indica si los elementos de arreglo nulos también se eliminan. Los valores nulos simples nunca se eliminan.

```
SELECT jsonbsetStripNulls(set(ARRAY[jsonb '{"speed": 10, "lights": null}',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}']));
/* {"{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\", null], \"speed\":
   20}"} */
SELECT jsonbsetStripNulls(set(ARRAY[jsonb '{"speed": 10, "lights": null}',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}']), true);
-- {"{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\"], \"speed\": 20}"}
SELECT jsonbsetStripNulls(set(ARRAY[jsonb
  '{"road": "Bvd Gén. Jacques", "category": "primary"}',
  '{"road": "Bvd de la Cambre", "category": "primary"}',
  '{"road": "rue de l''Abbaye", "category": null}']));
/* {"{\"road\": \"rue de l'Abbaye\"}", "{\"road\": \"Bvd Gén. Jacques\", \"category\":
   \"primary\"}", "{\"road\": \"Bvd de la Cambre\", \"category\": \"primary\"}"} */
```

Como se muestra a continuación, la función no elimina los valores nulos simples. Para este propósito puede utilizarse la función `setMinus`.

```
SELECT jsonbsetStripNulls(set(ARRAY[jsonb '{"speed": 10}', 'null',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}']));
/* {"null", "{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\"], \"speed\":
   20}"} */
SELECT setMinus(set(ARRAY[jsonb '{"speed": 10}', 'null',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}']), set(ARRAY[jsonb 'null']));
-- {"{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\"], \"speed\": 20}"}
```

- ¿Devuelve la ruta JSON algún elemento para el conjunto JSONB especificado?

```
jsonbset @? jsonpath → boolean[]
jsonbsetPathExists(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
jsonbsetPathExistsTz(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
```

El operador `@?` suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable. El resultado es un booleano por cada valor del conjunto, en el orden de los valores del conjunto.

```
SELECT jsonbset '{"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}';
-- {"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' @? '$.units';
-- {f,t}
SELECT jsonbsetPathExists(jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}', '$.speed ? (@ > $min)', '{"min": 15}');
-- {f,t}
```

- Devuelve el resultado de una comprobación de predicado de ruta JSON para un conjunto JSONB

```
jsonbset @@ jsonpath → boolean[]
jsonbsetPathMatch(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
jsonbsetPathMatchTz(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
```

El operador `@@` suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable. El resultado es un booleano por cada valor del conjunto, en el orden de los valores del conjunto.

```
SELECT jsonbset '{"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}';
-- {"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' @@ '$.speed > 15';
-- {f,t}
SELECT jsonbsetPathMatch(jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}', '$.speed > $min', '{"min": 15}');
-- {f,t}
```

- Devuelve todos los elementos devueltos por una ruta JSON de un conjunto JSONB, como un arreglo JSON

```
jsonbsetPathQueryArray(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
jsonbsetPathQueryArrayTz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

La función anterior puede suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT jsonbsetPathQueryArray(set(ARRAY[jsonb '{"speed":10}',
  '{"speed": 20, "units": "km/h"}']), '$ ? (@.speed >= $min && @.speed <= $max)',
  '{"min":10, "max":30}');
-- {"[{\"speed\": 10}]", "[{\"speed\": 20, \"units\": \"km/h\"}]"}
SELECT jsonbsetPathQueryArrayTz(set(ARRAY[jsonb
  '{"cameraId":25, "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}',
  '{"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}']),
  '$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
   @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {"[\"2001-01-07\"]", "[\"2001-01-06\", \"2001-01-09\"]"}
```

- Devuelve el primer elemento devuelto por una ruta JSON de un conjunto JSONB

```
jsonbsetPathQueryFirst(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
jsonbsetPathQueryFirstTz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

Las funciones anteriores pueden suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT jsonbsetPathQueryFirst(set(ARRAY[jsonb '{"speed":10}',
  '{"speed": 20, "units": "km/h"}']), '$.speed ? (@ >= $min && @ <= $max)',
  '{"min":10, "max":30}');
-- {10, 20}
SELECT jsonbsetPathQueryArrayTz(set(ARRAY[jsonb
  '{"cameraId":25, "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}',
  '{"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}']),
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {"[\"2001-01-07\"]", "[\"2001-01-06\", \"2001-01-09\"]"}
```

15.3. Valores JSONB temporales

El tipo temporal `tjsonb` permite representar la evolución en el tiempo de valores `jsonb`. Como todos los tipos temporales, viene en tres subtipos, a saber, instante, secuencia y conjunto de secuencias. A continuación se dan ejemplos de valores `tjsonb` en estos subtipos.

```
-- Instant
SELECT tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01';
-- "{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01
SELECT tjsonb '{{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02}';
/* {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02} */
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]';
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02],
  [{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-04]}';
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02], ["{\"location\": \"Point(2 2)\",
   \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(3 3)\", \"vehicleId\":
   1}"@2001-01-04]} */
```

Como puede verse arriba, para el subtipo `Instant` necesitamos encerrar el valor JSONB entre comillas y escapar las comillas internas; de lo contrario, la llave de apertura del valor JSONB se interpretaría como el comienzo de un subtipo `Sequence` con interpolación discreta.

El tipo `tjsonb` acepta modificadores de tipo (o `typmod` en la terminología de PostgreSQL) para especificar el subtipo. Los valores posibles para el subtipo son `Instant`, `Sequence` y `SequenceSet`. El argumento es opcional y, si no se especifica para una columna, se permiten valores de cualquier subtipo.

```
SELECT tjsonb(Instant) '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01';
-- "{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01
SELECT tjsonb(Sequence) '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Los valores JSONB temporales de subtipo secuencia o conjunto de secuencias se convierten a una forma normal de modo que valores equivalentes tengan representaciones idénticas. Para ello, los valores de instantes consecutivos se fusionan cuando es posible. Tres valores de instantes consecutivos pueden fusionarse en dos si sus valores JSONB son iguales. Asimismo, dos secuencias consecutivas que son adyacentes y tienen el mismo valor JSONB de final e inicio pueden fusionarse en una sola. A continuación se dan ejemplos de transformación a una forma normal.

```
SELECT asText(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]');
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-03] */
SELECT asText(tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03],
  ({"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-04]}');
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(2 2)\",
   \"vehicleId\": 1}"@2001-01-04]} */
```

15.4. Validez de los valores JSONB temporales

Los valores JSONB temporales deben satisfacer las restricciones especificadas en Sección 4.3 para que estén bien definidos. Se genera un error siempre que no se satisfaga alguna de estas restricciones. A continuación se dan ejemplos de valores incorrectos.

```
-- Null values are not allowed
SELECT tjsonb 'NULL@2001-01-01 08:05:00';
SELECT tjsonb 'Point(0 0)@NULL';
-- Base type is not a JSONB
SELECT tjsonb 'Point(0 0)@2001-01-01 08:05:00';
```

A continuación damos las funciones y operadores para JSONB temporal. La mayoría de las funciones y operadores para tipos temporales descritos en los capítulos anteriores pueden aplicarse a los tipos JSONB temporales. Por lo tanto, en las firmas de las funciones dadas anteriormente, la notación `base` representa un valor `jsonb` y la notación `ttype` representa un valor `tjsonb`. Para evitar redundancia, solo presentamos a continuación algunos ejemplos de estas funciones y operadores para valores JSONB temporales.

15.5. Entrada y salida

- Devuelve la representación de texto conocido (WKT)

```
asText({tjsonb,tjsonb[],jsonb,jsonpath}) → {text,text[]}
```

Sobre un valor JSONB o una ruta JSON, la función devuelve el texto que PostgreSQL escribe para el valor, como `jsonb::text` y `jsonpath::text`.

```
SELECT asText(jsonb '{"b": {"c": [1, {"d": null}]}, "a": "x"}');
-- {"a": "x", "b": {"c": [1, {"d": null}]}}
SELECT asText(jsonpath 'strict $.a[0 to 2].b');
-- strict $."a"[0 to 2]."b"
SELECT asText(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]');
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT asText(ARRAY[tjsonb '"{\"vehicleId\":1, \"location\": \"Point(1 1)\"}"@2001-01-01',
  '"{\"vehicleId\": 1, \"location\": \"Point(2 2)\"}"@2001-01-02']);
/* {"\"{\\\"location\\\": \\\"Point(1 1)\\\", \\\"vehicleId\\\":
   1}\"@2001-01-01","\"{\\\"location\\\": \\\"Point(2 2)\\\", \\\"vehicleId\\\":
   1}\"@2001-01-02"} */
```

Como puede verse arriba, para los arreglos de valores JSONB, los elementos deben encerrarse entre comillas y, por lo tanto, las comillas internas deben escaparse.

- Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)

```
asBinary(tjsonb,endian text='') → bytea
asHexWKB(tjsonb,endian text='') → text
```

El resultado se codifica utilizando la codificación little-endian (NDR) o big-endian (XDR). Si no se especifica ninguna codificación, se utiliza la codificación de la máquina.

```
SELECT asBinary(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 2)\"}"@2001-01-01');
/* \x0142000138000000000000000020000200800008009000000a000000090000106c6f636174696f6e76
   656869636c654964506f696e74283120322900200000000080010000040eba9c21c0000 */
SELECT asHexWKB(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 2)\"}"@2001-01-01');
/* 0142000138000000000000000020000200800008009000000A000000090000106C6F636174696F6E7665
   6869636C654964506F696E74283120322900200000000080010000040EBA9C21C0000 */
```

- Devuelve la representación JSON de características móviles (MF-JSON)

```
asMFJSON(text) → tjsonb
```

```
SELECT asMFJSON(tjsonb '[{"Position": "Point(1 1)"}@2001-01-01,
  {"Position": "Point(2 2)"}@2001-01-02]');
/* {"type":"MovingJsonb","values":[{"Position": "Point(1 1)"},{"Position": "Point(2
   2)"}],"datetimes":["2001-01-01T00:00:00","2001-01-02T00:00:00"],"lower_inc":true,"up
   per_inc":true,"interpolation":"Step"} */
```

- Entrada desde la representación de texto conocido (WKT)

```
tjsonbFromText(text) → tjsonb
jsonbFromText(text) → jsonb
jsonpathFromText(text) → jsonpath
```

Un valor JSONB y una ruta JSON se leen del texto que PostgreSQL lee para ellos, como `text::jsonb` y `text::jsonpath`; las funciones son la inversa de `asText`.

```
SELECT jsonbFromText('{"b": [1, 2], "a": null}');
-- {"a": null, "b": [1, 2]}
SELECT asText(jsonpathFromText('lax $.a.**{1 to last}'));
-- $."a".**{1 to last}
SELECT tjsonbFromText(text '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]');
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
```

- Entrada desde la representación binaria conocida (WKB) o desde la representación hexadecimal binaria conocida (HexWKB)

```
tjsonbFromBinary(bytea) → tjsonb
tjsonbFromHexWKB(text) → tjsonb
```

```
SELECT tjsonbFromBinary(asBinary( tjsonb '[{"vehicleId": 1,
  "location": "Point(1 2)"}@2001-01-01]'));
-- ["{\"location\": \"Point(1 2)\", \"vehicleId\": 1}"@2001-01-01]
SELECT tjsonbFromHexWKB(asHexWKB( tjsonb '[{"vehicleId": 1,
  "location": "Point(1 2)"}@2001-01-01]'));
-- ["{\"location\": \"Point(1 2)\", \"vehicleId\": 1}"@2001-01-01]
```

- Entrada desde la representación JSON de características móviles (MF-JSON)

```
tjsonbFromMFJSON(text) → tjsonb
```

```
SELECT tjsonbFromMFJSON('{"type": "MovingJsonb", "interpolation": "Step",
  "values": [{"Position": "Point(1 1)"}, {"Position": "Point(2 2)"}],
  "datetimes": ["2001-01-01T10:00:00", "2001-01-01T12:00:00"]}');
/* ["{\"Position\": \"Point(1 1)\"}"@2001-01-01 10:00:00, "{\"Position\": \"Point(2
   2)\"}"@2001-01-01 12:00:00] */
```

15.6. Constructores

- Constructor de valores JSONB temporales que tienen un valor constante

```
tjsonb(jsonb,timestamptz) → tjsonbInst
tjsonb(jsonb,tstzset) → tjsonbDiscSeq
tjsonb(jsonb,tstzspan) → tjsonbContSeq
tjsonb(jsonb,tstzspanset) → tjsonbSeqSet
```

```
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}', timestamptz '2001-01-01');
-- "{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
/* {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(1 1)\",
   \"vehicleId\": 1}"@2001-01-05} */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspan '[2001-01-01, 2001-01-02]');
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspanset '{[2001-01-01, 2001-01-02],[2001-01-03, 2001-01-04]}');
/* [["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-02], ["{\"location\": \"Point(1 1)\",
   \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(1 1)\", \"vehicleId\":
   1}"@2001-01-04]} */
```

- Constructor de valores JSONB temporales del subtipo secuencia

```
tjsonbSeq(tjsonbInst[],lowerInc boolean=true,upperInc boolean=true) → tjsonbSeq
```

```
SELECT tjsonbSeq(ARRAY[tjsonb '"{\"vehicleId\": 1,
  \"location\": \"Point(1 1)\"}"@2001-01-01',
  '"{\"vehicleId\": 1, \"location\": \"Point(2 2)\"}"@2001-01-02',
  '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-03']);
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02, "{\"location\": \"Point(1 1)\",
   \"vehicleId\": 1}"@2001-01-03] */
```

- Constructor de valores JSONB temporales del subtipo conjunto de secuencias

```
tjsonbSeqSet(tjsonb[]) → tjsonbSeqSet
tjsonbSeqSetGaps(tjsonbInst[],maxt interval=NULL,maxdist float=NULL) → tjsonbSeqSet
```

```
SELECT tjsonbSeqSet(ARRAY[tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]',
  '[{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-04]']);
/* [["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02], ["{\"location\": \"Point(2 2)\",
   \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(3 3)\", \"vehicleId\":
   1}"@2001-01-04]} */
SELECT tjsonbSeqSetGaps(ARRAY[tjsonb '"{\"vehicleId\": 1,
  \"location\": \"Point(1 1)\"}"@2001-01-01',
  '"{\"vehicleId\": 1, \"location\": \"Point(2 2)\"}"@2001-01-03',
  '"{\"vehicleId\": 1, \"location\": \"Point(3 3)\"}"@2001-01-05'], interval '1 day');
/* [["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01], ["{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-03], ["{\"location\": \"Point(3 3)\",
   \"vehicleId\": 1}"@2001-01-05]} */
```

15.7. Conversiones

- Convierte un JSONB temporal a un `tstzspan`

```
tjsonb::tstzspan
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]'::tstzspan;
-- [2001-01-01, 2001-01-02]
```

- Convierte entre un JSONB temporal y un texto

```
{tjsonb,text}::{text,tjsonb}
```

```
SELECT (text '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]')::tjsonb;
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT pg_typeof(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]'::text);
-- text
```

- Convierte entre un JSONB temporal y un texto temporal

```
{tjsonb,ttext}::{ttext,tjsonb}
```

```
SELECT (ttext '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]')::tjsonb;
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT pg_typeof(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]'::ttext);
-- ttext
```

15.8. Accesores

- Devuelve los valores

```
getValues(tjsonb) → jsonbset
```

```
SELECT getValues(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02]');
-- {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"}
```

- Devuelve el valor en una marca de tiempo

```
valueAtTimestamp(tjsonb,timestamptz) → jsonb
```

```
SELECT valueAtTimestamp(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]', '2001-01-02');
-- {"location": "Point(2 2)", "vehicleId": 1}
```

15.9. Transformaciones

- Transforma un JSONB temporal a otro subtipo

```
tjsonbInst(tjsonb) → tjsonbInst
tjsonbSeq(tjsonb,interp text='step') → tjsonbSeq
tjsonbSeqSet(tjsonb) → tjsonbSeqSet
```

```
SELECT tjsonbSeq(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01');
-- ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01]
SELECT tjsonbSeq(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01',
  'discrete');
-- {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01}
SELECT tjsonbSeqSet(tjsonb '"{\"vehicleId\": 1,\"location\":\"Point(1 1)\"}"@2001-01-01');
-- {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01]}
```

- Transforma un valor JSONB temporal a otra interpolación

```
setInterp(tjsonb,interp text) → tjsonb
```

```
SELECT setInterp(tjsonb '"{\"vehicleId\": 1,\"location\": \"Point(1 1)\"}"@2001-01-01',
  'step');
-- ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01]
SELECT setInterp(tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01],
  [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02]}', 'discrete');
/* {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-02} */
```

15.10. Operaciones JSON temporales

En esta sección, damos las versiones temporales de las funciones para los tipos `json` y `jsonb`. Remitimos a la documentación de PostgreSQL para explicaciones detalladas de estas funciones. Hemos buscado mantener en la medida de lo posible la misma sintaxis para las versiones no temporales y temporales de estas funciones, como se ilustra en este documento.

Como se indicó en Sección 4.4, una forma común de generalizar las operaciones tradicionales a los tipos temporales es aplicar la operación *en cada instante*, lo que produce un valor temporal como resultado. Considere por ejemplo el operador `->` que extrae un campo de objeto JSON con la clave dada.

```
SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10
```

Aplicar el mismo operador a un valor JSON o JSONB temporal produce el siguiente resultado

```
SELECT ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonb '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
```

que se obtiene aplicando el operador de PostgreSQL `->` en cada instante del valor JSON o JSONB temporal.

Al manipular colecciones JSON de estructura variable, puede ocurrir que un elemento esté definido en algunos de los documentos pero no en todos. PostgreSQL proporciona el modo lax para este propósito, donde el argumento `null_value_treatment` determina el comportamiento en el caso en que una función devuelve un valor NULL. El argumento puede tomar uno de los siguientes valores: `'raise_exception'`, `'use_json_null'`, `'delete_key'`, `'return_target'`, donde `'use_json_null'` es el valor por defecto. En PostgreSQL el modo lax se admite solo para operaciones de *actualización* (no de *consulta*) con la función `jsonb_set_lax`. En MobilityDB, hemos mantenido la misma semántica para la función correspondiente `tjsonbSetLax`, pero habilitamos un comportamiento similar para *todas* las operaciones JSON temporales que pueden devolver un valor nulo, salvo que reemplazamos el valor `'return_target'` por `'return_null'`, ya que el primero no tiene sentido para operaciones temporales. Para operadores como `->`, se utiliza el valor por defecto `'use_json_null'` y no se puede cambiar, mientras que para las funciones correspondientes `tjsonObjectField` y `tjsonbObjectField`, el último argumento especifica el comportamiento en el caso en que la función devuelve NULL. Ilustramos este comportamiento para la función `tjsonObjectField` a continuación.

- Extrae un campo de objeto JSON temporal especificado por una clave

```
{ttext,tjsonb} -> text → {ttext,tjsonb}
tjsonb ->> text → ttext
tjsonObjectField(ttext,text,null_handle text='use_json_null') → ttext
tjsonbObjectField(tjsonb,text,null_handle text='use_json_null') → tjsonb
tjsonbObjectFieldText(tjsonb,text,null_handle text='use_json_null') → ttext
```

```
SELECT ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonb '[{"position":"Point(1 1)", "speed":10}@2001-01-01,
  {"position":"Point(2 2)", "speed":20}@2001-01-03]' ->> text 'position';
-- {["Point(1 1)"@2001-01-01, "Point(2 2)"@2001-01-03]}
SELECT ttext '{[{"unit": "km", "speed": 10}@2001-01-01, {"unit": "km"}@2001-01-02,
  {"unit": "km", "speed": 10}@2001-01-03]}' -> text 'speed';
-- {["10"@2001-01-01, "null"@2001-01-02, "10"@2001-01-03]}
```

Mostramos a continuación el comportamiento de la función `tjsonObjectField` para los posibles valores del argumento `null_handle`. Todas las funciones JSON que pueden devolver un valor NULL se comportan de forma similar en MobilityDB.

```
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'raise_exception');
-- ERROR:  The lifted operation returned NULL
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'use_json_null');
-- {["10"@2001-01-01, "null"@2001-01-02, "10"@2001-01-03]}
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'delete_key');
-- {["10"@2001-01-01, "10"@2001-01-02), ["10"@2001-01-03]}
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'return_null');
-- NULL
```

- Extrae un campo de objeto JSON temporal especificado por una ruta

```
{ttext,tjsonb} #> text[] → {ttext,tjsonb}
tjsonExtractPath(ttext,path text[],null_handle text='use_json_null') → ttext
tjsonbExtractPath(tjsonb,path text[],null_handle text='use_json_null') → tjsonb
tjsonbExtractPathText(tjsonb,path text[],null_handle text='use_json_null') → ttext
```

```
SELECT ttext '[{"speed": {"unit": "km", "value": 10}}@2001-01-01,
  {"speed": {"unit": "km", "value": 20}}@2001-01-02]' #> ARRAY[text 'speed', 'value'];
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonb '[{"position":"Point(1 1)", "PoIs":["Grand Place", "La Bourse"]}@2001-01-01,
  {"position":"Point(2 2)",
  "PoIs":["Palais Royal", "Manneken Pis"]}@2001-01-03]' #>> ARRAY[text 'PoIs', '1'];
-- {["La Bourse"@2001-01-01, "Manneken Pis"@2001-01-03]}
```

- Extrae un elemento de un arreglo JSON temporal

```
tjsonArrayElement(ttext,integer,null_handle text='use_json_null') → ttext
tjsonbArrayElement(tjsonb,integer,null_handle text='use_json_null') → tjsonb
tjsonbArrayElementText(tjsonb,integer,null_handle text='use_json_null') → ttext
```

```
SELECT tjsonArrayElement(ttext '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal", "Manneken Pis"]@2001-01-02]', 0);
-- {["\"Grand Place\""@2001-01-01, "\"Palais Royal\""@2001-01-02]}
SELECT tjsonbArrayElement(tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal", "Manneken Pis"]@2001-01-02]', 1);
-- {["\"La Bourse\""@2001-01-01, "\"Manneken Pis\""@2001-01-02]}
```

- Extrae un valor alfanumérico temporal de un valor JSONB temporal dado por una clave

```
tbool(tjsonb,text,null_handle text='raise_exception') → tbool
tint(tjsonb,text,null_handle text='raise_exception') → tint
tfloat(tjsonb,text,interp text='linear',null_handle text='raise_exception') → tfloat
ttext(tjsonb,text,null_handle text='raise_exception') → ttext
```

Tenga en cuenta que el valor `use_json_null` no puede utilizarse para las funciones anteriores y por lo tanto se utiliza el valor por defecto `'raise_exception'`.

```
SELECT tbool(tjsonb '[{"speed": 10, "lights": "on"}@2001-01-01,
  {"speed": 20, "lights": "on"}@2001-01-02]', 'lights');
-- {[t@2001-01-01, t@2001-01-02]}
SELECT tint(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]', 'speed');
-- {[10@2001-01-01, 20@2001-01-02]}
SELECT tfloat(tjsonb '{[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02],
  [{"speed": 20}@2001-01-03, {"speed": 20}@2001-01-04]}', 'speed', 'step');
-- Interp=Step;{[10@2001-01-01, 20@2001-01-02], [20@2001-01-03, 20@2001-01-04]}
SELECT ttext(tjsonb '[{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03]', 'category');
-- {["primary"@2001-01-01, "residential"@2001-01-03]}
```

- Extrae un valor geo temporal de un valor JSONB temporal dado por una clave

```
tgeompoint(tjsonb,text,interp text='linear',null_handle text='raise_exception') →  ↩
    tgeompoint
tgeogpoint(tjsonb,text,interp text='linear',null_handle text='raise_exception') →  ↩
    tgeogpoint
tgeometry(tjsonb,text,null_handle text='raise_exception') → tgeometry
tgeography(tjsonb,text,null_handle text='raise_exception') → tgeography
```

El valor de la clave es una cadena en un formato de texto de una geometría o una geografía, como WKT o EWKT, que es un punto para los puntos temporales. Como para las funciones anteriores, el valor `use_json_null` no puede utilizarse.

```
SELECT asText(tgeompoint(tjsonb '[{"vehicle": "bus", "pos": "Point(0 0)"}@2001-01-01,
  {"vehicle": "bus", "pos": "Point(2 2)"}@2001-01-03]', 'pos'));
-- {[POINT(0 0)@2001-01-01, POINT(2 2)@2001-01-03]}
SELECT asEWKT(tgeogpoint(tjsonb '[{"pos": "Point(4.35 50.85)"}@2001-01-01,
  {"pos": "Point(4.40 50.80)"}@2001-01-02]', 'pos'));
-- SRID=4326;{[POINT(4.35 50.85)@2001-01-01, POINT(4.4 50.8)@2001-01-02]}
SELECT asText(tgeometry(tjsonb '[{"area": "Polygon((0 0,1 0,1 1,0 0))"}@2001-01-01,
  {"area": "Point(2 2)"}@2001-01-02]', 'area'));
-- {[POLYGON((0 0,1 0,1 1,0 0))@2001-01-01, POINT(2 2)@2001-01-02]}
```

- Concatenación JSONB temporal

```
tjsonb || {jsonb,tjsonb} → tjsonb
```

```
SELECT tjsonb '{[{"speed":10}@2001-01-01,
  {"speed":20}@2001-01-02]}' || '{"unit":"km"}'::jsonb;
/* {["{\"unit\": \"km\", \"speed\": 10}"@2001-01-01, "{\"unit\": \"km\", \"speed\":
   20}"@2001-01-02]} */
SELECT tjsonb '{[{"speed":10}@2001-01-01, {"speed":20}@2001-01-03]}' ||
  tjsonb '{[{"Position":"Point(1 1)"}@2001-01-02, {"Position":"Point(2 2)"}@2001-01-04]}';
/* {["{\"speed\": 10, \"Position\": \"Point(1 1)\"}"@2001-01-02, "{\"speed\": 20,
   \"Position\": \"Point(1 1)\"}"@2001-01-03]} */
```

- Eliminación JSONB temporal

```
tjsonb - {integer,text,text[]} → tjsonb
tjsonb #- text[] → tjsonb
```

```
SELECT tjsonb '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' - 'unit';
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal", "Manneken Pis"]@2001-01-02]' - 1;
-- {["[\"Grand Place\"]"@2001-01-01, "[\"Palais Royal\"]"@2001-01-02]}
SELECT tjsonb
  '[{"location":"Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]}@2001-01-01,
  {"location":"Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]}@2001-01-02]'
  #- ARRAY[text 'PoIs', '1'];
/* {["{\"PoIs\": [\"Grand Place\"], \"location\": \"Point(1 1)\"}"@2001-01-01,
   "{\"PoIs\": [\"Palais Royal\"], \"location\": \"Point(2 2)\"}"@2001-01-02]} */
```

Como se muestra a continuación, el resultado de una operación de eliminación puede ser un registro vacío o un arreglo vacío. Para eliminar estos valores, puede utilizarse la función `minusValues`.

```
SELECT tjsonb '{[{"unit": "km", "speed": 10, "light": true}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' - ARRAY[text 'unit', 'speed'];
-- {["{\"light\": true}"@2001-01-01, "{}"@2001-01-02]}
SELECT tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal"]@2001-01-02]' - 0;
-- {["[\"La Bourse\"]"@2001-01-01, "[]"@2001-01-02]}
```

```
SELECT minusValues(tjsonb '{[{"unit": "km", "speed": 10, "light": true}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' - ARRAY[text 'unit', 'speed'],
  jsonbset '{"[]","{}"}');
-- {["{\"light\": true}"@2001-01-01, "{\"light\": true}"@2001-01-02)}
SELECT minusValues(tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal"]@2001-01-02]' - 0, jsonbset '{"[]","{}"}');
-- {["[\"La Bourse\"]"@2001-01-01, "[\"La Bourse\"]"@2001-01-02)}
```

- Existencia JSONB temporal

```
tjsonb ? jsonb → tbool
tjsonb {?|, ?&} jsonb[] → tbool
```

Como en PostgreSQL, los operadores `?|` y `?&` comprueban, respectivamente, si *alguna* o *todas* las cadenas del arreglo de texto existen como claves de nivel superior o elementos de arreglo.

```
SELECT tjsonb '"{\"geom\": \"Point(1 1)\"}"@2001-01-01' ? text 'geom';
-- t@2001-01-01
SELECT tjsonb '{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
  {"geom": "Point(1 1)"}@2001-01-03}' ?| ARRAY[text 'geom'];
-- {t@2001-01-01, t@2001-01-02, t@2001-01-03}
SELECT tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]' ?& ARRAY['speed', 'units'];
-- [t@2001-01-01, t@2001-01-02]
```

- Contiene/contenido JSONB temporal

```
{tjsonb,jsonb} {@>, <@} {tjsonb,jsonb} → tbool
```

```
SELECT tjsonb '"{\"geom\": \"Point(1 1)\"}"@2001-01-01' @> jsonb '{"geom": "Point(1 1)"}';
-- t@2001-01-01
SELECT tjsonb '{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
  {"geom": "Point(1 1)"}@2001-01-03}' @> jsonb '{"geom": "Point(1 1)"}';
-- {t@2001-01-01, f@2001-01-02, t@2001-01-03}
SELECT jsonb '{"geom": "Point(1 1)"}' <@ tjsonb '{[{"geom": "Point(1 1)"}@2001-01-01,
  {"geom": "Point(1 1)"}@2001-01-02, {"geom": "Point(1 1)"}@2001-01-03],
  [{"geom": "Point(2 2)"}@2001-01-04, {"geom": "Point(2 2)"}@2001-01-05]}';
-- {[t@2001-01-01, t@2001-01-03], [f@2001-01-04, f@2001-01-05]}
```

- Asignación JSONB temporal

```
tjsonbSet(tjsonb,path text[],val jsonb,create_missing boolean=true) → tjsonb
tjsonbSetLax(tjsonb,path text[],val jsonb,create_missing boolean=true,
  handle_null text='use_json_null') → tjsonb
```

Devuelve el valor `tjsonb` con el elemento especificado por la ruta reemplazado por `jsonb`, o añadido si `create` es verdadero y el elemento no existe. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, y create_if_missing es verdadero, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo. La función `tjsonbSetLax` se comporta de forma idéntica a `tjsonbSet` si el valor dado no es NULL; de lo contrario, se comporta según el valor de `handle_null`, que debe ser uno de `'raise_exception'`, `'use_json_null'`, `'delete_key'` o `'return_target'`. Como se indicó anteriormente, mantuvimos el comportamiento de PostgreSQL para esta función habilitando el valor `'return_target'`, mientras que para todas las demás operaciones de *consulta*, este valor se ha reemplazado por `'return_null'`.

```
SELECT tjsonbSet(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb);
/* {["{\"speed\": 10, \"units\": \"km/h\"}"@2001-01-01, "{\"speed\": 20, \"units\":
   \"km/h\"}"@2001-01-02]} */
SELECT tjsonbSet(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb, false);
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonbSet(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]', ARRAY['units'], '"mi/h"'::jsonb);
/* {["{\"speed\": 10, \"units\": \"mi/h\"}"@2001-01-01, "{\"speed\": 20, \"units\":
   \"mi/h\"}"@2001-01-02]} */
SELECT tjsonbSetLax(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20}@2001-01-02]', ARRAY['units'], NULL, true, 'delete_key');
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
```

- Inserción JSONB temporal

```
tjsonbInsert(tjsonb,path text[],val jsonb,after boolean=false) → tjsonb
```

Devuelve el valor `tjsonb` con `jsonb` insertado. Si el elemento especificado por la ruta es un elemento de arreglo, el nuevo valor se insertará antes de ese elemento si `after` es falso, o después en caso contrario. Si el elemento especificado por la ruta es un campo de objeto, el nuevo valor se insertará solo si el objeto no contiene ya esa clave. Todos los pasos anteriores de la ruta deben existir, o se devuelve el objetivo sin cambios. Como ocurre con los operadores orientados a rutas, los enteros negativos que aparecen en la ruta cuentan desde el final de los arreglos JSON. Si el último paso de la ruta es un índice de arreglo que está fuera de rango, el nuevo valor se añade al principio del arreglo si el índice es negativo, o al final del arreglo si es positivo.

```
SELECT tjsonbInsert(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb);
```

```
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonbInsert(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb, false);
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonbInsert(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]', ARRAY['units'], '"mi/h"'::jsonb);
/* {["{\"speed\": 10, \"units\": \"mi/h\"}"@2001-01-01, "{\"speed\": 20, \"units\":
   \"mi/h\"}"@2001-01-02]} */
```

- Devuelve un valor JSON temporal sin nulos

```
tjsonStripNulls(ttext,strip_in_arrays boolean=false) → ttext
tjsonbStripNulls(tjsonb,strip_in_arrays boolean=false) → tjsonb
```

El último argumento indica si los elementos de arreglo nulos también se eliminan. Los valores nulos simples nunca se eliminan.

```
SELECT tjsonStripNulls(ttext '[{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02]');
/* ["{\"speed\":10}"@2001-01-01, "{\"speed\":20,\"PoIs\":[\"Grand Place\",\"La
   Bourse\",null]}"@2001-01-02] */
SELECT tjsonbStripNulls(tjsonb '[{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02]', true);
/* {["{\"speed\": 10}"@2001-01-01, "{\"PoIs\": [\"Grand Place\", \"La Bourse\"],
   \"speed\": 20}"@2001-01-02]} */
SELECT tjsonbStripNulls(tjsonb '{{"road": "Bvd Gén. Jacques",
  "category": "primary"}@2001-01-01,
  {"road": "Bvd de la Cambre", "category": "primary"}@2001-01-02,
  {"road": "rue de l''Abbaye", "category": null}@2001-01-03}');
/* {"{\"road\": \"Bvd Gén. Jacques\", \"category\": \"primary\"}"@2001-01-01,
   "{\"road\": \"Bvd de la Cambre\", \"category\": \"primary\"}"@2001-01-02,
   "{\"road\": \"rue de l'Abbaye\"}"@2001-01-03} */
```

Como se muestra a continuación, la función no elimina los valores nulos simples. Para este propósito puede utilizarse la función `minusValues`.

```
SELECT tjsonStripNulls(ttext '[{"speed": 10}@2001-01-01, null@2001-01-02,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03]');
/* ["{\"speed\":10}"@2001-01-01, "null"@2001-01-02, "{\"speed\":20,\"PoIs\":[\"Grand
   Place\",\"La Bourse\"]}"@2001-01-03] */
SELECT minusValues(ttext '[{"speed": 10}@2001-01-01, null@2001-01-02,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03]',
  set(ARRAY[text 'null']));
/* {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 10}"@2001-01-02), ["{\"speed\": 20,
   \"PoIs\": [\"Grand Place\", \"La Bourse\"]}"@2001-01-03]} */
```

- ¿Devuelve la ruta JSON algún elemento para el valor JSONB temporal especificado?

```
tjsonb @? jsonpath → tbool
tjsonbPathExists(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
tjsonbPathExistsTz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

El operador `@?` suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]' @? '$.units ? (@ == "km/h")';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonbPathExists(tjsonb '{{"road": "Bvd Gén. Jacques",
  "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
```

```
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
  '$.category ? (@ == "residential")');
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
SELECT tjsonbPathExistsTz(tjsonb '[{"speed":10, "cameraId": "25",
  "lastInspection": "2001-08-01 12:00:00"}@2001-01-01,
  {"speed":20, "cameraId": "35", "lastInspection": "2001-03-01 12:00:00"}@2001-01-02]',
  '$.lastInspection ? (@.datetime() > "2001-06-01".datetime() )');
-- [t@2001-01-01, f@2001-01-02]
```

- Devuelve el resultado de una comprobación de predicado de ruta JSON para un valor JSONB temporal

```
tjsonb @@ jsonpath → tbool
tjsonbPathMatch(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
tjsonbPathMatchTz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

El operador `@@` suprime los siguientes errores: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Las funciones anteriores también pueden suprimir estos tipos de errores estableciendo el último argumento en verdadero. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]' @@ '$.units == "km/h"';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonbPathMatch(tjsonb '{{"road": "Bvd Gén. Jacques",
  "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
  '$.category == "residential"');
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
SELECT tjsonbPathMatchTz(tjsonb '[{"speed":10, "cameraId": "25",
  "lastInspection": "2001-08-01 12:00:00"}@2001-01-01,
  {"speed":20, "cameraId": "35", "lastInspection": "2001-03-01 12:00:00"}@2001-01-02]',
  '$.lastInspection.datetime() > "2001-06-01".datetime()');
-- [t@2001-01-01, f@2001-01-02]
```

- Devuelve todos los elementos devueltos por una ruta JSON de un valor JSONB temporal, como un arreglo JSON

```
tjsonbPathQueryArray(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
tjsonbPathQueryArrayTz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

La función anterior puede suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT tjsonbPathQueryArray(tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$ ? (@.speed >= $min && @.speed <= $max)', '{"min":10, "max":30}');
-- {["[{\"speed\": 10}]"@2001-01-01, "[{\"speed\": 20, \"units\": \"km/h\"}]"@2001-01-02]}
SELECT tjsonbPathQueryArrayTz(tjsonb '[{"cameraId":25,
  "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}@2001-01-01,
  {"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}@2001-01-02]',
  '$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
   @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {["[\"2001-01-06\", \"2001-01-09\"]"@2001-01-01, "[\"2001-01-07\"]"@2001-01-02]}
```

- Devuelve el primer elemento devuelto por una ruta JSON de un valor JSONB temporal

```
tjsonbPathQueryFirst(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
tjsonbPathQueryFirstTz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

Las funciones anteriores pueden suprimir los siguientes errores estableciendo el último argumento en verdadero: campo de objeto o elemento de arreglo faltante, tipo de elemento JSON inesperado, errores de fecha/hora y numéricos. Este comportamiento puede ser útil al buscar en colecciones de documentos JSON de estructura variable.

```
SELECT tjsonbPathQueryFirst(tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$.speed ? (@ >= $min && @ <= $max)', '{"min":10, "max":30}');
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonbPathQueryArrayTz(tjsonb
  '[{"cameraId":25, "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}@2001-01-01,
   {"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}@2001-01-02]',
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {["[\"2001-01-06\", \"2001-01-09\"]"@2001-01-01, "[\"2001-01-07\"]"@2001-01-02]}
```

15.11. Restricciones

- Restringe a (el complemento de) un valor o un conjunto de valores

```
atValue(tjsonb,base) → tjsonb
minusValue(tjsonb,base) → tjsonb
atValues(tjsonb,values) → tjsonb
minusValues(tjsonb,values) → tjsonb
```

```
SELECT atValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
  jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
-- {["{\"location\": \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-03]}
SELECT minusValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
  jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-03)} */
```

15.12. Operaciones de caja delimitadora

- Operadores topológicos

```
{tjsonb,tstzspan} {&&, <@#, #@>, ~=, -|-} {tjsonb,tstzspan} → boolean
```

Los operadores temporales contiene y contenido se marcan con un # para distinguirlos de los operadores JSONB correspondientes.

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' &&
  tstzspan '[2001-01-01, 2001-03-01]';
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' #@>
  timestamptz '[2001-01-02]'::tstzspan;
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' -|-
  tjsonb '({"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true
```

- Operadores de posición

```
{tjsonb,tstzspan} {<<#, &<#, #>>, #&>} {tjsonb,tstzspan} → boolean
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' <<# tstzspan '[2001-01-01,
  2001-03-01]';
-- false
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' #>>
  timestamptz '[2001-01-01]'::tstzspan;
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' #&>
  tjsonb '[{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true
```

15.13. Comparaciones

- Comparaciones tradicionales

```
tjsonb {=, <>, <, >, <=, >=} tjsonb → boolean
```

```
SELECT tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02),
  [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
   {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]}' =
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- true
SELECT tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]}' <>
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- false
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' <
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- false
```

- Comparaciones siempre y alguna vez

```
{jsonb,tjsonb} {?=, %=} {jsonb,tjsonb} → boolean
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' ?= jsonb '{"vehicleId": 1,
  "location": "Point(1 1)"}';
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' %= jsonb '{"vehicleId": 1,
  "location": "Point(1 1)"}';
-- false
```

- Comparaciones temporales

```
{jsonb,tjsonb} {#=, #<>} {jsonb,tjsonb} → tbool
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03)' #= jsonb '{"vehicleId": 1,
  "location": "Point(1 1)"}';
-- [t@2001-01-01, t@2001-01-03)
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03)' #<>
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03)';
-- [f@2001-01-01, f@2001-01-03)
```

15.14. Agregaciones

Las funciones de agregación para valores JSONB temporales se ilustran a continuación.

- Conteo temporal

```
tCount(tjsonb) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' UNION
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-04]' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-03], (1@2001-01-03, 1@2001-01-04]}
```

- Conteo en ventana

```
wCount(tjsonb) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' UNION
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-04]' )
SELECT wCount(Temp, interval '2 days') FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-05], (1@2001-01-05, 1@2001-01-06]}
```

15.15. Indexación

Pueden crearse índices GiST y SP-GiST para columnas de tabla de valores JSONB temporales. A continuación se muestra un ejemplo de creación de índice:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

Los índices GiST y SP-GiST almacenan la caja delimitadora de los valores JSONB temporales, que es un `stbox`, como todos los tipos espaciotemporales.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en los valores JSONB temporales,
- `<<#`, `&<#`, `#&>`, `#>>`, que solo consideran la dimensión temporal en los valores JSONB temporales,
- `&&`, `@>`, `<@`, `~=`, `-|-` y `|=|` , que consideran tantas dimensiones como las que comparten la columna indexada y el argumento de la consulta.

Estos operadores trabajan sobre cajas delimitadoras, no sobre los valores completos.

Capítulo 16

Teselaciones temporales

Un *sistema de cuadrícula global discreta* (DGGS) divide la Tierra en una jerarquía de celdas, cada una identificada por un entero de 64 bits, de modo que una posición se convierte en un identificador de celda y una prueba de contención se convierte en una comparación de enteros. MobilityDB admite tres de ellos, y un tipo de índice de celda temporal registra el movimiento de un objeto como la secuencia de celdas que ocupa a lo largo del tiempo. Cada cuadrícula tesela la Tierra, y estas tres la teselan globalmente: un identificador de celda nombra una celda de una de ellas dondequiera que se lea, que es lo que permite a un tipo temporal transportar celdas y nada más.

Cuadrícula	Tipo de celda	Tipo de conjunto	Tipo temporal	Resoluciones
Uber H3	`h3index`	`h3indexset`	`th3index`	0–15
CARTO QUADBIN	`quadbin`	`quadbinset`	`tquadbin`	0–26
Google S2	`s2cell`	`s2cellset`	`ts2cell`	0–30

Figura 16.1 muestra la traza de un buque AIS que sale de Frederikshavn en las tres cuadrículas, en la resolución de cada una cuyas celdas son más próximas en área — 0,57 km^2 para H3, 0,43 km^2 para QUADBIN y 0,32 km^2 para S2 en esta latitud — con las celdas por las que pasa la traza sombreadas y un área marina protegida en verde. Un índice de celda temporal registra esa secuencia sombreada, de modo que la traza es una y la misma y cada cuadrícula la expresa como una secuencia distinta de identificadores. Preguntar si el buque entró en el área protegida se convierte entonces en una prueba entre dos conjuntos de identificadores enteros en lugar de una prueba entre dos geometrías. La fila inferior presenta el extracto delineado en negro arriba, una resolución más fina en cada cuadrícula, donde una celda H3 ocupa aproximadamente un séptimo del área de su padre y una celda QUADBIN o S2 aproximadamente un cuarto, y la secuencia sigue la traza lo bastante de cerca como para representarla.

Las tres difieren en la forma de una celda y en cómo se ramifica la jerarquía. H3 de Uber tesela la esfera en 122 celdas base en la resolución 0 y subdivide cada una en 7 hijos en cada resolución más fina, hasta la 15, de modo que una celda es un hexágono sobre un icosaedro y su identificador codifica la celda base, la resolución y la posición jerárquica. QUADBIN de CARTO se construye sobre el esquema de teselas de mapa deslizante (Web-Mercator): una única celda mundial en la resolución 0 se subdivide en cuatro hijos iguales en cada resolución más fina, hasta la 26, y el identificador codifica una etiqueta de cabecera, la resolución y las coordenadas de la tesela. S2 de Google proyecta las seis caras de un cubo circunscrito sobre la esfera y subdivide cada cara en cuatro hijos, hasta el nivel 30, de modo que una celda es un cuadrilátero esférico y el identificador codifica la cara, la posición a lo largo de una curva de Hilbert y el nivel.

Cada forma aporta algo. Un hexágono tiene seis vecinos todos a la misma distancia de su centro, de modo que un paso H3 es el mismo movimiento en todas las direcciones, que es lo que necesita un flujo o una superficie de densidad. Un cuadrado QUADBIN es una tesela de mapa deslizante, de modo que una celda ES la tesela que un servidor de mapas ya sirve y el identificador se convierte en la terna `(x,y,z)` y en la cadena quadkey de ese ecosistema. Una celda S2 es una celda de un árbol cuaternario de una cara del cubo proyectada sobre la esfera, de modo que los descendientes de una celda ocupan un intervalo contiguo de la curva de Hilbert y una prueba de ascendencia es un par de comparaciones de enteros. Sus autores documentan las cuadrículas en h3geo.org, CARTO y s2geometry.io, y la familia a la que pertenecen en Discrete global grid.

Responden a las mismas preguntas, de modo que este capítulo presenta cada operación una sola vez y nombra la cuadrícula únicamente donde la respuesta depende de ella. S2 llama *nivel* a una resolución; las dos palabras denotan la misma cantidad.

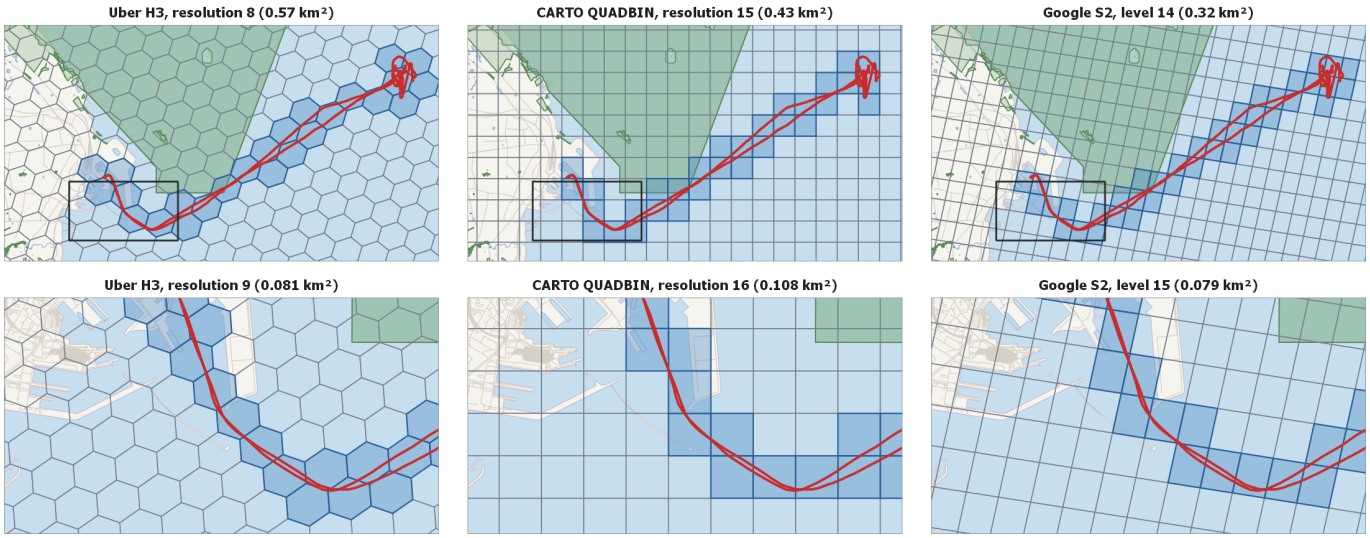


Figura 16.1: La traza de un buque AIS que sale de Frederikshavn en las tres cuadrículas, con las celdas por las que pasa la traza sombreadas y un área marina protegida en verde. La fila superior presenta la traza completa en las resoluciones cuyas celdas son más próximas en área y delinea en negro el extracto que presenta la fila inferior, una resolución más fina en cada cuadrícula. Traza del buque y áreas protegidas de los datos AIS daneses; mapa base © colaboradores de OpenStreetMap.

Un tipo de índice de celda comparte su representación en disco con `tbigint`: ambos almacenan un entero temporal de 64 bits. No todo entero es una celda, ya que cada cuadrícula da a sus celdas la estructura que define su especificación: QUADBIN su cabecera, resolución y bits no usados, S2 su cara y el bit uno que termina su posición, y H3 sus bits de modo. El tipo SQL distinto existe para que el verificador de tipos rechace una función específica de celdas aplicada a una trayectoria `tbigint` arbitraria, y a la inversa, y un entero que entra en un tipo de celda se comprueba como la entrada de texto del tipo comprueba su cadena: un valor S2 es una celda de su cuadrícula, un valor QUADBIN un índice bien formado de cualquier modo, y un valor H3 una celda, una arista dirigida o un vértice.

Como con otros tipos temporales, un tipo de índice de celda tiene cuatro subtipos: `Instant`, secuencia discreta, secuencia continua con interpolación de pasos y `SequenceSet`. Los identificadores de celda son discretos, de modo que la interpolación lineal entre dos de ellos carece de sentido y siempre se usa la interpolación de pasos.

Un ráster es una teselación propia: una rejilla de píxeles tendida sobre una huella, cuya celda es un píxel. Figura en este capítulo junto a las tres rejillas globales, y se distingue de ellas en lo que lleva una celda. Una celda H3, QUADBIN o S2 lleva identidad, de modo que lo que produce una trayectoria sobre una de ellas es un índice de celda temporal; una celda de ráster lleva un valor, de modo que lo que produce una trayectoria leída contra un ráster es un `tfloat` de los valores por los que pasa, y no existe un tipo ráster temporal. Por eso las secciones siguientes enuncian un ráster allí donde responde la operación que responde una celda, y enuncian aparte su lectura a lo largo de un viaje.

La extensión ráster de PostGIS almacena datos de cobertura en cuadrícula (elevación, temperatura, imágenes satelitales, ...) en el tipo `raster`. Cada ráster tiene una o más bandas; una banda contiene un arreglo 2D de valores de píxel, una extensión espacial, un tamaño de píxel y un valor centinela opcional para datos nulos (`nodata`).

Raquet es un formato ráster nativo en la nube que almacena teselas ráster como filas en un archivo Apache Parquet. Cada fila contiene los bytes de píxel de una tesela Web-Mercator junto con su identificador de celda QUADBIN, un entero de 64 bits que codifica el nivel de zoom y las coordenadas x/y intercaladas mediante Morton. No se requieren metadatos espaciales adicionales: el rectángulo delimitador y el mapeo de píxel a coordenadas quedan completamente determinados por el valor QUADBIN. La especificación llama a esa columna `block` y mantiene una fila más, en `block` = 0, cuya columna `metadata` indica en JSON el tamaño de la tesela, el tipo y el valor nodata de cada banda y la compresión de los bytes de las bandas.

Los dos se distinguen en lo que lleva un valor. Un valor `raquet` contiene sus píxeles y su celda QUADBIN en un único valor, de modo que su extensión y su rejilla de píxeles se derivan de la celda y responde únicamente a la rejilla Web-Mercator. Un `raster` de PostGIS declara un sistema de referencia espacial arbitrario y una georreferenciación afín arbitraria, y es el que toman las funciones de procesamiento de Sección 16.24: clip, transform, rescale, reclass, summaryStats y dumpAsPolygons. MEOS calcula ambos, de modo que ninguno necesita PostgreSQL para evaluarse.

Una cobertura multiespectral se lee banda por banda en ambas vías. La especificación RaQuet otorga a cada banda de una tesela una columna propia, `band_1`, `band_2` y así sucesivamente, por lo que tal cobertura es un conjunto de teselas de una sola banda que comparten una celda QUADBIN y rasterTileValue muestrea la banda que recibe, mientras que en la vía `raster` de PostGIS el argumento `band` la nombra. Lo que un valor `raquet` no transporta es la disposición entrelazada que la especificación ofrece para imágenes RGB, que empaqueta las bandas de una tesela en una única columna comprimida en JPEG o WebP.

16.1. Notación

La mayoría de las funciones y operadores para tipos temporales descritos en los capítulos anteriores pueden aplicarse a los tipos de índices de celdas temporales. Por lo tanto, en las signaturas de las funciones, la notación `base` representa una `cell` y la notación `ttype` representa también un `tcell`. Para evitar redundancia, presentamos a continuación únicamente las funciones y los operadores específicos de los tipos de índices de celdas.

- `cell` representa cualquier tipo de índice de celda, es decir, `h3index`, `quadbin` o `s2cell`, y en la posición de un nombre de función es el constructor de ese tipo,
- `cellset` representa cualquier tipo de conjunto de índices de celdas, es decir, `h3indexset`, `quadbinset` o `s2cellset`,
- `tcell` representa cualquier tipo de índice de celda temporal, es decir, `th3index`, `tquadbin` o `ts2cell`,
- `geo` y `tgeo` representan el tipo de geometría en el que responde una cuadrícula, plana para QUADBIN y geodésica para H3 y S2.
- `raster` es el tipo ráster de PostGIS y `raquet` una única tesela ráster Web-Mercator; ninguno de los dos es un tipo de celda, de modo que una operación que lee uno de ellos lo nombra.

16.2. Entrada y salida

Un valor de celda temporal se introduce utilizando la sintaxis temporal estándar con el identificador de celda de 64 bits subyacente. El analizador de entrada acepta la forma hexadecimal canónica que publica cada cuadrícula — la forma que emiten la CLI de H3 y `h3_cell_to_string`, y la forma que lee la implementación de referencia de CARTO — con un prefijo `0x` opcional, en cualquiera de las dos cajas de letra y con 16 dígitos significativos como máximo. El hexadecimal es idiomático en los tres ecosistemas y es lo que emite la función de salida, de modo que todos los ejemplos siguientes lo usan. El valor debe además denotar algo que su cuadrícula indexe: para H3 una celda, una arista dirigida o un vértice, de modo que un literal hexadecimal bien formado como `12345`, que no es ninguno de los tres, se rechaza en lugar de almacenarse como un valor sin ubicación. Como referencia, el hexadecimal `480fffffffffffff` usado en los ejemplos es la celda mundial QUADBIN de resolución 0, igual al decimal `5192650370358181887`.

Las funciones `asBinary` y `asHexWKB` serializan un valor `raster` o `raquet`, y rasterFromBinary, rasterFromHexWKB, raquetFromBinary y raquetFromHexWKB lo reconstruyen. Un ráster se escribe en la forma que escribe el ráster de PostGIS, de modo que una columna de rásteres hace un viaje de ida y vuelta entre PostgreSQL y cualquier programa que lea esa forma. Una tesela lleva sus píxeles y su georreferenciación QUADBIN en un único valor, de modo que hace un viaje de ida y vuelta a través de una cadena de bytes portable sin intervención de ninguna extensión espacial, y la forma hexadecimal permite que una columna de teselas haga ese viaje mediante un `COPY` basado en texto. Ambas funciones de salida aceptan un argumento opcional de ordenación de bytes, `NDR` para little-endian o `XDR` para big-endian, cuyo valor predeterminado es la ordenación del servidor.

- Entrada y salida de una celda temporal en su forma textual

```
tcell(text) → tcell
text(tcell) → text
```

El tipo estático acompañante `cell` contiene un único identificador de celda (no temporal) y `cellset` un conjunto finito de ellos; comparten la representación textual hexadecimal / decimal del tipo temporal, llevan los operadores estándar de igualdad y de orden a través de una clase de operadores btree y hash, y exponen los predicados de validez de más abajo. Un modificador de tipo restringe una columna a un subtipo, que es `Instant`, `Sequence` o `SequenceSet`.

```
SELECT th3index '831c02fffffffff@2001-01-01';
-- 831c02fffffffff@2001-01-01
SELECT th3index '[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]';
-- [831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]
SELECT th3index '{[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]}';
-- {[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]}
SELECT tquadbin '48a6227affffffff@2001-01-01';
-- 48a6227affffffff@2001-01-01
SELECT tquadbin '[48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-02]';
-- [48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-02]
SELECT tquadbin '{[48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-02]}';
-- {[48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-02]}
SELECT ts2cell '47c3c@2001-01-01';
-- 47c3c@2001-01-01
SELECT ts2cell '[47c3c@2001-01-01, 47c4@2001-01-02]';
-- [47c3c@2001-01-01, 47c4@2001-01-02]
SELECT ts2cell '{[47c3c@2001-01-01, 47c4@2001-01-02]}';
-- {[47c3c@2001-01-01, 47c4@2001-01-02]}
```

- Serializar una celda temporal como texto, como binario o como binario bien conocido hexadecimal, con o sin su SRID, o como MF-JSON

```
asText(tcell) → text
asEWKT(tcell) → text
asBinary(tcell,endian text='') → bytea
asEWKB(tcell,endian text='') → bytea
asHexWKB(tcell,endian text='') → text
asHexEWKB(tcell,endian text='') → text
asMFJSON(tcell) → text
```

Las formas extendidas `asEWKT`, `asEWKB` y `asHexEWKB` indican el SRID 4326 de una celda, que las demás formas omiten. La forma binaria bien conocida hexadecimal permite que una columna de celdas temporales se transfiera de ida y vuelta mediante un `COPY` basado en texto. En MF-JSON cada celda del arreglo `values` es su forma textual en una cadena JSON, el índice hexadecimal de H3 y QUADBIN y el token de S2, ya que un lector JSON que guarda sus números como números de doble precisión pierde los bits bajos de un identificador de celda de 64 bits, y el miembro `bbox` indica la caja espaciotemporal de las celdas.

```
SELECT asEWKT(th3index '831c02fffffffff@2001-01-01');
-- SRID=4326;831c02fffffffff@2001-01-01
SELECT asHexEWKB(th3index '831c02fffffffff@2001-01-01');
-- 01460061E6100000FFFFFFFF2FC031080060885EC91C0000
SELECT asHexWKB(th3index '831c02fffffffff@2001-01-01');
-- 01460021FFFFFFFF2FC031080060885EC91C0000
SELECT asHexWKB(tquadbin '48a6227affffffff@2001-01-01');
-- 01490001FFFFFFFF7A22A6480060885EC91C0000
SELECT asHexWKB(ts2cell '47c3c@2001-01-01');
-- 01570021000000000C0C3470060885EC91C0000
SELECT asMFJSON(tquadbin '48a6227affffffff@2001-01-01');
/* {"type":"MovingQuadbin","crs":{"type":"Name","properties":{"name":"EPSG:4326"}},
   "values":["48a6227affffffff"],"datetimes":["2001-01-01T00:00:00+00:00"],
   "interpolation":"None"} */
```

- Reconstruir una celda temporal a partir de su representación textual, binaria, hexadecimal o MF-JSON

```
tcellFromText(text) → tcell
tcellFromEWKT(text) → tcell
tcellFromBinary(bytea) → tcell
tcellFromEWKB(bytea) → tcell
tcellFromHexWKB(text) → tcell
tcellFromHexEWKB(text) → tcell
tcellFromMFJSON(text) → tcell
```

Un SRID indicado por una entrada extendida debe ser 4326, el SRID de una celda. La entrada MF-JSON lee una celda escrita como su forma textual en una cadena JSON y una celda escrita como el número JSON de su identificador de 64 bits.

```
SELECT ts2cellFromText('47c3c@2001-01-01');
-- 47c3c@2001-01-01
SELECT th3indexFromEWKT('SRID=4326;831c02fffffffff@2001-01-01');
-- 831c02fffffffff@2001-01-01
SELECT tquadbinFromEWKB(asEWKB(tquadbin '48a6227affffffff@2001-01-01'));
-- 48a6227affffffff@2001-01-01
SELECT tquadbinFromMFJSON('{"type":"MovingQuadbin","values":["48a6227affffffff"],
  "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"}');
-- 48a6227affffffff@2001-01-01
SELECT tquadbinFromMFJSON('{"type":"MovingQuadbin","values":[5234909528541102079],
  "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"}');
-- 48a6227affffffff@2001-01-01
SELECT th3indexFromHexWKB(asHexWKB(th3index '831c02fffffffff@2001-01-01'));
-- 831c02fffffffff@2001-01-01
SELECT tquadbinFromHexWKB(asHexWKB(tquadbin '48a6227affffffff@2001-01-01'));
-- 48a6227affffffff@2001-01-01
SELECT ts2cellFromHexWKB(asHexWKB(ts2cell '47c3c@2001-01-01'));
-- 47c3c@2001-01-01
```

- Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB) de una celda

```
asBinary(cell,endian text='') → bytea
asHexWKB(cell,endian text='') → text
```

Una celda se escribe como su identificador de 64 bits, codificado utilizando la codificación little-endian (NDR) o big-endian (XDR), la codificación de la máquina cuando no se especifica ninguna. El SRID 4326 de una celda, que fija su malla, se omite.

```
SELECT asBinary(quadbin '48427fffffffffff');
-- \x0100ffffffffff7f4248
SELECT asHexWKB(quadbin '48427fffffffffff', 'XDR');
-- 000048427FFFFFFFFFF
SELECT asHexWKB(s2cell '47c3c444c');
-- 0100000000C044C4C347
```

- Entrada de una celda desde la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)

```
cellFromBinary(bytea) → cell
cellFromHexWKB(text) → cell
```

Un SRID indicado por una entrada extendida debe ser 4326, el SRID de una celda, y el identificador debe codificar una celda de la malla.

```
SELECT quadbinFromHexWKB('0140E6100000FFFFFFFFFF7F4248');
-- 48427fffffffffff
SELECT quadbinFromHexWKB('0140110F0000FFFFFFFFFF7F4248');
-- ERROR:  Only SRID 4326 (WGS84) is accepted for the type quadbin, received 3857
```

- Devuelve la representación de texto de una celda

```
asText(cell) → text
```

En la función anterior, `cell` reemplaza `h3index`, `quadbin` o `s2cell`. El texto es el índice hexadecimal de H3 y QUADBIN y el token de S2, la forma que emite la función de salida; para `h3index`, el tipo de h3-pg, es el texto que escribe la función de salida de h3-pg.

```
SELECT asText(h3index '831c02fffffffff');
-- 831c02fffffffff
SELECT asText(quadbin '48a6227affffffff');
-- 48a6227affffffff
SELECT asText(s2cell '47c3c');
-- 47c3c
```

- Entrada desde la representación de texto de una celda

```
cellFromText(text) → cell
```

Hay una función por tipo de celda, el nombre de la función tiene como prefijo el nombre del tipo, que es `h3index`, `quadbin` o `s2cell`. La función es la inversa de `asText` y lee las formas que acepta un literal del tipo, tanto de una constante como de una columna `text`.

```
SELECT h3indexFromText('0x831C02FFFFFFFFF');
-- 831c02fffffffff
SELECT quadbinFromText('48a6227affffffff');
-- 48a6227affffffff
SELECT quadbinFromText('0x48a6227affffffff');
-- 48a6227affffffff
SELECT s2cellFromText('47c3c');
-- 47c3c
```

- Devuelve la representación binaria conocida (WKB) de un ráster o de una tesela Raquet

```
asBinary({raster,raquet},endian text='') → bytea
```

```
SELECT length(asBinary(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326)));
-- 61
SELECT get_byte(asBinary(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326),
  'XDR'), 0);
-- 0
SELECT length(asBinary(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817fffffffffff', 'uint8')));
-- 31
```

- Devuelve la representación hexadecimal binaria conocida (HexWKB) de un ráster o de una tesela Raquet

```
asHexWKB({raster,raquet},endian text='') → text
```

```
SELECT length(asHexWKB(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326)));
-- 122
SELECT length(asHexWKB(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817fffffffffff', 'uint8')));
-- 62
```

- Devuelve un ráster o una tesela Raquet a partir de su representación binaria conocida (WKB)

```
rasterFromBinary(bytea) → raster
raquetFromBinary(bytea) → raquet
```

```
SELECT rasterFromBinary(asBinary(r, 'XDR')) = r
FROM (SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r) t;
-- true
SELECT raquetFromBinary(asBinary(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817fffffffffff', 'uint8')))
  = raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8');
-- true
```

- Devuelve un ráster o una tesela Raquet a partir de su representación hexadecimal binaria conocida (HexWKB)

```
rasterFromHexWKB(text) → raster
raquetFromHexWKB(text) → raquet
```

```
SELECT rasterFromHexWKB(asHexWKB(r, 'XDR')) = r
FROM (SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r) t;
-- true
SELECT raquetFromHexWKB(asHexWKB(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817fffffffffff', 'uint8')))
  = raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8');
-- true
```

16.3. Constructores

Una tesela `raquet` se construye a partir de bytes de píxel ya disponibles o decodificando un archivo ráster, y lleva su georreferenciación QUADBIN en el propio valor. Un `raster` de PostGIS se construye del mismo modo a partir de los bytes de los píxeles de una banda y de su georreferenciación, o como una cuadrícula sin banda a la que se añaden bandas y cuyos píxeles se fijan después.

- Construir una celda temporal que contiene una celda sobre un valor de tiempo

```
tcell(cell,timestamptz) → tcellInst
tcell(cell,tstzset) → tcellDiscSeq
tcell(cell,tstzspan) → tcellSeq
tcell(cell,tstzspanset) → tcellSeqSet
```

```
SELECT th3index(h3index '871fa4418ffffff', timestamptz '2001-01-01');
-- 871fa4418ffffff@2001-01-01
SELECT th3index(h3index '871fa4418ffffff', tstzspan '[2001-01-01, 2001-01-03]');
-- [871fa4418ffffff@2001-01-01, 871fa4418ffffff@2001-01-03]
SELECT tquadbin(quadbin '48a6227ffffffff', timestamptz '2001-01-01');
-- 48a6227ffffffff@2001-01-01
SELECT tquadbin(quadbin '48a6227ffffffff', tstzspan '[2001-01-01, 2001-01-03]');
-- [48a6227ffffffff@2001-01-01, 48a6227ffffffff@2001-01-03]
SELECT ts2cell(s2cell '47c3c', timestamptz '2001-01-01');
-- 47c3c@2001-01-01
SELECT ts2cell(s2cell '47c3c', tstzspan '[2001-01-01, 2001-01-03]');
-- [47c3c@2001-01-01, 47c3c@2001-01-03]
```

- Construir una celda temporal de un subtipo dado a partir de sus componentes

```
tcellInst(tcell) → tcellInst
tcellSeq(tcell[],interp text='step') → tcellSeq
tcellSeqSet(tcell[]) → tcellSeqSet
```

```
SELECT th3indexSeq(ARRAY[th3index '871fa4418ffffff@2001-01-01',
  th3index '871fa44a8ffffff@2001-01-02'], 'step');
-- [871fa4418ffffff@2001-01-01, 871fa44a8ffffff@2001-01-02]
SELECT tquadbinSeq(ARRAY[tquadbin '48a6227ffffffff@2001-01-01',
  tquadbin '485623fffffffff@2001-01-02'], 'step');
-- [48a6227ffffffff@2001-01-01, 485623fffffffff@2001-01-02]
SELECT ts2cellSeq(ARRAY[ts2cell '47c3c@2001-01-01', ts2cell '47c4@2001-01-02'], 'step');
-- [47c3c@2001-01-01, 47c4@2001-01-02]
```

- Construye una tesela ráster Raquet a partir de sus bytes de píxeles, dimensiones, celda QUADBIN y tipo de píxel

```
raquet(bytea,width integer,height integer,quadbin,pixtype text,
  nodata float) → raquet
```

Empaqueta un arreglo de píxeles `pixels` de banda única en orden de fila mayor de `width × height` píxeles de tipo `pixtype` (uno de `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32` o `float64`), georreferenciado por la celda `quadbin`, en un único valor `raquet` autodescriptivo. El argumento `nodata` es opcional; cuando se omite (`NULL`) la tesela no lleva valor centinela de nodata. Se genera un error cuando el arreglo `pixels` es menor que `width × height ×` el tamaño del píxel. Los píxeles de más de un byte son little-endian, que es el orden de bytes que les da la especificación RaQuet sea cual sea el orden de bytes del servidor, de modo que una tesela conserva sus valores cuando se lee en otra máquina. Un `raquet` es un valor de longitud variable, libre de GDAL, con una representación portátil en texto Hex-WKB y binaria, distinto de y coexistente con el tipo `raster` de PostGIS usado por rasterValue. Una tabla Raquet indica las dimensiones, el tipo de píxel y el valor nodata de sus teselas una sola vez, en la fila de metadatos en `block` = 0, que la consulta siguiente lee y deja fuera de las teselas: 0 no es un índice QUADBIN, por lo que la conversión a `quadbin` lo rechaza. El constructor lee los bytes de píxel tal como están, por lo que las bandas de un archivo Raquet cuyos metadatos indican una compresión `gzip` se descomprimen antes de llegar a él.

```
-- Package the tiles of a Raquet table, whose band_1 column is uncompressed,
-- into raquet values keyed by their QUADBIN cell
CREATE TABLE elevation_tiles AS
SELECT r.block::quadbin AS quadbin, raquet(r.band_1,
    (m.meta->'tiling'->>'block_width')::integer,
    (m.meta->'tiling'->>'block_height')::integer, r.block::quadbin,
    m.meta->'bands'->0->>'type', (m.meta->'bands'->0->>'nodata')::float8) AS tile
FROM elevation_raquet r,
  (SELECT metadata::jsonb AS meta FROM elevation_raquet WHERE block = 0 LIMIT 1) m
WHERE r.block <> 0;
-- A 2 x 2 UINT8 tile with no nodata
SELECT raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8');
```

- Construye un ráster de una banda a partir de los bytes de sus píxeles y de su georreferenciación

```
raster(pixels bytea,width integer,height integer,upperleftx float,upperlefty float,
  scalex float,scaley float,skewx float,skewy float,srid integer,pixeltype text,
  nodataval float=NULL) → raster
```

El gemelo del constructor raquet para un `raster` de PostGIS, en el que la georreferenciación de un ráster ocupa el lugar de la celda QUADBIN de una tesela. Los `pixels` son los que devuelve pixels: fila a fila, cada uno en los bytes de su tipo, en little-endian cuando ocupa más de un byte, y un byte para los tipos `1BB`, `2BUI` y `4BUI`, que acotan un píxel a menos de un byte. El `pixeltype` se nombra como lo nombra PostGIS, `8BUI`, `16BSI`, `32BF`, etc., o como lo devuelve bandPixelType, donde el nombre `uint8` designa `8BUI`. Una banda ráster no tiene flotante de 16 bits ni entero de 64 bits, por lo que se rechazan `float16`, `int64` y `uint64`, así como unos bytes de otro número que el que ocupan los píxeles y un byte que supera la cota de un tipo que acota un píxel a menos de un byte. El `nodataval` es opcional; cuando se omite, la banda no indica valor nodata. Un ráster de una banda es el ráster construido a partir de sus propios píxeles y su georreferenciación, `raster(pixels(r), width(r), height(r), upperLeftX(r), upperLeftY(r), scaleX(r), scaleY(r), skewX(r), skewY(r), SRID(r), bandPixelType(r), bandNoDataValue(r))`, y conserva exactamente cada valor que contiene. Una banda de `1BB`, `2BUI` o `4BUI` ocupa un byte por píxel, como una de `8BUI`, y bandPixelType la nombra `uint8`, por lo que se reconstruye como `8BUI` con los mismos bytes, siendo la cota que indica su nombre lo único que pierde. Las demás bandas se añaden con addBand.

```
SELECT dumpValues(raster('\x0100feff2c010080'::bytea, 2, 2, 0, 2, 1, -1, 0, 0,
  3857, '16BSI'), 1);
-- {{1,-2},{300,-32768}}
```

- Construye un ráster sin banda

```
makeEmptyRaster(width integer,height integer,upperleftx float,upperlefty float,
  scalex float,scaley float,skewx float,skewy float,srid integer=0) → raster
```

El ráster que devuelve `ST_MakeEmptyRaster` de PostGIS: una cuadrícula de `width` × `height` píxeles, cada dimensión de 0 a 65535, georreferenciada por su esquina superior izquierda, su escala y su sesgo en el sistema de referencia espacial `srid`.

```
SELECT width(r), height(r), numBands(r), scaleX(r)
FROM (SELECT makeEmptyRaster(3, 2, 10, 20, 2, -1, 0, 0, 3857) AS r) t;
-- 3 | 2 | 0 | 2
```

- Añade una banda a un ráster

```
addBand(rast raster,index integer,pixeltype text,initialvalue float=0,
  nodataval float=NULL) → raster
addBand(torast raster,fromrast raster,fromband integer=1,
  torastindex integer=NULL) → raster
```

El ráster que devuelve `ST_AddBand` de PostGIS. La primera forma añade una banda de tipo `pixeltype`, nombrado como lo lee raster, en la que cada píxel contiene `initialvalue`, y que indica el valor nodata `nodataval` o ninguno cuando se omite. La segunda forma añade una copia de la banda `fromband` de `fromrast`, un ráster del mismo ancho y alto. La banda toma el número `index` o `torastindex`, de 1 a uno más que el número de bandas del ráster, y se añade tras la última banda cuando el número es `NULL`, escrito `NULL::integer` en la primera forma para que no se lea como un ráster de la segunda. Se rechazan un número fuera de ese rango, una banda que `fromrast` no tiene y un ráster de otro tamaño, allí donde PostGIS añade la banda en otro lugar o devuelve el ráster sin cambios.

```
SELECT numBands(r), bandPixelType(r, 1), bandNoDataValue(r, 2)
FROM (SELECT addBand(addBand(makeEmptyRaster(2, 2, 0, 2, 1, -1, 0, 0, 3857),
  NULL::integer, '8BUI', 1), 1, '32BF', 0, -9999) AS r) t;
-- 2 | float32 | NULL
SELECT numBands(addBand(r, r))
FROM (SELECT addBand(makeEmptyRaster(2, 2, 0, 2, 1, -1, 0, 0, 3857),
  NULL::integer, '8BUI', 1) AS r) t;
-- 2
```

- Decodifica un archivo ráster en una tesela ráster Raquet mediante GDAL

```
raquetRead(bytea,quadbin) → raquet
raquetRead(text,quadbin) → raquet
```

Lee los bytes de `rasterfile`, un ráster en cualquier formato compatible con GDAL (GeoTIFF, PNG, ...), a través del sistema de archivos virtual `/vsimem/` de GDAL, y empaqueta su primera banda, en orden de fila mayor, en un único valor `raquet` autodescriptivo georreferenciado por la celda `quadbin`. El tipo de datos de la banda debe ser uno de `Byte`, `Int16`, `Int32`, `Float32` o `Float64`; el valor de nodata de la banda, cuando está definido, se traslada a la tesela. Como el archivo se suministra como bytes, no se requiere acceso a archivos del lado del servidor. GDAL realiza la decodificación, por lo que el controlador de formato del ráster debe estar permitido por la configuración `postgis.gdal_enabled_drivers` de PostGIS (que deshabilita todos los controladores de forma predeterminada); habilítelo con, por ejemplo, `SET postgis.gdal_enabled_drivers = 'ENABLE_ALL'`. Esta es la contraparte de ingesta del constructor raquet, que construye una tesela a partir de bytes de píxeles ya decodificados. Cuando se omite `quadbin` o es `NULL`, el identificador de la tesela se deriva de la georreferenciación del ráster: el ráster debe llevar una referencia espacial `EPSG:3857` (Web-Mercator) y una geotransformación alineada con los ejes que describa una única tesela QUADBIN. Un ráster sin referencia espacial, uno que no esté en `EPSG:3857`, o una geotransformación rotada o no alineada con la cuadrícula de teselas provoca un error en lugar de ser forzado. La forma que recibe `text` lee en su lugar el archivo ráster del servidor en esa ruta, allí donde PostGIS permite leer una banda ráster almacenada fuera de la base de datos, como indica rasterValue.

```
-- Decode a GeoTIFF stored in a bytea column into a raquet tile
SELECT raquetRead(file_bytes, quadbin) AS tile FROM elevation_files;
-- Derive the QUADBIN cell from an EPSG:3857 GeoTIFF's georeferencing
SELECT raquetRead(file_bytes) AS tile FROM elevation_files;
```

16.4. Conversión de tipos

Un valor `tcell` comparte la misma representación en disco que `tbigint`, y una celda la misma representación que `bigint`. Una conversión entre ellos es una conversión de asignación: se aplica en una asignación, como al insertar un `tbigint` en una columna `tcell`, y en los demás casos una consulta escribe `::`, de modo que las funciones específicas de celdas no puedan invocarse silenciosamente sobre enteros arbitrarios.

Una tesela se convierte en la caja espaciotemporal de su huella, que es lo que da un índice espacial a una tabla de teselas.

- Convertir entre un `tcell` y un `tbigint`

```
{bigint,quadbin}::{quadbin,bigint}
{bigint,s2cell}::{s2cell,bigint}
{tbigint,tcell}::{tcell,tbigint}
```

La conversión desde un tipo de celda es una coerción binaria, ya que toda celda es un entero. La conversión hacia un tipo de celda comprueba cada valor como lo hace la entrada de texto del tipo y provoca un error en caso contrario, como lo hacen las entradas MF-JSON y WKB del tipo: un valor S2 debe ser una celda de su cuadrícula, un valor QUADBIN un índice bien formado de cualquier modo, como los que contiene el tipo `quadbin`, y un valor H3 una celda, una arista dirigida o un vértice, ya que un índice H3 nombra los tres. El tipo `h3index` y sus conversiones desde y hacia `bigint` son los de la extensión H3. La conversión da la vuelta completa: convertir una celda a un entero y de vuelta devuelve la celda de la que se partió.

```
SELECT (tbigint '590464338553208831@2001-01-01')::th3index;
-- 831c02fffffffff@2001-01-01
SELECT (tbigint '523490952854110207 9@2001-01-01')::tquadbin;
-- 48a6227affffffff@2001-01-01
SELECT (tbigint '5171187903383994368@2001-01-01')::ts2cell;
-- 47c3c@2001-01-01
SELECT 5234909528541102079::bigint::quadbin;
-- 48a6227affffffff
SELECT (tbigint '42@2001-01-01')::tquadbin;
-- ERROR:  The value 42 does not encode a valid quadbin index
```

- Convertir una celda temporal en un punto temporal geodésico o plano de centroides de celda

```
{th3index,tquadbin,ts2cell}::{tgeogpoint,tgeompoint}
```

Las conversiones siguen la regla de [?listitem].

La conversión de una celda QUADBIN responde un punto del plano Web-Mercator sobre el que se define la cuadrícula, mientras que la conversión de una celda S2 responde uno geodésico; cada cuadrícula declara la única conversión que exige su propia definición. H3 declara ambas, de modo que es la única cuadrícula en la que quien llama elige entre la representación geodésica y la plana en SRID 4326.

```
SELECT asText((th3index '831c02fffffffff@2001-01-01')::tgeogpoint, 6);
-- POINT(-144.523991 49.716503)@2001-01-01
SELECT asText((th3index '831c02fffffffff@2001-01-01')::tgeompoint, 6);
-- POINT(-144.523991 49.716503)@2001-01-01
SELECT asText((tquadbin '48a6227affffffff@2001-01-01')::tgeompoint, 6);
-- POINT(4.394531 50.847573)@2001-01-01
SELECT asText((ts2cell '47c3c@2001-01-01')::tgeogpoint, 6);
-- POINT(4.222019 50.936164)@2001-01-01
```

- Convierte una tesela Raquet en una caja espaciotemporal

```
stbox(raquet) → stbox
```

Devuelve la huella de la tesela: la envolvente de longitud y latitud de su celda QUADBIN, en SRID 4326 y sin dimensión temporal. La huella se deriva únicamente de la celda, por lo que teselas que difieren en píxeles, dimensiones o tipo de píxel la comparten. La conversión también está disponible como conversión de tipo.

```
-- The footprint of a tile covering the north-eastern quadrant at zoom 1
SELECT round(stbox(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff',
  'uint8')), 6);
-- SRID=4326;STBOX X((0,0),(180,85.051129))
-- Keep the tiles whose footprint overlaps a region of interest
SELECT * FROM elevation_tiles WHERE tile::stbox && stbox 'SRID=4326;
  STBOX X((0,0),(30,30))';
```

La huella admite los operadores, agregados y clases de operadores de `stbox`, por lo que una tabla de teselas se consulta por solapamiento espacial en cualquier nivel de zoom en lugar de por una prueba de igualdad sobre la celda QUADBIN en uno fijo. Indexar la conversión dota a la tabla de teselas de un índice espacial, y la extensión de un conjunto de teselas es la extensión de sus huellas.

```
-- A spatial index over the tile footprints
CREATE INDEX elevation_tiles_gist ON elevation_tiles USING gist (stbox(tile));
CREATE INDEX elevation_tiles_spgist ON elevation_tiles USING spgist (stbox(tile));
-- The extent covered by a set of tiles
SELECT extent(stbox(tile)) FROM elevation_tiles;
-- Tiles a trajectory passes through, at whatever zoom levels the table holds
SELECT t.tripid, r.tile FROM trips t,
  elevation_tiles r WHERE stbox(r.tile) && stbox(t.trip);
```

- Convierte un ráster en una caja espaciotemporal

```
stbox(raster) → stbox
```

Devuelve la extensión del ráster en el sistema de referencia que el propio ráster declara, sin dimensión temporal. Esta es la única forma de un `raster` de PostGIS que la extensión declara, porque PostGIS no tiene `stbox`: el ancho, el alto, la georreferenciación y las propiedades de las bandas se leen con los accesores propios de PostGIS, que aquí no se repiten. La conversión también está disponible como una conversión de tipo.

```
WITH rast AS (
  SELECT ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r
)
SELECT stbox(r) AS box, stbox(r) = r::stbox AS cast_agrees FROM rast;
-- SRID=4326;STBOX X((0,0),(3,3)) | t
-- The box states the reference system of the raster, not a fixed one
SELECT stbox(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857));
-- SRID=3857;STBOX X((0,0),(2,2))
```

La extensión lleva la rotación de la geotransformación, de modo que un ráster sesgado responde la caja que contiene sus cuatro esquinas rotadas y no la que daría su escala por sí sola.

```
WITH plain AS (
  SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r
), skewed AS (
  SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 1.0, 0.0, 4326) AS r
)
SELECT stbox((SELECT r FROM plain)) AS plain_box,
  stbox((SELECT r FROM skewed)) AS skewed_box;
-- SRID=4326;STBOX X((0,0),(2,2)) | SRID=4326;STBOX X((0,0),(4,2))
```

La extensión lleva los operadores, las agregaciones y las clases de operadores de `stbox`, de modo que una tabla de rásteres se consulta por solapamiento espacial y se indexa sobre la conversión.

```
SELECT stbox(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326)) &&
  stbox 'SRID=4326;STBOX X((1,1),(2,2))';
-- t
```

16.5. Accesores

numBands, SRID y los accesores de la esquina, la escala y la inclinación leen la malla de un `raster` de PostGIS; width, height y los accesores de banda leen por igual un ráster y una tesela `raquet`, y los demás accesores leen una tesela `raquet`.

Un valor `raquet` es autodescriptivo: los accesores siguientes leen la georreferenciación y la disposición que transporta, de modo que una tesela empaquetada con el constructor raquet o decodificada con raquetRead puede inspeccionarse sin conservar las columnas sueltas a partir de las cuales se construyó.

- Devolver el primer, el último o el enésimo identificador de celda distinto de un valor temporal

```
startValue(tcell) → cell
endValue(tcell) → cell
valueN(tcell,integer) → cell
```

`valueN` cuenta desde uno y devuelve `NULL` cuando `n` está fuera de rango.

```
SELECT startValue(th3index '831c02fffffffff@2001-01-01');
-- 831c02fffffffff
SELECT endValue(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}');
-- 831c00fffffffff
SELECT valueN(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}', 1);
-- 831c02fffffffff
SELECT startValue(tquadbin '48a6227affffffff@2001-01-01');
-- 48a6227affffffff
SELECT endValue(tquadbin '{48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-02}');
-- 485623fffffffffff
SELECT valueN(tquadbin '{48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-02}', 1);
-- 48a6227affffffff
SELECT startValue(ts2cell '47c3c@2001-01-01');
-- 47c3c
SELECT endValue(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}');
-- 47c4
SELECT valueN(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}', 1);
-- 47c3c
```

- Devolver el conjunto de identificadores de celda distintos que toma la trayectoria

```
getValues(tcell) → cellset
```

```
SELECT getValues(th3index '831c02fffffffff@2001-01-01');
-- {"831c02fffffffff"}
SELECT getValues(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}');
-- {"831c00fffffffff", "831c02fffffffff"}
SELECT getValues(tquadbin '48a6227affffffff@2001-01-01');
-- {"48a6227affffffff"}
SELECT getValues(tquadbin '{48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-02}');
-- {"485623fffffffffff", "48a6227affffffff"}
SELECT getValues(ts2cell '47c3c@2001-01-01');
-- {"47c3c"}
SELECT getValues(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}');
-- {"47c3c", "47c4"}
```

- Devolver el identificador de celda en una marca de tiempo dada, usando interpolación de pasos

```
valueAtTimestamp(tcell,timestamptz) → cell
```

```
SELECT valueAtTimestamp(th3index
  '[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-05]', '2001-01-03');
-- 831c02fffffffff
```

```
SELECT valueAtTimestamp(tquadbin
  '[48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-05]', '2001-01-03');
-- 48a6227affffffff
SELECT valueAtTimestamp(ts2cell
  '[47c3c@2001-01-01, 47c4@2001-01-05]', '2001-01-03');
-- 47c3c
```

- Devuelve el número de bandas de un ráster

```
numBands(raster) → integer
```

El número de banda que aceptan rasterValue y los operadores construidos sobre él va de 1 hasta este valor.

```
WITH rast1 AS (
  SELECT ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326),
    '32BF'::text, 0.0::float8, NULL::float8 ) AS r ), rast2 AS (
  SELECT ST_AddBand(r, '32BF'::text, 0.0::float8, NULL::float8) AS r FROM rast1 )
SELECT numBands((SELECT r FROM rast1)) AS num_bands_one,
  numBands((SELECT r FROM rast2)) AS num_bands_two;
-- 1 | 2
```

- Devuelve el identificador del sistema de referencia espacial de un ráster

```
SRID(raster) → integer
```

```
SELECT SRID(ST_MakeEmptyRaster(3, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857));
-- 3857
```

- Devuelve la coordenada X o Y de la esquina superior izquierda de un ráster

```
upperLeftX(raster) → float
upperLeftY(raster) → float
```

La esquina es el origen que establece la geotransformación, alrededor del cual la inclinación gira la malla, de modo que es una esquina de la extensión solo cuando ambas inclinaciones son cero.

```
SELECT upperLeftX(r), upperLeftY(r)
FROM (SELECT ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 1.5, -2.0, 0.5, 0.25, 3857) AS r) t;
-- 10 | 20
```

- Devuelve el ancho o el alto de píxel de un ráster, es decir, la componente X o Y de su escala

```
scaleX(raster) → float
scaleY(raster) → float
```

El alto de píxel es negativo para una malla cuyas filas van de norte a sur, que es como se escribe habitualmente un ráster.

```
SELECT scaleX(r), scaleY(r)
FROM (SELECT ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 1.5, -2.0, 0.5, 0.25, 3857) AS r) t;
-- 1.5 | -2
```

- Devuelve la componente X o Y de la inclinación de un ráster

```
skewX(raster) → float
skewY(raster) → float
```

Un ráster cuyas dos inclinaciones son cero está alineado con los ejes.

```
SELECT skewX(r), skewY(r)
FROM (SELECT ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 1.5, -2.0, 0.5, 0.25, 3857) AS r) t;
-- 0.5 | 0.25
```

- Devuelve la celda QUADBIN de una tesela Raquet

```
quadbin(raquet) → quadbin
```

```
SELECT quadbin(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8'));
-- 4817fffffffffff
```

- Devuelve el ancho en píxeles de un ráster o de una tesela Raquet

```
width({raster,raquet}) → integer
```

```
SELECT width(ST_MakeEmptyRaster(3, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326));
-- 3
SELECT width(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8'));
-- 2
```

- Devuelve el alto en píxeles de un ráster o de una tesela Raquet

```
height({raster,raquet}) → integer
```

```
SELECT height(ST_MakeEmptyRaster(3, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326));
-- 2
SELECT height(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8'));
-- 2
```

- Devuelve el nombre del tipo de dato de píxel de una banda de un ráster o de una tesela Raquet

```
bandPixelType(raster,band integer=1) → text
bandPixelType(raquet) → text
```

El nombre devuelto es el que aceptan los constructores, es decir, uno de `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32` o `float64`. Los constructores leen el nombre sin distinguir mayúsculas de minúsculas, de modo que el nombre que el ráster de PostGIS da a un tipo de píxel pasa a ellos tal cual; el nombre devuelto aquí es la grafía en minúsculas que la especificación RaQuet da al campo `type` de una tesela, tanto para una banda de PostGIS como para una tesela, de modo que una banda y una tesela del mismo tipo informan el mismo nombre.

```
SELECT bandPixelType(ST_AddBand(ST_MakeEmptyRaster(1, 1, 0, 0, 1), '16BSI'));
-- int16
-- Read back the layout of a packaged tile
SELECT quadbin(tile), width(tile), height(tile), bandPixelType(tile)
FROM (SELECT raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8')
  AS tile) t;
-- 4817fffffffffff | 2 | 2 | uint8
```

- Devuelve si una banda de un ráster o una tesela Raquet establece un valor nodata

```
bandHasNoDataValue(raster,band integer=1) → boolean
bandHasNoDataValue(raquet) → boolean
```

Una banda que no establece ninguno no tiene píxel que excluir, de modo que cada píxel que contiene lleva un valor.

```
SELECT bandHasNoDataValue(ST_AddBand(ST_MakeEmptyRaster(1, 1, 0, 0, 1), '16BSI'::text,
  0::float8, -9999::float8));
-- true
SELECT bandHasNoDataValue(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff',
  'uint8'));
-- false
```

- Devuelve el valor nodata de una banda de un ráster o de una tesela Raquet

```
bandNoDataValue(raster,band integer=1) → float
bandNoDataValue(raquet) → float
```

Una banda que no establece ningún valor nodata devuelve `NULL`.

```
SELECT bandNoDataValue(ST_AddBand(ST_MakeEmptyRaster(1, 1, 0, 0, 1), '16BSI'::text,
  0::float8, -9999::float8));
-- -9999
SELECT bandNoDataValue(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff',
  'uint8', 255));
-- 255
-- A tile built without a nodata value states none
SELECT bandNoDataValue(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff',
  'uint8'));
-- NULL
```

- Devuelve los bytes de píxeles de una banda de un ráster o de una tesela Raquet

```
pixels(rast raster,nband integer=1) → bytea
pixels(raquet) → bytea
```

Los bytes están en orden de fila mayor y empaquetados, `width` × `height` píxeles del tamaño de bandPixelType cada uno, en little-endian cuando un píxel ocupa más de un byte, que es la disposición que aceptan los constructores. Por lo tanto, una tesela y una banda de un ráster se reconstruyen a través de sus propios accesores, y cada valor que contiene la banda se conserva exactamente, allí donde dumpValues devuelve sus valores como números de doble precisión. Una banda de un tipo que acota un píxel a menos de un byte ocupa un byte por píxel.

```
SELECT pixels(raquet('\x01020304'::bytea, 2, 2, quadbin '4817fffffffffff', 'uint8'));
-- \x01020304
SELECT pixels(raster('\x0100feff2c010080'::bytea, 2, 2, 0, 2, 1, -1, 0, 0,
  3857, '16BSI'));
-- \x0100feff2c010080
```

16.6. Transformaciones

- Desplazar, escalar, o desplazar y escalar el lapso de tiempo de una celda temporal

```
shiftTime(tcell,interval) → tcell
scaleTime(tcell,interval) → tcell
shiftScaleTime(tcell,interval,interval) → tcell
```

```
SELECT shiftTime(th3index '871fa4418ffffff@2001-01-01', interval '1 day');
-- 871fa4418ffffff@2001-01-02
SELECT scaleTime(th3index '[871fa4418ffffff@2001-01-01,
  871fa44a8ffffff@2001-01-03]', interval '4 days');
-- [871fa4418ffffff@2001-01-01, 871fa44a8ffffff@2001-01-05]
SELECT shiftTime(tquadbin '48a6227affffffff@2001-01-01', interval '1 day');
-- 48a6227affffffff@2001-01-02
SELECT scaleTime(tquadbin '[48a6227affffffff@2001-01-01,
  485623fffffffffff@2001-01-03]', interval '4 days');
-- [48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-05]
SELECT shiftTime(ts2cell '47c3c@2001-01-01', interval '1 day');
-- 47c3c@2001-01-02
SELECT scaleTime(ts2cell '[47c3c@2001-01-01, 47c4@2001-01-03]', interval '4 days');
-- [47c3c@2001-01-01, 47c4@2001-01-05]
```

- Añadir un instante temporal o una secuencia temporal

```
appendInstant(tcell,tcellInst) → tcell
appendSequence(tcell,tcellSeq) → tcell
```

```
SELECT asText(appendInstant(th3index '831c02fffffffff@2001-01-01',
  th3index '831c00fffffffff@2001-01-02'));
-- [831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]
SELECT asText(appendSequence(th3index '[831c02fffffffff@2001-01-01]',
  th3index '[831c00fffffffff@2001-01-02]'));
-- {[831c02fffffffff@2001-01-01], [831c00fffffffff@2001-01-02]}
SELECT asText(appendInstant(tquadbin '48a6227affffffff@2001-01-01',
  tquadbin '485623fffffffffff@2001-01-02'));
-- [48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-02]
SELECT asText(appendSequence(tquadbin '[48a6227affffffff@2001-01-01]',
  tquadbin '[485623fffffffffff@2001-01-02]'));
-- {[48a6227affffffff@2001-01-01], [485623fffffffffff@2001-01-02]}
SELECT asText(appendInstant(ts2cell '47c3c@2001-01-01',
  ts2cell '47c4@2001-01-02'));
-- [47c3c@2001-01-01, 47c4@2001-01-02]
SELECT asText(appendSequence(ts2cell '[47c3c@2001-01-01]',
  ts2cell '[47c4@2001-01-02]'));
-- {[47c3c@2001-01-01], [47c4@2001-01-02]}
```

- Fusionar las celdas temporales

```
merge(tcell,tcell) → tcell
merge(tcell[]) → tcell
```

```
SELECT asText(merge(th3index '[831c02fffffffff@2001-01-01, 831c02fffffffff@2001-01-03]',
  th3index '[831c02fffffffff@2001-01-03, 831c00fffffffff@2001-01-05]'));
-- [831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-05]
SELECT asText(merge(ARRAY[th3index '831c02fffffffff@2001-01-01',
  th3index '831c00fffffffff@2001-01-03']));
-- {831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-03}
SELECT asText(merge(tquadbin '[48a6227affffffff@2001-01-01, 48a6227affffffff@2001-01-03]',
  tquadbin '[48a6227affffffff@2001-01-03, 485623fffffffffff@2001-01-05]'));
-- [48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-05]
SELECT asText(merge(ARRAY[tquadbin '48a6227affffffff@2001-01-01',
  tquadbin '485623fffffffffff@2001-01-03']));
-- {48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-03}
SELECT asText(merge(ts2cell '[47c3c@2001-01-01, 47c3c@2001-01-03]',
  ts2cell '[47c3c@2001-01-03, 47c4@2001-01-05]'));
-- [47c3c@2001-01-01, 47c4@2001-01-05]
SELECT asText(merge(ARRAY[ts2cell '47c3c@2001-01-01',
  ts2cell '47c4@2001-01-03']));
-- {47c3c@2001-01-01, 47c4@2001-01-03}
```

- Establecer la interpolación de una celda temporal

```
setInterp(tcell,interp text) → tcell
```

```
SELECT setInterp(th3index '{871fa4418ffffff@2001-01-01,
  871fa44a8ffffff@2001-01-02}', 'step');
-- {[871fa4418ffffff@2001-01-01], [871fa44a8ffffff@2001-01-02]}
SELECT setInterp(tquadbin '{48a6227affffffff@2001-01-01,
  485623fffffffffff@2001-01-02}', 'step');
-- {[48a6227affffffff@2001-01-01], [485623fffffffffff@2001-01-02]}
SELECT setInterp(ts2cell '{47c3c@2001-01-01,
  47c4@2001-01-02}', 'step');
-- {[47c3c@2001-01-01], [47c4@2001-01-02]}
```

16.7. Modificaciones

- Insertar una celda temporal en otra, o actualizar una con otra

```
insert(tcell,tcell,connect boolean=true) → tcell
update(tcell,tcell,connect boolean=true) → tcell
```

```
SELECT insert(th3index '[871fa4418ffffff@2001-01-01,
  871fa4418ffffff@2001-01-02]', th3index '[871fa44a8ffffff@2001-01-03,
  871fa44a8ffffff@2001-01-04]');
-- [871fa4418ffffff@2001-01-01, 871fa44a8ffffff@2001-01-03, 871fa44a8ffffff@2001-01-04]
SELECT insert(tquadbin '[48a6227affffffff@2001-01-01,
  48a6227affffffff@2001-01-02]', tquadbin '[485623fffffffffff@2001-01-03,
  485623fffffffffff@2001-01-04]');
/* [48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-03,
   485623fffffffffff@2001-01-04] */
SELECT insert(ts2cell '[47c3c@2001-01-01,
  47c3c@2001-01-02]', ts2cell '[47c4@2001-01-03,
  47c4@2001-01-04]');
-- [47c3c@2001-01-01, 47c4@2001-01-03, 47c4@2001-01-04]
```

- Eliminar un valor de tiempo de una celda temporal

```
deleteTime(tcell,times,connect boolean=true) → tcell
```

```
SELECT deleteTime(th3index '[871fa4418ffffff@2001-01-01,
  871fa4418ffffff@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [871fa4418ffffff@2001-01-01, 871fa4418ffffff@2001-01-04]
SELECT deleteTime(tquadbin '[48a6227affffffff@2001-01-01,
  48a6227affffffff@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [48a6227affffffff@2001-01-01, 48a6227affffffff@2001-01-04]
SELECT deleteTime(ts2cell '[47c3c@2001-01-01,
  47c3c@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [47c3c@2001-01-01, 47c3c@2001-01-04]
```

- Fija los valores de un bloque de píxeles de una banda de un ráster

```
setValues(rast raster,nband integer,x integer,y integer,newvalueset float[],
  noset boolean[]=NULL,keepnodata boolean=false) → raster
```

El ráster que devuelve `ST_SetValues` de PostGIS: la banda `nband` toma los valores de `newvalueset`, un arreglo de una fila o de filas de igual longitud, sobre el bloque de píxeles cuyo píxel superior izquierdo está en la columna `x` y la fila `y`, ambas a partir de 1. Un valor `NULL` fija el píxel al valor nodata de la banda, o al menor valor de su tipo cuando la banda no indica ninguno. Un píxel cuya marca en `noset` es verdadera queda como está, leyéndose las marcas sobre el bloque que comparten con los valores, y lo mismo un píxel que contiene el valor nodata cuando `keepnodata` es verdadero. Los píxeles del bloque que quedan fuera del ráster se omiten, y un valor que excede el tipo de píxel se lee en él como lo lee PostGIS. Se rechazan una banda que el ráster no tiene, un píxel `x`, `y` fuera de él y un arreglo de más de dos dimensiones.

```
SELECT dumpValues(setValues(addBand(makeEmptyRaster(3, 2, 0, 2, 1, -1, 0, 0,
  3857), NULL::integer, '16BSI', 0, -1), 1, 2, 1, ARRAY[[1, 2], [NULL, 4]]), 1);
-- {{0,1,2},{0,NULL,4}}
```

16.8. Restricciones

- Restringir una celda temporal a (el complemento de) una celda o un conjunto de celdas

```
atValue(tcell,cell) → tcell
minusValue(tcell,cell) → tcell
atValues(tcell,cellset) → tcell
minusValues(tcell,cellset) → tcell
```

```
SELECT atValue(th3index '{871fa4418ffffff@2001-01-01,
  871fa44a8ffffff@2001-01-02}', h3index '871fa4418ffffff');
-- {871fa4418ffffff@2001-01-01}
SELECT atValue(tquadbin '{48a6227affffffff@2001-01-01, 485623fffffffffff@2001-01-02}',
  quadbin '48a6227affffffff');
-- {48a6227affffffff@2001-01-01}
SELECT atValue(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}',
  s2cell '47c3c');
-- {47c3c@2001-01-01}
```

- Restringir una celda temporal a (el complemento de) un valor de tiempo

```
atTime(tcell,times) → tcell
minusTime(tcell,times) → tcell
```

```
SELECT atTime(th3index '[871fa4418ffffff@2001-01-01,
  871fa4418ffffff@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [871fa4418ffffff@2001-01-02, 871fa4418ffffff@2001-01-03]
SELECT atTime(tquadbin '[48a6227affffffff@2001-01-01, 48a6227affffffff@2001-01-04]',
  tstzspan '[2001-01-02, 2001-01-03]');
-- [48a6227affffffff@2001-01-02, 48a6227affffffff@2001-01-03]
SELECT atTime(ts2cell '[47c3c@2001-01-01, 47c3c@2001-01-04]',
  tstzspan '[2001-01-02, 2001-01-03]');
-- [47c3c@2001-01-02, 47c3c@2001-01-03]
```

16.9. Cobertura de geometrías estáticas

Estas funciones convierten una geometría ordinaria (no temporal) en la celda o el conjunto de celdas que la cubre en una resolución dada, y comprueban si una trayectoria de celdas temporales entra alguna vez en ese conjunto de celdas. La salida del conjunto de celdas junto con el predicado de intersección alguna vez forman el prefiltro espacial multiplataforma que usan las consultas portables de BerlinMOD.

- Devolver la celda de una cuadrícula que contiene una geometría de punto en la resolución dada

```
latLngToCell(geometry,resolution integer) → h3index
geoToQuadbinCell(geometry,integer) → quadbin
geoToS2Cell(geography,integer) → s2cell
```

La geometría debe ser un `POINT` no vacío, su longitud y su latitud se interpretan en el SRID 4326, y la resolución está entre 0 y 15 para H3, entre 0 y 26 para QUADBIN y entre 0 y 30 para S2; cualquier otro argumento provoca un error. La forma H3 toma el nombre de la función `h3_latlng_to_cell` de h3-pg; QUADBIN y S2 no tienen extensión anfitriona, y sus formas conservan los nombres que les da MobilityDB.

```
SELECT latLngToCell(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- 871fa4418ffffff
SELECT geoToQuadbinCell(geometry 'SRID=4326;Point(4.35 50.85)', 10);
-- 48a6227affffffff
SELECT geoToS2Cell(geography 'SRID=4326;Point(4.35 50.85)', 10);
-- 47c3c3
```

- Devolver el conjunto de celdas de una cuadrícula que cubren una geometría en la resolución dada

```
geoToH3IndexSet(geometry,integer) → h3indexset
geoToQuadbinSet(geometry,integer) → quadbinset
geoToS2CellSet(geography,integer) → s2cellset
```

El conjunto contiene las celdas que contienen un punto de la geometría, asignado cada punto a su celda como lo asigna `latLngToCell`, `geoToQuadbinCell` o `geoToS2Cell`. Es el conjunto del que toma sus valores una celda temporal de una trayectoria, de modo que una prueba de conjunto de celdas sobre él nunca pierde una trayectoria que comparte un punto con la geometría. Un `POINT` produce una única celda, una `LINESTRING` las celdas por las que pasan sus segmentos, un `POLYGON` las de sus anillos junto con las que encierra, y los valores `MULTI*` / `GEOMETRYCOLLECTION` devuelven la unión de sus componentes. Una curva, un `TIN` o un `TRIANGLE`, por sí solos o dentro de una colección, se rechazan, ya que una cobertura que omitiera sus celdas descartaría una trayectoria que el prefiltro debe conservar. H3 recorre cada segmento de celda en celda a través de los bordes de las celdas y, para un polígono, toma las celdas que no contienen ningún punto de un anillo y cuyo centro contiene el polígono, leyéndolas hacia fuera desde las celdas de los anillos. Una tesela QUADBIN está delimitada por dos meridianos y dos paralelos, de modo que QUADBIN recorre cada segmento de tesela en tesela y rellena las filas de teselas entre las teselas del borde de un polígono. Una celda S2 está delimitada por cuatro arcos de círculos máximos, el camino que sigue un segmento geodésico, de modo que S2 recorre cada segmento de una geografía de celda en celda y lee las caras del cubo hacia abajo en la jerarquía de celdas para obtener las celdas que encierra un polígono. Esta cobertura es la que necesita un prefiltro espacial, donde la función `h3_polygon_to_cells` de h3-pg devuelve las celdas cuyo centro contiene un polígono, razón por la que el nombre sigue siendo el de MobilityDB.

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- {"871fa4418ffffff"}
SELECT geoToQuadbinSet(geometry 'SRID=4326;Point(4.35 50.85)', 10);
-- {"48a6227affffffff"}
SELECT geoToS2CellSet(geography 'SRID=4326;Point(4.35 50.85)', 10);
-- {"47c3c3"}
```

- Devolver si una trayectoria de celdas temporales toma alguna vez una de las celdas de un conjunto de celdas

```
h3indexset ?= th3index → boolean
quadbinset ?= tquadbin → boolean
s2cellset ?= ts2cell → boolean
```

Combinado con el constructor del conjunto de celdas responde si un viaje pasa alguna vez por una región sin materializar la geometría de la trayectoria, un prefiltro conservador para el `eIntersects` exacto. Descansa sobre un núcleo que comparten todas las cuadrículas.

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7) ?=
  th3index '871fa4418ffffff@2001-01-01';
-- t
SELECT geoToQuadbinSet(geometry 'SRID=4326;Point(4.35 50.85)', 10) ?=
  tquadbin '48a6227affffffff@2001-01-01';
-- t
SELECT geoToS2CellSet(geography 'SRID=4326;Point(4.35 50.85)', 10) ?=
  ts2cell '47c3c3@2001-01-01';
-- t
```

16.10. Inspección

- Devolver la resolución de cada celda de una trayectoria

```
getResolution(cell) → integer
getResolution(tcell) → tint
```

Devolver la resolución de una celda, como un `integer` desde la forma estática y como un `tint` de la resolución de cada celda de una trayectoria desde la temporal. QUADBIN la llama nivel de zoom y S2 nivel, y el rango pertenece a la cuadrícula: de 0 a 15 para H3, de 0 a 26 para QUADBIN, de 0 a 30 para S2.

```
SELECT getResolution(h3index '831c02fffffffff');
-- 3
SELECT getResolution(th3index '831c02fffffffff@2001-01-01');
-- 3@2001-01-01
SELECT getResolution(quadbin '48a6227affffffff');
-- 10
SELECT getResolution(tquadbin '48a6227affffffff@2001-01-01');
-- 10@2001-01-01
SELECT getResolution(s2cell '47c3c');
-- 7
SELECT getResolution(ts2cell '47c3c@2001-01-01');
-- 7@2001-01-01
```

- Devolver un booleano temporal que indica en cada instante si el valor es una celda válida

```
isValidCell(cell) → boolean
isValidCell(tcell) → tbool
```

Un valor es una celda válida cuando su patrón de bits denota algo que su cuadrícula indexa: para QUADBIN, coordenadas de tesela dentro de los límites de la resolución que declara el identificador; para H3, una celda base y una secuencia de dígitos que lleva el icosaedro; para S2, una cara y una posición de Hilbert en un nivel entre 0 y 30.

Solo a H3 se le puede plantear la pregunta sobre un valor que responde `false`: su función de entrada admite cualquier patrón de bits y deja el veredicto a esta función, mientras que las funciones de entrada de QUADBIN y de S2 rechazan un patrón que no indexa nada, de modo que `tquadbin '0'` y `ts2cell '0'` provocan un error antes de que se haga la llamada.

```
SELECT isValidCell(h3index '831c02fffffffff');
-- t
SELECT isValidCell(th3index '831c02fffffffff@2001-01-01');
-- t@2001-01-01
SELECT isValidCell(th3index '0@2001-01-01');
-- f@2001-01-01
SELECT isValidCell(quadbin '48a6227affffffff');
-- t
SELECT isValidCell(tquadbin '48a6227affffffff@2001-01-01');
-- t@2001-01-01
SELECT isValidCell(s2cell '47c3c');
-- t
SELECT isValidCell(ts2cell '47c3c@2001-01-01');
-- t@2001-01-01
```

16.11. Jerarquía

- Devolver la celda padre temporal en la resolución dada

```
cellToParent(cell,integer) → cell
cellToParent(tcell,integer) → tcell
cellToParent(tcell) → tcell
```

Devolver el ancestro de una celda en la resolución más gruesa dada. La forma estática toma una única `cell`; la forma temporal devuelve el ancestro de cada celda de una trayectoria. La forma sin argumento desciende a la resolución inmediatamente más gruesa.

```
SELECT getResolution(cellToParent(h3index '8a2a1072b59ffff', 5));
-- 5
SELECT getResolution(cellToParent(th3index '8a2a1072b59ffff@2001-01-01', 5));
-- 5@2001-01-01
SELECT getResolution(cellToParent(quadbin '48a6227affffffff', 5));
-- 5
```

```
SELECT getResolution(cellToParent(tquadbin
  '48a6227affffffff@2001-01-01', 5));
-- 5@2001-01-01
SELECT getResolution(cellToParent(s2cell '47c3c', 5));
-- 5
SELECT getResolution(cellToParent(ts2cell '47c3c@2001-01-01', 5));
-- 5@2001-01-01
```

- Devolver las celdas que cubren una celda, o las celdas de un conjunto de celdas, en la resolución dada

```
cellToCover(h3index,integer) → h3indexset
cellToCover(quadbin,integer) → quadbinset
cellToCover(s2cell,integer) → s2cellset
coverCells(h3indexset,integer) → h3indexset
coverCells(quadbinset,integer) → quadbinset
coverCells(s2cellset,integer) → s2cellset
```

La cobertura contiene las celdas que contienen un punto de la celda, cada punto asignado a su celda como lo asigna `latLngToCell`, que es lo que `geoToH3IndexSet` establece para una geometría. Una trayectoria que pasa por la celda toma por tanto allí una celda de la cobertura, de modo que leer así los valores de una celda temporal establece una cobertura de su trayectoria en una resolución en la que sus valores no están almacenados, sin volver a leer la trayectoria. La resolución es la propia de la celda o una más gruesa, y una más fina produce un error. Una tesela QUADBIN y una celda S2 son cada una exactamente la unión de sus cuatro hijos, de modo que su cobertura en una resolución más gruesa es su ancestro en ella (`cellToParent`). El padre de una celda H3 no es su cobertura: los hijos de un hexágono no lo teselan, de modo que una celda H3 puede alcanzar celdas distintas de su padre.

```
SELECT cellToCover(h3index '8c1f005018733ff', 10);
-- {"8a1f00501877fff"}
SELECT coverCells(getValues(th3index '{8c1f005018733ff@2001-01-01,
  8c1f005018731ff@2001-01-02}'), 10);
-- {"8a1f00501877fff"}
SELECT cellToCover(quadbin '48a6227affffffff', 7);
-- {"4876227fffffffff"}
SELECT coverCells(getValues(tquadbin '{48a6227affffffff@2001-01-01,
  48a6227bffffffff@2001-01-02}'), 7);
-- {"4876227fffffffff"}
SELECT cellToCover(s2cell '47c3c', 5);
-- {"47c4"}
SELECT coverCells(cellToChildren(s2cell '47c3c', 9), 7);
-- {"47c3c"}
```

16.12. Conversiones de latitud y longitud

- Devolver la celda temporal en la resolución dada que contiene cada punto de un punto temporal geodésico o plano

```
th3index(tpoint,integer,borderInc boolean=true) → th3index
tquadbin(tpoint,integer,borderInc boolean=true) → tquadbin
ts2cell(tpoint,integer,borderInc boolean=true) → ts2cell
```

El SRID debe ser 4326 para las sobrecargas de `tgeompoint`. El constructor sobre un punto temporal se presenta para todas las cuadrículas, sobre un punto geodésico y uno plano.

Una trayectoria que no indica nada entre sus instantes, discreta o escalonada, produce las celdas que los contienen y conserva su interpolación; una que se desplaza entre ellos produce todas las celdas que atraviesa por el camino, ya que el segmento se recorre celda a celda en lugar de muestrearse. El segmento se recorre a lo largo del camino que sigue su punto: el círculo máximo de un `tgeogpoint`, de modo que una trayectoria que cruza el antimeridiano o pasa cerca de un polo produce las celdas de su ruta más corta, y la recta en longitud y latitud de un `tgeompoint`. Un círculo máximo se curva hacia el polo en longitud y latitud, de modo que el arco entre dos posiciones de una misma tesela QUADBIN puede salir de la tesela y volver

a ella. Una arista de celda S2 es un arco de círculo máximo, que una recta en longitud y latitud no es, de modo que esa recta entre dos posiciones de una misma celda S2 también puede salir de la celda y volver a ella. Para `th3index` y `ts2cell`, cada instante del resultado marca el momento en que la trayectoria entró en esa celda, y la última celda se mantiene hasta el final de la trayectoria, de modo que el resultado abarca su período. Con un límite superior exclusivo, una celda que la trayectoria alcanza solo en su final, sin mantenerla ningún tiempo, no forma parte del resultado. Dos cruces que caen en el mismo microsegundo, la menor separación que indica una marca de tiempo, dejan la celda en la que se entra en ese instante: una celda que se abandona antes de mantenerla ningún tiempo no forma parte del resultado. Un cruce en un extremo de un segmento se indica en el instante de ese extremo, como la división en el espacio de spaceSplit indica la tesela de una esquina, por lo que la cobertura de una trayectoria cortada en periodos semiabiertos sucesivos se fusiona en la cobertura de la trayectoria completa. Las celdas distintas del resultado, que devuelve `getValues`, son las teselas que cubre una trayectoria, la clave de combinación de una tabla Raquet, donde una tesela que el viaje cruza y la clave omite es una tesela que la combinación nunca lee.

La cobertura QUADBIN indica el teselado multidimensional de la trayectoria que spaceSplit indica sobre teselas de la misma cuadrícula. Una tesela contiene sus bordes oeste y norte, de modo que cada posición pertenece a la única tesela que la contiene, y un camino que pasa por el punto donde se encuentran cuatro teselas ocupa la tesela al este del meridiano y al sur del paralelo que se cruzan allí en el instante en que pasa por él. El instante de un cruce pertenece a la tesela que contiene la posición del cruce: la tesela en la que entra la trayectoria cuando se desplaza hacia el este o el sur, y la tesela que abandona cuando se desplaza hacia el oeste o el norte, donde el resultado es un conjunto de secuencias cuya secuencia siguiente comienza después de ese instante. Un cruce se fecha como lo fecha la restricción de la trayectoria a una tesela, truncando su parámetro al microsegundo, y cuando varios cruces caen en un mismo microsegundo, la última tesela alcanzada lo ocupa. El argumento opcional `borderInc` indica, como para `spaceSplit`, si se incluye el borde superior de la extensión de la trayectoria: sin él, una tesela que no contiene de la trayectoria más que ese borde, en su meridiano este o en su paralelo sur, no forma parte del resultado. Para `th3index` y `ts2cell`, cuyas celdas no tienen un eje alineado con la extensión, la malla asigna una posición en el borde de una celda a una de las celdas que se encuentran allí, y la celda en la que la trayectoria entra en su último instante es la única celda que ocupa solo en el borde superior de su extensión: `borderInc` indica si esa celda forma parte del resultado, y sin él la celda ocupada antes se mantiene hasta el final de la trayectoria con un límite exclusivo.

```
SELECT getResolution(th3index(tgeogpoint 'POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT getResolution(th3index(tgeompoint
  'SRID=4326;POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT getResolution(tquadbin(tgeompoint
  'SRID=4326;POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT getResolution(tquadbin(tgeogpoint 'POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT numInstants(tquadbin(tgeogpoint
  '[POINT(-170 65)@2001-01-01, POINT(-100 65)@2001-01-02]', 2));
-- 5
SELECT tquadbin(tgeompoint
  'SRID=4326;[Point(0 0)@2001-01-01, Point(45 33.3)@2001-01-03]', 2);
-- {[482cfffffffffff@2001-01-01], (4826fffffffffff@2001-01-01,
--   4826fffffffffff@2001-01-03]}
SELECT tquadbin(tgeompoint
  'SRID=4326;[Point(0 0)@2001-01-01, Point(45 33.3)@2001-01-03]', 2, false);
-- (4826fffffffffff@2001-01-01, 4826fffffffffff@2001-01-03]
SELECT getResolution(ts2cell(tgeogpoint 'POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT getResolution(ts2cell(tgeompoint
  'SRID=4326;POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT numInstants(ts2cell(tgeompoint
  'SRID=4326;[POINT(-15 44.5)@2001-01-01, POINT(15 44.5)@2001-01-02]', 0));
-- 4
```

- Dividir un punto temporal en las celdas QUADBIN que atraviesa {}

```
quadbinSplit(tpoint,integer,borderInc boolean=true) → {(cell,tpoint)}
```

La cobertura indica qué celda ocupa una trayectoria y cuándo. La división indica las mismas celdas y, junto a cada una, la trayectoria sobre los períodos que la cobertura indica para esa celda, de modo que un valor indexado por celda se lee en una sola pasada donde la cobertura por sí sola exige una restricción por celda. Las celdas de una división son las celdas de la cobertura, el fragmento de una celda es la trayectoria restringida a los períodos que la cobertura indica para ella, y los fragmentos se fusionan en la trayectoria. Una trayectoria que alcanza una celda en su último instante, o que pasa por la esquina que contiene una celda, la ocupa allí durante un instante, y la división indica ese fragmento sobre ese instante. El argumento opcional `borderInc` es el de tquadbin.

El segmento se recorre a lo largo del camino que sigue su punto, de modo que una trayectoria geodésica se divide en las celdas que atraviesan sus círculos máximos y una planar en las celdas que atraviesa su línea recta en longitud y latitud. El período de un fragmento es el que responde `getTime` para él, de modo que la división no lleva un período propio.

```
SELECT (s).cell, asText((s).tpoint, 6) AS tpoint
FROM (SELECT quadbinSplit(tgeompoint
  'SRID=4326;[Point(-135 -10)@2001-01-01, Point(-45 10)@2001-01-03]', 2) AS s) t;
-- 4823fffffffffff | {(POINT(-90 0)@2001-01-02, POINT(-45 10)@2001-01-03]}
-- 4828fffffffffff | {[POINT(-135 -10)@2001-01-01, POINT(-90 0)@2001-01-02)}
-- 4829fffffffffff | {[POINT(-90 0)@2001-01-02]}
SELECT count(*) FROM quadbinSplit(tgeogpoint
  '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 2
SELECT count(*) FROM quadbinSplit(tgeompoint
  'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 1
```

- Dividir un punto temporal en las celdas H3 que atraviesa {}

```
h3Split(tpoint,integer,borderInc boolean=true) → {(cell,tpoint)}
```

La cobertura indica qué celda ocupa una trayectoria y cuándo. La división indica las mismas celdas y, junto a cada una, la trayectoria sobre los períodos que la cobertura indica para esa celda, de modo que un valor indexado por celda se lee en una sola pasada donde la cobertura por sí sola exige una restricción por celda. Las celdas de una división son las celdas de la cobertura, el fragmento de una celda es la trayectoria restringida a los períodos que la cobertura indica para ella, y los fragmentos se fusionan en la trayectoria. El período de un fragmento es el que responde `getTime` para él, de modo que la división no lleva un período propio. El argumento opcional `borderInc` es el de la cobertura, de modo que sin él la celda en la que la trayectoria entra en su último instante no tiene fragmento.

El segmento se recorre a lo largo del camino que sigue su punto, de modo que una trayectoria geodésica se divide en las celdas que atraviesan sus círculos máximos y una planar en las celdas que atraviesa su línea recta en longitud y latitud. Una celda H3 es un hexágono sobre la esfera, y un círculo máximo es el camino más corto entre dos posiciones sobre ella, de modo que una trayectoria geodésica no atraviesa más de esas celdas que la línea recta en longitud y latitud entre las mismas posiciones.

```
SELECT (s).cell, asText((s).tpoint, 6) AS tpoint
FROM (SELECT h3Split(tgeompoint
  'SRID=4326;[Point(-0.1 51.5)@2001-01-01, Point(0.1 51.5)@2001-01-02]', 4) AS s) t;
-- 84194adffffffff | {[POINT(-0.1 51.5)@2001-01-01, POINT(0.004068 51.5)@2001-01-01 ↩
    12:29:17.362145)}
-- 84194e7fffffffff | {[POINT(0.004068 51.5)@2001-01-01 12:29:17.362145, POINT(0.1 51.5) ↩
    @2001-01-02]}
SELECT count(*) FROM h3Split(tgeogpoint
  '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 14
SELECT count(*) FROM h3Split(tgeompoint
  'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 16
```

- Dividir un punto temporal en las celdas S2 que atraviesa {}

```
s2Split(tpoint,integer,borderInc boolean=true) → {(cell,tpoint)}
```

La cobertura indica qué celda ocupa una trayectoria y cuándo. La división indica las mismas celdas y, junto a cada una, la trayectoria sobre los períodos que la cobertura indica para esa celda, de modo que un valor indexado por celda se lee en

una sola pasada donde la cobertura por sí sola exige una restricción por celda. Las celdas de una división son las celdas de la cobertura, el fragmento de una celda es la trayectoria restringida a los períodos que la cobertura indica para ella, y los fragmentos se fusionan en la trayectoria. El período de un fragmento es el que responde `getTime` para él, de modo que la división no lleva un período propio. El argumento opcional `borderInc` es el de la cobertura, de modo que sin él la celda en la que la trayectoria entra en su último instante no tiene fragmento.

El segmento se recorre a lo largo del camino que sigue su punto, de modo que una trayectoria geodésica se divide en las celdas que atraviesan sus círculos máximos y una planar en las celdas que atraviesa su línea recta en longitud y latitud. Los dos caminos atraviesan celdas distintas, y su número es una propiedad de la trayectoria y no de la cuadrícula: el arco de longitud -170 a -100 sobre la latitud 65 atraviesa tantas celdas S2 como la línea recta entre las mismas posiciones, mientras que las celdas mismas difieren.

```
SELECT count(*) FROM s2Split(tgeogpoint
  '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6);
-- 29
SELECT count(*) FROM s2Split(tgeompoint
  'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6);
-- 29
SELECT getValues(ts2cell(tgeogpoint
    '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6))
  = getValues(ts2cell(tgeompoint
    'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6));
-- f
```

- Devolver el centroide de una celda, o de cada celda de una trayectoria

```
cellToPoint({h3index,quadbin}) → geometry
cellToPoint(s2cell) → geography
cellToPoint({th3index,ts2cell}) → tgeogpoint
cellToPoint(tquadbin) → tgeompoint
tgeompoint(th3index) → tgeompoint
```

El centroide está en el SRID 4326, y su tipo depende de la cuadrícula. Una celda S2 responde un punto geodésico, ya que sus aristas son arcos de círculo máximo, y una tesela QUADBIN uno plano, ya que la tesela es un rectángulo en longitud y latitud. Una celda H3 responde un punto plano y una celda H3 temporal uno geodésico, y `tgeompoint` responde el mismo centroide como un punto plano, de modo que H3 es la única cuadrícula que publica ambas formas para una trayectoria. La operación se llama `cellToPoint` para todas las cuadrículas, siendo la ranura `cell_to_point` del descriptor lo que lo fija.

```
SELECT ST_AsText(cellToPoint(h3index '871fa44a8ffffff'), 6);
-- POINT(4.297401 50.8043)
SELECT asText(cellToPoint(th3index '871fa44a8ffffff@2001-01-01'), 6);
-- POINT(4.297401 50.8043)@2001-01-01
SELECT asText(tgeompoint(th3index '871fa44a8ffffff@2001-01-01'), 6);
-- POINT(4.297401 50.8043)@2001-01-01
SELECT ST_AsText(cellToPoint(quadbin '48a6227affffffff'), 6);
-- POINT(4.394531 50.847573)
SELECT asText(cellToPoint(tquadbin '48a6227affffffff@2001-01-01'), 6);
-- POINT(4.394531 50.847573)@2001-01-01
SELECT ST_AsText(cellToPoint(s2cell '47c3c'), 6);
-- POINT(4.222019 50.936164)
SELECT asText(cellToPoint(ts2cell '47c3c@2001-01-01'), 6);
-- POINT(4.222019 50.936164)@2001-01-01
```

- Devolver la frontera poligonal de una celda, o de cada celda de una trayectoria

```
cellToBoundary({h3index,quadbin}) → geometry
cellToBoundary(s2cell) → geography
cellToBoundary({th3index,ts2cell}) → tgeography
cellToBoundary(tquadbin) → tgeometry
```

La frontera está en el SRID 4326, y su tipo depende de la cuadrícula como el centroide de cellToPoint: una geografía para S2, una geometría para QUADBIN y, para H3, una geometría para una celda y una geografía para una trayectoria. El anillo

se cierra sobre su primer vértice, de modo que un hexágono H3 lleva siete puntos y un cuadrado QUADBIN o un cuadrilátero esférico S2 cinco.

```
SELECT ST_NPoints(cellToBoundary(h3index '871fa44a8ffffff'));
-- 7
SELECT ST_GeometryType(getValue(cellToBoundary(th3index
  '871fa44a8ffffff@2001-01-01'))::geometry);
-- ST_Polygon
SELECT ST_NPoints(getValue(cellToBoundary(th3index
  '871fa44a8ffffff@2001-01-01'))::geometry);
-- 7
SELECT ST_GeometryType(getValue(cellToBoundary(tquadbin
  '48a6227affffffff@2001-01-01'))::geometry);
-- ST_Polygon
SELECT ST_NPoints(getValue(cellToBoundary(tquadbin
  '48a6227affffffff@2001-01-01'))::geometry);
-- 5
SELECT ST_GeometryType(getValue(cellToBoundary(ts2cell '47c3c@2001-01-01'))::geometry);
-- ST_Polygon
SELECT ST_NPoints(getValue(cellToBoundary(ts2cell '47c3c@2001-01-01'))::geometry);
-- 5
```

16.13. Operaciones de cuadro delimitador

Los siguientes operadores comparan el *cuadro delimitador* espaciotemporal de las celdas temporales en lugar de la contención en la jerarquía de celdas. `tcell` es un tipo temporal espacial: su cuadro delimitador es un `stbox` que combina la extensión espacial de las fronteras de las celdas (su envolvente de longitud/latitud en el SRID 4326), geodésica para una celda H3 o S2 y plana para una celda QUADBIN, con el período de tiempo `tstzspan` de la trayectoria. Los operadores de más abajo comparan ese cuadro a lo largo de los ejes espaciales y/o del eje temporal. Véase la sección de indexación para las clases de operadores que hacen que estos operadores puedan usar índices.

---

nota
El operador `@>` comprueba la contención del cuadro delimitador espaciotemporal (la extensión espacial y el tiempo conjuntamente), *no* la contención en la jerarquía de celdas. Para comprobar si una celda es el ancestro de otra en una resolución dada, use cellToParent.

- Devolver el cuadro delimitador espaciotemporal de una celda temporal, o ese cuadro expandido en la dimensión espacial por un valor flotante

```
stbox(tcell) → stbox
expandSpace(tcell,float) → stbox
```

El cuadro es la envolvente de las fronteras de las celdas, geodésica para una celda H3 o S2 y plana para una celda QUADBIN, junto con el período de tiempo de la trayectoria, de modo que es la clave que almacenan ambas clases de operadores de índice de la sección de indexación.

```
SELECT round(stbox(th3index '871fa44a8ffffff@2001-01-01'), 6);
/* SRID=4326;GEODSTBOX XT(((4.280027,50.79295),(4.314775,50.815648)),
   [2001-01-01, 2001-01-01]) */
SELECT round(expandSpace(th3index '871fa44a8ffffff@2001-01-01', 1), 6);
/* SRID=4326;GEODSTBOX XT(((3.280027,49.79295),(5.314775,51.815648)),
   [2001-01-01, 2001-01-01]) */
SELECT round(stbox(tquadbin '48a6227affffffff@2001-01-01'), 6);
/* SRID=4326;STBOX XT(((4.21875,50.736455),(4.570313,50.958427)),
   [2001-01-01, 2001-01-01]) */
SELECT round(expandSpace(tquadbin '48a6227affffffff@2001-01-01', 1), 6);
/* SRID=4326;STBOX XT(((3.21875,49.736455),(5.570313,51.958427)),
```

```
   [2001-01-01, 2001-01-01]) */
SELECT round(stbox(ts2cell '47c3c@2001-01-01'), 6);
/* SRID=4326;GEODSTBOX XT(((3.780069,50.590023),(4.676786,51.283168)),
   [2001-01-01, 2001-01-01]) */
SELECT round(expandSpace(ts2cell '47c3c@2001-01-01', 1), 6);
/* SRID=4326;GEODSTBOX XT(((2.780069,49.590023),(5.676786,52.283168)),
   [2001-01-01, 2001-01-01]) */
```

- Operadores topológicos

```
{tstzspan,stbox,tcell} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tcell} → boolean
```

Solapamiento, contiene, está contenido en, igual y adyacente, sobre el cuadro delimitador

```
SELECT th3index '871fa44a8ffffff@2001-01-01' && tstzspan '[2001-01-01, 2001-01-02]';
-- t
SELECT th3index '871fa44a8ffffff@2001-01-01' ~= th3index '871fa44a8ffffff@2001-01-01';
-- t
SELECT tquadbin '485623fffffffffff@2001-01-01' && tstzspan '[2001-01-01, 2001-01-02]';
-- t
SELECT tquadbin '485623fffffffffff@2001-01-01' ~= tquadbin '485623fffffffffff@2001-01-01';
-- t
SELECT ts2cell '47c3c@2001-01-01' && tstzspan '[2001-01-01, 2001-01-02]';
-- t
SELECT ts2cell '47c3c@2001-01-01' ~= ts2cell '47c3c@2001-01-01';
-- t
```

- Operadores de posición

```
{stbox,tcell} {<<, &<, >>, &>} {stbox,tcell} → boolean
{stbox,tcell} {<<|, &<|, |>>, |&>} {stbox,tcell} → boolean
{tstzspan,stbox,tcell} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tcell} → boolean
```

Posición relativa a lo largo de los ejes espaciales (estrictamente a la izquierda, solapa por la izquierda, solapa por la derecha, estrictamente a la derecha; estrictamente debajo, solapa por debajo, solapa por encima, estrictamente encima) y a lo largo del eje temporal (estrictamente antes, solapa antes, estrictamente después, solapa después)

```
SELECT th3index '871fa44a8ffffff@2001-01-01' <<
  th3index '871fa44aefffffff@2001-01-01';
-- f
SELECT tquadbin '485623fffffffffff@2001-01-01' << tquadbin '48b6227abffffffff@2001-01-01';
-- f
SELECT ts2cell '47c3c@2001-01-01' << ts2cell '47c4@2001-01-01';
-- f
```

```
SELECT th3index '871fa44a8ffffff@2001-01-01' <<# th3index '871fa44aefffffff@2001-01-05';
-- t
SELECT th3index '871fa44a8ffffff@2001-01-01' #>>
  th3index '871fa44aefffffff@2001-01-05';
-- f
SELECT tquadbin '485623fffffffffff@2001-01-01' <<# tquadbin '48b6227abffffffff@2001-01-05';
-- t
SELECT tquadbin '485623fffffffffff@2001-01-01' #>> tquadbin '48b6227abffffffff@2001-01-05';
-- f
SELECT ts2cell '47c3c@2001-01-01' <<# ts2cell '47c4@2001-01-05';
-- t
SELECT ts2cell '47c3c@2001-01-01' #>> ts2cell '47c4@2001-01-05';
-- f
```

16.14. Funciones métricas

El área de una celda es la única métrica que comparten las tres cuadrículas, y `cellArea` la responde en metros cuadrados para cada una de ellas. La longitud de una arista dirigida y la distancia de círculo máximo entre dos puntos temporales se presentan en la sección específica de H3.

- Devolver el área de una celda, y el área temporal de cada celda de una trayectoria

```
cellArea(cell) → float
cellArea(tcell) → tfloat
```

El área de una celda se encoge hacia los polos en la cuadrícula QUADBIN, ya que los cuadrados de Web-Mercator cubren progresivamente menos terreno en latitudes más altas. Una celda H3 o S2 se define sobre la esfera, de modo que su área varía únicamente con la resolución y con la posición de la celda en el sólido base.

La unidad es el metro cuadrado, fijada por la ranura `cell_area` del descriptor `DggsCellOps` al que se conecta cada cuadrícula, de modo que `cellArea` responde la misma cantidad sea cual sea la cuadrícula que contiene la celda. La función `h3_cell_area` de h3-pg responde kilómetros cuadrados salvo que se le pida otra unidad, de modo que para una celda H3 `cellArea` responde un millón de veces su valor por defecto.

```
SELECT cellArea(th3index '871fa44a8ffffff@2001-01-01');
-- 4441914.27011093 2@2001-01-01
SELECT cellArea(tquadbin '48a6227affffffff@2001-01-01');
-- 609209287.1146417@2001-01-01
SELECT cellArea(ts2cell '47c3c@2001-01-01');
-- 4145164039.0857196@2001-01-01
```

16.15. Funciones que devuelven conjuntos

Las siguientes funciones auxiliares estáticas devuelven un `cellset`, el tipo compañero de colección finita de `cell`. Operan sobre una `cell` estática o sobre un `cellset` y no se elevan a `tcell`, quedando la elevación temporal aplazada a la espera de una primitiva `tset<T>`.

Las celdas QUADBIN que cubre una trayectoria son los valores de tquadbin sobre ella, que son la clave de unión por la que se lee una tabla Raquet.

- Todas las celdas dentro de `k` pasos de cuadrícula desde el origen

```
gridDisk(h3index,integer) → h3indexset
gridDisk(quadbin,integer) → quadbinset
gridDisk(h3indexset,integer) → h3indexset
gridDisk(quadbinset,integer) → quadbinset
s2EdgeNeighbors(s2cell) → s2cellset
```

El disco incluye el origen, que es todo el disco cuando `k` = 0. Su forma sigue a la cuadrícula: en la cuadrícula cuadrada QUADBIN el disco en el paso `k` es el bloque de `(2k+1)×(2k+1)` celdas centrado en el origen, recortado a la cuadrícula, que no tiene celdas más allá de sus bordes como en la referencia de CARTO: una columna no da la vuelta en el antimeridiano y una fila no cruza un polo, de modo que el disco de radio 1 de una celda en un borde contiene 6 celdas y el de una esquina 4, mientras que en la cuadrícula hexagonal H3 es el anillo de hexágonos a esa cantidad de pasos y todo lo que hay dentro de él. S2 presenta las cuatro celdas que comparten una arista con el origen y no toma un número de pasos, de modo que responde el disco en `k` = 1 menos el propio origen. Un conjunto de celdas da la unión de los discos de sus celdas, que en `k` = 1 amplía una cobertura con el anillo de celdas que la rodea, y un disco de radio 1 ampliado un paso es el disco de radio 2 alrededor del mismo origen.

```
SELECT gridDisk(h3index '871fa44a8ffffff', 1);
/* {"871fa44a8ffffff", "871fa44a9ffffff", "871fa44aaffffff", "871fa44abffffff",
   "871fa44acffffff", "871fa44adffffff", "871fa44aeffffff"} */
SELECT gridDisk(quadbin '48a6227affffffff', 1);
```

```
/* {"48a6226dffffffff", "48a6226fffffffff", "48a62278ffffffff", "48a62279ffffffff",
   "48a6227affffffff", "48a6227bffffffff", "48a622c5ffffffff", "48a622d0ffffffff",
   "48a622d1ffffffff"} */
SELECT numValues(gridDisk(gridDisk(h3index '871fa44a8ffffff', 1), 1));
-- 19
SELECT numValues(gridDisk(gridDisk(quadbin '48a6227affffffff', 1), 1));
-- 25
SELECT s2EdgeNeighbors(s2cell '47c3c');
-- {"47c14", "47c24", "47c34", "47c44"}
```

- Todos los hijos de una celda en la resolución de destino dada

```
cellToChildren(h3index,integer) → h3indexset
cellToChildren(quadbin,integer) → quadbinset
cellToChildren(s2cell,integer) → s2cellset
```

El número de hijos es el factor de ramificación de la cuadrícula: cuatro para una celda de árbol cuaternario QUADBIN o S2, siete para una celda H3. La función auxiliar es estática, siendo los hijos un conjunto por instante cuya elevación temporal queda aplazada a la espera de una primitiva `tset<T>`.

```
SELECT cellToChildren(h3index '831c02fffffffff', 4);
/* {"841c021fffffffff", "841c023fffffffff", "841c025fffffffff", "841c027fffffffff",
   "841c029fffffffff", "841c02bfffffffff", "841c02dfffffffff"} */
SELECT cellToChildren(quadbin '48a6227affffffff', 11);
-- {"48b6227a3fffffff", "48b6227a7fffffff", "48b6227abfffffff", "48b6227affffffff"}
SELECT cellToChildren(s2cell '47c3c', 8);
-- {"47c39", "47c3b", "47c3d", "47c3f"}
```

- Representación compactada de un conjunto de celdas, fusionándose las celdas más finas en su padre siempre que esté presente el conjunto completo de hijos del padre

```
compactCells(cellset) → cellset
```

Una tesela QUADBIN y una celda S2 son cada una exactamente la unión de sus cuatro hijos, de modo que las celdas de un conjunto QUADBIN o S2 pueden ser de cualquier resolución: se descarta una celda cubierta por una celda más gruesa del conjunto, y cada cuatro hijos de un mismo padre se reemplazan por ese padre, desde la resolución más fina hacia arriba, como la biblioteca S2 normaliza una unión de celdas. Una celda H3 cubre a sus siete hijos solo de forma aproximada, de modo que un conjunto H3 se compacta como lo compacta la biblioteca H3, a partir de celdas de una única resolución, y celdas de dos resoluciones provocan un error. Un conjunto compactado mezcla resoluciones, mientras que `?=` compara celdas, de modo que una cobertura compactada se descompacta a la resolución de una trayectoria antes de leer la trayectoria contra ella.

```
SELECT compactCells(cellToChildren(h3index '831c02fffffffff', 4));
-- {"831c02fffffffff"}
SELECT compactCells(cellToChildren(quadbin '48a6227affffffff', 12));
-- {"48a6227affffffff"}
SELECT compactCells(cellToChildren(s2cell '47c3c', 9));
-- {"47c3c"}
```

- Representación descompactada en la resolución dada, la inversa de `compactCells`

```
uncompactCells(cellset,integer) → cellset
```

Una celda más gruesa que la resolución se reemplaza por sus descendientes en la resolución. Una celda más fina que la resolución no puede expresarse en ella y provoca un error, como también un resultado QUADBIN o S2 de más de 4194304 celdas.

```
SELECT numValues(uncompactCells(compactCells(
  cellToChildren(h3index '831c02fffffffff', 4)), 4));
-- 7
SELECT numValues(uncompactCells(quadbinset '{48a6227affffffff}', 12));
-- 16
```

```
SELECT numValues(uncompactCells(compactCells(cellToChildren(s2cell '47c3c', 8)), 9));
-- 16
```

16.16. Relaciones siempre y alguna vez

- Relaciones siempre y alguna vez

```
eContains({geometry,tquadbin},{geometry,tquadbin}) → boolean
aContains({geometry,tquadbin},{geometry,tquadbin}) → boolean
eCovers({geometry,tquadbin},{geometry,tquadbin}) → boolean
aCovers({geometry,tquadbin},{geometry,tquadbin}) → boolean
eTouches({geometry,tquadbin},{geometry,tquadbin}) → boolean
aTouches({geometry,tquadbin},{geometry,tquadbin}) → boolean
eDisjoint({geometry,tquadbin},{geometry,tquadbin}) → boolean
eDisjoint({geography,th3index},{geography,th3index}) → boolean
eDisjoint({geography,ts2cell},{geography,ts2cell}) → boolean
aDisjoint({geometry,tquadbin},{geometry,tquadbin}) → boolean
aDisjoint({geography,th3index},{geography,th3index}) → boolean
aDisjoint({geography,ts2cell},{geography,ts2cell}) → boolean
eIntersects({geometry,tquadbin},{geometry,tquadbin}) → boolean
eIntersects({geography,th3index},{geography,th3index}) → boolean
eIntersects({geography,ts2cell},{geography,ts2cell}) → boolean
aIntersects({geometry,tquadbin},{geometry,tquadbin}) → boolean
aIntersects({geography,th3index},{geography,th3index}) → boolean
aIntersects({geography,ts2cell},{geography,ts2cell}) → boolean
eDwithin({geometry,tquadbin},{geometry,tquadbin},float) → boolean
eDwithin({geography,th3index},{geography,th3index},float) → boolean
eDwithin({geography,ts2cell},{geography,ts2cell},float) → boolean
aDwithin({geometry,tquadbin},{geometry,tquadbin},float) → boolean
aDwithin(th3index,th3index,float) → boolean
aDwithin(ts2cell,ts2cell,float) → boolean
```

Una relación de una celda es la de su frontera, tal como la devuelve cellToBoundary, y cada malla se relaciona en la única lectura que expresa sus celdas exactamente. Una tesela QUADBIN está delimitada por meridianos y paralelos, que son rectas en longitud y latitud, de modo que un `tquadbin` se relaciona a través de la geometría temporal de su frontera, con un `geometry` u otro `tquadbin`, con el conjunto completo de relaciones de los tipos de geometrías temporales. Una celda H3 o S2 está delimitada por arcos de círculo máximo, de modo que un `th3index` o un `ts2cell` se relaciona a través de la geografía temporal de su frontera, con un `geography` u otra celda de su malla, con la respuesta calculada sobre la esfera y la distancia en metros; tiene las relaciones que tiene una geografía temporal, que no expresan contención ni contacto. Un `geometry` pasado a una relación de H3 o S2 se lee como el `geography` al que PostGIS lo convierte. Cada relación tiene tres formas cuantificadas: la forma *alguna vez* con el prefijo `e` devuelve `true` cuando la relación se cumple en algún instante, la forma *siempre* con el prefijo `a` cuando se cumple en todo instante, y la forma *temporal* con el prefijo `t` de la sección siguiente devuelve un `tbool` del resultado instante a instante. `eContains` indica si una frontera contiene a la otra, solo interiores, y `eCovers` lo mismo con la frontera incluida; `eIntersects` si las fronteras comparten al menos un punto, `eDisjoint` si no comparten ninguno, `eTouches` si comparten un punto de la frontera pero ningún punto interior, y `eDwithin` si se encuentran dentro de la distancia pasada como tercer argumento. El último ejemplo es un punto que H3 asigna a la celda de resolución 0 y que se encuentra a 91 km fuera del polígono que une los vértices de la celda con rectas en longitud y latitud.

```
SELECT eIntersects(geography 'Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', th3index '871fa4418ffffff@2001-01-01');
-- t
SELECT eIntersects(geometry 'SRID=4326;Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', tquadbin '48a6227affffffff@2001-01-01');
-- t
SELECT eIntersects(geography 'Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', ts2cell '47c3c@2001-01-01');
-- t
SELECT eIntersects(th3index '8009fffffffffff@2001-01-01',
```

```
  geography 'Point(16.6328 71.9281)');
-- t
```

16.17. Relaciones espaciotemporales

- Relaciones espaciotemporales

```
tContains({geometry,tquadbin},{geometry,tquadbin}) → tbool
tCovers({geometry,tquadbin},{geometry,tquadbin}) → tbool
tDisjoint({geometry,tquadbin},{geometry,tquadbin}) → tbool
tDisjoint(th3index,th3index) → tbool
tDisjoint(ts2cell,ts2cell) → tbool
tDwithin({geometry,tquadbin},{geometry,tquadbin},float) → tbool
tIntersects({geometry,tquadbin},{geometry,tquadbin}) → tbool
tIntersects(th3index,th3index) → tbool
tIntersects(ts2cell,ts2cell) → tbool
tTouches({geometry,tquadbin},{geometry,tquadbin}) → tbool
```

La forma temporal responde un `tbool` que lleva el resultado en cada instante, mientras que las formas siempre y alguna vez de la sección anterior responden un único `boolean` sobre todo el lapso de tiempo. Una celda H3 o S2 se relaciona como la geografía temporal de su frontera, que tiene las relaciones temporales de disyunción e intersección entre dos valores temporales, de modo que dos celdas de la malla se relacionan sobre la esfera.

```
SELECT tIntersects(th3index '871fa4418ffffff@2001-01-01',
  th3index '871fa4418ffffff@2001-01-01');
-- t@2001-01-01
SELECT tIntersects(geometry 'SRID=4326;Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', tquadbin '48a6227affffffff@2001-01-01');
-- t@2001-01-01
SELECT tIntersects(ts2cell '47c3c@2001-01-01', ts2cell '47c3c@2001-01-01');
-- t@2001-01-01
```

16.18. Comparaciones

Las teselas también admiten el conjunto completo de operadores de comparación. El orden es total y se define primero por la celda QUADBIN, luego por el tipo de píxel, el ancho, el alto y finalmente los bytes de píxeles; está pensado para indexación y deduplicación, no para una interpretación espacial. Una clase de operadores btree por defecto hace que `ORDER BY`, `DISTINCT` y las uniones por mezcla funcionen sobre columnas `raquet`, y una clase de operadores hash por defecto habilita las uniones y agregaciones hash.

- Comparaciones tradicionales

```
cell {=, <>, <, >, <=, >=} cell → boolean
tcell {=, <>, <, >, <=, >=} tcell → boolean
```

Las funciones que respaldan estos operadores son `eq`, `ne`, `lt`, `le`, `ge` y `gt`, junto con `cmp({cell,tcell},{cell,tcell}) → integer`, que devuelve -1, 0 o 1. Los operadores sobre `h3index` son los de la extensión `h3`, y las funciones responden como ellos.

```
SELECT cmp(h3index '831c02fffffffff', h3index '8a2a1072b59ffff');
-- -1
SELECT th3index '831c02ffffffffff@2001-01-01' = th3index '831c02ffffffffff@2001-01-01';
-- t
SELECT tquadbin '48a6227affffffff@2001-01-01' = tquadbin '48a6227affffffff@2001-01-01';
-- t
SELECT ts2cell '47c3c@2001-01-01' = ts2cell '47c3c@2001-01-01';
-- t
```

- Comparaciones siempre y alguna vez

```
{cell,tcell} {?=, ?<>, %=, %<>} {cell,tcell} → boolean
```

El operando de sondeo puede ser un identificador `cell` escueto u otra trayectoria `tcell`, en cualquiera de las dos posiciones de operando. Las formas con el prefijo `e` se cumplen cuando la relación se cumple en al menos un instante, las formas con el prefijo `a` cuando se cumple a lo largo de todo el lapso de tiempo.

```
SELECT th3index '[831c02fffffffff@2001-01-01,
  880326b885fffff@2001-01-05]' ?= h3index '880326b885fffff';
-- t
SELECT tquadbin '[48a6227affffffff@2001-01-01,
  485623fffffffff@2001-01-05]' ?= quadbin '485623fffffffff';
-- t
SELECT ts2cell '[47c3c@2001-01-01,
  47c4@2001-01-05]' ?= s2cell '47c4';
-- t
```

- Comparaciones temporales

```
{cell,tcell} {#=, #<>} {cell,tcell} → tbool
```

Devolver un booleano temporal que da la igualdad, o la desigualdad, por instante entre una celda temporal y un sondeo

```
SELECT th3index '831c02fffffffff@2001-01-01' #= h3index '831c02fffffffff';
-- t@2001-01-01
SELECT tquadbin '48a6227affffffff@2001-01-01' #= quadbin '48a6227affffffff';
-- t@2001-01-01
SELECT ts2cell '47c3c@2001-01-01' #= s2cell '47c3c';
-- t@2001-01-01
```

- Compara dos teselas Raquet

```
raquet {=, <>, <, <=, >=, >} raquet → boolean
```

Las funciones que respaldan estos operadores son `eq`, `ne`, `lt`, `le`, `ge` y `gt`, junto con `cmp(raquet,raquet) → integer`, que devuelve -1, 0 o 1.

```
-- Two tiles with the same cell, layout and pixels are equal
SELECT raquet('\x0102'::bytea, 2, 1, quadbin '4817fffffffffff',
  'uint8') = raquet('\x0102'::bytea, 2, 1, quadbin '4817fffffffffff', 'uint8');
-- true
-- Deduplicate tiles, which the btree and hash operator classes support
SELECT DISTINCT tile FROM elevation_tiles;
```

- Valores hash

```
hash({cell,cellset,tcell,raquet}) → integer
hashExtended({cell,cellset,tcell,raquet},seed bigint) → bigint
```

La función `hash` devuelve un valor hash de 32 bits y la función `hashExtended` un valor hash de 64 bits calculado con la semilla dada, y los valores iguales tienen valores hash iguales. Sobre `h3index` responden como las funciones `h3index_hash` y `h3index_hash_extended` de la extensión `h3`, que proporciona el tipo.

```
SELECT hash(th3index '831c02fffffffff@2001-01-01') =
  hash(th3index '{831c02fffffffff@2001-01-01}');
-- t
SELECT hash(quadbinset '{48a6227affffffff, 485623fffffffff}') =
  hash(quadbinset '{485623fffffffff, 48a6227affffffff}');
-- t
SELECT hashExtended(s2cell '47c3c', 0) <> hashExtended(s2cell '47c3c', 1);
-- t
```

16.19. Agregaciones

A continuación se ilustran las funciones de agregación para celdas temporales.

- Conteo temporal

```
tCount(tcell) → {tintSeq,tintSeqSet}
```

```
SELECT tCount(temp) FROM (VALUES (th3index '831c02fffffffff@2001-01-01')) AS t(temp);
-- {1@2001-01-01}
SELECT tCount(temp) FROM (VALUES (tquadbin '48a6227affffffff@2001-01-01')) AS t(temp);
-- {1@2001-01-01}
SELECT tCount(temp) FROM (VALUES (ts2cell '47c3c@2001-01-01')) AS t(temp);
-- {1@2001-01-01}
```

- Conteo de ventana

```
wCount(tcell) → {tintSeq,tintSeqSet}
```

```
SELECT wCount(temp, interval '1 day') FROM (VALUES
  (th3index '831c02fffffffff@2001-01-01')) AS t(temp);
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT wCount(temp, interval '1 day') FROM (VALUES
  (tquadbin '48a6227affffffff@2001-01-01')) AS t(temp);
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT wCount(temp, interval '1 day') FROM (VALUES
  (ts2cell '47c3c@2001-01-01')) AS t(temp);
-- {[1@2001-01-01, 1@2001-01-02]}
```

- Agregar un conjunto de celdas temporales fusionándolas, o ensamblándolas en una secuencia o en un conjunto de secuencias

```
merge(tcell) → tcell
appendInstant(tcell[,interp text[,maxt interval]]) → tcell
appendSequence(tcell) → tcell
```

DuckDB, Apache Spark y Apache Flink las llaman `mergeAgg`, `appendInstantAgg` y `appendSequenceAgg` (Sección 18.10.1).

```
SELECT merge(temp) FROM (VALUES (th3index '831c02fffffffff@2001-01-01')) AS t(temp);
-- {831c02fffffffff@2001-01-01}
SELECT merge(temp) FROM (VALUES (tquadbin '48a6227affffffff@2001-01-01')) AS t(temp);
-- {48a6227affffffff@2001-01-01}
SELECT merge(temp) FROM (VALUES (ts2cell '47c3c@2001-01-01')) AS t(temp);
-- {47c3c@2001-01-01}
```

16.20. Indexación

Se pueden crear índices GiST y SP-GiST para columnas de tabla de celdas temporales. A continuación, se muestra un ejemplo de creación de índice:

```
CREATE INDEX Trips_Cells_SPGist_Idx ON Trips USING SPGist(Cells);
```

Los índices GiST y SP-GiST almacenan el cuadro delimitador para las celdas temporales, que es un `stbox`, como todos los tipos espaciotemporales: la extensión de las fronteras de las celdas, geodésica para las celdas H3 y S2 y plana para las celdas QUADBIN, junto con el período de tiempo.

Cada clase de operadores lleva el nombre del tipo temporal que indexa, de modo que una columna `th3index` toma `th3index_rtree_ops` para GiST y `th3index_quadtree_ops` o `th3index_kdtree_ops` para SP-GiST, y los

nombres de `tquadbin` y `ts2cell` igualmente. La clase de árbol cuaternario es la predeterminada de SP-GiST y la clase de árbol k-d se nombra explícitamente. También se definen una clase de operadores B-tree y una hash, que ordenan y aplican hash por el valor temporal subyacente.

Un índice GiST o SP-GiST puede acelerar las consultas que involucran a los siguientes operadores:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, que solo consideran la dimensión espacial en celdas temporales,
- `<<#`, `&<#`, `#&>`, `#>>`, que solo consideran la dimensión temporal en celdas temporales,
- `&&`, `@>`, `<@`, `~=`, `-|-`, que consideran tantas dimensiones como compartan la columna indexada y el argumento de consulta.

Estos operadores trabajan con cuadros delimitadores, no con los valores completos.

16.21. Operaciones específicas de H3

Una celda H3 es un hexágono sobre un icosaedro, de modo que tiene vecinos a través de seis aristas, vértices compartidos con ellos y una celda base entre las 122 que lleva el icosaedro. Doce celdas de cada resolución son pentágonos. Nada de eso tiene contrapartida en una cuadrícula de cuadrados o de cuadriláteros esféricos.

MobilityDB se apoya en la extensión h3 (h3-pg) y lee su tipo `h3index`, de modo que toda función de h3-pg sigue disponible junto a las siguientes. Una función de MobilityDB sobre una celda H3 toma el nombre de su homóloga de h3-pg sin el prefijo `h3_`, una operación que comparten las tres cuadrículas toma el nombre que comparten, y una sola signatura responde a cada operación.

h3-pg	MobilityDB
`h3_latlng_to_cell`	`latLngToCell`
`h3_cell_to_geometry`, `h3_cell_to_latlng`	`cellToPoint`
`h3_cell_to_boundary`	`cellToBoundary`
`h3_cell_to_parent`, `h3_cell_to_children`	`cellToParent`, `cellToChildren`
`h3_get_resolution`, `h3_is_valid_cell`	`getResolution`, `isValidCell`
`h3_grid_disk`	`gridDisk`
`h3_cell_area`	`cellArea`

- Devolver el número de celda base temporal (0–121) de cada celda de una trayectoria

```
th3GetBaseCellNumber(th3index) → tint
```

```
SELECT th3GetBaseCellNumber(th3index '831c02fffffffff@2001-01-01');
-- 14@2001-01-01
```

- Devolver un booleano temporal que indica si cada celda tiene orientación de clase III

```
th3IsResClassIii(th3index) → tbool
```

La clase III alterna con la resolución: las resoluciones impares son de clase III, las pares de clase II.

```
SELECT th3IsResClassIii(th3index '871fa44a8ffffff@2001-01-01');
-- t@2001-01-01
```

- Devolver un booleano temporal que indica si cada celda es un pentágono

```
th3IsPentagon(th3index) → tbool
```

12 celdas por resolución son pentágonos, las 110 restantes de cada celda base descienden como hexágonos.

```
SELECT th3IsPentagon(th3index '831c00fffffffff@2001-01-01');
-- t@2001-01-01
SELECT th3IsPentagon(th3index '831c02fffffffff@2001-01-01');
-- f@2001-01-01
```

- Devolver la celda hija central temporal en la resolución dada

```
th3CellToCenterChild(th3index,integer) → th3index
th3CellToCenterChild(th3index) → th3index
```

La forma sin argumento desciende a la resolución inmediatamente más fina.

```
SELECT th3CellToCenterChild(th3index '871fa44a8ffffff@2001-01-01', 8);
-- 881fa44a81fffff@2001-01-01
SELECT th3CellToCenterChild(th3index '871fa44a8ffffff@2001-01-01');
-- 881fa44a81fffff@2001-01-01
```

- Devolver la posición ordinal (basada en 0) de cada celda entre los hijos de su padre en la resolución dada

```
th3CellToChildPos(th3index,integer) → tbigint
```

```
SELECT th3CellToChildPos(th3index '871fa44a8ffffff@2001-01-01', 6);
-- 0@2001-01-01
```

- Devolver la celda hija temporal en una posición ordinal dada del padre dado, en la resolución de hijo dada

```
th3ChildPosToCell(tbigint,th3index,integer) → th3index
```

```
SELECT th3ChildPosToCell(tbigint '0@2001-01-01', th3index '871fa44a8ffffff@2001-01-01',
  8);
-- 881fa44a81fffff@2001-01-01
```

- Las celdas a *exactamente* `k` pasos de cuadrícula desde el origen

```
h3GridRing(h3index,integer) → h3indexset
```

Falla cerca de los pentágonos.

```
SELECT h3GridRing(h3index '871fa44a8ffffff', 1);
/* {"871fa44a9ffffff", "871fa44aaffffff", "871fa44abffffff", "871fa44acffffff",
   "871fa44adffffff", "871fa44aeffffff"} */
```

- El camino inclusivo de celdas entre dos celdas de la misma resolución

```
h3GridPathCells(h3index,h3index) → h3indexset
```

```
SELECT h3GridPathCells(h3index '871fa44a8ffffff', h3index '871fa44b1ffffff');
/* {"871fa4484ffffff", "871fa44a3ffffff", "871fa44a8ffffff", "871fa44aeffffff",
   "871fa44b1ffffff"} */
```

- Todas las aristas dirigidas salientes de una celda

```
h3OriginToDirectedEdges(h3index) → h3indexset
```

6 para un hexágono y 5 para un pentágono.

```
SELECT h3OriginToDirectedEdges(h3index '871fa44a8ffffff');
/* {"1171fa44a8ffffff", "1271fa44a8ffffff", "1371fa44a8ffffff", "1471fa44a8ffffff",
   "1571fa44a8ffffff", "1671fa44a8ffffff"} */
```

- Todos los vértices de una celda

```
h3CellToVertexes(h3index) → h3indexset
```

6 para un hexágono y 5 para un pentágono.

```
SELECT h3CellToVertexes(h3index '871fa44a8ffffff');
/* {"2071fa44a8ffffff", "2171fa44a8ffffff", "2271fa44a8ffffff", "2371fa44a8ffffff",
   "2471fa44a8ffffff", "2571fa44a8ffffff"} */
```

- Los índices de las caras del icosaedro (0..19) que interseca una celda

```
h3GetIcosahedronFaces(h3index) → intset
```

```
SELECT h3GetIcosahedronFaces(h3index '831c02fffffffff');
-- {6, 11}
```

- Devolver la longitud temporal de cada arista dirigida de una trayectoria, en la unidad solicitada

```
th3EdgeLength(th3index) → tfloat
```

```
SELECT th3EdgeLength(th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aefffffff@2001-01-01'));
-- 1.272084901305088@2001-01-01
```

- Devolver la distancia de círculo máximo temporal entre dos puntos temporales geodésicos (no entre celdas)

```
greatCircleDistance(tgeogpoint,tgeogpoint) → tfloat
```

```
SELECT greatCircleDistance(tgeogpoint 'Point(0 0)@2001-01-01',
  tgeogpoint 'Point(1 0)@2001-01-01');
-- 111.19505197522943@2001-01-01
```

- Devolver un booleano temporal que indica si dos celdas temporales son vecinas de cuadrícula en cada instante

```
th3AreNeighborCells(th3index,th3index) → tbool
```

```
SELECT th3AreNeighborCells(th3index '880326b885fffff@2001-01-01',
  th3index '880326b88dfffff@2001-01-01');
-- t@2001-01-01
```

- Devolver un identificador de arista dirigida temporal a partir de un par de celdas temporales de origen y destino

```
th3CellsToDirectedEdge(th3index,th3index) → th3index
```

```
SELECT th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aefffffff@2001-01-01');
-- 1671fa44a8ffffff@2001-01-01
```

- Devolver un booleano temporal que indica en cada instante si el valor es una arista dirigida H3 válida

```
isValidDirectedEdge(tcell) → tbool
```

Las aristas dirigidas H3 son ellas mismas identificadores de 64 bits (codificados de forma distinta a los identificadores de celda, `isValidDirectedEdge` los distingue). El tipo `th3index` de MobilityDB lleva aristas dirigidas por convención: el mismo tipo, interpretado a través de accesores específicos de aristas.

```
SELECT isValidDirectedEdge(th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aefffffff@2001-01-01'));
-- t@2001-01-01
SELECT isValidDirectedEdge(th3index '871fa44a8ffffff@2001-01-01');
-- f@2001-01-01
```

- Devolver la celda de origen o de destino temporal de una arista dirigida temporal

```
th3GetDirectedEdgeOrigin(th3index) → th3index
th3GetDirectedEdgeDestination(th3index) → th3index
```

```
SELECT th3GetDirectedEdgeOrigin(th3CellsToDirectedEdge(th3index
  '871fa44a8ffffff@2001-01-01', th3index '871fa44aefffffff@2001-01-01'));
-- 871fa44a8ffffff@2001-01-01
SELECT th3GetDirectedEdgeDestination(th3CellsToDirectedEdge(th3index
  '871fa44a8ffffff@2001-01-01', th3index '871fa44aefffffff@2001-01-01'));
-- 871fa44aefffffff@2001-01-01
```

- Devolver la frontera poligonal temporal de cada arista dirigida de una trayectoria, como una geografía temporal

```
th3DirectedEdgeToBoundary(th3index) → tgeography
```

```
SELECT ST_NPoints(getValue(th3DirectedEdgeToBoundary(
  th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aefffffff@2001-01-01')))::geometry);
-- 3
```

- Devolver el vértice H3 temporal en el número de vértice dado

```
th3CellToVertex(th3index,integer) → th3index
```

0–5 para hexágonos, 0–4 para pentágonos.

```
SELECT th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01', 0);
-- 2071fa44a8ffffff@2001-01-01
```

- Devolver las coordenadas geodésicas temporales de cada vértice de una trayectoria

```
th3VertexToPoint(th3index) → tgeogpoint
```

```
SELECT asText(th3VertexToPoint(th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01',
  0)), 6);
-- POINT(4.280027 50.807655)@2001-01-01
```

- Devolver un booleano temporal que indica en cada instante si el valor es un vértice H3 válido

```
isValidVertex(tcell) → tbool
```

```
SELECT isValidVertex(th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01', 0));
-- t@2001-01-01
```

- Devolver la distancia temporal en saltos de cuadrícula (número de pasos de una sola celda) entre dos celdas temporales

```
th3GridDistance(th3index,th3index) → tbigint
tcell <-> tcell → tbigint
```

De forma equivalente, el operador <-> devuelve la misma distancia.

```
SELECT th3index '880326b885fffff@2001-01-01' <-> th3index '880326b88dfffff@2001-01-01';
-- 1@2001-01-01
```

- Convertir entre una celda temporal y un par de coordenadas locales (I, J) temporales relativas a una celda temporal de origen

```
th3CellToLocalIj(th3index,th3index) → tgeompoint
th3LocalIjToCell(th3index,tgeompoint) → th3index
```

La coordenada local se expone como un `tgeompoint` con el eje I en la ranura `x` y J en `y`.

```
SELECT asText(th3CellToLocalIj(th3index '871fa44a8ffffff@2001-01-01',
  th3index '871fa44aeffffff@2001-01-01'));
-- POINT(497 320)@2001-01-01
SELECT th3LocalIjToCell(th3index '871fa44a8ffffff@2001-01-01',
  th3CellToLocalIj(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aeffffff@2001-01-01'));
-- 871fa44aeffffff@2001-01-01
```

16.22. Operaciones específicas de QUADBIN

QUADBIN se construye sobre el esquema de teselas de mapa deslizante (Web-Mercator), de modo que una celda se corresponde directamente con una terna de coordenadas de tesela `(x,y,z)` y con la cadena quadkey en base 4 que usan los servidores de teselas. Estas conversiones son el puente específico de QUADBIN hacia el ecosistema de teselado, y no tienen contrapartida en una cuadrícula hexagonal ni de cuadriláteros esféricos.

- Devolver si un valor es un identificador QUADBIN estructuralmente válido

```
isValidIndex(quadbin) → boolean
```

Etiqueta de cabecera y campo de resolución correctos. La entrada del tipo `quadbin` provoca un error ante cualquier otro valor, por lo que todo valor del tipo es un índice válido, mientras que isValidCell responde si el índice es una celda de datos: el índice 504b5fffffffffff lleva el modo 2.

```
SELECT isValidIndex(quadbin '480fffffffffffff');
-- t
SELECT isValidIndex(quadbin '504b5fffffffffff');
-- t
```

- Devolver la celda vecina en la misma resolución en la dirección dada

```
quadbinCellSibling(quadbin,text) → quadbin
```

`up` / `down` / `left` / `right`. La respuesta es NULL para una celda vecina más allá de un borde de la cuadrícula, que no tiene celdas allí: una columna no da la vuelta en el antimeridiano y una fila no cruza un polo.

```
SELECT quadbinCellSibling(
  quadbinCellSibling(quadbin '48427fffffffffff', 'right'), 'left')
  = quadbin '48427fffffffffff';
-- t
```

- Emitir la cadena quadkey en base 4 de tesela deslizante de una celda

```
quadbinCellToQuadkey(quadbin) → text
tquadbinCellToQuadkey(tquadbin) → ttext
```

Un dígito por nivel de zoom, primero el más significativo, correspondiéndose la celda mundial de resolución 0 con la cadena vacía. La forma estática toma un único `quadbin`, devolviendo la forma temporal el quadkey de cada celda de una trayectoria como un `ttext`.

```
SELECT quadbinCellToQuadkey(quadbin '480fffffffffffff');
-- (empty string)
SELECT quadbinCellToQuadkey(quadbin '48427fffffffffff');
-- 0213
SELECT quadbinCellToQuadkey(quadbin '48a6227affffffff');
-- 1202021322
```

- Devolver la distancia temporal en la cuadrícula entre dos celdas QUADBIN temporales

```
tquadbinGridDistance(tquadbin,tquadbin) → tbigint
tquadbin <-> tquadbin → tbigint
```

La distancia en la cuadrícula entre dos celdas del mismo zoom es el menor radio cuyo `gridDisk` alrededor de una contiene a la otra, la mayor de las diferencias entre sus columnas y sus filas de tesela. La cuadrícula no tiene celdas más allá de sus bordes, de modo que las celdas de los dos extremos de una fila están a una fila entera de distancia. De forma equivalente, el operador <-> devuelve la misma distancia.

```
SELECT asText(tquadbinGridDistance(
  tquadbin(quadbinTileToCell(1, 1, 3), tstzspan '[2001-01-01, 2001-01-03]'),
  tquadbin(quadbinTileToCell(4, 2, 3), tstzspan '[2001-01-02, 2001-01-04]')));
-- [3@2001-01-02, 3@2001-01-03]
SELECT tquadbin(quadbinTileToCell(0, 3, 3), timestamptz '2001-01-01') <->
  tquadbin(quadbinTileToCell(7, 3, 3), timestamptz '2001-01-01');
-- 7@2001-01-01
```

- Descomponer una celda de vuelta en su terna de coordenadas de tesela, devuelta como un arreglo de enteros que contiene x, después y, después z

```
quadbinCellToTile(quadbin) → integer[]
```

Es la inversa del constructor de teselas

```
SELECT quadbinCellToTile(quadbin '480fffffffffffff');
-- {0,0,0}
SELECT quadbinCellToTile(quadbin '48a6227affffffff');
-- {524,343,10}
```

- Construir una celda a partir de coordenadas de tesela de mapa deslizante: la columna `x`, la fila `y` y el zoom `z`

```
quadbinTileToCell(integer,integer,integer) → quadbin
```

```
SELECT quadbinTileToCell(0, 0, 0) = quadbin '480fffffffffffff';
-- t
SELECT quadbinTileToCell(3, 5, 4) = quadbin '48427fffffffffff';
-- t
SELECT quadbinTileToCell(524, 343, 10) = quadbin '48a6227affffffff';
-- t
```

16.23. Operaciones específicas de S2

Las operaciones de más abajo no tienen análogo en H3 ni en QUADBIN. Se presentan sobre una `s2cell` estática, ya que un identificador de celda las responde sin referencia al tiempo; las operaciones compartidas que presentan las secciones anteriores se aplican a un `ts2cell` a través de la notación `tcell`.

- Devolver la cara del cubo de 0 a 5 sobre la que se asienta una celda S2

```
s2GetFace(s2cell) → integer
```

S2 proyecta la esfera sobre las seis caras de un cubo circunscrito y tiende una curva de Hilbert sobre cada una, de modo que la cara son los tres primeros bits del identificador y dos celdas de caras distintas no comparten ningún ancestro.

```
SELECT s2GetFace(s2cell '47c3c3');
-- 2
SELECT s2GetFace(geoToS2Cell(geography 'SRID=4326;Point(-122.4 37.8)', 10));
-- 4
```

- Convertir entre una celda S2 y su token, la forma que publican las bibliotecas de S2

```
s2CellToToken(s2cell) → text
s2TokenToCell(text) → s2cell
ts2CellToToken(ts2cell) → ttext
```

Un token es el identificador en hexadecimal con sus ceros finales eliminados, de modo que una celda gruesa se lee más corta que una fina y el texto se ordena como lo hace el identificador.

```
SELECT s2CellToToken(s2cell '47c3c3');
-- 47c3c3
SELECT s2TokenToCell('47c3c3');
-- 47c3c3
SELECT asText(ts2CellToToken(ts2cell '47c3c3@2001-01-01'));
-- "47c3c3"@2001-01-01
```

- Devolver la distancia temporal en la cuadrícula entre dos celdas S2 temporales

```
ts2GridDistance(ts2cell,ts2cell) → tbigint
ts2cell <-> ts2cell → tbigint
```

La distancia en la cuadrícula entre dos celdas del mismo nivel es el número de pasos entre vecinas por una arista, las que devuelve `s2EdgeNeighbors`, de una a la otra, a través de las aristas de las caras del cubo como dentro de una cara. Dos celdas en caras opuestas están a un camino alrededor del cubo de distancia. De forma equivalente, el operador <-> devuelve la misma distancia.

```
SELECT asText(ts2GridDistance(
  ts2cell(s2cell '47c3c', tstzspan '[2001-01-01, 2001-01-03]'),
  ts2cell(s2cell '47c34', tstzspan '[2001-01-02, 2001-01-04]')));
-- [1@2001-01-02, 1@2001-01-03]
SELECT ts2cell(geoToS2Cell(geography 'Point(0 0)', 30),
    timestamptz '2001-01-01') <->
  ts2cell(geoToS2Cell(geography 'Point(180 0)', 30),
    timestamptz '2001-01-01');
-- 2147483648@2001-01-01
```

- Devolver el nivel de la celda más profunda que contiene a dos celdas S2

```
s2CommonAncestorLevel(s2cell,s2cell) → integer
```

Dos celdas de caras distintas del cubo no están contenidas por ninguna celda, y la función responde -1 para ellas.

```
SELECT s2CommonAncestorLevel(s2cell '47c3c3', s2cell '47c4');
-- 5
SELECT s2CommonAncestorLevel(s2cell '47c3c3', s2cell '808581');
-- -1
```

- Devolver verdadero si la primera celda S2 contiene a la segunda

```
s2CellContains(s2cell,s2cell) → boolean
```

```
SELECT s2CellContains(s2cell '47c4', s2cell '47c3c3');
-- t
```

- Devolver la primera y la última celda hoja que contiene una celda S2

```
s2RangeMin(s2cell) → s2cell
s2RangeMax(s2cell) → s2cell
```

Los descendientes de una celda S2 ocupan un intervalo contiguo de la curva de Hilbert, de modo que una prueba de ascendencia es el par de comparaciones `other >= s2RangeMin(cell)` y `other <= s2RangeMax(cell)`. Eso es lo que aporta S2 y no aporta una cuadrícula hexagonal ni de Web-Mercator, y es un filtro que alimenta el índice `stbox` más que una clave de índice propia.

```
SELECT s2RangeMin(s2cell '47c3c3');
-- 47c3c20000000001
SELECT s2RangeMax(s2cell '47c3c3');
-- 47c3c3fffffffff
```

- Devolver el descendiente de una celda S2 en la posición dada a lo largo de la curva de Hilbert

```
s2CellToChild(s2cell,level integer,position integer) → s2cell
```

El nivel es exactamente uno más fino que el de la propia celda, de modo que la posición va de 0 a 3; `cellToChildren`, que presenta la sección de jerarquía, responde todos los descendientes en cualquier nivel más fino.

```
SELECT s2CellToChild(s2cell '47c3c3', 11, 0);
-- 47c3c24
```

- Devolver la longitud en metros de una arista de una celda S2

```
s2EdgeLength(s2cell,edge integer) → float
```

La arista va del vértice del índice dado, que va de 0 a 3, al siguiente en sentido antihorario, y la longitud se mide sobre la esfera WGS84.

```
SELECT round(s2EdgeLength(s2cell '47c3c3', 0)::numeric, 3);
-- 9270.048
```

- Devolver las cuatro celdas que comparten una arista con una celda S2

```
s2EdgeNeighbors(s2cell) → s2cellset
```

Una celda S2 es un cuadrilátero, de modo que tiene cuatro vecinas por arista y ningún anillo de un radio dado; el disco de cuadrícula de H3 no tiene análogo en S2.

```
SELECT s2EdgeNeighbors(s2cell '47c3c3');
-- {"47c3c1", "47c3c5", "47c3dd", "47c3e9"}
```

Una cobertura de región, que responde el conjunto de celdas que aproxima una geometría arbitraria, es la única operación S2 para la que este capítulo no presenta ninguna signatura: las bibliotecas de S2 la publican y MobilityDB aún no lleva un núcleo para ella.

16.24. Operaciones específicas de ráster

Las secciones anteriores enuncian lo que un ráster comparte con las demás teselaciones. Las operaciones siguientes son propias del ráster: una celda de ráster lleva un *valor* donde una celda H3, QUADBIN o S2 lleva solo identidad, de modo que una trayectoria leída contra un ráster responde un `tfloat` en lugar de una celda temporal.

Un ráster es el mismo valor en todos los motores, y en DuckDB y en Spark se convierte además desde y hacia el ráster del anfitrión, el de la extensión de rásteres de DuckDB y el de Apache Sedona: Sección 18.11 enuncia lo que conserva cada conversión y cómo la escribe una consulta.

El operador de muestreo de ráster de MobilityDB lee una banda de ráster a lo largo de una trayectoria de punto móvil y devuelve un `tfloat`. Una posición fuera de la extensión del ráster o sobre un píxel `nodata` no lleva valor. Los operadores de muestreo son de valor doble sea cual sea el tipo de píxel de la banda, de modo que un tipo de píxel pertenece a la superficie de muestreo cuando todo valor que puede contener es exactamente representable en un número de doble precisión. Eso es lo que permite muestrear una banda de cualquier tipo en un único `tfloat`, y que una columna con teselas de varios tipos de píxel se lea con una sola consulta. Un tipo de píxel también se acepta con el nombre que le da el ráster de PostGIS, de modo que el tipo de banda que informa una llamada a `ST_BandPixelType` (`8BUI`, `16BSI`, `32BF`, ...) se pasa a los constructores de teselas tal cual. El ráster de PostGIS no lleva flotantes de 16 bits ni enteros de 64 bits, así que `float16`, `int64` y `uint64` toman la grafía que da su gramática, `16BF`, `64BSI` y `64BUI`. Sus tipos `1BB`, `2BUI` y `4BUI` acotan un píxel a menos de un byte mientras que PostGIS almacena cada uno de ellos a un byte por píxel, de modo que tal banda ya son los bytes de una banda de 8 bits sin signo y los constructores la leen como `uint8`; la cota es lo único que añade el nombre, y una tesela no la lleva. El nombre que informa una tesela es el que la especificación RaQuet da al tipo. Los tipos `uint64` e `int64` contienen valores fuera de ese dominio: una tesela de cualquiera de ellos se almacena y se vuelve a leer exactamente, mientras que muestrear un píxel cuyo valor ningún número de doble precisión nombra genera un error en lugar de responder con un valor vecino. Esto permite consultas como *¿qué perfil de elevación siguió cada vehículo?* o *¿qué viajes entraron en una banda de umbral de temperatura?*

Una tabla Raquet guarda la celda QUADBIN de una tesela en su columna `block` como un entero de 64 bits, junto a la fila de metadatos en `block` = 0, que no contiene ningún índice QUADBIN. Una consulta convierte las celdas de una trayectoria a `bigint` para leer esa columna, y una celda leída de ella a `quadbin`, comprobando la conversión que es un índice QUADBIN. El patrón de consulta típico une una trayectoria con una tabla Raquet en dos pasos:

```
-- Step 1: identify which tiles the trajectory overlaps
SELECT getValues(tquadbin(traj, 8));
-- Step 2: sample each matching tile, a 256 x 256 float32 band without nodata
SELECT rasterTileValueQuadbin(traj, band_1, 256, 256, block::quadbin, 'float32', 0,
  false) FROM elevation_raquet
WHERE block IN (SELECT unnest(getValues(tquadbin(traj, 8)))::bigint);
```

Los operadores de restricción y predicado que cierran la sección componen `rasterValue` con las funciones estándar de restricción temporal y predicado de MobilityDB. Aceptan un argumento opcional `band` (por defecto 1) e invocan `rasterValue` exactamente una vez por invocación.

Una función de ráster toma el nombre de su homóloga del ráster de PostGIS sin el prefijo `ST_`, y los nombres y los valores por defecto de sus argumentos. Una sola signatura responde a cada operación, de modo que una forma de PostGIS que solo fija un argumento de otra se escribe mediante esa otra forma. Un nombre que el estándar SQL reserva toma el tipo como prefijo, de modo que `ST_Value` es `rasterValue`, y las funciones de Well-Known Binary toman los nombres que toma todo tipo de MobilityDB.

Ráster de PostGIS	**MobilityDB**
`ST_Value`	`rasterValue`
`ST_Clip`	`clip`
`ST_Transform`	`transform`
`ST_Rescale`	`rescale`
`ST_Reclass`	`reclass`
`ST_SummaryStats`	`summaryStats`
`ST_DumpAsPolygons`	`dumpAsPolygons`
`ST_NumBands`, `ST_Width`, `ST_Height`, `ST_SRID`	`numBands`, `width`, `height`, `SI`
`ST_UpperLeftX`, `ST_UpperLeftY`, `ST_ScaleX`, `ST_ScaleY`, `ST_SkewX`, `ST_SkewY`	`upperLeftX`, `upperLeftY`, `s`
`ST_BandPixelType`, `ST_BandNoDataValue`	`bandPixelType`, `bandNoDat`
`ST_MakeEmptyRaster`, `ST_AddBand`, `ST_SetValues`	`makeEmptyRaster`, `addBand`
`ST_AsBinary`, `ST_AsHexWKB`	`asBinary`, `asHexWKB`
`ST_RastFromWKB`, `ST_RastFromHexWKB`	`rasterFromBinary`, `raster`

`asBinary` y `asHexWKB` toman un orden de bytes donde PostGIS toma `outasin`, y sin él devuelven los bytes de `ST_AsBinary` y `ST_AsHexWKB`.

- Lee una banda de ráster a lo largo de una trayectoria

```
rasterValue(tgeompoint,rast raster,band integer=1,exclude_nodata_value boolean=true,
  resample text='nearest') → tfloat
rasterValue(tgeompoint,path text,band integer=1) → tfloat
```

Lee la banda `band` a lo largo de `traj`, tomando el valor del píxel en el que cae cada posición. Una trayectoria que no afirma nada entre sus instantes se lee en sus instantes y devuelve un `tfloat` de conjunto de instantes. Una trayectoria que se mueve entre ellos pasa por los píxeles intermedios, por lo que cada segmento se recorre píxel a píxel, y devuelve un `tfloat` escalonado que mantiene cada valor desde el instante en que el trayecto alcanza el píxel que lo contiene, incluido un píxel del que el trayecto solo corta una esquina. Con un límite superior exclusivo, un píxel que la trayectoria alcanza solo en su final, sin mantenerlo ningún tiempo, no aporta ningún valor al resultado, por lo que las lecturas de una trayectoria cortada en periodos semiabiertos sucesivos se fusionan en la lectura de la trayectoria completa. Una posición fuera del ráster o sobre un píxel sin datos no lleva valor y termina el tramo, por lo que un trayecto que sale y vuelve a entrar en el ráster devuelve una secuencia por visita, y `NULL` cuando nunca encuentra un píxel con datos. Una banda cuyos píxeles no se pueden leer, como una banda almacenada fuera de la base de datos en un archivo que no se puede abrir, provoca un error en lugar de leerse como una banda sin valores. La forma que recibe `path` lee un archivo ráster del servidor, en cualquier formato compatible con GDAL, mediante GDAL, allí donde PostGIS permite leer una banda ráster almacenada fuera de la base de datos: `postgis.enable_outdb_rasters` debe estar activado y `postgis.gdal_enabled_drivers` debe habilitar el controlador del archivo. Devuelve lo que la forma que recibe un `raster` devuelve sobre el mismo ráster.

`exclude_nodata_value` y `resample` son los de `ST_Value` de PostGIS: con el primero falso una posición sobre un píxel nodata conserva el valor nodata donde de otro modo se descarta, y el segundo lee el píxel bajo la posición con `nearest` o `bilinear`, nombrado sin distinguir mayúsculas de minúsculas. Un valor bilineal varía dentro de un píxel. A lo largo de un segmento de una trayectoria que se desplaza entre sus instantes es, sobre cada cuadrado de medio píxel que el segmento atraviesa, un polinomio de grado dos en el tiempo, de modo que una trayectoria leída con `bilinear` devuelve un `tfloat` lineal que contiene el valor en los instantes de la trayectoria, donde cruza los bordes de los píxeles y las líneas que pasan por sus centros, y en el extremo que alcanza cada uno de esos tramos, lineal entre ellos, como el producto de dos flotantes temporales contiene su extremo. Su mínimo y su máximo son los del valor, y es el valor mismo donde la trayectoria se desplaza a lo largo de un eje de la malla, sobre el cual el valor es lineal; en otro caso, entre los instantes que contiene, es la cuerda del valor. El valor salta a través de un borde de píxel donde un píxel leído allí contiene nodata, que toma el valor del píxel en el que cae la posición, y una nueva secuencia comienza entonces en el instante en que la trayectoria cruza el borde.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT rasterValue(tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2001-01-01,
  POINT(2.5 2.5)@2001-01-02, POINT(0.5 0.5)@2001-01-03]', r)::text FROM rast;
/* Interp=Step;[10@2001-01-01, 20@2001-01-01 06:00:00, 30@2001-01-01 18:00:00,
   50@2001-01-02 06:00:00, 70@2001-01-02 18:00:00, 70@2001-01-03] */
-- A bilinear read reaching its maximum between two pixel centres
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4], [20.0::float4, 10.0::float4]]) AS r )
SELECT round(rasterValue(tgeompoint 'SRID=4326;[POINT(0.5 1.5)@2001-01-01,
  POINT(1.5 1.1)@2001-01-03]', r, 1, true, 'bilinear'), 6)::text FROM rast;
/* [10@2001-01-01, 15@2001-01-02, 16.125@2001-01-02 18:00:00, 16@2001-01-03] */
```

Aplicado a un conjunto de datos de trayectorias con un ráster de elevación del terreno:

```
-- Elevation profile per trip (requires a loaded elevation raster)
SELECT tripid, rasterValue(trip, elev) AS elevation FROM trips, elevation_raster;
-- Average elevation per trip
SELECT tripid, avgValue(rasterValue(trip, elev)) AS mean_elev FROM trips,
  elevation_raster;
-- Trips that crossed terrain above 200 m
SELECT tripid FROM trips, elevation_raster
```

```
WHERE atSpan(rasterValue(trip, elev), floatspan '[200, 9999]') IS NOT NULL;
```

Un ráster cargado como una tabla de teselas, como lo carga `raster2pgsql -t`, se lee a lo largo de una trayectoria fusionando las lecturas de sus teselas con `merge`. Una posición se encuentra en un píxel de una sola tesela, de modo que las lecturas de las teselas ocupan tiempos disjuntos, y su fusión responde la lectura del ráster que las teselas cubren, incluido un viaje que pasa de una tesela a la siguiente.

```
-- A 4x4 raster read along a trip whole, and as its four tiles of 2x2 pixels
WITH r AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4, 40.0::float4],
    [50.0::float4, 60.0::float4, 70.0::float4, 80.0::float4],
    [90.0::float4, 100.0::float4, 110.0::float4, 120.0::float4],
    [130.0::float4, 140.0::float4, 150.0::float4, 160.0::float4]]) AS rast ),
t AS (
  SELECT tgeompoint 'SRID=4326;[POINT(0.5 3.5)@2001-01-01, POINT(3.5 0.5)@2001-01-02]'
    AS trip )
SELECT merge(rasterValue(trip, tile)) = (SELECT rasterValue(trip, rast) FROM t, r)
FROM t, r, ST_Tile(rast, 2, 2) AS tile;
-- true
```

- Devuelve el número de píxeles de una banda de un ráster que contienen cada valor

```
valueCount(rast raster,nband integer=1,exclude_nodata_value boolean=true,
  searchvalues float[]=NULL,roundto float=0) → setof (value float,count integer)
valueCount(rast raster,nband integer,searchvalues float[],roundto float=0)
  → setof (value float,count integer)
valueCount(rast raster,searchvalues float[],roundto float=0)
  → setof (value float,count integer)
valueCount(rast raster,nband integer,exclude_nodata_value boolean,searchvalue float,
  roundto float=0) → integer
valueCount(rast raster,nband integer,searchvalue float,roundto float=0) → integer
valueCount(rast raster,searchvalue float,roundto float=0) → integer
```

El histograma de `ST_ValueCount` de PostGIS, con sus argumentos y valores por defecto: una fila por cada valor distinto de la banda, o una por cada valor buscado, que cuenta cero cuando ningún píxel lo contiene, y el número de píxeles que contienen un valor en las formas que toman un único valor buscado. Un valor se redondea al múltiplo más cercano de `roundto` antes de contarse cuando `roundto` es positivo. Una banda cuyos píxeles no tienen datos no devuelve ninguna fila cuando se excluyen los píxeles sin datos, y una banda que el ráster no tiene produce un error donde PostGIS devuelve `NULL`. Las formas de PostGIS que leen una columna ráster de una tabla consultan la base de datos, por lo que siguen siendo de PostGIS.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.24, 1.26, -9999], [2, 2, 2], [3, 1.24, 2]]::double precision[][]) AS r )
SELECT round(value::numeric, 2), count
FROM rast, valueCount(r, 1, true, NULL::float[], 0.1) ORDER BY 1;
-- 1.20 | 2
-- 1.30 | 1
-- 2.00 | 4
-- 3.00 | 1
```

- Devuelve la proporción de los píxeles de una banda de un ráster que contienen cada valor

```
valuePercent(rast raster,nband integer=1,exclude_nodata_value boolean=true,
  searchvalues float[]=NULL,roundto float=0) → setof (value float,percent float)
valuePercent(rast raster,nband integer,searchvalues float[],roundto float=0)
  → setof (value float,percent float)
valuePercent(rast raster,searchvalues float[],roundto float=0)
  → setof (value float,percent float)
```

```
valuePercent(rast raster,nband integer,exclude_nodata_value boolean,
  searchvalue float,roundto float=0) → float
valuePercent(rast raster,nband integer,searchvalue float,roundto float=0) → float
valuePercent(rast raster,searchvalue float,roundto float=0) → float
```

La proporción de `ST_ValuePercent` de PostGIS, entre 0 y 1, de los píxeles que lee valueCount, con los mismos argumentos y formas. Cuando no se lee ningún píxel, la proporción de un valor buscado es cero dividido por cero, `NaN`, como para PostGIS.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.24, 1.26, -9999], [2, 2, 2], [3, 1.24, 2]]::double precision[][]) AS r )
SELECT valuePercent(r, 2::float), valuePercent(r, 3::float) FROM rast;
-- 0.5 | 0.125
```

- Devuelve verdadero si los rásteres están sobre la misma rejilla

```
sameAlignment(rast1 raster,rast2 raster) → boolean
sameAlignment(raster) → boolean
sameAlignment(raster) → boolean
```

La prueba de `ST_SameAlignment` de PostGIS: dos rásteres están alineados cuando tienen la misma escala y la misma inclinación y una esquina de píxel de uno cae sobre una esquina de píxel del otro, de modo que sus píxeles pueden combinarse uno a uno. El agregado responde si cada ráster de una columna está alineado con el primero, omitiendo un ráster `NULL` como `setUnion` omite un conjunto `NULL`, donde PostGIS responde `NULL`; sobre ningún ráster responde `NULL`. Rásteres de SRID distintos producen un error, donde PostGIS responde falso con un aviso.

```
WITH r AS (SELECT
  ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS base,
  ST_MakeEmptyRaster(3, 3, 1.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS east1,
  ST_MakeEmptyRaster(3, 3, 0.5, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS easthalf)
SELECT sameAlignment(base, east1), sameAlignment(base, easthalf),
  (SELECT sameAlignment(x) FROM (VALUES (base), (east1), (easthalf)) t(x))
FROM r;
-- t | f | f
```

- Devuelve por qué dos rásteres no están sobre la misma rejilla

```
notSameAlignmentReason(rast1 raster,rast2 raster) → text
```

La razón de `ST_NotSameAlignmentReason` de PostGIS, en sus palabras: la primera propiedad en la que difieren las rejillas, o que los rásteres están alineados.

```
WITH r AS (SELECT
  ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS base,
  ST_MakeEmptyRaster(3, 3, 0.5, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS easthalf,
  ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 2.0, -1.0, 0.0, 0.0, 4326) AS scalex2)
SELECT notSameAlignmentReason(base, easthalf), notSameAlignmentReason(base, scalex2)
FROM r;
-- The rasters (pixel corner coordinates) are not aligned |
--   The rasters have different scales on the X axis
```

- Devuelve la unión de los rásteres de una columna

```
rasterUnion(raster) → raster
rasterUnion(raster,nband integer) → raster
rasterUnion(raster,uniontype text) → raster
rasterUnion(raster,nband integer,uniontype text) → raster
```

El agregado `ST_Union` de PostGIS sobre rásteres, nombrado como `geoUnion` nombra la unión de geometrías, ya que `union` es una palabra clave de SQL. El resultado cubre la unión de las extensiones de los rásteres, cada píxel la operación de píxeles de los píxeles que tienen un valor allí: `LAST`, por defecto, `FIRST`, `MIN`, `MAX`, `COUNT`, `SUM`, `MEAN` o `RANGE`, sin distinguir mayúsculas de minúsculas. Sin `nband` se unen todas las bandas, y una banda que le falta a un ráster se lee como nodata; con `nband` solo esa banda, que todo ráster debe tener. El tipo de píxel y el valor nodata son los de la banda leída primero, y un conteo es una banda de enteros sin signo de 32 bits sin valor nodata. Un ráster `NULL` se omite, y sobre ningún ráster el agregado devuelve `NULL`. El agregado se ejecuta en paralelo, combinando las uniones de los procesos, mientras que el de PostGIS no tiene función de combinación; `LAST` y `FIRST` dependen del orden de los rásteres, que un `ORDER BY` en el agregado fija. Rásteres con SRID distintos o sobre rejillas distintas producen un error, y también una operación de píxeles distinta de estas, que PostGIS lee como `LAST`.

```
WITH tiles AS (
  SELECT 1 AS id, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857) ↩
      ,
    '32BF'::text, 1.0::float8, -9999.0::float8) AS r
  UNION ALL
  SELECT 2, ST_AddBand(ST_MakeEmptyRaster(2, 2, 1.0, 1.0, 1.0, -1.0, 0.0, 0.0, 3857),
    '32BF'::text, 3.0::float8, -9999.0::float8) )
SELECT ST_DumpValues(rasterUnion(r, 'MEAN' ORDER BY id), 1) FROM tiles;
-- {{1,1,NULL},{1,2,3},{NULL,3,3}}
```

- Devuelve un ráster cuya banda contiene las clases a las que una expresión asigna sus valores

```
reclass(raster,nband integer,reclassexpr text,pixeltype text,nodataval float=NULL)
  → raster
```

La expresión sigue la gramática de `ST_Reclass` de PostGIS: una lista de asignaciones separadas por comas, cada una un rango de origen y un destino separados por dos puntos, siendo un rango un valor o dos separados por un guion. Un rango escrito sin corchetes incluye su extremo inferior y excluye el superior, mientras que `(` excluye el extremo inferior y `]` incluye el superior, de modo que `0-50` deja 50 sin asignar mientras que `[50-100]` lo toma. Un píxel que ningún rango toma contiene el valor nodata de la banda resultante, o cero cuando la banda no declara ninguno. Esa banda tiene el tipo de píxel `pixeltype`, con el nombre que le da PostGIS raster, como `8BUI`, y no declara valor nodata cuando se omite `nodataval`. El ráster conserva su malla y sus demás bandas, de modo que las clases pueden leerse a lo largo de una trayectoria con rasterValue.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT ST_DumpValues(reclass(r, 1, '0-50:1, 51-100:2', '8BUI', 0), 1) AS plain,
  ST_DumpValues(reclass(r, 1, '0-50:1, [50-100]:2', '8BUI', 0), 1) AS bracket
FROM rast;
-- {{1,1,1},{1,NULL,2},{2,2,2}} | {{1,1,1},{1,2,2},{2,2,2}}
-- Read the classes along a trajectory
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT rasterValue(tgeompoint 'SRID=4326;{POINT(0.5 2.5)@2001-01-01,
  POINT(1.5 1.5)@2001-01-02, POINT(2.5 0.5)@2001-01-03}',
  reclass(r, 1, '0-50:1, [50-100]:2', '8BUI', 0))::text FROM rast;
-- {1@2001-01-01, 2@2001-01-02, 2@2001-01-03}
```

- Adición, resta, multiplicación y división de rásteres, píxel a píxel

```
{float,raster} {+, -, *, /} {float,raster} → raster
```

Un ráster tiene los operadores aritméticos de un número temporal, de modo que un ráster derivado se escribe como la expresión que es, como `(nir - red) / (nir + red)` o `depth - 8.5`, donde PostGIS lo escribe como la expresión de `ST_MapAlgebra`. Toda banda del resultado tiene el tipo de píxel `64BF`, cualesquiera que sean los tipos de píxel de los operandos, de modo que un píxel contiene la aritmética tal como se calcula: una diferencia negativa de dos bandas sin signo, un producto más allá de su rango y una razón entre cero y uno se conservan en lugar de recortarse o truncarse, y reclass reduce el resultado donde se necesita un tipo de píxel menor. Dos rásteres se operan banda a banda sobre los píxeles que comparten, como dos números temporales sobre el tiempo que comparten: deben tener el mismo sistema de referencia, la misma malla y el mismo número de bandas, y dos rásteres que no comparten ningún píxel no tienen resultado. Un píxel sin datos en cualquiera de los operandos no tiene datos en el resultado, que toma el valor sin datos del primer operando que declara uno y no declara ninguno cuando ninguno lo hace, de modo que un resultado igual a cero es un valor. Una división por cero produce un error, sea el divisor un número o un píxel. Las funciones detrás de los operadores son `rasterAdd`, `rasterSub`, `rasterMul` y `rasterDiv`.

```
-- The normalized difference of two unsigned 16-bit bands
WITH bands AS (
  SELECT
    ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
      4326), '16BUI'::text, 0.0::float8, NULL::float8), 1, 1, 1,
      ARRAY[[3000, 500, 2000], [4000, 1000, 6000], [100, 0, 65535]]
        ::double precision[][]) AS nir,
    ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
      4326), '16BUI'::text, 0.0::float8, NULL::float8), 1, 1, 1,
      ARRAY[[1000, 1500, 2000], [1000, 3000, 2000], [300, 1, 0]]
        ::double precision[][]) AS red )
SELECT ST_BandPixelType((nir - red) / (nir + red), 1),
  ST_DumpValues((nir - red) / (nir + red), 1)
FROM bands;
-- 64BF | {{0.5,-0.5,0},{0.6,-0.5,0.5},{-0.5,-1,1}}
-- Read a derived raster along a trajectory: the depth under a vessel of draught 8.5
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS depth )
SELECT rasterValue(tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2001-01-01,
  POINT(2.5 2.5)@2001-01-02, POINT(0.5 0.5)@2001-01-03]', depth - 8.5)::text
FROM rast;
-- Interp=Step;[1.5@2001-01-01 00:00:00+00, 11.5@2001-01-01 06:00:00+00,
--   21.5@2001-01-01 18:00:00+00, 41.5@2001-01-02 06:00:00+00,
--   61.5@2001-01-02 18:00:00+00, 61.5@2001-01-03 00:00:00+00]
```

- Devuelve la pendiente de una banda de un ráster, píxel a píxel

```
slope(rast raster,nband integer=1,pixeltype text='32BF',units text='DEGREES',
  scale float=1.0,interpolate_nodata boolean=false) → raster
slope(rast raster,nband integer,customextent raster,pixeltype text='32BF',
  units text='DEGREES',scale float=1.0,interpolate_nodata boolean=false) → raster
```

La pendiente de `ST_Slope` de PostGIS, con sus argumentos y valores por defecto: los gradientes de cada píxel se leen de sus ocho vecinos, tomando un vecino sin datos o fuera del ráster el valor del píxel, y un píxel sin datos sigue sin datos. Las unidades son `DEGREES`, `RADIANS` o `PERCENT`, leídas por sus tres primeras letras sin distinguir mayúsculas, y otras unidades producen un error; PostGIS devuelve los radianes en grados. La escala es la razón entre las unidades verticales y las horizontales, como 111120 para elevaciones en metros sobre una malla en grados. Cuando `interpolate_nodata` es verdadero, un píxel sin datos toma primero la media de sus vecinos ponderada por el inverso de sus distancias, como la calcula `ST_InvDistWeight4ma` de PostGIS. El resultado toma la malla de `customextent` cuando se da.

```
-- A plane rising by 1 for each pixel to the east
WITH plane AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
```

```
    ARRAY[[0, 1, 2], [0, 1, 2], [0, 1, 2]]::double precision[][]) AS r )
SELECT ST_Value(slope(r), 2, 2), ST_Value(slope(r, units => 'PERCENT'), 2, 2)
FROM plane;
-- 45 | 100
```

- Devuelve el índice de rugosidad del terreno de una banda de un ráster, píxel a píxel

```
tri(rast raster,nband integer=1,pixeltype text='32BF',interpolate_nodata boolean=false)
  → raster
tri(rast raster,nband integer,customextent raster,pixeltype text='32BF',
  interpolate_nodata boolean=false) → raster
```

El índice de `ST_TRI` de PostGIS, con sus argumentos y valores por defecto: la media de las diferencias absolutas entre un píxel y sus ocho vecinos, leídos como los lee slope.

```
WITH dem AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[10, 20, 30, 40], [15, 25, -9999, 45], [20, 30, 40, 50],
    [25, 35, 45, 55]]::double precision[][]) AS r )
SELECT ST_DumpValues(tri(r), 1) FROM dem;
-- {{3.75,3.75,5,1.875},{5,7.5,NULL,3.75},{5,8.125,8.125,3.125},{2.5,5.625,5.625,3.75}}
```

- Devuelve el índice de posición topográfica de una banda de un ráster, píxel a píxel

```
tpi(rast raster,nband integer=1,pixeltype text='32BF',interpolate_nodata boolean=false)
  → raster
tpi(rast raster,nband integer,customextent raster,pixeltype text='32BF',
  interpolate_nodata boolean=false) → raster
```

El índice de `ST_TPI` de PostGIS, con sus argumentos y valores por defecto: el valor de un píxel menos la media de sus ocho vecinos, leídos como los lee slope.

```
WITH dem AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[10, 20, 30, 40], [15, 25, -9999, 45], [20, 30, 40, 50],
    [25, 35, 45, 55]]::double precision[][]) AS r )
SELECT ST_DumpValues(tpi(r, interpolate_nodata => true), 1) FROM dem;
-- {{-3.75,-1.875,-1.875,1.25},{-3.75,0,0,3.75},{-3.75,0,0,3.75},{-1.25,1.875,1.875,3.75}}
```

- Devuelve una estadística del vecindario de cada píxel de una banda de un ráster

```
mapAlgebra(rast raster,nband integer,stat text,pixeltype text=NULL,
  extenttype text='FIRST',customextent raster=NULL,distancex integer=0,
  distancey integer=0) → raster
```

Las estadísticas focales de `ST_MapAlgebra` de PostGIS con las funciones de retorno `ST_Min4ma`, `ST_Max4ma`, `ST_Sum4ma`, `ST_Mean4ma`, `ST_Range4ma` y `ST_StdDev4ma`, con sus argumentos y valores por defecto, la función de retorno nombrada por su estadística: `MIN`, `MAX`, `SUM`, `MEAN`, `RANGE` o `STDDEV`, sin distinguir mayúsculas de minúsculas, como `ST_Union` nombra sus operaciones. El vecindario de un píxel se extiende `distancex` píxeles al oeste y al este y `distancey` píxeles al norte y al sur; sus píxeles sin datos, y los que quedan fuera del ráster, se omiten. La suma de ningún píxel es 0, toda otra estadística de ningún píxel es sin datos, y la desviación estándar es la muestral. Como en PostGIS, un vecindario cuya distancia es cero solo a lo largo de un eje es el píxel solo. Un tipo de píxel dejado en NULL es el de la banda, y el resultado conserva el valor sin datos de la banda, o su valor mínimo cuando no indica ninguno. Todo tipo de extensión salvo `CUSTOM` es la extensión del ráster, y `CUSTOM` toma la grilla de `customextent`. Una estadística o un tipo de extensión fuera de estos produce un error, donde PostGIS lee un tipo de extensión desconocido como `INTERSECTION`.

```
WITH dem AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
```

```
    ARRAY[[10, 20, 30, 40], [15, 25, -9999, 45], [20, 30, 40, 50],
    [25, 35, 45, 55]]::double precision[][]) AS r )
SELECT ST_DumpValues(mapAlgebra(r, 1, 'MEAN', '64BF', distancex => 1, distancey => 1), 1)
FROM dem;
-- {{17.5,20,32,38.333333333333336},{20,23.75,35,41},{25,29.375,40.625,47},
--  {27.5,32.5,42.5,47.5}}
```

- Devuelve un ráster que conserva los píxeles de otro que una geometría cubre

```
clip(raster,geom geometry,crop boolean=true,touched boolean=false) → raster
clip(raster,nband integer[],geom geometry,nodataval float[]=NULL,crop boolean=true,
  touched boolean=false) → raster
```

Cada banda se lee contra una máscara en la que se graba la geometría, tal como la lee `ST_Clip` de PostGIS: un píxel que la geometría no cubre devuelve el valor nodata de su banda, y el menor valor que admite su tipo de píxel cuando la banda no declara ninguno. Con `crop` el resultado conserva la extensión que comparten el ráster y la geometría, y en caso contrario la extensión del ráster. La geometría está en el sistema de referencia del ráster; una en otro sistema genera un error. Con `nband` el resultado conserva las bandas nombradas, todas cuando es NULL, `nodataval` declara el valor nodata de los píxeles que la geometría no cubre, uno para todas las bandas o uno por banda, y con `touched` se conserva un píxel que la geometría toca donde de otro modo solo se conserva uno cuyo centro cubre.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT ST_DumpValues(clip(r, geometry 'SRID=4326;POLYGON((0 1,2 1,2 3,0 3,0 1))'), 1)
  AS cropped,
  ST_DumpValues(clip(r, geometry 'SRID=4326;POLYGON((0 1,2 1,2 3,0 3,0 1))', false), 1)
  AS kept_extent
FROM rast;
-- {{10,20},{40,50}} | {{10,20,NULL},{40,50,NULL},{NULL,NULL,NULL}}
```

- Devuelve un ráster expresado en otro sistema de referencia espacial

```
transform(raster,srid integer,algorithm text='NearestNeighbour',maxerr float=0.125,
  scalex float=0,scaley float=0) → raster
transform(raster,alignto raster,algorithm text='NearestNeighbour',
  maxerr float=0.125) → raster
transformPipeline(raster,pipeline text,srid integer=0,is_forward boolean=true,
  algorithm text='NearestNeighbour',maxerr float=0.125,scalex float=0,
  scaley float=0) → raster
```

Cada banda se lleva al sistema de destino y se remuestrea sobre la malla que implica la reproyección, mediante la misma deformación de GDAL que `ST_Transform` de PostGIS, de modo que el resultado expresa la misma cobertura leída a través de otro sistema. El algoritmo es uno de `NearestNeighbour`, `Bilinear`, `Cubic`, `CubicSpline`, `Lanczos`, `Max` y `Min`, nombrado sin distinguir mayúsculas de minúsculas, y `maxerr` es el error en píxeles de entrada que la deformación puede cometer, 0 para un cálculo exacto. `scalex` y `scaley` declaran el tamaño de píxel del resultado, 0 dejando que la deformación lo derive, y `alignto` toma el sistema de referencia, el tamaño de píxel, el origen de la malla y la inclinación de otro ráster, de modo que el resultado queda sobre su malla.

La función `transformPipeline` lleva el ráster a través de una operación de coordenadas, expresada como una cadena PROJ, un WKT o un código de autoridad como `urn:ogc:def:coordinateOperation:EPSG::16031`, en lugar de la que GDAL deriva de los dos sistemas, y la lee como lo hace `transformPipeline` sobre una geometría: primero la longitud y en grados. El resultado expresa el sistema de referencia dado por `srid`, 0 por defecto. Por defecto la operación se aplica hacia adelante; poner `is_forward` a falso aplica su inversa. Los demás argumentos son los de `transform`, de modo que una operación que GDAL derivaría responde como `transform` al mismo sistema.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT SRID(transform(r, 3857)) AS srid, width(transform(r, 3857)) AS width,
  height(transform(r, 3857)) AS height
FROM rast;
-- 3857 | 3 | 3
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT SRID(transformPipeline(r, '+proj=utm +zone=31 +ellps=WGS84', 32631)) AS srid,
  width(transformPipeline(r, '+proj=utm +zone=31 +ellps=WGS84', 32631)) AS width,
  height(transformPipeline(r, '+proj=utm +zone=31 +ellps=WGS84', 32631)) AS height
FROM rast;
-- 32631 | 3 | 3
```

- Devuelve un ráster remuestreado a otro tamaño de píxel

```
rescale(raster,scalex float,scaley float,algorithm text='NearestNeighbour',
  maxerr float=0.125) → raster
```

El resultado conserva el sistema de referencia y la esquina superior izquierda del ráster y expresa su cobertura sobre una malla del tamaño de píxel pedido, mediante la misma deformación de GDAL que `ST_Rescale` de PostGIS, de modo que una escala más gruesa lee menos píxeles sobre el mismo terreno. Las escalas están en las unidades del sistema de referencia del ráster, y el algoritmo y el error son los de transform.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT width(rescale(r, 0.5, -0.5)) AS width,
  height(rescale(r, 0.5, -0.5)) AS height,
  scaleX(rescale(r, 0.5, -0.5)) AS scalex
FROM rast;
-- 6 | 6 | 0.5
```

- Devuelve las estadísticas de los píxeles de una banda de ráster

```
summaryStats(raster,nband integer=1,exclude_nodata_value boolean=true) → summarystats
```

La banda se lee una sola vez y devuelve el registro `summarystats` de PostGIS que devuelve `ST_SummaryStats`: el número de píxeles contados, y la suma, la media, la desviación estándar, el mínimo y el máximo de sus valores. Se leen todos los píxeles, por lo que la respuesta es exacta y no muestreada; los píxeles que la banda declara como nodata quedan fuera salvo que `exclude_nodata_value` sea falso, y una banda sin ningún píxel que contar devuelve un recuento de cero y ninguna estadística.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
```

```
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT (summaryStats(r)).* FROM rast;
-- 9 | 450 | 50 | 25.81988897471611 | 10 | 90
```

- Devuelve las estadísticas de los píxeles de una banda sobre los rásteres de una columna

```
summaryStatsAgg(raster,nband integer,exclude_nodata_value boolean) → summarystats
summaryStatsAgg(raster,exclude_nodata_value boolean) → summarystats
```

El agregado `ST_SummaryStatsAgg` de PostGIS, que devuelve el registro `summarystats` que summaryStats devuelve para un ráster sobre todos los píxeles de la banda de los rásteres, la banda 1 cuando se omite `nband`. Se leen todos los píxeles, como en `summaryStats`, por lo que las formas no toman ningún porcentaje de muestreo y la desviación estándar es la de todos los píxeles y no la de una muestra. Un ráster `NULL` se omite, sobre ningún ráster el agregado devuelve `NULL`, y unos rásteres sin ningún píxel que contar devuelven un recuento de cero y ninguna estadística. El agregado se ejecuta en paralelo, combinando las estadísticas de los procesos, mientras que el de PostGIS no tiene función de combinación. Una banda que un ráster no tiene genera un error, mientras que PostGIS omite el ráster.

```
WITH tiles AS (
  SELECT ST_AddBand(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857),
    '32BF'::text, 1.0::float8, -9999.0::float8) AS r
  UNION ALL
  SELECT ST_AddBand(ST_MakeEmptyRaster(2, 2, 1.0, 1.0, 1.0, -1.0, 0.0, 0.0, 3857),
    '32BF'::text, 3.0::float8, -9999.0::float8) )
SELECT (summaryStatsAgg(r, 1, true)).* FROM tiles;
-- 8 | 16 | 2 | 1 | 1 | 3
```

- Devuelve el número de píxeles, la suma, la media, la desviación estándar, el mínimo o el máximo de las estadísticas de una banda de ráster

```
summarystatsCount(summarystats) → bigint
summarystatsSum(summarystats) → float
summarystatsMean(summarystats) → float
summarystatsStddev(summarystats) → float
summarystatsMin(summarystats) → float
summarystatsMax(summarystats) → float
```

Cada función lee un campo del registro `summarystats` que devuelven summaryStats y summaryStatsAgg, la forma en que todo motor lo lee (Capítulo 18), junto a la lectura de un campo por su nombre, `(summaryStats(r)).mean`, que PostgreSQL conserva. Sobre unas estadísticas que no cuentan ningún píxel el recuento es 0 y las demás funciones devuelven `NULL`.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT summarystatsCount(summaryStats(r)), summarystatsMean(summaryStats(r)),
  summarystatsStddev(summaryStats(r)) FROM rast;
-- 9 | 50 | 25.81988897471611
```

- Devuelve los polígonos de una banda de ráster, uno por cada grupo de píxeles que comparten un valor `{}`

```
dumpAsPolygons(raster,band integer=1,exclude_nodata_value boolean=true) → {geomval}
```

Una banda declara un valor por píxel, y esta función declara la misma banda como las regiones que cubren esos valores, devolviendo los registros `geomval` de PostGIS que devuelve `ST_DumpAsPolygons`. Cada polígono lleva el sistema de referencia del ráster, donde `ST_DumpAsPolygons` no declara ninguno, de modo que puede compararse con las trayectorias a lo largo de las cuales se lee la cobertura. Los píxeles que la banda declara como nodata quedan fuera salvo que `exclude_nodata_value` sea falso, una banda cuyos píxeles son todos nodata no cubre nada y no devuelve ninguna fila, y una banda que el ráster no tiene genera un error.

```
-- Classify a 3x3 synthetic elevation raster, then read each class as a region
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT val, ST_AsText(geom)
FROM rast, dumpAsPolygons(reclass(r, 1, '0-50:1, [50-100]:2', '8BUI', 0))
ORDER BY val;
-- 1 | POLYGON((0 3,0 1,1 1,1 2,3 2,3 3,0 3))
-- 2 | POLYGON((1 2,1 1,0 1,0 0,3 0,3 2,1 2))
```

- Devuelve los píxeles de una banda de ráster como puntos {}

```
pixelAsPoints(rast raster,band integer=1,exclude_nodata_value boolean=true)
  → setof (geom geometry,val float,x integer,y integer)
```

Los puntos de `ST_PixelAsPoints` de PostGIS: para cada píxel, fila por fila, su esquina superior izquierda en el sistema de referencia del ráster, su valor y su columna y su fila, empezando en 1. Los píxeles que la banda declara como nodata quedan fuera salvo que `exclude_nodata_value` sea falso, en cuyo caso conservan el valor que la banda tiene para ellos, y una banda que no declara valor nodata no deja ninguno fuera. Una banda que el ráster no tiene genera un error, donde PostGIS no devuelve ninguna fila.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 2.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.5, -9999, 3], [4, 5, 6]]::double precision[][]) AS r )
SELECT ST_AsText(geom), val, x, y FROM rast, pixelAsPoints(r) LIMIT 2;
-- POINT(10 20) | 1.5 | 1 | 1
-- POINT(14 20) |   3 | 3 | 1
```

- Devuelve los valores de las bandas de un ráster como arreglos

```
dumpValues(rast raster,nband integer[]=NULL,exclude_nodata_value boolean=true)
  → setof (nband integer,valarray float[])
dumpValues(rast raster,nband integer,exclude_nodata_value boolean=true) → float[]
```

Los valores de `ST_DumpValues` de PostGIS: un arreglo por banda, una fila del arreglo por fila del ráster, en el orden de las bandas dadas en `nband`, todas las bandas cuando es `NULL`; la forma con una banda devuelve solo su arreglo. Un píxel que la banda declara como nodata es `NULL` salvo que `exclude_nodata_value` sea falso, en cuyo caso es el valor que la banda tiene para él. Una banda que el ráster no tiene, o un número de banda `NULL`, genera un error, donde PostGIS no devuelve ninguna fila o lo omite.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 2.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.5, -9999, 3], [4, 5, 6]]::double precision[][]) AS r )
SELECT dumpValues(r, 1), dumpValues(r, 1, false) FROM rast;
-- {{1.5,NULL,3},{4,5,6}} | {{1.5,-9999,3},{4,5,6}}
```

- Muestrea una tesela ráster `raquet` a lo largo de una trayectoria

```
rasterTileValue(tgeompoint,rast raquet) → tfloat
```

Lee la tesela a lo largo de `traj` (SRID 4326) como rasterValue lee un ráster, devolviendo los valores de píxel como un `tfloat`. Es el equivalente tipado de rasterTileValueQuadbin: las dimensiones, la celda QUADBIN, el tipo de píxel y el tratamiento de nodata se toman del valor `raquet` en lugar de argumentos separados. Devuelve `NULL` cuando el trayecto nunca encuentra un píxel de la tesela con datos.

```
-- Sample pre-packaged raquet tiles along ship trajectories
SELECT t.tripid, rasterTileValue(t.trip, r.tile)
FROM trips t JOIN elevation_tiles r
  ON r.quadbin IN (SELECT unnest(getValues(tquadbin(t.trip, 8))));
```

- Muestrea un arreglo de teselas ráster `raquet` a lo largo de una trayectoria

```
rasterTileValue(tgeompoint,rast raquet[]) → tfloat
```

Lee cada tesela de `rast` a lo largo de `traj` (SRID 4326) y devuelve los valores de píxel como un único `tfloat`. Una trayectoria que sale de una tesela queda cubierta por varias, y cada una responde la parte del trayecto que está sobre ella. Las teselas de un mismo nivel de zoom particionan el plano, de modo que responden tiempos disjuntos. Las teselas de niveles de zoom distintos se solapan, y cuando dos de ellas responden el mismo tiempo se conserva el valor de la tesela de mayor zoom, que es la que tiene la resolución más fina. Devuelve `NULL` cuando el trayecto nunca encuentra un píxel de una tesela con datos.

```
-- Sample a trajectory across every tile it crosses, in one value
SELECT t.tripid, rasterTileValue(t.trip, array_agg(r.tile)) FROM trips t
JOIN elevation_tiles r ON r.quadbin IN (SELECT unnest(getValues(tquadbin(t.trip, 8))))
GROUP BY t.tripid, t.trip;
```

- Muestrea un chip ráster de Raquet a lo largo de una trayectoria usando una celda QUADBIN para la georreferenciación

```
rasterTileValueQuadbin(tgeompoint,pixels bytea,width integer,height integer,
  quadbin,pixtype text,nodata float,has_nodata boolean) → tfloat
```

Lee un arreglo de píxeles de banda única en orden de fila mayor a lo largo de `traj` (SRID 4326), como rasterValue lee un ráster. La georreferenciación de la tesela (rectángulo delimitador, mapeo de píxel a coordenadas) se deriva únicamente de `quadbin` mediante la transformada de tesela deslizante Web-Mercator. El argumento `pixtype` debe ser uno de `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32` o `float64`. Cuando `has_nodata` es verdadero, un píxel igual a `nodata` no lleva valor. Devuelve `NULL` cuando el trayecto nunca encuentra un píxel con datos.

```
-- Sample the 256 x 256 float32 elevation tiles of a Raquet table, stating no
-- nodata value, along every ship trajectory
SELECT t.tripid, rasterTileValueQuadbin(t.trip, r.band_1, 256, 256,
  r.block::quadbin, 'float32', 0, false) FROM trips t
JOIN elevation_raquet r
  ON r.block IN (SELECT unnest(getValues(tquadbin(t.trip, 8)))::bigint);
-- Average sea-surface temperature per trajectory
SELECT t.tripid, avgValue(rasterTileValueQuadbin(t.trip,
  r.band_1, 256, 256, r.block::quadbin, 'int16', -9999, true)) AS mean_sst FROM trips t
JOIN sst_raquet r ON r.block IN (SELECT unnest(getValues(tquadbin(t.trip, 6)))::bigint);
```

- Devuelve la parte de una trayectoria donde el valor ráster que lee cae dentro de un rango de flotante

```
atRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → tgeompoint
atRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → tgeompoint
```

Lee `rasterValue` a lo largo de `traj` y devuelve la sub-trayectoria restringida a los tiempos en que el valor del píxel está dentro de `vspan`. Devuelve `NULL` cuando ningún tiempo satisface la condición. La forma que recibe `path` lee un archivo ráster del servidor, como lo hace la forma correspondiente de rasterValue.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
```

```
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT asText(atRasterValue(tgeompoint 'SRID=4326;
  [POINT(0.5 2.5)@2001-01-01, POINT(1.5 1.5)@2001-01-02, POINT(0.5 0.5)@2001-01-03]', r,
  floatspan '[40, 90]')) FROM rast;
/* {[POINT(1 2)@2001-01-01 12:00:00, POINT(1.5 1.5)@2001-01-02,
   POINT(0.5 0.5)@2001-01-03]} */
```

- Devuelve la parte de una trayectoria donde el valor ráster que lee cae fuera de un rango de flotante

```
minusRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → tgeompoint
minusRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → tgeompoint
```

El complemento de `atRasterValue`: devuelve la sub-trayectoria restringida a los tiempos en que el valor del píxel está fuera de `vspan`. Devuelve `NULL` cuando el valor del píxel está dentro de `vspan` en todo momento. La forma que recibe `path` lee un archivo ráster del servidor, como lo hace la forma correspondiente de rasterValue.

```
-- Keep only the instant whose raster value (10) is below 40
SELECT asText(minusRasterValue(traj, elev_raster, floatspan '[40, 9999]')) FROM trips,
  elevation_raster;
```

- Devuelve verdadero si la trayectoria alguna vez lee un valor de píxel ráster dentro de un rango de flotante

```
eRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → boolean
eRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → boolean
```

Devuelve `true` cuando `traj` lee alguna vez un valor de píxel dentro de `vspan`, `false` en caso contrario. Devuelve `NULL` únicamente cuando la entrada es `NULL`. La forma que recibe `path` lee un archivo ráster del servidor, como lo hace la forma correspondiente de rasterValue.

```
-- Trips that ever crossed terrain above 200 m
SELECT tripid FROM trips,
  elevation_raster WHERE eRasterValue(trip, elev_raster, floatspan '[200, 9999]');
```

- Devuelve verdadero si cada valor de píxel que la trayectoria lee sobre el ráster está dentro de un rango de flotante

```
aRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → boolean
aRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → boolean
```

Devuelve `true` cuando cada valor de píxel que `traj` lee sobre el ráster está dentro de `vspan`, `false` cuando al menos uno no lo está. Devuelve `NULL` únicamente cuando la entrada es `NULL`. La forma que recibe `path` lee un archivo ráster del servidor, como lo hace la forma correspondiente de rasterValue.

```
-- Trips that stayed entirely below 100 m elevation
SELECT tripid FROM trips,
  elevation_raster WHERE aRasterValue(trip, elev_raster, floatspan '[0, 100]');
```

Para usar los operadores descritos aquí, MobilityDB debe ser compilado con `-DQUADBIN=ON` y `-DRASTER=ON`, ya que una tesela Raquet se identifica por una celda `quadbin`. En tal compilación, el archivo `mobilitydb.control` generado declara `requires = 'postgis, postgis_raster'`, por lo que un único `CASCADE` crea la pila completa:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE:  installing required extension "postgis"
-- NOTICE:  installing required extension "postgis_raster"
```

El ráster de PostGIS forma parte del paquete estándar `postgresql-`*N*`-postgis-3` en Debian/Ubuntu; no se requiere ningún paquete adicional. Pase `-DQUADBIN=ON` y `-DRASTER=ON` a CMake:

```
cmake -DQUADBIN=ON -DRASTER=ON ..
make -j$(nproc)
sudo make install
```

Las compilaciones de CI, entre ellas `.github/workflows/pgversion.yml`, pasan `-DALL=ON`, que incluye el ráster junto con las demás familias opcionales.

Capítulo 17

Nubes de puntos temporales

La extensión pgPointCloud almacena datos de nubes de puntos LIDAR en PostgreSQL mediante dos tipos estáticos: `pcpoint`, un único punto multidimensional cuyas dimensiones se describen mediante un *esquema* almacenado en la tabla de catálogo `pointcloud_formats`; y `pcpatch`, un lote comprimido de valores `pcpoint` que comparten el mismo `pcid` (identificador de esquema). Cada esquema fija las dimensiones presentes (`X`, `Y`, `Z`, `Intensity`, ...), su tipo numérico, escala y desplazamiento.

MobilityDB eleva estos tipos al mundo temporal mediante `tpcpoint` (un sensor LIDAR/GPS en movimiento) y `tpcpatch` (una serie temporal de grupos de puntos comprimidos). También añade los tipos de conjunto `pcpointset` / `pcpatchset` y el tipo de caja delimitadora espaciotemporal `tpcbox`. Una referencia completa de los tipos estáticos `pcpoint` y `pcpatch` se ofrece en la documentación de pgPointCloud; este capítulo cubre las adiciones de MobilityDB sobre pgPointCloud.

Para usar los tipos descritos aquí, MobilityDB debe compilarse con la opción de CMake `POINTCLOUD=ON`. En tal compilación, el archivo `mobilitydb.control` generado declara `requires = 'postgis, pointcloud'`, de modo que un único `CASCADE` crea la pila completa:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE:  installing required extension "postgis"
-- NOTICE:  installing required extension "pointcloud"
```

Crear las extensiones a mano en el orden incorrecto, sin `CASCADE`, genera un error de tipo ausente.

Tanto `tpcpoint` como `tpcpatch` admiten la E/S binaria estándar de MobilityDB a través de `asBinary` / `tpcpointFromBinary` / `tpcpatchFromBinary` y a través de `COPY ... (FORMAT BINARY)` de PostgreSQL. La codificación lleva el `pcid` de cada valor junto a la carga de dimensiones y, siempre que el backend de codificación conozca el XML de esquema de ese `pcid`, el esquema mismo, por lo que un valor volcado de un clúster se importa en otro que nunca ha visto el `pcid`.

17.1. Esquemas de nubes de puntos

Cada valor `pcpoint` y `pcpatch` lleva un `pcid` que se resuelve en un *esquema*, que declara las dimensiones que contiene el valor (X, Y, Z, ...), el tipo numérico con el que se almacena cada una, y su escala y desplazamiento. El esquema se declara en SQL, en dos tablas: `pointcloud_schemas` contiene lo que declara un esquema en conjunto, y `pointcloud_dimensions` una fila por dimensión. Un esquema 3D mínimo, con cada dimensión almacenada como `int32_t` con escala unitaria:

```
INSERT INTO pointcloud_schemas (pcid, srid, compression, description)
VALUES (1, 4326, 'none', 'Minimal 3D schema');
INSERT INTO pointcloud_dimensions (pcid, dim_no, dim_name, interpretation, dim_scale,
  dim_offset, active, description)
VALUES (1, 1, 'X', 'int32_t', 1, 0, true, 'Easting'),
  (1, 2, 'Y', 'int32_t', 1, 0, true, 'Northing'),
  (1, 3, 'Z', 'int32_t', 1, 0, true, 'Elevation');
```

El tamaño de cada dimensión y su desplazamiento dentro de un punto no aparecen porque se derivan de la interpretación, y las dimensiones X, Y, Z y M tampoco porque se resuelven a partir de los nombres. La columna `dim_no` declara el orden en el que se almacenan las dimensiones, de modo que no depende del orden en el que se insertan las filas, y `dim_name` es el nombre por el que se resuelven. Ningún nombre de columna necesita comillas en ninguno de los hosts en los que se materializan estas tablas. Un esquema es global a la base de datos y suele registrarse una sola vez, al cargar los datos; el SRID es por esquema y lo heredan todos los valores que llevan ese `pcid`.

Registrar un esquema en las dos tablas también escribe su documento XML en el catálogo `pointcloud_formats` de pgPointCloud, y un cambio en sus filas lo reescribe o lo elimina, de modo que los tipos `pcpoint` y `pcpatch` y las funciones `PC_` de pgPointCloud también leen el esquema. El documento se escribe en cuanto el esquema es uno que pgPointCloud acepta, es decir, en cuanto declara una dimensión X y una dimensión Y y una dimensión en cada posición, para un `pcid` menor que 65536. Para un `pcid` que declaran las dos tablas, el documento en `pointcloud_formats` es el que escriben sus filas.

Una base de datos cuyos esquemas ya están registrados en el catálogo `pointcloud_formats` de pgPointCloud, como documento XML, sigue funcionando: ese catálogo se lee para un `pcid` que las dos tablas anteriores no declaren.

Un cambio en las filas de estas tablas, o de `pointcloud_formats`, alcanza a toda sentencia que comience después de él: una sentencia construye y lee sus valores mediante los esquemas que declaran sus filas cuando comienza, como hace pgPointCloud.

La mezcla de esquemas se rechaza en el momento de la construcción: no se puede construir una secuencia `tpcpoint` cuyos instantes lleven valores `pcid` diferentes, y lo mismo vale para un `pcpointset` y un `pcpatchset`. El esquema es lo que da significado a las dimensiones, por lo que un valor que mezclara esquemas llevaría coordenadas que nada puede interpretar.

Tres funciones auxiliares responden lo que declara un `pcid` sin disponer de un valor de ese esquema y sin depender de la disposición de las tablas. Resuelven el esquema como se lee un valor de él, a partir de las dos tablas anteriores y, en su defecto, de `pointcloud_formats`, de modo que un `pcid` que declara pgPointCloud se responde como uno que declaran las tablas. Cada una responde `NULL` para un `pcid` que ningún esquema declara, y `pointCloudSchemaSRID` responde 0 para un esquema que no declara ningún sistema de referencia.

- Devuelve el sistema de referencia espacial en el que se expresa todo valor de un esquema

```
pointCloudSchemaSRID(integer) → integer
```

```
SELECT pointCloudSchemaSRID(1);
-- 4326
```

- Devuelve cómo se almacena un parche de un esquema

```
pointCloudSchemaCompression(integer) → text
```

```
SELECT pointCloudSchemaCompression(1);
-- none
```

- Devuelve cuántas dimensiones de un esquema contienen valores

```
pointCloudSchemaNDims(integer) → integer
```

```
SELECT pointCloudSchemaNDims(1);
-- 3
```

17.2. Nubes de puntos estáticas

Los tipos estáticos provienen de pgPointCloud. MobilityDB expone las funciones de acceso de coordenadas con conocimiento del esquema necesarias para proyectar los valores de pgPointCloud en el resto del ecosistema de tipos temporales.

17.2.1. Entrada y salida

El texto de un `pcpoint` y de un `pcpatch` es el hexadecimal de su representación binaria conocida (WKB) de pgPointCloud, el texto que leen y escriben las funciones de entrada y salida del tipo de pgPointCloud, y cada motor lo lee y lo escribe con las funciones siguientes.

- Devuelve la representación hexadecimal binaria conocida (HexWKB) de un pcpoint o de un pcpatch

```
asHexWKB({pcpoint,pcpatch}) → text
```

El resultado es el texto que escribe la función de salida del tipo de pgPointCloud, en el orden de bytes de la máquina.

```
SELECT asHexWKB(pcpoint(1, 1.0, 2.0, 3.0));
-- 01010000006400000C80000002C010000
SELECT asHexWKB(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)));
-- 01010000000000000002000000640000006400000064000000C8000000C8000000C8000000
```

- Entrada de un pcpoint o de un pcpatch a partir de su representación hexadecimal binaria conocida (HexWKB)

```
pcpointFromHexWKB(text) → pcpoint
pcpatchFromHexWKB(text) → pcpatch
```

La entrada se lee como la lee la función de entrada del tipo de pgPointCloud, en cualquier orden de bytes, bajo el esquema de su `pcid`.

```
SELECT getZ(pcpointFromHexWKB('01010000006400000C80000002C010000'));
-- 3
SELECT numPoints(pcpatchFromHexWKB('0101000000000000000200000064000000640000006400000064000000 ↩
    C8000000C8000000C8000000'));
-- 2
```

17.2.2. Constructores

La función `pcpoint_in` de pgPointCloud solo acepta hex-WKB, por lo que el constructor canónico es `PC_MakePoint(pcid, ARRAY[...]::float[])`. MobilityDB incluye dos sobrecargas SQL ligeras sobre él para literales de prueba más cortos y consultas ad hoc; la validación subyacente de dimensiones del esquema sigue ocurriendo dentro de `PC_MakePoint`.

- Construye un pcpoint 2D o 3D directamente a partir de coordenadas x/y(/z)

```
pcpoint(pcid integer,x float,y float) → pcpoint
pcpoint(pcid integer,x float,y float,z float) → pcpoint
```

```
SELECT pcpoint(1, 10.0, 20.0, 30.0);
-- Equivalent to pcpoint(1, 10.0, 20.0, 30.0)
```

- Construye un pcpoint a partir de las coordenadas de cada dimensión del esquema

```
pcpoint(pcid integer,coordinates float[]) → pcpoint
```

La disposición del esquema indica cada dimensión, incluidas las inactivas, que es como se dan sus valores a un esquema con más dimensiones de las que alcanzan las sobrecargas anteriores. Ese número es el que toma `PC_MakePoint` y NO es el número de dimensiones activas que informa `pointCloudSchemaNDims`

```
SELECT pcpoint(1, ARRAY[10.0, 20.0, 30.0]::float[]);
```

- Construye un pcpatch a partir de las coordenadas de sus puntos, un punto tras otro

```
pcpatch(pcid integer,coordinates float[]) → pcpatch
```

Para cada punto se da cada dimensión de la disposición del esquema, incluidas las inactivas. Ese número es el que toma `PC_MakePatch` y NO es el número de dimensiones activas que informa `pointCloudSchemaNDims`; un número que sea múltiplo del que no corresponde construye un parche de puntos erróneos en lugar de informar un error. Un parche construido así declara él mismo el esquema, al no haber ningún punto del que leerlo

```
SELECT pcpatch(1, ARRAY[1.0, 1.0, 1.0, 2.0, 2.0, 2.0]::float[]);
```

- Construye un pcpatch a partir de una lista variádica de pcpoints que comparten el mismo pcid

```
pcpatch(VARIADIC points pcpoint[]) → pcpatch
```

```
SELECT pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0));
-- Equivalent to pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0))
```

17.2.3. Accesores

- Devuelve el identificador de esquema de un pcpoint o pcpatch

```
pcid({pcpoint,pcpatch}) → integer
```

```
SELECT pcid(pcpoint(1, 10.0, 20.0, 30.0));
-- 1
```

- Devuelve una coordenada de un pcpoint, NULL si la dimensión está ausente del esquema

```
getX(pcpoint) → float
getY(pcpoint) → float
getZ(pcpoint) → float
```

```
WITH pt AS (SELECT pcpoint(1, 10.0, 20.0, 30.0) p)
  SELECT getX(p), getY(p), getZ(p) FROM pt;
-- 10 | 20 | 30
```

- Devuelve cualquier dimensión con nombre de un pcpoint, NULL si el nombre es desconocido para el esquema

```
getDim(pcpoint,text) → float
```

```
SELECT getDim(pcpoint(1, 10.0, 20.0, 30.0), 'X');
-- 10
```

- Devuelve el identificador de referencia espacial (heredado del esquema)

```
SRID({pcpoint,pcpatch}) → integer
```

```
SELECT SRID(pcpoint(1, 10.0, 20.0, 30.0));
-- 0
SELECT SRID(pcpatch(pcpoint(1, 10.0, 20.0, 30.0), pcpoint(1, 11.0, 21.0, 31.0)));
-- 0
```

- Devuelve el número de puntos de un pcpatch

```
numPoints(pcpatch) → integer
```

```
SELECT numPoints(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)));
-- 2
```

- Devuelve el n-ésimo punto de un pcpatch

```
pointN(pcpatch,integer) → pcpoint
```

Los puntos se numeran desde uno, un entero negativo cuenta desde el final del parche, y un entero que no designa ningún punto devuelve NULL.

```
SELECT pointN(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)), 2)
  = pcpoint(1, 2.0, 2.0, 2.0);
-- true
SELECT pointN(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)), 3)
  IS NULL;
-- true
```

- Devuelve el multipunto de las posiciones que ocupan los puntos de un parche

```
geometry(pcpatch) → geometry
geography(pcpatch) → geography
```

El esquema que nombra el `pcid` decide el SRID y si el resultado lleva una dimensión Z

```
SELECT ST_AsText(geometry(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- MULTIPOINT Z ((1 1 1),(2 2 2))
```

17.3. Conjuntos de nubes de puntos

Un `pcpointset` es un conjunto ordenado de valores `pcpoint` distintos; un `pcpatchset` es el tipo de conjunto análogo para `pcpatch`. Ambos siguen la semántica general de conjuntos de MobilityDB (véase Capítulo 2): los valores se deduplicen en la construcción, la representación binaria es canónica, y todos los operadores genéricos a nivel de conjunto (=, <>, <, <=, >, >=, `@>`, `<@`, `&&`, +, −, ⋆) y las funciones de acceso (`numValues`, `memSize`, `asText`, `asEWKT`, `asBinary`, `asEWKB`, `asHexWKB`, `asHexEWKB`, . . .) están disponibles (véase las representaciones binarias de un conjunto espacial). Las formas extendidas indican el SRID que el esquema del `pcid` declara en el catálogo `pointcloud_formats`, por lo que coinciden con las formas simples cuando ese esquema no declara ningún SRID.

Todos los valores de un conjunto deben compartir el mismo `pcid`. Mezclar esquemas en un único conjunto genera un error en el momento de la construcción.

17.3.1. Constructores

- Construye un conjunto a partir de un array de valores, todos los cuales deben compartir el mismo pcid

```
set(pcpoint[]) → pcpointset
set(pcpatch[]) → pcpatchset
```

```
SELECT numValues(set(ARRAY[pcpoint(1, 10.0, 20.0, 30.0),
  pcpoint(1, 10.0, 20.0, 30.0),  -- duplicate, deduped
  pcpoint(1, 11.0, 21.0, 31.0)]));
-- 2
```

17.4. Cajas delimitadoras de nubes de puntos

El tipo `tpcbox` es la caja delimitadora espaciotemporal para los tipos de nube de puntos temporal. Refleja la estructura de `stbox` (véase Capítulo 3), mismos ejes X/Y/Z/T, mismos bits de indicador para las dimensiones presentes, y añade un único campo: el `pcid`. Dos valores `tpcbox` con distintos valores de `pcid` describen esquemas distintos, y una operación entre ellos genera un error en lugar de comparar su extensión geométrica o temporal.

Propagar el `pcid` a través de la aritmética de cajas delimitadoras garantiza que la poda por caja delimitadora nunca descarte una fila cuyo esquema difiera del esquema de la consulta, y nunca devuelva un resultado entre esquemas incompatibles.

17.4.1. Entrada y salida

- Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB) de una caja de nube de puntos

```
asBinary(tpcbox,endian text='') → bytea
asHexWKB(tpcbox,endian text='') → text
```

El resultado es la representación de la `stbox` que abarca la caja seguida de su `pcid`, codificada utilizando la codificación little-endian (NDR) o big-endian (XDR), la codificación de la máquina cuando no se especifica ninguna. Como para todo tipo espacial, la representación plana no escribe el SRID, que escriben `asEWKB` y `asHexEWKB`.

```
SELECT asHexWKB(tpcboxX(0, 0, 10, 10, 1));
-- 0101000000000000000000000000000000244000000000000000000000000000000244001000000
SELECT asHexWKB(tpcboxX(0, 0, 10, 10, 5), 'XDR');
-- 0001000000000000000040240000000000000000000000000000402400000000000000000005
```

- Devuelve la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB) de una caja de nube de puntos

```
asEWKB(tpcbox,endian text='') → bytea
asHexEWKB(tpcbox,endian text='') → text
```

El resultado es la representación que escriben `asBinary` y `asHexWKB` con el SRID de la caja, el que indica el esquema de su `pcid`, como todo tipo espacial escribe su SRID en su representación extendida.

```
SELECT asHexEWKB(tpcboxX(0, 0, 10, 10, 1));
-- 0101000000000000000000000000000000244000000000000000000000000000000244001000000
SELECT asHexEWKB(tpcboxX(0, 0, 10, 10, 5), 'XDR');
-- 0041000010E600000000000000004024000000000000000000000000000000402400000000000000000005
```

- Entrada de una caja de nube de puntos desde la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)

```
tpcboxFromBinary(bytea) → tpcbox
tpcboxFromHexWKB(text) → tpcbox
```

El SRID del resultado es el que indica el esquema de su `pcid`, que la representación plana no escribe.

```
SELECT asText(tpcboxFromBinary(asBinary(tpcboxX(0, 0, 10, 10, 5))));
-- SRID=4326;TPCBOX(X((0,0),(10,10)), 5)
```

- Entrada de una caja de nube de puntos desde la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB)

```
tpcboxFromEWKB(bytea) → tpcbox
tpcboxFromHexEWKB(text) → tpcbox
```

El SRID que indica la entrada debe ser el que indica el esquema de su `pcid`, cuando ese esquema indica uno.

```
SELECT tpcboxFromHexEWKB(asHexEWKB(tpcboxX(0, 0, 10, 10, 1)));
-- TPCBOX(X((0,0),(10,10)), 1)
SELECT tpcboxFromHexEWKB(replace(asHexEWKB(tpcboxX(0, 0, 10, 10, 5)), 'E6100000', '110 ↩
    F0000'));
-- ERROR:  Could not parse tpcbox value: The value states SRID 3857 and the schema of pcid ↩
     5 states SRID 4326
```

- Devuelve la representación de texto conocido (WKT)

```
asText(tpcbox,maxdecimaldigits integer=15) → text
```

El argumento `maxdecimaldigits` se puede utilizar para definir el número máximo de decimales para la salida de los valores de coma flotante (por defecto 15).

```
SELECT asText(tpcbox 'TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)');
-- TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)
SELECT asText(tpcbox 'TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)', 1);
-- TPCBOX(X((0.1,1.2),(2.4,3.5)), 1)
```

- Entrada desde la representación de texto conocido (WKT)

```
tpcboxFromText(text) → tpcbox
```

La función es la inversa de `asText`: lee el texto en el que se escribe un literal del tipo, tanto de una constante como de una columna `text`.

```
SELECT tpcboxFromText('TPCBOX(ZT(((0,0,0),(10,10,10)),[2024-01-01, 2024-01-02]), 1)');
-- TPCBOX(ZT(((0,0,0),(10,10,10)),[2024-01-01, 2024-01-02]), 1)
```

17.4.2. Constructores

- Construye un tpcbox a partir de límites X/Y/Z/T explícitos y un pcid, cuyo esquema declara el SRID

```
tpcboxX(xmin float,ymin float,xmax float,ymax float,pcid integer=0) → tpcbox
tpcboxZ(xmin float,ymin float,zmin float,xmax float,ymax float,zmax float,
  pcid integer=0) → tpcbox
tpcboxT(period tstzspan,pcid integer=0) → tpcbox
tpcboxXT(xmin float,ymin float,xmax float,ymax float,period tstzspan,
  pcid integer=0) → tpcbox
tpcboxZT(xmin float,ymin float,zmin float,xmax float,ymax float,zmax float,
  period tstzspan,pcid integer=0) → tpcbox
```

El SRID no es un parámetro: se lee del esquema nombrado por el pcid, y un pcid que no nombra ningún esquema registrado es un error

```
SELECT tpcboxX(0, 0, 10, 10, 1);
-- TPCBOX(X((0,0),(10,10)), 1)
SELECT tpcboxZT(0, 0, 0, 10, 10, 10, tstzspan '[2024-01-01, 2024-01-02]', 1);
-- TPCBOX(ZT(((0,0,0),(10,10,10)),[2024-01-01, 2024-01-02]), 1)
```

17.4.3. Conversiones

- Convierte un pcpoint a un tpcbox degenerado (de extensión cero); el SRID y el pcid provienen del esquema

```
tpcbox(pcpoint) → tpcbox
```

```
SELECT tpcbox(pcpoint(1, 10.0, 20.0, 30.0));
-- TPCBOX(Z((10,20,30),(10,20,30)), 1)
```

- Convierte un pcpatch a un tpcbox

```
tpcbox(pcpatch) → tpcbox
```

La extensión en X y en Y del resultado es la de los límites PCBOUNDS integrados en el parche, y su extensión en Z, cuando el esquema que nombra el pcid tiene una dimensión Z, es la que indican los puntos del parche. El SRID es el que declara el esquema nombrado por el pcid.

```
SELECT tpcbox(pcpatch(pcpoint(1, 10.0, 20.0, 30.0)));
-- TPCBOX(Z((10,20,30),(10,20,30)), 1)
```

17.4.4. Accesores

- Devuelve si un tpcbox tiene establecidas las dimensiones X/Y, Z o T

```
hasX(tpcbox) → boolean
hasZ(tpcbox) → boolean
hasT(tpcbox) → boolean
```

```
WITH b AS (SELECT tpcboxX(0, 0, 10, 10, 1) AS b)
  SELECT hasX(b), hasZ(b) FROM b;
-- t | f
```

- Devuelve un límite de coordenada, NULL si la dimensión correspondiente está ausente

```
xmin(tpcbox) → float
xmax(tpcbox) → float
ymin(tpcbox) → float
ymax(tpcbox) → float
zmin(tpcbox) → float
zmax(tpcbox) → float
```

```
WITH b AS (SELECT tpcboxX(0, 0, 10, 10, 1) AS b)
  SELECT xmin(b), xmax(b) FROM b;
-- 0 | 10
```

- Devuelve un límite temporal, NULL si la caja no tiene dimensión T

```
tmin(tpcbox) → timestamptz
tmax(tpcbox) → timestamptz
```

```
SELECT tmin(tpcboxT(tstzspan '[2024-01-01, 2024-01-02]', 1));
-- 2024-01-01
```

- Devuelve el identificador de esquema y el SRID que declara dicho esquema

```
pcid(tpcbox) → integer
SRID(tpcbox) → integer
```

```
WITH b AS (SELECT tpcboxX(0, 0, 10, 10, 1) AS b)
  SELECT pcid(b), SRID(b) FROM b;
-- 1 | 4326
```

17.4.5. Transformaciones

- Devuelve un tpcbox con las coordenadas redondeadas al número de dígitos decimales indicado

```
round(tpcbox,integer=0) → tpcbox
```

```
SELECT round(tpcboxX(0.123456, 1.234567, 2.345678, 3.456789, 1), 2);
-- TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)
```

- Devuelve un tpcbox que declara un sistema de referencia que su esquema no declara

```
setSRID(tpcbox,integer) → tpcbox
```

El esquema nombrado por el pcid es el que contiene el sistema de referencia, por lo que cuando ese esquema declara uno la caja se rechaza: un pcid de 0 no nombra ningún esquema, y un esquema puede no declarar ninguno. No reproyecta las coordenadas, para ello, proyecte primero el tpcpoint subyacente y tome la caja delimitadora del resultado

```
SELECT SRID(setSRID(tpcboxX(0, 0, 10, 10), 4326));
-- 4326
SELECT setSRID(tpcboxX(0, 0, 10, 10, 1), 3857);
-- ERROR: The SRID of a TPCBox is the one its schema states: pcid 1 states SRID 4326
```

17.4.6. Operaciones de conjuntos

- Unión e intersección de dos tpcboxes

```
tpcbox {+, *} tpcbox → tpcbox
```

Ambos operandos deben compartir el mismo `pcid`, y la intersección es NULL cuando las dos cajas son disjuntas.

```
SELECT tpcboxX(0, 0, 5, 5, 1) + tpcboxX(3, 3, 10, 10, 1);
-- TPCBOX(X((0,0),(10,10)), 1)
SELECT tpcboxX(0, 0, 5, 5, 1) * tpcboxX(3, 3, 10, 10, 1);
-- TPCBOX(X((3,3),(5,5)), 1)
```

Nota: no se proporciona un `minus` genérico entre cajas, siguiendo la convención de `stbox` y `tbox`: la diferencia de dos cajas no es en general una caja.

17.4.7. Operaciones topológicas

Todos los operadores topológicos toman dos valores `tpcbox` que declaran el mismo esquema, el mismo sistema de referencia y el mismo tipo de coordenada, plana o esférica. Una discrepancia en cualquiera de los dos genera un error en lugar de responder falso: el `pcid` es lo que da a un número almacenado su significado como coordenada, por lo que dos cajas que nombran esquemas distintos no tienen una extensión común que comparar.

También toman dos cajas que comparten un eje. Una caja que contiene coordenadas y una caja que contiene un período no coinciden en ningún eje, por lo que un predicado topológico entre ellas no tiene nada que comparar y genera un error a su vez.

- Operadores topológicos

```
tpcbox {&&, @>, <@, ~=, -|-} tpcbox → boolean
```

```
SELECT tpcboxX(0, 0, 10, 10, 1) @> tpcboxX(2, 2, 8, 8, 1);
-- t
SELECT tpcboxX(2, 2, 8, 8, 1) <@ tpcboxX(0, 0, 10, 10, 1);
-- t
SELECT tpcboxX(0, 0, 5, 5, 1) && tpcboxX(3, 3, 10, 10, 1);
```

```
-- t
SELECT tpcboxX(0, 0, 2, 2) ~= tpcboxX(0, 0, 2, 2);
-- t
SELECT tpcboxX(0, 0, 2, 2) -|- tpcboxX(5, 5, 7, 7);
-- f
SELECT tpcboxX(0, 0, 5, 5, 1) && tpcboxX(0, 0, 5, 5, 2);
-- ERROR:  Operation on TPCBox values with different schemas: 1 vs 2
SELECT tpcboxX(0, 0, 10, 10, 1) && tpcboxT(tstzspan '[2024-01-01, 2024-01-02]', 0);
-- ERROR:  The temporal values must have at least one common dimension
SELECT tpcbox 'SRID=4326;TPCBOX(X((1,1),(2,2)), 1)' &&
  tpcbox 'GEODTPCBOX(X((1,1),(2,2)), 1)';
-- ERROR:  Operation on mixed planar and geodetic coordinates
```

17.4.8. Operaciones de posición

Dieciséis predicados de posición alineados por eje, paralelos a los de `stbox`. Cada uno comprueba la relación estricta o de solapamiento a lo largo de un eje. Los doce que leen coordenadas requieren el mismo esquema, el mismo sistema de referencia y el mismo tipo de coordenada, y una discrepancia en cualquiera de los tres genera un error; los cuatro que leen el eje temporal no requieren ninguno de los dos, ya que una marca de tiempo significa lo mismo sea cual sea el esquema en el que se leen las coordenadas. Todo predicado requiere que ambas cajas lleven el eje que él lee, de modo que los del eje Z necesitan una dimensión Z en ambas y los del eje T un período en ambas.

- Operadores de posición

```
tpcbox {<<, &<, >>, &>} tpcbox → boolean
tpcbox {<<|, &<|, |>>, |&>} tpcbox → boolean
tpcbox {<</, &</, />>, /&>} tpcbox → boolean
tpcbox {<<#, &<#, #>>, #&>} tpcbox → boolean
```

Los cuatro grupos prueban el eje X, el eje Y, el eje Z y el eje temporal, y cada uno indica, en orden, estrictamente antes, no se extiende después, estrictamente después y no se extiende antes a lo largo de su eje.

```
SELECT tpcboxX(0, 0, 2, 2) << tpcboxX(5, 5, 7, 7);
-- t
SELECT tpcboxX(0, 0, 2, 2) <<| tpcboxX(5, 5, 7, 7);
-- t
-- Neither box carries a z dimension or a time span, so neither of the next two
-- has an axis to compare along. The last pair states two schemas, which the
-- time axis does not read.
SELECT tpcboxX(0, 0, 2, 2) <</ tpcboxX(5, 5, 7, 7);
-- ERROR:  The tpcbox must have Z dimension
SELECT tpcboxX(0, 0, 2, 2) <<# tpcboxX(5, 5, 7, 7);
-- ERROR:  The tpcbox must have T dimension
SELECT tpcboxXT(0, 0, 5, 5, tstzspan '[2024-01-01, 2024-01-15]', 1) <<#
  tpcboxXT(0, 0, 5, 5, tstzspan '[2025-01-01, 2025-01-15]', 2);
-- t
```

17.4.9. Comparaciones

- Comparaciones tradicionales

```
tpcbox {=, <>, <, <=, >, >=} tpcbox → boolean
```

La función compara en el orden siguiente: pcid, srid, flags, período y a continuación límites espaciales en orden XYZ-min/max)

```
SELECT tpcboxX(0, 0, 2, 2) = tpcboxX(0, 0, 2, 2);
-- t
SELECT tpcboxX(0, 0, 2, 2) <> tpcboxX(5, 5, 7, 7);
```

```
-- t
SELECT tpcboxX(0, 0, 2, 2) < tpcboxX(5, 5, 7, 7);
-- t
```

17.5. Puntos de nube de puntos temporales

Un `tpcpoint` es el levantamiento de `pcpoint` a través de la maquinaria temporal de MobilityDB. Admite los mismos tres subtipos que los demás tipos temporales, instante, secuencia (con interpolación discreta o de pasos) y conjunto de secuencias. Todos los instantes de un único valor `tpcpoint` deben compartir el mismo `pcid`; el constructor impone esta restricción.

La caja delimitadora de un `tpcpoint` es un `tpcbox`, calculada en el momento de la construcción a partir de las dimensiones X/Y/Z de los valores `pcpoint` subyacentes junto con las marcas temporales de los instantes.

Presentamos a continuación las funciones y operadores para los puntos de nube de puntos temporales. La mayoría de las funciones y operadores para tipos temporales descritos en los capítulos anteriores pueden aplicarse a los puntos de nube de puntos temporales. Por lo tanto, en las signaturas de las funciones, la notación `base` representa un `pcpoint` y las notaciones `ttype` y `tpoint` representan también un `tpcpoint`. Para evitar redundancias, solo presentamos a continuación algunos ejemplos de estas funciones y operadores para los puntos de nube de puntos temporales.

Una columna o dominio puede fijar el esquema añadiendo el `pcid` como un typmod de entero único, exactamente igual que el `geometry(Point, SRID)` de PostGIS. La mezcla de esquemas en dicha columna es rechazada en el momento de INSERT o de la conversión. El mismo constructor se aplica a `tpcpatch`.

```
CREATE TABLE scans (id int, traj tpcpoint(1), full_scan tpcpatch(1));
-- Accepts pcid 1
INSERT INTO scans VALUES (1,
  tpcpoint(pcpoint(1, 1.0, 2.0, 3.0), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz));
-- Rejects any other pcid:
-- ERROR: Pcid of tpcpoint value (2) does not match column typmod pcid (1)
```

Omitir el typmod (`tpcpoint` sin pcid entre paréntesis) deja la columna sin restricción de esquema. La comprobación en tiempo de ejecución del agregado `extent` sigue rechazando entradas con pcid mixtos a mitad de consulta incluso en columnas sin restricciones.

17.5.1. Entrada y salida

- Devuelve la representación textual de un tpcpoint, o de un arreglo de ellos, con o sin su SRID, y convertir un tpcpoint a texto

```
asText(tpcpoint) → text
asText(tpcpoint[]) → text[]
asEWKT(tpcpoint) → text
asEWKT(tpcpoint[]) → text[]
tpcpoint::text → text
```

  `asEWKT` antepone el SRID que declara el esquema del `pcid`, y escribe solo la forma textual bajo un esquema que no declara ninguno, como el pcid 1 a continuación.

```
-- The pcpoint payload renders as hex-encoded WKB.
SELECT asText(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- 0101000000640000006400000064000000@2001-01-01
SELECT asEWKT(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- 0101000000640000006400000064000000@2001-01-01
```

- Devuelve la representación JSON de características móviles (MF-JSON) de un tpcpoint y la lee de vuelta; la ida y vuelta es sin pérdida

```
asMFJSON(tpcpoint,options integer=0,flags integer=0,maxdecimaldigits integer=15) → text
tpcpointFromMFJSON(text) → tpcpoint
```

Un `tpcpoint` se emite como `{"type":"MovingPCPoint", ... "coordinates":[[X,Y,Z], ...], "values":[{"pcid":N,"pt":[...]}, ...], "datetimes":[...]}`. Las coordenadas son las X, Y y Z de cada instante leídas de su `pcpoint` mediante la caché de esquemas, y un esquema que no indica ni X ni Y deja vacío el array de coordenadas de ese instante. Los valores son la forma de texto de pgPointCloud de cada punto, cada dimensión de su esquema en el orden en que el esquema las indica y en la forma decimal más corta que se lee de vuelta como el mismo double, sea cual sea el valor de `maxdecimaldigits`. `tpcpointFromMFJSON` reconstruye cada punto a partir de sus valores mediante el esquema de su `pcid`, de modo que una dimensión entera escalada se lee de vuelta como el entero que almacena.

```
SELECT asMFJSON(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
/* {"type":"MovingPCPoint","coordinates":[[1,1,1]],"values":[{"pcid":1,"pt":[1,1,1]}],
   "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
SELECT tpcpointFromMFJSON(asMFJSON(t)) = t
FROM (SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) AS t) s;
-- t
```

- Devuelve la representación binaria o binaria bien conocida hexadecimal de un tpcpoint, con o sin su SRID

```
asBinary(tpcpoint,endian text='') → bytea
asEWKB(tpcpoint,endian text='') → bytea
asHexWKB(tpcpoint,endian text='') → text
asHexEWKB(tpcpoint,endian text='') → text
```

```
SELECT tpcpointFromEWKB(asEWKB(t)) = t
FROM (SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) AS t) s;
-- t
```

- Devuelve un tpcpoint a partir de su representación textual, binaria o binaria bien conocida hexadecimal, con o sin su SRID

```
tpcpointFromText(text) → tpcpoint
tpcpointFromEWKT(text) → tpcpoint
tpcpointFromBinary(bytea) → tpcpoint
tpcpointFromEWKB(bytea) → tpcpoint
tpcpointFromHexWKB(text) → tpcpoint
tpcpointFromHexEWKB(text) → tpcpoint
```

El `pcid` que lleva cada punto nombra su esquema, por lo que la forma textual se lee de vuelta como la binaria, y un SRID indicado por la entrada debe ser el que declara ese esquema.

```
SELECT tpcpointFromText(asText(t)) = t
FROM (SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) AS t) s;
-- t
SELECT tpcpointFromEWKT('SRID=4326;' ||
  asText(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)));
-- ERROR:  The SRID of a pcpoint is the one its schema declares and cannot be set
```

`asText` no toma un argumento `maxdecimaldigits`, ya que un `pcpoint` se imprime como el hexadecimal de su WKB de pgPointCloud, el texto que escribe la función de salida del tipo de pgPointCloud, que no tiene decimales que redondear. MF-JSON lleva una clave `"bbox"` cuando `options` es 1, un objeto JSON TPCBox que añade un campo `"pcid"` junto al array `"bbox"` con forma de stbox estándar.

17.5.2. Constructores

- Constructor para puntos de nube de puntos temporales con un valor constante

```
tpcpoint(pcpoint,timestamptz) → tpcpointInst
tpcpoint(pcpoint,tstzset) → tpcpointDiscSeq
tpcpoint(pcpoint,tstzspan) → tpcpointContSeq
tpcpoint(pcpoint,tstzspanset) → tpcpointSeqSet
```

```
SELECT asText(tpcpoint(pcpoint(1, 1, 1, 1), timestamptz '2001-01-01'));
-- 0101000000640000006400000064000000@2001-01-01
SELECT asText(tpcpoint(pcpoint(1, 1, 1, 1), tstzspan '[2001-01-01, 2001-01-02]'));
-- [0101000000640000...64000000@2001-01-01, 0101000000640000...64000000@2001-01-02]
```

- Constructor para puntos de nube de puntos temporales de subtipo secuencia

```
tpcpointSeq(tpcpointInst[],interp text='step',leftInc boolean=true,
  rightInc boolean=true) → tpcpointSeq
```

La interpolación es `'discrete'` o `'step'`; las dimensiones que lleva un `pcpoint` no interpolan linealmente, por lo que `'linear'` genera `The temporal type cannot have linear interpolation`.

```
SELECT tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1.0, 2.0, 3.0), '2024-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4.0, 5.0, 6.0), '2024-01-02'::timestamptz)], 'step');
```

- Construye un pcpoint temporal de subtipo conjunto de secuencias a partir de un array de secuencias

```
tpcpointSeqSet(tpcpoint[]) → tpcpointSeqSet
```

```
SELECT numSequences(tpcpointSeqSet(ARRAY[tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)])]));
-- 1
SELECT numInstants(tpcpointSeqSet(ARRAY[tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)])]));
-- 2
```

17.5.3. Conversiones

- Convierte un punto de nube de puntos temporal a un punto de geometría temporal

```
tpcpoint::{tgeompoint,tgeogpoint}
```

Las conversiones siguen la regla de [?listitem].

Toda dimensión no espacial se proyecta y las marcas temporales se preservan. El resultado lleva el SRID que indica el esquema, y es tridimensional cuando el esquema indica una dimensión Z. No se proporciona la conversión inversa, ya que reconstruir un `pcpoint` necesita un esquema destino que el valor temporal por sí solo no lleva.

```
SELECT ST_AsText(startValue(tpcpoint(pcpoint(1, 10.0, 20.0, 30.0),
  '2024-01-01'::timestamptz)::tgeompoint));
-- POINT Z (10 20 30)
```

- Convierte un punto de nube de puntos temporal en su caja delimitadora

```
tpcbox(tpcpoint) → tpcbox
tpcpoint::tpcbox
```

La caja es la que responde la extensión del valor: la extensión espacial de sus puntos, su intervalo temporal, el SRID que indica su esquema y su `pcid`.

```
SELECT tpcpoint(pcpoint(1, 10.0, 20.0, 30.0), '2024-01-01'::timestamptz)::tpcbox;
-- TPCBOX(ZT(((10,20,30),(10,20,30)),[2024-01-01, 2024-01-01]), 1)
```

17.5.4. Accesores

- Devuelve el id de esquema de pgPointCloud compartido por todos los instantes

```
pcid(tpcpoint) → integer
```

```
SELECT pcid(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- 1
```

- Devuelve el SRID heredado del esquema

```
SRID({tpcpoint,tpcpatch}) → integer
```

```
-- The SRID is that of the schema registered for the pcid.
SELECT SRID(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- 0
SELECT SRID(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01'::timestamptz));
-- 0
```

- Proyecta un tpcpoint sobre una dimensión espacial, produciendo un tfloat

```
getX(tpcpoint) → tfloat
getY(tpcpoint) → tfloat
getZ(tpcpoint) → tfloat
```

```
SELECT startValue(getX(tpcpoint(pcpoint(1, 10.0, 20.0, 30.0),
  '2024-01-01'::timestamptz)));
-- 10
```

- Proyecta un tpcpoint sobre cualquier dimensión de esquema con nombre, produciendo un tfloat

```
getDim(tpcpoint,text) → tfloat
```

```
SELECT getDim(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]), 'X');
-- Interp=Step;[1@2001-01-01, 2@2001-01-02]
```

17.5.5. Transformaciones

- Transforma un tpcpoint en otro subtipo

```
tpcpointInst(tpcpoint) → tpcpoint
tpcpointSeq(tpcpoint,interp text='step') → tpcpoint
tpcpointSeqSet(tpcpoint) → tpcpoint
```

```
SELECT numInstants(tpcpointInst(tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz)));
-- 1
SELECT numInstants(tpcpointSeq(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)])));
-- 2
```

- Transforma un tpcpoint a otra interpolación

```
setInterp(tpcpoint,interp text) → tpcpoint
```

```
-- A tpcpoint carries step interpolation.
SELECT interp(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]));
-- Step
SELECT interp(setInterp(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)], 'discrete'), 'step'));
-- Step
```

17.5.6. Modificaciones

- Inserta un tpcpoint en otro, o actualizar otro con él

```
insert(tpcpoint,tpcpoint,connect boolean=true) → tpcpoint
update(tpcpoint,tpcpoint,connect boolean=true) → tpcpoint
```

```
SELECT numInstants(insert(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]),
  tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)])));
-- 3
```

- Elimina un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un tpcpoint

```
deleteTime(tpcpoint,times,connect boolean=true) → tpcpoint
```

```
-- Deleting the middle instant splits the sequence into {[2001-01-01, 2001-01-02),
-- (2001-01-02, 2001-01-03]}; each half stores its bound at the deleted timestamp, giving
-- four instants.
SELECT numInstants(deleteTime(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- 4
```

- Añade un instante o una secuencia a un tpcpoint

```
appendInstant(tpcpoint,tpcpoint) → tpcpoint
appendSequence(tpcpoint,tpcpoint) → tpcpoint
```

```
SELECT numInstants(appendInstant(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)));
-- 3
```

- Fusionar los valores temporales

```
merge(tpcpoint,tpcpoint) → tpcpoint
merge(tpcpoint[]) → tpcpoint
```

```
SELECT numInstants(merge(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01':: ↩
    timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]),
  tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)])));
-- 3
SELECT numInstants(merge(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]));
-- 2
```

17.5.7. Restricciones

- Restringe un tpcpoint a (al complemento de) un valor pcpoint o un conjunto de valores pcpoint

```
atValue(tpcpoint,pcpoint) → tpcpoint
minusValue(tpcpoint,pcpoint) → tpcpoint
atValues(tpcpoint,pcpointset) → tpcpoint
minusValues(tpcpoint,pcpointset) → tpcpoint
```

```
-- With step interpolation the matching value is held on [2001-01-01, 2001-01-02); its
-- closing bound is stored as a second instant.
SELECT numInstants(atValue(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), pcpoint(1, 1, 1, 1)));
-- 2
SELECT numInstants(minusValue(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), pcpoint(1, 1, 1, 1)));
-- 2
```

- Restringe un tpcpoint a un selector temporal (marca de tiempo, periodo, conjunto o spanset), o eliminarlo

```
atTime(tpcpoint,times) → tpcpoint
minusTime(tpcpoint,times) → tpcpoint
```

```
SELECT numInstants(atTime(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- 1
SELECT numInstants(atTime(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tstzspan '[2001-01-01, 2001-01-02]'));
-- 2
```

- Restringe un tpcpoint al extent espacial / temporal de un tpcbox, o eliminarlo

```
atTpcbox(tpcpoint,tpcbox,border_inc boolean=true) → tpcpoint
minusTpcbox(tpcpoint,tpcbox,border_inc boolean=true) → tpcpoint
```

Devuelve NULL cuando el pcid del tpcbox no coincide con el del tpcpoint. Internamente proyecta a un tgeompoint a través de la caché de esquemas, restringe la proyección mediante el stbox equivalente y devuelve el span temporal del resultado al tpcpoint original para preservar los valores pcpoint completos

```
SELECT numInstants(atTpcbox(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcboxZT(0, 0, 0, 2, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 2
-- The box does not intersect the tpcpoint, so nothing is removed.
SELECT numInstants(minusTpcbox(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcboxZT(10, 10, 10, 12, 12, 12, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 3
```

- Restringe un tpcpoint a los instantes anteriores o posteriores a una marca de tiempo

```
beforeTimestamp(tpcpoint,timestamptz,strict boolean=true) → tpcpoint
afterTimestamp(tpcpoint,timestamptz,strict boolean=true) → tpcpoint
```

El indicador strict excluye el instante en la marca de tiempo misma

```
SELECT timeSpan(beforeTimestamp(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- [2001-01-01, 2001-01-02)
SELECT timeSpan(afterTimestamp(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- (2001-01-02, 2001-01-03]
```

17.5.8. Operaciones de división

- Devuelve los valores distintos de un tpcpoint con el conjunto de spans durante el cual toma cada uno de ellos

```
unnest(tpcpoint) → {(value,time)}
```

```
SELECT (un).time FROM (SELECT unnest(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2024-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-03'::timestamptz)])) AS un) t;
-- {[2024-01-01, 2024-01-02), [2024-01-03, 2024-01-03]}
-- {[2024-01-02, 2024-01-03)}
```

- Fragmenta un tpcpoint en fragmentos, uno por cada intervalo de tiempo

```
timeSplit(tpcpoint,duration interval,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(time,tpcpoint)}
```

El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se genera el fragmento que comienza en ese borde y solo lo contiene.

```
SELECT (ts).time, numInstants((ts).temp) FROM (SELECT timeSplit(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2024-01-03'::timestamptz)]),
  interval '1 day', '2024-01-01'::timestamptz) AS ts) t;
-- 2024-01-01 | 2
-- 2024-01-02 | 2
-- 2024-01-03 | 1
```

- Devuelve el arreglo de lapsos de tiempo de los segmentos de un tpcpoint

```
spans(tpcpoint) → tstzspan[]
```

```
SELECT spans(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2024-01-03'::timestamptz)]));
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

- Devuelve los lapsos de tiempo de un tpcpoint dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

```
splitNSpans(tpcpoint,integer) → tstzspan[]
splitEachNSpans(tpcpoint,integer) → tstzspan[]
```

```
SELECT splitNSpans(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2024-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2024-01-03'::timestamptz)]), 2);
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

17.5.9. Operaciones de caja delimitadora

Los operadores de caja delimitadora evalúan contra el `tpcbox` del valor; véase Sección 17.4.7 y Sección 17.4.8 para la geometría por operador. Cada operador a continuación está conectado contra `tpcbox`, `tstzspan` y el propio `tpcpoint`.

- Operadores topológicos

```
{tstzspan,tpcbox,tpcpoint} {&&, @>, <@, ~=, -|-} {tstzspan,tpcbox,tpcpoint} → boolean
```

```
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) &&
  tstzspan '[2001-01-02, 2001-01-04]';
-- true
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) @>
  tpcboxZ(1, 1, 1, 2, 2, 2, 1);
-- true
```

- Operadores de posición

```
{tpcbox,tpcpoint} {<<, >>, <<|, |>>, <</, />>} {tpcbox,tpcpoint} → boolean
{tstzspan,tpcbox,tpcpoint} {<<#, #>>} {tstzspan,tpcbox,tpcpoint} → boolean
```

```
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) <<
  tpcboxZ(10, 10, 10, 12, 12, 12, 1);
-- true
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) <<#
  tstzspan '[2001-01-05, 2001-01-06]';
-- true
```

17.5.10. Operaciones de distancia

- Devuelve la distancia más cercana

```
nearestApproachDistance(tpcpoint,{tpcbox,tpcpoint}) → float
```

Dos valores cuyos esquemas difieren no son comparables, por lo que un desajuste de pcid genera `Operation on pcpoint values with different schemas`, nombrando ambos, en lugar de responder.

```
SELECT round(nearestApproachDistance( tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 10, 10, 10), '2001-01-01'::timestamptz))::numeric, 4);
-- 15.5885
-- A tpcbox without a time span measures the spatial approach only.
SELECT round(nearestApproachDistance(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcboxZ(10, 10, 10, 12, 12, 12, 1))::numeric, 4);
-- 12.1244
SELECT round((tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) |=|
  tpcpoint(pcpoint(1, 10, 10, 10), '2001-01-01'::timestamptz))::numeric, 4);
-- 15.5885
```

- Devuelve el instante del tpcpoint en el que los dos argumentos están a la distancia más cercana

```
nearestApproachInstant({geometry,tpcpoint},{geometry,tpcpoint}) → tpcpoint
```

```
SELECT asText(nearestApproachInstant(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4, 4, 4), '2001-01-02'::timestamptz)]), geometry 'Point Z(5 5 5)'));
-- 0101000000900100009001000090010000@2001-01-02
```

- Devuelve la línea que une el punto de acercamiento más cercano entre los dos argumentos

```
shortestLine({geometry,tpcpoint},{geometry,tpcpoint}) → geometry
```

```
SELECT ST_AsText(shortestLine(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4, 4, 4), '2001-01-02'::timestamptz)]), geometry 'Point Z(5 5 5)'));
-- LINESTRING Z (4 4 4,5 5 5)
```

- Devuelve la distancia temporal

```
{geometry,tpcpoint} <-> {geometry,tpcpoint} → tfloat
```

```
SELECT round(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4, 4, 4), '2001-01-02'::timestamptz)]) <->
  geometry 'Point Z(5 5 5)', 4);
-- [6.9282@2001-01-01, 1.7321@2001-01-02]
```

17.5.11. Relaciones siempre y alguna vez

Un `tpcpoint` se proyecta a un `tgeompoint` mediante la caché de esquemas y la relación se responde allí, de modo que un tpcpoint declara las relaciones que declara su destino de conversión.

- Relaciones ever y always

```
eContains(geometry,tpcpoint) → boolean
aContains(geometry,tpcpoint) → boolean
eCovers(geometry,tpcpoint) → boolean
aCovers(geometry,tpcpoint) → boolean
eDisjoint({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
aDisjoint({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
eDwithin({geometry,tpcpoint},{geometry,tpcpoint},dist float) → boolean
aDwithin({geometry,tpcpoint},{geometry,tpcpoint},dist float) → boolean
eIntersects({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
aIntersects({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
eTouches(geometry,tpcpoint) → boolean
aTouches(geometry,tpcpoint) → boolean
eTouches(tpcpoint,geometry) → boolean
aTouches(tpcpoint,geometry) → boolean
```

Un punto en movimiento ni contiene ni cubre una geometría, y dos puntos en movimiento no se tocan, por lo que `eContains`, `eCovers` y sus gemelas always toman primero la geometría, y `eTouches` y `aTouches` leen cualquiera de los dos órdenes pero no dos puntos. Son relaciones planares, por lo que un punto cuyo esquema indica una dimensión Z encuentra el rechazo `The tgeompoint cannot have Z dimension`.

```
SELECT eContains(geometry 'Polygon((0 0,0 4,4 4,4 0,0 0))',
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- ERROR:  The tgeompoint cannot have Z dimension
SELECT eIntersects(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
```

```
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- t
SELECT aIntersects(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- f
SELECT eIntersects(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]));
-- t
SELECT eDisjoint(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- t
SELECT aDisjoint(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- f
SELECT eDwithin(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (5 5 5)',
  4);
-- t
SELECT aDwithin(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (5 5 5)',
  4);
-- f
```

17.5.12. Relaciones espaciotemporales

Las relaciones *temporales* calculan la relación topológica o de distancia en cada instante y dan como resultado un `tbool`. Leen la misma proyección a `tgeompoint` que leen las relaciones ever/always, y están conectadas en los mismos tres órdenes de argumentos. Un tpcpoint lleva la Z que declara su esquema, si la hay: `tContains`, `tCovers` y `tTouches` son relaciones planares, de modo que un esquema cuyas dimensiones no incluyen Z las responde y uno que lleva Z es rechazado. Esa proyección lleva la interpolación de su origen, de modo que una secuencia tpcpoint se responde a lo largo de su trayectoria y no solo en sus instantes.

- Relaciones espaciotemporales

```
tDisjoint({geometry,tpcpoint},{geometry,tpcpoint}) → tbool
tDwithin({geometry,tpcpoint},{geometry,tpcpoint},float) → tbool
tIntersects({geometry,tpcpoint},{geometry,tpcpoint}) → tbool
tContains(geometry,tpcpoint) → tbool
tCovers(geometry,tpcpoint) → tbool
tTouches(geometry,tpcpoint) → tbool
tTouches(tpcpoint,geometry) → tbool
```

Un punto de nube de puntos temporal declara las relaciones que declara su destino de conversión, de modo que `tContains` y `tCovers` toman la geometría en primer lugar únicamente y `tTouches` no tiene dirección entre dos puntos de nube de puntos. Las tres son relaciones planares: un esquema cuyas dimensiones no incluyen Z las responde, y uno que lleva Z encuentra el rechazo `The tgeompoint cannot have Z dimension`.

```
SELECT tIntersects(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
```

```
-- t@2001-01-01
SELECT tDisjoint(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- f@2001-01-01
SELECT tDisjoint(tpcpoint(pcpoint(1, 9, 9, 9), '2001-01-02'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-02
SELECT tDisjoint(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 9, 9, 9), '2001-01-01'::timestamptz));
-- t@2001-01-01
SELECT tDwithin(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Point(1 2)', 2.0);
-- t@2001-01-01
SELECT tDwithin(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Point(1 5)', 2.0);
-- f@2001-01-01
```

17.5.13. Comparaciones

- Comparaciones tradicionales

```
tpcpoint {=, <>, <, <=, >, >=} tpcpoint → boolean
```

El orden es lexicográfico sobre las codificaciones canónicas de los dos valores, no geométrico.

```
SELECT tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz) = tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz);
-- t
SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) <> tpcpoint(pcpoint(1, 2,
  2, 2), '2001-01-02'::timestamptz);
-- t
SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) < tpcpoint(pcpoint(1, 2,
  2, 2), '2001-01-02'::timestamptz);
-- t
```

- Comparaciones alguna vez y siempre

```
{pcpoint,tpcpoint} {?=, %=, ?<>, %<>} {pcpoint,tpcpoint} → boolean
```

```
SELECT pcpoint(1, 1, 1, 1) ?= tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]);
-- t
SELECT tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) %= pcpoint(1, 1, 1, 1);
-- f
```

- Comparaciones temporales

```
{pcpoint,tpcpoint} {#=, #<>} {pcpoint,tpcpoint} → tbool
```

```
SELECT tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) #= pcpoint(1, 1, 1, 1);
-- [t@2001-01-01, f@2001-01-02, f@2001-01-03]
```

17.5.14. Simplificación

- Devuelve un tpcpoint simplificado asegurando que los puntos consecutivos estén separados al menos una cierta distancia o intervalo de tiempo, o simplificado mediante el algoritmo de Douglas-Peucker

```
minDistSimplify(tpcpoint,mindist float) → tpcpoint
minTimeDeltaSimplify(tpcpoint,mint interval) → tpcpoint
maxDistSimplify(tpcpoint,maxdist float,syncdist boolean=true) → tpcpoint
douglasPeuckerSimplify(tpcpoint,maxdist float,syncdist boolean=true) → tpcpoint
```

La simplificación se aplica únicamente a secuencias temporales o conjuntos de secuencias con interpolación lineal, y un tpcpoint tiene interpolación escalonada o discreta, por lo que cada función devuelve una copia del valor dado.

```
SELECT minDistSimplify(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), 2) =
  tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]);
-- t
```

17.6. Parches de nube de puntos temporales

Un `tpcpatch` es la elevación temporal de `pcpatch`. Es el espejo de `tpcpoint` en todos los aspectos, mismos subtipos, mismas funciones de acceso, mismas conversiones y operadores, excepto que cada instante contiene un lote comprimido completo de puntos en lugar de un único punto. La caja delimitadora es de nuevo un `tpcbox`, cuya extensión en X y en Y por instante es el `PCBOUNDS` embebido en el parche y cuya extensión en Z, cuando el esquema tiene una dimensión Z, es la que indican los puntos del parche.

Las operaciones sobre un `tpcpatch` se realizan a dos granularidades, porque los puntos de un parche residen en una carga comprimida que la capa de caja delimitadora no puede inspeccionar. A *nivel de parche* el parche de cada instante se conserva o se descarta como un todo, leyendo únicamente su extensión y la marca de tiempo del instante: esta es la granularidad de atTpcbox / minusTpcbox, de los operadores de caja delimitadora y del agregado `extent`. La extensión en X y en Y es la cabecera `PCBOUNDS` de cuatro valores double (xmin / xmax / ymin / ymax), leída sin descomprimir la carga; como `PCBOUNDS` es 2D, la extensión en Z, que prueba un `tpcbox` con Z, es la que indican los puntos del parche.

A *nivel de punto* se recorre en C cada punto de cada instante. points emite una fila por cada marca de tiempo de instante y punto, mientras que atTpcboxFine / minusTpcboxFine, atGeometry / minusGeometry y eIntersects(tpcpatch,geometry) aplican su predicado a las coordenadas reales y reconstruyen cada instante superviviente con los puntos que lo superaron, descartando los instantes que no conservan ninguno. Pode primero a nivel de parche, con `atTpcbox` o un escaneo de índice GiST / SP-GiST, y refine los supervivientes cuando se necesite fidelidad a nivel de punto.

17.6.1. Entrada y salida

- Devuelve la representación textual de un tpcpatch, o de un arreglo de ellos, con o sin su SRID, y convertir un tpcpatch a texto

```
asText(tpcpatch) → text
asText(tpcpatch[]) → text[]
asEWKT(tpcpatch) → text
asEWKT(tpcpatch[]) → text[]
tpcpatch::text → text
```

Como para un `tpcpoint`, `asEWKT` antepone el SRID que declara el esquema del `pcid`, y escribe solo la forma textual bajo un esquema que no declara ninguno.

```
-- The pcpatch payload renders as hex-encoded WKB.
SELECT left(asText(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)), 24);
```

```
-- EC0100000100000000000000
SELECT left(asEWKT(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)), 24);
-- EC0100000100000000000000
```

- Devuelve la representación binaria o binaria bien conocida hexadecimal de un tpcpatch, con o sin su SRID

```
asBinary(tpcpatch,endian text='') → bytea
asEWKB(tpcpatch,endian text='') → bytea
asHexWKB(tpcpatch,endian text='') → text
asHexEWKB(tpcpatch,endian text='') → text
```

```
SELECT tpcpatchFromHexEWKB(asHexEWKB(t)) = t
FROM (SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz) AS t) s;
-- t
```

- Devuelve un tpcpatch a partir de su representación textual, binaria o binaria bien conocida hexadecimal, con o sin su SRID

```
tpcpatchFromText(text) → tpcpatch
tpcpatchFromEWKT(text) → tpcpatch
tpcpatchFromBinary(bytea) → tpcpatch
tpcpatchFromEWKB(bytea) → tpcpatch
tpcpatchFromHexWKB(text) → tpcpatch
tpcpatchFromHexEWKB(text) → tpcpatch
```

El `pcid` que lleva cada parche nombra su esquema, por lo que la forma textual se lee de vuelta como la binaria, y un SRID indicado por la entrada debe ser el que declara ese esquema.

```
SELECT tpcpatchFromText(asText(t)) = t
FROM (SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz) AS t) s;
-- t
SELECT tpcpatchFromEWKT('SRID=4326;' || asText(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)));
-- ERROR:  The SRID of a pcpatch is the one its schema declares and cannot be set
```

- Devuelve la representación JSON de características móviles (MF-JSON) de un tpcpatch y la lee de vuelta; la ida y vuelta es sin pérdida

```
asMFJSON(tpcpatch,options integer=0,flags integer=0,maxdecimaldigits integer=15) → text
tpcpatchFromMFJSON(text) → tpcpatch
```

Un `tpcpatch` se emite bajo la etiqueta de tipo `"MovingPCPatch"`, y cada instante indica el objeto `{"pcid":N, "npoints":N, "bounds":[xmin,xmax,ymin,ymax], "pts":[[...], ...]}`. Los puntos son la forma de texto de pgPointCloud del parche, cada dimensión de cada punto en el orden en que su esquema las indica y en la forma decimal más corta que se lee de vuelta como el mismo double, sea cual sea el valor de `maxdecimaldigits`. `tpcpatchFromMFJSON` reconstruye cada parche a partir de sus puntos en la compresión que indica su esquema, y rechaza un punto que no indique todas las dimensiones del esquema.

```
SELECT asMFJSON(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz));
/* {"type":"MovingPCPatch","values":[{"pcid":1,"npoints":2,"bounds":[1,2,1,2],
   "pts":[[1,1,1],[2,2,2]]}],"datetimes":["2001-01-01T00:00:00"],"interpolation":"None"}  ↩
      */
SELECT tpcpatchFromMFJSON(asMFJSON(t)) = t
FROM (SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz) AS t) s;
-- t
```

Como para un `tpcpoint`, `asText` no toma un argumento `maxdecimaldigits`.

17.6.2. Constructores

- Constructor para parches de nube de puntos temporales con un valor constante

```
tpcpatch(pcpatch,timestamptz) → tpcpatchInst
tpcpatch(pcpatch,tstzset) → tpcpatchDiscSeq
tpcpatch(pcpatch,tstzspan) → tpcpatchContSeq
tpcpatch(pcpatch,tstzspanset) → tpcpatchSeqSet
```

```
SELECT tempSubtype(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz));
-- Instant
SELECT tempSubtype(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), tstzspan '[2001-01-01, 2001-01-02]'));
-- Sequence
```

- Construye un pcpatch temporal de subtipo secuencia

```
tpcpatchSeq(tpcpatch[]) → tpcpatch
tpcpatchSeq(tpcpatch[],text) → tpcpatch
```

```
SELECT numInstants(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)])), interp(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]));
-- 2 | Step
```

- Construye un pcpatch temporal de subtipo conjunto de secuencias

```
tpcpatchSeqSet(tpcpatch[]) → tpcpatch
```

```
SELECT numSequences(tpcpatchSeqSet(ARRAY[tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)])]));
-- 1
```

17.6.3. Conversiones

- Convierte un parche temporal en la geometría temporal de los multipuntos que ocupan sus parches

```
tgeometry(tpcpatch) → tgeometry
tgeography(tpcpatch) → tgeography
```

El parche de cada instante se convierte en el multipunto de las posiciones que ocupan sus puntos, leídas a través del esquema que nombra su `pcid`

```
SELECT asText(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)::tgeometry);
-- MULTIPOINT Z ((1 1 1),(2 2 2))@2001-01-01
```

- Convierte un parche de nube de puntos temporal en su caja delimitadora

```
tpcbox(tpcpatch) → tpcbox
tpcpatch::tpcbox
```

La caja es la que responde la extensión del valor: la extensión espacial de sus puntos, su intervalo temporal, el SRID que indica su esquema y su `pcid`.

```
SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)::tpcbox;
-- TPCBOX(ZT(((1,1,1),(2,2,2)),[2001-01-01, 2001-01-01]), 1)
```

17.6.4. Accesores

- Devuelve el número de puntos del parche contenido en el primer o último instante

```
startNumPoints(tpcpatch) → integer
endNumPoints(tpcpatch) → integer
```

```
SELECT startNumPoints(tpcpatch(pcpatch(pcpoint(1, 10.0, 20.0, 30.0)),
  '2024-02-01'::timestamptz));
-- 1
```

- Devuelve el número total de puntos de todos los parches de todos los instantes

```
numPoints(tpcpatch) → bigint
```

Se lee directamente de la cabecera de cada parche, sin descompresión.

```
SELECT numPoints(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1.0, 1.0, 1.0),
  pcpoint(1, 2.0, 2.0, 2.0)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3.0, 3.0, 3.0)), '2024-01-02'::timestamptz)]));
-- 3
```

- Emite una fila por cada marca de tiempo de instante y punto del parche

```
points(tpcpatch) → setof (t timestamptz,point pcpoint)
```

Recorre el parche de cada instante y lo descompone mediante la API C en memoria de pgPointCloud. Útil para combinar una columna tpcpatch con predicados por punto que los operadores de nivel de caja delimitadora no pueden expresar. El coste es O(total de puntos), una descompresión por instante más una emisión de fila por punto.

```
SELECT t,
  point FROM points(tpcpatch(pcpatch(pcpoint(1, 1.0, 1.0, 1.0),
  pcpoint(1, 2.0, 2.0, 2.0)), '2024-01-01'::timestamptz));
-- ('2024-01-01',
-- '0101000000640000006400000064000000'::pcpoint)
-- ('2024-01-01',
-- '01010000000C8000000C8000000C8000000'::pcpoint)
```

17.6.5. Transformaciones

- Transforma un tpcpatch en otro subtipo

```
tpcpatchInst(tpcpatch) → tpcpatch
tpcpatchSeq(tpcpatch,interp text='step') → tpcpatch
tpcpatchSeqSet(tpcpatch) → tpcpatch
```

```
SELECT tempSubtype(tpcpatchSeq(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)));
-- Sequence
```

- Transforma un tpcpatch a otra interpolación

```
setInterp(tpcpatch,interp text) → tpcpatch
```

```
SELECT interp(setInterp(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)],
  'discrete'), 'step'));
-- Step
```

17.6.6. Modificaciones

- Inserta un tpcpatch en otro, o actualizar otro con él

```
insert(tpcpatch,tpcpatch,connect boolean=true) → tpcpatch
update(tpcpatch,tpcpatch,connect boolean=true) → tpcpatch
```

```
SELECT numInstants(insert(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)])));
-- 3
```

- Elimina un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un tpcpatch

```
deleteTime(tpcpatch,times,connect boolean=true) → tpcpatch
```

```
-- Deleting the middle instant splits the sequence into {[2001-01-01, 2001-01-02),
-- (2001-01-02, 2001-01-03]}; each half stores its bound at the deleted timestamp, giving
-- four instants.
SELECT numInstants(deleteTime(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]),
  timestamptz '2001-01-02'));
-- 4
```

- Añade un instante o una secuencia a un tpcpatch

```
appendInstant(tpcpatch,tpcpatch) → tpcpatch
appendSequence(tpcpatch,tpcpatch) → tpcpatch
```

```
SELECT numInstants(appendInstant(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)));
-- 3
```

- Fusionar los valores temporales

```
merge(tpcpatch,tpcpatch) → tpcpatch
merge(tpcpatch[]) → tpcpatch
```

```
SELECT numInstants(merge(tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), ' ↩
    2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 2, 2, 2)), '2001-01-02'::timestamptz)]),
  tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 2, 2, 2)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3)), '2001-01-03'::timestamptz)])));
-- 3
SELECT numInstants(merge(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01':: ↩
    timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3)), '2001-01-03'::timestamptz)]));
-- 2
```

17.6.7. Restricciones

- Restringe un tpcpatch a (al complemento de) un valor pcpatch o un conjunto de valores pcpatch

```
atValue(tpcpatch,pcpatch) → tpcpatch
minusValue(tpcpatch,pcpatch) → tpcpatch
atValues(tpcpatch,pcpatchset) → tpcpatch
minusValues(tpcpatch,pcpatchset) → tpcpatch
```

```
-- The matching value is held up to the next instant; the closing bound is stored as a
-- second instant.
SELECT numInstants(atValue(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- 2
SELECT numInstants(minusValue(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- 1
```

- Restringe un tpcpatch a los instantes cuya extensión del parche se superpone al tpcbox, o eliminarlos

```
atTpcbox(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
minusTpcbox(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
```

La granularidad es a nivel de parche: cada instante superviviente conserva su payload pcpatch textualmente. La extensión en X y en Y de un parche es el `PCBOUNDS` 2D de pgPointCloud; cuando la caja tiene Z, la extensión en Z del parche es la que indican sus puntos, y un parche de un esquema sin Z no supera la caja. Devuelve NULL cuando el pcid del tpcbox no coincide.

```
SELECT numInstants(atTpcbox(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcboxXT(0, 0, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 1
-- The dropped patch stays present on the open interval before its instant,
-- so the complement keeps two instants.
SELECT numInstants(minusTpcbox(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcboxXT(0, 0, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 2
```

- Restringe un tpcpatch a los puntos de cada instante que contiene el tpcbox, o los elimina

```
atTpcboxFine(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
minusTpcboxFine(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
```

Cada instante superviviente contiene un pcpatch recién construido que incluye solo los puntos dentro, o fuera en la forma minus, del tpcbox en dos dimensiones, y en tres cuando la caja tiene dimensión Z. Los instantes cuyo parche filtra hasta cero puntos se descartan. Más lenta que la variante aproximada porque cada parche se descomprime y reconstruye; úsela cuando se necesite fidelidad a nivel de punto.

```
SELECT numInstants(atTpcboxFine(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcboxXT(0, 0, 1, 1, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 1
```

- Restringe un tpcpatch a los puntos cuya proyección XY intersecta (o no intersecta, en el caso de minus) una geometría 2D

```
atGeometry(tpcpatch,geometry) → tpcpatch
minusGeometry(tpcpatch,geometry) → tpcpatch
```

Z se ignora. El llamador debe garantizar la compatibilidad de SRID entre el esquema del parche y la geometría.

```
-- The polygon boundary touches the point (3 3) of the patch at 2001-01-02, so that patch
-- survives both the at and the minus side.
SELECT numInstants(atGeometry(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), geometry 'POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))'));
-- 2
SELECT numInstants(minusGeometry(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), geometry 'POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))'));
-- 1
```

- Restringe un tpcpatch a los instantes anteriores o posteriores a una marca de tiempo

```
beforeTimestamp(tpcpatch,timestamptz,strict boolean=true) → tpcpatch
afterTimestamp(tpcpatch,timestamptz,strict boolean=true) → tpcpatch
```

El indicador strict excluye el instante en la marca de tiempo misma

```
SELECT timeSpan(beforeTimestamp(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]),
  timestamptz '2001-01-02'));
-- [2001-01-01, 2001-01-02)
SELECT timeSpan(afterTimestamp(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]),
  timestamptz '2001-01-02'));
-- (2001-01-02, 2001-01-03]
```

17.6.8. Operaciones de división

- Devuelve los valores distintos de un tpcpatch con el conjunto de spans durante el cual toma cada uno de ellos

```
unnest(tpcpatch) → {(value,time)}
```

```
SELECT (un).time FROM (SELECT unnest(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-03'::timestamptz)])) AS un) t;
-- {[2024-01-02, 2024-01-03)}
-- {[2024-01-01, 2024-01-02), [2024-01-03, 2024-01-03]}
```

- Fragmenta un tpcpatch en fragmentos, uno por cada intervalo de tiempo

```
timeSplit(tpcpatch,duration interval,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(time,tpcpatch)}
```

El argumento opcional `borderInc` indica si se incluye el borde superior de la extensión y, por lo tanto, se genera el fragmento que comienza en ese borde y solo lo contiene.

```
SELECT (ts).time, numInstants((ts).temp) FROM (SELECT timeSplit(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2024-01-03'::timestamptz)]),
  interval '1 day', '2024-01-01'::timestamptz) AS ts) t;
-- 2024-01-01 | 2
-- 2024-01-02 | 2
-- 2024-01-03 | 1
```

- Devuelve el arreglo de lapsos de tiempo de los segmentos de un tpcpatch

```
spans(tpcpatch) → tstzspan[]
```

```
SELECT spans(tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-03'::timestamptz)]));
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

- Devuelve los lapsos de tiempo de un tpcpatch dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

```
splitNSpans(tpcpatch,integer) → tstzspan[]
splitEachNSpans(tpcpatch,integer) → tstzspan[]
```

```
SELECT splitNSpans(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-03'::timestamptz)]), 2);
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

17.6.9. Operaciones espaciales

- Devuelve la franja de un parche temporal, la región del plano que barren sus parches

```
swath(tpcpatch,maxt interval=NULL) → geometry
```

La huella de un instante es la envolvente convexa de las posiciones de los puntos de su parche, leídas mediante el esquema que nombra su `pcid` y proyectadas sobre el plano. Las huellas de dos instantes consecutivos se unen mediante la envolvente convexa de ambas, y la franja es la unión de estas bandas, junto con la huella de cada instante que no se une a ninguno de sus vecinos. Los instantes consecutivos se unen salvo que estén separados por más de `maxt`, de modo que una pausa en la adquisición divide la franja; cuando no se da `maxt` se une todo intervalo. Un parche es el conjunto de puntos que captura una adquisición, como un ping de una ecosonda multihaz, y la franja es entonces la banda del fondo marino que sondean los pings de una línea de levantamiento. Mientras que el área recorrida de la conversión a `tgeometry` son los propios sondeos, cuya área es nula, la franja cubre el fondo marino entre ellos. La unión se calcula de forma exacta.

```
SELECT ST_AsText(swath(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 0, 0, 0), pcpoint(1, 1, 0, 0), pcpoint(1, 2, 0, 0)),
    '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 0, 1, 0), pcpoint(1, 1, 1, 0), pcpoint(1, 2, 1, 0)),
    '2001-01-02'::timestamptz)])));
-- POLYGON((0 0,0 1,2 1,2 0,0 0))
SELECT ST_NumGeometries(swath(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 0, 0, 0), pcpoint(1, 2, 0, 0)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 0, 1, 0), pcpoint(1, 2, 1, 0)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 0, 5, 0), pcpoint(1, 2, 5, 0)), '2001-01-05'::timestamptz)]) ↩
    ,
  interval '1 day'));
-- 2
```

17.6.10. Operaciones de caja delimitadora

La misma superficie de operadores de caja que en Sección 17.5.9 se aplica a `tpcpatch`. El predicado se evalúa sobre el `tpcbox` del valor, cuya extensión en X y en Y por instante es el `PCBOUNDS` embebido en el parche y cuya extensión en Z es la que indican los puntos del parche.

- Operadores topológicos

```
{tstzspan,tpcbox,tpcpatch} {&&, @>, <@, ~=, -|-} {tstzspan,tpcbox,tpcpatch} → boolean
```

```
SELECT tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)])
  && tstzspan '[2001-01-01, 2001-01-03]';
-- true
```

- Operadores de posición

```
{tpcbox,tpcpatch} {<<, >>, <<|, |>>, <</, />>} {tpcbox,tpcpatch} → boolean
{tstzspan,tpcbox,tpcpatch} {<<#, #>>} {tstzspan,tpcbox,tpcpatch} → boolean
```

```
SELECT tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)])
  <<# tstzspan '[2001-01-05, 2001-01-06]';
-- true
```

17.6.11. Operaciones de distancia

- Devuelve la distancia más cercana

```
nearestApproachDistance(tpcpatch,{tpcbox,tpcpatch}) → float
```

```
SELECT round(nearestApproachDistance(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 8, 8, 8), pcpoint(1, 9, 9, 9)),
  '2001-01-01'::timestamptz))::numeric, 4);
-- 8.4853
```

17.6.12. Relaciones siempre y alguna vez

- Devuelve verdadero si y solo si al menos un punto de alguno de los instantes del tpcpatch intersecta la geometría (solo XY)

```
eIntersects(tpcpatch,geometry) → boolean
```

Un parche es un conjunto de puntos, por lo que ni contiene ni cubre una geometría, y dos parches no se tocan: `eContains`, `eCovers` y sus gemelas siempre toman primero la geometría, `eTouches` y `aTouches` leen cualquiera de los dos órdenes pero no dos parches, y `eDisjoint`, `eIntersects`, `eDwithin` y sus gemelas siempre leen los tres órdenes de argumentos. `eIntersects(tpcpatch,geometry)` es la que el tipo responde de forma nativa, recorriendo los puntos de cada instante y deteniéndose en el primer acierto; las demás convierten el parche a la geometría temporal de sus multipuntos.

```
SELECT eIntersects(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]), geometry 'POINT(1 1)');
-- t
```

17.6.13. Relaciones espaciotemporales

Las relaciones *temporales* calculan la relación topológica o de distancia en cada instante y dan como resultado un `tbool`. Un parche llega al motor de geometría a través de la conversión a `tgeometry` anterior, por lo que basta con que un punto del parche esté dentro del argumento para que el parche lo intersecte. Un tpcpatch lleva la Z que declara su esquema, y `tContains`, `tCovers` y `tTouches` son relaciones planas que rechazan una dimensión Z, de modo que el tipo responde las tres que un valor con Z puede responder.

- Relaciones espaciotemporales

```
tDisjoint({geometry,tpcpatch},{geometry,tpcpatch}) → tbool
tDwithin({geometry,tpcpatch},{geometry,tpcpatch},float) → tbool
tIntersects({geometry,tpcpatch},{geometry,tpcpatch}) → tbool
tContains(geometry,tpcpatch) → tbool
tCovers(geometry,tpcpatch) → tbool
tTouches(geometry,tpcpatch) → tbool
tTouches(tpcpatch,geometry) → tbool
```

Un parche llega al motor de geometría como el multipunto de las posiciones que ocupan sus puntos, y como conjunto de puntos declara la matriz de un punto temporal: `tContains` y `tCovers` toman primero la geometría, `tTouches` lee cualquiera de los dos órdenes pero no dos parches, y las demás leen los tres órdenes de argumentos. Las planares responden para un esquema cuyas dimensiones no incluyen Z, y un esquema que lleva Z encuentra el rechazo `The tgeometry cannot have Z dimension`.

```
SELECT tIntersects(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 9, 9, 9)),
  '2001-01-01'::timestamptz), geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-01
SELECT tDisjoint(tpcpatch(pcpatch(pcpoint(1, 9, 9, 9)), '2001-01-02'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-02
SELECT tDwithin(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01'::timestamptz),
  geometry 'Point(1 5)', 2.0);
-- f@2001-01-01
```

```
SELECT tIntersects(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 9, 9, 9)), '2001-01-02'::timestamptz)]),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- [t@2001-01-01, f@2001-01-02]
```

17.6.14. Comparaciones

- Comparaciones alguna vez y siempre

```
{pcpatch,tpcpatch} {?=, %=, ?<>, %<>} {pcpatch,tpcpatch} → boolean
```

```
SELECT pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)) ?= tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]);
-- t
SELECT tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz), tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]) %= pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2));
-- f
```

- Comparaciones temporales

```
{pcpatch,tpcpatch} {#=, #<>} {pcpatch,tpcpatch} → tbool
```

```
SELECT tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz), tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]) #= pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2));
-- [t@2001-01-01, f@2001-01-02]
```

17.7. Indexación

Las clases de operadores B-tree (`tpcpoint_btree_ops`, `tpcpatch_btree_ops`, `tpcbox_btree_ops`) y hash (`tpcpoint_hash_ops`, `tpcpatch_hash_ops`) admiten predicados de igualdad y ordenación. Las clases de operadores GiST R-tree y SP-GiST quadtree / kd-tree aceleran los predicados basados en la caja delimitadora (`&&`, `@>`, `<@`, `~=`, `-|-`) sobre los tres tipos `tpcbox`, `tpcpoint` y `tpcpatch`:

```
-- GiST (R-tree)
CREATE INDEX ON my_table USING gist(my_tpcpoint_column);
CREATE INDEX ON my_table USING gist(my_tpcpatch_column);
-- SP-GiST quadtree (default) or kd-tree
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column);
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column tpcpoint_kdtree_ops);
```

Las clases de operadores SP-GiST almacenan internamente un `stbox` (el filtro `pcid` se recupera mediante la revisión del operador), por lo que una consulta contra un índice sobre una columna cuyos valores mezclan múltiples `pcid` devolverá un conjunto ligeramente mayor de filas candidatas para revisión, algo irrelevante cuando cada tabla contiene valores de un único esquema, que es el caso común.

17.8. Agregaciones

- Agregado de caja delimitadora sobre una columna de valores tpcpoint, tpcpatch o tpcbox

```
extent({tpcpoint,tpcpatch,tpcbox}) → tpcbox
```

Devuelve el tpcbox más pequeño que contiene todas las entradas. Agregar a través de pcid distintos genera un error, la caja delimitadora sería ininterpretable ya que las dimensiones son relativas al pcid.

```
SELECT extent(v) FROM (VALUES (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
  (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
-- TPCBOX(ZT(((1,1,1),(2,2,2)),[2001-01-01, 2001-01-02]), 1)
```

- Conteo temporal y conteo por ventana de filas superpuestas en cada marca de tiempo

```
tCount({tpcpoint,tpcpatch}) → tint
wCount({tpcpoint,tpcpatch},interval) → tint
```

El resultado es un tint. Todas las filas de entrada deben compartir el mismo subtipo temporal (instante / secuencia / conjunto de secuencias).

```
SELECT tCount(v) FROM (VALUES (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
  (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
-- {1@2001-01-01, 1@2001-01-02}
SELECT wCount(v, interval '1 day') FROM (VALUES
  (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
  (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
-- {[1@2001-01-01, 2@2001-01-02], (1@2001-01-02, 1@2001-01-03]}
```

- Conteo de puntos de pcpatch por instante, sumado a través de las filas

```
tnpoints(tpcpatch) → tint
```

Distinto de `tCount(tpcpatch)`, que cuenta el número de parches (siempre 1 por instante). `tnpoints` se lee como «cuántos puntos había en la nube en el tiempo t», la primitiva natural para el control de calidad de la ingesta y los paneles de densidad a lo largo del tiempo.

```
SELECT tnpoints(v) FROM (VALUES (tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)),
  (tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz))) t(v);
-- {2@2001-01-01, 2@2001-01-02}
```

- Densidad de puntos por instante (`numPoints / área-bbox-xy`) sumada a través de las filas

```
tdensity(tpcpatch) → tfloat
```

Cada pcpatch lleva el `PCBOUNDS` en línea (xmin / xmax / ymin / ymax) por lo que el término de área de la caja delimitadora se lee directamente, sin recálculo. Un parche cuyos límites XY no tienen área, un solo punto o puntos sobre una línea, no tiene densidad: la división por cero genera un error. Una consulta sobre parches que pueden no tener área lee aquellos cuya caja tiene un área, `area(stbox(tpcbox(patch))) > 0`.

```
-- Each patch holds 2 points over a 1x1 xy-bbox.
SELECT tdensity(v) FROM (VALUES (tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)),
  (tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz))) t(v);
-- {2@2001-01-01, 2@2001-01-02}
```

- Combina múltiples valores temporales en uno solo con todos los instantes de entrada fusionados en orden temporal

```
merge(tpcpoint) → tpcpoint
merge(tpcpatch) → tpcpatch
```

Todas las entradas deben compartir el mismo pcid y subtipo.

```
SELECT numInstants(merge(v)) FROM (VALUES (tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz)),
  (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
-- 2
```

- Agregados de construcción en flujo: `appendInstant` ensambla instantes en una secuencia; `appendSequence` ensambla secuencias en un conjunto de secuencias

```
appendInstant({tpcpoint,tpcpatch}[,interp text[,maxt interval]]) → {tpcpoint,tpcpatch}
appendSequence(tpcpoint) → tpcpoint
appendSequence(tpcpatch) → tpcpatch
```

Útil para cargadores de trayectorias que consumen filas en orden temporal.

```
SELECT numInstants(appendInstant(v ORDER BY startTimestamp(v))) FROM (VALUES
  (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
  (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
-- 2
```

Capítulo 18

Escritura de consultas portables

MobilityDB se usa desde varios motores SQL: PostgreSQL mediante MobilityDB mismo, DuckDB mediante MobilityDuck, Apache Spark mediante MobilitySpark y Apache Flink mediante MobilityFlink. Los tipos y las funciones son los mismos en todos ellos, pero los operadores no: PostgreSQL permite a MobilityDB definir operadores como `&&` o `<<#`, mientras que los otros motores aceptan solo algunos de ellos o ninguno. DuckDB, por ejemplo, reserva `#` para las referencias posicionales a columnas, por lo que no acepta ningún operador que lo contenga.

Por esta razón todo operador es también una función, que ejecuta el mismo código y devuelve exactamente el mismo resultado, y una consulta escrita solo con funciones se ejecuta sin cambios en todos los motores. La superposición de cajas delimitadoras `t1.trip && t2.trip`, por ejemplo, se escribe `stboxOverlaps(t1.trip, t2.trip)`, y la prueba de posición temporal `p1.period <<# p2.period` se escribe `spanBefore(p1.period, p2.period)`. Las secciones siguientes dan la función de cada operador, agrupadas como las agrupan los capítulos de cada tipo.

El nombre de una función dice sobre qué opera. Una operación aritmética o de conjuntos toma el prefijo de la familia de su argumento: `set`, `span` o `spanset`, y `t` para un valor temporal, como en `spanUnion` o `tAdd`. Una prueba de posición toma el prefijo de lo que compara: `set`, `span` o `spanset`, o, para una caja o un valor temporal, el tipo de su caja delimitadora, `tbox`, `stbox` o `tpcbox`; un valor temporal booleano, de texto o JSONB se compara mediante el intervalo de tiempo que lo delimita, y toma por ello `span`. Estos nombres están además libres en todos los motores: `left` y `right` son funciones de Spark y Flink, por lo que las pruebas de posición se llaman `spanLeft` o `stboxLeft`, y una comparación «alguna vez» o «siempre» escribe su operación completa, como en `eLessThan`, porque `eLt` se leería como la función `elt` de Spark y Flink, que ignoran las mayúsculas de un nombre.

Algunas operaciones no tienen operador y se escriben de la misma manera en todas partes: las relaciones espaciales (`eIntersects`, `aIntersects`, `eTouches`, `eDwithin`, ...), las restricciones (`atTime`, `atGeometry`, `atValue`, ...) y la comparación de orden `cmp(a, b)`, que devuelve `-1`, `0` o `1`.

Spark y Flink piden algunos cambios más además de las funciones, que describe Sección 18.10.

18.1. Operaciones de conjuntos

La unión, la intersección y la diferencia de dos conjuntos, rangos o conjuntos de rangos, y la concatenación de dos conjuntos o de dos valores temporales.

Categoría	Operador	Función portable
Unión (set / span / span set)	+	setUnion / spanUnion / spansetUnion (a, b)
Intersección (set / span / span set)	*	setIntersection / spanIntersection / spansetIntersection (a, b)
Diferencia (set / span / span set)	-	setMinus / spanMinus / spansetMinus (a, b)
Concatenación (set / temporal)	||	setConcat / tConcat (a, b)

18.2. Operaciones de cuadro delimitador

Estas operaciones comparan las cajas delimitadoras de sus argumentos: el rango de un conjunto o de un conjunto de rangos, o la caja de un valor temporal, `tbox`, `stbox` o `tpcbox`.

18.2.1. Operaciones topológicas

Si dos cajas delimitadoras se superponen, se contienen una a otra, son adyacentes o son la misma. Las últimas filas prueban el intervalo de tiempo de un valor JSONB temporal y los identificadores de ruta de los puntos de red.

Categoría	Operador	Función portable
Cuadro delimitador	&&	setOverlaps / spanOverlaps / spansetOverlaps / tboxOverlaps / stboxOverlaps / tpcboxOverlaps (a, b)
	@>	setContains / spanContains / spansetContains / tboxContains / stboxContains / tpcboxContains (a, b)
	<@	setContained / spanContained / spansetContained / tboxContained / stboxContained / tpcboxContained (a, b)
	-|-	spanAdjacent / spansetAdjacent / tboxAdjacent / stboxAdjacent / tpcboxAdjacent (a, b)
	~=	spanSame / tboxSame / stboxSame / tpcboxSame (a, b)
Rango de tiempo de un valor JSONB temporal	#@>	tjsonbContains(a, b)
	<@#	tjsonbContained(a, b)
Identificador de ruta (punto de red)	@=	sameRid(a, b)
	@?	containsRid(a, b)
	?@	containedRid(a, b)
	@@	overlapsRid(a, b)

18.2.2. Operaciones de posición

Si una caja delimitadora está estrictamente antes, después, a la izquierda, a la derecha, debajo, encima, delante o detrás de otra y, en las formas que empiezan por `over`, si no se extiende más allá de ella por ese lado.

Categoría	Operador	Función portable
Posición temporal	<<#	setBefore / spanBefore / spansetBefore / tboxBefore / stboxBefore / tpcboxBefore (a, b)
	#>>	setAfter / spanAfter / spansetAfter / tboxAfter / stboxAfter / tpcboxAfter (a, b)
	&<#	setOverbefore / spanOverbefore / spansetOverbefore / tboxOverbefore / stboxOverbefore / tpcboxOverbefore (a, b)
	#&>	setOverafter / spanOverafter / spansetOverafter / tboxOverafter / stboxOverafter / tpcboxOverafter (a, b)
Espacio, eje X	<<	setLeft / spanLeft / spansetLeft / tboxLeft / stboxLeft / tpcboxLeft (a, b)
	>>	setRight / spanRight / spansetRight / tboxRight / stboxRight / tpcboxRight (a, b)
	&<	setOverleft / spanOverleft / spansetOverleft / tboxOverleft / stboxOverleft / tpcboxOverleft (a, b)
	&>	setOverright / spanOverright / spansetOverright / tboxOverright / stboxOverright / tpcboxOverright (a, b)
Espacio, eje Y	<<|	stboxBelow / tpcboxBelow (a, b)
	|>>	stboxAbove / tpcboxAbove (a, b)
	&<|	stboxOverbelow / tpcboxOverbelow (a, b)
	|&>	stboxOverabove / tpcboxOverabove (a, b)

Categoría	Operador	Función portable
Espacio, eje Z	<</	stboxFront / tpcboxFront (a, b)
	/>>	stboxBack / tpcboxBack (a, b)
	&</	stboxOverfront / tpcboxOverfront (a, b)
	/&>	stboxOverback / tpcboxOverback (a, b)

18.3. Operaciones de distancia

Los operandos de dimensiones distintas se leen en las dimensiones que ambos tienen, véase Sección 8.3.

`tDistance` devuelve la distancia entre dos valores en cada instante, como un flotante temporal, y `nearestApproachDistance` la menor de estas distancias. `distance` y sus formas de conjunto, rango y conjunto de rangos devuelven la distancia entre dos valores, y `th3GridDistance` la distancia en la cuadrícula entre dos celdas H3 temporales en cada instante, como un entero grande temporal.

`tquadbinGridDistance` y `ts2GridDistance` devuelven del mismo modo la distancia en la cuadrícula entre dos celdas QUADBIN o S2 temporales.

Categoría	Operador	Función portable
Distancia	<->	tDistance(a, b)
	|=|	nearestApproachDistance(a, b)
Distancia (base / set / span / span set)	<->	distance / setDistance / spanDistance / spansetDistance (a, b)
Distancia de rejilla (índice de celda)	<->	th3GridDistance(a, b)
	<->	tquadbinGridDistance(a, b)
	<->	ts2GridDistance(a, b)

18.4. Comparaciones

18.4.1. Comparaciones tradicionales

Las comparaciones de dos valores del mismo tipo. Con `cmp` ordenan valores, como lo hace un `ORDER BY`.

Operador	Función portable
=	eq(a, b)
<>	ne(a, b)
<	lt(a, b)
<=	le(a, b)
>	gt(a, b)
>=	ge(a, b)

18.4.2. Comparaciones alguna vez y siempre

Si un valor temporal es alguna vez, o siempre, igual a, distinto de, menor que o mayor que un valor u otro valor temporal.

Categoría	Operador	Función portable
Comparación alguna vez	?=	eEqual(a, b)
	?<>	eNotEqual(a, b)
	?<	eLessThan(a, b)
	?<=	eLessEqual(a, b)
	?>	eGreaterThan(a, b)

Categoría	Operador	Función portable
	?>=	eGreaterEqual(a, b)
Comparación siempre	%=	aEqual(a, b)
	%<>	aNotEqual(a, b)
	%<	aLessThan(a, b)
	%<=	aLessEqual(a, b)
	%>	aGreaterThan(a, b)
	%>=	aGreaterEqual(a, b)

18.4.3. Comparaciones temporales

La comparación de dos valores en cada instante, que devuelve un booleano temporal.

Operador	Función portable
#=	tEqual(a, b)
#<>	tNotEqual(a, b)
#<	tLessThan(a, b)
#<=	tLessEqual(a, b)
#>	tGreaterThan(a, b)
#>=	tGreaterEqual(a, b)

18.5. Operaciones booleanas

La conjunción, la disyunción y la negación de booleanos temporales en cada instante.

Operador	Función portable
&	tAnd(a, b)
|	tOr(a, b)
~	tNot(a)

18.6. Operaciones matemáticas

La aritmética de números temporales en cada instante, y la de rásteres píxel a píxel.

Operador	Función portable
+	tAdd / rasterAdd (a, b)
-	tSub / rasterSub (a, b)
*	tMul / rasterMul (a, b)
/	tDiv / rasterDiv (a, b)

18.7. Operaciones JSON temporales

El acceso a los campos y elementos de los conjuntos JSONB y de los valores JSONB temporales, la existencia de claves, las consultas de rutas JSON y el borrado de una ruta. Cada función toma el prefijo del tipo de su primer argumento, `jsonbset` o `tjsonb`.

Categoría	Operador	Función portable
Acceso a campo / elemento JSON	->	jsonbsetObjectField / tjsonbObjectField / jsonbsetArrayElement / tjsonbArrayElement (a, b)

Categoría	Operador	Función portable
	->>	jsonbsetObjectFieldText / tjsonbObjectFieldText / jsonbsetArrayElementText / tjsonbArrayElementText (a, b)
Existencia de clave JSON	?	jsonbsetExists / tjsonbExists (a, b)
	?&	jsonbsetExistsAll / tjsonbExistsAll (a, b)
	?|	jsonbsetExistsAny / tjsonbExistsAny (a, b)
Ruta JSON	@?	jsonbsetPathExists / tjsonbPathExists (a, b)
	@@	jsonbsetPathMatch / tjsonbPathMatch (a, b)
	#-	jsonbsetDeletePath / tjsonbDeletePath (a, b)

18.8. Valores base provistos por el anfitrión

Algunos tipos base de MobilityDB provienen de PostgreSQL mismo o de una extensión que MobilityDB requiere: `jsonb` y `jsonpath` de PostgreSQL, `h3index` de la extensión `h3`, y `pcpoint` y `pcpatch` de la extensión `pointcloud`. En PostgreSQL sus operadores y funciones conservan los nombres que les da quien los provee, y todos los motores, PostgreSQL incluido, los llaman también con los nombres de la tabla siguiente.

Sus valores se comparan con las funciones de Sección 18.4.1 y con `cmp`, y se resumen con `hash` y `hashExtended`, que Spark y Flink llaman `jsonbHash`, `jsonbHashExtended`, `h3indexHash` y `h3indexHashExtended`, pues estos motores nombran la función de resumen de cada tipo según el tipo. Un valor se lee de su texto con `jsonbFromText`, `jsonpathFromText` y `h3indexFromText`, y se escribe como texto con `asText`; en PostgreSQL un literal y una conversión desde o hacia `text` hacen lo mismo. Un `pcpoint` o un `pcpatch` se escribe como la forma hexadecimal de su Well-Known Binary, se lee con `pcpointFromHexWKB` y `pcpatchFromHexWKB` y se escribe con `asHexWKB`.

PostgreSQL	Función portable
jsonb '...', text::jsonb	jsonbFromText(a)
jsonb::text	asText(a)
jsonpath '...', text::jsonpath	jsonpathFromText(a)
jsonpath::text	asText(a)
h3index '...', text::h3index	h3indexFromText(a)
h3index::text	asText(a)
pcpoint '...', text::pcpoint	pcpointFromHexWKB(a)
pcpoint::text	asHexWKB(a)
pcpatch '...', text::pcpatch	pcpatchFromHexWKB(a)
pcpatch::text	asHexWKB(a)
jsonb -> text	jsonbObjectField(a, b)
jsonb -> integer	jsonbArrayElement(a, b)
jsonb ->> text	jsonbObjectFieldText(a, b)
jsonb ->> integer	jsonbArrayElementText(a, b)
jsonb #> text[], jsonb_extract_path	jsonbExtractPath(a, b)
jsonb #>> text[], jsonb_extract_path_text	jsonbExtractPathText(a, b)
jsonb ? text	jsonbExists(a, b)
jsonb ?| text[]	jsonbExistsAny(a, b)
jsonb ?& text[]	jsonbExistsAll(a, b)
jsonb @> jsonb	jsonbContains(a, b)
jsonb <@ jsonb	jsonbContained(a, b)
jsonb || jsonb	jsonbConcat(a, b)
jsonb - text	jsonbDelete(a, b)
jsonb - text[]	jsonbDeleteArray(a, b)
jsonb - integer	jsonbDeleteIndex(a, b)
jsonb #- text[]	jsonbDeletePath(a, b)
jsonb @? jsonpath, jsonb_path_exists	jsonbPathExists(a, b)
jsonb_path_exists_tz	jsonbPathExistsTz(a, b)
jsonb @@ jsonpath, jsonb_path_match	jsonbPathMatch(a, b)
jsonb_path_match_tz	jsonbPathMatchTz(a, b)

PostgreSQL	Función portable
jsonb_path_query	jsonbPathQuery(a, b)
jsonb_path_query_tz	jsonbPathQueryTz(a, b)
jsonb_path_query_array	jsonbPathQueryArray(a, b)
jsonb_path_query_array_tz	jsonbPathQueryArrayTz(a, b)
jsonb_path_query_first	jsonbPathQueryFirst(a, b)
jsonb_path_query_first_tz	jsonbPathQueryFirstTz(a, b)
jsonb_array_length	jsonbArrayLength(a)
jsonb_array_elements	jsonbArrayElements(a)
jsonb_array_elements_text	jsonbArrayElementsText(a)
jsonb_each	jsonbEach(a)
jsonb_each_text	jsonbEachText(a)
jsonb_object_keys	jsonbObjectKeys(a)
jsonb_insert	jsonbInsert(a, b, c)
jsonb_set	jsonbSet(a, b, c)
jsonb_set_lax	jsonbSetLax(a, b, c)
jsonb_pretty	jsonbPretty(a)
jsonb_strip_nulls	jsonbStripNulls(a)
PC_NumPoints(pcpatch)	numPoints(a)
PC_PointN(pcpatch, integer)	pointN(a, b)

18.9. Agregaciones

Todo agregado de MobilityDB, como `tCount`, `tSum` o `extent`, se llama con el mismo nombre en todos los motores, con una excepción. Once agregados comparten su nombre con una función: `tMin`, `tMax`, `merge`, `appendInstant`, `appendSequence`, `sameAlignment`, `tAnd`, `tOr`, `setUnion`, `spanUnion` y `spansetUnion`. PostgreSQL permite que un agregado y una función compartan un nombre, y MobilityDB llama a estos agregados por él. DuckDB, Apache Spark y Apache Flink tienen una sola función por nombre, y los llaman con el nombre seguido de `Agg`: `tMinAgg`, `mergeAgg`, `setUnionAgg`, etc. (Sección 18.10.1).

Un agregado calcula su resultado en hasta tres pasos, cada uno una función nombrada según el agregado y el paso. La función de transición `<aggregate>Transition` añade un valor a un estado en curso, la función de combinación `<aggregate>Combine` une dos estados calculados en paralelo, y la función final `<aggregate>Final` convierte el estado en el resultado. Los pasos de combinación y final son opcionales: `extent` no tiene función final, pues su estado es ya su resultado, y los agregados de adición no tienen función de combinación, pues añaden sus valores en orden temporal. Una consulta nunca llama a estas funciones; cada motor construye el agregado a partir de ellas.

La función de transición toma el estado y un valor, y tiene una versión por tipo de valor, como el agregado: `tMinTransition` toma un `tint`, un `tbigint`, un `tfloat` o un `ttext`. Las funciones de combinación y final toman solo estados, y un estado conoce el tipo de los valores que contiene, por lo que cada una es una sola función por agregado: `tMinCombine` combina los estados del mínimo temporal de todos los tipos de valor. En PostgreSQL, donde una función final declara el tipo que devuelve, la función final se declara una vez por tipo de valor con `FINALFUNC_EXTRA`, como `tMinFinal(internal, tint)` que devuelve un `tint`, y todas sus versiones ejecutan el mismo código. En Spark y Flink cada agregado es un solo objeto, que elige su función de transición según el tipo del valor.

La tabla siguiente lista todos los agregados y sus funciones; un guion (—) marca un paso que el agregado no tiene.

Categoría	Transición	Combinación	Final
Conteo	tCountTransition	tCountCombine	tCountFinal
	wCountTransition	wCountCombine	wCountFinal
Suma	tSumTransition	tSumCombine	tSumFinal
	wSumTransition	wSumCombine	wSumFinal
Promedio	tAvgTransition	tAvgCombine	tAvgFinal
	wAvgTransition	wAvgCombine	wAvgFinal
Mínimo / máximo	tMinTransition	tMinCombine	tMinFinal
	tMaxTransition	tMaxCombine	tMaxFinal

Categoría	Transición	Combinación	Final
	wMinTransition	wMinCombine	wMinFinal
	wMaxTransition	wMaxCombine	wMaxFinal
Booleano	tAndTransition	tAndCombine	tAndFinal
	tOrTransition	tOrCombine	tOrFinal
Centroide	tCentroidTransition	tCentroidCombine	tCentroidFinal
Nube de puntos	tnpointsTransition	tnpointsCombine	tnpointsFinal
	tdensityTransition	tdensityCombine	tdensityFinal
Caja delimitadora	extentTransition	extentCombine	—
Unión	setUnionTransition	setUnionCombine	setUnionFinal
	spanUnionTransition	spanUnionCombine	spanUnionFinal
	spansetUnionTransition	spansetUnionCombine	spansetUnionFinal
Fusión	mergeTransition	mergeCombine	mergeFinal
Anexado	appendInstantTransition	—	appendInstantFinal
	appendSequenceTransition	—	appendSequenceFinal
Ráster	rasterUnionTransition	rasterUnionCombine	rasterUnionFinal
	sameAlignmentTransition	sameAlignmentCombine	sameAlignmentFinal
	summaryStatsAggTransition	summaryStatsAggCombine	summaryStatsAggFinal

18.10. Ejecución de una consulta en Apache Spark y Apache Flink

Escrita solo con funciones, una consulta ya no depende de los operadores. Spark y Flink difieren de PostgreSQL en otras tres cosas, y cada una pide un cambio. Ya definen algunos de los nombres que MobilityDB usa: ambos tienen `round`, `lower` y `length`, y Flink rechaza `set` y `unnest` sin comillas; tal nombre toma un prefijo que nombra la clase o el tipo base de su argumento (Sección 18.10.1). Nunca llaman por usted a la función de entrada o de salida de un tipo, por lo que un literal, una conversión y el texto de un valor se convierten en llamadas a funciones que la consulta escribe (Sección 18.10.2). Y cada motor llama a su manera a una función que devuelve filas (Sección 18.10.3). La tabla siguiente resume estos cambios; una función que no lista conserva su nombre.

Lo que contiene la consulta	MobilityDB	Spark y Flink
Un operador: su función con nombre, como la dan las secciones anteriores	trip && box	stboxOverlaps(trip, box)
Un nombre que el motor ya posee, sobre un valor cuyo tipo base tiene la operación: el tipo base delante	round(speed(trip), 2)	floatRound(speed(trip), 2)
Un nombre que el motor ya posee, sobre otro valor conjunto, rango, conjunto de rangos o temporal: la clase delante	lower(timeSpan(trip))	spanLower(timeSpan(trip))
Un nombre que el motor ya posee, sobre un cuadro delimitador o un tipo base: el tipo delante	round(box, 2)	stboxRound(box, 2)
Un literal con tipo: el constructor de texto de su tipo	tint '1@2001-01-01'	tintFromText('1@2001-01-01')
El texto de un valor, que PostgreSQL escribe mediante la función de salida de su tipo: asText	SELECT speed(trip)	SELECT asText(speed(trip))
Una conversión: la función que su entrada da a su lado, como tbool::tint es tint(tbool)	flag::tint	tint(flag)

18.10.1. Los nombres que poseen los motores

Un nombre que el motor ya define toma el prefijo de la clase o del tipo base de su argumento, y las funciones que van juntas toman el mismo prefijo: `spanLowerInc` va con `spanLower`, `geoCumulativeLength` con `geoLength`. Spark y Flink usan los mismos nombres, por lo que una consulta escrita para uno se ejecuta en el otro, y `round(1.23456, 2)` sigue significando el redondeo propio del motor de un número. Los nombres que cambian son los siguientes.

Nombre en MobilityDB	Clase del argumento	Nombre en Spark y Flink
lower, upper, initcap	textset, ttext	textLower, textUpper, textInitcap
lower, upper, lowerInc, upperInc	rango, conjunto de rangos	spanLower, spanUpper, spanLowerInc, spanUpperInc, spansetLower, spansetUpper, spansetLowerInc, spansetUpperInc
lowerInc, upperInc	temporal	temporalLowerInc, temporalUpperInc
round	float y sus tipos conjunto, rango, conjunto de rangos y temporal	floatRound
	geometría y geografía, sus conjuntos y tipos temporales, trgeometry	geoRound
	cbuffer, cbufferset, tcbuffer; npoint, npointset, tnpoint; nsegment; pose, poseset, tpose; posechain, posechainset, tposechain; tbox; stbox; tpcbox	cbufferRound, npointRound, nsegmentRound, poseRound, posechainRound, tboxRound, stboxRound, tpcboxRound
abs	tint, tbigint, tfloat	intAbs, bigintAbs, floatAbs
ceil, floor, degrees, radians, cos, sin, tan, exp, ln, log10	float y sus tipos conjunto, rango, conjunto de rangos y temporal	floatCeil, floatFloor, floatDegrees, floatRadians, floatCos, floatSin, floatTan, floatExp, floatLn, floatLog10
transform	los conjuntos y tipos temporales de geometría y geografía, trgeometry	geoTransform
transformPipeline	geometría y geografía, sus conjuntos y tipos temporales, trgeometry	geoTransformPipeline
	cbuffer, cbufferset, tcbuffer; pose, poseset, tpose; posechain, posechainset, tposechain; stbox; raster	cbufferTransform, poseTransform, posechainTransform, stboxTransform, rasterTransform, y el . . . TransformPipeline de cada uno
translate, affine, rotate, rotateX, rotateY, rotateZ, scale, transscale	geometry, tgeompoint, tgeometry	geoTranslate, geoAffine, geoRotate, geoRotateX, geoRotateY, geoRotateZ, geoScale, geoTransscale
translate, rotate, rotateX, rotateY, rotateZ	tcbuffer; tpose; tposechain	cbufferTranslate, cbufferRotate, cbufferRotateZ; poseTranslate, poseRotate, poseRotateX, poseRotateY, poseRotateZ; lo mismo para posechain (posechainTranslate, . . .)
	trgeometry	geoTranslate, geoRotate, geoRotateX, geoRotateY, geoRotateZ
width, height, bandPixelType, bandHasNoDataValue, bandNoDataValue, pixels, asHexWKB, stbox	raster; raquet	rasterWidth, rasterHeight, rasterBandPixelType, rasterBandHasNoDataValue, rasterBandNoDataValue, rasterPixels, rasterAsHexWKB, rasterStbox; raquetWidth, raquetHeight, raquetBandPixelType, raquetBandHasNoDataValue, raquetBandNoDataValue, raquetPixels, raquetAsHexWKB, raquetStbox
length, cumulativeLength	geometría, geografía, tgeompoint, tgeogpoint, trgeometry	geoLength, geoCumulativeLength
	tnpoint	npointLength, npointCumulativeLength
contains, covers, disjoint, intersects, touches, dwithin, relate	geometría, geografía	geoContains, geoCovers, geoDisjoint, geoIntersects, geoTouches, geoDwithin, geoRelate
reverse	geometría	geoReverse
collect	geometría[]	geoCollect

Nombre en MobilityDB	Clase del argumento	Nombre en Spark y Flink
transform	geometría, geografía	geoTransform
hash, hashExtended	conjunto, rango, conjunto de rangos, cuadro delimitador, temporal, tipo base	setHash, spanHash, spansetHash, tboxHash, stboxHash, temporalHash, cbufferHash, . . . , y sus . . . HashExtended
insert, update, merge	temporal	temporalInsert, temporalUpdate, temporalMerge
tMin, tMax, merge, appendInstant, appendSequence, tAnd, tOr (los agregados)	temporal	tMinAgg, tMaxAgg, mergeAgg, appendInstantAgg, appendSequenceAgg, tAndAgg, tOrAgg
setUnion, spanUnion, spansetUnion (los agregados)	valor base, conjunto, rango, conjunto de rangos	setUnionAgg, spanUnionAgg, spansetUnionAgg
sameAlignment (el agregado)	raster	sameAlignmentAgg
wAvg (el agregado)	tint, tbigint, tfloat	wAvgAgg
jsonbSet, jsonbSetLax	jsonb	pgJsonbSet, pgJsonbSetLax
unnest	conjunto, temporal	setUnnest, temporalUnnest
set (el constructor)	cualquiera	setMake

Las funciones `jsonbSet` y `jsonbSetLax` toman el prefijo `pg` de PostgreSQL, que las proporciona, ya que Spark y Flink, que ignoran las mayúsculas de un nombre, leerían `jsonbSet` como `jsonbset`, la conversión de un textset en un jsonbset.

Los nombres que comparten un raster y un raquet también toman el prefijo de su tipo, aunque el motor no defina ese nombre: ambos viajan como WKB, por lo que un motor con una sola función por nombre no distingue un raster dado a `width` de un raquet.

18.10.2. Lectura y escritura de valores

PostgreSQL lee y escribe un valor mediante las cuatro funciones que declara su tipo, y las llama por usted: un literal tipado, una conversión desde texto y un `COPY` de texto pasan por la función de entrada, el texto que muestra un cliente por la función de salida, y un `COPY` binario o un controlador en modo binario por las funciones de recepción y envío. Spark y Flink no llaman a ninguna de ellas. Un valor viaja entre ellos como sus bytes Well-Known Binary (WKB), y cada conversión es una función que la consulta llama, cada una con su inversa, como muestra la tabla siguiente.

Lo que usa PostgreSQL	Dirección	Spark y Flink
la función de entrada: un literal con tipo, una conversión desde texto	texto a valor	ttypeFromText(text), settypeFromText(text), boxFromText(text), . . . , una por tipo, con el nombre del tipo como prefijo; con un SRID también tspatialFromEWKT(text)
la función de salida: el texto que imprime un cliente	valor a texto	asText(value); con un SRID también asEWKT(value)
la función de recepción: COPY binario, un controlador binario	binario a valor	ttypeFromBinary(bytea), ttypeFromHexWKB(text), y lo mismo para todo otro tipo; con un SRID también tspatialFromEWKB(bytea), tspatialFromHexEWKB(text)
la función de envío	valor a binario	asBinary(value), asHexWKB(value); con un SRID también asEWKB(value), asHexEWKB(value)
ninguna	MF-JSON	ttypeFromMFJSON(text), asMFJSON(ttype)

Cada tipo tiene las funciones de la tabla que se le aplican: las formas `E` cuando lleva un SRID que sus formas de texto y binaria simples omiten, y MF-JSON cuando es temporal. Esto incluye los tipos que PostgreSQL lee y escribe solo mediante sus cuatro funciones: los tipos de conjunto, rango y conjunto de rangos de enteros, enteros grandes, flotantes, fechas y marcas de tiempo, textset y jsonbset, tbox y stbox, tbool, tint, tbigint, tfloat y ttext, y quadbin, s2cell, nsegment y tpcbox.

El texto de un raquet es su WKB hexadecimal, por lo que se lee con `raquetFromHexWKB` y se escribe con `asHexWKB`, como un ráster de PostGIS con `ST_RastFromHexWKB` y `ST_AsHexWKB`. Una caja, stbox o tpcbox, escribe su SRID tanto en su texto como en su WKB, y no tiene por ello forma `E`. Toda salida WKB toma el orden de bytes `endian`, y toda salida

de texto de un valor con flotantes o coordenadas el número de dígitos decimales `maxdecimaldigits`. Una columna de bytes WKB, como una columna TemporalParquet, se lee en todos los motores con el constructor binario de su tipo, como en `tgeompointFromBinary(col)`.

18.10.3. Funciones que devuelven filas

Una función que devuelve varias filas, como `timeSplit` o `unnest`, se llama en PostgreSQL como una tabla en la cláusula `FROM`. En Spark y Flink la misma función, `unnest` bajo los nombres `setUnnest` y `temporalUnnest`, devuelve un arreglo con un elemento por fila, un arreglo de registros cuando devuelve varias columnas. Flink convierte el arreglo en filas con `CROSS JOIN UNNEST`, y Spark con `LATERAL VIEW explode`, o `inline` para un arreglo de registros.

18.10.4. Ejemplo

La consulta

```
SELECT vehicle, round(length(trip)::numeric, 1), lower(timeSpan(trip))
FROM Trips
WHERE trip && stbox 'SRID=3857;STBOX X((0,0),(1000,1000))';
```

se escribe, en Spark y Flink,

```
SELECT vehicle, round(geoLength(trip), 1), spanLower(timeSpan(trip))
FROM Trips
WHERE stboxOverlaps(trip, stboxFromText('SRID=3857;STBOX X((0,0),(1000,1000))'));
```

Aquí `round` se aplica a un número, por lo que es la función propia del motor y conserva su nombre.

18.11. Los rásteres de los motores

Un `raster` de MobilityDB es el mismo tipo, con las mismas funciones, en todos los motores (Sección 16.24). Dos motores tienen además un ráster propio: DuckDB mediante su extensión de rásteres, y Apache Spark mediante Apache Sedona; Apache Flink no tiene ninguno. Cada anfitrión nombra todo lo que añade con un prefijo, `RT_` en DuckDB y `RS_` en Sedona, y MobilityDB no nombra nada con ninguno de los dos, de modo que el nombre de una función indica sobre qué ráster trabaja: `RT_Read` y `RS_FromGeoTiff` son del anfitrión, `summaryStats` y `rasterValue` de MobilityDB. Una consulta calcula con el ráster de MobilityDB y convierte un valor en la frontera solo donde necesita al anfitrión: para leer un archivo ráster con el lector del anfitrión, o para pasar un ráster a una de las funciones del anfitrión. La conversión conserva cada valor de píxel y la georreferenciación, de modo que un ráster convertido al ráster del anfitrión y de vuelta es el ráster que era.

Lo que hace la consulta	DuckDB con su extensión de rásteres	Spark con Apache Sedona
Disponer del ráster del anfitrión	LOAD raster	SedonaContext.create(spark), junto al registro de MobilitySpark
Leer un archivo ráster	RT_Read(file), y luego raster(databand_1, tile_x, tile_y, metadata) para cada tesela	RS_FromGeoTiff(content), y luego la conversión de Sección 18.11.2
Pasar un ráster a las funciones del anfitrión	CAST(rast AS RT_DATACUBE) para sus píxeles, rasterMetadata(rast) para su georreferenciación	RS_AddBandFromArray sobre RS_MakeEmptyRaster, a partir de sus accesores
Pasar las estadísticas de una banda al anfitrión	CAST(stats AS RT_STATS), y de vuelta CAST(stats AS summarystats)	ninguna

Los dos anfitriones admiten tipos de píxel distintos. Cada tipo de píxel de MobilityDB se convierte en el tipo de cada anfitrión de su fila, y un ráster de un tipo para el que un anfitrión no tiene ninguno no tiene ráster en ese anfitrión.

Tipo de píxel del ráster	Datacube de DuckDB	Tipo de píxel de Sedona
1BB, 2BUI, 4BUI	UINT8	ninguno
8BUI	UINT8	B
8BSI	INT8	ninguno
16BUI	UINT16	US
16BSI	INT16	S
32BUI	UINT32	ninguno
32BSI	INT32	I
32BF	FLOAT	F
64BF	DOUBLE	D

18.11.1. DuckDB y su extensión de rásteres

La extensión de rásteres de DuckDB, una extensión de la comunidad, lee un archivo ráster con `RT_Read` como una tabla con una fila por tesela: los píxeles de cada banda en una columna `databand_N` de tipo `RT_DATACUBE`, la georreferenciación de todo el archivo en una columna JSON `metadata`, y la posición de la tesela en `tile_x` y `tile_y`. No tiene un valor ráster propio, solo estas columnas. Un ráster de MobilityDB se construye a partir de las cuatro columnas de una tesela, y devuelve un datacube y una columna de metadatos, las dos columnas a partir de las cuales las propias funciones de la extensión leen una tesela. Los metadatos que escribe `rasterMetadata` son los de un archivo que contiene el ráster como su única tesela, de modo que el ráster se reconstruye a partir de ellos en la tesela (0, 0).

El tipo `RT_DATACUBE` solo existe una vez cargada la extensión, de modo que una consulta que lo nombra se ejecuta después de `LOAD raster`; antes, DuckDB responde que no existe ningún tipo `RT_DATACUBE`. Las conversiones son conversiones de tipo y funciones explícitas: un ráster nunca se convierte por sí solo en un datacube al pasarlo a una función `RT_`, puesto que un datacube deja atrás la georreferenciación.

Desde	Hacia	SQL
una tesela de RT_Read, o cualquier datacube con sus metadatos	raster	raster(databand_1, tile_x, tile_y, metadata)
raster	RT_DATACUBE	CAST(rast AS RT_DATACUBE)
raster	los metadatos de una tesela	rasterMetadata(rast)
summarystats	RT_STATS	CAST(stats AS RT_STATS)
RT_STATS	summarystats	CAST(stats AS summarystats)

Un datacube tiene un solo tipo de píxel para todas sus bandas, y cada tipo de píxel de MobilityDB pasa a ser el tipo de datacube que GDAL le asigna, como ocurre con un ráster de PostGIS escrito mediante GDAL. Los tipos de 1, 2 y 4 bits no tienen un tipo más pequeño en un datacube: sus valores se conservan exactamente en 8 bits, y un ráster reconstruido a partir del datacube tiene el tipo `8BUI`.

Una conversión conserva todos los valores o produce un error; nunca responde con una aproximación. Un datacube tiene un solo tipo de píxel y un solo valor nodata, de modo que un ráster cuyas bandas difieren en alguno de ellos no puede convertirse a `RT_DATACUBE`. Un datacube de tipo `UINT64` o `INT64` no tiene un tipo de píxel de MobilityDB al que convertirse. Un datacube o una columna de metadatos mal formados, truncados, con un tipo o un formato desconocido, o que declaran más píxeles de los que contienen, se rechazan en lugar de leerse. El sistema de referencia de coordenadas de los metadatos se lee como `EPSG:`*n* y da el SRID del ráster; un ráster combinado con un valor de otro SRID produce el error que produce cualquier mezcla de SRID. Un ráster convertido a datacube escribe sus píxeles sin comprimir; un datacube se lee sea cual sea la compresión con que la extensión lo escribió, `RAW`, `SNAPPY`, `GZIP`, `ZSTD` o `LZ4`.

```
LOAD raster;
-- The tiles of 2 x 2 pixels of a 4 x 4 GeoTIFF of elevations, as MobilityDB rasters
CREATE TABLE elevation AS
SELECT tile_x, tile_y, raster(databand_1, tile_x, tile_y, metadata) AS rast
FROM RT_Read('elevation.tif', blocksize_x := 2, blocksize_y := 2);
-- MobilityDB's raster functions over them
SELECT count(*), summarystatsMean(summaryStatsAgg(rast, true)) FROM elevation;
-- 4 | 57.06666666666667
-- A MobilityDB raster handed to a function of the extension
```

```
SELECT tile_x, tile_y, RT_CubeStats(CAST(rast AS RT_DATACUBE), 0).mean
FROM elevation ORDER BY tile_y, tile_x;
-- 0 | 0 | 53.5
-- 1 | 0 | 68.5
-- 0 | 1 | 46.0
-- 1 | 1 | 61.333333333333336
-- A raster converted to the extension's columns and back is the raster it was
SELECT bool_and(raster(CAST(rast AS RT_DATACUBE), 0, 0, rasterMetadata(rast)) = rast)
FROM elevation;
-- true
```

18.11.2. Spark y Apache Sedona

Apache Sedona registra su tipo ráster y sus funciones `RS_` en una sesión de Spark mediante `SedonaContext.create(spark)`, junto a las funciones de MobilitySpark; ningún nombre de una de ellas es un nombre de la otra. Un ráster de Sedona es un valor, que se lee de un archivo con `RS_FromGeoTiff` y se escribe con `RS_AsGeoTiff`, y se convierte desde y hacia un ráster de MobilityDB mediante las funciones que ambos ya tienen: la georreferenciación pasa por los accesores de cada uno, y los valores de una banda como un arreglo.

Un ráster de Sedona se construye a partir de un ráster `r` de MobilityDB mediante `RS_MakeEmptyRaster` con la georreferenciación de `r` y `RS_AddBandFromArray` con los valores de una banda, su tipo de píxel y su valor nodata. Un ráster de MobilityDB se construye a partir de un ráster `s` de Sedona mediante `raster(pixels, width, height, upperleftx, upperlefty, scalex, scaley, skewx, skewy, srid, pixeltype[, nodataval])` sobre los accesores `RS_` de `s`, siendo sus píxeles los bytes de los valores de `RS_BandAsArray(s, 1)` en su orden; cada valor se convierte en sus bytes como el píxel de una banda de un solo píxel, `rasterPixels(addBand(makeEmptyRaster(1, 1, 0.0, 0.0, 1.0, -1.0, 0.0, 0.0, 0), 1, pixeltype, v), 1)`. Ambas direcciones leen una banda; un ráster de varias bandas se convierte banda por banda.

Los seis tipos de píxel que admite Sedona se convierten en ambas direcciones, con o sin valor nodata, sin cambiar ningún valor, y un ráster convertido a Sedona y de vuelta es el ráster que era, byte a byte; los demás no tienen ráster de Sedona.

```
-- A Sedona raster from the georeferencing and the band of a MobilityDB raster r
SELECT RS_AddBandFromArray(
    RS_MakeEmptyRaster(1, CASE rasterBandPixelType(r, 1) WHEN 'uint8' THEN 'B'
        WHEN 'int16' THEN 'S' WHEN 'uint16' THEN 'US' WHEN 'int32' THEN 'I'
        WHEN 'float32' THEN 'F' WHEN 'float64' THEN 'D' END,
      rasterWidth(r), rasterHeight(r), upperLeftX(r), upperLeftY(r),
      scaleX(r), scaleY(r), skewX(r), skewY(r), SRID(r)),
    flatten(dumpValues(r, 1, false)), 1, rasterBandNoDataValue(r, 1)) AS s
FROM t;
```

Apéndice A

MobilityDB Reference

A.1. Tipos de MobilityDB

MobilityDB define cuatro *tipos de plantilla* (o *template types*) que actúan como constructores de tipos sobre *tipos de base*. Estos tipos de plantilla son `set`, `span`, `spanset` y `temporal`. Tabla A.1 muestra los tipos definidos sobre estos tipos de plantilla. Además, MobilityDB define dos tipos de cuadros delimitadores, a saber, `tbox` y `stbox`.

Tipos de base	**Tipos de plantilla**			
	set	**span**	**spanset**	**temporal**
bool				tbool
text	textset			ttext
jsonb	jsonbset			tjsonb
integer	intset	intspan	intspanset	tint
bigint	bigintset	bigintspan	bigintspanset	tbigint
float	floatset	floatspan	floatspanset	tfloat
date	dateset	datespan	datespanset	
timestamptz	tstzset	tstzspan	tstzspanset	
geometry	geomset			tgeompoint, tgeometry
geography	geogset			tgeogpoint, tgeography
pose	poseset			tpose, trgeometry
posechain	posechainset			tposechain
npoint	npointset			tnpoint
cbuffer	cbufferset			tcbuffer
h3index	h3indexset			th3index
quadbin	quadbinset			tquadbin
s2cell	s2cellset			ts2cell
pcpoint	pcpointset			tpcpoint
pcpatch	pcpatchset			tpcpatch

Cuadro A.1: Instancias actuales de los tipos de plantilla en MobilityDB

A.2. Set and Span Types

A.2.1. Entrada y salida

- asText: Devuelve la representación de texto conocido (WKT)
- asEWKT: Devuelve la representación extendida de texto conocido (EWKT) de un conjunto espacial

- asBinary, asHexWKB: Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)
- asEWKB, asHexEWKB: Devuelve la representación extendida binaria conocida (EWKB) o la representación hexadecimal extendida binaria conocida (HexEWKB) de un conjunto espacial
- settypeFromText, spantypeFromText, spansettypeFromText: Entrada desde la representación de texto conocido (WKT)
- settypeFromBinary, spantypeFromBinary, spansettypeFromBinary: Entrada desde la representación binaria conocida (WKB)
- settypeFromHexWKB, spantypeFromHexWKB, spansettypeFromHexWKB: Entrada desde la representación hexadecimal binaria conocida (HexWKB)
- spatialsettypeFromText, spatialsettypeFromEWKT: Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT) de un conjunto espacial
- spatialsettypeFromEWKB, spatialsettypeFromHexEWKB: Entrada desde la representación extendida binaria conocida (EWKB) o desde la representación hexadecimal extendida binaria conocida (HexEWKB) de un conjunto espacial

A.2.2. Constructores

- set: Constructor para tipos de conjunto
- span: Constructor para tipos de rango
- spanset: Constructor for conjunto de rangos

A.2.3. Conversión de tipos

- ::, set, span, spanset: Convierte un valor de base en un valor de conjunto, rango o conjunto de rangos
- ::, spanset: Convierte un valor de conjunto en un valor de conjunto de rangos
- ::, geometry, geography: Convierte un conjunto geo en la colección de sus valores
- ::, geomset, geogset: Convierte entre una geometría y una geografía
- ::, spanset: Convierte un valor de rango a un valor de conjunto de rangos
- span: Convierte un conjunto de valores o un conjunto de rangos a un rango, ignorando las posibles brechas de tiempo
- ::, span, range: Convierte un valor de de rango en MobilityDB hacia y desde un valor de rango de PostgreSQL
- ::, spanset, multirange: Convierte un valor de conjunto de rangos de MobilityDB hacia y desde un valor de multirango de PostgreSQL

A.2.4. Accesores

- memSize: Devuelve el tamaño de la memoria en bytes
- lower, upper: Devuelve el límite inferior o superior
- lowerInc, upperInc: ¿Es el límite inferior or superior inclusivo?
- width: Devuelve el ancho del rango como un número de punto flotante
- duration: Devuelve el rango de tiempo
- numValues, getValues: Devuelve el número de valores o los valores
- startValue, endValue, valueN: Devuelve el valor inicial, final o enésimo

- unnest: Transforma un conjunto en un conjunto de filas, una por cada valor del conjunto, en el orden del conjunto {}
- numSpans, spans: Devuelve el número de rangos o los rangos
- startSpan, endSpan, spanN: Devuelve el rango inicial, final o enésimo
- numDates, dates: Devuelve el (number of) different dates
- startDate, endDate, dateN: Devuelve el start, end, or n-th date
- numTimestamps, timestamps: Devuelve el número o las marcas de tiempo diferentes
- startTimestamp, endTimestamp, timestampN: Devuelve la marca de tiempo inicial, final, o enésima

A.2.5. Transformaciones

- expand: Expande o reducir los límites de un rango con un valor o un intervalo de tiempo
- shift: Desplaza con un valor o rango de tiempo
- scale: Escala con un valor o un rango de tiempo
- shiftScale: Desplaza y escalear con los valores o rangos de tiempo
- floor, ceil: Redondea al entero inferior o superior
- round: Redondea a un número de decimales
- degrees, radians: Convierte a grados o radianes
- lower, upper, initcap: Transforma en minúsculas, majúsculas o initcap
- ||: Concatenación de texto
- tprecision: Establece la precisión temporal del valor de tiempo al rango con respecto al origen

A.2.6. Sistema de referencia espacial

- SRID, setSRID: Devuelve o especifica el identificador de referencia espacial
- transform, transformPipeline: Transforma a una referencia espacial diferente

A.2.7. Operaciones de conjuntos

- +, -, *: Unión, diferencia o intersección de conjuntos o de rangos

A.2.8. Operaciones de cuadro delimitador

A.2.8.1. Operaciones topológicas

- &&: ¿Se superponen los valores (tienen valores en común)?
- @>: ¿Contiene el primer valor el segundo?
- <@: ¿Está el primer valor contenido en el segundo?
- -|-: ¿Es el primer valor adyacente al segundo?

A.2.8.2. Operaciones de posición

- <<, <<#: ¿Está el primer valor estrictamente a la izquierda del segundo?
- >>, #>>: ¿Está el primer valor estrictamente a la derecha del segundo?
- &<, &<#: ¿No está el primer valor a la derecha del segundo?
- &>, #&>: ¿No está el primer valor a la izquierda del segundo?

A.2.8.3. Operaciones de división

- splitNSpans: Devuelve una matriz de N rangos obtenida fusionando los elementos de un conjunto o los rangos de un conjunto de rangos
- splitEachNSpans: Devuelve una matriz de rangos obtenida fusionando N elementos consecutivos de un conjunto o N rangos consecutivos de un conjunto de rangos

A.2.9. Operaciones de distancia

- <->: Devuelve la distancia mínima

A.2.10. Comparaciones

- =, <>, <, >, <=, >=, cmp: Comparaciones tradicionales
- hash, hashExtended: Valores hash

A.2.11. Agregaciones

- extent: Rango o período delimitador
- setUnion, spanUnion, spansetUnion: Unión agregada
- tCount: Conteo temporal

A.3. Box Types

A.3.1. Entrada y salida

- asText: Devuelve la representación de texto conocido (WKT)
- asBinary, asHexWKB: Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)
- asEWKB, asHexEWKB: Devuelve la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB) de una caja espaciotemporal
- boxFromText: Entrada desde la representación de texto conocido (WKT)
- boxFromBinary, boxFromHexWKB: Entrada desde la representación binaria conocida (WKB) o desde la representación hexadecimal binaria conocida (HexWKB)
- stboxFromEWKB, stboxFromHexEWKB: Entrada de una caja espaciotemporal desde la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB)

A.3.2. Constructores

- tbox: Constructor para `tbox`
- stboxX, stboxZ, stboxT, stboxXT, stboxZT, geodstboxZ, geodstboxZT, stbox: Constructor para `stbox`

A.3.3. Conversión de tipos

- ::, intspan, floatspan, tstzspan: Convierte un `tbox` a otro tipo
- ::, tbox: Convierte otro tipo a un `tbox`
- ::, box2d, box3d, geometry, geography, tstzspan: Convierte un `stbox` a otro tipo
- ::, stbox: Convierte otro tipo a un `stbox`

A.3.4. Accesores

- hasX, hasZ, hasT: ¿Tiene dimensión X/Z/T?
- isGeodetic: ¿Es geodética?
- xMin, yMin, zMin, tMin: Devuelve el valor mínimo de X/Y/Z/T
- xMax, yMax, zMax, tMax: Devuelve el valor máximo de X/Y/Z/T
- xMinInc, tMinInc: Es el mínimo valor X/T inclusivo?
- xMaxInc, tMaxInc: Es el máximo valor X/T inclusivo?
- area, perimeter, volume: Área, volumen, perímetro

A.3.5. Transformaciones

- shiftValue, scaleValue, shiftScaleValue: Desplaza y/o escalear el rango de valores de un cuadro delimitador con uno o dos valores
- shiftTime, scaleTime, shiftScaleTime: Desplaza y/o escalar el período de un cuadro delimitador con uno o dos intervalos
- getSpace: Devuelve la dimensión espacial del cuadro delimitador, eliminando la dimensión temporal, si existe
- expandValue, expandSpace, expandTime: Extiende la dimensión numérica, espacial o temporal del cuadro delimitador con un valor o un intervalo
- round: Redondea el valor o las coordenadas del cuadro delimitador a un número de decimales

A.3.6. Sistema de referencia espacial

- SRID, setSRID: Devuelve o especifica el identificador de referencia espacial
- transform, transformPipeline: Transforma a una referencia espacial diferente

A.3.7. Operaciones de división

- quadSplit: Divide el cuadro delimitador en cuadrantes u octantes {}

A.3.8. Operaciones de conjuntos

- +, *: Unión e intersección de dos cuadros delimitadores

A.3.9. Operaciones de cuadro delimitador

A.3.9.1. Operaciones topológicas

- &&: ¿Se superponen los cuadros delimitadores?
- @>: ¿Contiene el primer cuadro delimitador el segundo?
- <@: ¿Está el primer cuadro delimitador contenido en el segundo?
- ~=: ¿Son los cuadros delimitadores iguales en sus dimensiónes comunes?
- -|-: ¿Son los cuadros delimitadores adyacentes?

A.3.9.2. Operaciones de posición

- <<, <<|, <</, <<#: ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente menores que los del segundo?
- >>, |>>, />>, #>>: ¿Son los valores X/Y/Z/T del primer cuadro delimitador estrictamente mayores que los del segundo?
- &<, &<|, &</, &<#: ¿No son los valores X/Y/Z/T del primer cuadro delimitador mayores que los del segundo?
- &>, |&>, /&>, #&>: ¿No son los valores X/Y/Z/T del primer cuadro delimitador menores que los del segundo?

A.3.10. Comparaciones

- =, <>, <, >, <=, >=, cmp: Comparaciones tradicionales
- hash, hashExtended: Valores hash

A.3.11. Aggregaciones

- extent: Extensión del cuadro delimitador

A.4. Temporal Types

A.4.1. Entrada y salida

- asText: Devuelve la representación de texto conocido (WKT)
- asMFJSON: Devuelve la representación JSON de características móviles (MF-JSON)
- asBinary, asHexWKB: Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)
- ttypeFromText: Entrada desde la representación de texto conocido (WKT)
- ttypeFromMFJSON: Entrada desde la representación JSON de características móviles (MF-JSON)
- ttypeFromBinary, ttypeFromHexWKB: Entrada desde la representación binaria conocida (WKB) o desde la representación hexadecimal binaria conocida (HexWKB)

A.4.2. Constructores

- ttype: Constructores para tipos temporales que tienen un valor constante
- ttypeSeq: Constructores para tipos temporales de subtipo secuencia
- ttypeSeqSet, ttypeSeqSetGaps: Constructores para tipos temporales de subtipo conjunto de secuencias

A.4.3. Conversión de tipos

- :::: Convierte un valor temporal a un intervalo de marcas de tiempo

A.4.4. Accesores

- memSize: Devuelve el tamaño de la memoria en bytes
- tempSubtype: Devuelve el subtipo temporal
- tempBasetype: Devuelve el tipo base
- interp: Devuelve la interpolación
- getValue, getTimestamp: Devuelve el valor o la marca de tiempo de un instantes
- getValues, getTime: Devuelve los valores o el tiempo en el que se define el valor temporal
- timeSpan: Devuelve el período delimitador en el que se define el valor temporal
- startValue, endValue, valueN: Devuelve el valor inicial, final, o enésimo
- valueAtTimestamp: Devuelve el valor en una marca de tiempo
- duration: Devuelve el intervalo de tiempo
- lowerInc, upperInc: ¿Es el instante inicial/final inclusivo?
- numInstants, instants: Devuelve el número o los instantes diferentes
- startInstant, endInstant, instantN: Devuelve el instante inicial, final o enésimo
- numTimestamps, timestamps: Devuelve el número o las marcas de tiempo diferentes
- startTimestamp, endTimestamp, timestampN: Devuelve la marca de tiempo inicial, final o enésima
- numSequences, sequences: Devuelve el número o las secuencias
- startSequence, endSequence, sequenceN: Devuelve la secuencia inicial, final, o enésima
- segments: Devuelve los segmentos

A.4.5. Transformaciones

- ttypeInst, ttypeSeq, ttypeSeqSet: Transforma un tipo temporal a otro subtipo
- setInterp: Transforma un valor temporal a otra interpolación
- segmentMinDuration, segmentMaxDuration: Devuelve un valor booleano temporal que indica si cada segmento de una secuencia (o conjunto de sequencias) temporal(es) continua tiene, respectivamente, al menos o como máximo una duración determinada
- shiftTime, scaleTime, shiftScaleTime: Desplaza y/o escalear el intervalo de tiempo de un valor temporal a uno o dos intervalos
- unnest: Transforma un valor temporal no lineal en un conjunto de filas, cada una es una pareja compuesta de un valor de base y un conjunto de períodos durante el cual el valor temporal tiene el valor de base {}

A.4.6. Modificaciones

- insert: Inserta un valor temporal en otro
- update: Actualiza un valor temporal con otro
- deleteTime: Elimina de un valor temporal un valor de tiempo
- appendInstant: Anexa un instante temporal a un valor temporal
- appendSequence: Anexa una secuencia temporal a un valor temporal
- merge: Fusiona los valores temporales

A.4.7. Restricciones

- atValue, minusValue, atValues, minusValues, atSpan, minusSpan, atSpanset, minusSpanset: Restringe a (al complemento de) un valor o un conjunto de valores o, para un número temporal, un span o un conjunto de spans de valores
- atTime, minusTime: Restringe a (al complemento de) un valor de tiempo
- beforeTimestamp, afterTimestamp: Mantiene los instantes de un valor temporal que son anteriores o posteriores a una marca de tiempo, o iguales a ella

A.4.8. Comparaciones

A.4.8.1. Comparaciones tradicionales

- =, <>, <, >, <=, >=, cmp: Comparaciones tradicionales
- hash, hashExtended: Valores hash

A.4.8.2. Comparaciones alguna vez y siempre

- ?=, ?<>, ?<, ?>, ?<=, ?>=, %=, %<>, %<, %>, %<=, %>=: Comparaciones alguna vez y siempre

A.4.8.3. Comparaciones temporales

- #=, #<>, #<, #>, #<=, #>=: Comparaciones temporales

A.4.9. Funciones de utilidad

- mobilitydbVersion: Versión de la extensión MobilityDB
- mobilitydbFullVersion: Versión de la extensión MobilityDB y de sus dependencias

A.5. Temporal Alphanumeric Types

A.5.1. Conversión de tipos

- ::, valueSpan(tnumber), timeSpan(tnumber), tbox(tnumber), valueSpan, timeSpan, tbox: Convierte un número temporal en un rango o un cuadro delimitador temporal
- ::, tint(tbool), tint: Convierte entre un booleano temporal y un entero temporal
- tnumber, ::, tnumber(tnumber): Convierte entre enteros temporales, enteros grandes temporales y flotantes temporales

A.5.2. Accessores

- valueSpan: Devuelve el intervalo de valores ignorando las brechas potenciales
- valueSet: Devuelve el conjunto de valores de un número temporal
- minValue, maxValue, avgValue: Devuelve el valor mínimo, máximo, o promedio
- minInstant, maxInstant: Devuelve el instante con el valor mínimo o máximo

A.5.3. Transformaciones

- shiftValue, scaleValue, shiftScaleValue: Desplaza y/o escalear el rango de valores de un número temporal con uno o dos números
- stops: Extrae de un flotante temporal con interpolación lineal las subsecuencias donde los valores permanecen en un rango de un ancho dado durante al menos una duración dada

A.5.4. Restrictions

- atMin, atMax, minusMin, minusMax: Restringe al (complemento del) valor mínimo o máximo
- atTbox, minusTbox: Restringe a (al complemento de) un `tbox`

A.5.5. Operaciones booleanas

- &, |: Y temporal, o temporal
- ~: No temporal
- whenTrue: Devuelve el tiempo cuando un booleano temporal toma el valor verdadero

A.5.6. Operaciones matemáticas

- +, -, *, /: Adición, resta, multiplicación y división temporal
- abs: Devuelve el valor absoluto del número temporal
- floor, ceil: Redondea al entero inferior o superior
- round: Redondea a un número de posiciones decimales
- degrees, radians: Convierte a grados o radianes
- deltaValue: Devuelve la diferencia de valor entre instantes consectivos del número temporal
- trend: Devuelve la tendencia de un flotante temporal con interpolación lineal, que indica si su valor es creciente, constante o decreciente, representado, respectivamente, por 1, 0 y -1
- derivative: Devuelve la derivada sobre el tiempo del número flotante temporal en unidades por segundo
- integral: Devuelve el área bajo la curva
- twAvg: Devuelve el promedio ponderado en el tiempo
- ln, log10: Devuelve el logaritmo natural y el logaritmo base 10 de un número flotante temporal
- exp: Devuelve el exponencial (e elevado a la potencia dada) de un número flotante temporal
- sin, cos, tan: Devuelve el seno, coseno o tangente de un número flotante temporal (argumento en radianes)

A.5.7. Operaciones de texto

- ||: Concatenación de texto
- lower, upper, initcap: Transforma en minúsculas, mayúsculas o initcap

A.6. Temporal Geometry Types

A.6.1. Funciones sobre Geometrías

- orientedEnvelope: Devuelve el rectángulo de área mínima que encierra una geometría
- convexHull: Devuelve la geometría convexa más pequeña que encierra una geometría
- isSimple: Devuelve verdadero si una geometría no tiene ningún punto en el que se cruce o se toque a sí misma
- buffer: Devuelve la geometría que contiene todos los puntos situados a una distancia dada de una geometría
- relate: Devuelve la matriz de intersección DE-9IM de dos geometrías o si coincide con un patrón
- contains, covers, disjoint, intersects, touches, geoEquals: Devuelve verdadero si dos geometrías satisfacen una relación espacial
- dwithin: Devuelve verdadero si dos geometrías están a una distancia dada
- area, perimeter, centroid: Devuelve el área, el perímetro o el centroide de una geometría o una geografía
- distance, shortestLine, maxDistance: Devuelve la distancia, la línea que une los puntos más cercanos o la distancia máxima de dos geometrías o geografías
- boundary, reverse, numGeometries, geometryN, numPoints: Devuelve la frontera, la inversión o las partes de una geometría
- lineInterpolatePoint, lineSubstring, lineLocatePoint: Devuelve el punto en una fracción de una línea, la parte entre dos fracciones o la fracción más cercana a un punto
- collect, makeLine: Devuelve la colección de un arreglo de geometrías o la línea que las une
- asText, asEWKT, asEWKB, asGeoJSON: Devuelve la representación Well-Known Text, Extended Well-Known Text, Extended Well-Known Binary o GeoJSON de una geometría o una geografía
- transform: Transforma una geometría o una geografía a un identificador de referencia espacial
- segmentize: Densifica una geometría o una geografía de modo que ningún segmento sea más largo que una distancia
- clusterKMeans, clusterDBSCAN, clusterIntersecting, clusterWithin: Devuelve los grupos de un arreglo de geometrías
- intersection: Devuelve la región que dos geometrías comparten
- geoUnion: Devuelve la región que dos geometrías cubren en conjunto
- difference: Devuelve la región de la primera geometría que la segunda no cubre
- symDifference: Devuelve la región que cubre exactamente una de dos geometrías

A.6.2. Entrada y salida

- asText, asEWKT: Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)
- asMFJSON: Devuelve la representación JSON de características móviles (MF-JSON)
- asBinary, asEWKB, asHexEWKB: Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)
- tspatialFromText, tspatialFromEWKT: Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)
- tspatialFromMFJSON: Entrada desde la representación JSON de características móviles (MF-JSON)
- tspatialFromBinary, tspatialFromEWKB, tspatialFromHexWKB, tspatialFromHexEWKB, geometryFromHexEWKB, geographyFromHexEWKB: Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

A.6.3. Conversión de tipos

- stbox, ::, stbox(tspatial): Convierte un valor espaciotemporal a un rango temporal o a un cuadro delimitador espaciotemporal
- ::: Convierte entre una geometría temporal y una geografía temporal
- ::: Convierte entre un punto temporal y una trayectoria PostGIS

A.6.4. Accessores

- trajectory, traversedArea: Devuelve la trayectoria o el área atravesada
- centroid: Devuelve el centroide como un punto temporal
- getX, getY, getZ: Devuelve los valores de las coordenadas X/Y/Z como un número flotante temporal
- isSimple: Devuelve verdadero si el punto temporal no se auto-intersecta espacialmente
- length: Devuelve la longitud atravesada por el punto temporal
- cumulativeLength: Devuelve la longitud acumulada atravesada por el punto temporal
- speed: Devuelve la velocidad del punto temporal en unidades por segundo
- twCentroid: Devuelve el centroide ponderado en el tiempo
- direction: Devuelve la dirección, es decir, el acimut entre las ubicaciones inicial y final
- azimuth: Devuelve el acimut temporal
- angularDifference: Devuelve la diferencia angular temporal
- bearing: Devuelve el rumbo temporal

A.6.5. Transformaciones

- round: Redondea los valores de las coordenadas a un número de decimales
- makeSimple: Devuelve una matriz de fragmentos del punto temporal que son simples
- geoMeasure: Construye una geometría/geografía con medida M a partir de un punto temporal y un número flotante temporal
- force2D, force3D: Devuelve una geometría temporal sin Z o con una Z, como lo hacen las funciones de PostGIS `ST_Force2D` y `ST_Force3D` para una geometría
- affine: Devuelve la transformación afín 3D de una geometría temporal para traducirla, rotarla y/o escalarla en un solo paso
- rotate: Devuelve el punto temporal rotado en sentido antihorario sobre el punto de origen
- rotateX: Devuelve una geometría temporal rotada en sentido antihorario alrededor del eje x
- rotateY: Devuelve una geometría temporal rotada en sentido antihorario alrededor del eje y
- rotateZ: Devuelve una geometría temporal rotada en sentido antihorario alrededor del eje z
- scale: Devuelve un punto temporal escalado por factores dados
- transscale: Devuelve una geometría temporal trasladada por desplazamientos dados y luego escalada por factores dados
- translate: Devuelve una geometría temporal trasladada por desplazamientos dados
- asMVTGeom: Transforma un punto geométrico temporal en el espacio de coordenadas de un Mapbox Vector Tile
- stops: Extrae de un punto temporal con interpolación lineal las subsecuencias donde el punto permanece dentro de un área con un tamaño máximo especificado durante al menos la duración dada

A.6.6. Restricciones

- atGeometry, minusGeometry: Restringe a (al complemento de) una geometría
- atStbox, minusStbox: Restringe a (al complemento de) un `stbox`
- atElevation, minusElevation: Restringe a (al complemento de) un rango de altitud

A.6.7. Sistema de referencia espacial

- SRID, setSRID: Devuelve o establece el identificador de referencia espacial
- transform, transformPipeline: Transforma a una referencia espacial diferente

A.6.8. Operaciones de cuadro delimitador

- expandSpace: Devuelve el cuadro delimitador espaciotemporal expandido en la dimensión espacial por un valor flotante

A.6.9. Operaciones de distancia

- |=|: Devuelve la distancia más pequeña que haya existido
- nearestApproachInstant: Devuelve el instante del primer punto temporal en el que los dos argumentos están a la distancia más cercana
- minDistance: Devuelve la distancia espacial mínima entre dos conjuntos de geometrías temporales, ignorando el tiempo
- eDwithinPairs, aDwithinPairs, tDwithinPairs, eIntersectsPairs, aIntersectsPairs, tIntersectsPairs, eDisjointPairs, aDisjointPairs, tDisjointPairs, eTouchesPairs, aTouchesPairs, tTouchesPairs: Devuelve los pares de índices que cumplen la relación entre dos arreglos de geometrías temporales, generalizando las relaciones espaciales escalares a una unión espacial conjunto-conjunto
- shortestLine: Devuelve la línea que conecta el punto de aproximación más cercano
- <->: Devuelve la distancia temporal

A.6.10. Relaciones siempre y alguna vez

- eContains, aContains: Contiene alguna vez o siempre
- eCovers, aCovers: Cubre alguna vez o siempre
- eDisjoint, aDisjoint: Está disjunto alguna vez o siempre
- eDwithin, aDwithin: Está alguna vez o siempre a distancia de
- eIntersects, aIntersects: Intersecta alguna vez o siempre
- eTouches, aTouches: Toca alguna vez o siempre

A.6.11. Relaciones espaciotemporales

- tContains: Contiene temporal
- tDisjoint: Disjunto temporal
- tDwithin: Está a distancia de temporal
- tIntersects: Intersección temporal
- tTouches: Toca temporal

A.7. Operations for Temporal Types: Analytics Operations

A.7.1. Simplificación

- minDistSimplify, minTimeDeltaSimplify: Simplifica un flotante o una geometría/geografía temporal asegurándose de que los valores consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo
- maxDistSimplify, douglasPeuckerSimplify: Simplifica un flotante o un punto temporal usando el algoritmo de Douglas-Peucker

A.7.2. Reducción

- tsample: Muestrea un valor temporal con respecto a un intervalo
- tprecision: Reduce la precisión temporal de un valor temporal con respecto a un intervalo calculando el promedio/centroide ponderado por el tiempo en cada intervalo de tiempo

A.7.3. Similaridad

- hausdorffDistance, frechetDistance, dynTimeWarpDistance, averageHausdorffDistance, lcssDistance: Devuelve la distancia de Hausdorff discreta, la distancia de Fréchet discreta, la distancia de distorsión de tiempo dinámica (Dynamic Time Warp o DTW), la distancia de Hausdorff promedio, o la distancia de subsecuencia común más larga (Longest Common SubSequence o LCSS) entre dos valores temporales
- frechetDistancePath, dynTimeWarpPath: Devuelve las parejas de correspondencia entre dos valores temporales con respecto a la distancia de Fréchet discreta o la distance de distorsión de tiempo dinámica (Dynamic Time Warp o DTW) {}

A.7.4. Filtro de Kalman extendido

- extendedKalmanFilter: Devuelve un flotante temporal o un punto temporal suavizado, obtenido al filtrar una trayectoria ruidosa con un filtro de Kalman extendido

A.7.5. Operaciones de división

- splitNSpans: Devuelve una matriz de N rangos de tiempo a partir de los instantes o segmentos de un valor temporal
- splitEachNSpans: Devuelve una matriz de rangos de tiempo obtenida fusionando N instantes o segmentos consecutivos de un valor temporal
- splitNTboxes: Devuelve una matriz de N cuadros temporales obtenida fusionando los instantes o segmentos de un número temporal
- splitEachNTboxes: Devuelve una matriz de cuadros temporales obtenida fusionando N instantes o segmentos consecutivos de un número temporal
- splitNStboxes: Devuelve ya sea una matriz de N cuadros espaciales obtenida fusionando los segmentos de una (multi)línea o una matriz de N cuadros espaciotemporales obtenida fusionando los instantes o segmentos de un punto temporal
- splitEachNStboxes: Devuelve ya sea una matriz de cuadros espaciales obtenida fusionando N segmentos consecutivos de una (multi)línea o una matriz de cuadros espaciotemporales obtenida fusionando N instantes o segmentos consecutivos de un punto temporal

A.7.6. Mosaicos multidimensionales

A.7.6.1. Operaciones de intervalos

- bins: Devuelve una matriz de intervalos que cubre el rango de valores o de tiempo con intervalos de la misma amplitud o duración
- getBin: Devuelve el rango que contiene un número o una marca de tiempo

A.7.6.2. Operaciones de mosaicos

- valueTiles, timeTiles, valueTimeTiles: Devuelve el conjunto de mosaicos que cubre un cuadro delimitador temporal con mosaicos del mismo tamaño y/o duración {}
- spaceTiles, timeTiles, spaceTimeTiles: Devuelve el conjunto de mosaicos que cubre un cuadro delimitador espaciotemporal con mosaicos del mismo tamaño y/o duración {}
- getValueTile, getTboxTimeTile, getValueTimeTile: Devuelve el mosaico temporal que cubre un valor y/o una marca de tiempo
- getSpaceTile, getStboxTimeTile, getSpaceTimeTile: Devuelve el mosaico espaciotemporal que cubre un punto y/o una marca de tiempo

A.7.6.3. Operaciones de cuadro delimitador

- valueBins, timeBins: Devuelve una matriz de rangos de valores o de tiempo obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores o de tiempo
- valueBoxes, timeBoxes, valueTimeBoxes: Devuelve una matriz de cuadros temporales obtenidos a partir de los instantes o segmentos de un número temporal con respecto a un mosaico de valores y/o tiempo
- spaceBoxes, timeBoxes, spaceTimeBoxes: Devuelve una matriz de cuadros espaciotemporales obtenidos a partir de los instantes o segmentos de un punto temporal con respecto a un mosaico espacial y/o temporal

A.7.6.4. Operaciones de división

- valueSplit, timeSplit, valueTimeSplit: Fragmenta un número temporal con respecto a intervalos de valores y/o de tiempo {}
- spaceSplit, timeSplit, spaceTimeSplit: Fragmenta un punto temporal con respecto a una malla espacial o espacio-temporal {}

A.7.7. Agregación

- tCount: Conteo temporal
- extent: Extensión del cuadro delimitador
- tAnd, tOr: Y, o temporal
- tMin, tMax, tSum, tAvg: Mínimo, máximo, suma y promedio temporal
- wCount: Conteo de ventana
- wMin, wMax, wSum, wAvg: Mínimo, máximo, suma y promedio de ventana
- tCentroid: Centroide temporal

A.8. Temporal Poses and Pose Chains

A.8.1. Poses estáticas y cadenas de poses estáticas

A.8.1.1. Entrada y salida

- asText, asEWKT: Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)

- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)
- poseFromText, poseFromEWKT: Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)
- poseFromBinary, poseFromEWKB, poseFromHexWKB, poseFromHexEWKB: Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

A.8.1.2. Constructores

- geodpose: Constructor de poses geodésicos a partir de un punto geográfico
- pose: Constructor para poses

A.8.1.3. Conversión de tipos

- ::, stbox: Convierte una pose y, opcionalmente, una marca de tiempo o un período, en una caja espaciotemporal
- ::: Convierte una pose, planar o geodésica, en un punto geográfico
- ::: Convierte una pose, planar o geodésica, en un punto geométrico

A.8.1.4. Accesores

- point: Devuelve el punto, que para una cadena de poses es el punto de su marco más interior
- quaternion: Devuelve el cuaternión de orientación de una pose

A.8.1.5. Transformaciones

- round: Redondea el punto y la orientación de la pose al número de posiciones decimales

A.8.1.6. Sistema de referencia espacial

- SRID, setSRID: Devuelve o establece el identificador de referencia espacial, que para una cadena de poses es el identificador de su marco exterior
- transform, transformPipeline: Transforma a un identificador de referencia espacial

A.8.1.7. Operaciones de distancia

- <->: Devuelve la distancia
- |=|: Devuelve la distancia de aproximación más cercana

A.8.1.8. Comparaciones

- =, <>, <, >, <=, >=: Comparaciones tradicionales

A.8.2. Composición de cadenas de poses

- posechain: Construye una cadena de poses a partir de un arreglo de poses ordenadas desde el marco más exterior hacia dentro
- posechain: Convierte una pose en una cadena de poses de un solo eslabón
- pose: Devuelve la pose de un marco que define la cadena, leída en el marco exterior de la cadena
- appendPose: Devuelve una cadena de poses con una pose añadida como su eslabón más interior
- numPoses: Devuelve el número de eslabones de una cadena de poses
- startPose, endPose, poseN, poses: Devuelve los eslabones de una cadena de poses
- numPoses: Devuelve el número de eslabones que tiene todo valor de una cadena de poses temporal

A.8.3. Entrada y salida

- asText, asEWKT: Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)
- asMFJSON: Devuelve la representación JSON de características móviles (MF-JSON)
- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)
- tposeFromText, tposeFromEWKT: Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)
- tposeFromMFJSON, tposechainFromMFJSON: Entrada desde la representación JSON de características móviles (MF-JSON)
- tposeFromBinary, tposeFromEWKB, tposeFromHexWKB, tposeFromHexEWKB: Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

A.8.4. Constructores

- tpose: Constructor para poses temporales con valor constante
- tposeSeq: Constructor para poses temporales de subtipo secuencia
- tposeSeqSet, tposeSeqSetGaps: Constructor para poses temporales de subtipo conjunto de secuencias
- tpose: Construye una pose temporal 2D a partir de un punto geométrico temporal y un flotante temporal

A.8.5. Conversión de tipos

- ::: Convierte una pose temporal, planar o geodésica, en un punto geométrico temporal
- ::: Convierte una pose temporal, planar o geodésica, en un punto geográfico temporal

A.8.6. Accesores

- getValues: Devuelve los valores
- points: Devuelve los puntos
- trajectory: Devuelve la trayectoria
- valueAtTimestamp: Devuelve el valor en una marca de tiempo
- speed: Devuelve la velocidad de una pose temporal como un flotante temporal: la magnitud de la velocidad de la componente de posición (distancia recorrida por unidad de tiempo)
- angularSpeed: Devuelve la velocidad angular de una pose temporal como un flotante temporal con interpolación por pasos (radianes por unidad de tiempo)
- getValue, getValues, timeSpan: Devuelve el valor de un instante temporal, el conjunto de valores o el lapso de tiempo

A.8.7. Transformaciones

- tposeInst, tposeSeq, tposeSeqSet: Transforma a otro subtipo
- setInterp: Transforma a otra interpolación
- round: Redondea los puntos y las orientaciones de la pose temporal al número de posiciones decimales
- translate: Devuelve un valor temporal trasladado por los desplazamientos dados 
- rotate: Devuelve un valor temporal rotado en sentido antihorario alrededor del origen o de un punto 
- rotateX: Devuelve un valor temporal rotado en sentido antihorario alrededor del eje x 
- rotateY: Devuelve un valor temporal rotado en sentido antihorario alrededor del eje y 
- rotateZ: Devuelve un valor temporal rotado en sentido antihorario alrededor del eje z 
- force2D: Devuelve una pose temporal proyectada sobre el suelo 

A.8.8. Modificaciones

- appendInstant, appendSequence: Agrega un instante temporal o una secuencia temporal
- merge: Fusiona las poses temporales
- insert, update, deleteTime: Inserta o actualizar la segunda cadena de poses temporal en la primera, o eliminar de ella un valor de tiempo

A.8.9. Restricciones

- atValue, minusValue, atValues, minusValues: Restringe al (complemento de) un valor o un conjunto de valores
- atGeometry, minusGeometry: Restringe al (complemento de) una geometría
- atElevation, minusElevation: Restringe al (complemento de) un span de elevación
- atTime, atValue: Restringe una cadena de poses temporal a un tiempo o a un valor

A.8.10. Sistema de referencia espacial

- SRID, setSRID: Devuelve o establece el identificador de referencia espacial, que para una cadena de poses temporal es el identificador de su marco exterior
- transform, transformPipeline: Transforma a un identificador de referencia espacial

A.8.11. Operaciones de cuadro delimitador

- &&, <@, @>, ~=, -|-: Operadores topológicos
- ::: Operadores de posición
- expandSpace, stboxes: Devuelve la caja delimitadora espaciotemporal expandida en la dimensión espacial, o la matriz de cajas que cubren la cadena de poses temporal

A.8.12. Operaciones de distancia

- |=|: Devuelve la distancia más pequeña alguna vez
- nearestApproachInstant: Devuelve el instante de la primera pose temporal en el que los dos argumentos están a la distancia más cercana
- shortestLine: Devuelve la línea que conecta el punto de aproximación más cercano
- <->: Devuelve la distancia temporal

A.8.13. Relaciones siempre y alguna vez

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Relaciones alguna vez y siempre

A.8.14. Relaciones espaciotemporales

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Relaciones espaciotemporales

A.8.15. Comparaciones

- ::: Comparaciones tradicionales
- ?=, %=, ?<>, %<>: Comparaciones alguna vez y siempre
- ::: Comparaciones temporales
- eEqual: Compara dos cadenas de poses temporales, o una cadena de poses temporal y una cadena de poses

A.8.16. Agregaciones

- tCount: Conteo temporal
- wCount: Conteo de ventana
- extent: Extensión del cuadro delimitador
- appendInstant: Agrega los instantes agregados en una secuencia
- appendSequence: Agrega las secuencias agregadas en un conjunto de secuencias

A.8.17. Operaciones espaciales

- centroid: Devuelve el punto de geometría temporal dado por los puntos de una pose temporal
- convexHull: Devuelve la envolvente convexa de los puntos de una pose temporal

A.8.18. Simplificación

- minDistSimplify, minTimeDeltaSimplify: Devuelve una pose temporal simplificada asegurando que los puntos consecutivos estén separados al menos una cierta distancia o intervalo de tiempo
- maxDistSimplify, douglasPeuckerSimplify: Devuelve una pose temporal simplificada mediante el algoritmo de Douglas-Peucker

A.8.19. Soporte de OGC GeoPose v1.0

A.8.19.1. Entrada y salida JSON

- asGeoPose, poseFromGeoPose: Convierte hacia o desde la codificación JSON de OGC GeoPose v1.0 (clase de conformidad Basic-Quaternion, Basic-YPR o Advanced)

A.8.19.2. Renormalización de cuaterniones

- poseNormalize: Devuelve una pose cuyo cuaternión de orientación se ha llevado a una norma unitaria
- poseInverse: Devuelve la inversa de una pose

A.8.19.3. Accesores de ángulos de Euler

- yaw, pitch, roll: Devuelve el ángulo yaw, pitch o roll de una pose, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose
- ypr: Devuelve la orientación de una pose como una terna yaw / pitch / roll, en radianes
- yaw, pitch, roll: Eleva los accesores de ángulos de Euler sobre una pose temporal, devolviendo un flotante temporal en radianes

A.8.19.4. Registro de metadatos de marcos

- geoPoseFrames: Devuelve todos los marcos que nombran el estándar y los codificadores
- geoPoseFrameSRID: Devuelve el SRID de un marco que MEOS registra, o `NULL` si el marco es paramétrico (LTP, BODY)
- geoPoseFrameName: Devuelve el nombre legible de un marco que MEOS registra
- geoPoseFrameIsGeographic: Devuelve `true` para un marco lat/lon/h que MEOS registra, `false` para uno cartesiano o proyectado

A.8.19.5. Transformación rígida cuerpo↔mundo

- applyPose: Aplica la transformación de cuerpo rígido codificada por una pose (la correspondencia *cuerpo al mundo* de OGC GeoPose) a una geometría en el marco del cuerpo, produciendo la geometría correspondiente en el marco del mundo

A.8.19.6. E/S JSON temporal

- asGeoPose, tposeFromGeoPose: Convierte una pose temporal hacia o desde su codificación OGC GeoPose v1.0
- asGeoPoseStream: Escribe una pose temporal como un Stream de OGC GeoPose
- asGeoPoseStreamHeader, asGeoPoseStreamElement: Escribe la cabecera de Stream de OGC GeoPose de una pose temporal, o el elemento de Stream de uno de sus instantes

A.8.19.7. Documentos compuestos de cadena y de grafo

- asGeoPose, tposechainFromGeoPose: Escribe o lee un documento de cadena compuesta OGC GeoPose
- asGeoPose: Escribe un documento de grafo compuesto OGC GeoPose

A.9. Temporal Network Points

A.9.1. Tipos de red estáticos

A.9.1.1. Entrada y salida

- asText, asEWKT: Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)
- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)
- npointFromText, npointFromEWKT, nsegmentFromText, nsegmentFromEWKT: Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)
- npointFromBinary, npointFromEWKB, npointFromHexWKB, npointFromHexEWKB: Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)
- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB) de un segmento de red
- nsegmentFromBinary, nsegmentFromEWKB, nsegmentFromHexWKB, nsegmentFromHexEWKB: Entrada de un segmento de red desde la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)

A.9.1.2. Constructores

- npoint, nsegment: Constructores de puntos o segmentos de red

A.9.1.3. Conversión de tipos

- :::: Convierte entre un punto o un segmento de red y una geometría
- stbox: Construye una caja espaciotemporal a partir de un punto de red y, opcionalmente, una marca de tiempo o un período

A.9.1.4. Accesores

- route: Devuelve el identificador de ruta
- getPosition: Devuelve la posición
- startPosition, endPosition: Devuelve la posición inicial/final

A.9.1.5. Transformaciones

- round: Redondea la(s) posición(es) del punto de red or el segmento de red en el número de posiciones decimales

A.9.1.6. Operaciones espaciales

- SRID: Devuelve el identificador de referencia espacial

A.9.1.7. Comparaciones

- =, <>, <, >, <=, >=: Comparaciones tradicionales

A.9.2. Constructores

- tnpoint, tnpointSeq, tnpointSeqSet: Constructor para puntos de red temporales con valor constante
- tnpointSeq: Constructor para puntos de red temporal de subtipo secuencia
- tnpointSeqSet, tnpointSeqSetGaps: Constructor para puntos de red temporal de subtipo conjunto de secuencias

A.9.3. Conversión de tipos

- :::: Convierte entre un punto de red temporal y un punto de geometría temporal

A.9.4. Entrada y Salida MF-JSON

- asMFJSON, tnpointFromMFJSON: Genera un punto de red temporal como MF-JSON y lo lee de vuelta; el ciclo de ida y vuelta es sin pérdida

A.9.5. Accessores

- getValues: Devuelve los valores
- routes: Devuelve los identificadores de ruta
- valueAtTimestamp: Devuelve el valor en una marca de tiempo
- length: Devuelve la longitud atravesada por el punto de red temporal
- cumulativeLength: Devuelve la longitud acumulada atravesada por el punto de red temporal
- speed: Devuelve la velocidad del punto de red temporal en unidades por segundo

A.9.6. Transformaciones

- tnpointInst, tnpointSeq, tnpointSeqSet: Transforma un punto de red temporal en otro subtipo
- setInterp: Transforma un punto de red temporal en otra interpolación
- round: Redondea la fracción del punto de red temporal en el número de lugares decimales

A.9.7. Modificaciones

- appendInstant, appendSequence: Agrega un instante temporal o una secuencia temporal
- merge: Fusiona los puntos de red temporales

A.9.8. Restricciones

- atValue, minusValue, atValues, minusValues: Restringe al (complemento de) un valor o un conjunto de valores
- atGeometry, minusGeometry: Restringe al (complemento de) una geometría

A.9.9. Operaciones espaciales

- twCentroid: Devuelve el centroide ponderado en el tiempo

A.9.10. Operaciones de cuadro delimitador

- ?@, @?, @@, @=: Operadores de rutas, que comparan los identificadores de ruta que visita un punto de red temporal: contenido en, contiene, se superpone e igual
- &&, <@, @>, ~=, -|-: Operadores topológicos
- ::: Operadores de posición

A.9.11. Operaciones de distancia

- |=|: Devuelve la distancia más cercana
- nearestApproachInstant: Devuelve el instante del primer punto de red temporal en el que los dos argumentos están a la distancia más cercana
- shortestLine: Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos
- <->: Devuelve la distancia temporal

A.9.12. Relaciones siempre y alguna vez

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Relaciones alguna vez y siempre

A.9.13. Relaciones espaciotemporales

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Relaciones espaciotemporales

A.9.14. Comparaciones

- =, <>, <, >, <=, >=: Comparaciones tradicionales
- ?=, ?<>, %=, %<>: Comparaciones alguna vez y siempre
- #=, #<>: Comparaciones temporales

A.9.15. Agregacions

- tCount: Conteo temporal
- wCount: Conteo de ventana
- tCentroid: Centroide temporal

A.9.16. Agregación

- extent: Devuelve el cuadro delimitador espaciotemporal que cubre cada posición materializada en un conjunto de puntos de red

A.10. Temporal Circular Buffers

A.10.1. Búferes circulares estáticos

A.10.1.1. Entrada y salida

- asText, asEWKT: Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)
- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)
- cbufferFromText, cbufferFromEWKT: Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)
- cbufferFromBinary, cbufferFromEWKB, cbufferFromHexWKB, cbufferFromHexEWKB: Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

A.10.1.2. Constructores

- cbuffer: Constructor para búferes circulares

A.10.1.3. Conversión de tipos

- ::: Convierte entre un búfer circular y una geometría
- stbox: Convierte un búfer circular y, opcionalmente, una marca de tiempo o un período, a un cuadro espaciotemporal

A.10.1.4. Accesores

- point: Devuelve el punto
- radius: Devuelve el radio

A.10.1.5. Transformaciones

- round: Redondea el punto y el radio del búfer circular en el número de lugares decimales

A.10.1.6. Operaciones espaciales

- SRID, setSRID: Devuelve o establece el identificador de referencia espacial
- transform, transformPipeline: Transforma a una referencia espacial diferente

A.10.1.7. Operaciones de distancia

- <->: Devuelve la distancia
- |=|: Devuelve la distancia de aproximación más cercana

A.10.1.8. Comparaciones

- =, <>, <, >, <=, >=: Comparaciones tradicionales

A.10.2. Entrada y salida

- asText, asEWKT: Devuelve la representación de texto conocido (WKT) o la representación extendida de texto conocido (EWKT)
- asMFJSON: Devuelve la representación JSON de características móviles (MF-JSON)
- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria conocida (WKB), la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)
- tcbufferFromText, tcbufferFromEWKT: Entrada desde la representación de texto conocido (WKT) o desde la representación extendida de texto conocido (EWKT)
- tcbufferFromMFJSON: Entrada desde la representación JSON de características móviles (MF-JSON)
- tcbufferFromBinary, tcbufferFromEWKB, tcbufferFromHexWKB, tcbufferFromHexEWKB: Entrada desde la representación binaria conocida (WKB), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

A.10.3. Constructores

- tcbuffer: Constructor para búferes circulares temporales con valor constante
- tcbufferSeq: Constructor para búferes circulares temporales de subtipo secuencia
- tcbufferSeqSet, tcbufferSeqSetGaps: Constructor para búferes circulares temporales de subtipo conjunto de secuencias
- tcbuffer: Construye un búfer circular temporal a partir de un punto de geometría temporal y un flotante temporal

A.10.4. Conversión de tipos

- ::: Convierte un búfer circular temporal a un punto de geometría temporal
- ::: Convierte un búfer circular temporal a un flotante temporal

A.10.5. Accesores

- getValues: Devuelve los valores
- points, radius: Devuelve los puntos o los radios
- valueAtTimestamp: Devuelve el valor en una marca de tiempo

A.10.6. Transformaciones

- tcbufferInst, tcbufferSeq, tcbufferSeqSet: Transforma un búfer circular temporal en otro subtipo
- setInterp: Transforma un búfer circular temporal en otra interpolación
- round: Redondea los puntos y los radios del búfer circular temporal en el número de lugares decimales
- stops: Extrae las subsecuencias en las que el búfer circular temporal permanece dentro de un área de un tamaño máximo dado durante al menos la duración dada
- translate: Devuelve un valor temporal trasladado por los desplazamientos dados
- rotate: Devuelve un valor temporal rotado en sentido antihorario alrededor del origen o de un punto
- rotateZ: Devuelve un valor temporal rotado en sentido antihorario alrededor del eje z

A.10.7. Modificaciones

- appendInstant, appendSequence: Agrega un instante temporal o una secuencia temporal
- merge: Fusiona los búferes circulares temporales

A.10.8. Restricciones

- atValue, minusValue, atValues, minusValues: Restringe al (complemento de) un valor o un conjunto de valores
- atGeometry, minusGeometry: Restringe al (complemento de) una geometría
- atStbox, minusStbox: Restringe un búfer circular temporal a (el complemento de) una caja espaciotemporal

A.10.9. Operaciones espaciales

- SRID, setSRID: Devuelve o establece el identificador de referencia espacial
- transform, transformPipeline: Transforma a una referencia espacial diferente
- traversedArea: Devuelve el área atravesada por el búfer circular temporal
- centroid: Devuelve el punto geométrico temporal dado por los centros de un búfer circular temporal
- convexHull: Devuelve la envolvente convexa del área atravesada por el búfer circular temporal

A.10.10. Operaciones de cuadro delimitador

- &&, <@, @>, ~=, -|-: Operadores topológicos
- ::: Operadores de posición
- expandSpace: Devuelve la caja delimitadora espaciotemporal expandida en la dimensión espacial por un valor flotante

A.10.11. Operaciones de distancia

- |=|: Devuelve la distancia más cercana
- nearestApproachInstant: Devuelve el instante del primer búfer circular temporal en el que los dos argumentos están a la distancia más cercana
- shortestLine: Devuelve la línea que conecta el punto de aproximación más cercano entre los dos argumentos
- <->: Devuelve la distancia temporal
- minDistance: Devuelve la distancia espacial mínima entre dos conjuntos de búferes circulares temporales, ignorando el tiempo

A.10.12. Relaciones siempre y alguna vez

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Relaciones alguna vez y siempre

A.10.13. Relaciones espaciotemporales

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Relaciones espaciotemporales

A.10.14. Comparaciones

- =, <>, <, >, <=, >=: Comparaciones tradicionales
- ?=, ?<>, %=, %<>: Comparaciones alguna vez y siempre
- #=, #<>: Comparaciones temporales

A.10.15. Agregaciones

- extent: Extensión del cuadro delimitador
- tCount: Conteo temporal
- wCount: Conteo de ventana
- tCentroid: Agrega los centros de un conjunto de búferes circulares temporales en su centroide temporal, convirtiendo a `tgeompoint`

A.10.16. Simplificación

- minDistSimplify, minTimeDeltaSimplify: Devuelve un búfer circular temporal simplificado asegurándose de que los centros consecutivos estén al menos separados por una cierta distancia o intervalo de tiempo
- maxDistSimplify, douglasPeuckerSimplify: Devuelve un búfer circular temporal simplificado usando el algoritmo de Douglas-Peucker

A.11. Temporal Rigid Geometries

A.11.1. Entrada y salida

- asText: Devuelve la representación de texto conocido (WKT)
- asEWKT: Devuelve la representación extendida de texto conocido (EWKT), prefijada con el SRID
- asMFJSON: Devuelve la representación JSON de características móviles (MF-JSON)
- asEWKB, asHexWKB, asHexEWKB: Devuelve la representación extendida binaria conocida (EWKB), la representación hexadecimal binaria conocida (HexWKB), o la representación hexadecimal extendida binaria conocida (HexEWKB)
- trgeometryFromText, trgeometryFromMFJSON, trgeometryFromEWKB, trgeometryFromEWKT, trgeometryFromHexWKB, trgeometryFromHexEWKB: Entrada desde la representación de texto conocido (WKT), desde la representación extendida de texto conocido (EWKT), desde la representación JSON de características móviles (MF-JSON), desde la representación extendida binaria conocida (EWKB), desde la representación hexadecimal binaria conocida (HexWKB), o desde la representación hexadecimal extendida binaria conocida (HexEWKB)

A.11.2. Constructores

- trgeometry: Construye una geometría rígida temporal a partir de una geometría de referencia y una pose temporal

A.11.3. Conversión de tipos

- tpose: Devuelve la pose temporal subyacente
- tgeompoint: Devuelve la trayectoria del centroide (antena) como un punto temporal

A.11.4. Accesores

- startValue, endValue, valueAtTimestamp: Devuelve la geometría materializada al inicio, al final o en una marca de tiempo elegida
- numInstants, numSequences, numTimestamps: Devuelve el número de instantes, secuencias o marcas de tiempo distintos
- instants, sequences, segments: Devuelve el arreglo de instantes, secuencias o segmentos entre instantes constituyentes
- points: Devuelve el conjunto de puntos del centroide (antena) distintos vistos a lo largo de la trgeometry
- yaw, pitch, roll: Devuelve el ángulo yaw, pitch o roll como un flotante temporal, en radianes, bajo la convención Tait-Bryan intrínseca ZYX que exige la clase de conformidad Basic-YPR de OGC GeoPose
- geometry: Devuelve la geometría de referencia estática

A.11.5. Área recorrida

- traversedArea: Devuelve el área atravesada por la geometría rígida temporal

A.11.6. Funciones espaciales

- centroid: Devuelve el centroide del cuerpo en movimiento como un punto temporal
- convexHull: Devuelve la envolvente convexa del área recorrida por el cuerpo en movimiento
- bodyPointTrajectory: Devuelve la trayectoria en el marco del mundo de un punto del marco del cuerpo transportado por la geometría rígida en movimiento

A.11.7. Métricas de movimiento

- length, cumulativeLength: Devuelve la longitud total recorrida por el centroide, o su longitud acumulada a lo largo del tiempo
- speed: Devuelve la velocidad del centroide como un flotante temporal
- angularSpeed: Devuelve la velocidad angular como un flotante temporal
- twCentroid: Devuelve el centroide ponderado en el tiempo de la trayectoria del centroide como una geometría

A.11.8. Transformaciones

- round: Redondea todos los componentes numéricos a un número dado de posiciones decimales
- setInterp: Convierte entre interpolaciones lineal y escalonada
- shiftTime, scaleTime, shiftScaleTime: Desplaza, escalar, o desplazar y escalar el dominio temporal de una trgeometry
- unnest: Transforma una trgeometry no lineal en un conjunto de filas, una por pose distinta, cada una es una pareja compuesta de la geometría de referencia con la pose aplicada y el conjunto de períodos durante el cual la trgeometry toma esa pose {}
- translate: Devuelve un valor temporal trasladado por los desplazamientos dados
- rotate: Devuelve un valor temporal rotado en sentido antihorario alrededor del origen o de un punto
- rotateX: Devuelve un valor temporal rotado en sentido antihorario alrededor del eje x
- rotateY: Devuelve un valor temporal rotado en sentido antihorario alrededor del eje y
- rotateZ: Devuelve un valor temporal rotado en sentido antihorario alrededor del eje z
- force2D: Devuelve una geometría rígida temporal proyectada sobre el suelo

A.11.9. Modificaciones

- appendInstant: Agrega un único instante a una trgeometry existente, ampliando la trayectoria de pose subyacente
- appendSequence: Agrega una secuencia completa a una trgeometry existente
- merge: Combina dos trgeometries con la misma geometría de referencia en un único conjunto de secuencias

A.11.10. Restricciones

- atValue, minusValue, atValues, minusValues: Restringe al (complemento de) una pose o un conjunto de poses
- atGeometry, minusGeometry: Restringe una trgeometry a (el complemento de) una geometría
- atStbox, minusStbox: Restringe una trgeometry a (el complemento de) una caja espaciotemporal

A.11.11. Operaciones de distancia

- <->: Devuelve la distancia temporal
- |=|: Devuelve la distancia de aproximación más cercana
- nearestApproachInstant: Devuelve el instante del primer argumento en el que los dos argumentos están más cerca
- shortestLine: Devuelve la línea entre los puntos más cercanos de los dos argumentos en su instante de aproximación más cercana

A.11.12. Operaciones de caja delimitadora

- &&, @>, <@, ~=, -|-: Operadores topológicos estándar sobre cajas delimitadoras
- <<, >>, &<, &>, <</, />>, <<#, #>>: Operadores de posición espacial y temporal

A.11.13. Relaciones siempre y alguna vez

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Relaciones siempre y alguna vez

A.11.14. Relaciones espaciotemporales

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Relaciones espaciotemporales

A.11.15. Comparaciones

- =, <>, <, >, <=, >=: Comparaciones tradicionales
- ?=, %=, ?<>, %<>: Comparaciones alguna vez y siempre
- %=, %<>: Comparaciones temporales

A.11.16. Mosaicos

- spaceBoxes, timeBoxes, spaceTimeBoxes: Devuelve las cajas espaciotemporales de una geometría rígida temporal dividida con respecto a una cuadrícula espacial, temporal o espaciotemporal

A.11.17. Cajas delimitadoras

- stboxes, spans: Devuelve el arreglo de cajas espaciotemporales o lapsos de tiempo de los segmentos de una geometría rígida temporal
- splitNStboxes, splitEachNStboxes, splitNSpans, splitEachNSpans: Devuelve las cajas espaciotemporales o los lapsos de tiempo de una geometría rígida temporal dividida en un número dado de contenedores, o con un número dado de segmentos por contenedor

A.11.18. Agregaciones

- tCount: Número de geometrías rígidas temporales solapadas en cada instante, equivalente a `tCount` sobre la tpose subyacente
- extent: Caja delimitadora agregada de una columna de geometrías rígidas temporales

A.12. Temporal JSON

A.12.1. Operaciones sobre valores JSONB

- jsonbObjectField, jsonbObjectFieldText: Extrae un campo de objeto JSON especificado por una clave
- jsonbExtractPath, jsonbExtractPathText: Extrae un valor JSON especificado por una ruta
- jsonbArrayElement, jsonbArrayElementText: Extrae un elemento de un arreglo JSON

- jsonbArrayLength: Devuelve el número de elementos de un arreglo JSON
- jsonbArrayElements, jsonbArrayElementsText: Devuelve los elementos de un arreglo JSON como filas
- jsonbEach, jsonbEachText: Devuelve los campos de un objeto JSON como filas de una clave y un valor
- jsonbObjectKeys: Devuelve las claves de un objeto JSON como filas
- jsonbPretty: Devuelve el texto indentado de un valor JSON
- jsonbConcat: Concatena dos valores JSON
- jsonbDelete, jsonbDeleteArray, jsonbDeleteIndex, jsonbDeletePath: Elimina de un valor JSON una clave, las claves de un arreglo, un elemento de arreglo o el elemento en una ruta
- jsonbExists, jsonbExistsAny, jsonbExistsAll: ¿Existe un texto, o alguno o todos los textos de un arreglo, como clave de nivel superior o elemento de arreglo de un valor JSON?
- jsonbContains, jsonbContained: ¿Contiene el primer valor JSON al segundo, o está contenido en él?
- jsonbSet, jsonbSetLax: Devuelve un valor JSON con el elemento en una ruta establecido en un valor
- jsonbInsert: Devuelve un valor JSON con un valor insertado en una ruta
- jsonbStripNulls: Devuelve un valor JSON sin sus campos de objeto cuyo valor es nulo
- jsonbPathExists, jsonbPathExistsTz: ¿Devuelve la ruta JSON algún elemento para un valor JSON?
- jsonbPathMatch, jsonbPathMatchTz: Devuelve el resultado de una comprobación de predicado de ruta JSON para un valor JSON
- jsonbPathQuery, jsonbPathQueryTz, jsonbPathQueryArray, jsonbPathQueryArrayTz, jsonbPathQueryFirst, jsonbPathQueryFirstTz: Devuelve los elementos que una ruta JSON devuelve para un valor JSON, como filas, como un arreglo JSON, o el primero de ellos
- eq, cmp, hash, hashExtended: Compara dos valores JSON, o devuelve el valor hash de un valor JSON

A.12.2. Operaciones sobre conjuntos JSONB

- ->, ->>, jsonbsetObjectField, jsonbsetObjectFieldText: Extrae un campo de objeto JSON especificado por una clave
- ->, ->>, jsonbsetExtractPath, jsonbsetExtractPathText: Extrae un campo de objeto JSON especificado por una ruta
- jsonbsetArrayElement, jsonbsetArrayElementText: Extrae un elemento de un arreglo JSON
- intset, floatset, textset: Extrae un valor alfanumérico de un conjunto JSONB dado por una clave
- ||: Concatenación de un conjunto JSONB
- -: Eliminación en un conjunto JSONB
- ?: Existencia en un conjunto JSONB
- jsonbsetSet, jsonbsetSetLax: Asignación en un conjunto JSONB
- jsonbsetInsert: Devuelve el valor `jsonbset` con `jsonb` insertado
- jsonbsetStripNulls: Devuelve un conjunto JSONB sin nulos
- @?, jsonbsetPathExists, jsonbsetPathExistsTz: ¿Devuelve la ruta JSON algún elemento para el conjunto JSONB especificado?
- @@, jsonbsetPathMatch, jsonbsetPathMatchTz: Devuelve el resultado de una comprobación de predicado de ruta JSON para un conjunto JSONB
- jsonbsetPathQueryArray, jsonbsetPathQueryArrayTz: Devuelve todos los elementos devueltos por una ruta JSON de un conjunto JSONB, como un arreglo JSON
- jsonbsetPathQueryFirst, jsonbsetPathQueryFirstTz: Devuelve el primer elemento devuelto por una ruta JSON de un conjunto JSONB

A.12.3. Entrada y salida

- asText: Devuelve la representación de texto conocido (WKT)
- asBinary, asHexWKB: Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)
- asMFJSON: Devuelve la representación JSON de características móviles (MF-JSON)
- tjsonbFromText, jsonbFromText, jsonpathFromText: Entrada desde la representación de texto conocido (WKT)
- tjsonbFromBinary, tjsonbFromHexWKB: Entrada desde la representación binaria conocida (WKB) o desde la representación hexadecimal binaria conocida (HexWKB)
- tjsonbFromMFJSON: Entrada desde la representación JSON de características móviles (MF-JSON)

A.12.4. Constructores

- tjsonb: Constructor de valores JSONB temporales que tienen un valor constante
- tjsonbSeq: Constructor de valores JSONB temporales del subtipo secuencia
- tjsonbSeqSet, tjsonbSeqSetGaps: Constructor de valores JSONB temporales del subtipo conjunto de secuencias

A.12.5. Conversiones

- :::: Convierte un JSONB temporal a un `tstzspan`
- :::: Convierte entre un JSONB temporal y un texto
- :::: Convierte entre un JSONB temporal y un texto temporal

A.12.6. Accesores

- getValues: Devuelve los valores
- valueAtTimestamp: Devuelve el valor en una marca de tiempo

A.12.7. Transformaciones

- tjsonbInst, tjsonbSeq, tjsonbSeqSet: Transforma un JSONB temporal a otro subtipo
- setInterp: Transforma un valor JSONB temporal a otra interpolación

A.12.8. Operaciones JSON temporales

- ->, ->>, tjsonObjectField, tjsonbObjectField, tjsonbObjectFieldText: Extrae un campo de objeto JSON temporal especificado por una clave
- ->, ->>, tjsonExtractPath, tjsonbExtractPath, tjsonbExtractPathText: Extrae un campo de objeto JSON temporal especificado por una ruta
- tjsonArrayElement, tjsonbArrayElement, tjsonbArrayElementText: Extrae un elemento de un arreglo JSON temporal
- tbool, tint, tfloat, ttext: Extrae un valor alfanumérico temporal de un valor JSONB temporal dado por una clave
- tgeompoint, tgeogpoint, tgeometry, tgeography: Extrae un valor geo temporal de un valor JSONB temporal dado por una clave

- ||: Concatenación JSONB temporal
- -: Eliminación JSONB temporal
- ?: Existencia JSONB temporal
- @>, <@: Contiene/contenido JSONB temporal
- tjsonbSet, tjsonbSetLax: Asignación JSONB temporal
- tjsonbInsert: Inserción JSONB temporal
- tjsonStripNulls, tjsonbStripNulls: Devuelve un valor JSON temporal sin nulos
- @?, tjsonbPathExists, tjsonbPathExistsTz: ¿Devuelve la ruta JSON algún elemento para el valor JSONB temporal especificado?
- @@, tjsonbPathMatch, tjsonbPathMatchTz: Devuelve el resultado de una comprobación de predicado de ruta JSON para un valor JSONB temporal
- tjsonbPathQueryArray, tjsonbPathQueryArrayTz: Devuelve todos los elementos devueltos por una ruta JSON de un valor JSONB temporal, como un arreglo JSON
- tjsonbPathQueryFirst, tjsonbPathQueryFirstTz: Devuelve el primer elemento devuelto por una ruta JSON de un valor JSONB temporal

A.12.9. Restricciones

- atValue, minusValue, atValues, minusValues: Restringe a (el complemento de) un valor o un conjunto de valores

A.12.10. Operaciones de caja delimitadora

- &&, <@, @>, ~=, -|-: Operadores topológicos
- :::: Operadores de posición

A.12.11. Comparaciones

- =, <>, <, >, <=, >=: Comparaciones tradicionales
- ?=, %=: Comparaciones siempre y alguna vez
- :::: Comparaciones temporales

A.12.12. Agregaciones

- tCount: Conteo temporal
- wCount: Conteo en ventana

A.13. Temporal Tessellations

A.13.1. Entrada y salida

- tcell: Entrada y salida de una celda temporal en su forma textual
- asText, asEWKT, asBinary, asEWKB, asHexWKB, asHexEWKB, asMFJSON: Serializar una celda temporal como texto, como binario o como binario bien conocido hexadecimal, con o sin su SRID, o como MF-JSON
- tcellFromText, tcellFromEWKT, tcellFromBinary, tcellFromEWKB, tcellFromHexWKB, tcellFromHexEWKB, tcellFromMFJSON: Reconstruir una celda temporal a partir de su representación textual, binaria, hexadecimal o MF-JSON
- asBinary, asHexWKB: Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB) de una celda
- h3indexFromBinary, h3indexFromHexWKB, quadbinFromBinary, quadbinFromHexWKB, s2cellFromBinary, s2cellFromHexWKB: Entrada de una celda desde la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)
- asText: Devuelve la representación de texto de una celda
- h3indexFromText, quadbinFromText, s2cellFromText: Entrada desde la representación de texto de una celda
- asBinary: Devuelve la representación binaria conocida (WKB) de un ráster o de una tesela Raquet
- asHexWKB: Devuelve la representación hexadecimal binaria conocida (HexWKB) de un ráster o de una tesela Raquet
- rasterFromBinary, raquetFromBinary: Devuelve un ráster o una tesela Raquet a partir de su representación binaria conocida (WKB)
- rasterFromHexWKB, raquetFromHexWKB: Devuelve un ráster o una tesela Raquet a partir de su representación hexadecimal binaria conocida (HexWKB)

A.13.2. Constructores

- tcell: Construir una celda temporal que contiene una celda sobre un valor de tiempo
- tcellInst, tcellSeq, tcellSeqSet: Construir una celda temporal de un subtipo dado a partir de sus componentes
- raquet: Construye una tesela ráster Raquet a partir de sus bytes de píxeles, dimensiones, celda QUADBIN y tipo de píxel
- raster: Construye un ráster de una banda a partir de los bytes de sus píxeles y de su georreferenciación
- makeEmptyRaster: Construye un ráster sin banda
- addBand: Añade una banda a un ráster
- raquetRead: Decodifica un archivo ráster en una tesela ráster Raquet mediante GDAL

A.13.3. Conversión de tipos

- :::: Convertir entre un `tcell` y un `tbigint`
- :::: Convertir una celda temporal en un punto temporal geodésico o plano de centroides de celda
- stbox: Convierte una tesela Raquet en una caja espaciotemporal
- stbox: Convierte un ráster en una caja espaciotemporal

A.13.4. Accesores

- startValue, endValue, valueN: Devolver el primer, el último o el enésimo identificador de celda distinto de un valor temporal
- getValues: Devolver el conjunto de identificadores de celda distintos que toma la trayectoria
- valueAtTimestamp: Devolver el identificador de celda en una marca de tiempo dada, usando interpolación de pasos
- numBands: Devuelve el número de bandas de un ráster
- SRID: Devuelve el identificador del sistema de referencia espacial de un ráster
- upperLeftX, upperLeftY: Devuelve la coordenada X o Y de la esquina superior izquierda de un ráster
- scaleX, scaleY: Devuelve el ancho o el alto de píxel de un ráster, es decir, la componente X o Y de su escala
- skewX, skewY: Devuelve la componente X o Y de la inclinación de un ráster
- quadbin: Devuelve la celda QUADBIN de una tesela Raquet
- width: Devuelve el ancho en píxeles de un ráster o de una tesela Raquet
- height: Devuelve el alto en píxeles de un ráster o de una tesela Raquet
- bandPixelType: Devuelve el nombre del tipo de dato de píxel de una banda de un ráster o de una tesela Raquet
- bandHasNoDataValue: Devuelve si una banda de un ráster o una tesela Raquet establece un valor nodata
- bandNoDataValue: Devuelve el valor nodata de una banda de un ráster o de una tesela Raquet
- pixels: Devuelve los bytes de píxeles de una banda de un ráster o de una tesela Raquet

A.13.5. Transformaciones

- shiftTime, scaleTime, shiftScaleTime: Desplazar, escalar, o desplazar y escalar el lapso de tiempo de una celda temporal
- appendInstant, appendSequence: Añadir un instante temporal o una secuencia temporal
- merge: Fusionar las celdas temporales
- setInterp: Establecer la interpolación de una celda temporal

A.13.6. Modificaciones

- insert, update: Insertar una celda temporal en otra, o actualizar una con otra
- deleteTime: Eliminar un valor de tiempo de una celda temporal
- setValues: Fija los valores de un bloque de píxeles de una banda de un ráster

A.13.7. Restricciones

- atValue, minusValue, atValues, minusValues: Restringir una celda temporal a (el complemento de) una celda o un conjunto de celdas
- atTime, minusTime: Restringir una celda temporal a (el complemento de) un valor de tiempo

A.13.8. Cobertura de geometrías estáticas

- latLngToCell, geoToQuadbinCell, geoToS2Cell: Devolver la celda de una cuadrícula que contiene una geometría de punto en la resolución dada
- cellset, geoToH3IndexSet, geoToQuadbinSet, geoToS2CellSet: Devolver el conjunto de celdas de una cuadrícula que cubren una geometría en la resolución dada
- eEqual: Devolver si una trayectoria de celdas temporales toma alguna vez una de las celdas de un conjunto de celdas

A.13.9. Inspección

- getResolution: Devolver la resolución de cada celda de una trayectoria
- isValidCell: Devolver un booleano temporal que indica en cada instante si el valor es una celda válida

A.13.10. Jerarquía

- cellToParent: Devolver la celda padre temporal en la resolución dada
- cellToCover, coverCells: Devolver las celdas que cubren una celda, o las celdas de un conjunto de celdas, en la resolución dada

A.13.11. Conversiones de latitud y longitud

- tcell: Devolver la celda temporal en la resolución dada que contiene cada punto de un punto temporal geodésico o plano
- quadbinSplit: Dividir un punto temporal en las celdas QUADBIN que atraviesa {}
- h3Split: Dividir un punto temporal en las celdas H3 que atraviesa {}
- s2Split: Dividir un punto temporal en las celdas S2 que atraviesa {}
- cellToPoint, tgeompoint: Devolver el centroide de una celda, o de cada celda de una trayectoria
- cellToBoundary: Devolver la frontera poligonal de una celda, o de cada celda de una trayectoria

A.13.12. Operaciones de cuadro delimitador

- stbox, expandSpace: Devolver el cuadro delimitador espaciotemporal de una celda temporal, o ese cuadro expandido en la dimensión espacial por un valor flotante
- &&, @>, <@, ~=, -|-: Operadores topológicos
- <<, &<, &>, >>, <<|, &<|, |&>, |>>, <<#, &<#, #>>, #&>: Operadores de posición

A.13.13. Funciones métricas

- cellArea: Devolver el área de una celda, y el área temporal de cada celda de una trayectoria

A.13.14. Funciones que devuelven conjuntos

- gridDisk: Todas las celdas dentro de `k` pasos de cuadrícula desde el origen
- cellToChildren: Todos los hijos de una celda en la resolución de destino dada
- compactCells: Representación compactada de un conjunto de celdas, fusionándose las celdas más finas en su padre siempre que esté presente el conjunto completo de hijos del padre
- uncompactCells: Representación descompactada en la resolución dada, la inversa de `compactCells`

A.13.15. Relaciones siempre y alguna vez

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Relaciones siempre y alguna vez

A.13.16. Relaciones espaciotemporales

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Relaciones espaciotemporales

A.13.17. Comparaciones

- =, <>, <, >, <=, >=, cmp: Comparaciones tradicionales
- ?=, ?<>, ?<, ?>, ?<=, ?>=, ?cmp: Comparaciones siempre y alguna vez
- #=, #<>: Comparaciones temporales
- =, <>, <, <=, >=, >, cmp: Compara dos teselas Raquet
- hash, hashExtended: Valores hash

A.13.18. Agregaciones

- tCount: Conteo temporal
- wCount: Conteo de ventana
- merge, appendInstant, appendSequence: Agregar un conjunto de celdas temporales fusionándolas, o ensamblándolas en una secuencia o en un conjunto de secuencias

A.13.19. Operaciones específicas de H3

- th3GetBaseCellNumber: Devolver el número de celda base temporal (0–121) de cada celda de una trayectoria
- th3IsResClassIii: Devolver un booleano temporal que indica si cada celda tiene orientación de clase III
- th3IsPentagon: Devolver un booleano temporal que indica si cada celda es un pentágono
- th3CellToCenterChild: Devolver la celda hija central temporal en la resolución dada
- th3CellToChildPos: Devolver la posición ordinal (basada en 0) de cada celda entre los hijos de su padre en la resolución dada
- th3ChildPosToCell: Devolver la celda hija temporal en una posición ordinal dada del padre dado, en la resolución de hijo dada
- h3GridRing: Las celdas a *exactamente* `k` pasos de cuadrícula desde el origen
- h3GridPathCells: El camino inclusivo de celdas entre dos celdas de la misma resolución
- h3OriginToDirectedEdges: Todas las aristas dirigidas salientes de una celda
- h3CellToVertexes: Todos los vértices de una celda
- h3GetIcosahedronFaces: Los índices de las caras del icosaedro (0..19) que interseca una celda
- th3EdgeLength: Devolver la longitud temporal de cada arista dirigida de una trayectoria, en la unidad solicitada
- greatCircleDistance: Devolver la distancia de círculo máximo temporal entre dos puntos temporales geodésicos (no entre celdas)

- th3AreNeighborCells: Devolver un booleano temporal que indica si dos celdas temporales son vecinas de cuadrícula en cada instante
- th3CellsToDirectedEdge: Devolver un identificador de arista dirigida temporal a partir de un par de celdas temporales de origen y destino
- isValidDirectedEdge: Devolver un booleano temporal que indica en cada instante si el valor es una arista dirigida H3 válida
- th3GetDirectedEdgeOrigin, th3GetDirectedEdgeDestination: Devolver la celda de origen o de destino temporal de una arista dirigida temporal
- th3DirectedEdgeToBoundary: Devolver la frontera poligonal temporal de cada arista dirigida de una trayectoria, como una geografía temporal
- th3CellToVertex: Devolver el vértice H3 temporal en el número de vértice dado
- th3VertexToPoint: Devolver las coordenadas geodésicas temporales de cada vértice de una trayectoria
- isValidVertex: Devolver un booleano temporal que indica en cada instante si el valor es un vértice H3 válido
- <->: Devolver la distancia temporal en saltos de cuadrícula (número de pasos de una sola celda) entre dos celdas temporales
- th3CellToLocalIj, th3LocalIjToCell: Convertir entre una celda temporal y un par de coordenadas locales (I, J) temporales relativas a una celda temporal de origen

A.13.20. Operaciones específicas de QUADBIN

- isValidIndex: Devolver si un valor es un identificador QUADBIN estructuralmente válido
- quadbinCellSibling: Devolver la celda vecina en la misma resolución en la dirección dada
- quadbinCellToQuadkey, tquadbinCellToQuadkey: Emitir la cadena quadkey en base 4 de tesela deslizante de una celda
- tquadbinGridDistance, <->: Devolver la distancia temporal en la cuadrícula entre dos celdas QUADBIN temporales
- quadbinCellToTile: Descomponer una celda de vuelta en su terna de coordenadas de tesela, devuelta como un arreglo de enteros que contiene x, después y, después z
- quadbinTileToCell: Construir una celda a partir de coordenadas de tesela de mapa deslizante: la columna `x`, la fila `y` y el zoom `z`

A.13.21. Operaciones específicas de S2

- s2GetFace: Devolver la cara del cubo de 0 a 5 sobre la que se asienta una celda S2
- s2CellToToken, s2TokenToCell, ts2CellToToken: Convertir entre una celda S2 y su token, la forma que publican las bibliotecas de S2
- ts2GridDistance, <->: Devolver la distancia temporal en la cuadrícula entre dos celdas S2 temporales
- s2CommonAncestorLevel: Devolver el nivel de la celda más profunda que contiene a dos celdas S2
- s2CellContains: Devolver verdadero si la primera celda S2 contiene a la segunda
- s2RangeMin, s2RangeMax: Devolver la primera y la última celda hoja que contiene una celda S2
- s2CellToChild: Devolver el descendiente de una celda S2 en la posición dada a lo largo de la curva de Hilbert
- s2EdgeLength: Devolver la longitud en metros de una arista de una celda S2
- s2EdgeNeighbors: Devolver las cuatro celdas que comparten una arista con una celda S2

A.13.22. Operaciones específicas de ráster

- rasterValue: Lee una banda de ráster a lo largo de una trayectoria
- valueCount: Devuelve el número de píxeles de una banda de un ráster que contienen cada valor
- valuePercent: Devuelve la proporción de los píxeles de una banda de un ráster que contienen cada valor
- sameAlignment: Devuelve verdadero si los rásteres están sobre la misma rejilla
- notSameAlignmentReason: Devuelve por qué dos rásteres no están sobre la misma rejilla
- rasterUnion: Devuelve la unión de los rásteres de una columna
- reclass: Devuelve un ráster cuya banda contiene las clases a las que una expresión asigna sus valores
- +, -, *, /: Adición, resta, multiplicación y división de rásteres, píxel a píxel
- slope: Devuelve la pendiente de una banda de un ráster, píxel a píxel
- tri: Devuelve el índice de rugosidad del terreno de una banda de un ráster, píxel a píxel
- tpi: Devuelve el índice de posición topográfica de una banda de un ráster, píxel a píxel
- mapAlgebra: Devuelve una estadística del vecindario de cada píxel de una banda de un ráster
- clip: Devuelve un ráster que conserva los píxeles de otro que una geometría cubre
- transform, transformPipeline: Devuelve un ráster expresado en otro sistema de referencia espacial
- rescale: Devuelve un ráster remuestreado a otro tamaño de píxel
- summaryStats: Devuelve las estadísticas de los píxeles de una banda de ráster
- summaryStatsAgg: Devuelve las estadísticas de los píxeles de una banda sobre los rásteres de una columna
- summarystatsCount, summarystatsSum, summarystatsMean, summarystatsStddev, summarystatsMin, summarystatsMax: Devuelve el número de píxeles, la suma, la media, la desviación estándar, el mínimo o el máximo de las estadísticas de una banda de ráster
- dumpAsPolygons: Devuelve los polígonos de una banda de ráster, uno por cada grupo de píxeles que comparten un valor {}
- pixelAsPoints: Devuelve los píxeles de una banda de ráster como puntos {}
- dumpValues: Devuelve los valores de las bandas de un ráster como arreglos
- rasterTileValue: Muestrea una tesela ráster `raquet` a lo largo de una trayectoria
- rasterTileValue: Muestrea un arreglo de teselas ráster `raquet` a lo largo de una trayectoria
- rasterTileValueQuadbin: Muestrea un chip ráster de Raquet a lo largo de una trayectoria usando una celda QUADBIN para la georreferenciación
- atRasterValue: Devuelve la parte de una trayectoria donde el valor ráster que lee cae dentro de un rango de flotante
- minusRasterValue: Devuelve la parte de una trayectoria donde el valor ráster que lee cae fuera de un rango de flotante
- eRasterValue: Devuelve verdadero si la trayectoria alguna vez lee un valor de píxel ráster dentro de un rango de flotante
- aRasterValue: Devuelve verdadero si cada valor de píxel que la trayectoria lee sobre el ráster está dentro de un rango de flotante

A.14. Temporal Point Clouds

A.14.1. Esquemas de nubes de puntos

- pointCloudSchemaSRID: Devuelve el sistema de referencia espacial en el que se expresa todo valor de un esquema
- pointCloudSchemaCompression: Devuelve cómo se almacena un parche de un esquema
- pointCloudSchemaNDims: Devuelve cuántas dimensiones de un esquema contienen valores

A.14.2. Nubes de puntos estáticas

A.14.2.1. Entrada y salida

- asHexWKB: Devuelve la representación hexadecimal binaria conocida (HexWKB) de un pcpoint o de un pcpatch
- pcpointFromHexWKB, pcpatchFromHexWKB: Entrada de un pcpoint o de un pcpatch a partir de su representación hexadecimal binaria conocida (HexWKB)

A.14.2.2. Constructores

- pcpoint: Construye un pcpoint 2D o 3D directamente a partir de coordenadas x/y(/z)
- pcpoint: Construye un pcpoint a partir de las coordenadas de cada dimensión del esquema
- pcpatch: Construye un pcpatch a partir de las coordenadas de sus puntos, un punto tras otro
- pcpatch: Construye un pcpatch a partir de una lista variádica de pcpoints que comparten el mismo pcid

A.14.2.3. Accesores

- pcid: Devuelve el identificador de esquema de un pcpoint o pcpatch
- getX, getY, getZ: Devuelve una coordenada de un pcpoint, NULL si la dimensión está ausente del esquema
- getDim: Devuelve cualquier dimensión con nombre de un pcpoint, NULL si el nombre es desconocido para el esquema
- SRID: Devuelve el identificador de referencia espacial (heredado del esquema)
- numPoints: Devuelve el número de puntos de un pcpatch
- pointN: Devuelve el n-ésimo punto de un pcpatch
- geometry: Devuelve el multipunto de las posiciones que ocupan los puntos de un parche

A.14.3. Conjuntos de nubes de puntos

A.14.3.1. Constructores

- set: Construye un conjunto a partir de un array de valores, todos los cuales deben compartir el mismo pcid

A.14.4. Cajas delimitadoras de nubes de puntos

A.14.4.1. Entrada y salida

- asBinary, asHexWKB: Devuelve la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB) de una caja de nube de puntos
- asEWKB, asHexEWKB: Devuelve la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB) de una caja de nube de puntos
- tpcboxFromBinary, tpcboxFromHexWKB: Entrada de una caja de nube de puntos desde la representación binaria conocida (WKB) o la representación hexadecimal binaria conocida (HexWKB)
- tpcboxFromEWKB, tpcboxFromHexEWKB: Entrada de una caja de nube de puntos desde la representación binaria conocida extendida (EWKB) o la representación hexadecimal binaria conocida extendida (HexEWKB)
- asText: Devuelve la representación de texto conocido (WKT)
- tpcboxFromText: Entrada desde la representación de texto conocido (WKT)

A.14.4.2. Constructores

- tpcbox, tpcbox_z, tpcbox_t, tpcbox_xt, tpcbox_zt: Construye un tpcbox a partir de límites X/Y/Z/T explícitos y un pcid, cuyo esquema declara el SRID

A.14.4.3. Conversiones

- tpcbox: Convierte un pcpoint a un tpcbox degenerado (de extensión cero); el SRID y el pcid provienen del esquema
- tpcbox: Convierte un pcpatch a un tpcbox

A.14.4.4. Accesores

- hasX, hasZ, hasT: Devuelve si un tpcbox tiene establecidas las dimensiones X/Y, Z o T
- xmin, xmax, ymin, ymax, zmin, zmax: Devuelve un límite de coordenada, NULL si la dimensión correspondiente está ausente
- tmin, tmax: Devuelve un límite temporal, NULL si la caja no tiene dimensión T
- pcid, SRID: Devuelve el identificador de esquema y el SRID que declara dicho esquema

A.14.4.5. Transformaciones

- round: Devuelve un tpcbox con las coordenadas redondeadas al número de dígitos decimales indicado
- setSRID: Devuelve un tpcbox que declara un sistema de referencia que su esquema no declara

A.14.4.6. Operaciones de conjuntos

- +, *: Unión e intersección de dos tpcboxes

A.14.4.7. Operaciones topológicas

- &&, @>, <@, ~=, -|-: Operadores topológicos

A.14.4.8. Operaciones de posición

- <<, &<, >>, &>, <<|, &<|, |>>, |&>, <</, &</, />>, /&>, <<#, &<#, #>>, #&>: Operadores de posición

A.14.4.9. Comparaciones

- =, <>, <, <=, >=, >: Comparaciones tradicionales

A.14.5. Puntos de nube de puntos temporales

A.14.5.1. Entrada y salida

- asText, asEWKT: Devuelve la representación textual de un tpcpoint, o de un arreglo de ellos, con o sin su SRID, y convertir un tpcpoint a texto
- asMFJSON, tpcpointFromMFJSON: Devuelve la representación JSON de características móviles (MF-JSON) de un tpcpoint y la lee de vuelta; la ida y vuelta es sin pérdida
- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria o binaria bien conocida hexadecimal de un tpcpoint, con o sin su SRID
- tpcpointFromText, tpcpointFromEWKT, tpcpointFromBinary, tpcpointFromEWKB, tpcpointFromHexWKB, tpcpointFromHexEWKB: Devuelve un tpcpoint a partir de su representación textual, binaria o binaria bien conocida hexadecimal, con o sin su SRID

A.14.5.2. Constructores

- tpcpoint: Constructor para puntos de nube de puntos temporales con un valor constante
- tpcpointSeq: Constructor para puntos de nube de puntos temporales de subtipo secuencia
- tpcpointSeqSet: Construye un pcpoint temporal de subtipo conjunto de secuencias a partir de un array de secuencias

A.14.5.3. Conversiones

- ::: Convierte un punto de nube de puntos temporal a un punto de geometría temporal
- tpcbox, ::: Convierte un punto de nube de puntos temporal en su caja delimitadora

A.14.5.4. Accesores

- pcid: Devuelve el id de esquema de pgPointCloud compartido por todos los instantes
- SRID: Devuelve el SRID heredado del esquema
- getX, getY, getZ: Proyecta un tpcpoint sobre una dimensión espacial, produciendo un tfloat
- getDim: Proyecta un tpcpoint sobre cualquier dimensión de esquema con nombre, produciendo un tfloat

A.14.5.5. Transformaciones

- tpcpointInst, tpcpointSeq, tpcpointSeqSet: Transforma un tpcpoint en otro subtipo
- setInterp: Transforma un tpcpoint a otra interpolación

A.14.5.6. Modificaciones

- insert, update: Inserta un tpcpoint en otro, o actualizar otro con él
- deleteTime: Elimina un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un tpcpoint
- appendInstant, appendSequence: Añade un instante o una secuencia a un tpcpoint
- merge: Fusionar los valores temporales

A.14.5.7. Restricciones

- atValue, minusValue, atValues, minusValues: Restringe un tpcpoint a (al complemento de) un valor pcpoint o un conjunto de valores pcpoint
- atTime, minusTime: Restringe un tpcpoint a un selector temporal (marca de tiempo, periodo, conjunto o spanset), o eliminarlo
- atTpcbox, minusTpcbox: Restringe un tpcpoint al extent espacial / temporal de un tpcbox, o eliminarlo
- beforeTimestamp, afterTimestamp: Restringe un tpcpoint a los instantes anteriores o posteriores a una marca de tiempo

A.14.5.8. Operaciones de división

- unnest: Devuelve los valores distintos de un tpcpoint con el conjunto de spans durante el cual toma cada uno de ellos
- timeSplit: Fragmenta un tpcpoint en fragmentos, uno por cada intervalo de tiempo
- spans: Devuelve el arreglo de lapsos de tiempo de los segmentos de un tpcpoint
- splitNSpans, splitEachNSpans: Devuelve los lapsos de tiempo de un tpcpoint dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

A.14.5.9. Operaciones de caja delimitadora

- &&, @>, <@, ~=, -|-: Operadores topológicos
- <<, >>, <<|, |>>, <</, />>, <<#, #>>: Operadores de posición

A.14.5.10. Operaciones de distancia

- |=|: Devuelve la distancia más cercana
- nearestApproachInstant: Devuelve el instante del tpcpoint en el que los dos argumentos están a la distancia más cercana
- shortestLine: Devuelve la línea que une el punto de acercamiento más cercano entre los dos argumentos
- <->: Devuelve la distancia temporal

A.14.5.11. Relaciones siempre y alguna vez

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Relaciones ever y always

A.14.5.12. Relaciones espaciotemporales

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Relaciones espaciotemporales

A.14.5.13. Comparaciones

- =, <>, <, <=, >=, >: Comparaciones tradicionales
- ?=, %=: Comparaciones alguna vez y siempre
- #=: Comparaciones temporales

A.14.5.14. Simplificación

- minDistSimplify, minTimeDeltaSimplify, maxDistSimplify, douglasPeuckerSimplify: Devuelve un tpcpoint simplificado asegurando que los puntos consecutivos estén separados al menos una cierta distancia o intervalo de tiempo, o simplificado mediante el algoritmo de Douglas-Peucker

A.14.6. Parches de nube de puntos temporales

A.14.6.1. Entrada y salida

- asText, asEWKT: Devuelve la representación textual de un tpcpatch, o de un arreglo de ellos, con o sin su SRID, y convertir un tpcpatch a texto
- asBinary, asEWKB, asHexWKB, asHexEWKB: Devuelve la representación binaria o binaria bien conocida hexadecimal de un tpcpatch, con o sin su SRID
- tpcpatchFromText, tpcpatchFromEWKT, tpcpatchFromBinary, tpcpatchFromEWKB, tpcpatchFromHexWKB, tpcpatchFromHexEWKB: Devuelve un tpcpatch a partir de su representación textual, binaria o binaria bien conocida hexadecimal, con o sin su SRID
- asMFJSON, tpcpatchFromMFJSON: Devuelve la representación JSON de características móviles (MF-JSON) de un tpcpatch y la lee de vuelta; la ida y vuelta es sin pérdida

A.14.6.2. Constructores

- tpcpatch: Constructor para parches de nube de puntos temporales con un valor constante
- tpcpatchSeq: Construye un pcpatch temporal de subtipo secuencia
- tpcpatchSeqSet: Construye un pcpatch temporal de subtipo conjunto de secuencias

A.14.6.3. Conversiones

- tgeometry: Convierte un parche temporal en la geometría temporal de los multipuntos que ocupan sus parches
- tpcbox, ::: Convierte un parche de nube de puntos temporal en su caja delimitadora

A.14.6.4. Accesores

- startNumPoints, endNumPoints: Devuelve el número de puntos del parche contenido en el primer o último instante
- numPoints: Devuelve el número total de puntos de todos los parches de todos los instantes
- points: Emite una fila por cada marca de tiempo de instante y punto del parche

A.14.6.5. Transformaciones

- tpcpatchInst, tpcpatchSeq, tpcpatchSeqSet: Transforma un tpcpatch en otro subtipo
- setInterp: Transforma un tpcpatch a otra interpolación

A.14.6.6. Modificaciones

- insert, update: Inserta un tpcpatch en otro, o actualizar otro con él
- deleteTime: Elimina un selector de tiempo (marca de tiempo, conjunto, período o conjunto de spans) de un tpcpatch
- appendInstant, appendSequence: Añade un instante o una secuencia a un tpcpatch
- merge: Fusionar los valores temporales

A.14.6.7. Restricciones

- atValue, minusValue, atValues, minusValues: Restringe un tpcpatch a (al complemento de) un valor pcpatch o un conjunto de valores pcpatch
- atTpcbox, minusTpcbox: Restringe un tpcpatch a los instantes cuya extensión del parche se superpone al tpcbox, o eliminarlos
- atTpcboxFine, minusTpcboxFine: Restringe un tpcpatch a los puntos de cada instante que contiene el tpcbox, o los elimina
- atGeometry, minusGeometry: Restringe un tpcpatch a los puntos cuya proyección XY intersecta (o no intersecta, en el caso de minus) una geometría 2D
- beforeTimestamp, afterTimestamp: Restringe un tpcpatch a los instantes anteriores o posteriores a una marca de tiempo

A.14.6.8. Operaciones de división

- unnest: Devuelve los valores distintos de un tpcpatch con el conjunto de spans durante el cual toma cada uno de ellos
- timeSplit: Fragmenta un tpcpatch en fragmentos, uno por cada intervalo de tiempo
- spans: Devuelve el arreglo de lapsos de tiempo de los segmentos de un tpcpatch
- splitNSpans, splitEachNSpans: Devuelve los lapsos de tiempo de un tpcpatch dividido en un número dado de contenedores, o con un número dado de segmentos por contenedor

A.14.6.9. Operaciones espaciales

- swath: Devuelve la franja de un parche temporal, la región del plano que barren sus parches

A.14.6.10. Operaciones de caja delimitadora

- &&, @>, <@, ~=, -|-: Operadores topológicos
- <<, >>, <<|, |>>, <</, />>, <<#, #>>: Operadores de posición

A.14.6.11. Operaciones de distancia

- |=|: Devuelve la distancia más cercana

A.14.6.12. Relaciones siempre y alguna vez

- eIntersects, aIntersects, eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eTouches, aTouches, eDwithin, aDwithin: Devuelve verdadero si y solo si al menos un punto de alguno de los instantes del tpcpatch intersecta la geometría (solo XY)

A.14.6.13. Relaciones espaciotemporales

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Relaciones espaciotemporales

A.14.6.14. Comparaciones

- ?=, %=: Comparaciones alguna vez y siempre
- #=: Comparaciones temporales

A.14.7. Agregaciones

- extent: Agregado de caja delimitadora sobre una columna de valores tpcpoint, tpcpatch o tpcbox
- tCount, wCount: Conteo temporal y conteo por ventana de filas superpuestas en cada marca de tiempo
- tnpoints: Conteo de puntos de pcpatch por instante, sumado a través de las filas
- tdensity: Densidad de puntos por instante (`numPoints / área-bbox-xy`) sumada a través de las filas
- merge: Combina múltiples valores temporales en uno solo con todos los instantes de entrada fusionados en orden temporal
- appendInstant, appendSequence: Agregados de construcción en flujo: `appendInstant` ensambla instantes en una secuencia; `appendSequence` ensambla secuencias en un conjunto de secuencias

A.14.8. Indexes

- GiST, SP-GiST, B-tree, and hash opclasses for tpcpoint, tpcpatch, and tpcbox

Apéndice B

Generador de datos sintéticos

En muchas circunstancias, es necesario tener un conjunto de datos de prueba para evaluar enfoques de implementación alternativos o realizar evaluaciones comparativas. A menudo se requiere que tal conjunto de datos tenga requisitos particulares en tamaño o en las características intrínsecas de sus datos. Incluso si un conjunto de datos del mundo real pudiera estar disponible, puede que no sea ideal para tales experimentos por múltiples razones. Por lo tanto, un generador de datos sintéticos que pueda personalizarse para producir datos de acuerdo con los requisitos dados suele ser la mejor solución. Obviamente, los experimentos con datos sintéticos deben complementarse con experimentos con datos del mundo real para tener una comprensión profunda del problema en cuestión.

MobilityDB proporciona un generador simple de datos sintéticos que se puede utilizar para tales fines. En particular, se utilizó este generador de datos para generar la base de datos utilizada para las pruebas de regresión en MobilityDB. El generador de datos está programado en PL/pgSQL para que se pueda personalizar fácilmente. Se encuentra en el directorio `datagen` en el repositorio. En este apéndice, presentamos brevemente la funcionalidad básica del generador. Primero enumeramos las funciones que generan valores aleatorios para varios tipos de datos de PostgreSQL, PostGIS y MobilityDB y luego damos ejemplos de cómo se usan estas funciones para generar tablas de dichos valores. Los parámetros de las funciones no están especificados, consulte los archivos fuente donde se pueden encontrar explicaciones detalladas sobre los distintos parámetros.

B.1. Generador para tipos PostgreSQL

- `random_bool`: Generar un booleano aleatorio
- `random_int`: Generar un entero aleatorio
- `random_int_array`: Generar una matriz de enteros aleatorios
- `random_int4range`: Generar un rango aleatorio de enteros
- `random_float`: Generar un número flotante aleatorio
- `random_float_array`: Generar una matriz de números flotantes aleatorios
- `random_text`: Generar un texto aleatorio
- `random_timestamptz`: Generar una marca de tiempo con zona horaria aleatoria
- `random_timestamptz_array`: Generar una matriz de marcas de tiempo con zona horaria aleatorias
- `random_minutes`: Generar un intervalo de minutos aleatorio
- `random_tstzrange`: Generar rango aleatorio de marcas de tiempo con zona horaria
- `random_tstzrange_array`: Generar una matriz de rangos aleatorios de marcas de tiempo con zona horaria

B.2. Generador para tipos PostGIS

- `random_geom_point`: Generar un punto geométrico 2D aleatorio
- `random_geom_point3D`: Generar un punto geométrico 3D aleatorio
- `random_geog_point`: Generar un punto geográfico 2D aleatorio
- `random_geog_point3D`: Generar un punto geográfico 3D aleatorio
- `random_geom_point_array`: Generar una matriz de puntos geométricos 2D aleatorios
- `random_geom_point3D_array`: Generar una matriz de puntos geométricos 3D aleatorios
- `random_geog_point_array`: Generar una matriz puntos geográficos 2D aleatorios
- `random_geog_point3D_array`: Generar una matriz de puntos geográficos 3D aleatorios
- `random_geom_linestring`: Generar una cadena lineal geométrica 2D aleatoria
- `random_geom_linestring3D`: Generar una cadena lineal geométrica 3D aleatoria
- `random_geog_linestring`: Generar una cadena lineal geográfica 2D aleatoria
- `random_geog_linestring3D`: Generar una cadena lineal geográfica 3D aleatoria
- `random_geom_polygon`: Generar un polígono geométrico 2D sin agujeros aleatorio
- `random_geom_polygon3D`: Generar un polígono geométrico 3D sin agujeros aleatorio
- `random_geog_polygon`: Generar un polígono geográfico 2D sin agujeros aleatorio
- `random_geog_polygon3D`: Generar un polígono geográfico 3D sin agujeros aleatorio
- `random_geom_multipoint`: Generar un multipunto geométrico 2D aleatorio
- `random_geom_multipoint3D`: Generar un multipunto geométrico 3D aleatorio
- `random_geog_multipoint`: Generar un multipunto geográfico 2D aleatorio
- `random_geog_multipoint3D`: Generar un multipunto geográfico 3D aleatorio
- `random_geom_multilinestring`: Generar una multicadena lineal geométrica 2D aleatoria
- `random_geom_multilinestring3D`: Generar una multicadena lineal geométrica 3D aleatoria
- `random_geog_multilinestring`: Generar una multicadena lineal geográfica 2D aleatoria
- `random_geog_multilinestring3D`: Generar una multicadena lineal geográfica 3D aleatoria
- `random_geom_multipolygon`: Generar un multipolígono geométrico 2D sin agujeros aleatorio
- `random_geom_multipolygon3D`: Generar un multipolígono geométrico 3D sin agujeros aleatorio
- `random_geog_multipolygon`: Generar un multipolígono geográfico 2D sin agujeros aleatorio
- `random_geog_multipolygon3D`: Generar un multipolígono geográfico 3D sin agujeros aleatorio

B.3. Generador para tipos de rango, de tiempo y de cuadro delimitador MobilityDB

- `random_intspan`: Generar un rango aleatorio de enteros
- `random_floatspan`: Generar un rango aleatorio de números flotantes
- `random_tstzspan`: Generar un `tstzspan` aleatorio
- `random_tstzspan_array`: Generar una matriz de valores `tstzspan` aleatorios
- `random_tstzset`: Generar un `tstzset` aleatorio
- `random_tstzspanset`: Generar un `tstzspanset` aleatorio
- `random_tbox`: Generar un `tbox` aleatorio
- `random_stbox`: Generar un `stbox` 2D aleatorio
- `random_stbox3D`: Generar un `stbox` 3D aleatorio
- `random_geodstbox`: Generar un `stbox` geodético 2D aleatorio
- `random_geodstbox3D`: Generar un `stbox` geodético 3D aleatorio

B.4. Generador para tipos temporales MobilityDB

- `random_tbool_inst`: Generar un booleano temporal aleatorio de subtipo instante
- `random_tint_inst`: Generar un entero temporal aleatorio de subtipo instante
- `random_tfloat_inst`: Generar un flotante temporal aleatorio de subtipo instante
- `random_ttext_inst`: Generar un texto temporal aleatorio de subtipo instante
- `random_tgeompoint_inst`: Generar un punto geométrico temporal 2D aleatorio de subtipo instante
- `random_tgeompoint3D_inst`: Generar un punto geométrico temporal 3D aleatorio de subtipo instante
- `random_tgeogpoint_inst`: Generar un punto geográfico temporal 2D aleatorio de subtipo instante
- `random_tgeogpoint3D_inst`: Generar un punto geográfico temporal 3D aleatorio de subtipo instante
- `random_tbool_discseq`: Generar un booleano temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_tint_discseq`: Generar un entero temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_tfloat_discseq`: Generar un flotante temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_ttext_discseq`: Generar un texto temporal aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeompoint_discseq`: Generar un punto temporal geométrico 2D aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeompoint3D_discseq`: Generar un punto geométrico temporal 3D aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeogpoint_discseq`: Generar un punto geográfico temporal 2D aleatorio de subtipo secuencia con interpolación discreta
- `random_tgeogpoint3D_discseq`: Generar un punto geográfico temporal 3D aleatorio de subtipo secuencia con interpolación discreta
- `random_tbool_seq`: Generar un booleano temporal aleatorio de subtipo secuencia

- `random_tint_seq`: Generar un entero temporal aleatorio de subtipo secuencia
- `random_tfloat_seq`: Generar un flotante temporal aleatorio de subtipo secuencia
- `random_ttext_seq`: Generar un texto temporal aleatorio de subtipo secuencia
- `random_tgeompoint_seq`: Generar un punto geométrico temporal 2D aleatorio de subtipo secuencia
- `random_tgeompoint3D_seq`: Generar un punto geométrico temporal 3D aleatorio de subtipo secuencia
- `random_tgeogpoint_seq`: Generar un punto geográfico temporal 2D aleatorio de subtipo secuencia
- `random_tgeogpoint3D_seq`: Generar un punto geográfico temporal 3D aleatorio de subtipo secuencia
- `random_tbool_seqset`: Generar un booleano temporal aleatorio de subtipo conjunto de secuencias
- `random_tint_seqset`: Generar un entero temporal aleatorio de subtipo conjunto de secuencias
- `random_tfloat_seqset`: Generar un flotante temporal aleatorio de subtipo conjunto de secuencias
- `random_ttext_seqset`: Generar un texto temporal aleatorio de subtipo conjunto de secuencias
- `random_tgeompoint_seqset`: Generar un punto geométrico temporal 2D aleatorio de subtipo conjunto de secuencias
- `random_tgeompoint3D_seqset`: Generar un punto geométrico temporal 3D aleatorio de subtipo conjunto de secuencias
- `random_tgeogpoint_seqset`: Generar un punto geográfico temporal 2D aleatorio de subtipo conjunto de secuencias
- `random_tgeogpoint3D_seqset`: Generar un punto geográfico temporal 3D aleatorio de subtipo conjunto de secuencias

B.5. Generación de tablas con valores aleatorios

Los archivos `create_test_tables_temporal.sql` y `create_test_tables_tpoint.sql` dan ejemplos de utilización de las funciones que generan valores aleatorios listadas arriba. Por ejemplo, el primer archivo define la función siguiente.

```
CREATE OR REPLACE FUNCTION create_test_tables_temporal(size integer DEFAULT 100)
RETURNS text AS $$ DECLARE perc integer;
BEGIN perc := size * 0.01;
IF perc < 1 THEN perc := 1; END IF;
-- ... Table generation ...
RETURN 'The End';
END;
$$ LANGUAGE 'plpgsql';
```

La función tiene un parámetro `size` que define el número de filas en las tablas. Si no se proporciona, crea por defecto tablas de 100 filas. La función define una variable `perc` que calcula el 1 % del tamaño de las tablas. Este parámetro se utiliza, por ejemplo, para generar tablas con un 1 % de valores nulos. A continuación ilustramos algunos de los comandos que generan tablas.

La creación de una tabla `tbl_float` que contiene valores aleatorios `float` en el rango [0,100] con 1 % de valores nulos se da a continuación.

```
CREATE TABLE tbl_float AS
/* Add perc NULL values */
SELECT k, NULL AS f FROM generate_series(1, perc) AS k UNION
SELECT k, random_float(0, 100) FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_tbox` que contiene valores aleatorios `tbox` donde los límites de los valores están en el rango [0,100] y los límites de las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] se da a continuación.

```
CREATE TABLE tbl_tbox AS
/* Add perc NULL values */
SELECT k, NULL AS b FROM generate_series(1, perc) AS k UNION
SELECT k, random_tbox(0, 100, '2001-01-01', '2001-12-31', 10, 10)
FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_floatspan` que contiene valores aleatorios `floatspan` donde los límites de los valores están en el rango [0,100] y la máxima diferencia entre los límites inferiores y superiores es 10 se da a continuación.

```
CREATE TABLE tbl_floatspan AS
/* Add perc NULL values */
SELECT k, NULL AS f FROM generate_series(1, perc) AS k UNION
SELECT k, random_floatspan(0, 100, 10) FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_tstzset` que contiene valores aleatorios `tstzset` que tienen entre 5 y 10 marcas de tiempo donde las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] y el máximo intervalo entre marcas de tiempo consecutivas es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tstzset AS
/* Add perc NULL values */
SELECT k, NULL AS ts FROM generate_series(1, perc) AS k UNION
SELECT k, random_tstzset('2001-01-01', '2001-12-31', 10, 5, 10)
FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_tstzspan` que contiene valores aleatorios `tstzspan` donde las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] y la máxima diferencia entre los límites inferiores y superiores es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tstzspan AS
/* Add perc NULL values */
SELECT k, NULL AS p FROM generate_series(1, perc) AS k UNION
SELECT k,
  random_tstzspan('2001-01-01', '2001-12-31', 10) FROM generate_series(perc+1, size) AS k;
```

La creación de una tabla `tbl_geom_point` que contiene valores aleatorios `geometry` 2D point valores, donde las coordenadas x e y están en el rango [0, 100] y en SRID 3812 se da a continuación.

```
CREATE TABLE tbl_geom_point AS
SELECT 1 AS k, geometry 'SRID=3812;point empty' AS g UNION
SELECT k, random_geom_point(0, 100, 0, 100, 3812) FROM generate_series(2, size) k;
```

Observe que la tabla contiene un valor de punto vacío. Si no se proporciona el SRID, se establece de forma predeterminada en 0.

La creación de una tabla `tbl_geog_point3D` que contiene valores aleatorios `geography` 3D point valores, donde las coordenadas x, y, y z están, respectivament, en los rangos [-10, 32], [35, 72] y [0, 1000] y en SRID 7844 se da a continuación.

```
CREATE TABLE tbl_geog_point3D AS
SELECT 1 AS k, geography 'SRID=7844;pointZ empty' AS g UNION
SELECT k,
  random_geog_point3D(-10, 32, 35, 72, 0, 1000, 7844) FROM generate_series(2, size) k;
```

Nótese que los valores de latitud y longitud se eligen para cubrir aproximadamente la Europa continental. Si no se proporciona el SRID, se establece de forma predeterminada en 4326.

La creación de una tabla `tbl_geom_linestring` que contiene valores aleatorios `geometry` 2D linestring valores que tienen entre 5 y 10 vértices, donde las coordenadas x e y están en el rango [0, 100] y en SRID 3812 y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k,
  random_geom_linestring(0, 100, 0, 100, 10, 5, 10, 3812) FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geom_linestring` que contiene valores aleatorios `geometry` 2D linestring valores que tienen entre 5 y 10 vértices, donde las coordenadas x e y están en el rango [0, 100] y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k,
  random_geom_linestring(0, 100, 0, 100, 10, 5, 10) FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geom_polygon3D` que contiene valores aleatorios `geometry` 3D polygon valores sin agujeros, que tienen entre 5 y 10 vértices, donde las coordenadas x, y, y z están en el rango [0, 100] y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_polygon3D AS
SELECT 1 AS k, geometry 'polygon Z empty' AS g UNION
SELECT k, random_geom_polygon3D(0, 100, 0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geom_multipoint` que contiene valores aleatorios `geometry` 2D multipunto valores que tienen entre 5 y 10 points, donde las coordenadas x e y están en el rango [0, 100] y la máxima diferencia entre valores de coordenadas consecutivos es 10 unidades en el SRID subyacente se da a continuación.

```
CREATE TABLE tbl_geom_multipoint AS
SELECT 1 AS k, geometry 'multipoint empty' AS g UNION
SELECT k,
  random_geom_multipoint(0, 100, 0, 100, 10, 5, 10) FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geog_multilinestring` que contiene valores aleatorios `geography` 2D multilinestring valores que tienen entre 5 y 10 linestrings, cada una teniendo entre 5 y 10 vértices, donde las coordenadas x e y estan, respectivamente, en el rangos [-10, 32] y [35, 72] y la máxima diferencia entre valores de coordenadas consecutivos es 10 se da a continuación.

```
CREATE TABLE tbl_geog_multilinestring AS
SELECT 1 AS k, geography 'multilinestring empty' AS g UNION
SELECT k, random_geog_multilinestring(-10, 32, 35, 72, 10, 5, 10, 5, 10)
FROM generate_series(2, size) k;
```

La creación de una tabla `tbl_geometry3D` que contiene valores aleatorios `geometry` 3D de varios tipos se da a continuación. Esta función requiere que las tablas para los diversos tipos de geometría se hayan creado previamente.

```
CREATE TABLE tbl_geometry3D (k serial PRIMARY KEY, g geometry);
INSERT INTO tbl_geometry3D(g)
(SELECT g FROM tbl_geom_point3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_linestring3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_polygon3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipoint3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multilinestring3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipolygon3D ORDER BY k LIMIT (size * 0.2));
```

La creación de una tabla `tbl_tbool_inst` que contiene valores aleatorios `tbool` valores de subtipo instante donde las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] se da a continuación.

```
CREATE TABLE tbl_tbool_inst AS
/* Add perc NULL values */
SELECT k, NULL AS inst FROM generate_series(1, perc) AS k UNION
SELECT k,
  random_tbool_inst('2001-01-01', '2001-12-31') FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT inst FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows with the same timestamp */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT tboolInst(random_bool(), getTimestamp(inst))
  FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
```

Como se puede ver arriba, la tabla tiene un porcentaje de valores nulos, de duplicados y de filas con la misma marca de tiempo.

La creación de una tabla `tbl_tint_discseq` que contiene valores aleatorios `tint` valores de subtipo secuencia con interpolación discreta que tienen entre 5 y 10 marcas de tiempo donde the integer valores están en el rango [0, 100], las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre dos valores consecutivos es 10 y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tint_discseq AS
/* Add perc NULL values */
SELECT k, NULL AS ti FROM generate_series(1, perc) AS k UNION
SELECT k, random_tint_discseq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS ti
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows with the same timestamp */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti + random_int(1, 2) FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc rows that meet */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, endTimestamp(ti)-startTimestamp(ti))
  FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc rows that overlap */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute',(endTimestamp(ti)-startTimestamp(ti))/2))
  FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);
```

Como se puede ver arriba, la tabla tiene un porcentaje de valores nulos, de duplicados, de filas con la misma marca de tiempo, de filas que se encuentran y de filas que se superponen.

La creación de una tabla `tbl_tfloat_seq` que contiene valores aleatorios `tfloat` valores de subtipo secuencia que tienen entre 5 y 10 marcas de tiempo donde los valores `float` están en el rango [0, 100], las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre dos valores consecutivos es 10 y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tfloat_seq AS
/* Add perc NULL values */
SELECT k, NULL AS seq FROM generate_series(1, perc) AS k UNION
SELECT k, random_tfloat_seq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS seq
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq + random_int(1, 2) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, timeSpan(seq)) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, date_trunc('minute',timeSpan(seq)/2))
  FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);
```

La creación de una tabla `tbl_ttext_seqset` que contiene valores aleatorios `ttext` de subtipo conjunto de secuencias que tienen entre 5 y 10 sequences, cada una teniendo entre 5 y 10 marcas de tiempo, donde los valores de texto tienen máximo 10 caracteres, las marcas de tiempo están en el rango [2001-01-01, 2001-12-31] y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```
CREATE TABLE tbl_ttext_seqset AS
/* Add perc NULL values */
SELECT k, NULL AS ts FROM generate_series(1, perc) AS k UNION
SELECT k, random_ttext_seqset('2001-01-01', '2001-12-31', 10, 10, 5, 10, 5, 10) AS ts
FROM generate_series(perc+1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts || text 'A' FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, timeSpan(ts)) FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
  FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);
```

La creación de una tabla `tbl_tgeompoint_discseq` que contiene valores aleatorios `tgeompoint` 2D valores de subtipo secuencia con interpolación discreta que tienen entre 5 y 10 instantes, donde the x e y coordenadas están en el rango [0, 100] y en SRID 3812, las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre coordenadas successivas máximo 10 unidades en el SRID subyacente y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```
CREATE TABLE tbl_tgeompoint_discseq AS
SELECT k, random_tgeompoint_discseq(0, 100, 0, 100, '2001-01-01', '2001-12-31',
  10, 10, 5, 10, 3812) AS ti FROM generate_series(1, size) k;
/* Add perc duplicates */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT ti FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT round(ti,6) FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, endTimestamp(ti)-startTimestamp(ti))
  FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute',(endTimestamp(ti)-startTimestamp(ti))/2))
  FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
```

Finalmente, la creación de una tabla `tbl_tgeompoint3D_seqset` que contiene valores aleatorios `tgeompoint` 3D valores de subtipo conjunto de secuencias que tienen entre 5 y 10 sequences, cada una teniendo entre 5 y 10 marcas de tiempo, donde las coordenadas x, y, y z están en el rango [0, 100] y en SRID 3812, las marcas de tiempo están en el rango [2001-01-01, 2001-12-31], la máxima diferencia entre coordenadas successivas máximo 10 unidades en el SRID subyacente y el máximo intervalo entre dos instantes consecutivos es 10 minutos se da a continuación.

```
DROP TABLE IF EXISTS tbl_tgeompoint3D_seqset;
CREATE TABLE tbl_tgeompoint3D_seqset AS
SELECT k, random_tgeompoint3D_seqset(0, 100, 0, 100, 0, 100, '2001-01-01', '2001-12-31',
  10, 10, 5, 10, 5, 10, 3812) AS ts FROM generate_series(1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT ts FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT round(ts,3) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts,
  timeSpan(ts)) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
  FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
```

B.6. Generador para tipos de red temporales

- `random_fraction`: Generar una fracción aleatoria en el rango [0,1]
- `random_npoint`: Genera un punto de red aleatorio
- `random_nsegment`: Genera un segmento de red aleatorio
- `random_tnpoint_inst`: Generar un punto de red temporal aleatorio de subtipo instant
- `random_tnpoint_discseq`: Generar un punto de red temporal aleatorio de subtipo secuencia e interpolación discreta
- `random_tnpoint_seq`: Generar un punto de red temporal aleatorio de subtipo secuencia e interpolación linear o escalonada
- `random_tnpoint_seqset`: Generar un punto de red temporal aleatorio de subtipo conjunto de secuencias

El archivo `/datagen/npoint/create_test_tables_tnpoint.sql` da ejemplos de utilización de las funciones que generan valores aleatorios listadas arriba.

B.7. Generador para tipos JSONB temporales

- `random_jsonb`: Generar un documento JSONB aleatorio
- `random_jsonb_array`: Generar una matriz aleatoria de documentos JSONB
- `random_jsonb_set`: Generar un conjunto aleatorio de documentos JSONB
- `random_tjsonb_inst`: Generar un JSONB temporal aleatorio de subtipo instante
- `random_tjsonb_discseq`: Generar un JSONB temporal aleatorio de subtipo secuencia e interpolación discreta
- `random_tjsonb_seq`: Generar un JSONB temporal aleatorio de subtipo secuencia e interpolación escalonada
- `random_tjsonb_seqset`: Generar un JSONB temporal aleatorio de subtipo conjunto de secuencias

El archivo `/datagen/json/create_test_tables_tjsonb.sql` da ejemplos de utilización de las funciones que generan valores aleatorios listadas arriba.